

# **CORE JAVA**

## **With**

# **SCJP / OCJP**

### **Study Material**

## **Chapter 1 : Language Fundamentals**



**DURGA M.Tech**

**(Sun certified & Realtime Expert)**

**Ex. IBM Employee**

**Trained Lakhs of Students  
for last 14 years across INDIA**

**India's No.1 Software Training Institute**

# **DURGASOFT**

**www.durgasoft.com Ph: 9246212143 ,8096969696**

# Language Fundamentals

## Agenda :

1. Introduction
2. Identifiers
  - o Rules to define java identifiers:
3. Reserved words
  - o Reserved words for data types: (8)
  - o Reserved words for flow control:(11)
  - o Keywords for modifiers:(11)
  - o Keywords for exception handling:(6)
  - o Class related keywords:(6)
  - o Object related keywords:(4)
  - o Void return type keyword
  - o Unused keywords
  - o Reserved literals
  - o Enum
  - o Conclusions
4. Data types
  - o Integral data types
    - Byte
    - Short
    - Int
    - long
  - o Floating Point Data types
  - o boolean data type
  - o Char data type
  - o Java is pure object oriented programming or not ?
  - o Summary of java primitive data type
5. Literals
  - o Integral Literals
  - o Floating Point Literals
  - o Boolean literals
  - o Char literals
  - o String literals
  - o 1.7 Version enhancements with respect to Literals
    - Binary Literals
    - Usage of \_ (underscore) symbol in numeric literals
6. Arrays
  1. Introduction
  2. Array declaration
    - Single dimensional array declaration
    - Two dimensional array declaration
    - Three dimensional array declaration
  3. Array construction
    - Multi dimensional array creation
  4. Array initialization
  5. Array declaration, construction, initialization in a single line.

6. length Vs length() method
7. Anonymous arrays
8. Array element assignments
9. Array variable assignments

**Types of variables**

- o Primitive variables
- o Reference variables
- o Instance variables
- o Static variables
- o Local variables
- o Conclusions

**Un initialized arrays**

- o Instance level
- o Static level
- o Local level

**Var arg method**

- o Single Dimensional Array Vs Var-Arg Method

**Main method**

- o 1.7 Version Enhancements with respect to main()

**Command line arguments****Java coding standards**

- o Coding standards for classes
- o Coding standards for interfaces
- o Coding standards for methods
- o Coding standards for variables
- o Coding standards for constants
- o Java bean coding standards
  - Syntax for setter method
  - Syntax for getter method
- o Coding standards for listeners
  - To register a listener
  - To unregister a listener

**Various Memory areas present inside JVM**

## Identifier :

A name in java program is called identifier. It may be class name, method name, variable name and label name.

Example:

```

class Test      1
{
  public static void main(String[] args){
  int x=10;      2      3      4
  }              5
}

```

Rules to define java identifiers:

Rule 1: The only allowed characters in java identifiers are:

- 1) a to z
- 2) A to Z
- 3) 0 to 9
- 4) \_ (underscore)
- 5) \$

Rule 2: If we are using any other character we will get compile time error.

Example:

- 1) total\_number-----valid
- 2) Total#-----invalid

Rule 3: identifiers are not allowed to starts with digit.

Example:

- 1) ABC123-----valid
- 2) 123ABC-----invalid

Rule 4: java identifiers are case sensitive up course java language itself treated as case sensitive language.

Example:

```

class Test{
int number=10;
int Number=20;
int NUMBER=20; we can differentiate with case.
int NuMbEr=30;
}

```

Rule 5: There is no length limit for java identifiers but it is not recommended to take more than 15 lengths.

Rule 6: We can't use reserved words as identifiers.

Example:

```
int if=10; -----invalid
```

**Rule 7:** All predefined java class names and interface names we use as identifiers.

**Example 1:**

```
class Test
{
    public static void main(String[] args){
        int String=10;
        System.out.println(String);
    }
}
Output:
10
```

**Example 2:**

```
class Test
{
    public static void main(String[] args){
        int Runnable=10;
        System.out.println(Runnable);
    }
}
Output:
10
```

Even though it is legal to use class names and interface names as identifiers but it is not a good programming practice.

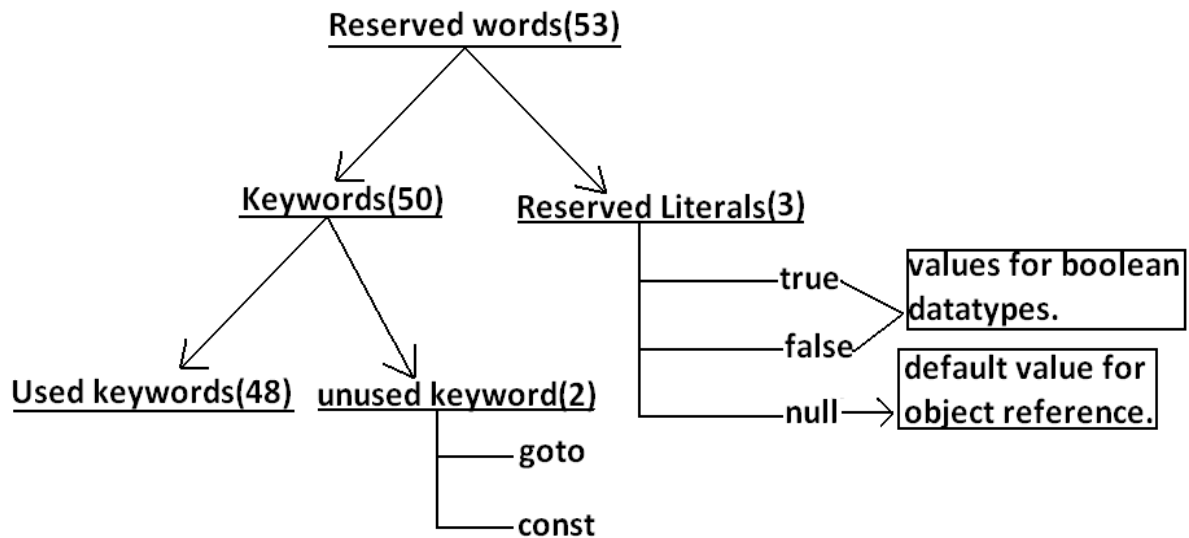
Which of the following are valid java identifiers?

- 1) **\_\$\_** (valid)
- 2) **Ca\$h** (valid)
- 3) **Java2share** (valid)
- 4) **all@hands** (invalid)
- 5) **123abc** (invalid)
- 6) **Total#** (invalid)
- 7) **Int** (valid)
- 8) **Integer** (valid)
- 9) **int** (invalid)
- 10) **tot123**

## Reserved words:

In java some identifiers are reserved to associate some functionality or meaning such type of reserved identifiers are called reserved words.

Diagram:



### Reserved words for data types: (8)

- 1) byte
- 2) short
- 3) int
- 4) long
- 5) float
- 6) double
- 7) char
- 8) boolean

### Reserved words for flow control:(11)

- 1) if
- 2) else
- 3) switch
- 4) case
- 5) default
- 6) for
- 7) do
- 8) while
- 9) break
- 10) continue
- 11) return

**Keywords for modifiers:(11)**

- 1) public
- 2) private
- 3) protected
- 4) static
- 5) final
- 6) abstract
- 7) synchronized
- 8) native
- 9) strictfp(1.2 version)
- 10) transient
- 11) volatile

**Keywords for exception handling:(6)**

- 1) try
- 2) catch
- 3) finally
- 4) throw
- 5) throws
- 6) assert(1.4 version)

**Class related keywords:(6)**

- 1) class
- 2) package
- 3) import
- 4) extends
- 5) implements
- 6) interface

**Object related keywords:(4)**

- 1) new
- 2) instanceof
- 3) super
- 4) this

**Void return type keyword:**

If a method won't return anything compulsory that method should be declared with the void return type in java but it is optional in C++.

- 1) void

**Unused keywords:**

**goto:** Create several problems in old languages and hence it is banned in java.

**Const:** Use final instead of this.

By mistake if we are using these keywords in our program we will get compile time error.

**Reserved literals:**

- 1) true values for boolean data type.
- 2) false
- 3) null----- default value for object reference.

**Enum:**

This keyword introduced in 1.5v to define a group of named constants

Example:

```
enum Beer
{
    KF, RC, KO, FO;
}
```

**Conclusions :**

1. All reserved words in java contain only lowercase alphabet symbols.
2. New keywords in java are:
3. strictfp-----1.2v
4. assert-----1.4v
5. enum-----1.5v
6. In java we have only new keyword but not delete because destruction of useless objects is the responsibility of Garbage Collection.
7. instanceof but not instanceOf
8. strictfp but not strictFp
9. const but not Constant
10. synchronized but not synchronize
11. extends but not extend
12. implements but not implement
13. import but not imports
14. int but not Int
- 15.

**Which of the following list contains only java reserved words ?**

1. final, finally, finalize (invalid) //here finalize is a method in Object class.
2. throw, throws, thrown(invalid) //thrown is not available in java
3. break, continue, return, exit(invalid) //exit is not reserved keyword
4. goto, constant(invalid) //here constant is not reserved keyword
5. byte, short, Integer, long(invalid) //here Integer is a wrapper class
6. extends, implements, imports(invalid) //imports keyword is not available in java
7. finalize, synchronized(invalid) //finalize is a method in Object class
8. instanceof, sizeOf(invalid) //sizeOf is not reserved keyword
9. new, delete(invalid) //delete is not a keyword
10. None of the above(valid)

**Which of the following are valid java keywords?**

1. public(valid)
2. static(valid)
3. void(valid)
4. main(invalid)



5. String(invalid)
6. args(invalid)

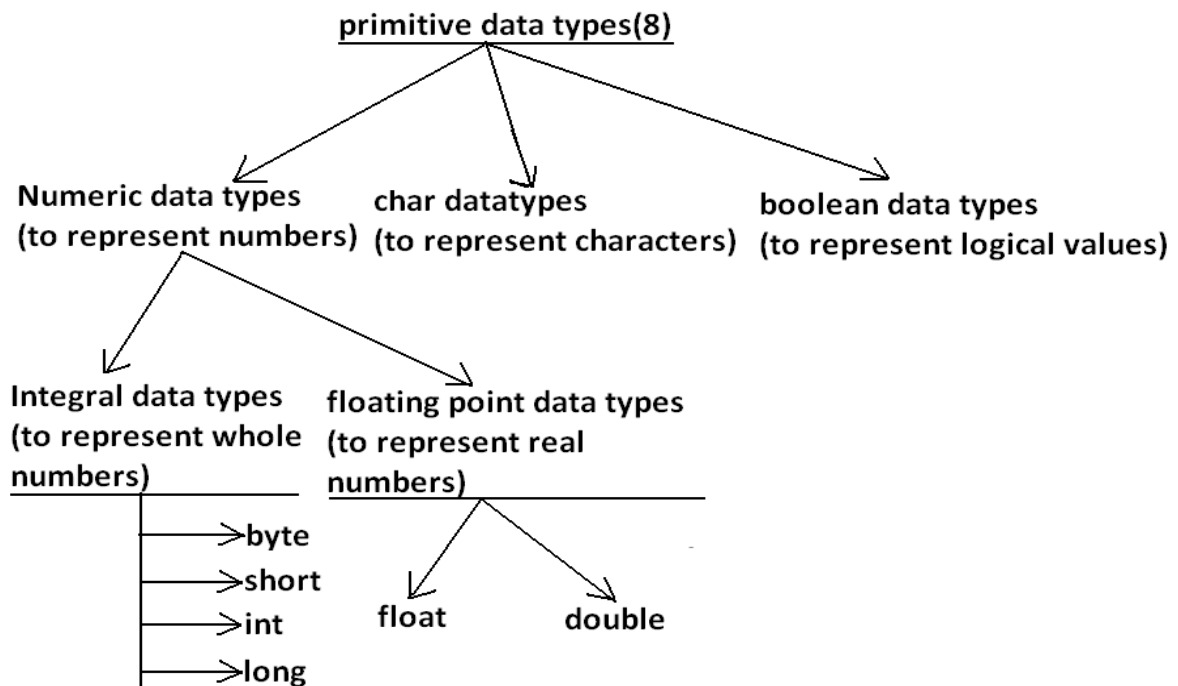
## Data types:

Every variable has a type, every expression has a type and all types are strictly define more over every assignment should be checked by the compiler by the type compatibility hence java language is considered as strongly typed programming language.

**Java is pure object oriented programming or not?**

Java is not considered as pure object oriented programming language because several oops features (like multiple inheritance, operator overloading) are not supported by java moreover we are depending on primitive data types which are non objects.

Diagram:



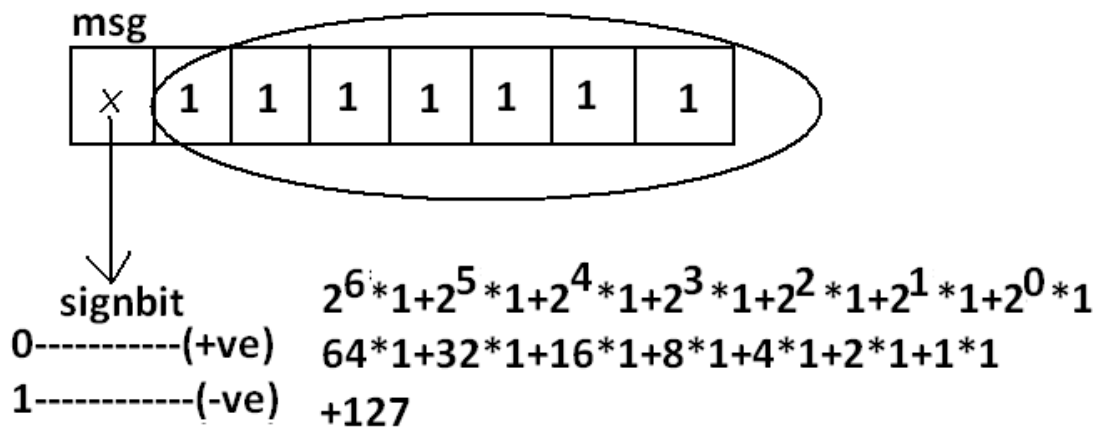
Except Boolean and char all remaining data types are considered as signed data types because we can represent both "+ve" and "-ve" numbers.

**Integral data types :**

**Byte:**

Size: 1byte (8bits)  
Maxvalue: +127  
Minvalue: -128

Range: -128 to 127 [ $-2^7$  to  $2^7-1$ ]



- The most significant bit acts as sign bit. "0" means "+ve" number and "1" means "-ve" number.
- "+ve" numbers will be represented directly in the memory whereas "-ve" numbers will be represented in 2's complement form.

Example:

```
byte b=10;
byte b2=130;//C.E:possible loss of precision
           found : int
           required : byte
byte b=10.5;//C.E:possible loss of precision
byte b=true;//C.E:incompatible types
byte b="ashok";//C.E:incompatible types
           found : java.lang.String
           required : byte
```

byte data type is best suitable if we are handling data in terms of streams either from the file or from the network.

**Short:**

The most rarely used data type in java is short.

Size: 2 bytes

Range: -32768 to 32767 ( $-2^{15}$  to  $2^{15}-1$ )

Example:

```
short s=130;
short s=32768;//C.E:possible loss of precision
short s=true;//C.E:incompatible types
```

Short data type is best suitable for 16 bit processors like 8086 but these processors are completely outdated and hence the corresponding short data type is also out data type.

**Int:**

This is most commonly used data type in java.

Size: 4 bytes

Range: -2147483648 to 2147483647 ( $-2^{31}$  to  $2^{31}-1$ )

Example:

```
int i=130;
```

```
int i=10.5;//C.E:possible loss of precision
```

```
int i=true;//C.E:incompatible types
```

**long:**

Whenever int is not enough to hold big values then we should go for long data type.

Example:

To hold the no. Of characters present in a big file int may not enough hence the return type of length() method is long.

```
long l=f.length();//f is a file
```

Size: 8 bytes

Range:  $-2^{63}$  to  $2^{63}-1$

Note: All the above data types (byte, short, int and long) can be used to represent whole numbers. If we want to represent real numbers then we should go for floating point data types.

**Floating Point Data types:**

| Float                                                                        | double                                                                          |
|------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| If we want to 5 to 6 decimal places of accuracy then we should go for float. | If we want to 14 to 15 decimal places of accuracy then we should go for double. |
| Size:4 bytes.                                                                | Size:8 bytes.                                                                   |
| Range:-3.4e38 to 3.4e38.                                                     | -1.7e308 to 1.7e308.                                                            |
| float follows single precision.                                              | double follows double precision.                                                |

**boolean data type:**

Size: Not applicable (virtual machine dependent)

Range: Not applicable but allowed values are true or false.

Which of the following boolean declarations are valid?

Example 1:

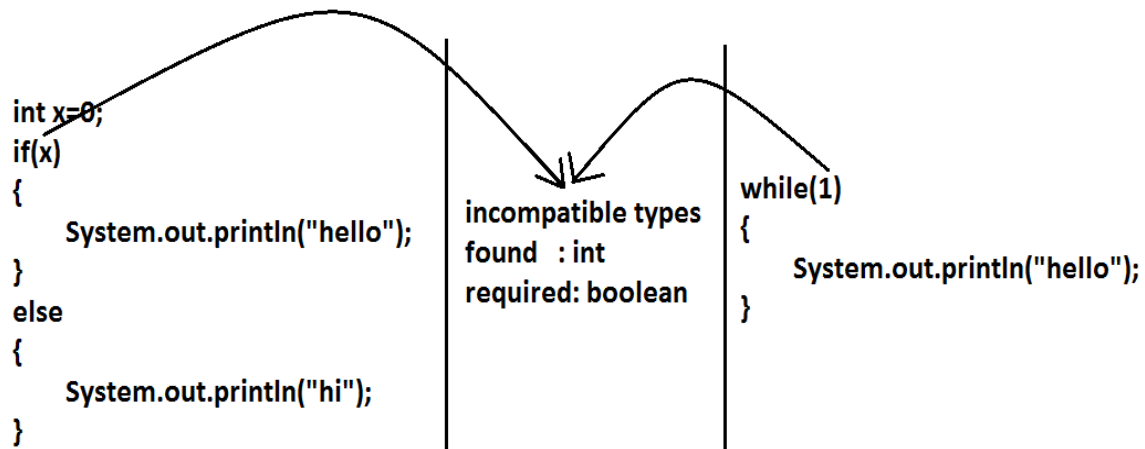
```
boolean b=true;
```

```
boolean b=True;//C.E:cannot find symbol
```

```
boolean b="True";//C.E:incompatible types
```

```
boolean b=0;//C.E:incompatible types
```

Example 2:



### Char data type:

In old languages like C & C++ are ASCII code based the no.Of ASCII code characters are < 256 to represent these 256 characters 8 - bits enough hence char size in old languages 1 byte.

In java we are allowed to use any worldwide alphabets character and java is Unicode based and no.Of unicode characters are > 256 and <= 65536 to represent all these characters one byte is not enough compulsory we should go for 2 bytes.

Size: 2 bytes

Range: 0 to 65535

Example:

```
char ch1=97;
```

```
char ch2=65536;//C.E:possible loss of precision
```

### Summary of java primitive data type:

| data type | Size           | Range                                               | Corresponding Wrapper class | Default value             |
|-----------|----------------|-----------------------------------------------------|-----------------------------|---------------------------|
| byte      | 1 byte         | $-2^7$ to $2^7-1$ (-128 to 127)                     | Byte                        | 0                         |
| short     | 2 bytes        | $-2^{15}$ to $2^{15}-1$ (-32768 to 32767)           | Short                       | 0                         |
| int       | 4 bytes        | $-2^{31}$ to $2^{31}-1$ (-2147483648 to 2147483647) | Integer                     | 0                         |
| long      | 8 bytes        | $-2^{63}$ to $2^{63}-1$                             | Long                        | 0                         |
| float     | 4 bytes        | -3.4e38 to 3.4e38                                   | Float                       | 0.0                       |
| double    | 8 bytes        | -1.7e308 to 1.7e308                                 | Double                      | 0.0                       |
| boolean   | Not applicable | Not applicable(but allowed values true false)       | Boolean                     | false                     |
| char      | 2 bytes        | 0 to 65535                                          | Character                   | 0(represents blank space) |

The default value for the object references is "null".

## Literals:

Any constant value which can be assigned to the variable is called literal.

Example:

`int x=10`

Diagram illustrating the components of the code `int x=10`:

- `int`: datatype | keyword
- `x`: name of variable | identifier
- `10`: constant value | literal

## Integral Literals:

For the integral data types (byte, short, int and long) we can specify literal value in the following ways.

1) Decimal literals: Allowed digits are 0 to 9.

Example: `int x=10;`

2) Octal literals: Allowed digits are 0 to 7. Literal value should be prefixed with zero.

Example: `int x=010;`

3) Hexa Decimal literals:

- The allowed digits are 0 to 9, A to Z.
- For the extra digits we can use both upper case and lower case characters.
- This is one of very few areas where java is not case sensitive.
- Literal value should be prefixed with `0x(or)0X`.

Example: `int x=0x10;`

These are the only possible ways to specify integral literal.

Which of the following are valid declarations?

1. `int x=0777; //(valid)`
2. `int x=0786; //C.E:integer number too large: 0786(invalid)`
3. `int x=0xFACE; (valid)`
4. `int x=0xbeef; (valid)`
5. `int x=0xBeer; //C.E: ';' expected(invalid) //:int x=0xBeer; ^// ^`
6. `int x=0xabb2cd;(valid)`

**Example:**

```
int x=10;
int y=010;
int z=0x10;
System.out.println(x+"----"+y+"----"+z); //10----8----16
```

By default every integral literal is int type but we can specify explicitly as long type by suffixing with small "l" (or) capital "L".

**Example:**

```
int x=10;(valid)
long l=10L;(valid)
long l=10;(valid)
int x=10l;//C.E:possible loss of precision(invalid)
           found : long
           required : int
```

There is no direct way to specify byte and short literals explicitly. But whenever we are assigning integral literal to the byte variables and its value within the range of byte compiler automatically treats as byte literal. Similarly short literal also.

**Example:**

```
byte b=127;(valid)
byte b=130;//C.E:possible loss of precision(invalid)
short s=32767;(valid)
short s=32768;//C.E:possible loss of precision(invalid)
```

**Floating Point Literals:**

Floating point literal is by default double type but we can specify explicitly as float type by suffixing with f or F.

**Example:**

```
float f=123.456;//C.E:possible loss of precision(invalid)
float f=123.456f;(valid)
double d=123.456;(valid)
```

We can specify explicitly floating point literal as double type by suffixing with d or D.

**Example:**

```
double d=123.456D;
```

We can specify floating point literal only in decimal form and we can't specify in octal and hexadecimal forms.

**Example:**

```
double d=123.456;(valid)
double d=0123.456;(valid) //it is treated as decimal value but not octal
double d=0x123.456;//C.E:malformed floating point literal(invalid)
```

Which of the following floating point declarations are valid?

1. float f=123.456; //C.E:possible loss of precision(invalid)
2. float f=123.456D; //C.E:possible loss of precision(invalid)
3. double d=0x123.456; //C.E:malformed floating point literal(invalid)
4. double d=0xFace; (valid)
5. double d=0xBeef; (valid)

We can assign integral literal directly to the floating point data types and that integral literal can be specified in decimal, octal and Hexa decimal form also.

**Example:**

```
double d=0xBeef;
System.out.println(d);//48879.0
```

But we can't assign floating point literal directly to the integral types.

**Example:**

```
int x=10.0;//C.E:possible loss of precision
```

We can specify floating point literal even in exponential form also(significant notation).

**Example:**

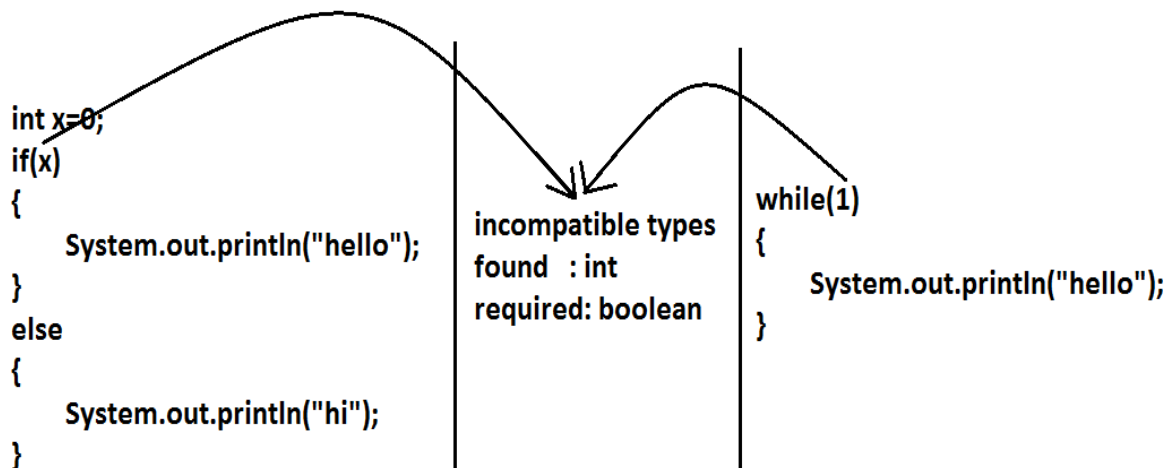
```
double d=10e2;//==>10*102(valid)
System.out.println(d);//1000.0
float f=10e2;//C.E:possible loss of precision(invalid)
float f=10e2F;(valid)
```

### Boolean literals:

The only allowed values for the boolean type are true (or) false where case is important. i.e., lower case

**Example:**

1. `boolean b=true;(valid)`
2. `boolean b=0;//C.E:incompatible types(invalid)`
3. `boolean b=True;//C.E:cannot find symbol(invalid)`
4. `boolean b="true";//C.E:incompatible types(invalid)`



## Char literals:

1) A char literal can be represented as single character within single quotes.

Example:

1. `char ch='a';`(valid)
2. `char ch=a;`//C.E:cannot find symbol(invalid)
3. `char ch="a";`//C.E:incompatible types(invalid)
4. `char ch='ab';`//C.E:unclosed character literal(invalid)

2) We can specify a char literal as integral literal which represents Unicode of that character.

We can specify that integral literal either in decimal or octal or hexadecimal form but allowed values range is 0 to 65535.

Example:

1. `char ch=97;` (valid)
2. `char ch=0xFace;` (valid)  
`System.out.println(ch);` //?
3. `char ch=65536;` //C.E: possible loss of precision(invalid)

3) We can represent a char literal by Unicode representation which is nothing but '\uxxxx' (4 digit hexa-decimal number) .

Example:

1. `char ch="\ubeef";`
2. `char ch1="\u0061";`  
`System.out.println(ch1);` //a
3. `char ch2="\u0062;` //C.E:cannot find symbol
4. `char ch3="\iface";` //C.E:illegal escape character
5. Every escape character in java acts as a char literal.

Example:

- 1) `char ch='\n';` //(valid)
- 2) `char ch='\l';` //C.E:illegal escape character(invalid)



| Escape Character | Description          |
|------------------|----------------------|
| <code>\n</code>  | New line             |
| <code>\t</code>  | Horizontal tab       |
| <code>\r</code>  | Carriage return      |
| <code>\f</code>  | Form feed            |
| <code>\b</code>  | Back space character |
| <code>\'</code>  | Single quote         |
| <code>\"</code>  | Double quote         |
| <code>\\</code>  | Back space           |

Which of the following char declarations are valid?

1. `char ch=a; //C.E:cannot find symbol(invalid)`
2. `char ch='ab'; //C.E:unclosed character literal(invalid)`
3. `char ch=65536; //C.E:possible loss of precision(invalid)`
4. `char ch=\uface; //C.E:illegal character: \64206(invalid)`
5. `char ch='/n'; //C.E:unclosed character literal(invalid)`
6. none of the above. (valid)

### String literals:

Any sequence of characters with in double quotes is treated as String literal.

Example:

`String s="Ashok"; (valid)`

### 1.7 Version enhancements with respect to Literals :

The following 2 are enhancements

1. Binary Literals
2. Usage of '\_' in Numeric Literals

### Binary Literals :

For the integral data types until 1.6v we can specified literal value in the following ways

1. Decimal
2. Octal
3. Hexa decimal

But from 1.7v onwards we can specified literal value in binary form also.

The allowed digits are 0 to 1.

Literal value should be prefixed with 0b or 0B .

```
int x = 0b111;
System.out.println(x); // 7
```

Usage of \_ symbol in numeric literals :

From 1.7v onwards we can use underscore(\_) symbol in numeric literals.

```
double d = 123456.789; //valid
```

```
double d = 1_23_456.7_8_9; //valid
```

```
double d = 123_456.7_8_9; //valid
```

The main advantage of this approach is readability of the code will be improved At the time of compilation ' \_ ' symbols will be removed automatically , hence after compilation the above lines will become double d = 123456.789

We can use more than one underscore symbol also between the digits.

```
Ex : double d = 1_23__456.789;
```

We should use underscore symbol only between the digits

```
double d=_1_23_456.7_8_9; //invalid
```

```
double d=1_23_456.7_8_9_; //invalid
```

```
double d=1_23_456_.7_8_9; //invalid
```

```
double d='a';
```

```
System.out.println(d); //97
```

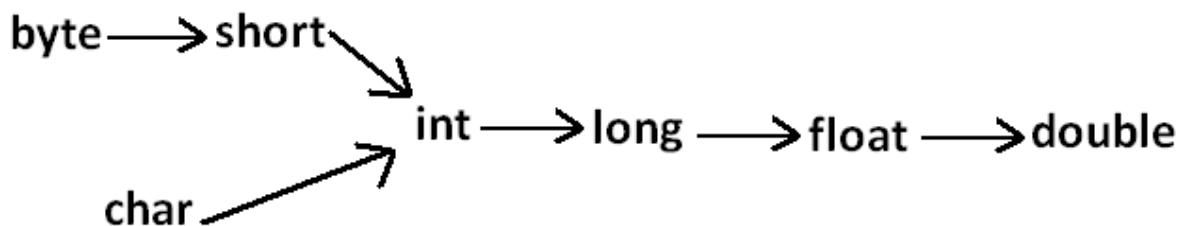
integral data types

```
float f=10L;
```

```
System.out.println(f); //10.0
```

floating-point data types

Diagram:



## Arrays

- 1) Introduction
- 2) Array declaration
- 3) Array construction
- 4) Array initialization
- 5) Array declaration, construction, initialization in a single line.
- 6) length Vs length() method
- 7) Anonymous arrays
- 8) Array element assignments
- 9) Array variable assignments.

### Introduction

An array is an indexed collection of fixed number of homogeneous data elements.

The main advantage of arrays is we can represent multiple values with the same name so that readability of the code will be improved.

But the main disadvantage of arrays is:

Fixed in size that is once we created an array there is no chance of increasing or decreasing the size based on our requirement that is to use arrays concept compulsory we should know the size in advance which may not possible always.

We can resolve this problem by using collections.

### Array declarations:

#### Single dimensional array declaration:

Example:

```
int[] a;//recommended to use because name is clearly separated from the  
type  
int []a;  
int a[];
```

At the time of declaration we can't specify the size otherwise we will get compile time error.

Example:

```
int[] a;//valid  
int[5] a;//invalid
```

#### Two dimensional array declaration:

Example:

```
int[][] a;  
int [][]a;  
int a[][];  
int[] []a;  
int[] a[];  
int []a[];
```

All are valid.(6 ways)

**Three dimensional array declaration:****Example:**

```
int[][][] a;
int [][][]a;
int a[][][];
int[] [][]a;
int[] a[][];
int[] []a[];
int[][] []a;
int[][] a[];
int []a[][];
int [][]a[];
```

All are valid.(10 ways)

**Which of the following declarations are valid?**

- 1) `int[] a1,b1; //a-1,b-1 (valid)`
- 2) `int[] a2[],b2; //a-2,b-1 (valid)`
- 3) `int[] []a3,b3; //a-2,b-2 (valid)`
- 4) `int[] a,[b]; //C.E: expected (invalid)`

**Note :**

If we want to specify the dimension before the variable that rule is applicable only for the 1st variable.

Second variable onwards we can't apply in the same declaration.

**Example:**

```
int[] []a,[b];
```

└── invalid

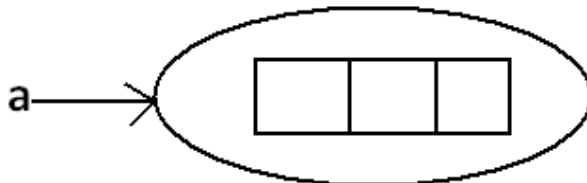
└── valid

**Array construction:**

Every array in java is an object hence we can create by using new operator.

**Example:**

```
int[] a=new int[3];
```

**Diagram:**

For every array type corresponding classes are available but these classes are part of java language and not available to the programmer level.

| Array Type | corresponding class name |
|------------|--------------------------|
| int[]      | [I                       |
| int[][]    | [[I                      |
| double[]   | [D                       |

**Rule 1:**

At the time of array creation compulsory we should specify the size otherwise we will get compile time error.

**Example:**

```
int[] a=new int[3];
int[] a=new int[]; //C.E:array dimension missing
```

**Rule 2:**

It is legal to have an array with size zero in java.

**Example:**

```
int[] a=new int[0];
System.out.println(a.length); //0
```

**Rule 3:**

If we are taking array size with -ve int value then we will get runtime exception saying `NegativeArraySizeException`.

**Example:**

```
int[] a=new int[-3]; //R.E:NegativeArraySizeException
```

**Rule 4:**

The allowed data types to specify array size are byte, short, char, int. By mistake if we are using any other type we will get compile time error.

**Example:**

```
int[] a=new int['a']; //(valid)
byte b=10;
int[] a=new int[b]; //(valid)
short s=20;
int[] a=new int[s]; //(valid)
int[] a=new int[10l]; //C.E:possible loss of precision //(invalid)
int[] a=new int[10.5]; //C.E:possible loss of precision //(invalid)
```

**Rule 5:**

The maximum allowed array size in java is maximum value of int size [2147483647].

**Example:**

```
int[] a1=new int[2147483647]; (valid)
int[] a2=new int[2147483648];
//C.E:integer number too large: 2147483648 (invalid)
```

In the first case we may get RE : `OutOfMemoryError`.

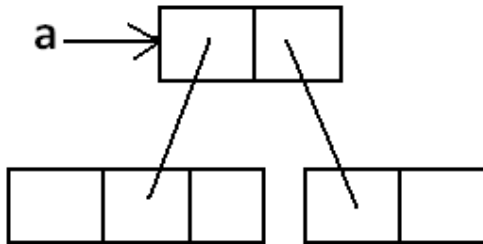
**Multi dimensional array creation:**

In java multidimensional arrays are implemented as array of arrays approach but not matrix form.

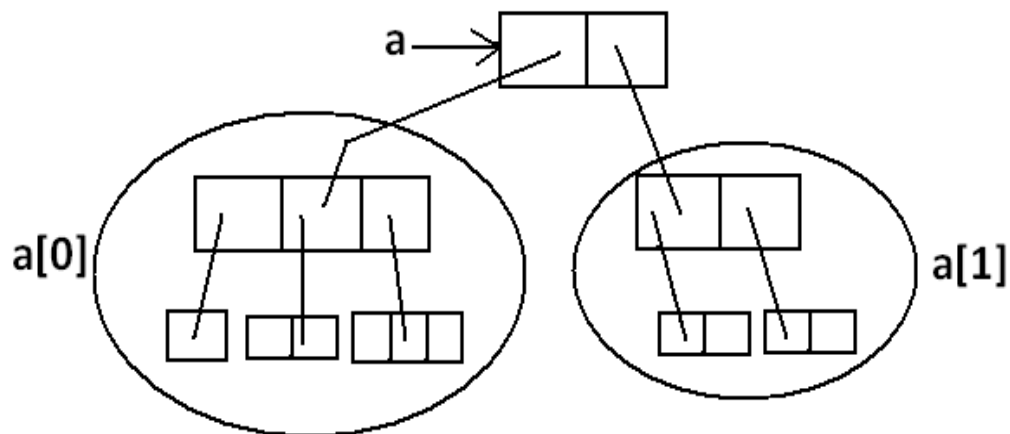
The main advantage of this approach is to improve memory utilization.

**Example 1:**

```
int[][] a=new int[2][];
a[0]=new int[3];
a[1]=new int[2];
```

**Diagram:****memory representation****Example 2:**

```
int[][][] a=new int[2][][]];
a[0]=new int[3][];
a[0][0]=new int[1];
a[0][1]=new int[2];
a[0][2]=new int[3];
a[1]=new int[2][2];
```

**Diagram:****memory representation****Which of the following declarations are valid?**

- 1) `int[] a=new int[]//C.E: array dimension missing(invalid)`
- 2) `int[][] a=new int[3][4];(valid)`
- 3) `int[][] a=new int[3][];(valid)`
- 4) `int[][] a=new int[][4];//C.E:' ' expected(invalid)`
- 5) `int[][][] a=new int[3][4][5];(valid)`
- 6) `int[][][] a=new int[3][4][];(valid)`
- 7) `int[][][] a=new int[3][][5];//C.E:' ' expected(invalid)`

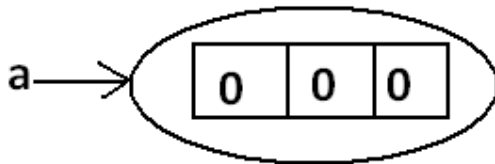
**Array Initialization:**

Whenever we are creating an array every element is initialized with default value automatically.

**Example 1:**

```
int[] a=new int[3];
System.out.println(a);//[I@3e25a5
System.out.println(a[0]);//0
```

Diagram:



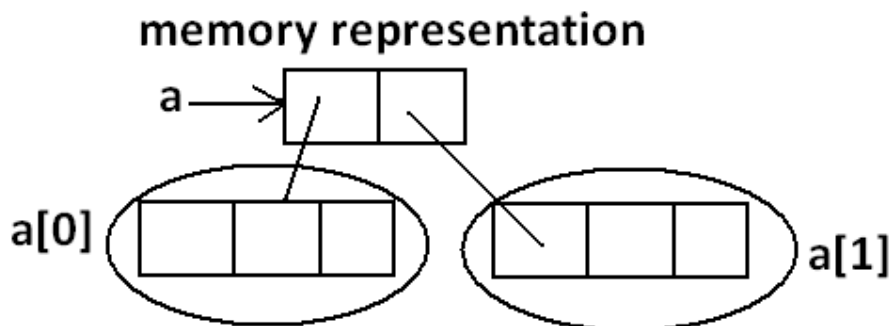
**Note:** Whenever we are trying to print any object reference internally toString() method will be executed which is implemented by default to return the following.  
classname@hexadecimalstringrepresentationofhashcode.

**Example 2:**

**int[][] a=new int[2][3]; base size**

```
System.out.println(a);//[[I@3e25a5
System.out.println(a[0]);//[I@19821f
System.out.println(a[0][0]);//0
```

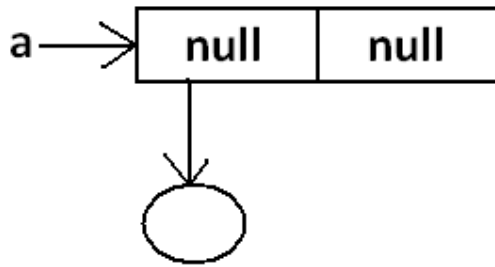
Diagram:



**Example 3:**

```
int[][] a=new int[2][];
System.out.println(a);//[[I@3e25a5
System.out.println(a[0]);//null
System.out.println(a[0][0]);//R.E:NullPointerException
```

Diagram:

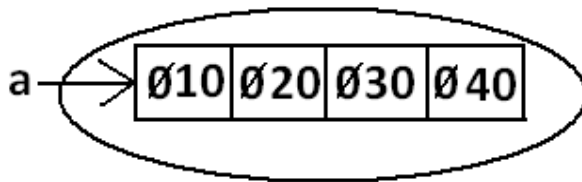


Once we created an array all its elements by default initialized with default values. If we are not satisfied with those default values then we can replays with our customized values.

**Example:**

```
int[] a=new int[4];  
a[0]=10;  
a[1]=20;  
a[2]=30;  
a[3]=40;  
a[4]=50;//R.E:ArrayIndexOutOfBoundsException: 4  
a[-4]=60;//R.E:ArrayIndexOutOfBoundsException: -4
```

**Diagram:**



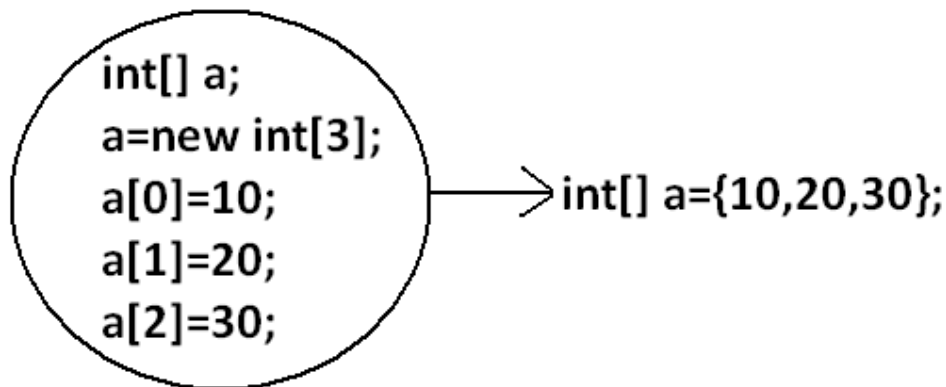
**Note:** if we are trying to access array element with out of range index we will get Runtime Exception saying `ArrayIndexOutOfBoundsException`.

**Declaration, construction and initialization of an array in a single line:**

We can perform declaration, construction and initialization of an array in a single line.



Example:



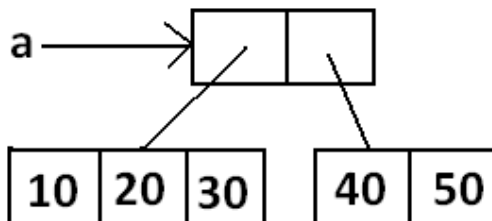
```
char[] ch={'a','e','i','o','u'};(valid)
String[] s={"balayya","venki","nag","chiru"};(valid)
```

We can extend this short cut even for multi dimensional arrays also.

Example:

```
int[][] a={{10,20,30},{40,50}};
```

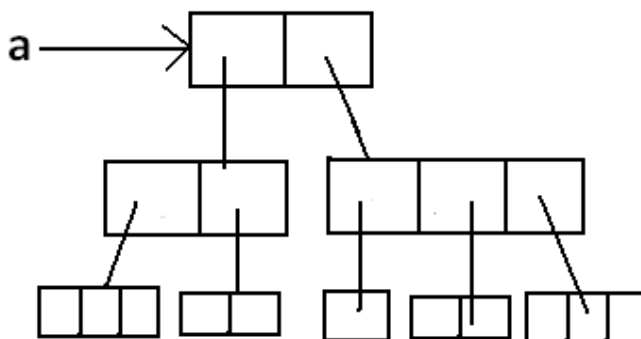
Diagram:



Example:

```
int[][][] a={{{10,20,30},{40,50}},{{60},{70,80},{90,100,110}}};
```

Diagram:

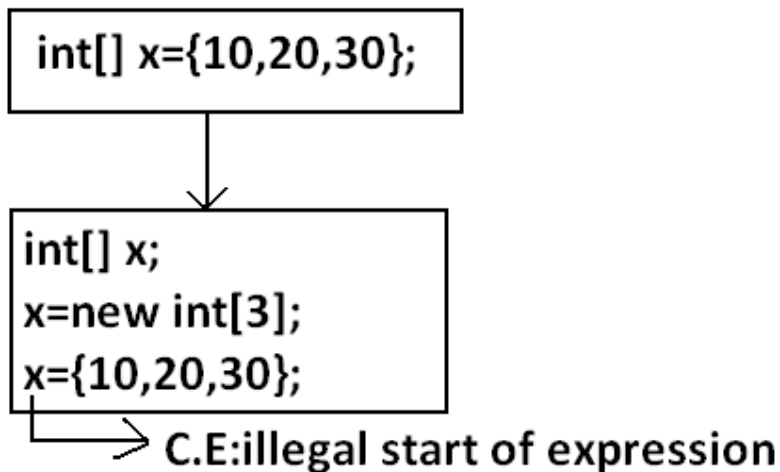


```
int[][][] a={{{10,20,30},{40,50}},{{60},{70,80},{90,100,110}}};
System.out.println(a[0][1][1]); //50(valid)
System.out.println(a[1][0][2]); //R.E:ArrayIndexOutOfBoundsException:
2(invalid)
```

```
System.out.println(a[1][2][1]); //100(valid)
System.out.println(a[1][2][2]); //110(valid)
System.out.println(a[2][1][0]); //R.E:ArrayIndexOutOfBoundsException:
2(invalid)
System.out.println(a[1][1][1]); //80(valid)
```

- If we want to use this short cut compulsory we should perform declaration, construction and initialization in a single line.
- If we are trying to divide into multiple lines then we will get compile time error.

Example:



**length Vs length():**

**length:**

1. It is the final variable applicable only for arrays.
2. It represents the size of the array.

Example:

```
int[] x=new int[3];
System.out.println(x.length()); //C.E: cannot find symbol
System.out.println(x.length()); //3
```

**length() method:**

1. It is a final method applicable for String objects.
2. It returns the no of characters present in the String.

Example:

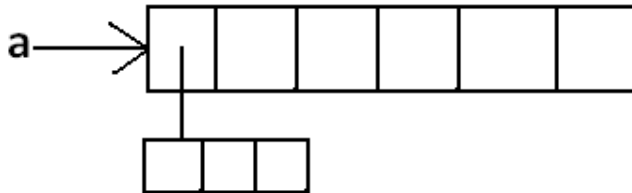
```
String s="bhaskar";
System.out.println(s.length()); //C.E:cannot find symbol
System.out.println(s.length()); //7
```

In multidimensional arrays length variable represents only base size but not total size.

Example:

```
int[][] a=new int[6][3];
System.out.println(a.length);//6
System.out.println(a[0].length);//3
```

Diagram:



length variable applicable only for arrays where as length()method is applicable for String objects.

There is no direct way to find total size of multi dimensional array but indirectly we can find as follows

$x[0].length + x[1].length + x[2].length + \dots$

**Anonymous Arrays:**

- Sometimes we can create an array without name such type of nameless arrays are called anonymous arrays.
- The main objective of anonymous arrays is "just for instant use".
- We can create anonymous array as follows.
- `new int[]{10,20,30,40}; (valid)`
- `new int[][]{{10,20},{30,40}}; (valid)`
- At the time of anonymous array creation we can't specify the size otherwise we will get compile time error.

Example:

```
new int[3]{10,20,30,40}; //C.E: ';' expected (invalid)
new int[]{10,20,30,40}; (valid)
```

Based on our programming requirement we can give the name for anonymous array then it is no longer anonymous.

Example:

```
int[] a=new int[]{10,20,30,40}; (valid)
```

Example:

```
class Test
{
    public static void main(String[] args)
    {
        System.out.println(sum(new int[]{10,20,30,40})); //100
    }
    public static int sum(int[] x)
    {
        int total=0;
        for(int x1:x)
        {
```

```

        total=total+x1;
    }
    return total;
}
}

```

In the above program just to call sum() , we required an array but after completing sum() call we are not using that array any more, anonymous array is best suitable.

### Array element assignments:

#### Case 1:

In the case of primitive array as array element any type is allowed which can be promoted to declared type.

#### Example 1:

For the int type arrays the allowed array element types are byte, short, char, int.

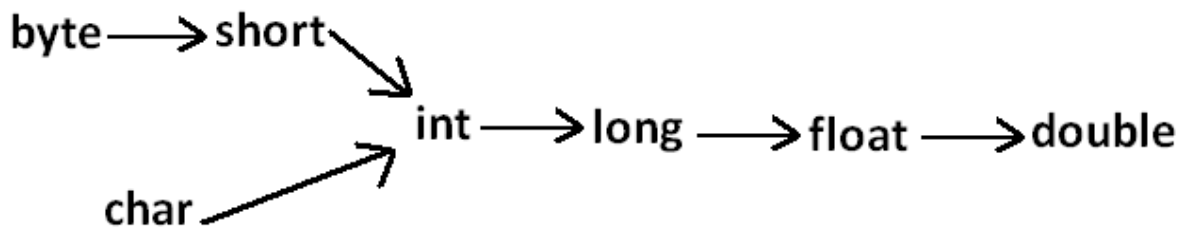
```

int[] a=new int[10];
a[0]=97;//(valid)
a[1]='a';//(valid)
byte b=10;
a[2]=b;//(valid)
short s=20;
a[3]=s;//(valid)
a[4]=101;//C.E:possible loss of precision

```

#### Example 2:

For float type arrays the allowed element types are byte, short, char, int, long, float.



#### Case 2:

In the case of Object type arrays as array elements we can provide either declared type objects or its child class objects.

#### Example 1:

```

Object[] a=new Object[10];
a[0]=new Integer(10);//(valid)
a[1]=new Object();//(valid)
a[2]=new String("bhaskar");//(valid)

```

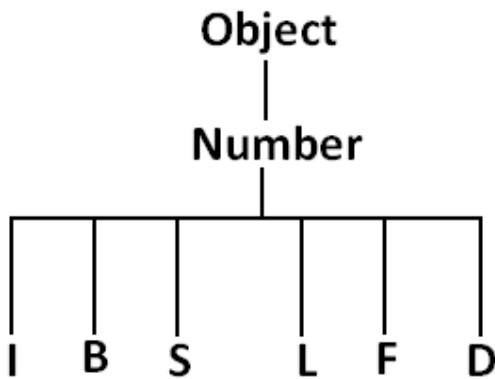
#### Example 2:

```

Number[] n=new Number[10];
n[0]=new Integer(10);//(valid)
n[1]=new Double(10.5);//(valid)
n[2]=new String("bhaskar");//C.E:incompatible types//(invalid)

```

Diagram:



Case 3:

In the case of interface type arrays as array elements we can provide its implemented class objects.

Example:

```

Runnable[] r=new Runnable[10];
r[0]=new Thread();
r[1]=new String("bhaskar");//C.E: incompatible types
  
```

| Array Type                     | Allowed Element Type                                        |
|--------------------------------|-------------------------------------------------------------|
| 1) Primitive arrays.           | 1) Any type which can be promoted to declared type.         |
| 2) Object type arrays.         | 2) Either declared type or its child class objects allowed. |
| 3) Interface type arrays.      | 3) Its implemented class objects allowed.                   |
| 4) Abstract class type arrays. | 4) Its child class objects are allowed.                     |

Array variable assignments:

Case 1:

- Element level promotions are not applicable at array object level.
- Ex : A char value can be promoted to int type but char array cannot be promoted to int array.

Example:

```

int[] a={10,20,30};
char[] ch={'a','b','c'};
int[] b=a;//(valid)
int[] c=ch;//C.E:incompatible types(invalid)
  
```

Which of the following promotions are valid?

- 1)char ————— int (valid)
- 2)char[] ————— int[] (invalid)
- 3)int ————— long (valid)
- 4)int[] ————— long[] (invalid)
- 5)double ————— float (invalid)
- 6)double[] ————— float[] (invalid)
- 7)String ————— Object (valid)
- 8)String[] ————— Object[] (valid)

**Note:** In the case of object type arrays child type array can be assign to parent type array variable.

**Example:**

```
String[] s={"A", "B"};  
Object[] o=s;
```

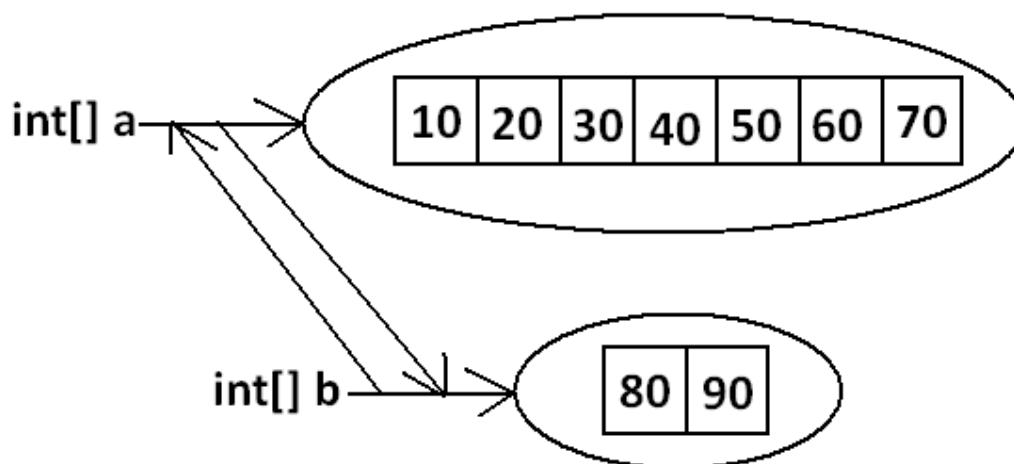
**Case 2:**

Whenever we are assigning one array to another array internal elements won't be copy just reference variables will be reassigned hence sizes are not important but types must be matched.

**Example:**

```
int[] a={10, 20, 30, 40, 50, 60, 70};  
int[] b={80, 90};  
a=b; //(valid)  
b=a; //(valid)
```

**Diagram:**



**Case 3:**

Whenever we are assigning one array to another array dimensions must be matched that is in the place of one dimensional array we should provide the same type only otherwise we will get compile time error.

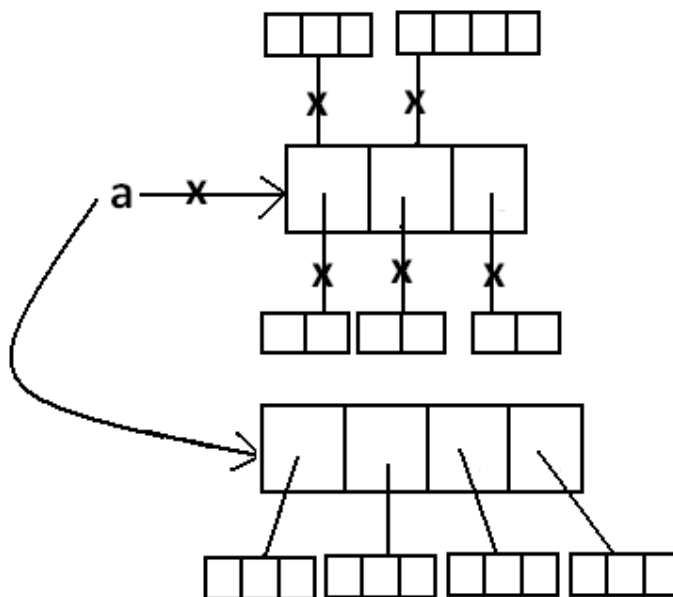
**Example:**

```
int[][] a=new int[3][];
a[0]=new int[4][5];//C.E:incompatible types(invalid)
a[0]=10;//C.E:incompatible types(invalid)
a[0]=new int[4];//(valid)
```

**Note:** Whenever we are performing array assignments the types and dimensions must be matched but sizes are not important.

**Example 1:**

```
int[][] a=new int[3][2];
a[0]=new int[3];
a[1]=new int[4];
a=new int[4][3];
```

**Diagram:**

**Total how many objects created?**

**Ans: 11**

**How many objects eligible for GC: 6**

**Example 2:**

```
class Test
{
    public static void main(String[] args)
    {
        String[] argh={"A", "B"};
        args=argh;
        System.out.println(args.length);//2
    }
}
```

```

        for(int i=0;i<=args.length;i++)
        {
            System.out.println(args[i]);
        }
    }
}

```

Output:

java Test x y

R.E: ArrayIndexOutOfBoundsException: 2

java Test x

R.E: ArrayIndexOutOfBoundsException: 2

java Test

R.E: ArrayIndexOutOfBoundsException: 2

Note: Replace with i<args.length

**Example 3:**

```

class Test
{
    public static void main(String[] args)
    {
        String[] argh={"A","B"};
        args=argh;
        System.out.println(args.length);//2
        for(int i=0;i<args.length;i++)
        {
            System.out.println(args[i]);
        }
    }
}

```

Output:

2

A

B

**Example 4:**

```

class Test
{
    public static void main(String[] args)
    {
        String[] argh={"A","B"};
        args=argh;

        for(String s : args) {
            System.out.println(s);
        }
    }
}

```

Output:

A

B

## Types of Variables

**Division 1 :** Based on the type of value represented by a variable all variables are divided into 2 types. They are:

1. Primitive variables
2. Reference variables



**Primitive variables:**

Primitive variables can be used to represent primitive values.

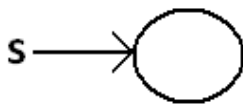
Example: `int x=10;`

**Reference variables:**

Reference variables can be used to refer objects.

Example: `Student s=new Student();`

Diagram:



**Division 2 :** Based on the behaviour and position of declaration all variables are divided into the following 3 types.

1. Instance variables
2. Static variables
3. Local variables

**Instance variables:**

- If the value of a variable is varied from object to object such type of variables are called instance variables.
- For every object a separate copy of instance variables will be created.
- Instance variables will be created at the time of object creation and destroyed at the time of object destruction hence the scope of instance variables is exactly same as scope of objects.
- Instance variables will be stored on the heap as the part of object.
- Instance variables should be declared with in the class directly but outside of any method or block or constructor.
- Instance variables can be accessed directly from Instance area. But cannot be accessed directly from static area.
- But by using object reference we can access instance variables from static area.

Example:

```

class Test
{
    int i=10;
    public static void main(String[] args)
    {
        //System.out.println(i);
        //C.E:non-static variable i cannot be referenced from a static
        context(invalid)
        Test t=new Test();
        System.out.println(t.i);//10(valid)
        t.methodOne();
    }
}
  
```

```

    }
    public void methodOne()
    {
        System.out.println(i);//10(valid)
    }
}

```

For the instance variables it is not required to perform initialization JVM will always provide default values.

**Example:**

```

class Test
{
    boolean b;
    public static void main(String[] args)
    {
        Test t=new Test();
        System.out.println(t.b);//false
    }
}

```

Instance variables also known as object level variables or attributes.

**Static variables:**

- If the value of a variable is not varied from object to object such type of variables is not recommended to declare as instance variables. We have to declare such type of variables at class level by using static modifier.
- In the case of instance variables for every object a separate copy will be created but in the case of static variables for entire class only one copy will be created and shared by every object of that class.
- Static variables will be created at the time of class loading and destroyed at the time of class unloading hence the scope of the static variable is exactly same as the scope of the .class file.
- Static variables will be stored in method area. Static variables should be declared within the class directly but outside of any method or block or constructor.
- Static variables can be accessed from both instance and static areas directly.
- We can access static variables either by class name or by object reference but usage of class name is recommended.
- But within the same class it is not required to use class name we can access directly.

### java TEST

1. Start JVM.
2. Create and start Main Thread by JVM.
3. Locate(find) Test.class by main Thread.
4. Load Test.class by main Thread. // static variable creation
5. Execution of main() method.
6. Unload Test.class // static variable destruction
7. Terminate main Thread.
8. Shutdown JVM.

Example:

```
class Test
{
    static int i=10;
    public static void main(String[] args)
    {
        Test t=new Test();
        System.out.println(t.i);//10
        System.out.println(Test.i);//10
        System.out.println(i);//10
    }
}
```

For the static variables it is not required to perform initialization explicitly, JVM will always provide default values.

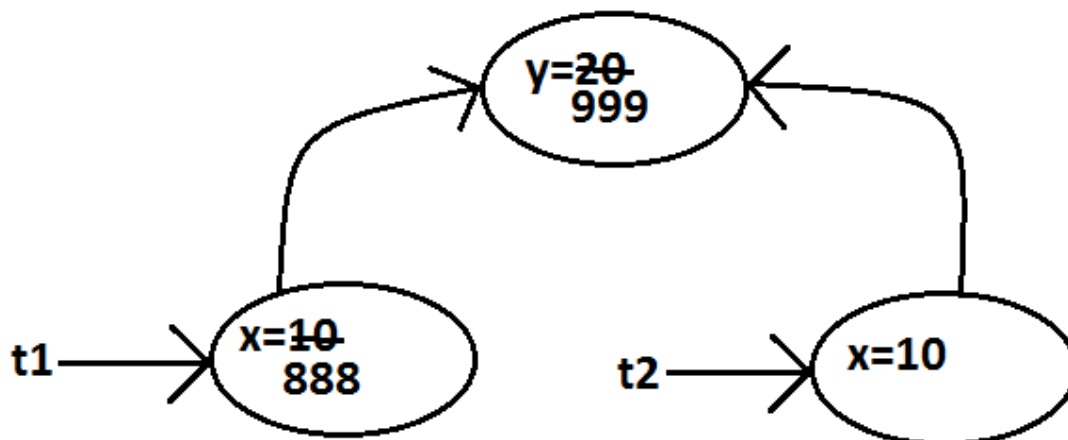
Example:

```
class Test
{
    static String s;
    public static void main(String[] args)
    {
        System.out.println(s);//null
    }
}
```

Example:

```
class Test
{
    int x=10;
    static int y=20;
    public static void main(String[] args)
    {
        Test t1=new Test();
        t1.x=888;
        t1.y=999;
        Test t2=new Test();
        System.out.println(t2.x+"-----"+t2.y);//10----999
    }
}
```

Diagram:



Static variables also known as class level variables or fields.

**Local variables:**

Some times to meet temporary requirements of the programmer we can declare variables inside a method or block or constructors such type of variables are called local variables or automatic variables or temporary variables or stack variables.

Local variables will be stored inside stack.

The local variables will be created as part of the block execution in which it is declared and destroyed once that block execution completes. Hence the scope of the local variables is exactly same as scope of the block in which we declared.

**Example 1:**

```
class Test
{
    public static void main(String[] args)
    {
        int i=0;
        for(int j=0;j<3;j++)
        {
            i=i+j;
        }
    }
}
```

**System.out.println(i+"----"+j);**

C.E

javac Test.java

Test.java:10: cannot find symbol  
symbol : variable j  
location: class Test

```
}
}
```

**Example 2:**

```
class Test
{
    public static void main(String[] args)
    {
        try
        {
            int i=Integer.parseInt("ten");
        }
        catch(NullPointerException e)
        {
        }
    }
}
```

**System.out.println(i);**

C.E

```
javac Test.java
Test.java:11: cannot find symbol
symbol : variable i
location: class Test
```

```
    }
}
}
```

- The local variables will be stored on the stack.
- For the local variables JVM won't provide any default values compulsory we should perform initialization explicitly before using that variable.

Example:

|                                                                                                                                             |                                                                                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>class Test {     public static void main(String[] args)     {         int x;         System.out.println("hello");//hello     } }</pre> | <pre>class Test {     public static void main(String[] args)     {         int x;         System.out.println(x);//C.E:variable x might     } }</pre> |
|---------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|

Example:

```
class Test
{
    public static void main(String[] args)
    {
        int x;
        if(args.length>0)
        {
            x=10;
        }
        System.out.println(x);
        //C.E:variable x might not have been initialized
    }
}
```

**Example:**

```
class Test
{
    public static void main(String[] args)
    {
        int x;
        if(args.length>0)
        {
            x=10;
        }
        else
        {
            x=20;
        }
        System.out.println(x);
    }
}
```

**Output:**

```
java Test x
10
java Test x y
10
java Test
20
```

- It is never recommended to perform initialization for the local variables inside logical blocks because there is no guarantee of executing that block always at runtime.
- It is highly recommended to perform initialization for the local variables at the time of declaration at least with default values.

**Note:** The only applicable modifier for local variables is final. If we are using any other modifier we will get compile time error.

**Example:**

```
class Test
{
    public static void main(String[] args)
    {
        expression
        public int x=10; //(invalid)
        private int x=10; //(invalid)
        protected int x=10; //(invalid)    C.E: illegal start of
        static int x=10; //(invalid)
        volatile int x=10; //(invalid)
        transient int x=10; //(invalid)
        final int x=10; //(valid)
    }
}
```

**Conclusions:**

1. For the static and instance variables it is not required to perform initialization explicitly JVM will provide default values. But for the local variables JVM won't

provide any default values compulsory we should perform initialization explicitly before using that variable.

2. For every object a separate copy of instance variable will be created whereas for entire class a single copy of static variable will be created. For every Thread a separate copy of local variable will be created.
3. Instance and static variables can be accessed by multiple Threads simultaneously and hence these are not Thread safe but local variables can be accessed by only one Thread at a time and hence local variables are Thread safe.
4. If we are not declaring any modifier explicitly then it means default modifier but this rule is applicable only for static and instance variables but not local variable.

## Un Initialized arrays

Example:

```
class Test
{
    int[] a;
    public static void main(String[] args)
    {
        Test t1=new Test();
        System.out.println(t1.a);//null
        System.out.println(t1.a[0]);//R.E:NullPointerException
    }
}
```

Instance level:

Example 1:

```
int[] a;
System.out.println(obj.a);//null
System.out.println(obj.a[0]);//R.E:NullPointerException
```

Example 2:

```
int[] a=new int[3];
System.out.println(obj.a);//[I@3e25a5
System.out.println(obj.a[0]);//0
```

Static level:

Example 1:

```
static int[] a;
System.out.println(a);//null
System.out.println(a[0]);//R.E:NullPointerException
```

Example 2:

```
static int[] a=new int[3];
System.out.println(a);//[I@3e25a5
System.out.println(a[0]);//0
```

Local level:

Example 1:

```
int[] a;
System.out.println(a); //C.E: variable a might not have been initialized
System.out.println(a[0]);
```

Example 2:

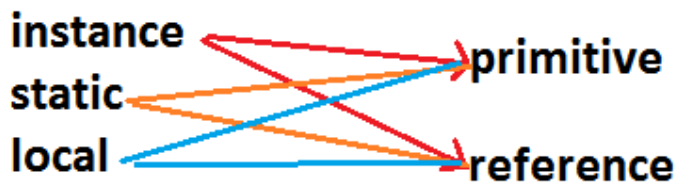
```
int[] a=new int[3];
System.out.println(a);//[I@3e25a5
System.out.println(a[0]);//0
```

Once we created an array every element is always initialized with default values irrespective of whether it is static or instance or local array.

Every variable in java should be either instance or static or local.

Every variable in java should be either primitive or reference

Hence the following are the various possible combinations for variables



```
class Test {
    int[] a=new int[3];    // instance-reference
    static int x=20;       //static-primitive
    public static void main(String[] args) {
        String s="xyz";    //local-reference
    }
}
```

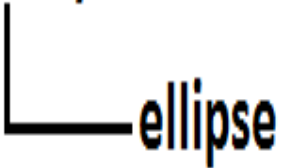
Var- arg methods (variable no of argument methods) (1.5)

- Until 1.4v we can't declared a method with variable no. Of arguments.
- If there is a change in no of arguments compulsory we have to define a new method.
- This approach increases length of the code and reduces readability.
- But from 1.5 version onwards we can declare a method with variable no. Of arguments such type of methods are called var-arg methods.



We can declare a var-arg method as follows.

`methodOne(int... x)`



We can call or invoke this method by passing any no. Of int values including zero number also.

**Example:**

```
class Test
{
    public static void methodOne(int... x)
    {
        System.out.println("var-arg method");
    }
    public static void main(String[] args)
    {
        methodOne();
        methodOne(10);
        methodOne(10, 20, 30);
    }
}
```

**Output:**

```
var-arg method
var-arg method
var-arg method
```

Internally var-arg parameter implemented by using single dimensional array hence within the var-arg method we can differentiate arguments by using index.

**Example:**

```
class Test
{
    public static void sum(int... x)
    {
        int total=0;
        for(int i=0;i<x.length;i++)
        {
            total=total+x[i];
        }
        System.out.println("The sum :"+total);
    }
    public static void main(String[] args)
    {
        sum();
        sum(10);
        sum(10, 20);
        sum(10, 20, 30, 40);
    }
}
```

```

    }
}
Output:
The sum: 0
The sum: 10
The sum: 30
The sum: 100

```

**Example:**

```

class Test
{
    public static void sum(int... x)
    {
        int total=0;
        for(int x1 : x)
        {
            total=total+x1;
        }
        System.out.println("The sum :"+total);
    }
    public static void main(String[] args)
    {
        sum();
        sum(10);
        sum(10,20);
        sum(10,20,30,40);
    }
}

```

**Output:**  
The sum: 0  
The sum: 10  
The sum: 30  
The sum: 100

**Case 1:**

Which of the following var-arg method declarations are valid?

1. methodOne(int... x) (valid)
2. methodOne(int ...x) (valid)
3. methodOne(int...x) (valid)
4. methodOne(int x...) (invalid)
5. methodOne(int. ..x) (invalid)
6. methodOne(int .x..) (invalid)

**Case 2:**

We can mix var-arg parameter with general parameters also.

**Example:**

```

methodOne(int a,int... b)           //valid
methodOne(String s,int... x)       //valid

```

**Case 3:**

if we mix var-arg parameter with general parameter then var-arg parameter should be

the last parameter.

**Example:**

```
methodOne(int a,int... b)           //valid
methodOne(int... a,int b)          //(invalid)
```

**Case 4:**

With in the var-arg method we can take only one var-arg parameter. i.e., if we are trying to more than one var-arg parameter we will get CE.

**Example:**

```
methodOne(int... a,int... b) //(invalid)
```

**Case 5:**

```
class Test
{
    public static void methodOne(int i)
    {
        System.out.println("general method");
    }
    public static void methodOne(int... i)
    {
        System.out.println("var-arg method");
    }
    public static void main(String[] args)
    {
        methodOne();//var-arg method
        methodOne(10,20);//var-arg method
        methodOne(10);//general method
    }
}
```

In general var-arg method will get least priority that is if no other method matched then only var-arg method will get the chance this is exactly same as default case inside a switch.

**Case 6:**

For the var-arg methods we can provide the corresponding type array as argument.

**Example:**

```
class Test
{
```

```
    public static void methodOne(int... i)  int[] i
    {
        System.out.println("var-arg method");
    }
    public static void main(String[] args)
    {
        methodOne(new int[]{10,20,30});//var-arg method
    }
}
```

**Case 7:**

```
class Test
{
    public void methodOne(int[] i){}
```

```

        public void methodOne(int... i){}
    }

```

Output:

Compile time error.

Cannot declare both methodOne(int...) and methodOne(int[]) in Test

### Single Dimensional Array Vs Var-Arg Method:

Case 1:

Wherever single dimensional array present we can replace with var-arg parameter.

**methodOne(int[] i)  $\Rightarrow$  methodOne(int... i) (valid)**

Example:

```

class Test
{
    public static void main(String... args)
    {
        System.out.println("var-arg main method");//var-arg main
    }
}

```

Case 2:

Wherever var-arg parameter present we can't replace with single dimensional array.

**methodOne(int... i)  $\Rightarrow$  methodOne(int[] i) (invalid)**

Note :

1. **methodOne(int... x)**  
we can call this method by passing a group of int values and x will become 1D array. (i.e., int[] x)
2. **methodOne(int[]... x)**  
we can call this method by passing a group of 1D int[] and x will become 2D array. ( i.e., int[][] x)

Above reasons this case 2 is invalid.

Example:

```

class Test
{
    public static void methodOne(int[]... x)
    {
        for(int[] a:x)
        {
            System.out.println(a[0]);
        }
    }
}

```

```

    }
}
public static void main(String[] args)
{
    int[] l={10,20,30};
    int[] m={40,50};
    methodOne(l,m);
}

```

Output:

10

40

Analysis:

**methodOne(int... x)**  
**methodOne(10,20);**  $x \rightarrow$ 

|    |    |
|----|----|
| 10 | 20 |
|----|----|

**methodOne(int[]... x)**  $x \rightarrow$ 

|  |  |
|--|--|
|  |  |
|--|--|

  
**int[] l={10,20,30};**  
**int[] m={40,50};**  
**methodOne(l,m);**

|    |    |    |
|----|----|----|
| 10 | 20 | 30 |
|----|----|----|

|    |    |
|----|----|
| 40 | 50 |
|----|----|

## Main Method

Whether the class contains main() method or not, and whether it is properly declared or not, these checking's are not responsibilities of the compiler, at runtime JVM is responsible for this.

If JVM unable to find the required main() method then we will get runtime exception saying `NoSuchMethodError: main`.

Example:

```

class Test
{
}

```

Output:

```

javac Test.java

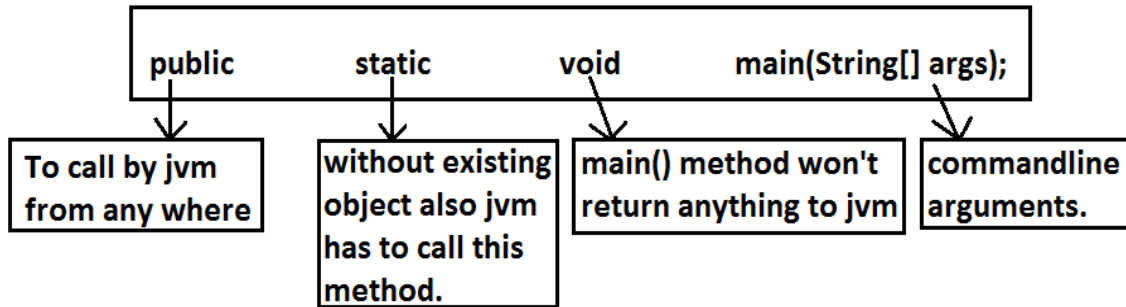
```

```

java Test R.E: NoSuchMethodError: main

```

At runtime JVM always searches for the main() method with the following prototype.



If we are performing any changes to the above syntax then the code won't run and will get Runtime exception saying `NoSuchMethodError`.

Even though above syntax is very strict but the following changes are acceptable to `main()` method.

1. The order of modifiers is not important that is instead of `public static` we can take `static public`.
2. We can declare `string[]` in any acceptable form
  - o `String[] args`
  - o `String []args`
  - o `String args[]`
3. Instead of `args` we can use any valid java identifier.
4. We can replace `string[]` with var-arg parameter.  
Example: `main(String... args)`
5. `main()` method can be declared with the following modifiers.  
`final`, `synchronized`, `strictfp`.
6. `class Test {`
7. `static final synchronized strictfp public void main(String... ask){`
8. `System.out.println("valid main method");`
9. `}`
10. `}`
11. output :
12. valid main method

Which of the following `main()` method declarations are valid ?

1. `public static void main(String args){}` (invalid)
2. `public synchronized final strictfp void main(String[] args){}` (invalid)
3. `public static void Main(String... args){}` (invalid)
4. `public static int main(String[] args){}` //int return type we can't take //(invalid)
5. `public static synchronized final strictfp void main(String... args){}`(valid)
6. `public static void main(String... args){}`(valid)
7. `public void main(String[] args){}`(invalid)

In which of the above cases we will get compile time error ?

No case, in all the cases we will get runtime exception.

Case 1 :

Overloading of the `main()` method is possible but JVM always calls `string[]` argument `main()` method only.

**Example:**

```
class Test
{
    public static void main(String[] args)
    {
        System.out.println("String[] array main method");    //overloaded
    }
    public static void main(int[] args)
    {
        System.out.println("int[] array main method");
    }
}
```

Output:

String[] array main method

The other overloaded method we have to call explicitly then only it will be executed.

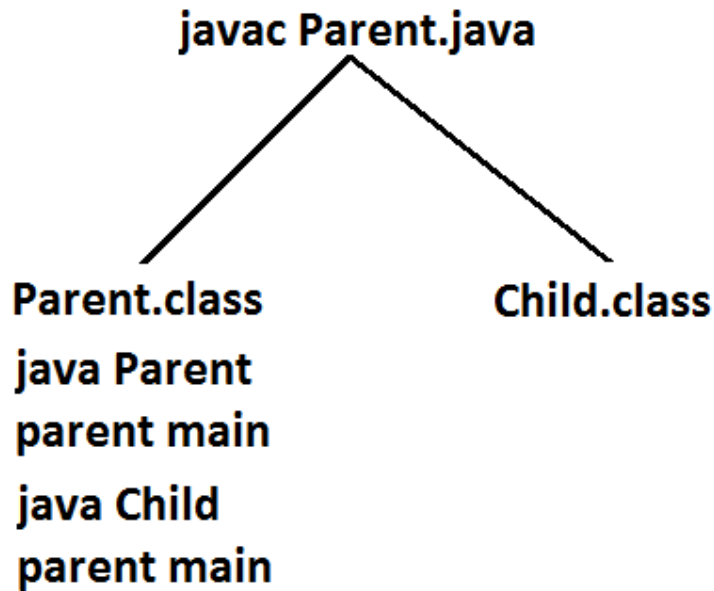
**Case 2:**

Inheritance concept is applicable for static methods including main() method hence while executing child class if the child class doesn't contain main() method then the parent class main() method will be executed.

**Example 1:**

```
class Parent
{
    public static void main(String[] args)
    {
        System.out.println("parent main");    //Parent.java
    }
}
class Child extends Parent
{}
```

Analysis:

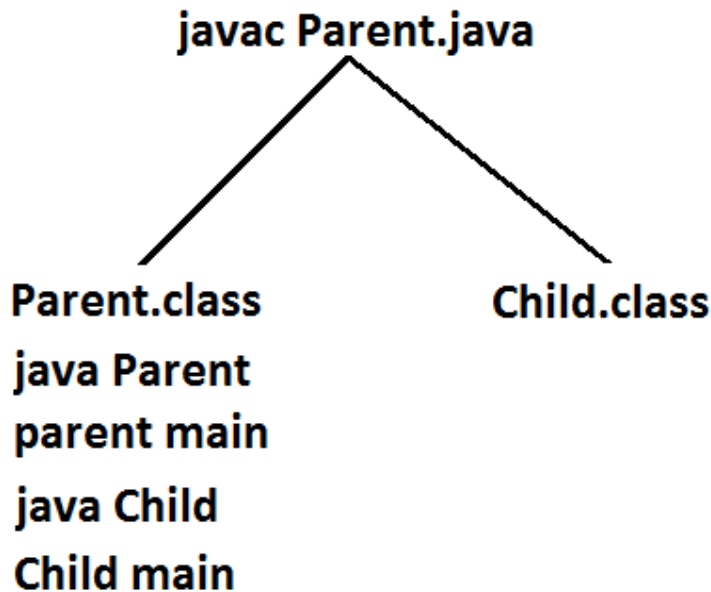


Example 2:

```
class Parent
{
    public static void main(String[] args)
    {
        System.out.println("parent main");           // Parent.java
    }
}
class Child extends Parent
{
    public static void main(String[] args)
    {
        System.out.println("Child main");
    }
}
```



Analysis:



It seems to be overriding concept is applicable for static methods but it is not overriding it is method hiding.

### 1.7 Version Enhancements with respect to main() :

Case 1 :

- Untill 1.6v if our class doesn't contain main() method then at runtime we will get Runtime Exception saying NoSuchMethodError:main
- But from 1.7 version onwards instead of NoSuchMethodError we will get more meaning full description

```
class Test {
}
```

1.6 version :  
javac Test.java  
java Test  
RE: NoSuchMethodError:main

1.7 version :  
javac Test.java  
java Test  
Error: main method not found in class Test, please define the main method as  
public static void main(String[] args)

**Case 2 :**

From 1.7 version onwards to start program execution compulsory main method should be required, hence even though the class contains static block if main method not available then won't be executed

```
class Test {  
    static {  
        System.out.println("static block");  
    }  
}
```

1.6 version :

```
javac Test.java
```

```
java Test
```

```
output :
```

```
static block
```

```
RE: NoSuchMethodError:main
```

1.7 version :

```
javac Test.java
```

```
java Test
```

```
Error: main method not found in class Test, please define the main method  
as
```

```
public static void main(String[] args)
```

**Case 3 :**

```
class Test {  
    static {  
        System.out.println("static block");  
        System.exit(0);  
    }  
}
```

1.6 version :

```
javac Test.java
```

```
java Test
```

```
output :
```

```
static block
```

1.7 version :

```
javac Test.java
```

```
java Test
```

```
Error: main method not found in class Test, please define the main method  
as
```

```
public static void main(String[] args)
```

**Case 4 :**

```
class Test {  
    static {  
        System.out.println("static block");  
    }  
    public static void main(String[] args) {  
        System.out.println("main method");  
    }  
}
```

1.6 version :

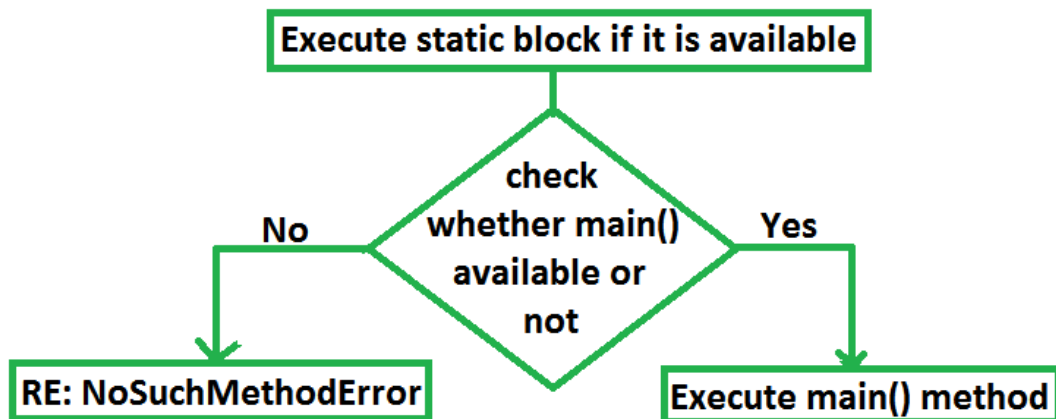
```
javac Test.java
```

```
java Test
```

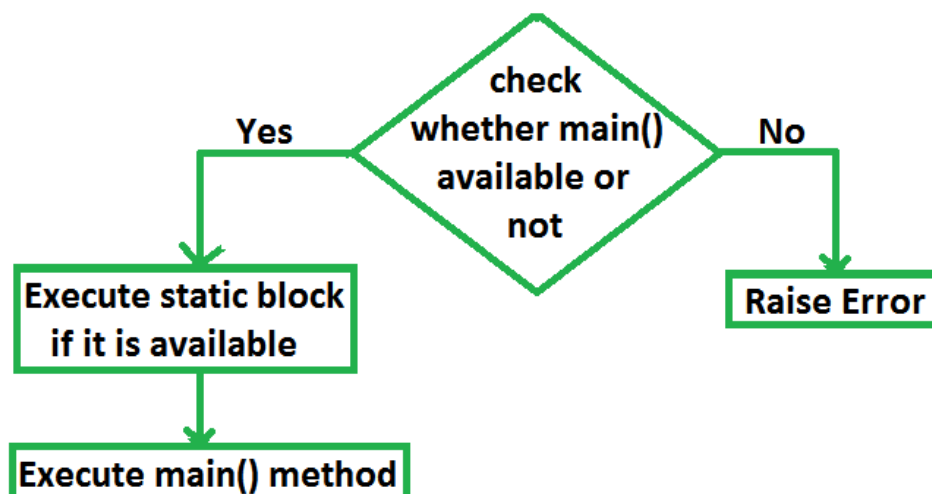
output :  
static block  
main method

1.7 version :  
javac Test.java  
java Test  
output :  
static block  
main method

### 1.6 version :



### 1.7 version :



### Command line arguments:

The arguments which are passing from command prompt are called command line arguments.

The main objective of command line arguments are we can customize the behavior of the main() method.

```
java Test 10 20 30
```

args[0]  
args[1]  
args[2]  
args.length → 3

**Example 1:**

```
class Test
{
    public static void main(String[] args)
    {
        for(int i=0;i<=args.length;i++)
        {
            System.out.println(args[i]);
        }
    }
}
```

**Output:**

```
java Test x y z
```

```
ArrayIndexOutOfBoundsException: 3
```

Replace `i<=args.length` with `i<args.length` then it will run successfully.

**Example 2 :**

```
class Test
{
    public static void main(String[] args)
    {
        String[] argh={"X","Y","Z"};
        args=argh;
        for(String s : args)
        {
            System.out.println(s);
        }
    }
}
```

**Output:**

```
java Test A B C
```

```
X
```

```
Y
```

```
Z
```

```
java Test A B
```

```
X
```

```
Y
```

```
Z
```

```
java Test
```

```
X
```

Y  
Z

Within the main() method command line arguments are available in the form of String

hence "+" operator acts as string concatenation but not arithmetic addition.

**Example 3 :**

```
class Test
{
    public static void main(String[] args)
    {
        System.out.println(args[0]+args[1]);
    }
}
```

Output:

E:\SCJP>javac Test.java

E:\SCJP>java Test 10 20

1020

Space is the separator between 2 command line arguments and if our command line argument itself contains space then we should enclose with in double quotes.

**Example 4 :**

```
class Test
{
    public static void main(String[] args)
    {
        System.out.println(args[0]);
    }
}
```

Output:

E:\SCJP>javac Test.java

E:\SCJP>java Test "Sai Charan"

Sai Charan

## Java coding standards

- Whenever we are writing java code , It is highly recommended to follow coding standards , which improves the readability and understandability of the code.
- Whenever we are writing any component(i.e., class or method or variable) the name of the component should reflect the purpose or functionality.

Example:

|                                                                                                                        |                                                                                                                                                                                       |
|------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>class A {     public int methodOne(int x,int y)     {         return x+y;     } }</pre> <p>Ameerpet standards</p> | <pre>package com.durgasoft.scjpdemo; class Calc {     public static int add(int number1,int number2)     {         return number1+number2;     } }</pre> <p>Hitech-city standards</p> |
|------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Coding standards for classes:

- Usually class names are nouns.
- Should starts with uppercase letter and if it contains multiple words every inner word should starts with upper case letter.

Example:

**String, Customer, Object, Student, StringBuffer** → nouns

Coding standards for interfaces:

- Usually interface names are adjectives.
- Should starts with upper case letter and if it contains multiple words every inner word should starts with upper case letter.

Example:

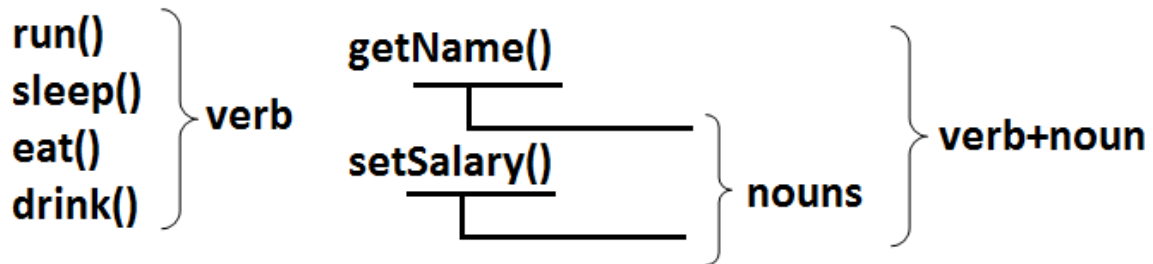
1. Serializable
2. Runnable
3. Cloneable

these are adjectives

Coding standards for methods:

- Usually method names are either verbs or verb-noun combination.
- Should starts with lowercase character and if it contains multiple words every inner word should starts with upper case letter.(camel case convention)

Example:



Coding standards for variables:

- Usually variable names are nouns.
- Should starts with lowercase alphabet symbol and if it contains multiple words every inner word should starts with upper case character.(camel case convention)

Example:

length  
name  
salary  
age  
mobileNumber

nouns

Coding standards for constants:

- Usually constants are nouns.
- Should contain only uppercase characters and if it contains multiple words then these words are separated with underscore symbol.
- Usually we can declare constants by using public static final modifiers.

Example:

MAX\_VALUE

MIN\_VALUE

NORM\_PRIORITY

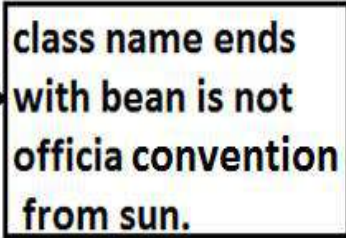
nouns

Java bean coding standards:

A java bean is a simple java class with private properties and public getter and setter methods.

Example:

```
class StudentBean
{
    private String name;
    public void setName(String name)
    {
        this.name=name;
    }
    public String getName()
    {
        return name;
    }
}
```



Syntax for setter method:

1. Method name should be prefixed with set.
2. It should be public.
3. Return type should be void.
4. Compulsory it should take some argument.

Syntax for getter method:

1. The method name should be prefixed with get.
2. It should be public.
3. Return type should not be void.
4. It is always no argument method.



**Note:** For the boolean properties the getter method can be prefixed with either get or is. But recommended to use is.

**Example:**

|                                                                                                   |  |                                                                                                  |
|---------------------------------------------------------------------------------------------------|--|--------------------------------------------------------------------------------------------------|
| <pre>private boolean empty;<br/>public boolean getEmpty()<br/>{<br/>    return empty;<br/>}</pre> |  | <pre>private boolean empty;<br/>public boolean isEmpty()<br/>{<br/>    return empty;<br/>}</pre> |
| (valid)                                                                                           |  | (valid)                                                                                          |
| <b>both are valid.</b>                                                                            |  |                                                                                                  |

### Coding standards for listeners:

To register a listener:

Method name should be prefixed with add.

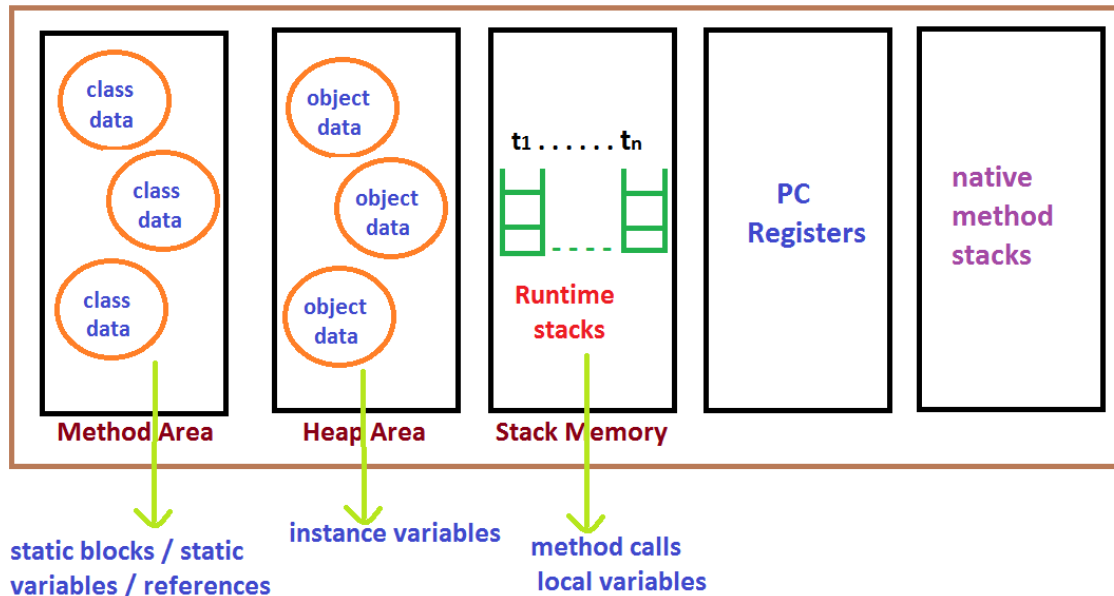
1. `public void addMyActionListener(MyActionListener l)` (valid)
2. `public void registerMyActionListener(MyActionListener l)` (invalid)
3. `public void addMyActionListener(ActionListener l)` (invalid)

To unregister a listener:

The method name should be prefixed with remove.

1. `public void removeMyActionListener(MyActionListener l)` (valid)
2. `public void unregisterMyActionListener(MyActionListener l)` (invalid)
3. `public void removeMyActionListener(ActionListener l)` (invalid)
4. `public void deleteMyActionListener(MyActionListener l)` (invalid)

## Various Memory areas present inside JVM :



1. Class level binary data including static variables will be stored in method area.
2. Objects and corresponding instance variables will be stored in Heap area.
3. For every method the JVM will create a Runtime stack all method calls performed by that Thread and corresponding local variables will be stored in that stack.  
Every entry in stack is called Stack Frame or Action Record.
4. The instruction which has to execute next will be stored in the corresponding PC Registers.
5. Native method invocations will be stored in native method stacks.

# **CORE JAVA**

## **With**

# **SCJP / OCJP**

### **Study Material**

#### **Chapter 2 : OPERATORS & ASSIGNMENTS**



**DURGA M.Tech**

**(Sun certified & Realtime Expert)**

**Ex. IBM Employee**

**Trained Lakhs of Students  
for last 14 years across INDIA**

**India's No.1 Software Training Institute**

# **DURGASOFT**

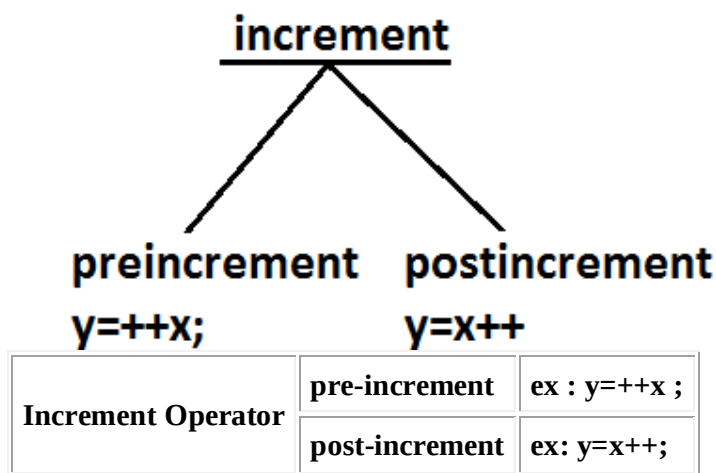
**[www.durgasoft.com](http://www.durgasoft.com) Ph: 9246212143 ,8096969696**

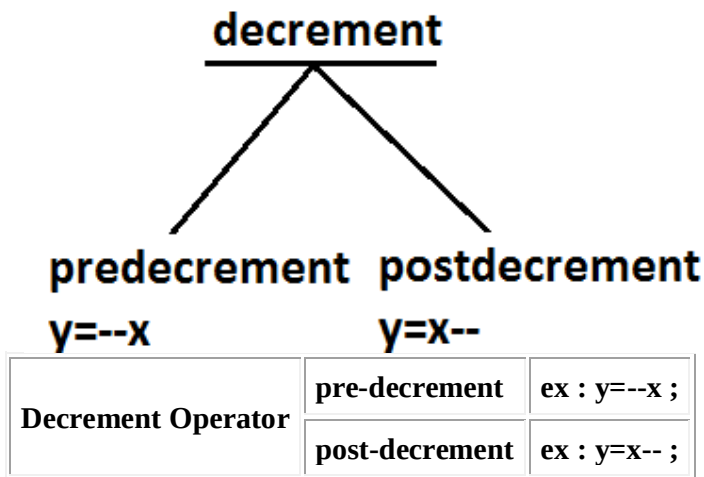
# OPERATORS & ASSIGNMENTS

## Agenda:

1. increment & decrement operators
2. arithmetic operators
3. string concatenation operators
4. Relational operators
5. Equality operators
6. instanceof operators
7. Bitwise operators
8. Short circuit operators
9. type cast operators
10. assignment operator
11. conditional operator
12. new operator
13. [ ] operator
14. Precedence of java operators
15. Evaluation order of java operands
16. new Vs newInstance( )
17. instanceof Vs instanceof( )
18. ClassNotFoundException Vs NoClassDefFoundError

## Increment & Decrement operators :





The following table will demonstrate the use of increment and decrement operators.

| Expression | initial value of x | value of y | final value of x |
|------------|--------------------|------------|------------------|
| y=++x      | 10                 | 11         | 11               |
| y=x++      | 10                 | 10         | 11               |
| y=--x      | 10                 | 9          | 9                |
| y=x--      | 10                 | 10         | 9                |

Ex :

```
class Test{
public static void main(String[] args){
int x=4;
int y=++x;
System.out.println("value of y :"+y);
} output:
} 5
```

```
class Test{
public static void main(String[] args){
int x=4;
int y=++4;
System.out.println("value of y :"+y);
} output:
} compile time error
```

Test.java:4: unexpected type  
required: variable  
found : value  
int y=++4;

1. Increment & decrement operators we can apply only for variables but not for constant values. otherwise we will get compile time error .

Ex :

```
int x = 4;
int y = ++x;
System.out.println(y); //output : 5
```

Ex 2 :

```
int x = 4;
int y = ++4;
System.out.println(y);
```

```
C.E: unexpected type
required: variable
found : value
```

2. We can't perform nesting of increment or decrement operator, other wise we will get compile time error

```
class Test{
public static void main(String[] args){
int x=4;
int y=++(++x); it will become constant
System.out.println("value of y :"+y);
} output:
} compile time error
```

```
Test.java:4: unexpected type
required: variable
found : value
int y=++(++x);
```

```
int x= 4;
int y = ++(++x);
System.out.println(y);
```

```
C.E: unexpected type
required: variable
found : value
```

3. For the final variables we can't apply increment or decrement operators ,other wise we will get compile time error

```
class Test{
public static void main(String[] args){
final int x=4;
x++;
System.out.println("value of x:"+x);
} output:
} compile time error
```

**Test.java:4: cannot assign a value to final variable x**  
**x++;**

Ex:

```
final int x = 4;
x++;                // x = x + 1
System.out.println(x);
```

C.E : can't assign a value to final variable 'x' .

4. We can apply increment or decrement operators even for primitive data types except boolean .

Ex:

```
int x=10;
x++;
System.out.println(x);    //output :11
```

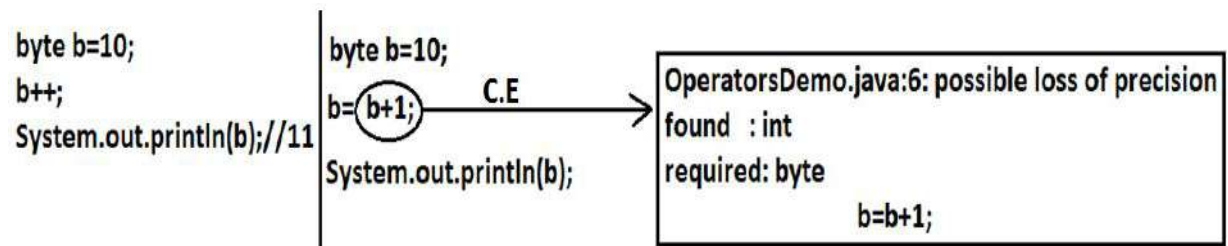
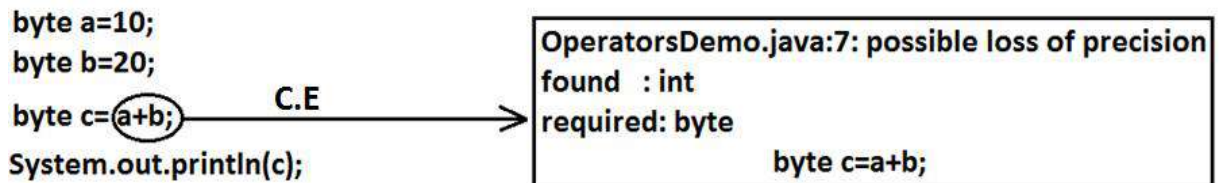
```
char ch='a';
ch++;
System.out.println(ch); //b
```

```
double d=10.5;
d++;
System.out.println(d); //11.5
```

```
boolean b=true;
b++;
System.out.println(b);
CE : operator ++ can't be applied to boolean
```

Difference between b++ and b = b+1?

If we are applying any arithmetic operators b/w 2 operands 'a' & 'b' the result type is max(int, type of a, type of b)



Ex 1:

```
byte a=10;
byte b=20;
byte c=a+b;
System.out.println(c);
```

//byte c=byte(a+b); valid

CE : possible loss of precision  
 found : int  
 required : byte

Ex 2:

```
byte b=20;
byte b=b+1;
System.out.println(c);
```

//byte b=(byte)b+1 ; valid

CE : possible loss of precision  
 found : int  
 required : byte

In the case of Increment & Decrement operators internal type casting will be performed automatically by the compiler

**b++; means**

**b=(type of b)(b+1);**

**b=(byte)(b+1);**

b++; => b=(type of b)b+1;

Ex:

```
byte b=10;
b++;
System.out.println(b); //output : 11
```



## Arithmetic Operator :

1. If we apply any Arithmetic operation b/w 2 variables a & b , the result type is always max(int , type of a , type of b)
2. Example :
- 3.
4. byte + byte=int
5. byte+short=int
6. short+short=int
7. short+long=long
8. double+float=double
9. int+double=double
10. char+char=int
11. char+int=int
12. char+double=double
- 13.
14. System.out.println('a' + 'b'); // output : 195
15. System.out.println('a' + 1); // output : 98
16. System.out.println('a' + 1.2); // output : 98.2

|                |                     |
|----------------|---------------------|
| byte+byte=int  | int+long=long       |
| byte+short=int | float+double=double |
| byte+int=int   | long+long=long      |
| char+char=int  | long+float=float    |
| char+int=int   |                     |
| byte+char=int  |                     |

17. In integral arithmetic (byte , int , short , long) there is no way to represents infinity , if infinity is the result we will get the ArithmeticException / by zero  
 System.out.println(10/0); // output RE : ArithmeticException / by zero  
 But in floating point arithmetic(float , double) there is a way represents infinity.  
 System.out.println(10/0.0); // output : infinity

System.out.println(10/0);  $\xrightarrow{\text{R.E}}$  Exception in thread "main" java.lang.ArithmeticException: / by zero

For the Float & Double classes contains the following constants :

1. POSITIVE\_INFINITY
2. NEGATIVE\_INFINITY

Hence , if infinity is the result we won't get any ArithmeticException in floating point arithmetics

Ex :

System.out.println(10/0.0); // output : infinity  
 System.out.println(-10/0.0); // output : - infinity

18. NaN(Not a Number) in integral arithmetic (byte , short , int , long) there is no way to represent undefined the results. Hence the result is undefined we will get ArithmeticException in integral arithmetic

`System.out.println(0/0);` // output RE : *ArithmeticException / by zero*

But floating point arithmetic (float , double) there is a way to represents undefined the results .

For the Float , Double classes contains a constant NaN , Hence the result is undefined we won't get ArithmeticException in floating point arithmetics .

`System.out.println(0.0/0.0);` // output : NaN

`System.out.println(-0.0/0.0);` // output : NaN

19. For any 'x' value including NaN , the following expressions returns false

`System.out.println(0/0);`  $\xrightarrow{\text{R.E}}$  Exception in thread "main" java.lang.ArithmeticException: / by zero

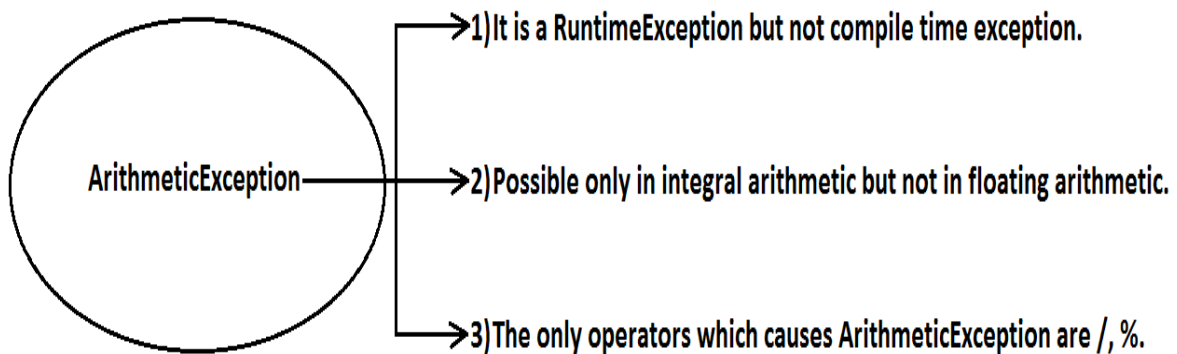
```

20. // Ex :      x=10;
21. System.out.println(10 < Float.NaN );           // false
22. System.out.println(10 <= Float.NaN );          // false
23. System.out.println(10 > Float.NaN );           // false
24. System.out.println(10 >= Float.NaN );          // false
25. System.out.println(10 == Float.NaN );          // false
26. System.out.println(Float.NaN == Float.NaN );   // false
27.
28. System.out.println(10 != Float.NaN );           //true
29. System.out.println(Float.NaN != Float.NaN );   //true

```

30. ArithmeticException :

1. It is a RuntimeException but not compile time error
2. It occurs only in integral arithmetic but not in floating point arithmetic.
3. The only operations which cause ArithmeticException are : ' / ' and ' % '



## String Concatenation operator :

1. The only overloaded operator in java is '+' operator some times it access arithmetic addition operator & some times it access String concatenation operator.
2. If acts as one argument is String type , then '+' operator acts as concatenation and If both arguments are number type , then operator acts as arithmetic operator

3. Ex :

```
4. String a="ashok";
   int b=10 , c=20 , d=30 ;
   System.out.println(a+b+c+d); //output : ashok102030
   System.out.println(b+c+d+a); //output : 60ashok
   System.out.println(b+c+a+d); //output : 30ashok30
   System.out.println(b+a+c+d); //output : 10ashok 2030
```

Example :

```
String a="bhaskar";
int b=10,c=20,d=30;
a=b+c+d; // C.E
System.out.println(c);
```

```
E:\scjp>javac OperatorsDemo.java
OperatorsDemo.java:7: incompatible types
found   : int
required: java.lang.String
a=b+c+d;
```

Example :

```
String a="bhaskar";
int b=10,c=20,d=30;
a=a+b+c;
c=b+d;
c=a+b+d;
System.out.println(a);//bhaskar1020
System.out.println(c);//40
System.out.println(c);
```

```
E:\scjp>javac OperatorsDemo.java
OperatorsDemo.java:9: incompatible types
found   : java.lang.String
required: int
c=a+b+d;
```

5. consider the following declaration

```
String a="ashok";
int b=10 , c=20 , d=30 ;
```

6. Example :

```
a=b+c+d ;
```

```
CE : incompatible type
      found : int
      required : java.lang.String
```

7. Example :

8.

```
a=a+b+c ; // valid
```

9. Example :

```

10.      b=a+c+d ;
11.
12.      CE : incompatible type
13.          found : java.lang.String
14.          required : int
15.  Example :
16.      b=b+c+d ;          // valid

```

## Relational Operators(< , <= , > , >= )

1. We can apply relational operators for every *primitive type* except *boolean* .

```

System.out.println(10>10.5);//false
System.out.println('a'>95.5);//true
System.out.println('z'>'a');//true
System.out.println(true>>false);

```

→ C.E

```

E:\scjp>javac OperatorsDemo.java
OperatorsDemo.java:8: operator > cannot be applied to boolean,boolean
    System.out.println(true>>false);

```

- ```

2.  System.out.println(10 < 10.5);    //true
3.  System.out.println('a' > 100.5);  //false
4.  System.out.println('b' > 'a');    //true
5.  System.out.println(true > false);
6.  //CE : operator > can't be applied to boolean , boolean

```

7. We can't apply relational operators for object types

```

System.out.println("bhaskar">"bhaskar");

```

→ C.E

```

OperatorsDemo.java:5: operator > cannot be applied to java.lang.String,java.lang.String
    System.out.println("bhaskar">"bhaskar");

```

- ```

8.  System.out.println("ashok123" > "ashok");
9.  // CE: operator > can't be applied to java.lang.String ,
    java.lang.String

```

10. Nesting of relational operator is not allowed

```

System.out.println(10<20<30);

```

→ C.E

```

E:\scjp>javac OperatorsDemo.java
OperatorsDemo.java:5: operator < cannot be applied to boolean,int
    System.out.println(10<20<30);

```

11. `System.out.println(10 > 20 > 30); // System.out.println(true > 30);`
12. `//CE : operator > can't be applied to boolean , int`

## Equality Operators : (== , !=)

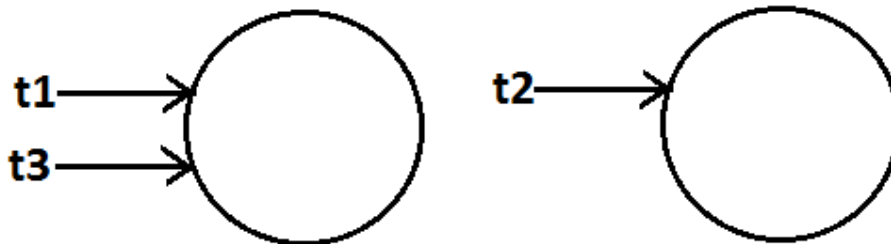
1. We can apply equality operators for every primitive type including boolean type also

2. `System.out.println(10 == 20) ; //false`
3. `System.out.println('a' == 'b' ); //false`
4. `System.out.println('a' == 97.0 ) //true`
5. `System.out.println(false == false) //true`

6. We can apply equality operators for object types also .

For object references r1 and r2 , r1 == r2 returns true if and only if both r1 and r2 pointing to the same object. i.e., == operator meant for reference-comparison Or address-comparison.

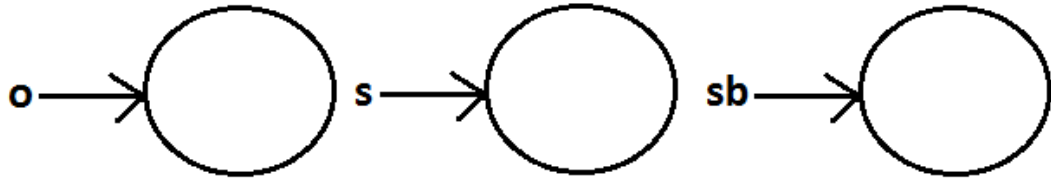
7. `Thread t1=new Thread( ) ;`
8. `Thread t2=new Thread( );`
9. `Thread t3=t1 ;`
10. `System.out.println(t1==t2); //false`
11. `System.out.println(t1==t3); //true`



12. To use the equality operators between object type compulsory these should be some relation between argument types(child to parent , parent to child) , Otherwise we will get Compiletime error incompatible types

13. `Thread t=new Thread( ) ;`
14. `Object o=new Object( );`
15. `String s=new String("durga");`
16. `System.out.println(t ==o); //false`
17. `System.out.println(o==s); //false`
18. `System.out.println(s==t);`
19. `CE : incompatible types : java.lang.String and java.lang.Thread`

System.out.println(s==sb);  $\xrightarrow{C.E}$  E:\scjp>javac OperatorsDemo.java  
OperatorsDemo.java:10: incomparable types: java.lang.String and java.lang.StringBuffer  
System.out.println(s==sb);



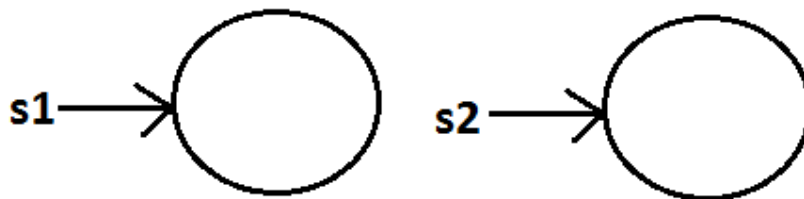
20. For any object reference of on  $r==null$  is always false , but  $null==null$  is always true .

```
21.      String s= new String("ashok");
22.      System.out.println(s==null); //output : false
23.      String s=null ;
24.      System.out.println(r==null); //true
25.      System.out.println(null==null); //true
```

26. What is the difference between  $==$  operator and `.equals()` method ?

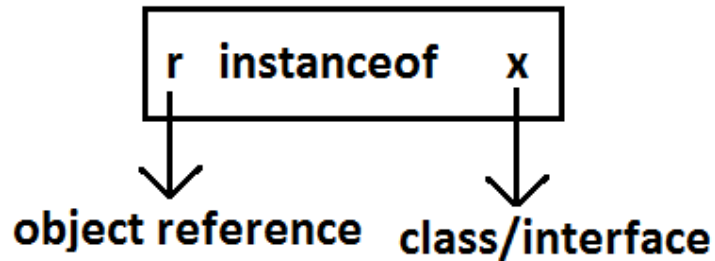
In general we can use `.equals()` for content comparison where as  $==$  operator for reference comparison

```
27.
28.      String s1=new String("ashok");
29.      String s2=new String("ashok");
30.      System.out.println(s1==s2); //false
31.      System.out.println(s1.equals(s2)); //true
```



## instanceof operator :

1. We can use the instanceof operator to check whether the given an object is particular type or not



```

2.      Object o=l.get(0);           // l is an array name
3.      if(o instanceof Student) {
4.          Student s=(Student)o ;
5.          //perform student specific operation
6.      }
7.      elseif(o instanceof Customer) {
8.          Customer c=(Customer)o;
9.          //perform Customer specific operations
10.     }
  
```

11. O instanceof X here O is object reference , X is ClassName/Interface name

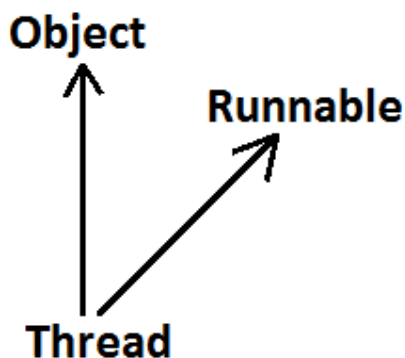
```

12.      Thread t = new Thread( );
13.      System.out.println(t instanceof Thread);    //true
14.      System.out.println(t instanceof Object);    //true
15.      System.out.println(t instanceof Runnable);  //true
  
```

Ex :

```

public class Thread extends Object implements Runnable {
}
  
```



16. To use instance of operator compulsory there should be some relation between argument types (either child to parent Or parent to child Or same type)  
Otherwise we will get compile time error saying inconvertible types

```
String s=new String("bhaskar");
System.out.println(s instanceof Thread);
```

```
E:\scjp>javac OperatorsDemo.java
OperatorsDemo.java:6: incompatible types
found   : java.lang.String
required: java.lang.Thread
System.out.println(s instanceof Thread);
```

```
17.
18.      Thread t=new Thread( );
19.      System.out.println(t instanceof String);
20.      CE : incompatible errors
21.      found : java.lang.Thread
22.      required : java.lang.String
```

23. Whenever we are checking the parent object is child type or not by using instanceof operator that we get false.

```
24.      Object o=new Object( );
25.      System.out.println(o instanceof String );
      //false
26.
27.      Object o=new String("ashok");
28.      System.out.println(o instanceof String);    //true
```

29. For any class or interface X null instanceof X is always returns false

```
30.      System.out.println(null instanceof X);    //false
```

## Bitwise Operators : ( & , | , ^ )

1. & (AND) : If both arguments are true then only result is true.
2. | (OR) : if at least one argument is true. Then the result is true.
3. ^ (X-OR) : if both are different arguments. Then the result is true.

Example:

```
System.out.println(true&false); //false
System.out.println(true|false); //true
System.out.println(true^false); //true
```

We can apply bitwise operators even for integral types also.

Example:

```
System.out.println(4&5); //4
System.out.println(4|5); //5
System.out.println(4^5); //1
```

using binary digits  
4-->100  
5-->101



Example :

|                                           |     |     |     |     |
|-------------------------------------------|-----|-----|-----|-----|
| <code>System.out.println(4&amp;5);</code> | //4 | 100 | 100 | 100 |
| <code>System.out.println(4 5);</code>     | //5 | 101 | 101 | 101 |
| <code>System.out.println(4^5);</code>     | //1 | 100 | 101 | 001 |

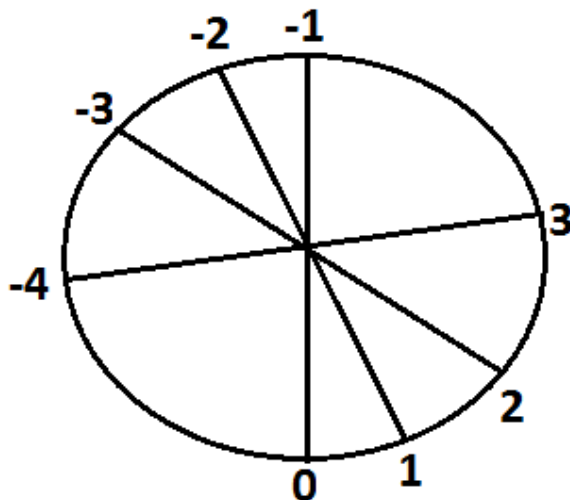
**Bitwise complement (~) (tilde symbol) operator:**

1. We can apply this operator only for *integral types* but not for boolean types.

`System.out.println(~true);`  $\xrightarrow{\text{C.E}}$

```
E:\scjp>javac OperatorsDemo.java
OperatorsDemo.java:5: operator ~ cannot be applied to boolean
System.out.println(~true);
```

2. Example :
3. `System.out.println(~true);` // CE :operator ~ can not be applied to boolean
4. `System.out.println(~4);` //-5
- 5.
6. description about above program :
7. 4--> 0 000.....0100                      0-----+ve
8. ~4--> 1 111.....1011                      1--- -ve
- 9.
10. 2's complement of ~4 --> 000....0100 add 1
11. result is : 000...0101 =5
12. Note : The most significant bit access as sign bit 0 means +ve number , 1 means -ve number.  
+ve number will be represented directly in memory where as -ve number will be represented in 2's complement form.



## Boolean complement (!) operator:

This operator is applicable only for *boolean* types but not for integral types.

`System.out.println(!4);` 

```
E:\scjp>javac OperatorsDemo.java
OperatorsDemo.java:5: operator ! cannot be applied to int
System.out.println(!4);
```

Example :

Example:

```
System.out.println(!true); //false
System.out.println(!false); //true
System.out.println(!4); //CE : operator ! can not be applied to int
```

Summary:

```
&
|      Applicable for both boolean and integral types.
^
~ -----Applicable for integral types only but not for boolean types.
! -----Applicable for boolean types only but not for integral types.
```

## Short circuit (&&, ||) operators:

These operators are exactly same as normal bitwise operators &(AND), |(OR) except the following differences.

| & ,                                             | && ,                                                          |
|-------------------------------------------------|---------------------------------------------------------------|
| Both arguments should be evaluated always.      | Second argument evaluation is optional.                       |
| Relatively performance is low.                  | Relatively performance is high.                               |
| Applicable for both integral and boolean types. | Applicable only for boolean types but not for integral types. |

**x&& y** : y will be evaluated if and only if x is true.(If x is false then y won't be evaluated i.e., If x is true then only y will be evaluated)

**x|| y** : y will be evaluated if and only if x is false.(If x is true then y won't be evaluated i.e., If x is false then only y will be evaluated)

Example :

```
int x=10 , y=15 ;
if(++x < 10 || ++y > 15) {    //instead of || using &&, |
operators
    x++;
}
else {
    y++;
```

```

    }
    System.out.println(x+"----"+y);

```

**Output:**

| operator | x  | y  |
|----------|----|----|
| &        | 11 | 17 |
|          | 12 | 16 |
| &&       | 11 | 16 |
|          | 12 | 16 |

**Example :**

```

int x=10 ;
if(++x < 10  && ((x/0)>10) ) {
    System.out.println("Hello");
}
else {
    System.out.println("Hi");
}

```

output : Hi

## Type Cast Operator :

There are 2 types of type-casting

1. implicit
2. explicit

### implicit type casting :

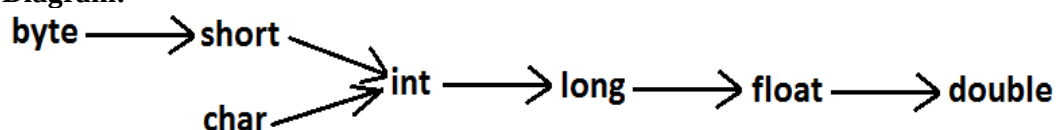
```

int x='a';
System.out.println(x); //97

```

1. The compiler is responsible to perform this type casting.
2. When ever we are assigning lower datatype value to higher datatype variable then implicit type cast will be performed .
3. It is also known as Widening or Upcasting.
4. There is no lose of information in this type casting.
5. The following are various possible implicit type casting.

Diagram:



- 6.
7. Example 1:

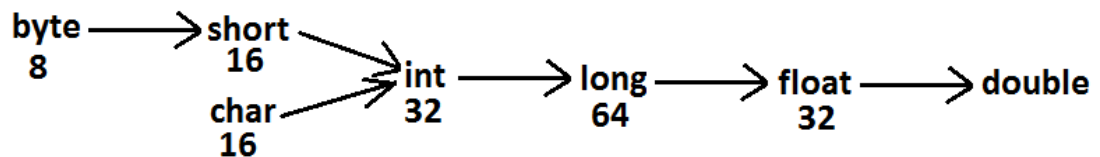
8. `int x='a';`
9. `System.out.println(x); //97`
10. **Note:** Compiler converts char to int type automatically by implicit type casting.
11. **Example 2:**
12. `double d=10;`
13. `System.out.println(d); //10.0`

**Note:** Compiler converts int to double type automatically by implicit type casting.

### Explicit type casting:

1. Programmer is responsible for this type casting.
2. Whenever we are assigning bigger data type value to the smaller data type variable then explicit type casting is required.
3. Also known as Narrowing or down casting.
4. There may be a chance of lose of information in this type casting.
5. The following are various possible conversions where explicit type casting is required.

Diagram:



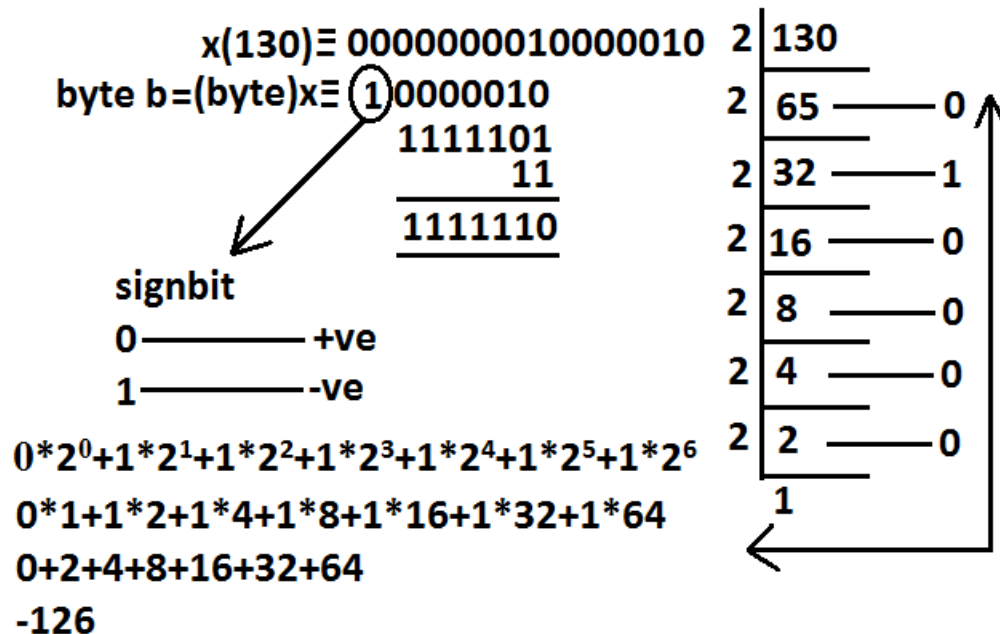
`int x=130;`  
`byte b=x;`

```

E:\scjp>javac OperatorsDemo.java
OperatorsDemo.java:6: possible loss of precision
found   : int
required: byte
    byte b=x;
  
```

- 6.
7. **Example :**
- 8.
9. `int x=130;`
10. `byte b=(byte)x;`
11. `System.out.println(b); // -126`

12.



13. Example 2 :

14.

15. `int x=130;`16. `byte b=x;`17. `System.out.println(b);` //CE : possible loss of precision

18. When ever we are assigning higher datatype value to lower datatype value variable by explicit type-casting ,the most significant bits will be lost i.e., we have considered least significant bits.

19. Example 3 :

20.

21. `int x=150;`22. `short s=(short)x;`23. `byte b=(byte)x;`24. `System.out.println(s);` //15025. `System.out.println(b);` //-106

26. When ever we are assigning floating point value to the integral types by explicit type casting , the digits of after decimal point will be lost .

27. Example 4:

28.

29. `double d=130.456 ;`

30.

31. `int x=(int)d ;`32. `System.out.println(x);` //130

33.

34. `byte b=(byte)d ;`35. `System.out.println(b);` //-206**float x=150.1234f;****int i=(int)x;****System.out.println(i);//150****double d=130.456;****int i=(int)d;****System.out.println(i);//130**

## Assignment Operator :

There are 3 types of assignment operators

1. Simple assignment:

*Example:* `int x=10;`

2. Chained assignment:

3. Example:

4. `int a,b,c,d;`

5. `a=b=c=d=20;`

6. `System.out.println(a+"---"+b+"---"+c+"---"+d); //20---20---20---20`

7. `int b , c , d ;`

8. `int a=b=c=d=20 ; //valid`

We can't perform chained assignment directly at the time of declaration.

`int a=b=c=d=20;` **C.E** →

cannot find symbol  
variable b  
variable c  
variable d

Example 2:

`int a=b=c=d=30;`

CE : can not find symbol

symbol : variable b

location : class Test

9. Compound assignment:

1. Sometimes we can mixed assignment operator with some other operator to form compound assignment operator.

2. Ex:

3. `int a=10 ;`

4. `a +=20 ;`

5. `System.out.println(a); //30`

6. The following is the list of all possible compound assignment operators in java.

|                 |                     |                            |
|-----------------|---------------------|----------------------------|
| <code>+=</code> |                     |                            |
| <code>-=</code> | <code>&amp;=</code> | <code>&gt;&gt;=</code>     |
| <code>*=</code> | <code> =</code>     | <code>&gt;&gt;&gt;=</code> |
| <code>/=</code> | <code>^=</code>     | <code>&lt;&lt;=</code>     |
| <code>%=</code> |                     |                            |

7. In the case of compound assignment operator internal type casting will be performed automatically by the compiler (similar to increment and decrement operators.)

```
byte b=10;
b=b+1;
System.out.println(b);
```

C.E



```
E:\scjp>javac OperatorsDemo.java
OperatorsDemo.java:6: possible loss of precision
found   : int
required: byte
    b=b+1;
```

```
byte b=10;
b++;
System.out.println(b); //11
```

```
byte b=10;
//b+=1;
b=(byte)(b+1);
System.out.println(b); //11
```

```
int a,b,c,d;
a=b=c=d=20;
a+=b-=c*=d/=2;
System.out.println(a+"---"+b+"---"+c+"---"+d);
// -160---180---200---10
```

```
byte b=10;
b=b+1;
System.out.println(b);
```

CE :

```
possible loss of precission
found   : int
required : byte
```

```
byte b=10;
b++;
System.out.println(b); //11
```

```
byte b=10;
b+=1;
System.out.println(b); //11
```

```
byte b=127;
b+=3;
System.out.println(b);
// -126
```

Ex :

```
int a , b , c , d ;
a=b=c=d=20 ;
a += b-= c *= d /= 2 ;
System.out.println(a+"---"+b+"---"+c+"---"+d); // -160...-180---200---10
```

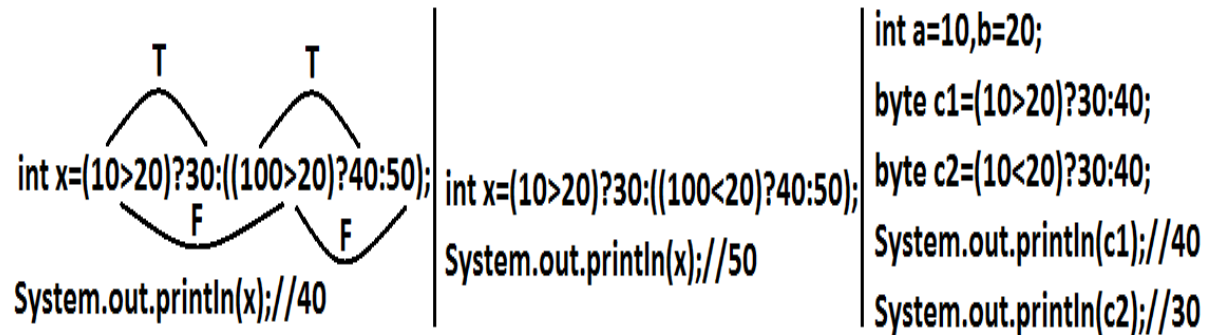
## Conditional Operator (? :)

The only possible ternary operator in java is conditional operator

Ex 1 :  
`int x=(10>20)?30:40;  
 System.out.println(x); //40`

Ex 2 :  
`int x=(10>20)?30:((40>50)?60:70);  
 System.out.println(x); //70`

Nesting of conditional operator is possible



```

int a=10,b=20;
byte c1=(a>b)?30:40;
byte c2=(a<b)?30:40;
System.out.println(c1);
System.out.println(c2);

```

C.E →

```

E:\scjp>javac OperatorsDemo.java
OperatorsDemo.java:6: possible loss of precision
found   : int
required: byte
    byte c1=(a>b)?30:40;

```

## new operator :

1. We can use "new" operator to create an object.
2. There is no "delete" operator in java because destruction of useless objects is the responsibility of garbage collector.



## [ ] operator:

We can use this operator to declare under construct/create arrays.

## Java operator precedence:

1. Unary operators: [], x++, x--, ++x, --x, ~, !, new, <type>
2. Arithmetic operators: \*, /, %, +, - .
3. Shift operators: >>, >>>, << .
4. Comparison operators: <, <=, >, >=, instanceof.
5. Equality operators: ==, !=
6. Bitwise operators: &, ^, | .
7. Short circuit operators: &&, || .
8. Conditional operator: (:)
9. Assignment operators: +=, -=, \*=, /=, %= ...

## Evaluation order of java operands:

There is no precedence for operands before applying any operator all operands will be evaluated from left to right.

Example:

```
class OperatorsDemo {
    public static void main(String[] args){
        System.out.println(m1(1)+m1(2)*m1(3)/m1(4)*m1(5)+m1(6));
    }
    public static int m1(int i) {
        System.out.println(i);
        return i;
    }
}
```

| output: | Analysis:   |
|---------|-------------|
| 1       | 1+2*3/4*5+6 |
| 2       | 1+6/4*5+6   |
| 3       | 1+1*5+6     |
| 4       | 1+5+6       |
| 5       | 12          |
| 6       |             |
| 12      |             |

|                                                        |                                                        |                                                                                       |
|--------------------------------------------------------|--------------------------------------------------------|---------------------------------------------------------------------------------------|
| <pre>int x=10; x=++x; System.out.println(x);//11</pre> | <pre>int x=10; x=x+1; System.out.println(x);//11</pre> | <pre>int x=10; int y=x++; System.out.println(y);//10 System.out.println(x);//11</pre> |
|--------------------------------------------------------|--------------------------------------------------------|---------------------------------------------------------------------------------------|

Ex 2:

```
int i=1;
i+=++i + i++ + ++i + i++;
System.out.println(i); //13
```

description :

```
i=i + ++i + i++ + ++i + i++ ;
i=1+2+2+4+4;
i=13;
```

## new Vs newInstance( ) :

1. new is an operator to create an objects , if we know class name at the beginning then we can create an object by using new operator .
2. newInstance( ) is a method presenting class " Class " , which can be used to create object.
3. If we don't know the class name at the beginning and its available dynamically Runtime then we should go for newInstance() method
4. 

```
public class Test {
```
5. 

```
    public static void main(String[] args) Throws Exception {
```
6. 

```
        Object o=Class.forName(arg[0]).newInstance( ) ;
```
7. 

```
        System.out.println(o.getClass().getName( ) );
```
8. 

```
    }
```
9. 

```
}
```
10. If dynamically provide class name is not available then we will get the RuntimeException saying ClassNotFoundException
11. To use newInstance( ) method compulsory corresponding class should contains no argument constructor , otherwise we will get the RuntimeException saying InstantiationException.

## Difference between **new** and **newInstance()** :

| <b>new</b>                                                                                                                                | <b>newInstance()</b>                                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| new is an operator , which can be used to create an object                                                                                | newInstance() is a method , present in class Class , which can be used to create an object .                                                                                    |
| We can use new operator if we know the class name at the beginning.<br>Test t= new Test( );                                               | We can use the newInstance() method , If we don't class name at the beginning and available dynamically Runtime.<br>Object o=Class.forName(arg[0]).newInstance( );              |
| If the corresponding .class file not available at Runtime then we will get RuntimeException saying NoClassDefFoundError , It is unchecked | If the corresponding .class file not available at Runtime then we will get RuntimeException saying ClassNotFoundException , It is checked                                       |
| To used new operator the corresponding class not required to contain no argument constructor                                              | To used newInstance() method the corresponding class should compulsory contain no argument constructor , Other wise we will get RuntimeException saying InstantiationException. |

## Difference between **ClassNotFoundException** & **NoClassDefFoundError** :

1. For hard coded class names at Runtime in the corresponding .class files not available we will get NoClassDefFoundError , which is unchecked  
Test t = new Test( );  
In Runtime Test.class file is not available then we will get NoClassDefFoundError
2. For Dynamically provided class names at Runtime , If the corresponding .class files is not available then we will get the RuntimeException saying ClassNotFoundException  
Ex : Object o=Class.forName("Test").newInstance( );  
At Runtime if Test.class file not available then we will get the ClassNotFoundException , which is checked exception

## Difference between instanceof and isInstance( ) :

| instanceof                                                                                                                                                          | isInstance( )                                                                                                                                                                                                                                                                   |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>instanceof an operator which can be used to check whether the given object is particular type or not</p> <p>We know at the type at beginning it is available</p> | <p>isInstance( ) is a method , present in class Class , we can use isInstance( ) method to checked whether the given object is particular type or not</p> <p>We don't know at the type at beginning it is available Dynamically at Runtime.</p>                                 |
| <pre>String s = new String("ashok"); System.out.println(s instanceof Object );  //true If we know the type at the beginning only.</pre>                             | <pre>class Test { public static void main(String[] args) { Test t = new Test( ); System.out.println( Class.forName(args[0]).isInstance( ));  //arg[0] --- We don't know the type at beginning } }  java Test Test //true java Test String //false java Test Object //true</pre> |
| <pre>int x= 10 ; x=x++; System.out.println(x); //10</pre>                                                                                                           | <ol style="list-style-type: none"> <li>1. consider old value of x for assignment x=10</li> <li>2. Increment x value x=11</li> <li>3. Perform assignment with old considered x value x=10</li> </ol>                                                                             |

# **CORE JAVA**

## **With**

# **SCJP / OCJP**

## **Study Material**

### **Chapter 3 : Flow Control**



**DURGA M.Tech**

**(Sun certified & Realtime Expert)**

**Ex. IBM Employee**

**Trained Lakhs of Students  
for last 14 years across INDIA**

**India's No.1 Software Training Institute**

# **DURGASOFT**

**[www.durgasoft.com](http://www.durgasoft.com) Ph: 9246212143 ,8096969696**

# Flow Control

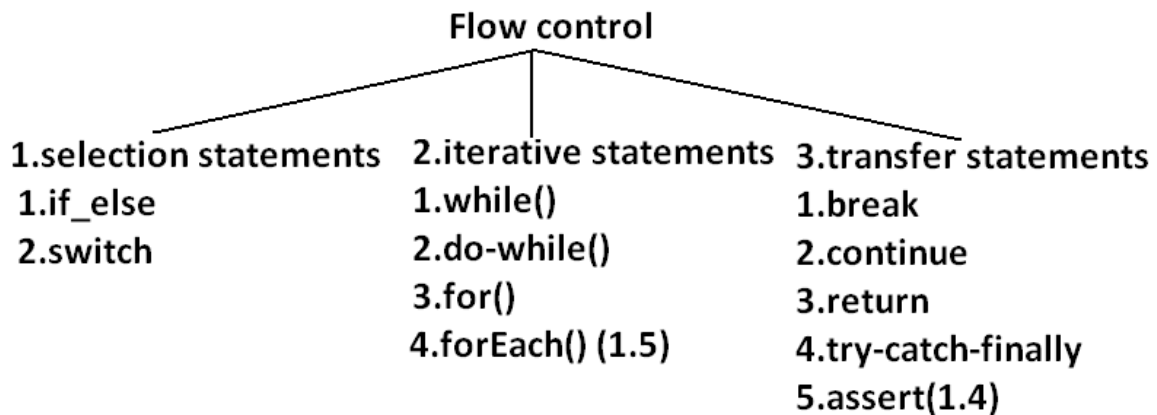
## Agenda :

1. Introduction
2. Selection statements
  - i. if-else
  - ii. Switch
    - Case Summary
    - fall-through inside a switch
    - default case
3. Iterative Statements
  - i. While loop
    - Unreachable statement in while
  - ii. Do-while
    - Unreachable statement in do while
  - iii. For Loop
    - Initialization section
    - Conditional check
    - Increment and decrement section
    - Unreachable statement in for loop
  - iv. For each
    - Iterator Vs Iterable(1.5v)
    - Difference between Iterable and Iterator
4. Transfer statements
  - o Break statement
  - o Continue statement
  - o Labeled break and continue statements
  - o Do-while vs continue (The most dangerous combination)

## Introduction :

Flow control describes the order in which all the statements will be executed at run time.

Diagram:



## Selection statements:

### if-else:

syntax:

```
if(b)
{
//action if b is true
}else{
//action if b is false
}
```

An arrow points from the variable `b` in the `if(b)` condition to the word `boolean`, indicating that `b` must be a boolean value.

The argument to the if statement should be Boolean by mistake if we are providing any other type we will get "compile time error".

**Example 1:**

```
public class ExampleIf{
public static void main(String args[]){
int x=0;
if(x)
{
System.out.println("hello");
}else{
System.out.println("hi");
}}}
```

OUTPUT:

Compile time error:

D:\Java&gt;javac ExampleIf.java

ExampleIf.java:4: incompatible types

found : int

required: boolean

if(x)

**Example 2:**

```
public class ExampleIf{
public static void main(String args[]){
int x=10;
if(x=20)
{
System.out.println("hello");
}
else{
System.out.println("hi");
}}}
```

OUTPUT:

Compile time error

D:\Java&gt;javac ExampleIf.java

ExampleIf.java:4: incompatible types

found : int

required: boolean

if(x=20)

**Example 3:**

```
public class ExampleIf{
public static void main(String args[]){
int x=10;
if(x==20)
{
System.out.println("hello");
}else{
System.out.println("hi");
}}}
```

OUTPUT:

Hi

**Example 4:**

```
public class ExampleIf{
public static void main(String args[]){
boolean b=false;
if(b=true)
{
System.out.println("hello");
}else{
System.out.println("hi");
}}}
```

OUTPUT:

Hello

**Example 5:**



```
public class ExampleIf{
public static void main(String args[]){
boolean b=false;
if(b==true)
{
System.out.println("hello");
}else{
System.out.println("hi");
}}}
OUTPUT:
Hi
```

Both else part and curly braces are optional.

Without curly braces we can take only one statement under if, but it should not be declarative statement.

**Example 6:**

```
public class ExampleIf{
public static void main(String args[]){
if(true)
System.out.println("hello");
}}
OUTPUT:
Hello
```

**Example 7:**

```
public class ExampleIf{
public static void main(String args[]){
if(true);
}}
OUTPUT:
No output
```

**Example 8:**

```
public class ExampleIf{
public static void main(String args[]){
if(true)
int x=10;
}}
OUTPUT:
Compile time error
D:\Java>javac ExampleIf.java
ExampleIf.java:4: '.class' expected
int x=10;
ExampleIf.java:4: not a statement
int x=10;
```

**Example 9:**

```
public class ExampleIf{
public static void main(String args[]){
if(true){
int x=10;
}}}
OUTPUT:
D:\Java>javac ExampleIf.java
D:\Java>java ExampleIf
```

**Example 10:**

```
public class Example10 {  
    public static void main(String args[]) {  
        if (true)  
            System.out.println("hello");  
            System.out.println("hi");  
        }  
    }  
}
```

—————> dependent statement on if  
—————> this is independent statement on if

**OUTPUT:**

Hello  
Hi

Semicolon(;) is a valid java statement which is called empty statement and it won't produce any output.

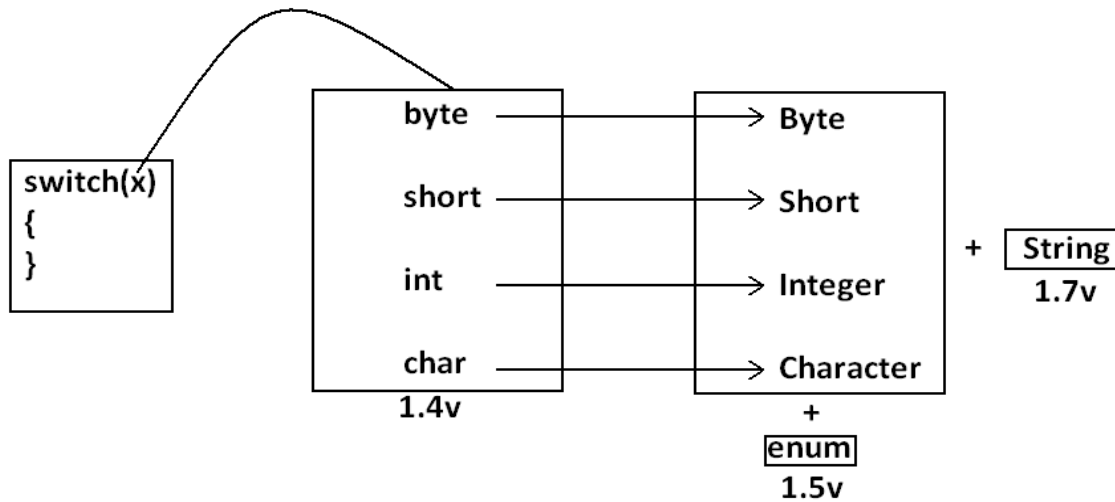
**Switch:**

If several options are available then it is not recommended to use if-else we should go for switch statement. Because it improves readability of the code.

**Syntax:**

```
switch(x)  
{  
    case 1:  
        action1  
    case 2:  
        action2  
    .  
    .  
    .  
    default:  
        default action  
}
```

Until 1.4 version the allowed types for the switch argument are byte, short, char, int but from 1.5 version onwards the corresponding wrapper classes (Byte, Short, Character, Integer) and "enum" types are also allowed.

**Diagram:**

- Curly braces are mandatory.(except switch case in all remaining cases curly braces are optional )
- Both case and default are optional.
- Every statement inside switch must be under some case (or) default. Independent statements are not allowed.

**Example 1:**

```
public class ExampleSwitch{
public static void main(String args[]){
int x=10;
switch(x)
{
System.out.println("hello");
}}}
```

OUTPUT:

Compile time error.

D:\Java>javac ExampleSwitch.java

ExampleSwitch.java:5: case, default, or '}' expected

System.out.println("hello");

Every case label should be "compile time constant" otherwise we will get compile time error.

**Example 2:**

```
public class ExampleSwitch{
public static void main(String args[]){
int x=10;
int y=20;
```

```
switch(x)
{
case 10:
System.out.println("10");
case y:
System.out.println("20");
}}
OUTPUT:
Compile time error
D:\Java>javac ExampleSwitch.java
ExampleSwitch.java:9: constant expression required
case y:
If we declare y as final we won't get any compile time error.
```

**Example 3:**

```
public class ExampleSwitch{
public static void main(String args[]){
int x=10;
final int y=20;
switch(x)
{
case 10:
System.out.println("10");
case y:
System.out.println("20");
}}
OUTPUT:
10
20
```

But switch argument and case label can be expressions , but case label should be constant expression.

**Example 4:**

```
public class ExampleSwitch{
public static void main(String args[]){
int x=10;
switch(x+1)
{
case 10:
case 10+20:
case 10+20+30:
}}
OUTPUT:
No output.
```

Every case label should be within the range of switch argument type.

**Example 5:**

```
public class ExampleSwitch{
public static void main(String args[]){
byte b=10;
switch(b)
{
case 10:
System.out.println("10");
case 100:
System.out.println("100");
```

```
case 1000:
System.out.println("1000");
}}}
OUTPUT:
Compile time error
D:\Java>javac ExampleSwitch.java
ExampleSwitch.java:10: possible loss of precision
found   : int
required: byte
case 1000:
```

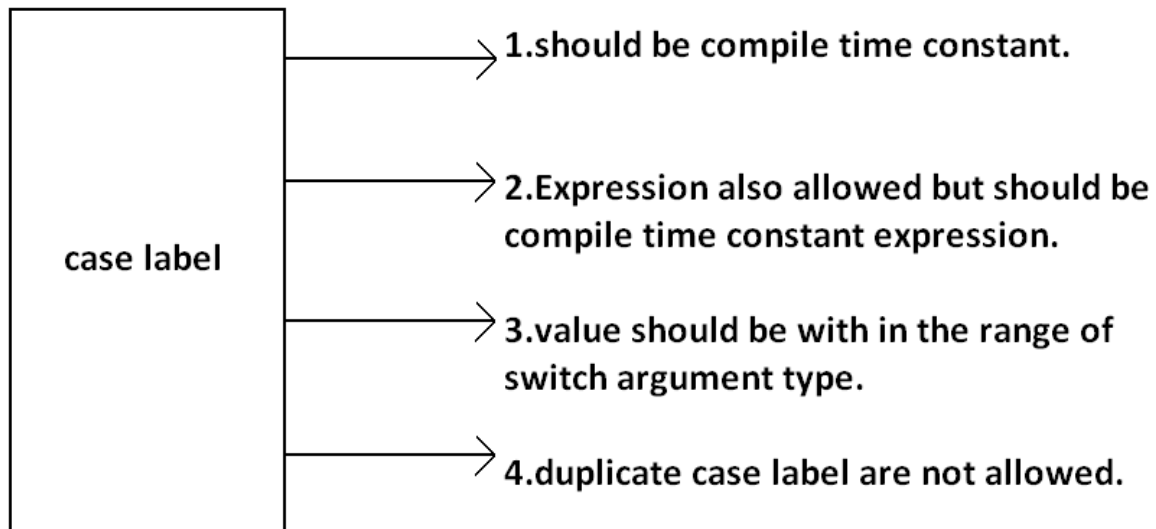
**Example :**

```
public class ExampleSwitch{
public static void main(String args[]){
byte b=10;
switch(b+1)
{
case 10:
System.out.println("10");
case 100:
System.out.println("100");
case 1000:
System.out.println("1000");
}}}
OUTPUT:
```

Duplicate case labels are not allowed.

**Example 6:**

```
public class ExampleSwitch{
public static void main(String args[]){
int x=10;
switch(x)
{
case 97:
System.out.println("97");
case 99:
System.out.println("99");
case 'a':
System.out.println("100");
}}}
OUTPUT:
Compile time error.
D:\Java>javac ExampleSwitch.java
ExampleSwitch.java:10: duplicate case label
case 'a':
```

**CASE SUMMARY:****Diagram:****FALL-THROUGH INSIDE THE SWITCH:**

With in the switch statement if any case is matched from that case onwards all statements will be executed until end of the switch (or) break. This is call "fall-through" inside the switch .

The main advantage of fall-through inside a switch is we can define common action for multiple cases

**Example 7:**

```
public class ExampleSwitch{
public static void main(String args[]){
int x=0;
switch(x)
{
case 0:
System.out.println("0");
case 1:
System.out.println("1");
break;
case 2:
System.out.println("2");
default:
System.out.println("default");
}
```

}}}

OUTPUT:

| x=0 | x=1 | x=2     | x=3     |
|-----|-----|---------|---------|
| 0   | 1   | 2       | default |
| 1   |     | default |         |

**DEFAULT CASE:**

- With in the switch we can take the default only once
- If no other case matched then only default case will be executed
- With in the switch we can take the default any where, but it is convention to take default as last case.

**Example 8:**

```
public class ExampleSwitch{
public static void main(String args[]){
int x=0;
switch(x)
{
default:
System.out.println("default");
case 0:
System.out.println("0");
break;
case 1:
System.out.println("1");
case 2:
System.out.println("2");
}}}
```

OUTPUT:

| X=0 | x=1 | x=2 | x=3     |
|-----|-----|-----|---------|
| 0   | 1   | 2   | default |
|     | 2   |     | 0       |

**ITERATIVE STATEMENTS:****While loop:**

if we don't know the no of iterations in advance then best loop is while loop:

**Example 1:**

```
while(rs.next())
{
}
```

**Example 2:**

```
while(e.hasMoreElements())
{
-----
-----
-----
}
```

**Example 3:**

```
while(itr.hasNext())
{
-----
}
```

```

-----
-----
}

```

The argument to the while statement should be Boolean type. If we are using any other type we will get compile time error.

**Example 1:**

```

public class ExampleWhile{
public static void main(String args[]){
while(1)
{
System.out.println("hello");
}}}
OUTPUT:
Compile time error.
D:\Java>javac ExampleWhile.java
ExampleWhile.java:3: incompatible types
found   : int
required: boolean
while(1)

```

Curly braces are optional and without curly braces we can take only one statement which should not be declarative statement.

**Example 2:**

```

public class ExampleWhile{
public static void main(String args[]){
while(true)
System.out.println("hello");
}}
OUTPUT:
Hello (infinite times).

```

**Example 3:**

```

public class ExampleWhile{
public static void main(String args[]){
while(true);
}}
OUTPUT:
No output.

```

**Example 4:**

```

public class ExampleWhile{
public static void main(String args[]){
while(true)
int x=10;
}}
OUTPUT:
Compile time error.
D:\Java>javac ExampleWhile.java
ExampleWhile.java:4: '.class' expected
int x=10;
ExampleWhile.java:4: not a statement
int x=10;

```

**Example 5:**

```

public class ExampleWhile{
public static void main(String args[]){
while(true)
{

```



```
int x=10;
}}}  
OUTPUT:  
No output.
```

### Unreachable statement in while:

#### Example 6:

```
public class ExampleWhile{  
    public static void main(String args[]){  
        while(true)  
        {  
            System.out.println("hello");  
        }  
        System.out.println("hi");  
    }  
}  
OUTPUT:  
Compile time error.  
D:\Java>javac ExampleWhile.java  
ExampleWhile.java:7: unreachable statement  
    System.out.println("hi");
```

#### Example 7:

```
public class ExampleWhile{  
    public static void main(String args[]){  
        while(false)  
        {  
            System.out.println("hello");  
        }  
        System.out.println("hi");  
    }  
}  
OUTPUT:  
D:\Java>javac ExampleWhile.java  
ExampleWhile.java:4: unreachable statement  
{
```

#### Example 8:

```
public class ExampleWhile{  
    public static void main(String args[]){  
        int a=10,b=20;  
        while(a<b)  
        {  
            System.out.println("hello");  
        }  
        System.out.println("hi");  
    }  
}  
OUTPUT:  
Hello (infinite times).
```

#### Example 9:

```
public class ExampleWhile{  
    public static void main(String args[]){  
        final int a=10,b=20;  
        while(a<b)  
        {  
            System.out.println("hello");  
        }  
        System.out.println("hi");  
    }  
}  
OUTPUT:  
Compile time error.  
D:\Java>javac ExampleWhile.java
```

ExampleWhile.java:8: unreachable statement  
System.out.println("hi");

**Example 10:**

```
public class ExampleWhile{
public static void main(String args[]){
final int a=10;
while(a<20)
{
System.out.println("hello");
}
System.out.println("hi");
}}
```

OUTPUT:

D:\Java>javac ExampleWhile.java  
ExampleWhile.java:8: unreachable statement  
System.out.println("hi");

**Note:**

- Every final variable will be replaced with the corresponding value by compiler.
- If any operation involves only constants then compiler is responsible to perform that operation.
- If any operation involves at least one variable compiler won't perform that operation. At runtime jvm is responsible to perform that operation.

**Example 11:**

```
public class ExampleWhile{
public static void main(String args[]){
int a=10;
while(a<20)
{
System.out.println("hello");
}
System.out.println("hi");
}}
```

OUTPUT:

Hello (infinite times).

## Do-while:

If we want to execute loop body at least once then we should go for do-while.

**Syntax:**

```
do
{
-----
-----
-----
}while(b); —————>semicolon is the mandatory.
```

Curly braces are optional.

Without curly braces we can take only one statement between do and while and it should not be declarative statement.

**Example 1:**

```
public class ExampleDoWhile{
public static void main(String args[]){
do
System.out.println("hello");
while(true);
}}
```

Output:

Hello (infinite times).

**Example 2:**

```
public class ExampleDoWhile{
public static void main(String args[]){
do;
while(true);
}}
```

Output:

Compile successful.

**Example 3:**

```
public class ExampleDoWhile{
public static void main(String args[]){
do
int x=10;
while(true);
}}
```

Output:

```
D:\Java>javac ExampleDoWhile.java
ExampleDoWhile.java:4: '.class' expected
int x=10;
ExampleDoWhile.java:4: not a statement
int x=10;
ExampleDoWhile.java:4: ')' expected
int x=10;
```

**Example 4:**

```
public class ExampleDoWhile{
public static void main(String args[]){
do
{
int x=10;
}while(true);
}}
```

Output:

Compile successful.

**Example 5:**

```
public class ExampleDoWhile{
public static void main(String args[]){
do while(true)
System.out.println("hello");
while(true);
}}
```

Output:

Hello (infinite times).

**Rearrange the above Example:**

```
public class ExampleDoWhile{
public static void main(String args[]){
do
    while(true)
        System.out.println("hello");
while(true);
}}
```

Output:  
Hello (infinite times).

**Example 6:**

```
public class ExampleDoWhile{
public static void main(String args[]){
do
while(true);
}}
```

Output:  
Compile time error.  
D:\Java>javac ExampleDoWhile.java  
ExampleDoWhile.java:4: while expected  
while(true);  
ExampleDoWhile.java:5: illegal start of expression  
}

**Unreachable statement in do while:****Example 7:**

```
public class ExampleDoWhile{
public static void main(String args[]){
do
{
System.out.println("hello");
}
while(true);
System.out.println("hi");
}}
```

Output:  
Compile time error.  
D:\Java>javac ExampleDoWhile.java  
ExampleDoWhile.java:8: unreachable statement  
System.out.println("hi");

**Example 8:**

```
public class ExampleDoWhile{
public static void main(String args[]){
do
{
System.out.println("hello");
}
while(false);
System.out.println("hi");
}}
```

Output:  
Hello  
Hi

**Example 9:**

```
public class ExampleDowhile{
public static void main(String args[]){
int a=10,b=20;
do
{
System.out.println("hello");
}
while(a<b);
System.out.println("hi");
}}
Output:
Hello (infinite times).
```

**Example 10:**

```
public class ExampleDowhile{
public static void main(String args[]){
int a=10,b=20;
do
{
System.out.println("hello");
}
while(a>b);
System.out.println("hi");
}}
Output:
Hello
Hi
```

**Example 11:**

```
public class ExampleDowhile{
public static void main(String args[]){
final int a=10,b=20;
do
{
System.out.println("hello");
}
while(a<b);
System.out.println("hi");
}}
Output:
Compile time error.
D:\Java>javac ExampleDowhile.java
ExampleDowhile.java:9: unreachable statement
System.out.println("hi");
```

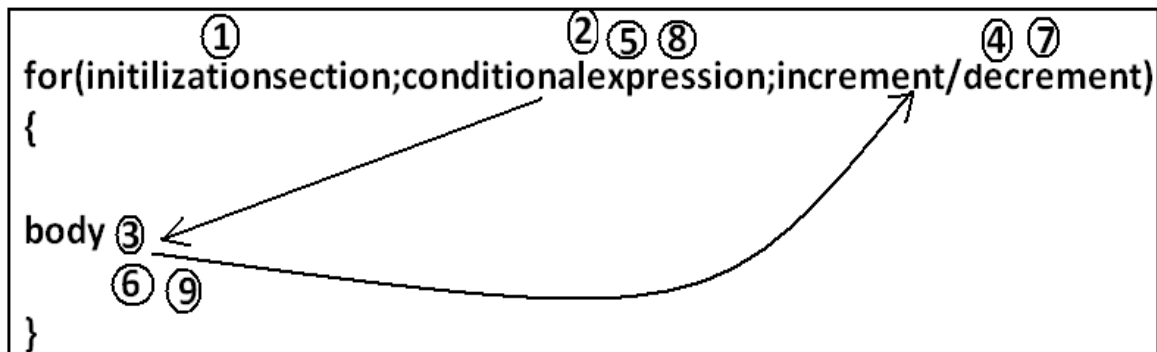
**Example 12:**

```
public class ExampleDowhile{
public static void main(String args[]){
final int a=10,b=20;
do
{
System.out.println("hello");
}
while(a>b);
System.out.println("hi");
}}
Output:
D:\Java>javac ExampleDowhile.java
D:\Java>java ExampleDowhile
Hello
Hi
```

## For Loop:

This is the most commonly used loop and best suitable if we know the no of iterations in advance.

### Syntax:



Curly braces are optional and without curly braces we can take only one statement which should not be declarative statement.

## Initilizationsection:

This section will be executed only once.

Here usually we can declare loop variables and we will perform initialization.

We can declare multiple variables but should be of the same type and we can't declare different type of variables.

### Example:

```
Int i=0,j=0; valid
Int i=0,Boolean b=true; invalid
Int i=0,int j=0; invalid
```

In initialization section we can take any valid java statement including "s.o.p" also.

### Example 1:

```
public class ExampleFor{
    public static void main(String args[]){
        int i=0;
        for(System.out.println("hello u r sleeping");i<3;i++){
            System.out.println("no boss, u only sleeping");
        }}
    }
```

Output:

```
D:\Java>javac ExampleFor.java
D:\Java>java ExampleFor
Hello u r sleeping
No boss, u only sleeping
No boss, u only sleeping
No boss, u only sleeping
```

## Conditional check:

We can take any java expression but should be of the type Boolean.  
Conditional expression is optional and if we are not taking any expression compiler will place true.

## Increment and decrement section:

Here we can take any java statement including s.o.p also.

### Example:

```
public class ExampleFor{
public static void main(String args[]){
int i=0;
for(System.out.println("hello");i<3;System.out.println("hi")){
i++;
}}}
```

Output:

```
D:\Java>javac ExampleFor.java
D:\Java>java ExampleFor
Hello
Hi
Hi
Hi
```

All 3 parts of for loop are independent of each other and all optional.

### Example:

```
public class ExampleFor{
public static void main(String args[]){
for(;;){
System.out.println("hello");
}}}
```

Output:

Hello (infinite times).

Curly braces are optional and without curly braces we can take exactly one statement and it should not be declarative statement.

## Unreachable statement in for loop:

### Example 1:

```
public class ExampleFor{
public static void main(String args[]){
for(int i=0;true;i++){
System.out.println("hello");
}
System.out.println("hi");
}}
```

Output:

Compile time error.

```
D:\Java>javac ExampleFor.java
ExampleFor.java:6: unreachable statement
System.out.println("hi");
```

### Example 2:

```
public class ExampleFor{
public static void main(String args[]){
for(int i=0;false;i++){
System.out.println("hello");
}
System.out.println("hi");
}}
```

Output:

Compile time error.

D:\Java>javac ExampleFor.java

ExampleFor.java:3: unreachable statement

```
for(int i=0;false;i++){
```

#### Example 3:

```
public class ExampleFor{
public static void main(String args[]){
for(int i=0;;i++){
System.out.println("hello");
}
System.out.println("hi");
}}
```

Output:

Compile time error.

D:\Java>javac ExampleFor.java

ExampleFor.java:6: unreachable statement

```
System.out.println("hi");
```

#### Example 4:

```
public class ExampleFor{
public static void main(String args[]){
int a=10,b=20;
for(int i=0;a<b;i++){
System.out.println("hello");
}
System.out.println("hi");
}}
```

Output:

Hello (infinite times).

#### Example 5:

```
public class ExampleFor{
public static void main(String args[]){
final int a=10,b=20;
for(int i=0;a<b;i++){
System.out.println("hello");
}
System.out.println("hi");
}}
```

Output:

D:\Java>javac ExampleFor.java

ExampleFor.java:7: unreachable statement

```
System.out.println("hi");
```



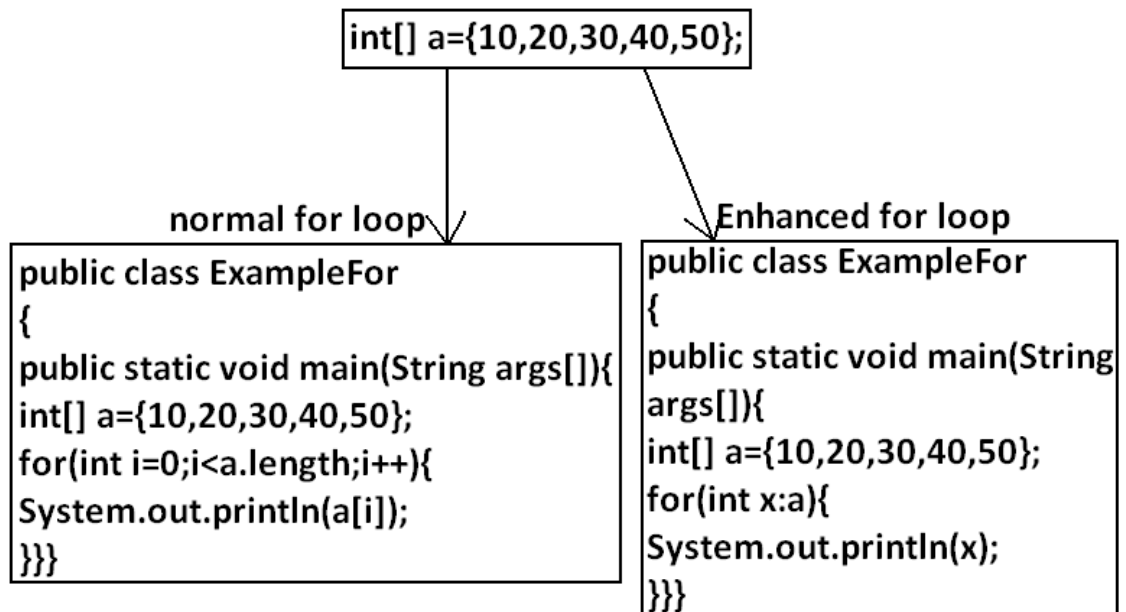
## For each:(Enhanced for loop)

- For each Introduced in 1.5version.
- Best suitable to retrieve the elements of arrays and collections.

### Example 1:

Write code to print the elements of single dimensional array by normal for loop and enhanced for loop.

### Example:



Output :

D:\Java>javac ExampleFor.java

D:\Java>java ExampleFor

10

20

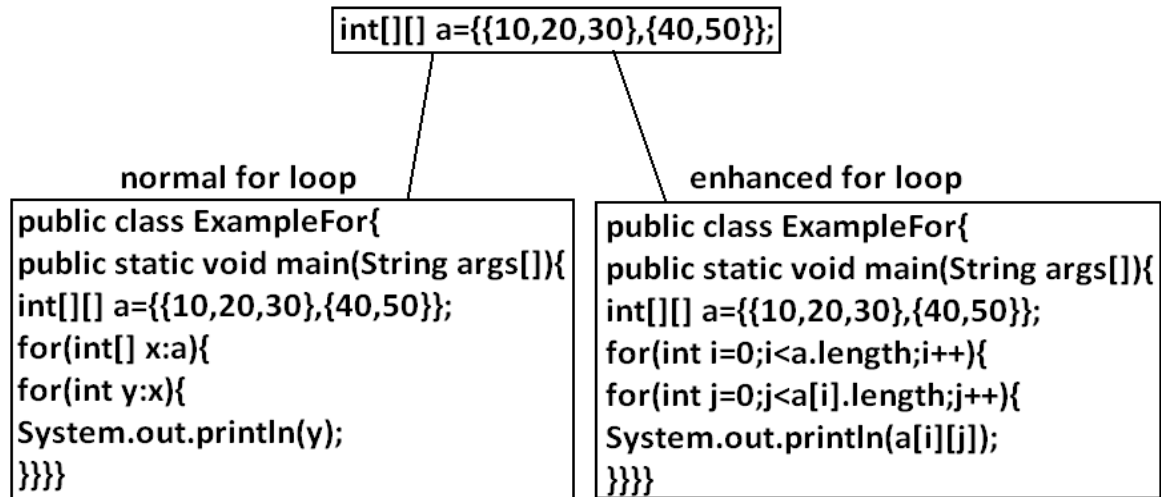
30

40

50

### Example 2:

Write code to print the elements of 2 dimensional arrays by using normal for loop and enhanced for loop.

**Example 3:**

Write equivalent code by For Each loop for the following for loop.

```
public class ExampleFor{
public static void main(String args[]){
for(int i=0;i<10;i++)
{
System.out.println("hello");
}}}
```

Output:

D:\Java>javac ExampleFor1.java

D:\Java>java ExampleFor1

```
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
```

- We can't write equivalent for each loop.
- For each loop is the more convenient loop to retrieve the elements of arrays and collections, but its main limitation is it is not a general purpose loop.
- By using normal for loop we can print elements either from left to right or from right to left. But using for-each loop we can always print array elements only from left to right.

## Iterator Vs Iterable(1.5v)

**Syntax :**

```
for(each item : target)
{
    -----
    -----
}
```

Iterable

array / Collection

- The target element in for-each loop should be Iterable object.
- An object is set to be iterable iff corresponding class implements java.lang.Iterable interface.
- Iterable interface introduced in 1.5 version and it's contains only one method iterator().

Syntax : public Iterator iterator();

Every array class and Collection interface already implements Iterable interface.

## Difference between Iterable and Iterator:

| Iterable                                              | Iterator                                                          |
|-------------------------------------------------------|-------------------------------------------------------------------|
| It is related to forEach loop                         | It is related to Collection                                       |
| The target element in forEach loop should be Iterable | We can use Iterator to get objects one by one from the collection |
| Iterator present in java.lang package                 | Iterator present in java.util package                             |
| contains only one method iterator()                   | contains 3 methods hasNext(), next(), remove()                    |
| Introduced in 1.5 version                             | Introduced in 1.2 version                                         |

## Transfer statements:

### Break statement:

We can use break statement in the following cases.

- Inside switch to stop fall-through.
- Inside loops to break the loop based on some condition.

- Inside label blocks to break block execution based on some condition.

Inside switch :

We can use break statement inside switch to stop fall-through

Example 1:

```
class Test{
public static void main(String args[]){
int x=0;
switch(x)
{
case 0:
    System.out.println("hello");
    break ;
case 1:
    System.out.println("hi");
}
}
```

Output:

```
D:\Java>javac Test.java
D:\Java>java Test
Hello
```

Inside loops :

We can use break statement inside loops to break the loop based on some condition.

Example 2:

```
class Test{
public static void main(String args[]){
for(int i=0; i<10; i++) {
    if(i==5)
        break;
    System.out.println(i);
}
}}
```

Output:

```
D:\Java>javac Test.java
D:\Java>java Test
0
1
2
3
4
```

Inside Labeled block :

We can use break statement inside label blocks to break block execution based on some condition.

Example:

```
class Test{
public static void main(String args[]){
int x=10;
l1 : {
    System.out.println("begin");
    if(x==10)
        break l1;
}
```

```

    System.out.println("end");
  }
  System.out.println("hello");
}
}

```

Output:

D:\Java>javac Test.java

D:\Java>java Test

begin

hello

These are the only places where we can use break statement. If we are using anywhere else we will get compile time error.

Example:

```

class Test{
public static void main(String args[]){
int x=10;
if(x==10)
break;
System.out.println("hello");
}
}

```

Output:

Compile time error.

D:\Java>javac Test.java

Test.java:5: break outside switch or loop

break;

## Continue statement:

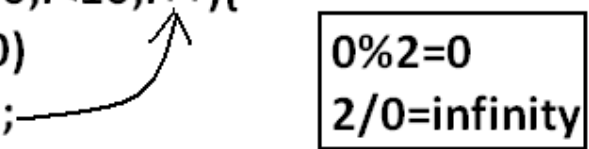
We can use continue statement to skip current iteration and continue for the next iteration.

Example:

```

class Test{
public static void main(String args[]){
int x=2;
for(int i=0;i<10;i++){
if(i%x==0)
continue;
System.out.println(i);
}}

```



Output:

```
D:\Java>javac Test.java
```

```
D:\Java>java Test
```

```
1
```

```
3
```

```
5
```

```
7
```

```
9
```

We can use continue only inside loops if we are using anywhere else we will get compile time error saying "continue outside of loop".

Example:

```
class Test
{
public static void main(String args[]){
int x=10;
if(x==10);
continue;
System.out.println("hello");
}
}
```

Output:

Compile time error.

```
D:\Enum>javac Test.java
```

```
Test.java:6: continue outside of loop
```

```
continue;
```

Labeled break and continue statements:

In the nested loops to break (or) continue a particular loop we should go for labeled break and continue statements.

Syntax:

```

l1:
for(;;){
.....
.....
l2:
for(;;){
.....
.....
l3:
for(;;){
.....
.....
break l1;
break l2;
break l3;
.....
.....
}
.....
}
.....
}
.....
}

```

**Example:**

```

class Test
{
public static void main(String args[]){
l1:
for(int i=0;i<3;i++)
{
for(int j=0;j<3;j++)
{
if(i==j)
break;
System.out.println(i+"....."+j);
}
}
}
}

```

```

    }
  }
}

```

**Break:**

```

1.....0
2.....0
2.....1

```

**Break l1:**

No output.

**Continue:**

```

0.....1
0.....2
1.....0
1.....2
2.....0
2.....1

```

**Continue l1:**

```

1.....0
2.....0
2.....1

```

### Do-while vs continue (The most dangerous combination):

```

class Test
{
    public static void main(String args[]){
        int x=0;
        do
        {
            ++x;
            System.out.println(x);
            if(++x<5)
            continue;
            ++x;
            System.out.println(x);
        }while(++x<10);
    }
}

```



Output:

1  
4  
6  
8  
10

Compiler won't check unreachability in the case of if-else it will check only in loops.

**Example 1:**

```
class Test
{
    public static void main(String args[]){
        while(true)
        {
            System.out.println("hello");
        }
        System.out.println("hi");
    }
}
```

Output:

Compile time error.

D:\Enum>javac Test.java

Test.java:8: unreachable statement

System.out.println("hi");

**Example 2:**

```
class Test
{
    public static void main(String args[]){
        if(true)
        {
            System.out.println("hello");
        }
        else
        {
            System.out.println("hi");
        }
    }
}
```

Output:

Hello

# **CORE JAVA**

## **With**

# **SCJP / OCJP**

### **Study Material**

#### **Chapter 4: Declaration & Access Modifiers**



**DURGA M.Tech**

**(Sun certified & Realtime Expert)**

**Ex. IBM Employee**

**Trained Lakhs of Students  
for last 14 years across INDIA**

**India's No.1 Software Training Institute**

# **DURGASOFT**

**www.durgasoft.com Ph: 9246212143 ,8096969696**

# Declaration and Access Modifiers

## Agenda

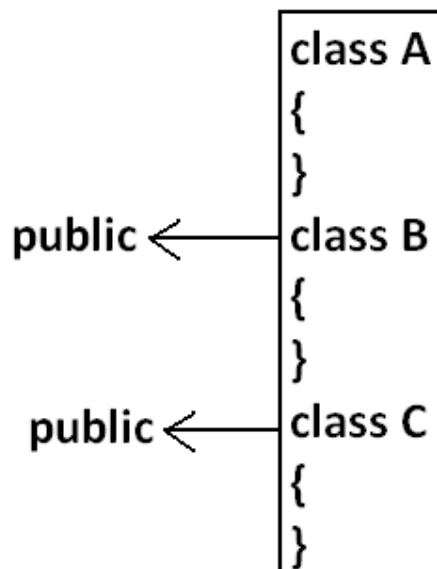
1. Java source file structure
  - o Import statement
  - o Types of Import Statements
    - Explicit class import
    - Implicit class import
  - o Difference between C language #include and java language import ?
  - o 1.5 versions new features
  - o Static import
    - Without static import
    - With static import
  - o Explain about System.out.println statement ?
  - o What is the difference between general import and static import ?
  - o Package statement
    - How to compile package Program
    - How to execute package Program
  - o Java source file structure
2. Class Modifiers
  - o Only applicable modifiers for Top Level classes
  - o What is the difference between access specifier and access modifier ?
  - o Public Classes
  - o Default Classes
  - o Final Modifier
    - Final Methods
    - Final Class
  - o Abstract Modifier
    - Abstract Methods
    - Abstract class
  - o The following are the various illegal combinations for methods
  - o What is the difference between abstract class and abstract method ?
  - o What is the difference between final and abstract ?
  - o Strictfp
  - o What is the difference between abstract and strictfp ?
3. Member modifiers
  - o Public members
  - o Default member
  - o Private members
  - o Protected members
  - o Compression of private, default, protected and public
  - o Final variables
    - Final instance variables
      - At the time of declaration
      - Inside instance block
      - Inside constructor
    - Final static variables
      - At the time of declaration

- Inside static block
  - Final local variables
- Formal parameters
- Static modifier
- Native modifier
  - Pseudo code
- Synchronized
- Transient modifier
- Volatile modifier
- Summary of modifier
- 4. Interfaces
  - Interface declarations and implementations
  - Extends vs implements
  - Interface methods
  - Interface variables
  - Interface naming conflicts
    - Method naming conflicts
    - Variable naming conflicts
  - Marker interface
  - Adapter class
  - Interface vs abstract class vs concrete class
  - Difference between interface and abstract class?
  - Conclusions

## Java source file structure:

- A java Program can contain any no. Of classes but at most one class can be declared as public. "If there is a public class the name of the Program and name of the public class must be matched otherwise we will get compile time error".
- If there is no public class then any name we gives for java source file.

### Example:



**Case 1:**

If there is no public class then we can use any name for java source file there are no restrictions.

Example:

A.java  
B.java  
C.java  
Ashok.java

**case 2:**

If class B declared as public then the name of the Program should be B.java otherwise we will get compile time error saying "class B is public, should be declared in a file named B.java".

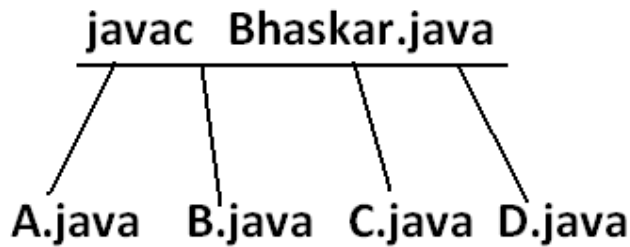
**Case 3:**

- If both B and C classes are declared as public and name of the file is B.java then we will get compile time error saying "class C is public, should be declared in a file named C.java".
- It is highly recommended to take only one class for source file and name of the Program (file) must be same as class name. This approach improves readability and understandability of the code.

**Example:**

```
class A
{
    public static void main(String args[]){
        System.out.println("A class main method is executed");
    }
}
class B
{
    public static void main(String args[]){
        System.out.println("B class main method is executed");
    }
}
class C
{
    public static void main(String args[]){
        System.out.println("C class main method is executed");
    }
}
class D
{
}
```

Output:



```

D:\Java>java A
A class main method is executed
D:\Java>java B
B class main method is executed
D:\Java>java C
C class main method is executed
D:\Java>java D
Exception in thread "main" java.lang.NoSuchMethodError: main
D:\Java>java Ashok
Exception in thread "main" java.lang.NoClassDefFoundError: Ashok
  
```

- We can compile a java Program but not java class in that Program for every class one dot class file will be created.
- We can run a java class but not java source file whenever we are trying to run a class the corresponding class main method will be executed.
- If the class won't contain main method then we will get runtime exception saying "NoSuchMethodError: main".
- If we are trying to execute a java class and if the corresponding .class file is not available then we will get runtime execution saying "NoClassDefFoundError: Ashok".

## Import statement:

```

class Test{
public static void main(String args[]){
ArrayList l=new ArrayList();
}
}
  
```

Output:

Compile time error.

```

D:\Java>javac Test.java
Test.java:3: cannot find symbol
symbol   : class ArrayList
location: class Test
  
```

```

ArrayList l=new ArrayList();
  
```

- We can resolve this problem by using fully qualified name "java.util.ArrayList l=new java.util.ArrayList();". But problem with using fully qualified name every time is it increases length of the code and reduces readability.
- We can resolve this problem by using import statements.

**Example:**

```
import java.util.ArrayList;  
class Test{  
public static void main(String args[]){  
ArrayList l=new ArrayList();  
}  
}  
Output:  
D:\Java>javac Test.java
```

Hence whenever we are using import statement it is not require to use fully qualified names we can use short names directly. This approach decreases length of the code and improves readability.

**Case 1: Types of Import Statements:**

There are 2 types of import statements.

- 1) Explicit class import
- 2) Implicit class import.

**Explicit class import:****Example: Import java.util.ArrayList**

- This type of import is highly recommended to use because it improves readability of the code.
- Best suitable for Hi-Tech city where readability is important.

**Implicit class import:****Example: import java.util.\*;**

- It is never recommended to use because it reduces readability of the code.
- Best suitable for Ameerpet where typing is important.

**Case 2:**

Which of the following import statements are meaningful ?

```
import java.util; ✗  
import java.util.ArrayList.*; ✗  
import java.util.*; ✓  
import java.util.ArrayList; ✓
```

**Case 3:**

consider the following code.

```
class MyArrayList extends java.util.ArrayList  
{  
}
```

- The code compiles fine even though we are not using import statements because we used fully qualified name.
- Whenever we are using fully qualified name it is not required to use import statement. Similarly whenever we are using import statements it is not required to use fully qualified name.

**Case 4:****Example:**

```
import java.util.*;  
import java.sql.*;  
class Test  
{  
    public static void main(String args[])  
    {  
        Date d=new Date();  
    }  
}
```

Output:

Compile time error.

D:\Java>javac Test.java

Test.java:7: reference to Date is ambiguous,  
both class java.sql.Date in java.sql and class java.util.Date in java.util  
match

Date d=new Date();

**Note:** Even in the List case also we may get the same ambiguity problem because it is available in both util and awt packages.



**Case 5:**

While resolving class names compiler will always gives the importance in the following order.

1. Explicit class import
2. Classes present in current working directory.
3. Implicit class import.

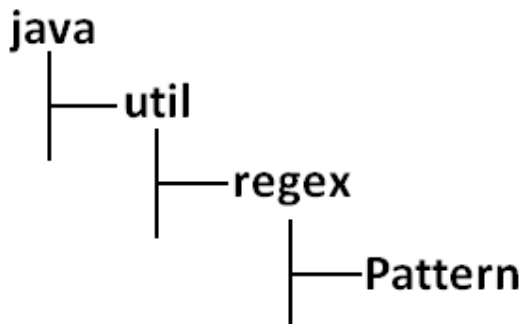
**Example:**

```
import java.util.Date;  
import java.sql.*;  
class Test  
{  
    public static void main(String args[]){  
        Date d=new Date();  
    }  
}
```

The code compiles fine and in this case util package Date will be considered.

**Case 6:**

Whenever we are importing a package all classes and interfaces present in that package are by default available but not sub package classes.

**Example:**

To use pattern class in our Program directly which import statement is required ?

1. `import java.*;` ✗
2. `import java.util.*;` ✗
3. `import java.util.regex.*;` ✓
4. `import java.util.regex.Pattern;` ✓

**Case7:**

In any java Program the following 2 packages are not require to import because these are available by default to every java Program.

1. `java.lang` package
2. default package(current working directory)

**Case 8:**

"Import statement is totally compile time concept" if more no of imports are there then more will be the compile time but there is "no change in execution time".

**Difference between C language `#include` and java language `import` ?**

| <code>#include</code>                                                                            | <code>import</code>                                                                                    |
|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| It can be used in C & C++                                                                        | It can be used in Java                                                                                 |
| At compile time only compiler copy the code from standard library and placed in current program. | At runtime JVM will execute the corresponding standard library and use it's result in current program. |
| It is static inclusion                                                                           | It is dynamic inclusion                                                                                |
| wastage of memory                                                                                | No wastage of memory                                                                                   |
| Ex : <code>&lt;jsp:@ file=""&gt;</code>                                                          | Ex : <code>&lt;jsp:include &gt;</code>                                                                 |

- In the case of C language `#include` all the header files will be loaded at the time of include statement hence it follows static loading.
- But in java `import` statement no ".class" will be loaded at the time of import statements in the next lines of the code whenever we are using a particular class then only corresponding ".class" file will be loaded. Hence it follows "dynamic loading" or "load-on -demand" or "load-on-fly".

## 1.5 versions new features :

1. For-Each
2. Var-arg
3. Queue
4. Generics
5. Auto boxing and Auto unboxing
6. Co-varient return types
7. Annotations

8. Enum
9. Static import
10. String builder

### Static import:

This concept introduced in 1.5 versions. According to sun static import improves readability of the code but according to worldwide Programming experts (like us) static imports creates confusion and reduces readability of the code. Hence if there is no specific requirement never recommended to use a static import.

Usually we can access static members by using class name but whenever we are using static import it is not require to use class name we can access directly.

### Without static import:

```
class Test
{
    public static void main(String args[]){
        System.out.println(Math.sqrt(4));
        System.out.println(Math.max(10,20));
        System.out.println(Math.random());
    }
}
```

Output:

```
D:\Java>javac Test.java
D:\Java>java Test
2.0
20
0.841306154315576
```

### With static import:

```
import static java.lang.Math.sqrt;
import static java.lang.Math.*;
class Test
{
    public static void main(String args[]){
        System.out.println(sqrt(4));
        System.out.println(max(10,20));
        System.out.println(random());
    }
}
```

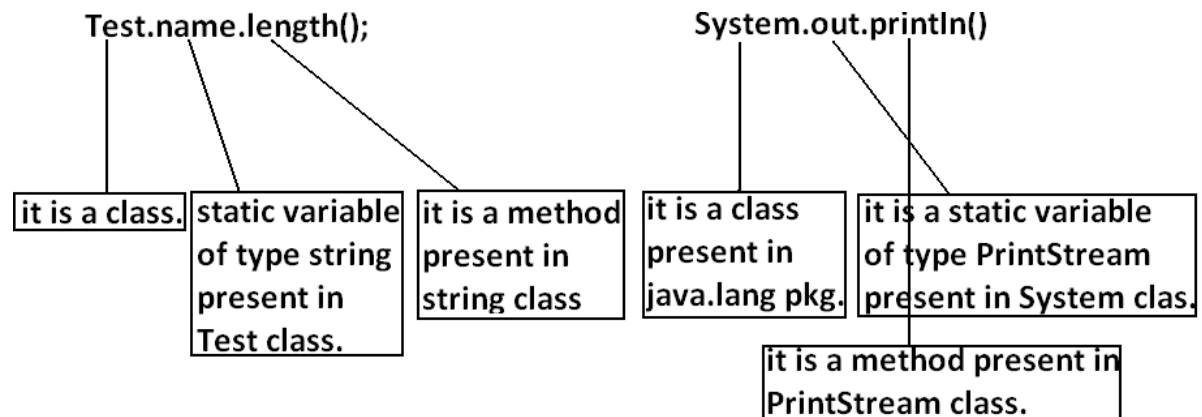
Output:

```
D:\Java>javac Test.java
D:\Java>java Test
2.0
20
0.4302853847363891
```

Explain about System.out.println statement ?Example 1 and Example 2:

```
1)
class Test
{
static String name="bhaskar";
}
```

```
2)
import java.io.*;
class System
{
static PrintStream out;
}
```

Example 3:

```
import static java.lang.System.out;
class Test
{
public static void main(String args[]){
out.println("hello");
out.println("hi");
}}
```

Output:

D:\Java&gt;javac Test.java

D:\Java&gt;java Test

hello

hi

Example 4:

```
import static java.lang.Integer.*;
import static java.lang.Byte.*;
class Test
{
public static void main(String args[]){
System.out.println(MAX_VALUE);
}}
```

Output:

Compile time error.

D:\Java>javac Test.java

Test.java:6: reference to MAX\_VALUE is ambiguous,  
both variable MAX\_VALUE in java.lang.Integer and variable MAX\_VALUE in  
java.lang.Byte match

System.out.println(MAX\_VALUE);

**Note:** Two packages contain a class or interface with the same is very rare hence ambiguity problem is very rare in normal import.

But 2 classes or interfaces can contain a method or variable with the same name is very common hence ambiguity problem is also very common in static import.

While resolving static members compiler will give the precedence in the following order.

1. Current class static members
2. Explicit static import
3. implicit static import.

**Example:**

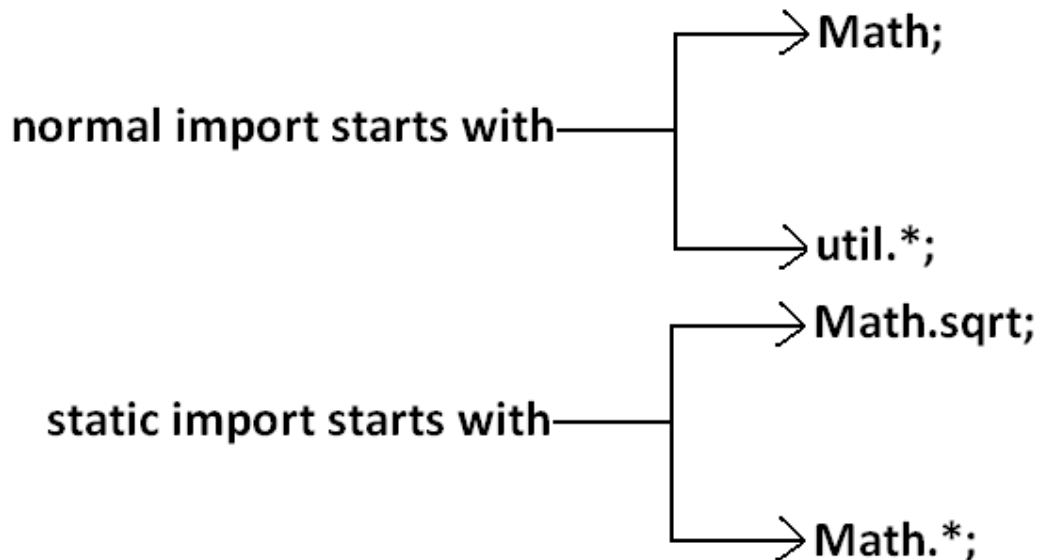
```
//import static java.lang.Integer.MAX_VALUE;—————> line2
import static java.lang.Byte.*;
class Test
{
//static int MAX_VALUE=999;—————> line1
public static void main(String args[])throws Exception{
System.out.println(MAX_VALUE);
}
}
```

- If we comet line one then we will get Integer class MAX\_VALUE 2147483647.
- If we comet lines one and two then Byte class MAX\_VALUE will be considered 127.

Which of the following import statements are valid ?

1. `import java.lang.Math.*;` ✗
2. `import static java.lang.Math.*;` ✓
3. `import java.lang.Math;` ✓
4. `import static java.lang.Math;` ✗
5. `import static java.lang.Math.sqrt.*;` ✗
6. `import java.lang.Math.sqrt;` ✗
7. `import static java.lang.Math.sqrt();` ✗
8. `import static java.lang.Math.sqrt;` ✓

Diagram:



Usage of static import reduces readability and creates confusion hence if there is no specific requirement never recommended to use static import.

What is the difference between general import and static import ?

- We can use normal imports to import classes and interfaces of a package. whenever we are using normal import we can access class and interfaces directly by their short name it is not require to use fully qualified names.
- We can use static import to import static members of a particular class. whenever we are using static import it is not require to use class name we can access static members directly.

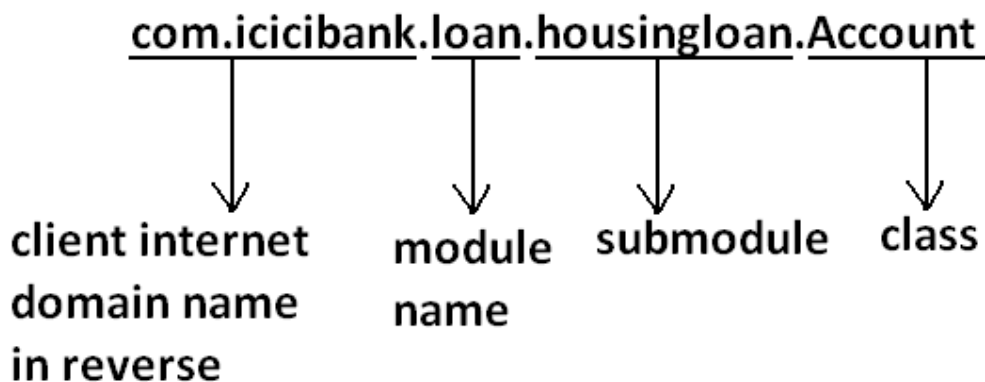
## Package statement:

It is an encapsulation mechanism to group related classes and interfaces into a single module.

The main objectives of packages are:

- To resolve name conflicts.
- To improve modularity of the application.
- To provide security.
- There is one universally accepted naming convention for packages that is to use internet domain name in reverse.

### Example:



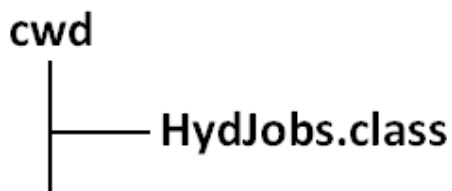
### How to compile package Program:

#### Example:

```
package com.durgajobs.itjobs;  
class HydJobs  
{  
    public static void main(String args[]){  
        System.out.println("package demo");  
    }  
}
```

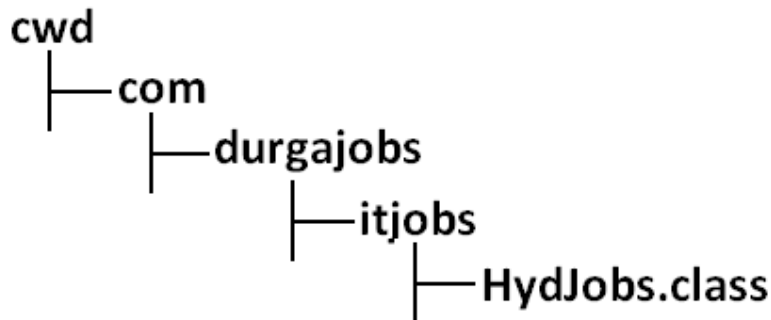
Javac HydJobs.java generated class file will be placed in current working directory.

### Diagram:



- `Javac -d . HydJobs.java`
- `-d` means destination to place generated class files `."` means current working directory.
- Generated class file will be placed into corresponding package structure.

Diagram:

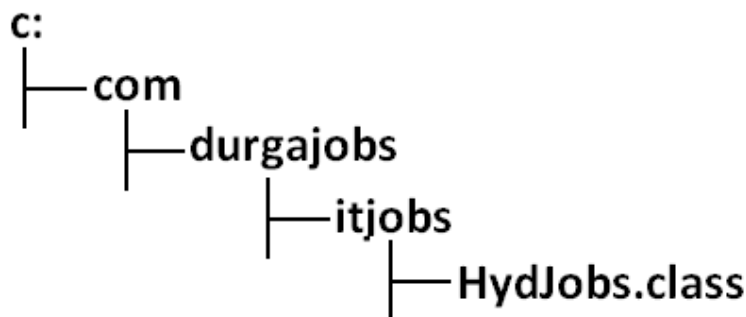


- If the specified package structure is not already available then this command itself will create the required package structure.
- As the destination we can use any valid directory.
- If the specified destination is not available then we will get compile time error.

Example:

D:\Java>javac -d c: HydJobs.java

Diagram:



If the specified destination is not available then we will get compile time error.

Example:

D:\Java>javac -d z: HydJobs.java

If Z: is not available then we will get compile time error.



**How to execute package Program:**

D:\Java>java com.durgajobs.itjobs.HydJobs

At the time of execution compulsory we should provide fully qualified name.

**Conclusion 1:**

In any java Program there should be at most one package statement that is if we are taking more than one package statement we will get compile time error.

**Example:**

```
package pack1;  
package pack2;  
class A  
{  
}
```

Output:

Compile time error.

D:\Java>javac A.java

A.java:2: class, interface, or enum expected  
package pack2;

**Conclusion 2:**

In any java Program the 1st non comment statement should be package statement [if it is available] otherwise we will get compile time error.

**Example:**

```
import java.util.*;  
package pack1;  
class A  
{  
}
```

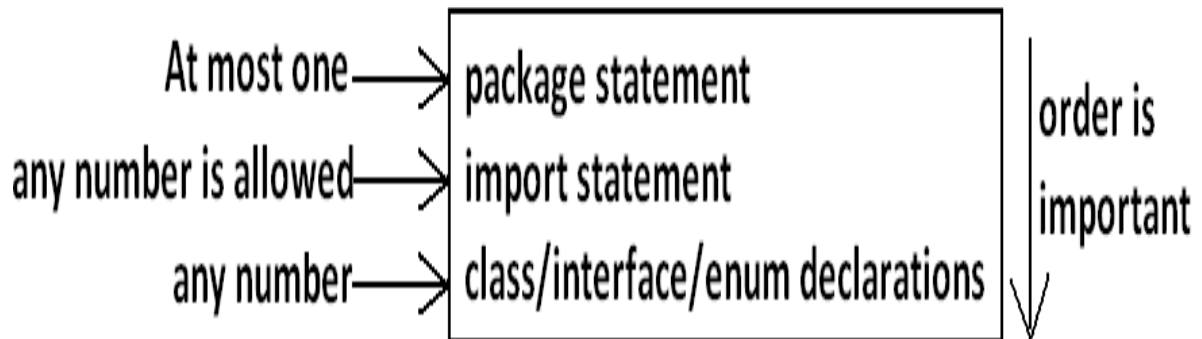
Output:

Compile time error.

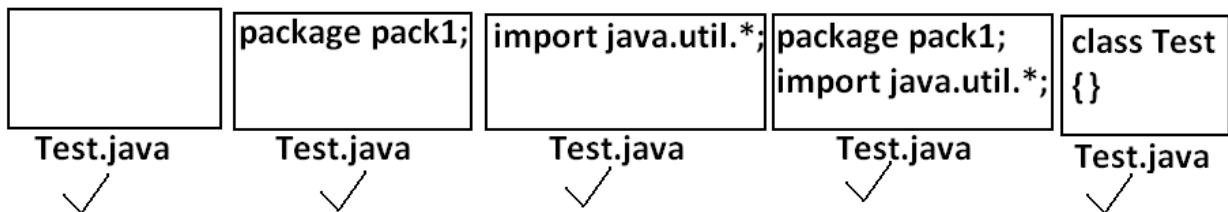
D:\Java>javac A.java

A.java:2: class, interface, or enum expected  
package pack1;

## Java source file structure:



All the following are valid java Programs.



**Note:** An empty source file is a valid java Program.

## Class Modifiers

Whenever we are writing our own classes compulsory we have to provide some information about our class to the jvm.

Like

1. Whether this class can be accessible from anywhere or not.
2. Whether child class creation is possible or not.
3. Whether object creation is possible or not etc.

We can specify this information by using the corresponding modifiers.

The only applicable modifiers for Top Level classes are:

1. Public
2. Default
3. Final
4. Abstract
5. Strictfp

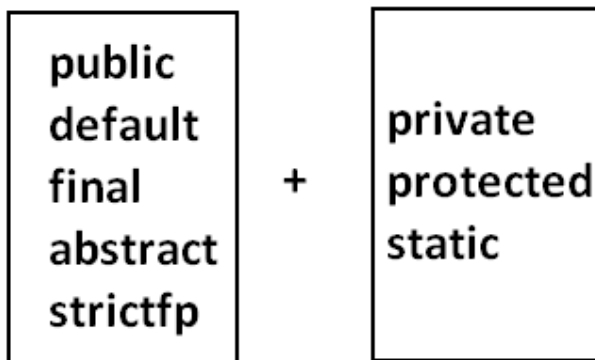
If we are using any other modifier we will get compile time error.

**Example:**

```
private class Test
{
    public static void main(String args[]){
        int i=0;
        for(int j=0;j<3;j++){
            {
                i=i+j;
            }
            System.out.println(i);
        }
    }
}
OUTPUT:
Compile time error.
D:\Java>javac Test.java
Test.java:1: modifier private not allowed here
private class Test
```

But For the inner classes the following modifiers are allowed.

**Diagram:**



**What is the difference between access specifier and access modifier ?**

- In old languages 'C' (or) 'C++' public, private, protected, default are considered as access specifiers and all the remaining are considered as access modifiers.
- But in java there is no such type of division all are considered as access modifiers.

### Public Classes:

If a class declared as public then we can access that class from anywhere. With in the package or outside the package.

**Example:**

**Program1:**

```
package pack1;
public class Test
```

```
{
public void methodOne(){
System.out.println("test class methodone is executed");
}}
```

Compile the above Program:

```
D:\Java>javac -d . Test.java
```

**Program2:**

```
package pack2;
import pack1.Test;
class Test1
{
public static void main(String args[]){
Test t=new Test();
t.methodOne();
}}
```

OUTPUT:

```
D:\Java>javac -d . Test1.java
```

```
D:\Java>java pack2.Test1
```

Test class methodone is executed.

If class Test is not public then while compiling Test1 class we will get compile time error saying pack1.Test is not public in pack1; cannot be accessed from outside package.

### Default Classes:

If a class declared as the default then we can access that class only within the current package hence default access is also known as "package level access".

**Example:**

**Program 1:**

```
package pack1;
class Test
{
public void methodOne(){
System.out.println("test class methodone is executed");
}}
```

**Program 2:**

```
package pack1;
import pack1.Test;
class Test1
{
public static void main(String args[]){
Test t=new Test();
t.methodOne();
}}
```

OUTPUT:

```
D:\Java>javac -d . Test.java
```

```
D:\Java>javac -d . Test1.java
```

```
D:\Java>java pack1.Test1
```

Test class methodone is executed

### Final Modifier:

Final is the modifier applicable for classes, methods and variables.

## Final Methods:

- Whatever the methods parent has by default available to the child.
- If the child is not allowed to override any method, that method we have to declare with final in parent class. That is final methods cannot overridden.

### Example:

#### Program 1:

```
class Parent
{
    public void property(){
        System.out.println("cash+gold+land");
    }
    public final void marriage(){
        System.out.println("subbalakshmi");
    }
}
```

#### Program 2:

```
class child extends Parent
{
    public void marriage(){
        System.out.println("Thamanna");
    }
}
```

#### OUTPUT:

Compile time error.

D:\Java>javac Parent.java

D:\Java>javac child.java

child.java:3: marriage() in child cannot override marriage() in Parent;  
overridden method is final  
public void marriage(){

## Final Class:

If a class declared as the final then we can't create the child class that is inheritance concept is not applicable for final classes.

### Example:

#### Program 1:

```
final class Parent
{
}
```

#### Program 2:

```
class child extends Parent
{
}
```

#### OUTPUT:

Compile time error.

D:\Java>javac Parent.java

D:\Java>javac child.java

child.java:1: cannot inherit from final Parent  
class child extends Parent

**Note:** Every method present inside a final class is always final by default whether we are declaring or not. But every variable present inside a final class need not be final.

**Example:**

```
final class parent
{
    static int x=10;
    static
    {
        x=999;
    }
}
```

The main advantage of final keyword is we can achieve security.

Whereas the main disadvantage is we are missing the key benefits of oops: polymorphism (because of final methods), inheritance (because of final classes) hence if there is no specific requirement never recommended to use final keyword.

### Abstract Modifier:

Abstract is the modifier applicable only for methods and classes but not for variables.

### Abstract Methods:

Even though we don't have implementation still we can declare a method with abstract modifier.

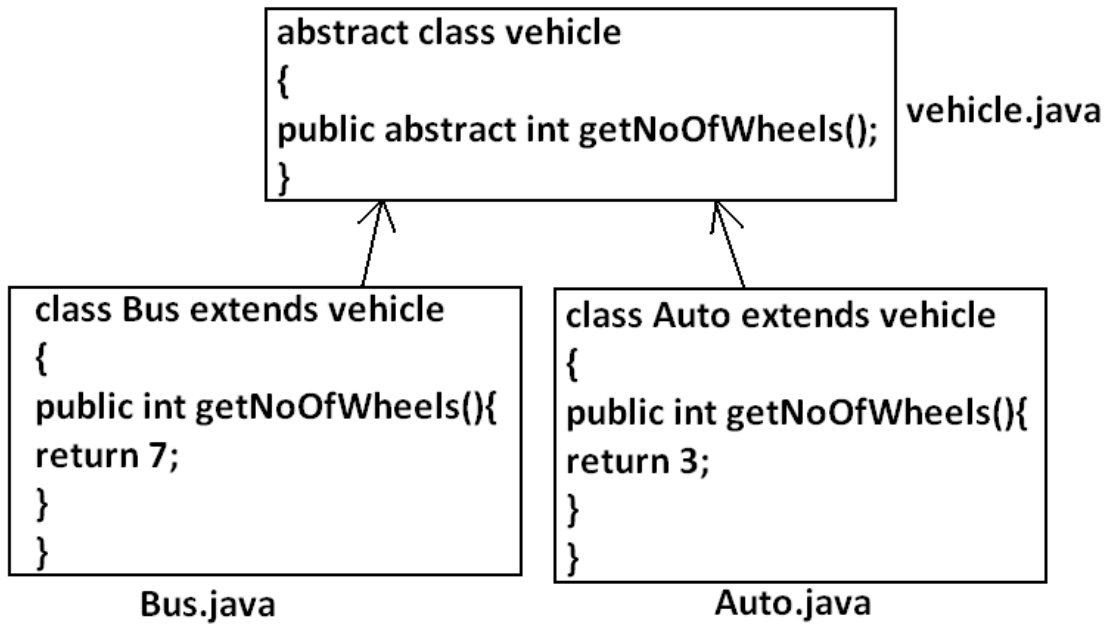
That is abstract methods have only declaration but not implementation.

Hence abstract method declaration should compulsorily end with semicolon.

**Example:**

```
public abstract void methodOne(); —————> valid
public abstract void methodOne(){} —————> invalid
```

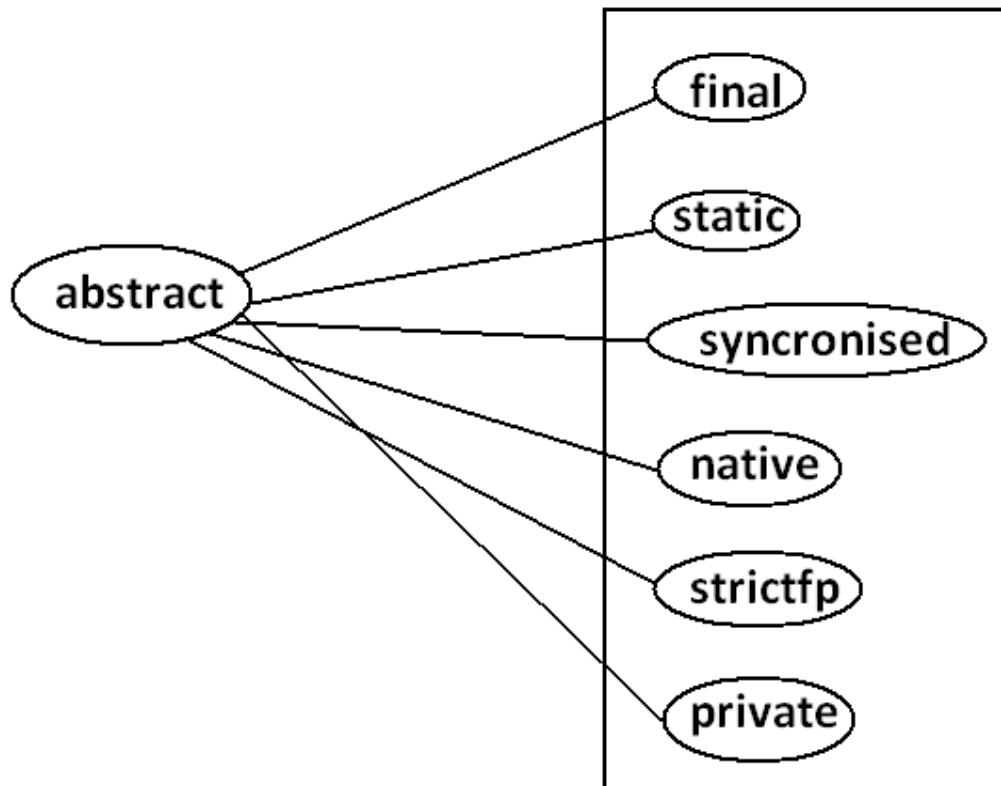
Child classes are responsible to provide implementation for parent class abstract methods.

Example:Program:

- The main advantage of abstract methods is , by declaring abstract method in parent class we can provide guide lines to the child class such that which methods they should compulsory implement.
- Abstract method never talks about implementation whereas if any modifier talks about implementation then the modifier will be enemy to abstract and that is always illegal combination for methods.

The following are the various illegal combinations for methods.

Diagram:



All the 6 combinations are illegal.

### Abstract class:

For any java class if we are not allow to create an object such type of class we have to declare with abstract modifier that is for abstract class instantiation is not possible.

#### Example:

```
abstract class Test
{
    public static void main(String args[]){
        Test t=new Test();
    }
}
```

Output:

Compile time error.

D:\Java>javac Test.java

Test.java:4: Test is abstract; cannot be instantiated

Test t=new Test();



**What is the difference between abstract class and abstract method ?**

- If a class contain at least on abstract method then compulsory the corresponding class should be declare with abstract modifier. Because implementation is not complete and hence we can't create object of that class.
- Even though class doesn't contain any abstract methods still we can declare the class as abstract that is an abstract class can contain zero no of abstract methods also.

**Example1:** HttpServlet class is abstract but it doesn't contain any abstract method.

**Example2:** Every adapter class is abstract but it doesn't contain any abstract method.

**Example1:**

```
class Parent
{
public void methodOne();
}
```

Output:

Compile time error.

D:\Java>javac Parent.java

Parent.java:3: missing method body, or declare abstract  
public void methodOne();

**Example2:**

```
class Parent
{
public abstract void methodOne(){}
}
```

Output:

Compile time error.

Parent.java:3: abstract methods cannot have a body

public abstract void methodOne(){}  
}

**Example3:**

```
class Parent
{
public abstract void methodOne();
}
```

Output:

Compile time error.

D:\Java>javac Parent.java

Parent.java:1: Parent is not abstract and does not  
override abstract method methodOne() in Parent  
class Parent

If a class extends any abstract class then compulsory we should provide implementation for every abstract method of the parent class otherwise we have to declare child class as abstract.

**Example:**

```
abstract class Parent
{
public abstract void methodOne();
public abstract void methodTwo();
}
class child extends Parent
```

```
{  
public void methodOne(){}  
}
```

Output:

Compile time error.

D:\Java>javac Parent.java

Parent.java:6: child is not abstract and does not  
override abstract method methodTwo() in Parent  
class child extends Parent

If we declare class child as abstract then the code compiles fine but child of child is responsible to provide implementation for methodTwo().

### What is the difference between final and abstract ?

- For abstract methods compulsory we should override in the child class to provide implementation. Whereas for final methods we can't override hence abstract final combination is illegal for methods.
- For abstract classes we should compulsory create child class to provide implementation whereas for final class we can't create child class. Hence final abstract combination is illegal for classes.
- Final class cannot contain abstract methods whereas abstract class can contain final method.

### Example:

|                                                                                |                                                                                  |
|--------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <pre>final class A<br/>{<br/>public abstract void<br/>methodOne();<br/>}</pre> | <pre>abstract class A<br/>{<br/>public final void methodOne(){<br/>}<br/>}</pre> |
| invalid                                                                        | valid                                                                            |

### Note:

Usage of abstract methods, abstract classes and interfaces is always good Programming practice.

### **Strictfp:**

- strictfp is the modifier applicable for methods and classes but not for variables.
- Strictfp modifier introduced in 1.2 versions.
- Usually the result of floating point of arithmetic is varying from platform to platform , to overcome this problem we should use strictfp modifier.

- If a method declare as the Strictfp then all the floating point calculations in that method has to follow IEEE754 standard, So that we will get platform independent results.

Example:

|                                    |                  |                       |
|------------------------------------|------------------|-----------------------|
| <b>System.out.println(10.0/3);</b> |                  |                       |
| <u><b>P4</b></u>                   | <u><b>P3</b></u> | <u><b>IEEE754</b></u> |
| <b>3.333333333333333</b>           | <b>3.333333</b>  | <b>3.333</b>          |

If a class declares as the Strictfp then every concrete method(which has body) of that class has to follow IEEE754 standard for floating point arithmetic, so we will get platform independent results.

What is the difference between abstract and strictfp ?

- Strictfp method talks about implementation where as abstract method never talks about implementation hence abstract, strictfp combination is illegal for methods.
- But we can declare a class with abstract and strictfp modifier simultaneously. That is abstract strictfp combination is legal for classes but illegal for methods.

Example:

```
public abstract strictfp void methodOne(); (invalid)
abstract strictfp class Test (valid)
{
}
```

## Member modifiers:

### Public members:

If a member declared as the public then we can access that member from anywhere "but the corresponding class must be visible" hence before checking member visibility we have to check class visibility.

**Example:****Program 1:**

```
package pack1;
class A
{
    public void methodOne(){
        System.out.println("a class method");
    }
}
D:\Java>javac -d . A.java
```

**Program 2:**

```
package pack2;
import pack1.A;
class B
{
    public static void main(String args[]){
        A a=new A();
        a.methodOne();
    }
}
Output:
Compile time error.
D:\Java>javac -d . B.java
B.java:2: pack1.A is not public in pack1;
        cannot be accessed from outside package
import pack1.A;
```

In the above Program even though methodOne() method is public we can't access from class B because the corresponding class A is not public that is both classes and methods are public then only we can access.

**Default member:**

If a member declared as the default then we can access that member only within the current package hence default member is also known as package level access.

**Example 1:****Program 1:**

```
package pack1;
class A
{
    void methodOne(){
        System.out.println("methodOne is executed");
    }
}
```

**Program 2:**

```
package pack1;
import pack1.A;
class B
{
    public static void main(String args[]){
        A a=new A();
        a.methodOne();
    }
}
Output:
D:\Java>javac -d . A.java
D:\Java>javac -d . B.java
D:\Java>java pack1.B
```

methodOne is executed

### Example 2:

#### Program 1:

```
package pack1;
class A
{
    void methodOne(){
        System.out.println("methodOne is executed");
    }
}
```

#### Program 2:

```
package pack2;
import pack1.A;
class B
{
    public static void main(String args[]){
        A a=new A();
        a.methodOne();
    }
}
```

Output:

Compile time error.

D:\Java>javac -d . A.java

D:\Java>javac -d . B.java

B.java:2: pack1.A is not public in pack1; cannot be accessed from outside package  
import pack1.A;

### **Private members:**

- If a member declared as the private then we can access that member only with in the current class.
- Private methods are not visible in child classes where as abstract methods should be visible in child classes to provide implementation hence private, abstract combination is illegal for methods.

### **Protected members:**

- If a member declared as the protected then we can access that member within the current package anywhere but outside package only in child classes.  
Protected=default+kids.
- We can access protected members within the current package anywhere either by child reference or by parent reference
- But from outside package we can access protected members only in child classes and should be by child reference only that is we can't use parent reference to call protected members from outside package.

### Example:

#### Program 1:

```
package pack1;
public class A
{
    protected void methodOne(){
        System.out.println("methodOne is executed");
    }
}
```

**Program 2:**

```

package pack1;
class B extends A
{
    public static void main(String args[]){
        A a=new A();
        a.methodOne();
        B b=new B();
        b.methodOne();
        A a1=new B();
        a1.methodOne();
    }
}

```

Output:

```

D:\Java>javac -d . A.java
D:\Java>javac -d . B.java
D:\Java>java pack1.B
methodOne is executed
methodOne is executed
methodOne is executed

```

**Example 2:**

```

package pack2;
import pack1.A;
public class C extends A
{
    public static void main(String args[]){
        A a=new A();
        a.methodOne();
        C c=new C();
        c.methodOne();
        A a1=new B();
        a1.methodOne();
    }
}

```

output:

compile time error.

D:\Java&gt;javac -d . C.java

C.java:7: methodOne() has protected access in pack1.A

a.methodOne();

**Compression of private, default, protected and public:**

| visibility                                | private | default | protected                                   | public |
|-------------------------------------------|---------|---------|---------------------------------------------|--------|
| 1)With in the same class                  | ✓       | ✓       | ✓                                           | ✓      |
| 2)From child class of same package        | ✗       | ✓       | ✓                                           | ✓      |
| 3)From non-child class of same package    | ✗       | ✓       | ✓                                           | ✓      |
| 4)From child class of outside package     | ✗       | ✗       | ✓<br>but we should use child reference only | ✓      |
| 5)From non-child class of outside package | ✗       | ✗       | ✗                                           | ✓      |

- The least accessible modifier is private.
- The most accessible modifier is public.

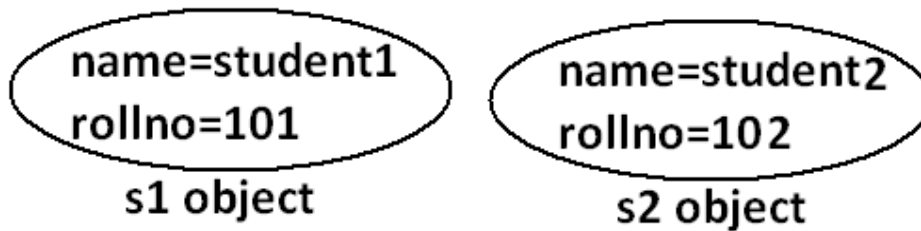
Private<default<protected<public

Recommended modifier for variables is private where as recommended modifier for methods is public.

**Final variables:****Final instance variables:**

- If the value of a variable is varied from object to object such type of variables are called instance variables.
- For every object a separate copy of instance variables will be created.

**DIAGRAM:**



For the instance variables it is not required to perform initialization explicitly jvm will always provide default values.

**Example:**

```

class Test
{
    int i;
    public static void main(String args[]){
        Test t=new Test();
        System.out.println(t.i);
    }
}

```

Output:

D:\Java>javac Test.java

D:\Java>java Test

0

If the instance variable declared as the final compulsory we should perform initialization explicitly and JVM won't provide any default values.

whether we are using or not otherwise we will get compile time error.

**Example:**

**Program 1:**

```

class Test
{

```

```

    int i;
}

```

Output:

D:\Java>javac Test.java

D:\Java>

**Program 2:**

```

class Test
{

```

```

    final int i;
}

```

Output:

Compile time error.

D:\Java>javac Test.java

Test.java:1: variable i might not have been initialized

```

class Test

```

**Rule:**

For the final instance variables we should perform initialization before constructor completion. That is the following are various possible places for this.

**1) At the time of declaration:**

**Example:**



```
class Test
{
    final int i=10;
}
Output:
D:\Java>javac Test.java
D:\Java>
```

## 2) Inside instance block:

### Example:

```
class Test
{
    final int i;
    {
        i=10;
    }
}
Output:
D:\Java>javac Test.java
D:\Java>
```

## 3) Inside constructor:

### Example:

```
class Test
{
    final int i;
    Test()
    {
        i=10;
    }
}
Output:
D:\Java>javac Test.java
D:\Java>
```

If we are performing initialization anywhere else we will get compile time error.

### Example:

```
class Test
{
    final int i;
    public void methodOne(){
        i=10;
    }
}
Output:
Compile time error.
D:\Java>javac Test.java
Test.java:5: cannot assign a value to final variable i
i=10;
```

## **Final static variables:**

- If the value of a variable is not varied from object to object such type of variables is not recommended to declare as the instance variables. We have to declare those variables at class level by using static modifier.

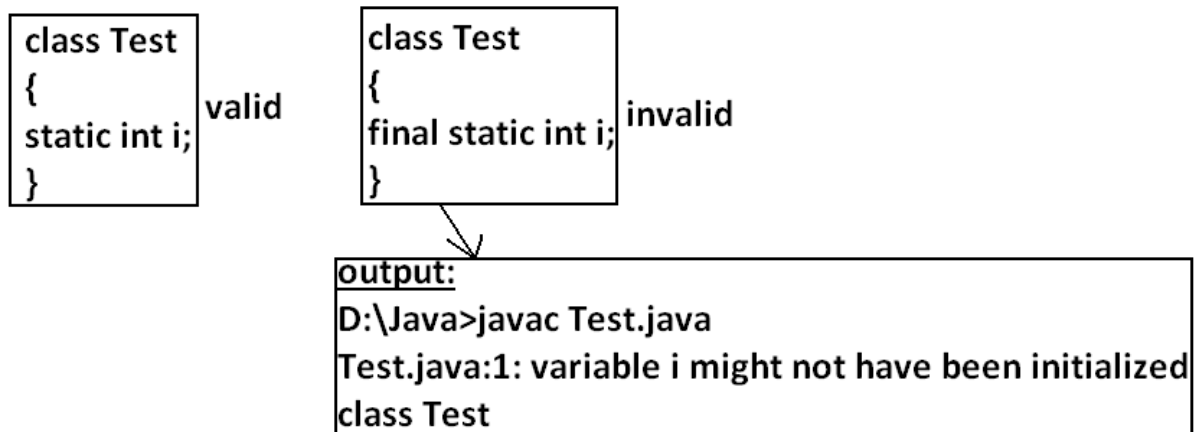
- In the case of instance variables for every object a separate copy will be created but in the case of static variables a single copy will be created at class level and shared by every object of that class.
- For the static variables it is not required to perform initialization explicitly jvm will always provide default values.

**Example:**

```
class Test
{
    static int i;
    public static void main(String args[]){
        System.out.println("value of i is :"+i);
    }
}
```

Output:  
D:\Java>javac Test.java  
D:\Java>java Test  
Value of i is: 0

If the static variable declare as final then compulsory we should perform initialization explicitly whether we are using or not otherwise we will get compile time error.(The JVM won't provide any default values)

**Example:****Rule:**

For the final static variables we should perform initialization before class loading completion otherwise we will get compile time error. That is the following are possible places.

**1) At the time of declaration:****Example:**

```
class Test
```

```
{
final static int i=10;
}
Output:
D:\Java>javac Test.java
D:\Java>
```

## 2) Inside static block:

### Example:

```
class Test
{
final static int i;
static
{
i=10;
}}
Output:
Compile successfully.
```

If we are performing initialization anywhere else we will get compile time error.

### Example:

```
class Test
{
final static int i;
public static void main(String args[]){
i=10;
}}
Output:
Compile time error.
D:\Java>javac Test.java
Test.java:5: cannot assign a value to final variable i
i=10;
```

## Final local variables:

- To meet temporary requirement of the Programmer sometime we can declare the variable inside a method or block or constructor such type of variables are called local variables.
- For the local variables jvm won't provide any default value compulsory we should perform initialization explicitly before using that variable.

### Example:

```
class Test
{
public static void main(String args[]){
int i;
System.out.println("hello");
}}
Output:
D:\Java>javac Test.java
D:\Java>java Test
Hello
```

### Example:

```
class Test
{
```

```
public static void main(String args[]){
int i;
System.out.println(i);
}}
Output:
Compile time error.
D:\Java>javac Test.java
Test.java:5: variable i might not have been initialized
System.out.println(i);
```

Even though local variable declared as the final before using only we should perform initialization.

**Example:**

```
class Test
{
public static void main(String args[]){
final int i;
System.out.println("hello");
}}
Output:
D:\Java>javac Test.java
D:\Java>java Test
hello
```

**Note:** The only applicable modifier for local variables is final if we are using any other modifier we will get compile time error.

**Example:**

```
class Test
{
public static void main(String args[])
{
private int x=10;————(invalid)
public int x=10; —————(invalid)
volatile int x=10;————(invalid)
transient int x=10; ———— (invalid)
final int x=10;—————(valid)
}
}
```

Output:

Compile time error.  
D:\Java>javac Test.java  
Test.java:5: illegal start of expression  
private int x=10;

### Formal parameters:

- The formal parameters of a method are simply acts as local variables of that method hence it is possible to declare formal parameters as final.
- If we declare formal parameters as final then we can't change its value within the method.

#### Example:

```
class Test{  
    public static void main(String args[]){  
        methodOne(10,20);  
    }  
    public static void methodOne(final int x,int y){  
        //x=100;  
        y=200;  
        System.out.println(x+" .... "+y);  
    }  
}
```

→ Formal parameters

output:  
compile time error.  
D:\Java>javac Test.java  
Test.java:6: final parameter x may not be assigned  
x=100;

- For instance and static variables JVM will provide default values but if instance and static declared as final JVM won't provide default value compulsory we should perform initialization whether we are using or not .
- For the local variables JVM won't provide any default values we have to perform explicitly before using that variables , this rule is same whether local variable final or not.

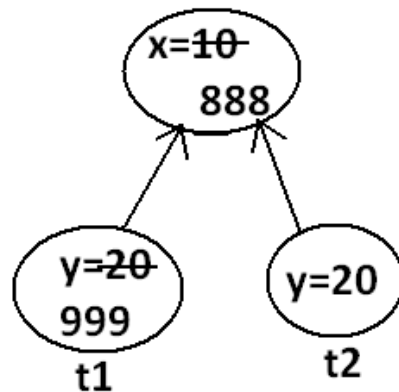
### Static modifier:

- Static is the modifier applicable for methods, variables and blocks.
- We can't declare a class with static but inner classes can be declaring as the static.

- In the case of instance variables for every object a separate copy will be created but in the case of static variables a single copy will be created at class level and shared by all objects of that class.

Example:

```
class Test{
static int x=10;
int y=20;
public static void main(String args[]){
Test t1=new Test();
t1.x=888;
t1.y=999;
Test t2=new Test();
System.out.println(t2.x+"....."+t2.y);
}
}
```

Output:

```
D:\Java>javac Test.java
D:\Java>java Test
888.....20
```

- Instance variables can be accessed only from instance area directly and we can't access from static area directly.
- But static variables can be accessed from both instance and static areas directly.

- 1) Int x=10;
- 2) Static int x=10;
- 3) Public void methodOne(){  
    System.out.println(x);  
    }
- 4) Public static void methodOne(){  
    System.out.println(x);  
    }

**Which are the following declarations are allow within the same class simultaneously ?**

a) 1 and 3

**Example:**

```
class Test
{
int x=10;
public void methodOne(){
System.out.println(x);
}}
Output:
Compile successfully.
```

b) 1 and 4

**Example:**

```
class Test
{
int x=10;
public static void methodOne(){
System.out.println(x);
}}
Output:
Compile time error.
D:\Java>javac Test.java
Test.java:5: non-static variable x cannot be referenced from a static
context
System.out.println(x);
```

c) 2 and 3

**Example:**

```
class Test
{
static int x=10;
public void methodOne(){
System.out.println(x);
}}
Output:
Compile successfully.
```

d) 2 and 4

**Example:**

```
class Test
{
static int x=10;
public static void methodOne(){
System.out.println(x);
}}
Output:
Compile successfully.
```

e) 1 and 2

**Example:**

```
class Test
{
int x=10;
static int x=10;
}
Output:
```

Compile time error.  
D:\Java>javac Test.java  
Test.java:4: x is already defined in Test  
static int x=10;

f) 3 and 4

**Example:**

```
class Test{
public void methodOne(){
System.out.println(x);
}
public static void methodOne(){
System.out.println(x);
}}
```

Output:

Compile time error.  
D:\Java>javac Test.java  
Test.java:5: methodOne() is already defined in Test  
public static void methodOne(){

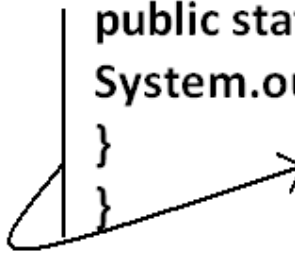
For static methods implementation should be available but for abstract methods implementation is not available hence static abstract combination is illegal for methods.

**case 1:**

Overloading concept is applicable for static method including main method also. But JVM will always call String[] args main method .  
The other overloaded method we have to call explicitly then it will be executed just like a normal method call .

**Example:**

```
class Test{
public static void main(String args[]){
System.out.println("String() method is called");
}
public static void main(int args[]){
System.out.println("int() method is called");
}
}
```



This method we have to call explicitly.



Output :  
String() method is called

### case 2:

Inheritance concept is applicable for static methods including main() method hence while executing child class, if the child doesn't contain main() method then the parent class main method will be executed.

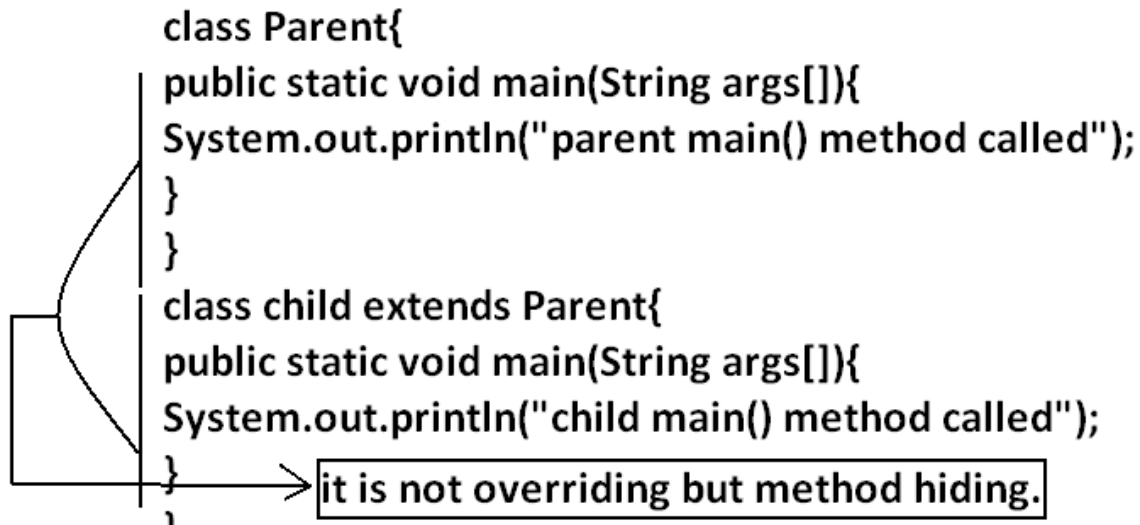
Example:

```
class Parent{  
public static void main(String args[]){  
System.out.println("parent main() method called");  
}  
}  
class child extends Parent{  
}  
Output:
```

```
javac    Parent.java  
  ↓      ↓  
Parent.class  Child.class  
      ↙      ↘  
    java Parent  
D:\Java>java Parent  
parent main() method called  
D:\Java>java child  
parent main() method called
```

Example:

```
class Parent{
    public static void main(String args[]){
        System.out.println("parent main() method called");
    }
}
class child extends Parent{
    public static void main(String args[]){
        System.out.println("child main() method called");
    }
}
```



it is not overriding but method hiding.

Output:

```
javac    Parent.java
  ↓      ↓
Parent.class  Child.class
      |
      |
java Parent ←
D:\Java>java Parent
parent main() method called
D:\Java>java child
child main() method called
```

- It seems to be overriding concept is applicable for static methods but it is not overriding it is method hiding.

### Native modifier:

- Native is a modifier applicable only for methods but not for variables and classes.
- The methods which are implemented in non java are called native methods or foreign methods.

The main objectives of native keyword are:

- To improve performance of the system.
- To use already existing legacy non-java code.
- To achieve machine level communication(memory level - address)
- Pseudo code to use native keyword in java.

To use native keyword:

### Pseudo code:

```

class Native{
static
{
1)load native library. System.loadLibrary("Native library");
}
2)native method declaration. ← public native void methodOne();
}
class Client
{
public static void main(String args[]){
Native n=new Native();
3) invoke a native method. ← n.methodOne();
}
}

```

- For native methods implementation is already available and we are not responsible to provide implementation hence native method declaration should compulsory ends with semicolon.
  - `Public native void methodOne()----`invalid
  - `Public native void methodOne();---`valid
- For native methods implementation is already available where as for abstract methods implementation should not be available child class is responsible to provide that, hence abstract native combination is illegal for methods.
- We can't declare a native method as `strictfp` because there is no guaranty whether the old language supports IEEE754 standard or not. That is native `strictfp` combination is illegal for methods.
- For native methods inheritance, overriding and overloading concepts are applicable.
- The main advantage of native keyword is performance will be improves.
- The main disadvantage of native keyword is usage of native keyword in java breaks platform independent nature of java language.

### Synchronized:

1. Synchronized is the modifier applicable for methods and blocks but not for variables and classes.
2. If a method or block declared with synchronized keyword then at a time only one thread is allow to execute that method or block on the given object.
3. The main advantage of synchronized keyword is we can resolve data inconsistency problems.
4. But the main disadvantage is it increases waiting time of the threads and effects performance of the system. Hence if there is no specific requirement never recommended to use synchronized keyword.

For synchronized methods compulsory implementation should be available , but for abstract methods implementation won't be available , Hence abstract - synchronized combination is illegal for methods.

### Transient modifier:

1. Transient is the modifier applicable only for variables but not for methods and classes.
2. At the time of serialization if we don't want to serialize the value of a particular variable to meet the security constraints then we should declare that variable with transient modifier.
3. At the time of serialization jvm ignores the original value of the transient variable and save default value that is transient means "not to serialize".
4. Static variables are not part of object state hence serialization concept is not applicable for static variables duo to this declaring a static variable as transient there is no use.
5. Final variables will be participated into serialization directly by their values due to this declaring a final variable as transient there is no impact.

**Volatile modifier:**

1. **Volatile is the modifier applicable only for variables but not for classes and methods.**
2. **If the value of variable keeps on changing such type of variables we have to declare with volatile modifier.**
3. **If a variable declared as volatile then for every thread a separate local copy will be created by the jvm, all intermediate modifications performed by the thread will take place in the local copy instead of master copy.**
4. **Once the value got finalized before terminating the thread that final value will be updated in master copy.**
5. **The main advantage of volatile modifier is we can resolve data inconsistency problems, but creating and maintaining a separate copy for every thread increases complexity of the Programming and affects performance of the system. Hence if there is no specific requirement never recommended to use volatile modifier and it's almost outdated.**
6. **Volatile means the value keeps on changing whereas final means the value never changes hence final volatile combination is illegal for variables.**

**Summary of modifier:**

| Modifiers    | Classes |       | Methods | Variables | Blocks | Interfaces |       | enum  |       | Constructors |
|--------------|---------|-------|---------|-----------|--------|------------|-------|-------|-------|--------------|
|              | Outer   | Inner |         |           |        | Outer      | inner | Outer | Inner |              |
| public       | ✓       | ✓     | ✓       | ✓         | ✗      | ✓          | ✓     | ✓     | ✓     | ✓            |
| private      | ✗       | ✓     | ✓       | ✓         | ✗      | ✗          | ✓     | ✗     | ✓     | ✓            |
| protected    | ✗       | ✓     | ✓       | ✓         | ✗      | ✗          | ✓     | ✗     | ✓     | ✓            |
| <default>    | ✓       | ✓     | ✓       | ✓         | ✗      | ✓          | ✓     | ✓     | ✓     | ✓            |
| final        | ✓       | ✓     | ✓       | ✓         | ✗      | ✗          | ✗     | ✗     | ✗     | ✗            |
| static       | ✗       | ✓     | ✓       | ✓         | ✓      | ✗          | ✓     | ✗     | ✓     | ✗            |
| synchronized | ✗       | ✗     | ✓       | ✗         | ✓      | ✗          | ✗     | ✗     | ✗     | ✗            |
| abstract     | ✓       | ✓     | ✓       | ✗         | ✗      | ✓          | ✓     | ✗     | ✗     | ✗            |
| native       | ✗       | ✗     | ✓       | ✗         | ✗      | ✗          | ✗     | ✗     | ✗     | ✗            |
| strictfp     | ✓       | ✓     | ✓       | ✗         | ✗      | ✓          | ✓     | ✓     | ✓     | ✗            |
| transient    | ✗       | ✗     | ✗       | ✓         | ✗      | ✗          | ✗     | ✗     | ✗     | ✗            |
| volatile     | ✗       | ✗     | ✗       | ✓         | ✗      | ✗          | ✗     | ✗     | ✗     | ✗            |

**Conclusions:**

- The Only Applicable Modifiers for Constructors are *public*, *private*, *protected*, and *<default>*.
- The Only Applicable Modifiers for Local Variable is *final*.
- The Only Modifier which is applicable for Classes but Not for Interfaces is *final*.
- The Modifiers which are Applicable for Classes but Not for enum are *final* and *abstract*.
- The Modifiers which are Applicable for Inner Classes but Not for Outer Classes are *public*, *protected*, and *static*.
- The Only Modifier which is Applicable for Methods is *native*.
- The Modifiers which are Applicable for Variables are *transient* and *volatile*.

**Java Modifiers Important Questions**

- 1) For the Top Level Classes which Modifiers are Allowed?
- 2) Is it Possible to Declare a Class as static, private, and protected?
- 3) What are Extra Modifiers Applicable for Inner Classes when compared with Outer Classes?
- 4) What is a final Class?
- 5) Explain the Differences between final, finally and finalize?
- 6) Is Every Method Present in final Class is final?
- 7) Is Every Variable Present Inside a final Class is final?
- 8) What is abstract Class?
- 9) What is abstract Method?
- 10) If a Class contain at least One abstract Method is it required to declared that Class Compulsory abstract?
- 11) If a Class doesn't contain any abstract Methods is it Possible to Declare that Class as abstract?
- 12) Whenever we are extending abstract Class is it Compulsory required to Provide Implementation for Every abstract Method of that Class?
- 13) Is final Class can contain abstract Method?
- 14) Is abstract Class can contain final Methods?
- 15) Can You give Example for abstract Class which doesn't contain any abstract Method?
- 16) Which of the following Modifiers Combinations are Legal for Methods?
  - public - static
  - static - abstract
  - abstract - final
  - final - synchronized
  - synchronized - native
  - native - abstract
- 17) Which of the following Modifiers Combinations are Legal for Classes?
  - public - final
  - final - abstract
  - abstract - strictfp
  - strictfp - public
- 18) What is the Difference between abstract Class and Interface?

- 19) What is strictfp Modifier?
- 20) Is it Possible to Declare a Variable with strictfp?
- 21) abstract - strictfp Combination, is Legal for Classes OR Methods?
- 22) Is it Possible to Override a native Method?
- 23) What is the Difference between Instance and Static Variable?
- 24) What is the Difference between General Static Variable and final Static Variable?
- 25) Which Modifiers are Applicable for Local Variable?
- 26) When the Static Variables will be Created?
- 27) What are Various Memory Locations of Instance Variables, Local Variables and Static Variables?
- 28) Is it Possible to Overload a main()?
- 29) Is it Possible to Override Static Methods?
- 30) What is native Key Word and where it is Applicable?
- 31) What is the Main Advantage of the native Key Word?
- 32) If we are using native Modifier how we can Maintain Platform Independent Nature?
- 33) How we can Declare a native Method?
- 34) Is abstract Method can contain Body?
- 35) What is synchronized Key Word where we can Apply?
- 36) What are Advantages and Disadvantages of synchronized Key Word?
- 37) Which Modifiers are the Most Dangerous in Java?
- 38) What is Serialization and Explain how its Process?
- 39) What is Deserialization?
- 40) By using which Classes we can Achieve Serialization and Deserialization?
- 41) What is Serializable interface and Explain its Methods?
- 42) What is a Marker Interface and give an Example?
- 43) Without having any Method in Serializable Interface, how we can get Serializable Ability for Our Object?



44)What is the Purpose of transient Key Word and Explain its Advantages?

45)Is it Possible to Serialize Every Java Object?

46)Is it Possible to Declare a Method, a Class with transient?

47)If we Declare Static Variable with transient is there any Impact?

48)What is the Impact of declaring final Variable a transient?

49)What is volatile Variable?

50)Is it Possible to Declare a Class OR a Method with volatile?

51)What is the Advantage and Disadvantage of volatile Modifier?

**Note :**

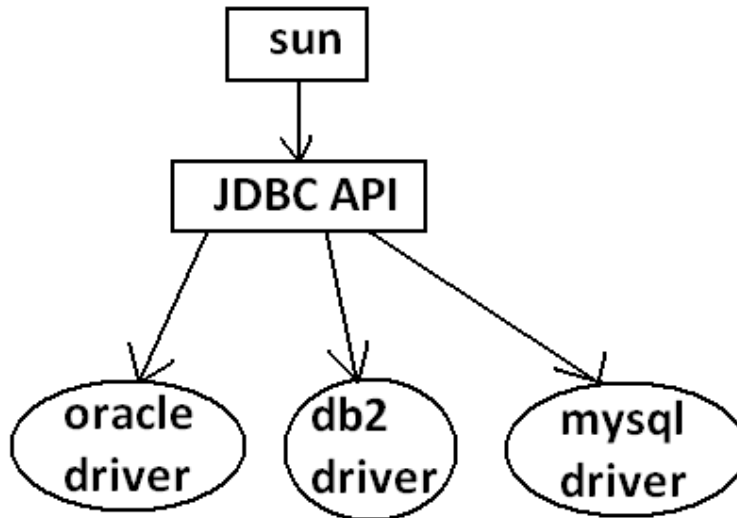
1. The modifiers which are applicable for inner classes but not for outer classes are private, protected, static.
2. The modifiers which are applicable only for methods native.
3. The modifiers which are applicable only for variables transient and volatile.
4. The modifiers which are applicable for constructor public, private, protected, default.
5. The only applicable modifier for local variables is final.
6. The modifiers which are applicable for classes but not for enums are final , abstract.
7. The modifiers which are applicable for classes but not for interface are final.

## Interfaces:

**Def1:** Any service requirement specification (srs) is called an interface.

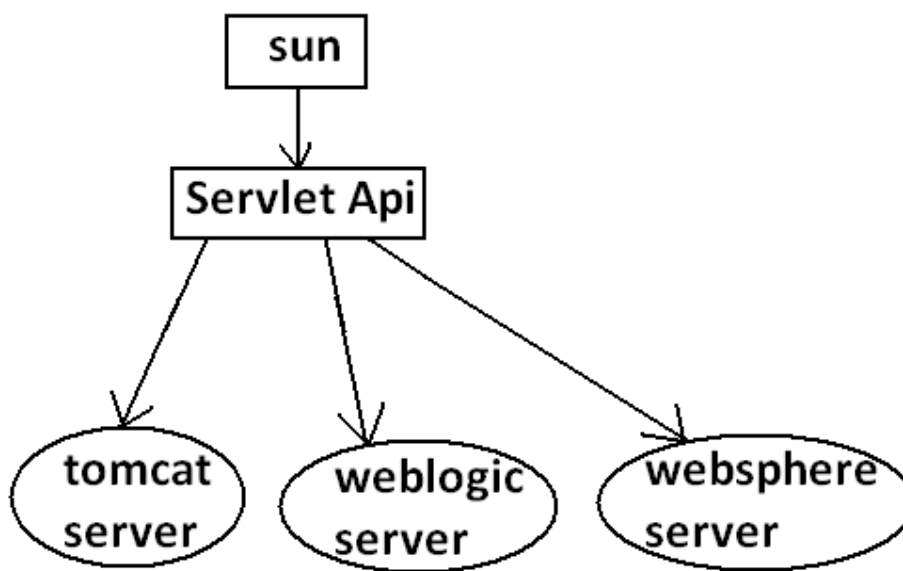
**Example1:** Sun people responsible to define JDBC API and database vendor will provide implementation for that.

Diagram:



**Example2:** Sun people define Servlet API to develop web applications web server vendor is responsible to provide implementation.

Diagram:



**Def2:** From the client point of view an interface define the set of services what is expecting. From the service provider point of view an interface defines the set of services what is offering. Hence an interface is considered as a contract between client and service provider.

**Example:** ATM GUI screen describes the set of services what bank people offering, at the same time the same GUI screen the set of services what customer is expecting hence this GUI screen acts as a contract between bank and customer.

**Def3:** Inside interface every method is always abstract whether we are declaring or not hence interface is considered as 100% pure abstract class.

**Summery def:** Any service requirement specification (SRS) or any contract between client and service provider or 100% pure abstract classes is considered as an interface.

### Declaration and implementation of an interface:

#### Note1:

Whenever we are implementing an interface compulsory for every method of that interface we should provide implementation otherwise we have to declare class as abstract in that case child class is responsible to provide implementation for remaining methods.

#### Note2:

Whenever we are implementing an interface method compulsory it should be declared as public otherwise we will get compile time error.

#### Example:

```
interface Interf
{
void methodOne();
void methodTwo();
}
```

```
abstract class ServiceProvider implements Interf{
    public void methodOne(){
    }}
    > 1
    > 2
```

```
class SubServiceProvider extends ServiceProvider
{
}
Output:
Compile time error.
D:\Java>javac SubServiceProvider.java
SubServiceProvider.java:1:
SubServiceProvider is not abstract and does not override
    abstract method methodTwo() in Interf
class SubServiceProvider extends ServiceProvider
```

### Extends vs implements:

A class can extends only one class at a time.

Example:

```
class One{
public void methodOne(){
}
}
class Two extends One{
}
```

A class can implements any no. Of interfaces at a time.

Example:

```
interface One{
public void methodOne();
}
interface Two{
public void methodTwo();
}
class Three implements One,Two{
public void methodOne(){
}
public void methodTwo(){
}
}
```

A class can extend a class and can implement any no. Of interfaces simultaneously.

```
interface One{
void methodOne();
}
class Two
{
public void methodTwo(){
}
}
class Three extends Two implements One{
public void methodOne(){
}
}
```

An interface can extend any no. Of interfaces at a time.

**Example:**

```
interface One{
void methodOne();
}
interface Two{
void methodTwo();
}
interface Three extends One,Two
{
}
```

Which of the following is true?

1. A class can extend any no.Of classes at a time.
2. An interface can extend only one interface at a time.
3. A class can implement only one interface at a time.
4. A class can extend a class and can implement an interface but not both simultaneously.
5. An interface can implement any no.Of interfaces at a time.
6. None of the above.

Ans: 6

Consider the expression X extends Y for which of the possibility of X and Y this expression is true?

1. Both x and y should be classes.
2. Both x and y should be interfaces.
3. Both x and y can be classes or can be interfaces.
4. No restriction.

Ans: 3

X extends Y, Z ?

X, Y, Z should be interfaces.

X extends Y implements Z ?

X, Y should be classes.

Z should be interface.

X implements Y, Z ?

X should be class.

Y, Z should be interfaces.

X implements Y extend Z ?

**Example:**

```
interface One{
}
class Two {
}
class Three implements One extends Two{
}
```

Output:  
 Compile time error.  
 D:\Java>javac Three.java  
 Three.java:5: '{' expected  
 class Three implements One extends Two{

### Interface methods:

Every method present inside interface is always public and abstract whether we are declaring or not. Hence inside interface the following method declarations are equal.

```
void methodOne();
public void methodOne();
abstract void methodOne();           Equal
public abstract void methodOne();
```

**public:** To make this method available for every implementation class.

**abstract:** Implementation class is responsible to provide implementation .

As every interface method is always public and abstract we can't use the following modifiers for interface methods.

Private, protected, final, static, synchronized, native, strictfp.

Inside interface which method declarations are valid?

1. public void methodOne(){}
2. private void methodOne();
3. public final void methodOne();
4. public static void methodOne();
5. public abstract void methodOne();

Ans: 5

### Interface variables:

- An interface can contain variables
- The main purpose of interface variables is to define requirement level constants.
- Every interface variable is always public static and final whether we are declaring or not.

#### Example:

```
interface interf
{
  int x=10;
}
```

**public:** To make it available for every implementation class.

**static:** Without existing object also we have to access this variable.

**final:** Implementation class can access this value but cannot modify.

Hence inside interface the following declarations are equal.

```
int x=10;
public int x=10;
static int x=10;
final int x=10;
public static int x=10;
public final int x=10;
static final int x=10;
public static final int x=10;
```

Equal

- As every interface variable by default public static final we can't declare with the following modifiers.
  - Private
  - Protected
  - Transient
  - Volatile
- For the interface variables compulsory we should perform initialization at the time of declaration only otherwise we will get compile time error.

**Example:**

```
interface Interf
{
int x;
}
```

Output:

Compile time error.

D:\Java>javac Interf.java

Interf.java:3: = expected

int x;

Which of the following declarations are valid inside interface ?

1. int x;
2. private int x=10;
3. public volatile int x=10;
4. public transient int x=10;
5. public static final int x=10;

Ans: 5

Interface variables can be access from implementation class but cannot be modified.

**Example:**

```
interface Interf
{
int x=10;
}
```

**Example 1:**

```

class Test implements Interf
{
    public static void main(String args[]){
        x=20;
        System.out.println("value of x"+x);
    }
}

```

**output:**

compile time error.

D:\Java&gt;javac Test.java

Test.java:4: cannot assign a value to final variable x

x=20;

**Example 2:**

```

class Test implements Interf
{
    public static void main(String args[]){
        int x=20;
        //here we declaring the variable x.
        System.out.println(x);
    }
}

```

**Output:**

D:\Java&gt;javac Test.java

D:\Java&gt;java Test

20\

**Interface naming conflicts:****Method naming conflicts:****Case 1:**

If two interfaces contain a method with same signature and same return type in the implementation class only one method implementation is enough.

**Example 1:**

```

interface Left
{
    public void methodOne();
}

```

**Example 2:**

```

interface Right
{

```



```
public void methodOne();
}
```

**Example 3:**

```
class Test implements Left,Right
{
public void methodOne()
{
}}
}}
```

Output:

```
D:\Java>javac Left.java
D:\Java>javac Right.java
D:\Java>javac Test.java
```

**Case 2:**

if two interfaces contain a method with same name but different arguments in the implementation class we have to provide implementation for both methods and these methods acts as a overloaded methods

**Example 1:**

```
interface Left
{
public void methodOne();
}
```

**Example 2:**

```
interface Right
{
public void methodOne(int i);
}
```

**Example 3:**

```
class Test implements Left,Right
{
public void methodOne()
{
}
public void methodOne(int i)
{
}}
}}
```

Output:

```
D:\Java>javac Left.java
D:\Java>javac Right.java
D:\Java>javac Test.java
```

**Case 3:**

If two interfaces contain a method with same signature but different return types then it is not possible to implement both interfaces simultaneously.

**Example 1:**

```
interface Left
{
public void methodOne();
}
```

**Example 2:**

```
interface Right
{
public int methodOne(int i);
}
```

We can't write any java class that implements both interfaces simultaneously.

Is a java class can implement any no. Of interfaces simultaneously ?

Yes, except if two interfaces contains a method with same signature but different return types.

### Variable naming conflicts:

Two interfaces can contain a variable with the same name and there may be a chance variable naming conflicts but we can resolve variable naming conflicts by using interface names.

#### Example 1:

```
interface Left
{
    int x=888;
}
```

#### Example 2:

```
interface Right
{
    int x=999;
}
```

#### Example 3:

```
class Test implements Left,Right
{
    public static void main(String args[]){
        //System.out.println(x);
        System.out.println(Left.x);
        System.out.println(Right.x);
    }
}
```

Output:

```
D:\Java>javac Left.java
D:\Java>javac Right.java
D:\Java>javac Test.java
D:\Java>java Test
888
999
```

### Marker interface:

If an interface doesn't contain any methods and by implementing that interface if our objects will get some ability such type of interfaces are called Marker interface (or) Tag interface (or) Ability interface.

#### Example:

```
Serializable
Cloneable
RandomAccess          These are marked for some ability
SingleThreadModel
.
.
.
.
```

**Example 1:**

By implementing Serializable interface we can send that object across the network and we can save state of an object into a file.

**Example 2:**

By implementing SingleThreadModel interface Servlet can process only one client request at a time so that we can get "Thread Safety".

**Example 3:**

By implementing Cloneable interface our object is in a position to provide exactly duplicate cloned object.

Without having any methods in marker interface how objects will get ability ?  
Internally JVM is responsible to provide required ability.

Why JVM is providing the required ability in marker interfaces ?  
To reduce complexity of the programming.

Is it possible to create our own marker interface ?

Yes, but customization of JVM must be required.

Ex : Sleepable , Jumpable , ....

**Adapter class:**

- Adapter class is a simple java class that implements an interface only with empty implementation for every method.
- If we implement an interface directly for each and every method compulsory we should provide implementation whether it is required or not. This approach increases length of the code and reduces readability.

**Example 1:**

```
interface X{
void m1();
void m2();
void m3();
void m4();
//.
//.
//.
//.
void m5();
}
```

**Example 2:**

```
class Test implements X{
public void m3(){
System.out.println("m3() method is called");
}
```

```

}
public void m1(){}
public void m2(){}
public void m4(){}
public void m5(){}
}

```

- We can resolve this problem by using adapter class.
- Instead of implementing an interface if we can extend adapter class we have to provide implementation only for required methods but not for all methods of that interface.
- This approach decreases length of the code and improves readability.

**Example 1:**

```

abstract class AdapterX implements X{
public void m1(){}
public void m2(){}
public void m3(){}
public void m4(){}
//.
//.
//.
public void m1000(){}
}

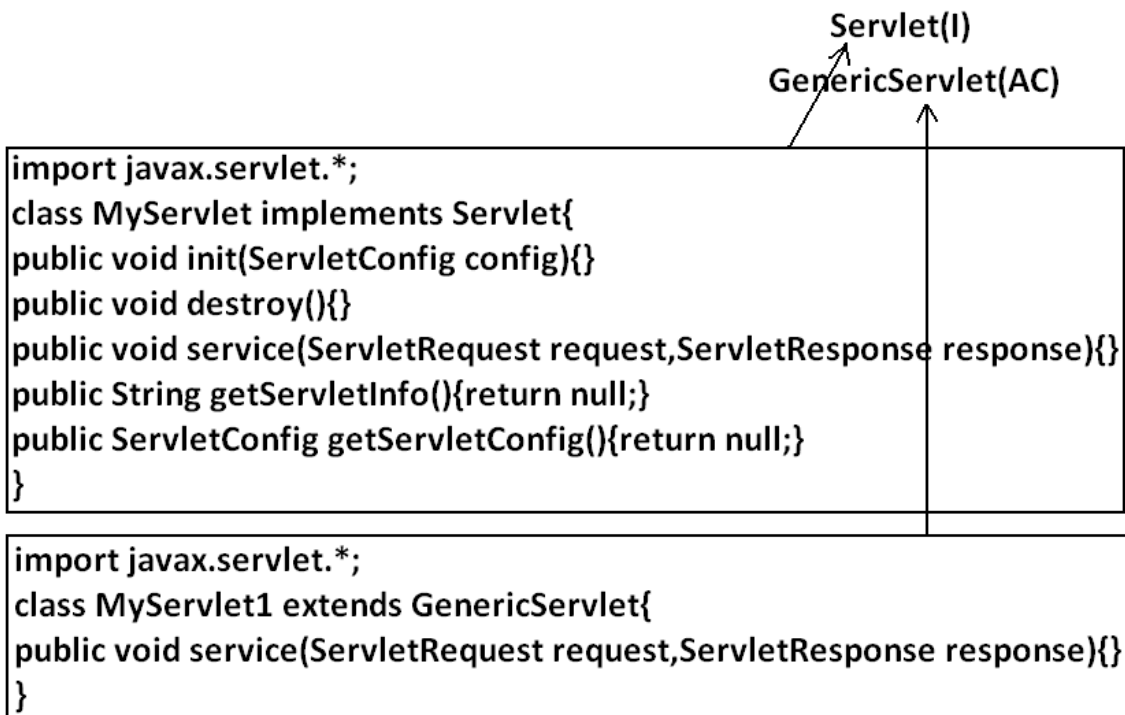
```

**Example 2:**

```

public class Test extend AdapterX{
public void m3(){
}}

```

**Example:**

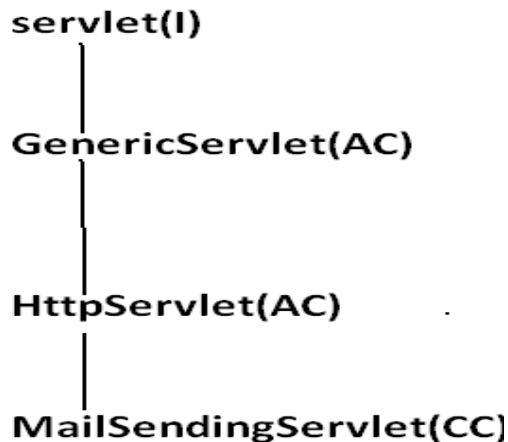
Generic Servlet simply acts as an adapter class for Servlet interface.

Note : marker interface and Adapter class are big utilities to the programmer to simplify programming.

**What is the difference between interface, abstract class and concrete class?  
When we should go for interface, abstract class and concrete class?**

- If we don't know anything about implementation just we have requirement specification then we should go for interface.
- If we are talking about implementation but not completely (partial implementation) then we should go for abstract class.
- If we are talking about implementation completely and ready to provide service then we should go for concrete class.

Example:



### What is the Difference between interface and abstract class ?

| interface                                                                                                                                      | Abstract class                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| If we don't know anything about implementation just we have requirement specification then we should go for interface.                         | If we are talking about implementation but not completely (partial implementation) then we should go for abstract class. |
| Every method present inside interface is always public and abstract whether we are declaring or not.                                           | Every method present inside abstract class need not be public and abstract.                                              |
| We can't declare interface methods with the modifiers private, protected, final, static, synchronized, native, strictfp.                       | There are no restrictions on abstract class method modifiers.                                                            |
| Every interface variable is always public static final whether we are declaring or not.                                                        | Every abstract class variable need not be public static final.                                                           |
| Every interface variable is always public static final we can't declare with the following modifiers. Private, protected, transient, volatile. | There are no restrictions on abstract class variable modifiers.                                                          |
| For the interface variables compulsory we should perform initialization at the time of declaration otherwise we will get compile time error.   | It is not require to perform initialization for abstract class variables at the time of declaration.                     |
| Inside interface we can't take static and instance blocks.                                                                                     | Inside abstract class we can take both static and instance blocks.                                                       |
| Inside interface we can't take constructor.                                                                                                    | Inside abstract class we can take constructor.                                                                           |

We can't create object for abstract class but abstract class can contain constructor what is the need ?

abstract class constructor will be executed when ever we are creating child class object to perform initialization of child object.

#### Example:

```
class Parent{
    Parent()
    {
        System.out.println(this.hashCode());
    }
}
class child extends Parent{
    child(){
        System.out.println(this.hashCode());
    }
}
class Test{
    public static void main(String args[]){
        child c=new child();
    }
}
```

```
System.out.println(c.hashCode());
}
}
```

Note : We can't create object for abstract class either directly or indirectly.

**Every method present inside interface is abstract but in abstract class also we can take only abstract methods then what is the need of interface concept ?**

We can replace interface concept with abstract class. But it is not a good programming practice. We are misusing the roll of abstract class. It may create performance problems also.

(this is just like recruiting IAS officer for sweeping purpose)

### Example :

```
interface X {
    -----
    -----
}
class Test implements X {
    -----
    -----
}
```

```
Test t=new Test();
//takes 2 sec
1) performance is high
2) while implementing X
   we can extend some
   other classes
```

```
abstract class X {
    -----
    -----
}
class Test extends X {
    -----
    -----
}
```

```
Test t=new Test();
//takes 20sec
1) performance is low
2) while extending X we can't
   extend any other classes
```

If every thing is abstract then it is recommended to go for interface.

**Why abstract class can contain constructor where as interface doesn't contain constructor ?**

The main purpose of constructor is to perform initialization of an object i.e., provide values for the instance variables, Inside interface every variable is always static and there is no chance of existing instance variables. Hence constructor is not required for interface.

But abstract class can contains instance variable which are required for the child object to perform initialization for those instance variables constructor is required in the case of abstract class.

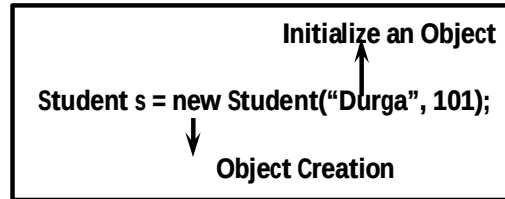
### **Interfaces Important Questions**

- 1) What is Interface?
- 2) What is Difference between Interface and Abstract Class?
- 3) When we should Go for Interface and Abstract Class and Concrete Class?
- 4) What Modifiers Applicable for Interfaces?
- 5) Explain about Interface Variables and what Modifiers are Applicable for them?
- 6) Explain about Interface Methods and what Modifiers are Applicable for them?
- 7) Can Java Class implement any Number of Interfaces?
- 8) If 2 Interfaces contains a Method with Same Signature but different Return Types, then how we can implement Both Interfaces Simultaneously?
- 9) Difference between extends and implements Key Word?
- 10) We cannot Create an Object of Abstract Class then what is Necessity of having Constructor Inside Abstract Class?
- 11) What is a Marker Interface? Give an Example?
- 12) What is Adapter Class and Explain its Usage?
- 13) An Interface contains only Abstract Methods and an Abstract Class also can contain only Abstract Methods then what is the Necessity of Interface?
- 14) In Your Previous Project where You used the following Marker Interface, Abstract Class, Interface and Adapter Class?



**The Purpose of Constructor is**

- To Initialize an Object but Not to Create an Object.
- Whenever we are creating an Object after Object Creation automatically Constructor will be executed to Initialize that Object.
- Object Creation by New Operator and then Initialization by Constructor.



- Before Constructor Only Object is Ready and Hence within the Constructor we can Access Object Properties Like Hash Code.

```

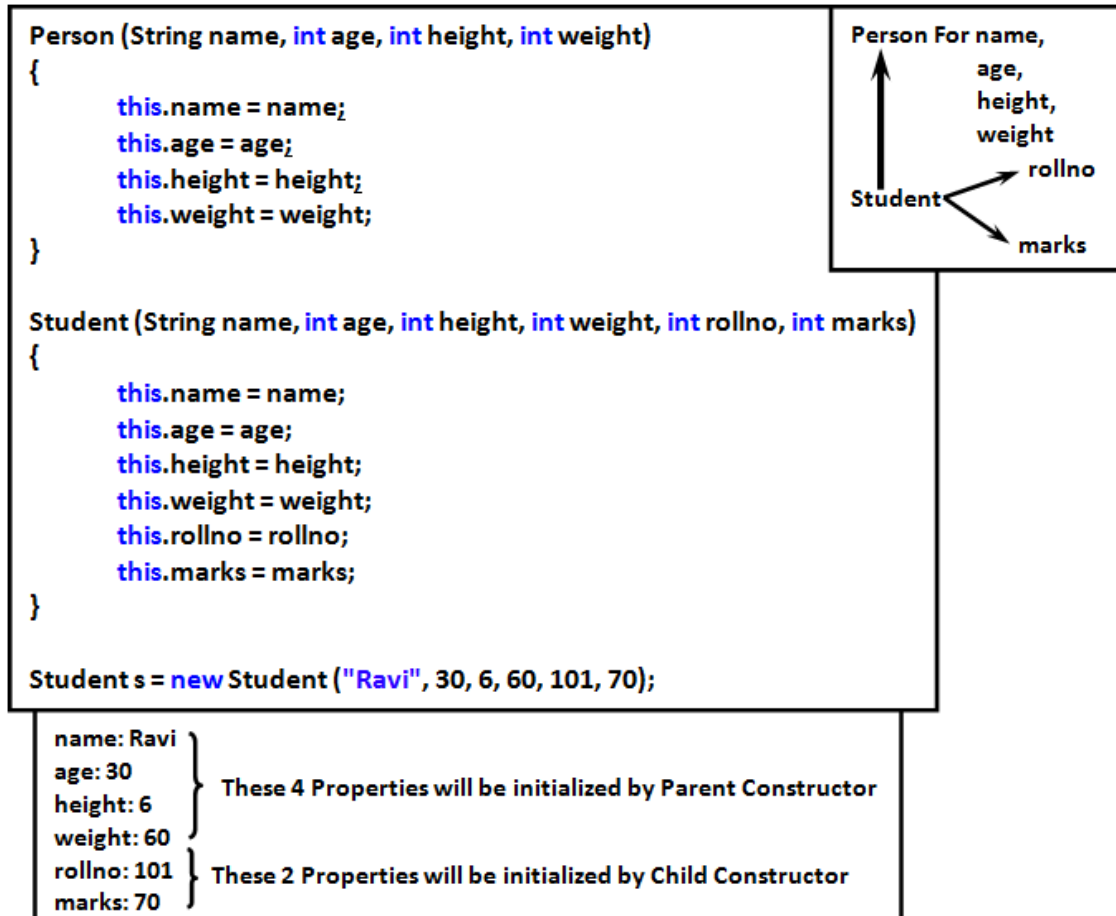
class Test {
    Test() {
        System.out.println(this); // Test@6e3d60
        System.out.println(this.hashCode()); //7224672
    }
    public static void main(String[] args) {
        Test t = new Test();
    }
}
    
```

- Whenever we are creating Child Class Object automatically Parent Constructor will be executed but Parent Object won't be created.
- The Purpose of Parent Constructor Execution is to Initialize Child Object Only. Of Course for the Instance Variables which are inheriting from parent Class.

```

class P {
    P() {
        System.out.println(this.hashCode()); //7224672
    }
}
class C extends P {
    C() {
        System.out.println(this.hashCode()); //7224672
    }
}
class Test {
    public static void main(String[] args) {
        C c = new C();
        System.out.println(c.hashCode()); //7224672
    }
}
    
```

- In the Above Example whenever we are creating Child Object Both Parent and Child Constructors will be executed for Child Object Purpose Only.
- In the Above Example we are Just creating Student Object but Both Person and Student Constructor to Initialize Student Object Only.



- Whenever we are creating an Object automatically Constructor will be executed but it May be One Constructor OR Multiple Constructors.
- Either Directly OR Indirectly we can't Create Object for Abstract Class but Abstract Class can contain Constructor what is the Need.
- Whenever we are creating Child Object Automatically Abstract Class Constructor will be executed to Perform Initialization of Child Object for the Properties whether inheriting from Abstract Class (Code Reusability).

Without abstract Class Constructor

```
class Student extends Person {  
    int rollno;  
    int marks;  
    Student(String name, int age, int height, int weight) {  
        this.name = name;  
        this.age = age;  
        this.height = height;  
        this.weight = weight;  
        this.rollno = rollno;  
        this.marks = marks;  
    }  
}
```

```
Student s = new Student("Durga", 30, 6, 60, 101, 70);
```

```
abstract class Person {  
    String name;  
    int age, height, weight;  
}
```

```
class Teacher extends Person {  
    String subject;  
    double salary;  
}
```

```
Teacher(String name, int age, int height, int weight, String subject, double salary) {  
    this.name = name;  
    this.age = age;  
    this.height = height;  
    this.weight = weight;  
    this.rollno = rollno;  
    this.marks = marks;  
    this.subject = subject;  
    this.salary = salary;  
}
```

```
Teacher t = new Teacher("Ravi", 50, 6, 70, "Java", 50K);
```

**With abstract Class Constructor**

```
abstract class Person {  
    String name;  
    int age;  
    int height;  
    int weight;  
    public Person(String name, int age, int height, int weight) {  
        super();  
        this.name = name;  
        this.age = age;  
        this.height = height;  
        this.weight = weight;  
    }  
}
```

```
class Student extends Person {  
    int rollNo, marks;  
    Student(String name, int age, int height, int weight, int rollNo, int marks) {  
        super(name, age, height, weight);  
        this.rollNo = rollNo;  
        this.marks = marks;  
    }  
}
```

```
class Teacher extends Person {  
    String sub;  
    double salary;  
    Teacher(String name, int age, int height, int weight, String sub, double salary) {  
        super(name, age, height, weight);  
        this.sub = sub;  
        this.salary = salary;  
    }  
}
```

**Any Way we can't Create Object for Abstract Class and Interface. But Abstract Class can contain Constructor but Interface doesn't Why?**

- The Main Purpose of Constructor is to Perform Initialization of an Object i.e. to Provide Values for Instance Variables.
- Abstract Class can contain Instance Variables which are required for Child Class Object to Perform Initialization of these Instance Variables Constructor is required Inside Abstract Class.
- But Every Variable Present Inside Interface is Always public, static and final whether we are declaring OR Not and Every Interface Variable we should Perform Initialization at the Time of Declaration and Hence Inside Interface there is No Chance of existing Instance Variable.
- Due to this Initialization of Instance Variables Concept Not Applicable for Interfaces.
- Hence Constructor Concept Not required for Interface.

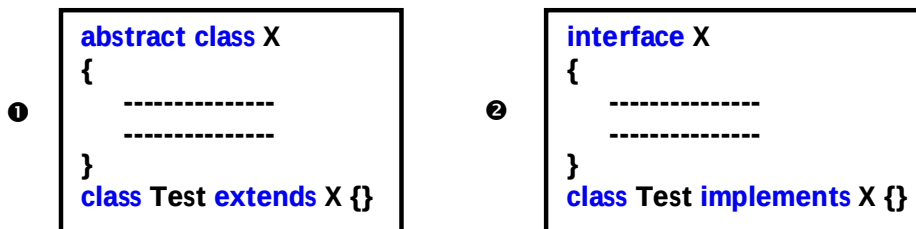
```

abstract class Person {
    String name;
    int age;
    Person(String name, int age) {
        this.name = name;
        this.age = age; → Current Child Class Object
    }
}

```

**Inside Interface we can take Only Abstract Methods. But in Abstract Class Also we can take Only Abstract Methods Based on Our Requirement. Then what is the Need of Interface? i.e. Is it Possible to Replace Interface Concept with Abstract Class?**

We can Replace Interface with Abstract Class but it is Not Good Programming Practice (This is Like requesting IAS Officer for sweeping Activity)



- While extending X we can't extend any Other Class.
- While implementing Interface we can extend any Other Class and Hence we won't Miss any Inheritance Benefit.
- In the Case ① Object Creation is Costly.  
Test t = new Test(); → 2 Mins
- In the Case ② Object Creation is Not Costly.  
Test t = new Test(); → 2 Sec
- Hence if everything is Abstract Highly Recommended to Use Interface but Not Abstract Class.
- If we are talking About Implementation of Course Partial Implementation then Only we should go for Abstract Class.

**Which of the following are Valid?**

- 1) The Purpose of Constructor is to Create an Object. ✗
- 2) The Purpose of Constructor is to Initialize an Object but Not to Create Object. ✓
- 3) Once Constructor completes then Only Object Creation completes. ✗
- 4) First Object will be Created and then Constructor will be executed. ✓
- 5) The Purpose of new Key Word is to Create Object and the Purpose of Constructor is to Initialize that Object. ✓
- 6) We can't Create Object for Abstract Class Directly but Indirectly we can Create. ✓
- 7) Whenever we are creating Child Class Object Automatically Parent Class Object will be created Internally. ✗
- 8) Whenever we are creating Child Class Object Automatically Abstract Class Constructor will be executed. ✓
- 9) Whenever we are creating Child Class Object Automatically Parent Object will be Created. ✗
- 10) Whenever we are creating Child Class Object Automatically Parent Constructor will be executed but Parent Object won't be Created. ✓
- 11) Either Directly OR Indirectly we can't Create Object for Abstract Class and Hence Constructor Concept is Not Applicable for Abstract Class. ✗
- 12) Interface can contain Constructor. ✗

# **CORE JAVA**

## **With**

# **SCJP / OCJP**

### **Study Material**

#### **Chapter 5: OOPS**



**DURGA M.Tech**

**(Sun certified & Realtime Expert)**

**Ex. IBM Employee**

**Trained Lakhs of Students  
for last 14 years across INDIA**

**India's No.1 Software Training Institute**

# **DURGASOFT**

**www.durgasoft.com Ph: 9246212143 ,8096969696**

# Object Oriented Programming (OOPS)

## Agenda:

1. Data Hiding
  2. Abstraction
  3. Encapsulation
  4. Tightly Encapsulated Class
  5. IS-A Relationship(Inheritance)
    - o Multiple inheritance
    - o Cyclic inheritance
  6. HAS-A Relationship
    - o Composition
    - o Aggregation
  7. Method Signature
  8. Polymorphism
    - o Overloading
      - Automatic promotion in overloading
    - o Overriding
      - Rules for overriding
      - Checked Vs Un-checked Exceptions
      - Overriding with respect to static methods
      - Overriding with respect to Var-arg methods
      - Overriding with respect to variables
      - Differences between overloading and overriding ?
    - o Method Hiding
  9. Static Control Flow
    - o Static control flow parent to child relationship
    - o Static block
  10. Instance Control Flow
    - o Instance control flow in Parent to Child relationship
  11. Constructors
    - o Constructor Vs instance block
    - o Rules to write constructors
    - o Default constructor
    - o Prototype of default constructor
    - o super() vs this():
    - o Overloaded constructors
    - o Recursive functions
  12. Coupling
  13. Cohesion
  14. Object Type Casting
    - o Compile time checking
    - o Runtime checking
- 
- Difference between ArrayList l=new ArrayList() & List l=new ArrayList() ?
  - In how many ways get an object in java ?



- Singleton classes
- Factory method

## Data Hiding :

- Our internal data should not go out directly that is outside person can't access our internal data directly.
- By using private modifier we can implement data hiding.

Example:

```
class Account {  
    private double balance;  
    .....;  
    .....;  
}
```

After providing proper username and password only , we can access our Account information.

The main advantage of data hiding is security.

Note: recommended modifier for data members is private.

## Abstraction :

- Hide internal implementation and just highlight the set of services, is called abstraction.
- By using abstract classes and interfaces we can implement abstraction.

Example :

By using ATM GUI screen bank people are highlighting the set of services what they are offering without highlighting internal implementation.

The main advantages of Abstraction are:

1. We can achieve security as we are not highlighting our internal implementation.(i.e., outside person doesn't aware our internal implementation.)
2. Enhancement will become very easy because without effecting end user we can able to perform any type of changes in our internal system.
3. It provides more flexibility to the end user to use system very easily.
4. It improves maintainability of the application.
5. It improves modularity of the application.
6. It improves easyness to use our system.

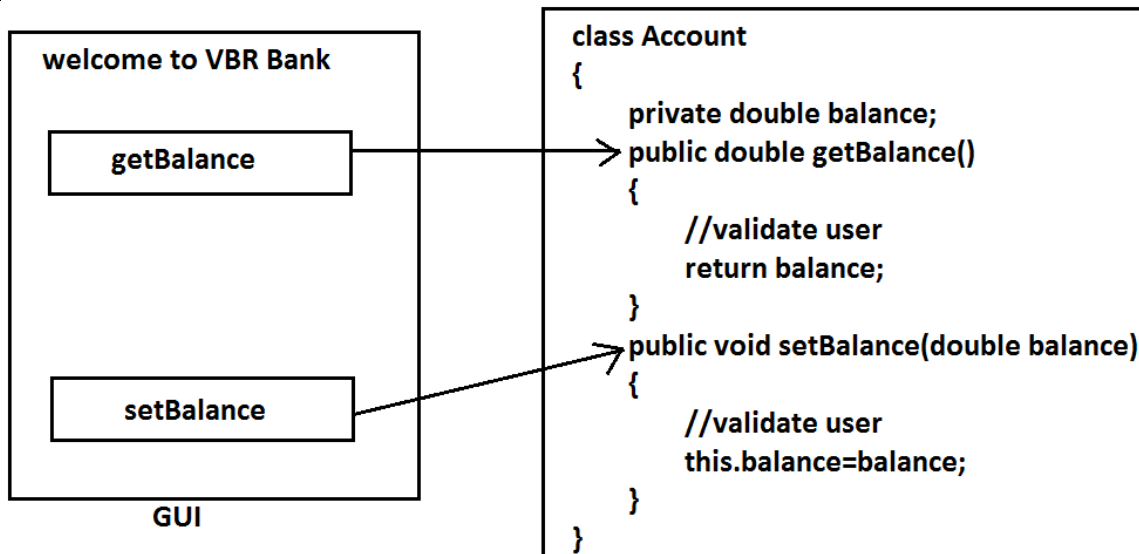
By using interfaces (GUI screens) we can implement abstraction

## Encapsulation :

- Binding of data and corresponding methods into a single unit is called Encapsulation .
- If any java class follows data hiding and abstraction such type of class is said to be encapsulated class.

Encapsulation=Datahiding+Abstraction

Example:



Every data member should be declared as private and for every member we have to maintain getter & Setter methods.

The main advantages of encapsulation are :

1. We can achieve security.
2. Enhancement will become very easy.
3. It improves maintainability and modularity of the application.
4. It provides flexibility to the user to use system very easily.

The main disadvantage of encapsulation is it increases length of the code and slows down execution.

## Tightly encapsulated class :

A class is said to be tightly encapsulated if and only if every variable of that class declared as private whether the variable has getter and setter methods are not , and whether these methods declared as public or not, these checkings are not required to perform.

Example:

```
class Account {
    private double balance;
    public double getBalance() {
        return balance;
    }
}
```

Which of the following classes are tightly encapsulated?

```
class A
{
    private int x=10; (valid)
}
class B extends A
{
    int y=20;(invalid)
}
class C extends A
{
    private int z=30; (valid)
}
```

Which of the following classes are tightly encapsulated?

```
class A {
    int x=10;    //not
}
class B extends A {
    private int y=20; //not
}
class C extends B {
    private int z=30; //not
}
```

Note: if the parent class is not tightly encapsulated then no child class is tightly encapsulated.

## IS-A Relationship(inheritance) :

1. Also known as inheritance.
2. By using "extends" keywords we can implement IS-A relationship.
3. The main advantage of IS-A relationship is reusability.

Example:

```
class Parent {
    public void methodOne(){ }
}
class Child extends Parent {
    public void methodTwo() { }
}
```

class Test

```
{
    public static void main(String[] args)
    {
        Parent p=new Parent();
        p.methodOne();
        p.methodTwo();
        Child c=new Child();
        c.methodOne();
        c.methodTwo();
        Parent p1=new Child();
        p1.methodOne();
        p1.methodTwo();
        Child c1=new Parent();
    }
}
```

C.E: cannot find symbol  
symbol : method methodTwo()  
location: class Parent

C.E: incompatible types  
found : Parent  
required: Child

Conclusion :

1. Whatever the parent has by default available to the child but whatever the child has by default not available to the parent. Hence on the child reference we can call both parent and child class methods. But on the parent reference we can call only methods available in the parent class and we can't call child specific methods.
2. Parent class reference can be used to hold child class object but by using that reference we can call only methods available in parent class and child specific methods we can't call.
3. Child class reference cannot be used to hold parent class object.

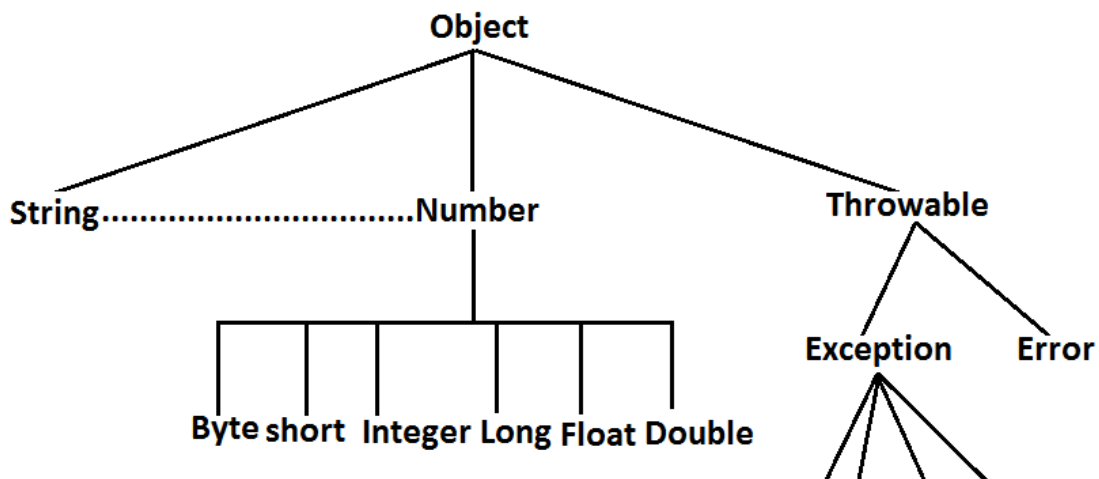
**Example:**

The common methods which are required for housing loan, vehicle loan, personal loan and education loan we can define into a separate class in parent class loan. So that automatically these methods are available to every child loan class.

Example:

```
class Loan {
    //common methods which are required for any type of loan.
}
class HousingLoan extends Loan {
    //Housing loan specific methods.
}
class EducationLoan extends Loan {
    //Education Loan specific methods.
}
```

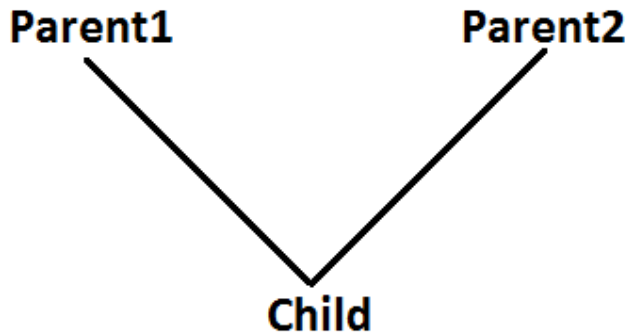
- For all java classes the most commonly required functionality is define inside object class hence object class acts as a root for all java classes.
- For all java exceptions and errors the most common required functionality defines inside Throwable class hence Throwable class acts as a root for exception hierarchy.

**Diagram:**

## Multiple inheritance :

Having more than one Parent class at the same level is called multiple inheritance.

Example:



Any class can extend only one class at a time and can't extend more than one class simultaneously hence java won't provide support for multiple inheritance.

Example:

```
class A{}
class B{}
class C extends A,B
{}
```

(invalid)

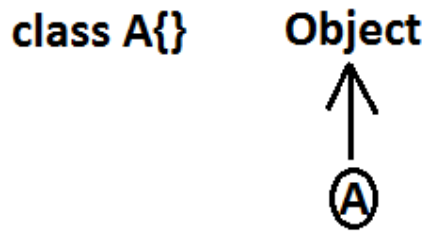
But an interface can extend any no. Of interfaces at a time hence java provides support for multiple inheritance through interfaces.

Example:

```
interface A{}
interface B{}
interface C extends A,B{}
```

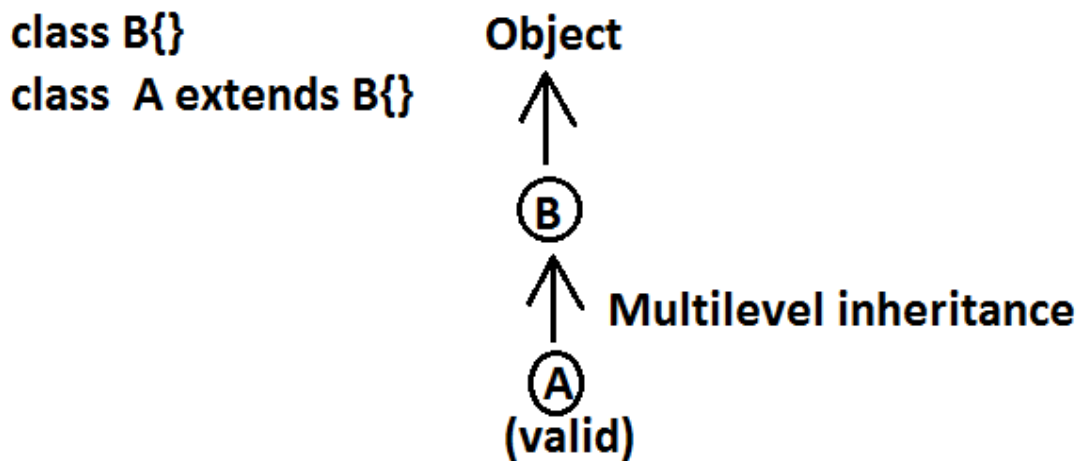
If our class doesn't extend any other class then only our class is the direct child class of object.

Example:

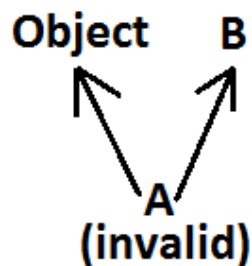


If our class extends any other class then our class is not direct child class of object, It is indirect child class of object , which forms multilevel inheritance.

Example 1:

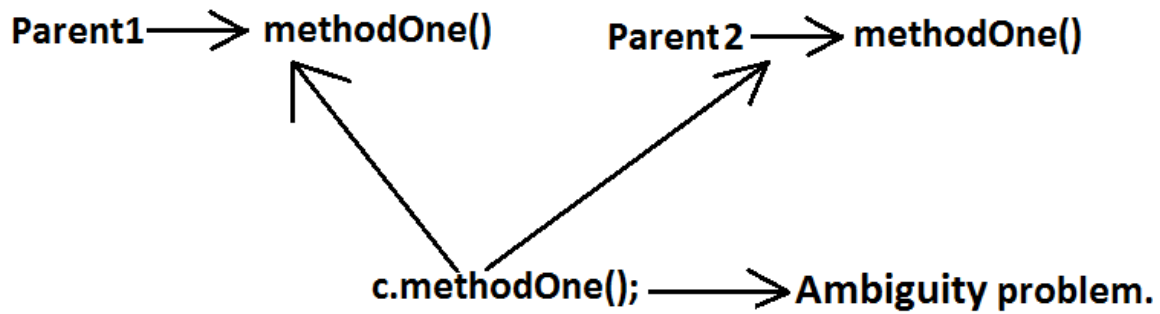


Example 2:



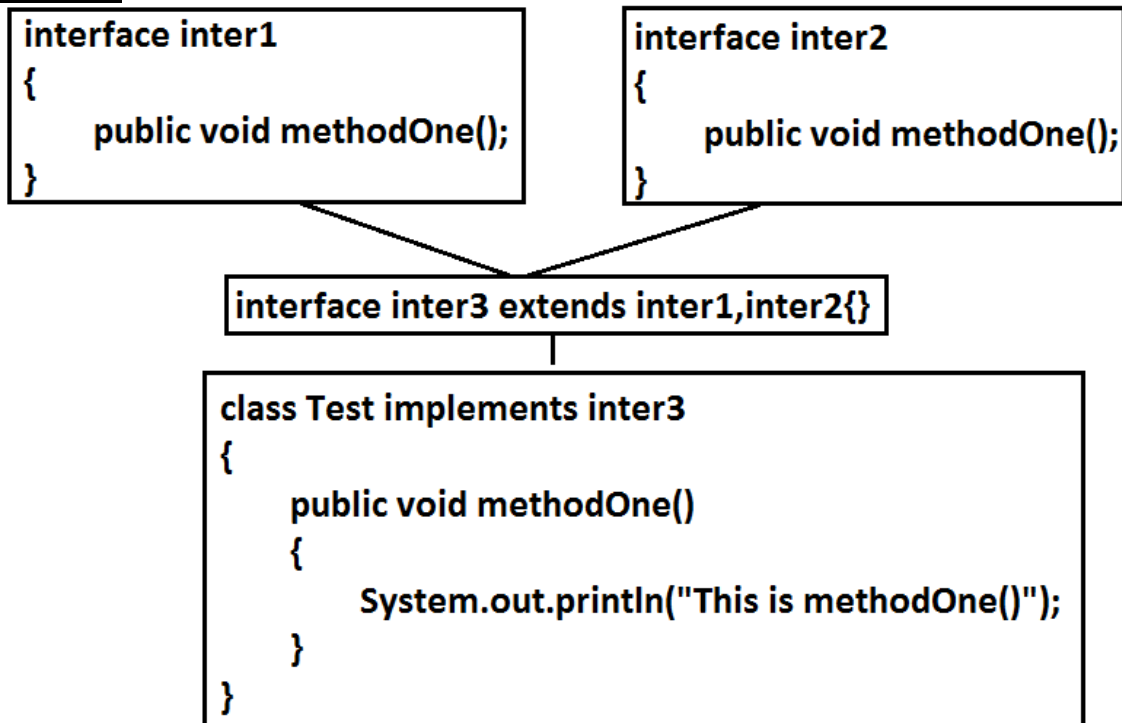
Why java won't provide support for multiple inheritance?

There may be a chance of raising ambiguity problems.

Example:

Why ambiguity problem won't be there in interfaces?

Interfaces having dummy declarations and they won't have implementations hence no ambiguity problem.

Example:

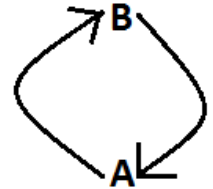


## Cyclic inheritance :

Cyclic inheritance is not allowed in java.

Example 1:

```
class A extends B{} } (invalid)
class B extends A{} } C.E:cyclic inheritance involving A
```



Example 2:

```
class A extends A{}  $\xrightarrow{\text{C.E}}$  cyclic inheritance involving A
```

## HAS-A relationship:

1. HAS-A relationship is also known as composition (or) aggregation.
2. There is no specific keyword to implement HAS-A relationship but mostly we can use new operator.
3. The main advantage of HAS-A relationship is reusability.

Example:

```
class Engine
{
    //engine specific functionality
}
class Car
{
    Engine e=new Engine();
    //.....;
    //.....;
    //.....;
}
```

- class Car HAS-A engine reference.
- The main dis-advantage of HAS-A relationship increases dependency between the components and creates maintains problems.

## Composition vs Aggregation:

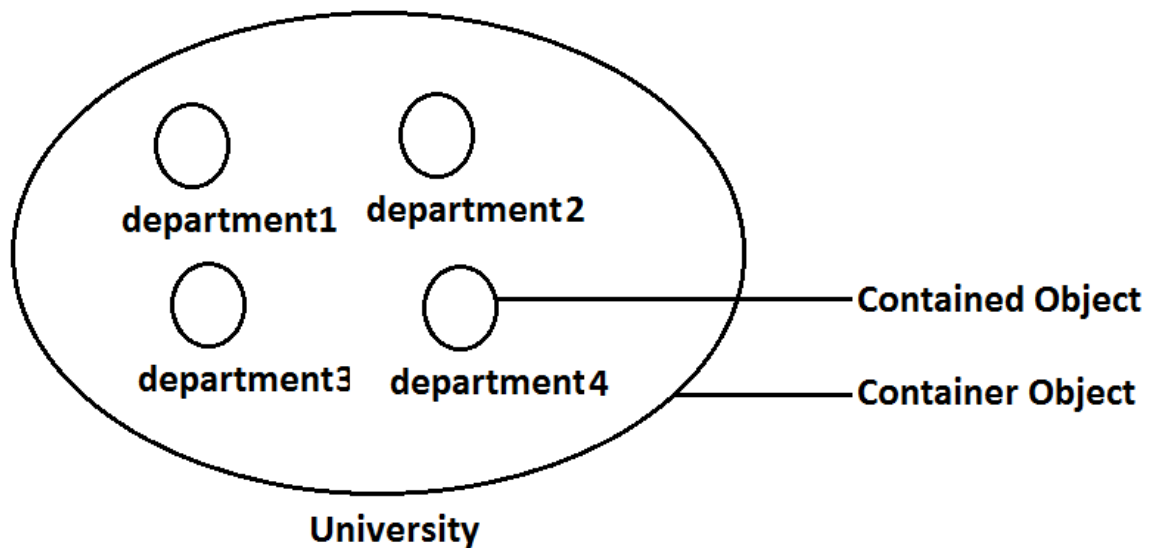
### Composition:

Without existing container object if there is no chance of existing contained objects then the relationship between container object and contained object is called composition which is a strong association.

Example:

University consists of several departments whenever university object destroys automatically all the department objects will be destroyed that is without existing university object there is no chance of existing dependent object hence these are strongly associated and this relationship is called composition.

Example:



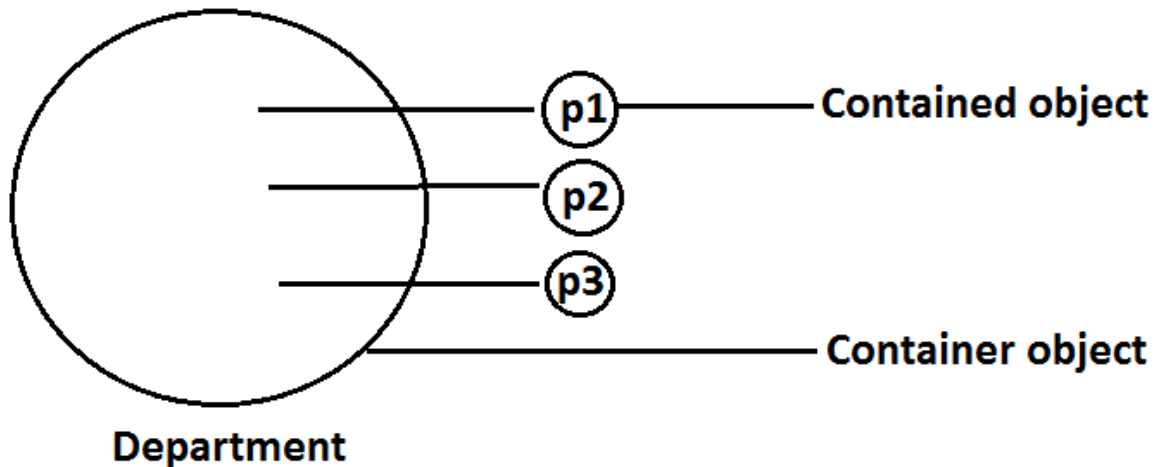
### Aggregation :

Without existing container object if there is a chance of existing contained objects such type of relationship is called aggregation. In aggregation objects have weak association.

Example:

Within a department there may be a chance of several professors will work whenever we are closing department still there may be a chance of existing professor object without existing department object the relationship between department and professor is called aggregation where the objects having weak association.

Example:



Note :

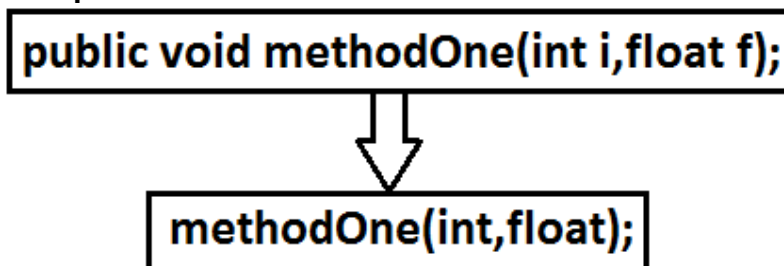
In composition container , contained objects are strongly associated, and but container object holds contained objects directly

But in Aggregation container and contained objects are weakly associated and container object just now holds the reference of contained objects.

### Method signature:

In java, method signature consists of name of the method followed by argument types.

Example:



- In java return type is not part of the method signature.
- Compiler will use method signature while resolving method calls.

```
class Test {
    public void m1(double d) { }
    public void m2(int i) { }
    public static void main(String ar[]) {
        Test t=new Test();
        t.m1(10.5);
        t.m2(10);
        t.m3(10.5); //CE
    }
}
```

```
}  
}  
CE : cannot find symbol  
symbol : method m3(double)  
location : class Test
```

Within the same class we can't take 2 methods with the same signature otherwise we will get compile time error.

Example:

```
public void methodOne() { }  
public int methodOne() {  
    return 10;  
}
```

Output:

Compile time error

methodOne() is already defined in Test

## Polymorphism:

Same name with different forms is the concept of polymorphism.

Example 1: We can use same abs() method for int type, long type, float type etc.

Example:

1. abs(int)
2. abs(long)
3. abs(float)

Example 2:

We can use the parent reference to hold any child objects.

We can use the same List reference to hold ArrayList object, LinkedList object, Vector object, or Stack object.

Example:

1. List l=new ArrayList();
2. List l=new LinkedList();
3. List l=new Vector();
4. List l=new Stack();

Diagram:

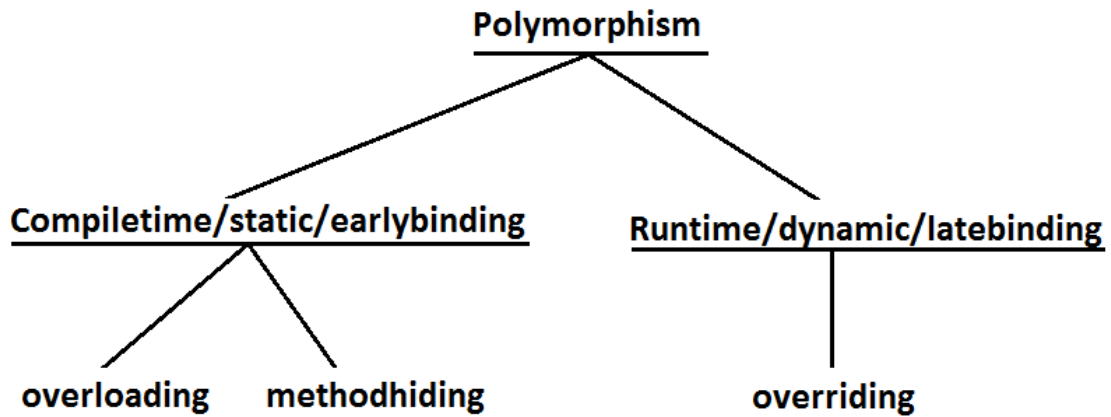
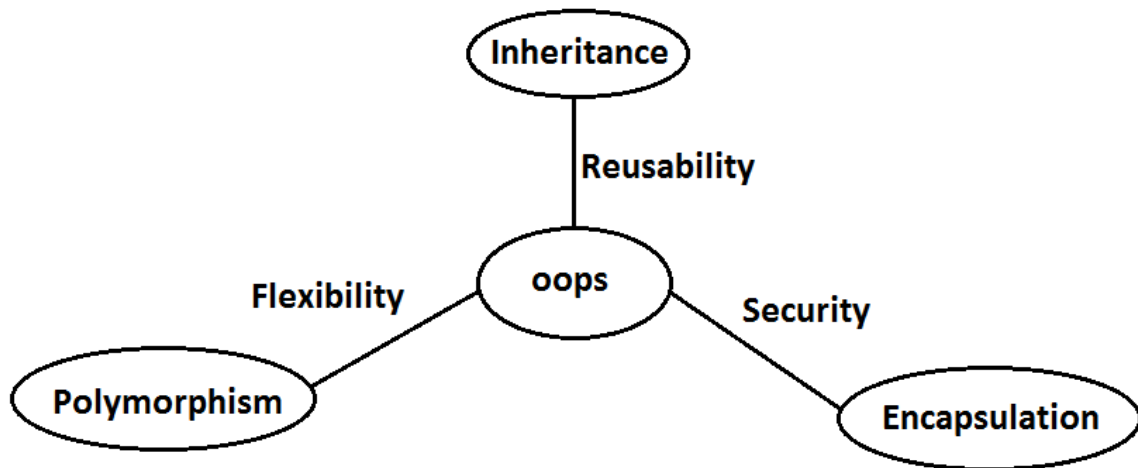


Diagram: 3 Pillars of OOPS



- 1) Inheritance talks about reusability.
- 2) Polymorphism talks about flexibility.
- 3) Encapsulation talks about security.

**Beautiful definition of polymorphism:**

A boy starts love with the word friendship, but girl ends love with the same word friendship, word is the same but with different attitudes. This concept is nothing but polymorphism.

## Overloading :

1. Two methods are said to be overload if and only if both having the same name but different argument types.

2. In 'C' language we can't take 2 methods with the same name and different types. If there is a change in argument type compulsory we should go for new method name.

Example :

**abs()** ————— for int type  
**labs()** ————— for long type  
**fabs()** ————— for float type

.

.

**etc**

3. Lack of overloading in "C" increases complexity of the programming.  
 4. But in java we can take multiple methods with the same name and different argument types.

Example:

**abs(int)**  
**abs(long)**  
**abs(float)**

.

.

.

5. Having the same name and different argument types is called method overloading.  
 6. All these methods are considered as overloaded methods.  
 7. Having overloading concept in java reduces complexity of the programming.

8. Example:

```
9. class Test {
10.     public void methodOne() {
11.         System.out.println("no-arg method");
12.     }
13.     public void methodOne(int i) {
14.         System.out.println("int-arg method"); //overloaded methods
15.     }
16.     public void methodOne(double d) {
17.         System.out.println("double-arg method");
18.     }
19.     public static void main(String[] args) {
20.         Test t=new Test();
21.         t.methodOne();//no-arg method
22.         t.methodOne(10);//int-arg method
23.         t.methodOne(10.5);//double-arg method
24.     }
25. }
```

26. **Conclusion :** In overloading compiler is responsible to perform method resolution(decision) based on the reference type(but not based on run time

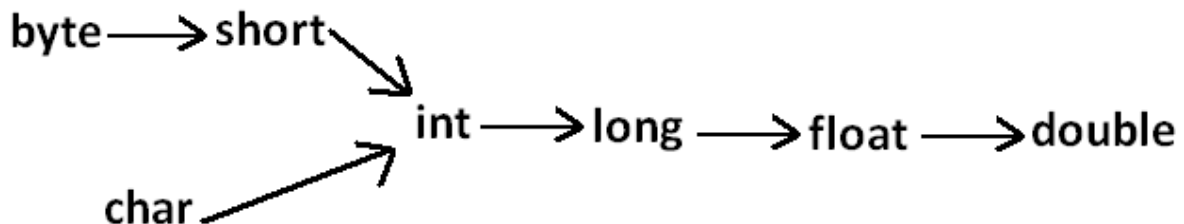
object). Hence overloading is also considered as compile time polymorphism(or) static polymorphism (or)early binding.

### Case 1: Automatic promotion in overloading.

- In overloading if compiler is unable to find the method with exact match we won't get any compile time error immediately.
- 1st compiler promotes the argument to the next level and checks whether the matched method is available or not if it is available then that method will be considered if it is not available then compiler promotes the argument once again to the next level. This process will be continued until all possible promotions still if the matched method is not available then we will get compile time error. This process is called automatic promotion in overloading.

The following are various possible automatic promotions in overloading.

Diagram :



Example:

```

class Test
{
    public void methodOne(int i)
    {
        System.out.println("int-arg method");
    }
    public void methodOne(float f)          //overloaded methods
    {
        System.out.println("float-arg method");
    }
    public static void main(String[] args)
    {
        Test t=new Test();
        //t.methodOne('a');//int-arg method
        //t.methodOne(10l);//float-arg method
        t.methodOne(10.5);//C.E:cannot find symbol
    }
}
  
```

Case 2:

```

class Test
{
    public void methodOne(String s)
    {
  
```

```

        System.out.println("String version");
    }
    public void methodOne(Object o)           //Both methods are said to
                                              //be
overloaded methods.
    {
        System.out.println("Object version");
    }
    public static void main(String[] args)
    {
        Test t=new Test();
        t.methodOne("arun");//String version
        t.methodOne(new Object());//Object version
        t.methodOne(null);//String version
    }
}

```

**Note :**

While resolving overloaded methods exact match will always get high priority,  
While resolving overloaded methods child class will get the more priority than parent class

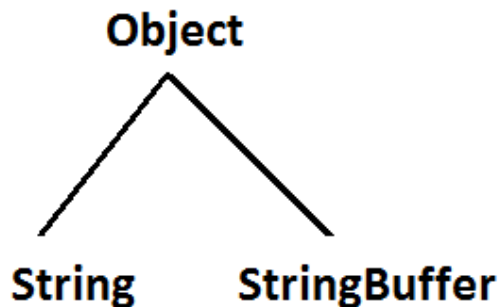
**Case 3:**

```

class Test{
    public void methodOne(String s)      {
        System.out.println("String version");
    }
    public void methodOne(StringBuffer s) {
        System.out.println("StringBuffer version");
    }
    public static void main(String[] args){
        Test t=new Test();
        t.methodOne("arun");//String version
        t.methodOne(new StringBuffer("sai"));//StringBuffer version
        t.methodOne(null);//CE : reference to m1() is ambiguous
    }
}

```

Output:



**Case 4:**

```

class Test {
    public void methodOne(int i,float f) {
        System.out.println("int-float method");
    }
    public void methodOne(float f,int i) {
        System.out.println("float-int method");
    }
    public static void main(String[] args){

```



```

        Test t=new Test();
        t.methodOne(10,10.5f);//int-float method
        t.methodOne(10.5f,10);//float-int method
        t.methodOne(10,10); //C.E:
        //CE:reference to methodOne is ambiguous,
        //both method methodOne(int,float) in Test
        //and method methodOne(float,int) in Test match
        t.methodOne(10.5f,10.5f);//C.E:
        cannot find symbol
        symbol : methodOne(float, float)
        location : class Test
    }
}

```

**Case 5 :**

```

class Test{
    public void methodOne(int i) {
        System.out.println("general method");
    }
    public void methodOne(int...i) {
        System.out.println("var-arg method");
    }
    public static void main(String[] args){
        Test t=new Test();
        t.methodOne();//var-arg method
        t.methodOne(10,20);//var-arg method
        t.methodOne(10);//general method
    }
}

```

In general var-arg method will get less priority that is if no other method matched then only var-arg method will get chance for execution it is almost same as default case inside switch.

**Case 6:**

```

class Animal{ }
class Monkey extends Animal{}
class Test{
    public void methodOne(Animal a) {
        System.out.println("Animal version");
    }
    public void methodOne(Monkey m) {
        System.out.println("Monkey version");
    }
    public static void main(String[] args){
        Test t=new Test();
        Animal a=new Animal();
        t.methodOne(a);//Animal version
        Monkey m=new Monkey();
        t.methodOne(m);//Monkey version
        Animal a1=new Monkey();
        t.methodOne(a1);//Animal version
    }
}

```

In overloading method resolution is always based on reference type and runtime object won't play any role in overloading.

## Overriding :

1. Whatever the Parent has by default available to the Child through inheritance, if the Child is not satisfied with Parent class method implementation then Child is allow to redefine that Parent class method in Child class in its own way this process is called overriding.
2. The Parent class method which is overridden is called overridden method.
3. The Child class method which is overriding is called overriding method.

### 4. Example 1:

```

5.
6. class Parent {
7.     public void property(){
8.         System.out.println("cash+land+gold");
9.     }
10.    public void marry()    {
11.        System.out.println("subbalakshmi"); //overridden
12.    }
13. }
14. class Child extends Parent{                //overriding
15.     public void marry()    {
16.         System.out.println("3sha/4me/9tara/anushka");
17.     } //overriding method
18. }
19. class Test {
20.     public static void main(String[] args){
21.         Parent p=new Parent();
22.         p.marry();//subbalakshmi(parent method)
23.         Child c=new Child();
24.         c.marry();//Trisha/nayanatara/anushka(child method)
25.         Parent p1=new Child();
26.         p1.marry();//Trisha/nayanatara/anushka(child method)
27.     }
28. }

```

29. In overriding method resolution is always takes care by JVM based on runtime object hence overriding is also considered as runtime polymorphism or dynamic polymorphism or late binding.
30. The process of overriding method resolution is also known as dynamic method dispatch.

**Note:** In overriding runtime object will play the role and reference type is dummy.

### Rules for overriding :

1. In overriding method names and arguments must be same. That is method signature must be same.
2. Until 1.4 version the return types must be same but from 1.5 version onwards co-variant return types are allowed.
3. According to this Child class method return type need not be same as Parent class method return type its Child types also allowed.
4. Example:
5. class Parent {

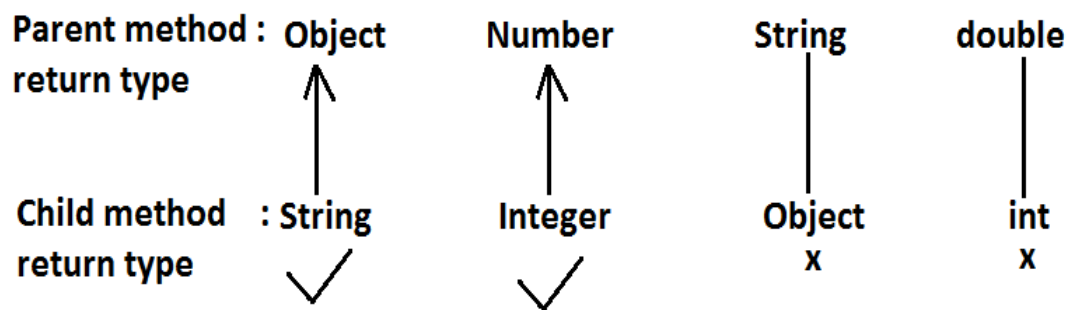
```

6.  public Object methodOne()    {
7.      return null;
8.  }
9.  }
10. class Child extends Parent {
11.     public String methodOne() {
12.         return null;
13.     }
14. }
15.
16. C:> javac -source 1.4 Parent.java //error

```

It is valid in "1.5" but invalid in "1.4".

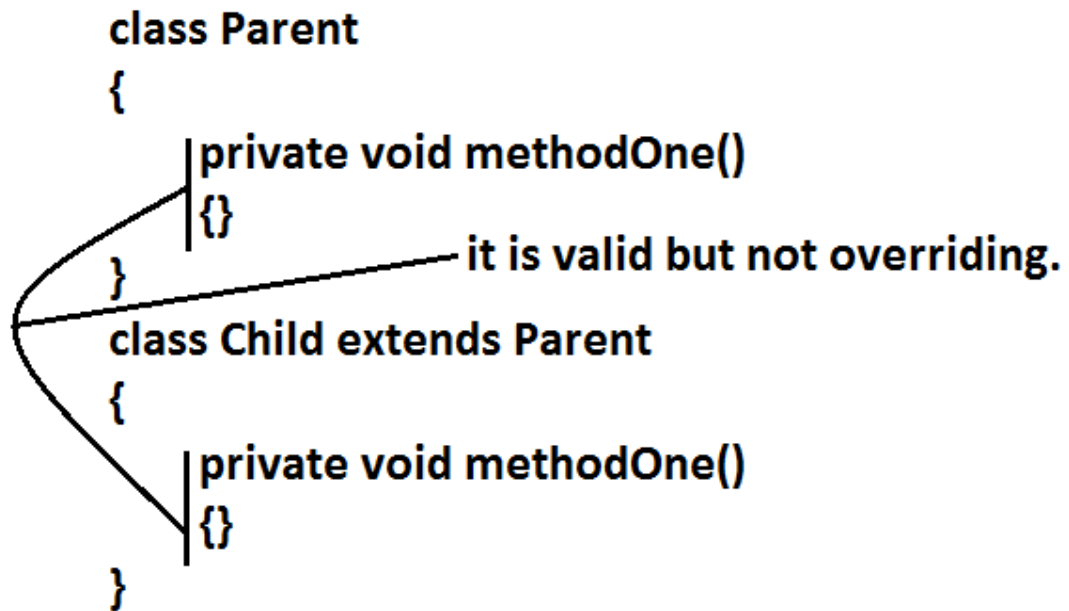
**Diagram:**



Co-variant return type concept is applicable only for object types but not for primitives.

17. Private methods are not visible in the Child classes hence overriding concept is not applicable for private methods. Based on own requirement we can declare the same Parent class private method in child class also. It is valid but not overriding.

Example:



Parent class final methods we can't override in the Child class.

```

18. Example:
19. class Parent {
20.     public final void methodOne() {}
21. }
22. class Child extends Parent{
23.     public void methodOne(){ }
24. }
25. Output:
26. Compile time error:
27. methodOne() in Child cannot override methodOne()
28.     in Parent; overridden method is final

```

Parent class non final methods we can override as final in child class. We can override native methods in the child classes.

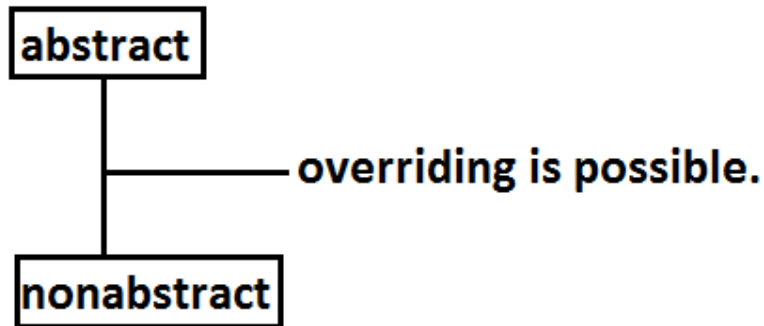
29. We should override Parent class abstract methods in Child classes to provide implementation.

```

30. Example:
31. abstract class Parent {
32.     public abstract void methodOne();
33. }
34. class Child extends Parent {
35.     public void methodOne() { }
36. }

```

Diagram:



37. We can override a non-abstract method as abstract  
this approach is helpful to stop availability of Parent method implementation to the next level child classes.

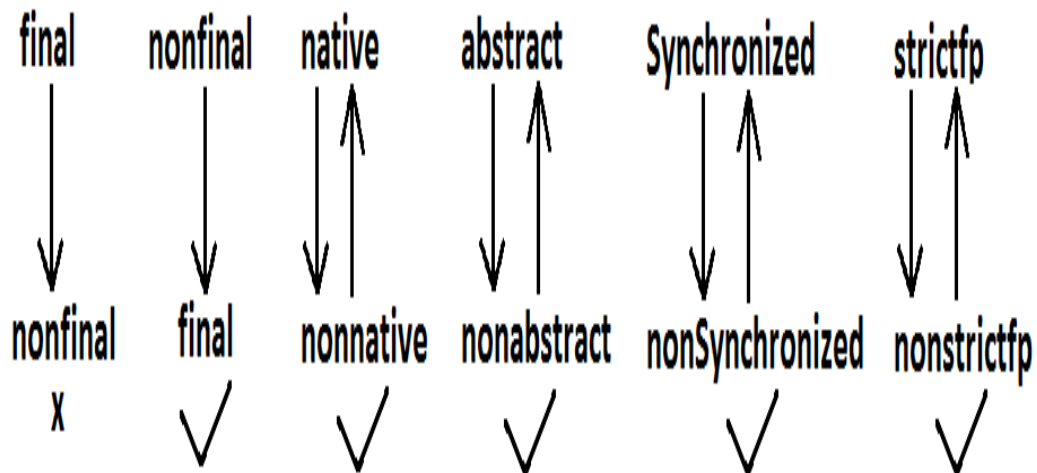
38. Example:

```

39. class Parent {
40.     public void methodOne() { }
41. }
42. abstract class Child extends Parent {
43.     public abstract void methodOne();
44. }
  
```

Synchronized, strictfp, modifiers won't keep any restrictions on overriding.

Diagram:



45. While overriding we can't reduce the scope of access modifier.

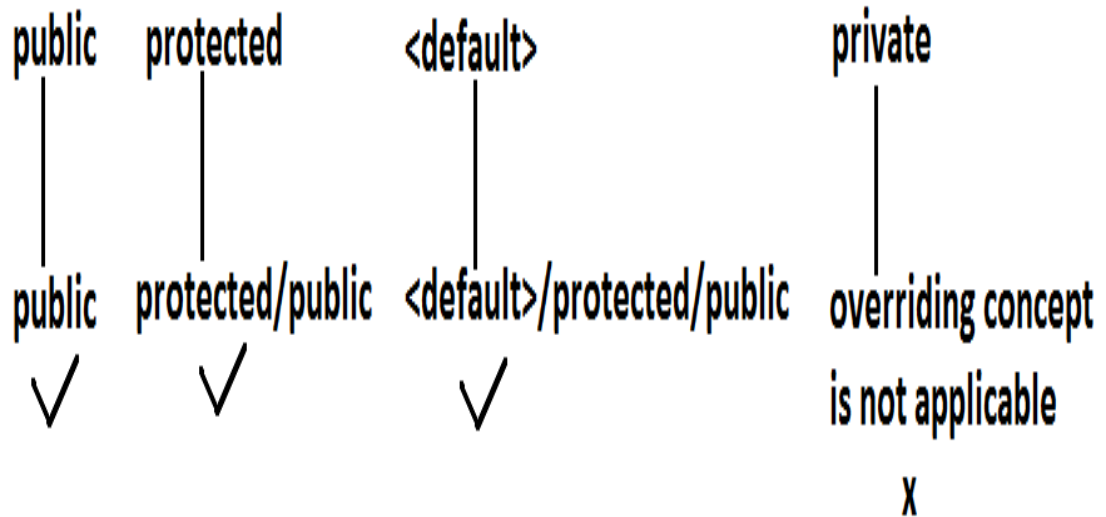
46. Example:

```

47. class Parent {
48.     public void methodOne() { }
49. }
50. class Child extends Parent {
51.     protected void methodOne( ) { }
52. }
  
```

53. Output:  
 54. Compile time error :  
 55. methodOne() in Child cannot override methodOne() in Parent;  
 56. attempting to assign weaker access privileges; was public

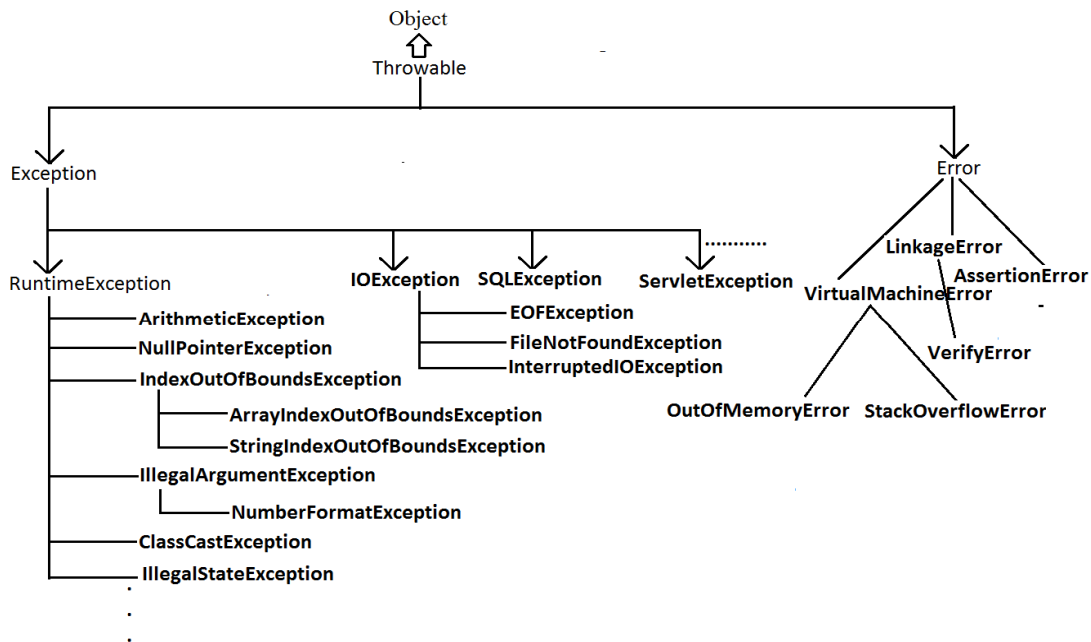
Diagram:



**private < default < protected < public**

## Checked Vs Un-checked Exceptions :

- The exceptions which are checked by the compiler for smooth execution of the program at runtime are called checked exceptions.
- The exceptions which are not checked by the compiler are called un-checked exceptions.
- RuntimeException and its child classes, Error and its child classes are unchecked except these the remaining are checked exceptions.

**Diagram:**

**Rule:** While overriding if the child class method throws any checked exception compulsory the parent class method should throw the same checked exception or its parent otherwise we will get compile time error.

But there are no restrictions for un-checked exceptions.

Example:

```

class Parent {
    public void methodOne() {}
}
class Child extends Parent{
    public void methodOne()throws Exception {}
}
  
```

Output:

Compile time error :

methodOne() in Child cannot override methodOne() in Parent;  
overridden method does not throw java.lang.Exception

Examples :

- ① Parent: public void methodOne()throws Exception } valid  
Child: public void methodOne()
- ② Parent: public void methodOne() } invalid  
Child : public void methodOne()throws Exception
- ③ Parent: public void methodOne()throws Exception } valid  
Child: public void methodOne()throws Exception
- ④ Parent: public void methodOne()throws IOException } invalid  
Child: public void methodOne()throws Exception
- ⑤ Parent: public void methodOne()throws IOException } valid  
Child: public void methodOne()throws EOFException,FileNotFoundException
- ⑥ Parent: public void methodOne()throws IOException } invalid  
Child : public void methodOne()throws EOFException,InterruptedException
- ⑦ Parent: public void methodOne()throws IOException } valid  
Child: public void methodOne()throws EOFException,ArithmeticException
- ⑧ Parent: public void methodOne()  
Child: public void methodOne()throws  
ArithmeticException,NullPointerException,ClassCastException,RuntimeException } valid

## Overriding with respect to static methods:

### Case 1:

We can't override a static method as non static.

Example:

```
class Parent
{
    public static void methodOne(){}
    //here static methodOne() method is a class level
}
class Child extends Parent
{
    public void methodOne(){}
    //here methodOne() method is a object level hence
    // we can't override methodOne() method
}
```

output :

CE: methodOne in Child can't override methodOne() in Parent ;  
overriden method is static

### Case 2:



Similarly we can't override a non static method as static.

### Case 3:

```
class Parent
{
    public static void methodOne() {}
}
class Child extends Parent {
    public static void methodOne() {}
}
```

It is valid. It seems to be overriding concept is applicable for static methods but it is not overriding it is method hiding.

## METHOD HIDING :

All rules of method hiding are exactly same as overriding except the following differences.

| Overriding                                                                                            | Method hiding                                                                                                 |
|-------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| 1. Both Parent and Child class methods should be non static.                                          | 1. Both Parent and Child class methods should be static.                                                      |
| 2. Method resolution is always takes care by JVM based on runtime object.                             | 2. Method resolution is always takes care by compiler based on reference type.                                |
| 3. Overriding is also considered as runtime polymorphism (or) dynamic polymorphism (or) late binding. | 3. Method hiding is also considered as compile time polymorphism (or) static polymorphism (or) early bidding. |

Example:

```
class Parent {
    public static void methodOne() {
        System.out.println("parent class");
    }
}
class Child extends Parent{
    public static void methodOne(){
        System.out.println("child class");
    }
}
class Test{
    public static void main(String[] args)    {
        Parent p=new Parent();
        p.methodOne();//parent class
        Child c=new Child();
        c.methodOne();//child class
        Parent p1=new Child();
        p1.methodOne();//parent class
    }
}
```

**Note:** If both Parent and Child class methods are non static then it will become overriding and method resolution is based on runtime object. In this case the output is

Parent class  
Child class  
Child class

## Overriding with respect to Var-arg methods:

A var-arg method should be overridden with var-arg method only. If we are trying to override with normal method then it will become overloading but not overriding.

Example:

```
class Parent {
    public void methodOne(int... i){
        System.out.println("parent class");
    }
}
class Child extends Parent {    //overloading but not overriding.
    public void methodOne(int i) {
        System.out.println("child class");
    }
}
class Test {
    public static void main(String[] args) {
        Parent p=new Parent();
        p.methodOne(10);//parent class
        Child c=new Child();
        c.methodOne(10);//child class
        Parent p1=new Child();
        p1.methodOne(10);//parent class
    }
}
```

In the above program if we replace child class method with var arg then it will become overriding. In this case the output is

Parent class  
Child class  
Child class

## Overriding with respect to variables:

- Overriding concept is not applicable for variables.
- Variable resolution is always takes care by compiler based on reference type.

Example:

```
class Parent
{
    int x=888;
}
class Child extends Parent
{
    int x=999;
}
class Test
{
    public static void main(String[] args)
```

```

    {
        Parent p=new Parent();
        System.out.println(p.x);//888
        Child c=new Child();
        System.out.println(c.x);//999
        Parent p1=new Child();
        System.out.println(p1.x);//888
    }
}

```

**Note:** In the above program Parent and Child class variables, whether both are static or non static whether one is static and the other one is non static there is no change in the answer.

### Differences between overloading and overriding ?

| Property                          | Overloading                                                | Overriding                                                                                                                                                                              |
|-----------------------------------|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1) Method names                   | Must be same.                                              | Must be same.                                                                                                                                                                           |
| 2) Argument type                  | Must be different(at least order)                          | Must be same including order.                                                                                                                                                           |
| 3) Method signature               | Must be different.                                         | Must be same.                                                                                                                                                                           |
| 4) Return types                   | No restrictions.                                           | Must be same until 1.4v but from 1.5v onwards we can take co-variant return types also.                                                                                                 |
| 5) private, static, final methods | Can be overloaded.                                         | Can not be overridden.                                                                                                                                                                  |
| 6) Access modifiers               | No restrictions.                                           | Weakening/reducing is not allowed.                                                                                                                                                      |
| 7) Throws clause                  | No restrictions.                                           | If child class method throws any checked exception compulsory parent class method should throw the same checked exceptions or its parent but no restrictions for un-checked exceptions. |
| 8) Method resolution              | Is always takes care by compiler based on referenced type. | Is always takes care by JVM based on runtime object.                                                                                                                                    |
| 9) Also known as                  | Compile time polymorphism (or) static(or)early binding.    | Runtime polymorphism (or) dynamic (or) late binding.                                                                                                                                    |

**Note:**

1. In overloading we have to check only method names (must be same) and arguments (must be different) the remaining things like return type extra not required to check.

2. But In overriding we should compulsory check everything like method names, arguments, return types, throws keyword, modifiers etc.

Consider the method in parent class

Parent: public void methodOne(int i) throws IOException

In the child class which of the following methods we can take..

1. public void methodOne(int i) //valid(overriding)
2. private void methodOne() throws Exception //valid(overloading)
3. public native void methodOne(int i); //valid(overriding)
4. public static void methodOne(double d) //valid(overloading)
5. public static void methodOne(int i)  
Compile time error :

methodOne(int) in Child cannot override methodOne(int) in Parent; overriding method is static

6. public static abstract void methodOne(float f)  
Compile time error :
  1. illegal combination of modifiers: abstract and static
  2. Child is not abstract and does not override abstract method methodOne(float) in Child

What is the difference between ArrayList l=new ArrayList() & List l=new ArrayList() ?

|                                                                           |                                                                                                                      |
|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| ArrayList al=new ArrayList();<br>[Child c=new Child();]                   | List l=new ArrayList();<br>[Parent p=new Child();]                                                                   |
| If we know runtime object type exactly then we have to used this approach | If we don't know exact Runtime object type then we have to used this approach                                        |
| By using child reference we can call both parent & child calss methods.   | By using parent reference we can call only method available in parent class and child specific method we can't call. |

- We can use ArrayList reference to hold ArrayList object where as we can use List reference to hold any list implemented class object (ArrayList, LinkedList, Vector, Stack)
- By using ArrayList reference we can call both List and ArrayList methods but by using List reference we can call only List interface specific methods and we can't call ArrayList specific methods.

## IIQ : In how many ways we can create an object ? (or) In how many ways get an object in java ?

1. By using new Operator :
2. `Test t = new Test();`
3. By using newInstance() :(Reflection Mechanism)
4. `Test t=(Test)Class.forName("Test").newInstance();`
5. By using Clone() :
6. `Test t1 = new Test();`
7. `Test t2 = (Test)t1.Clone();`
8. By using Factory methods :
9. `Runtime r = Runtime.getRuntime();`
10. `DateFormat df = DateFormat.getInstance();`
11. By using Deserialization :
12. `FileInputStream fis = new FileInputStream("abc.ser");`
13. `ObjectInputStream ois = new ObjectInputStream(fis);`
14. `Test t = (Test)ois.readObject();`

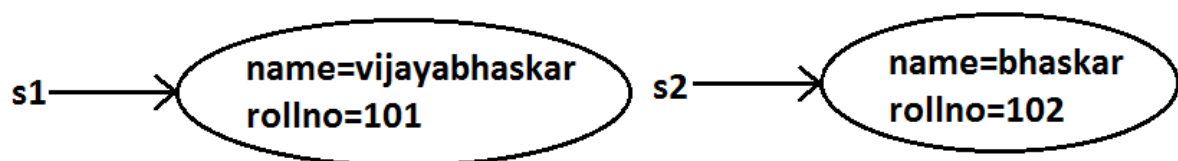
## Constructors :

1. Object creation is not enough compulsory we should perform initialization then only the object is in a position to provide the response properly.
2. Whenever we are creating an object some piece of the code will be executed automatically to perform initialization of an object this piece of the code is nothing but constructor.
3. Hence the main objective of constructor is to perform initialization of an object.

Example:

```
class Student
{
    String name;
    int rollno;
    Student(String name,int rollno) //Constructor
    {
        this.name=name;
        this.rollno=rollno;
    }
    public static void main(String[] args)
    {
        Student s1=new Student("vijayabhaskar",101);
        Student s2=new Student("bhaskar",102);
    }
}
```

Diagram:



### Constructor Vs instance block:

1. Both instance block and constructor will be executed automatically for every object creation but instance block 1st followed by constructor.
2. The main objective of constructor is to perform initialization of an object.
3. Other than initialization if we want to perform any activity for every object creation we have to define that activity inside instance block.
4. Both concepts having different purposes hence replacing one concept with another concept is not possible.
5. Constructor can take arguments but instance block can't take any arguments hence we can't replace constructor concept with instance block.
6. Similarly we can't replace instance block purpose with constructor.

Demo program to track no of objects created for a class:

```
class Test
{
    static int count=0;
    {
        count++;      //instance block
    }
    Test()
    {}
    Test(int i)
    {}
    public static void main(String[] args)
    {
        Test t1=new Test();
        Test t2=new Test(10);
        Test t3=new Test();
        System.out.println(count);//3
    }
}
```

### Rules to write constructors:

1. Name of the constructor and name of the class must be same.
2. Return type concept is not applicable for constructor even void also by mistake if we are declaring the return type for the constructor we won't get any compile time error and runtime error compiler simply treats it as a method.
3. Example:
4. class Test
5. {
6. void Test() //it is not a constructor and it is a method
7. {}
8. }
9. It is legal (but stupid) to have a method whose name is exactly same as class name.
10. The only applicable modifiers for the constructors are public, default, private, protected.
11. If we are using any other modifier we will get compile time error.

Example:

```
class Test
{
    static Test()
    {}
}
```

Output:

Modifier static not allowed here

### Default constructor:

1. For every class in java including abstract classes also constructor concept is applicable.
2. If we are not writing at least one constructor then compiler will generate default constructor.
3. If we are writing at least one constructor then compiler won't generate any default constructor. Hence every class contains either compiler generated constructor (or) programmer written constructor but not both simultaneously.

### Prototype of default constructor:

1. It is always no argument constructor.
2. The access modifier of the default constructor is same as class modifier. (This rule is applicable only for public and default).
3. Default constructor contains only one line. `super();` it is a no argument call to super class constructor.

| Programmers code                            | Compiler generated code                                                         |
|---------------------------------------------|---------------------------------------------------------------------------------|
| <code>class Test { }</code>                 | <pre>class Test {     Test()     {         super();     } }</pre>               |
| <code>public class Test { }</code>          | <pre>public class Test {     public Test()     {         super();     } }</pre> |
| <pre>class Test {     void Test(){} }</pre> | <pre>class Test {     Test()     {         super();     } }</pre>               |

|                                                                                           |                                                                                                                 |
|-------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
|                                                                                           | <pre>     }     void Test()     {} } </pre>                                                                     |
| <pre> class Test {     Test(int i)     {} } </pre>                                        | <pre> class Test {     Test(int i)     {         super();     } } </pre>                                        |
| <pre> class Test {     Test()     {         super();     } } </pre>                       | <pre> class Test {     Test()     {         super();     } } </pre>                                             |
| <pre> class Test {     Test(int i)     {         this();     }     Test()     {} } </pre> | <pre> class Test {     Test(int i)     {         this();     }     Test()     {         super();     } } </pre> |

### super() vs this():

The 1st line inside every constructor should be either super() or this() if we are not writing anything compiler will always generate super().

**Case 1:** We have to take super() (or) this() only in the 1st line of constructor. If we are taking anywhere else we will get compile time error.

Example:

```

class Test
{
    Test()
    {
        System.out.println("constructor");
        super();
    }
}

```

Output:

Compile time error.

Call to super must be first statement in constructor

**Case 2:** We can use either super() (or) this() but not both simultaneously.

Example:

```

class Test
{
    Test()
    {
        super();
    }
}

```



```

        this();
    }
}

```

Output:

Compile time error.

Call to this must be first statement in constructor

**Case 3:** We can use super() (or) this() only inside constructor. If we are using anywhere else we will get compile time error.

Example:

```

class Test
{
    public void methodOne()
    {
        super();
    }
}

```

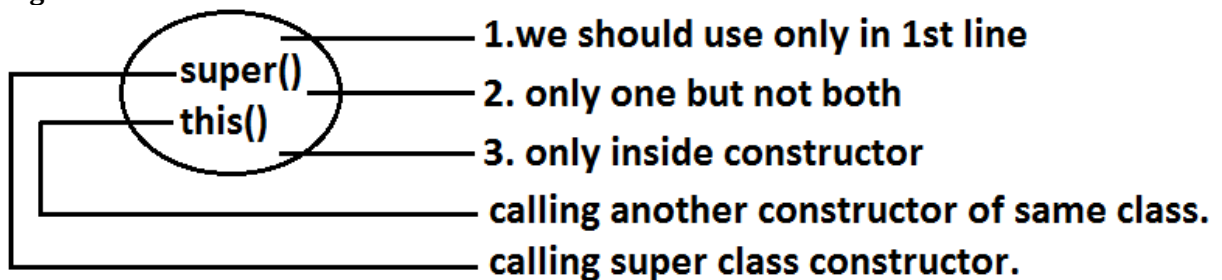
Output:

Compile time error.

Call to super must be first statement in constructor

That is we can call a constructor directly from another constructor only.

Diagram:



Example:

|                                                                                                                              |                                                                                                            |
|------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| super(), this()                                                                                                              | super, this                                                                                                |
| These are constructors calls.                                                                                                | These are keywords                                                                                         |
| We can use these to invoke super class & current constructors directly                                                       | We can use refers parent class and current class instance members.                                         |
| We should use only inside constructors as first line, if we are using outside of constructor we will get compile time error. | We can use anywhere (i.e., instance area) except static area , other wise we will get compile time error . |

Example:

```

class Test
{
    public static void main(String[] args)
    {
        System.out.println(super.hashCode());
    }
}

```

Output:

Compile time error.

Non-static variable super cannot be referenced from a static context.

### Overloaded constructors :

A class can contain more than one constructor and all these constructors having the same name but different arguments and hence these constructors are considered as overloaded constructors.

Example:

```
class Test {
    Test(double d){
        System.out.println("double-argument constructor");
    }
    Test(int i) {
        this(10.5);
        System.out.println("int-argument constructor");
    }
    Test() {
        this(10);
        System.out.println("no-argument constructor");
    }
    public static void main(String[] args) {
        Test t1=new Test(); //no-argument constructor/int-argument
                           //constructor/double-argument constructor
        Test t2=new Test(10);
                           //int-argument constructor/double-argument constructor
        Test t3=new Test(10.5); //double-argument constructor
    }
}
```

- Parent class constructor by default won't available to the Child. Hence Inheritance concept is not applicable for constructors and hence overriding concept also not applicable to the constructors. But constructors can be overloaded.
- We can take constructor in any java class including abstract class also but we can't take constructor inside interface.

Example:

| class Test                       | abstract class Test              | interface Test1                   |
|----------------------------------|----------------------------------|-----------------------------------|
| <pre>{     Test()     {} }</pre> | <pre>{     Test()     {} }</pre> | <pre>{     Test1()     {} }</pre> |
| valid                            | valid                            | invalid                           |

We can't create object for abstract class but abstract class can contain constructor what is the need ?

Abstract class constructor will be executed for every child class object creation to perform initialization of child class object only.

Which of the following statement is true ?

1. Whenever we are creating child class object then automatically parent class object will be created.(false)
2. Whenever we are creating child class object then parent class constructor will be executed.(true)

Example:

```
abstract class Parent
{
    Parent()
    {
        System.out.println(this.hashCode());
        //11394033//here this means child class object
    }
}
class Child extends Parent
{
    Child()
    {
        System.out.println(this.hashCode()); //11394033
    }
}
class Test
{
    public static void main(String[] args)
    {
        Child c=new Child();
        System.out.println(c.hashCode()); //11394033
    }
}
```

**Case 1:** recursive method call is always runtime exception where as recursive constructor invocation is a compile time error.

Note:

### Recursive functions:

A function is called using two methods (types).

1. Nested call
2. Recursive call

**Nested call:**

- Calling a function inside another function is called nested call.
- In nested call there is a calling function which calls another function(called function).

Example:

```
public static void methodOne()
{
    methodTwo();
}
public static void methodTwo()
{
    methodOne();
}
```

**Recursive call:**

- Calling a function within same function is called recursive call.
- In recursive call called and calling function is same.

Example:

```
public void methodOne()
{
    methodOne();
}
```

Example:

```
class Test
{
    public static void methodOne()
    {
        methodTwo();
    }
    public static void methodTwo()
    {
        methodOne();
    }
    public static void main(String[] args)
    {
        methodOne();
        System.out.println("hello");
    }
}
```

R.E:StackOverflowError

```
class Test
{
    Test(int i)
    {
        this();
    }
    Test()
    {
        this(10);
    }
    public static void main(String[] args)
    {
        System.out.println("hello");
    }
}
```

C.E:recursive constructor invocation

**Note: Compiler is responsible for the following checkings.**

1. Compiler will check whether the programmer wrote any constructor or not. If he didn't write at least one constructor then compiler will generate default constructor.
2. If the programmer wrote any constructor then compiler will check whether he wrote `super()` or `this()` in the 1st line or not. If his not writing any of these compiler will always write (generate) `super()`.
3. Compiler will check is there any chance of recursive constructor invocation. If there is a possibility then compiler will raise compile time error.

**Case 2:**

|                                                                    |                                                                         |                                                                                                                                                  |                                                                                                                                            |
|--------------------------------------------------------------------|-------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>class Parent { } class Child extends Parent {     valid</pre> | <pre>class Parent {     Parent() } class Child extends Parent { }</pre> | <pre>class Parent {     Parent(int i) } class Child extends Parent {     Child()     {         super();     } } output: compile time error</pre> | <pre>E:\scjp&gt;javac Child.java Child.java:10: cannot find symbol symbol : constructor Parent() location: class Parent     super();</pre> |
|--------------------------------------------------------------------|-------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|

- If the Parent class contains any argument constructors while writing Child classes we should takes special care with respect to constructors.
- Whenever we are writing any argument constructor it is highly recommended to write no argument constructor also.

**Case 3:**

```
class Parent
{
    Parent()throws java.io.IOException
    {}
}
class Child extends Parent
{
}
Output:
Compile time error
Unreported exception java.io.IOException in default constructor.
```

Example:

```
class Parent
{
    Parent()throws java.io.IOException
    {}
}
class Child extends Parent
{
    Child()throws Exception
    {
        super();
    }
}
```

If Parent class constructor throws some checked exception compulsory Child class constructor should throw the same checked exception (or) its Parent.

### Singleton classes :

For any java class if we are allow to create only one object such type of class is said to be singleton class.

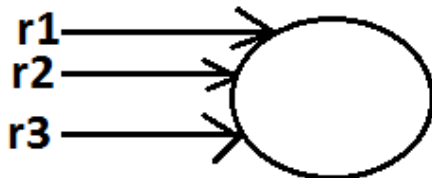
Example:

- 1) Runtime class
- 2) ActionServlet
- 3) ServiceLocator
- 4) BusinessDelegate

```
Runtime r1=Runtime.getRuntime();
    //getRuntime() method is a factory method
Runtime r2=Runtime.getRuntime();
Runtime r3=Runtime.getRuntime();
.....
.....
```

```
System.out.println(r1==r2);//true
System.out.println(r1==r3);//true
```

Diagram:



Advantage of Singleton class :

If the requirement is same then instead of creating a separate object for every person we will create only one object and we can share that object for every required person we can achieve this by using singleton classes. That is the main advantages of singleton classes are Performance will be improved and memory utilization will be improved.

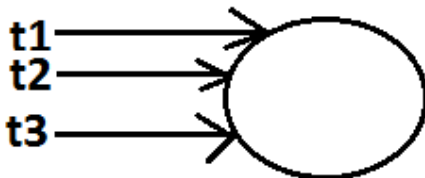
Creation of our own singleton classes:

We can create our own singleton classes for this we have to use private constructor, static variable and factory method.

Example:

```
class Test
{
    private static Test t=null;
    private Test()
    {}
    public static Test getTest()
        //getTest() method is a factory method
    {
        if(t==null)
        {
            t=new Test();
        }
        return t;
    }
}
class Client
{
    public static void main(String[] args)
    {
        System.out.println(Test.getTest().hashCode()); //1671711
        System.out.println(Test.getTest().hashCode()); //1671711
        System.out.println(Test.getTest().hashCode()); //1671711
        System.out.println(Test.getTest().hashCode()); //1671711
    }
}
```

Diagram:



Note:

We can create any xxxton classes like(double ton,tribe ton...etc)

Example:

```
class Test
{
    private static Test t1=null;
    private static Test t2=null;
    private Test()
    {}
    public static Test getTest()
        //getTest() method is a factory method
    {
        if(t1==null)
        {
            t1=new Test();
            return t1;
        }
    }
}
```

```

        else if(t2==null)
        {
            t2=new Test();
            return t2;
        }
        else
        {
            if(Math.random()<0.5) //Math.random() limit : 0<=x<1
                return t1;
            else
                return t2;
        }
    }
}
class Client
{
    public static void main(String[] args)
    {
        System.out.println(Test.getTest().hashCode()); //1671711
        System.out.println(Test.getTest().hashCode()); //11394033
        System.out.println(Test.getTest().hashCode()); //11394033
        System.out.println(Test.getTest().hashCode()); //1671711
    }
}

```

**IIQ : We are not allowed to create child class but class is not final , How it is Possible ?**

**By declaring every constructor has private.**

```

class Parent {
    private Parent() {
    }
}

```

**We can't create child class for this class**

**Note : When ever we are creating child class object automatically parent class constructor will be executed but parent object won't be created.**

```

class Parent {
    Parent() {
        System.out.println(this.hashCode()); //123
    }
}
class Child extends Parent {
    Child() {
        System.out.println(this.hashCode()); //123
    }
}
class Test {
    public static void main(String ar[]) {
        Child c=new Child();
        System.out.println(c.hashCode()); //123
    }
}

```

**Which of the following is true ?**

1. The name of the constructor and name of the class need not be same.(false)
2. We can declare return type for the constructor but it should be void. (false)
3. We can use any modifier for the constructor. (false)
4. Compiler will always generate default constructor. (false)



5. The modifier of the default constructor is always default. (false)
6. The 1st line inside every constructor should be super always. (false)
7. The 1st line inside every constructor should be either super or this and if we are not writing anything compiler will always place this().(false)
8. Overloading concept is not applicable for constructor. (false)
9. Inheritance and overriding concepts are applicable for constructors. (false)
10. Concrete class can contain constructor but abstract class cannot. (false)
11. Interface can contain constructor. (false)
12. Recursive constructor call is always runtime exception. (false)
13. If Parent class constructor throws some un-checked exception compulsory Child class constructor should throw the same un-checked exception or it's Parent. (false)
14. Without using private constructor we can create singleton class. (false)
15. None of the above.(true)

### **Factory method:**

By using class name if we are calling a method and that method returns the same class object such type of method is called factory method.

**Example:**

```
Runtime r=Runtime.getRuntime();//getRuntime is a factory method.  
DateFormat df=DateFormat.getInstance();
```

If object creation required under some constraints then we can implement by using factory method.

## Static control flow :

### Example:

```

class Base
{
    ①static int i=10; ⑦
    ②static
    {
        methodOne(); ⑧
        System.out.println("first static block"); ⑩
    }
    ③public static void main(String[] args)
    {
        methodOne(); ⑬
        System.out.println("main method"); ⑮
    }
    ④public static void methodOne()
    {
        System.out.println(j); ⑨, ⑭
    }
    ⑤static
    {
        System.out.println("second static block"); ⑪
    }
    ⑥static int j=20; ⑫
}
  
```

### Analysis:

i=0[R/W]  
 j=0[R/W]  
 i=10[R&W]  
 j=20[R&W]

- 1.identification of static members from top to bottom[1 to 6]
- 2.execution of static variable assignments and static blocks from top to bottom[7 to 12]
- 3.execution of main method[13 to 15]

Output:

E:\scjp>javac Base.java

E:\scjp>java Base

0

First static block

Second static block

20

Main method

### Read indirectly write only state (or) RIWO :

With in the static block if we are trying to read any variable then that read is considered as "direct read"

If we are calling a method , and with in the method if we are trying to read a method , that read is called Indirect read

If a variable is in RIWO state then we can't perform read operation directly otherwise we will get compile time error saying " illegal forward reference ".

### Example:

```
class Test
{
    static int i=10;
    static
    {
        System.out.println(i);//10
        System.exit(0);
    }
}
```

```
class Test
{
    static
    {
        System.out.println(i);
    }
    static int i=10;
}
```

output:  
compile time error:  
illegal forward reference

```
class Test
{
    static
    {
        methodOne();
    }
    public static void methodOne()
    {
        System.out.println(i);
    }
    static int i=10;
}
```

output:  
Runtime exception:  
0  
NoSuchMethodError: main

## Static control flow parent to child relationship :

```

class Base
{
    ① static int i=10; ⑫
    ② static
    {
        methodOne(); ⑬
        System.out.println("base static block"); ⑮
    }
    ③ public static void main(String[] args)
    {
        methodOne();
        System.out.println("base main");
    }
    ④ public static void methodOne()
    {
        System.out.println(i); ⑭
    }
    ⑤ static int j=20; ⑯
}

class Derived extends Base
{
    ⑥ static int x=100; ⑰
    ⑦ static
    {
        methodTwo(); ⑱
        System.out.println("derived first static block"); ⑳
    }
    ⑧ public static void main(String[] args)
    {
        methodTwo(); ㉓
        System.out.println("derived main"); ㉕
    }
    ⑨ public static void methodTwo()
    {
        System.out.println(y); ㉑, ㉔
    }
    ⑩ static
    {
        System.out.println("derived second static block"); ㉑
    }
    ⑪ static int y=200; ㉒
}

```

Analysis:

```
i=0[RIWO]
j=0[RIWO]
x=0[RIWO]
y=0[RIWO]
i=10[R&w]
j=20[R&w]
x=100[R&w]
y=200[R&w]
```

Output:

```
E:\scjp>java Derived
0
Base static block
0
Derived first static block
Derived second static block
200
Derived main
Output:
E:\scjp>java Base
0
Base static block
20
Basic main
```

Whenever we are executing Child class the following sequence of events will be performed automatically.

1. Identification of static members from Parent to Child. [1 to 11]
2. Execution of static variable assignments and static blocks from Parent to Child.[12 to 22]
3. Execution of Child class main() method.[23 to 25].

**Note :** When ever we are loading child class autimatically the parent class will be loaded but when ever we are loading parent class the child class don't be loaded automatically.

**Static block:**

- Static blocks will be executed at the time of class loading hence if we want to perform any activity at the time of class loading we have to define that activity inside static block.
- With in a class we can take any no. Of static blocks and all these static blocks will be executed from top to bottom.

**Example:**

The native libraries should be loaded at the time of class loading hence we have to define that activity inside static block.

Example:

```
class Test
{
    static
    {
        System.loadLibrary("native library path");
    }
}
```

Ex 2 : Every JDBC driver class internally contains a static block to register the driver with DriverManager hence programmer is not responsible to define this explicitly.

Example:

```
class Driver
{
    static
    {
        //Register this driver with DriverManager
    }
}
```

**IIQ :** Without using main() method is it possible to print some statements to the console?

**Ans :** Yes, by using static block.

Example:

```
class Google
{
    static
    {
        System.out.println("hello i can print");
        System.exit(0);
    }
}
```

Output:

Hello i can print

**IIQ :** Without using main() method and static block is it possible to print some statements to the console ?

Example 1:

```
class Test
{
    static int i=methodOne();
    public static int methodOne()
    {
```

```

System.out.println("hello i can print");
        System.exit(0);
        return 10;
    }
}

```

Output:

Hello i can print

Example 2:

```

class Test
{
    static Test t=new Test();
    Test()
    {
        System.out.println("hello i can print");
        System.exit(0);
    }
}

```

Output:

Hello i can print

Example 3:

```

class Test
{
    static Test t=new Test();
    {
        System.out.println("hello i can print");
        System.exit(0);
    }
}

```

Output:

Hello i can print

**IIQ : Without using System.out.println() statement is it possible to print some statement to the console ?**

Example:

```

class Test
{
    public static void main(String[] args)
    {
        System.err.println("hello");
    }
}

```

**Note :** Without using main() method we can able to print some statement to the sonsole , but this rule is applicable untill 1.6 version from 1.7 version onwards to run java program main() method is mandatory.

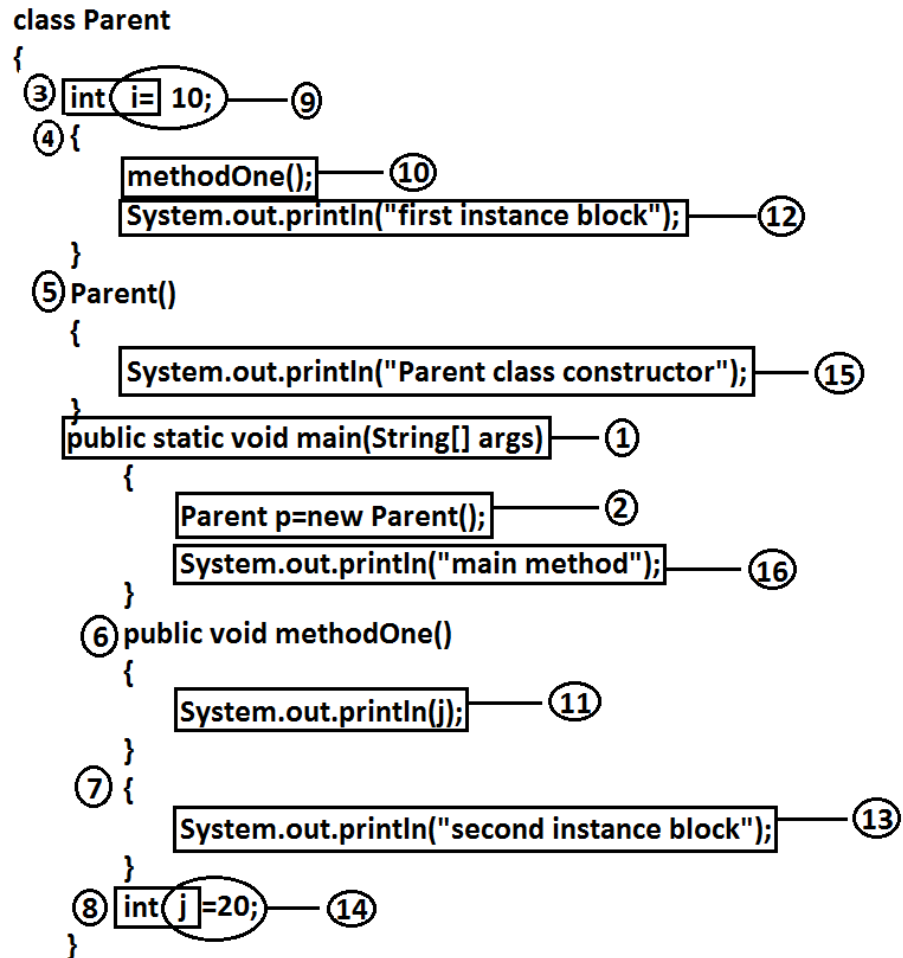
```

class Test {
    static {
        System.out.println("ststic block");
        System.exit(0);
    }
}

```

It is valid in 1.6 version but invalid or won't run in 1.7 version

## Instance control flow:



Analysis:

i=0 [RIW0]

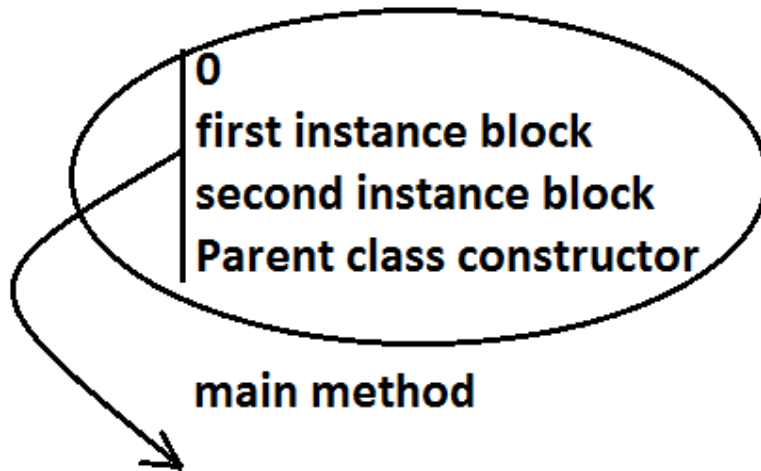
j=0 [RIW0]

i=10 [R&W]

j=20 [R&W]



Output:



Whenever we are executing a java class static control flow will be executed. In the Static control flow Whenever we are creating an object the following sequence of events will be performed automatically.

1. Identification of instance members from top to bottom(3 to 8).
2. Execution of instance variable assignments and instance blocks from top to bottom(9 to 14).
3. Execution of constructor.

**Note:** static control flow is one time activity and it will be executed at the time of class loading.

But instance control flow is not one time activity for every object creation it will be executed.

**Instance control flow in Parent to Child relationship :**

Example:

```
class Parent
{
    int x=10;
    {
        methodOne();
        System.out.println("Parent first instance block");
    }
    Parent()
    {
        System.out.println("parent class constructor");
    }
    public static void main(String[] args)
    {
        Parent p=new Parent();
        System.out.println("parent class main method");
    }
    public void methodOne()
    {
        System.out.println(y);
    }
    int y=20;
```

```

}
class Child extends Parent
{
    int i=100;
    {
        methodTwo();
        System.out.println("Child first instance block");
    }
    Child()
    {
        System.out.println("Child class constructor");
    }
    public static void main(String[] args)
    {
        Child c=new Child();
        System.out.println("Child class main method");
    }
    public void methodTwo()
    {
        System.out.println(j);
    }
    {
        System.out.println("Child second instance block");
    }
    int j=200;
}

```

Output:

E:\scjp>javac Child.java

E:\scjp>java Child

0

Parent first instance block

Parent class constructor

0

Child first instance block

Child second instance block

Child class constructor

Child class main method

Whenever we are creating child class object the following sequence of events will be executed automatically.

1. Identification of instance members from Parent to Child.
2. Execution of instance variable assignments and instance block only in Parent class.
3. Execution of Parent class constructor.
4. Execution of instance variable assignments and instance blocks in Child class.
5. Execution of Child class constructor.

**Note:** Object creation is the most costly operation in java and hence if there is no specific requirement never recommended to create objects.

Example 1:

public class Initialization

```

{
    private static String methodOne(String msg) //-->1
    {
        System.out.println(msg);
    }
}

```

```

        return msg;
    }
    public Initilization() //-->4
    {
        m=methodOne("1"); //-->9
    }
    {
        m=methodOne("2"); //-->5        //-->7
    }
    String m=methodOne("3");        //-->6 , //-->8
    public static void main(String[] args) //-->2
    {
        Object obj=new Initilization(); //-->3
    }
}

```

Analysis:

**1**  
**3**  
**2**  
**m=methodOne()[RIWO]**

Output:

2  
3  
1

Example 2:

```

public class Initilization
{
    private static String methodOne(String msg) //-->1
    {
        System.out.println(msg);
        return msg;
    }
    static String m=methodOne("1"); //-->2,        //-->5
    {
        m=methodOne("2");
    }
    static        //-->3
    {
        m=methodOne("3");        //-->6
    }
    public static void main(String[] args) //-->4
    {
        Object obj=new Initilization();
    }
}

```

Output:

1  
3  
2

We can't access instance variables directly from static area because at the time of execution of static area JVM may not identify those members.

Example:

```
class Test
{
    int i=10;
    public static void main(String[] args)
    {
        System.out.println(i);
    }
}
```

**output:**

**compile time error**

**non-static variable i cannot be referenced from a static context**

- But from the instance area we can access instance members directly.
- Static members we can access from anywhere directly because these are identified already at the time of class loading only.

## Type casting:

Parent class reference can be used to hold Child class object but by using that reference we can't call Child specific methods.

Example:

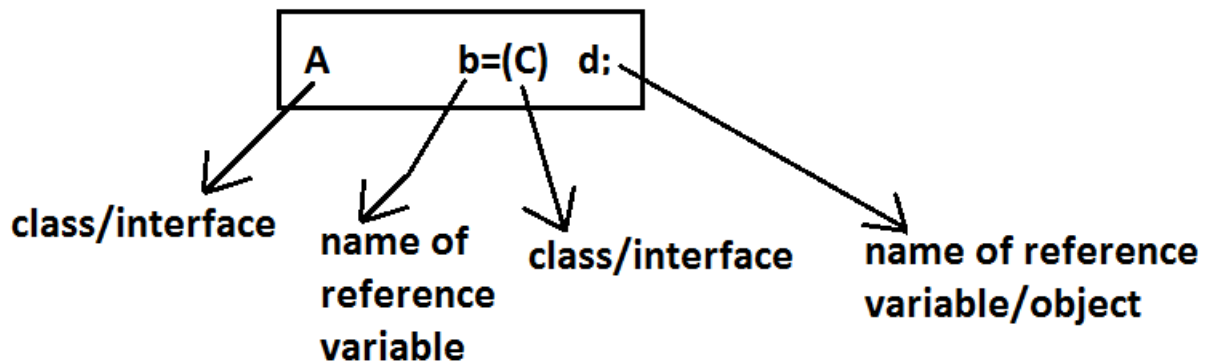
```
Object o=new String("ashok");//valid
System.out.println(o.hashCode());//valid
System.out.println(o.length());//
C.E:cannot find symbol,
    symbol : method length(),
    location: class java.lang.Object
```

Similarly we can use interface reference to hold implemented class object.

Example:

```
Runnable r=new Thread();
```

Type casting syntax:



## Compile time checking :

Rule 1: The type of "d" and "c" must have some relationship [either Child to Parent (or) Parent to Child (or) same type] otherwise we will get compile time error saying inconvertible types.

Example 1:

```
Object o=new String("bhaskar");  
StringBuffer sb=(StringBuffer)o; (valid)
```

Example 2:

```
String s=new String("bhaskar");  
StringBuffer sb=(StringBuffer)s; (inval id)
```

output:

compile time error

E:\scjp>javac Test.java

Test.java:6: inconvertible types

found : java.lang.String

required: java.lang.StringBuffer

StringBuffer sb=(StringBuffer)s;

Rule 2: "C" must be either same (or) derived type of "A" otherwise we will get compile time error saying incompatible types.

Found: C

Required: A

Example 1:

```
Object o=new String("bhaskar");  
StringBuffer sb=(StringBuffer)o; (valid)
```

Example 2:

```
Object o=new String("bhaskar");  
StringBuffer sb=(String)o; (invalid)
```

output:

compile time error

E:\scjp>javac Test.java

Test.java:6: incompatible types

found : java.lang.String

required: java.lang.StringBuffer

StringBuffer sb=(String)o;

Runtime checking :

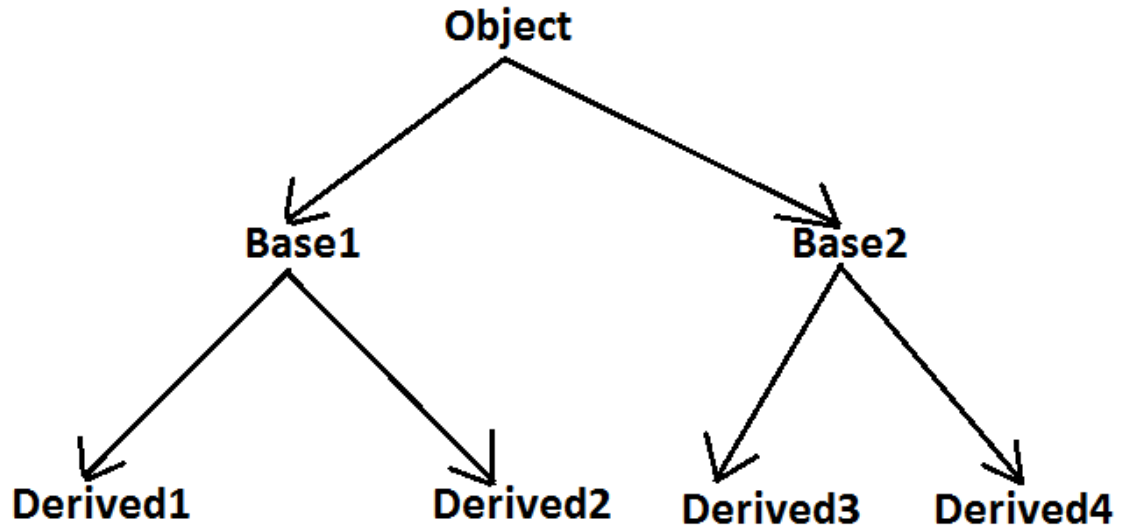
The underlying object type of "d" must be either same (or) derived type of "C" otherwise we will get runtime exception saying ClassCastException.

Example:

```
Object o=new String("bhaskar");  
StringBuffer sb=(StringBuffer)o;
```

→ Runtime Exception: ClassCastException

Diagram:



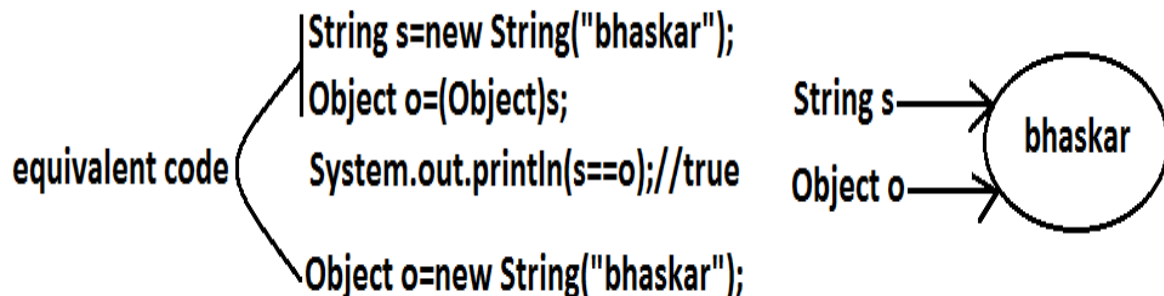
```

Base1 b=new Derived2();//valid
Object o=(Base1)b;//valid
Object o1=(Base2)o;//invalid
Object o2=(Base2)b;//invalid
Base2 b1=(Base1)(new Derived1());//invalid
Base2 b2=(Base2)(new Derived3());//valid
Base2 b2=(Base2)(new Derived1());//invalid
  
```

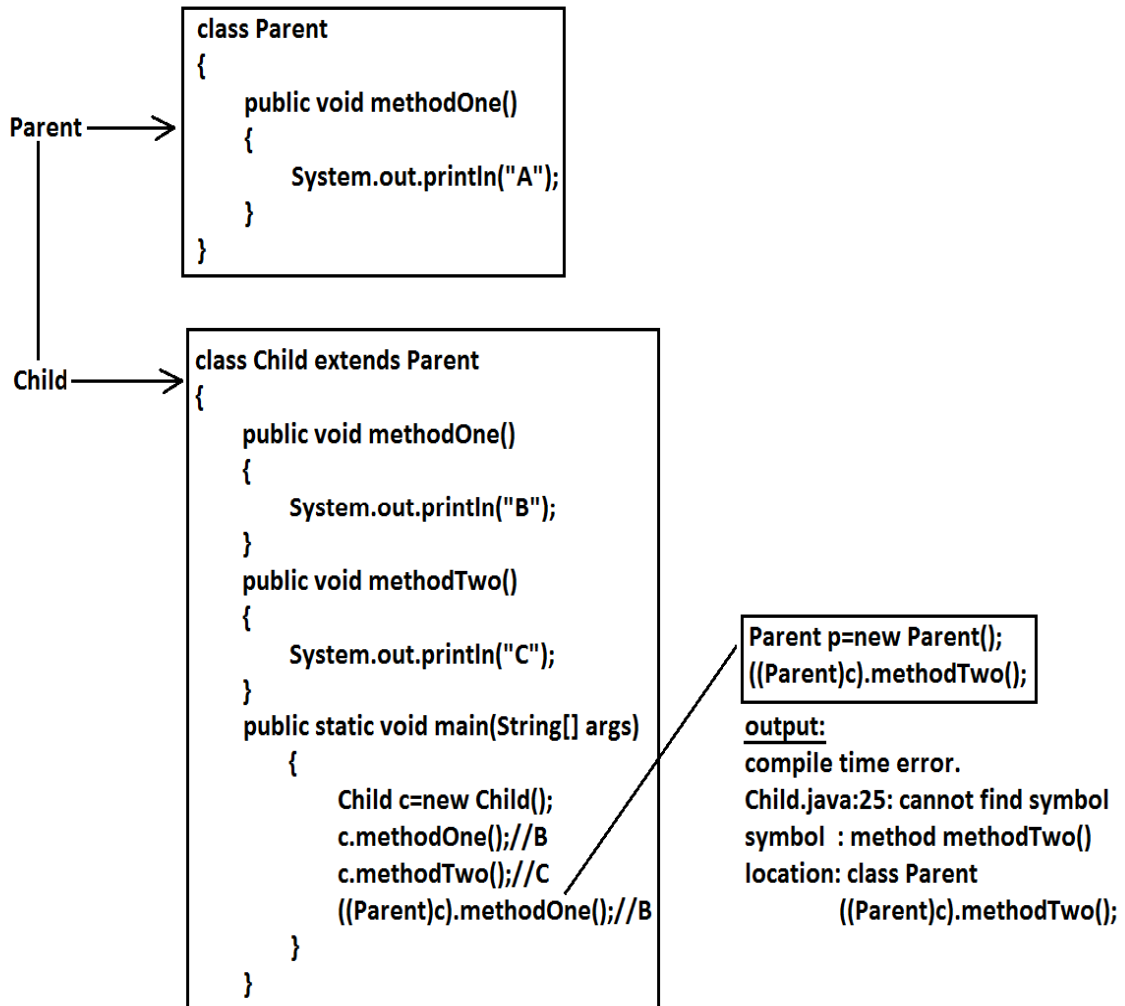
Through Type Casting just we are converting the type of object but not object itself that is we are performing type casting but not object casting.

Through Type Casting we are not create any new objects for the existing objects we are providing another type of reference variable(mostly Parent type).

Example:

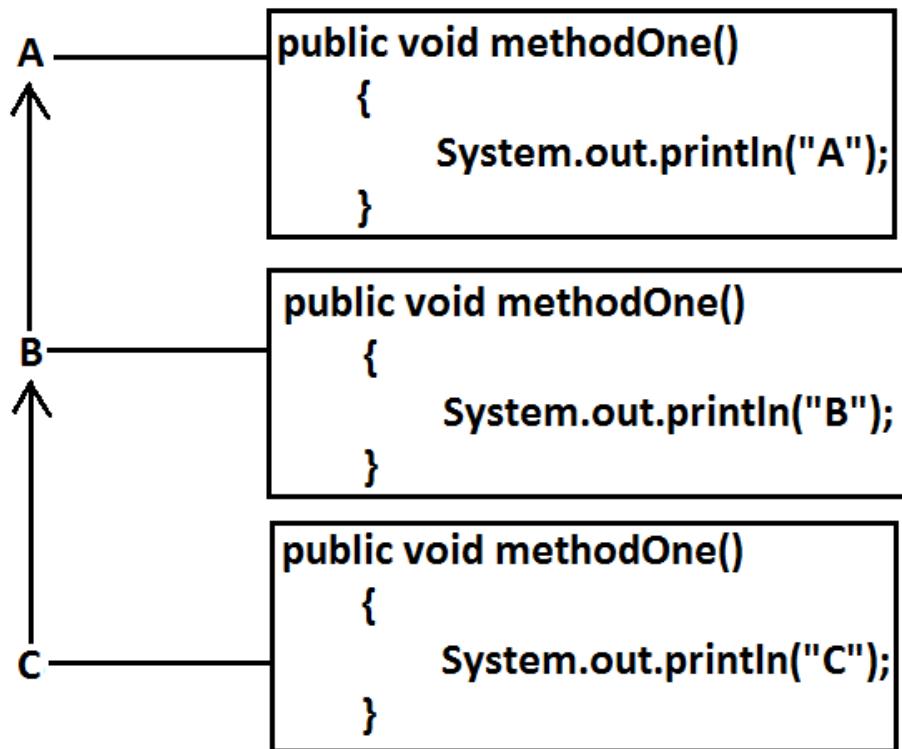


## Example 1:





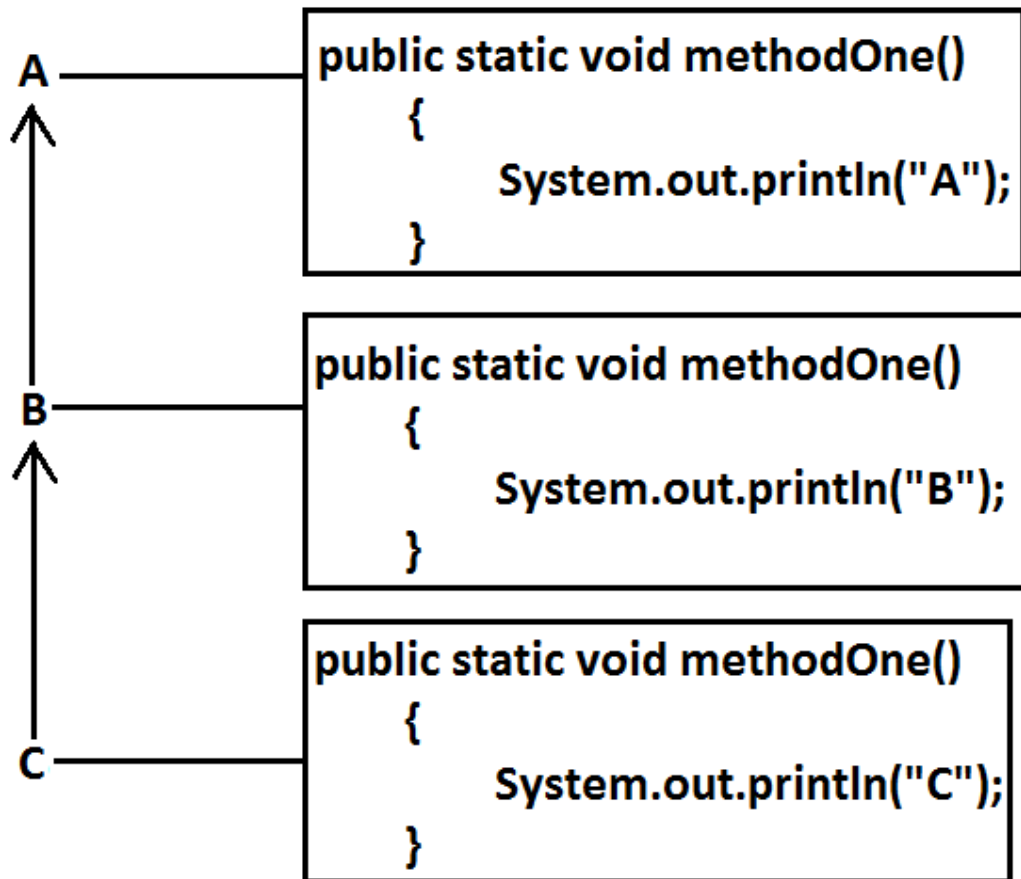
Example 2:



It is overriding and method resolution is based on runtime object.

```
C c=new C();
c.methodOne();//c
((B)c).methodOne();//c
((A)((B)c)).methodOne();//c
```

## Example 3:

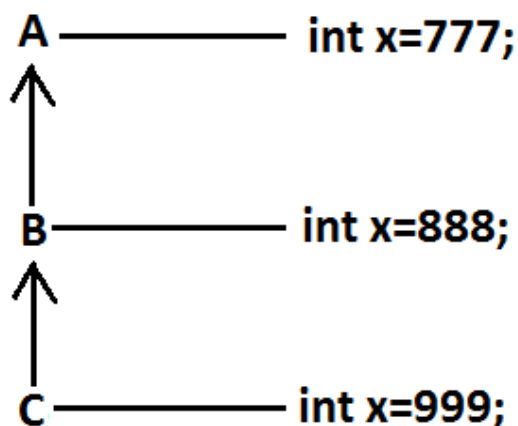


It is method hiding and method resolution is based on reference type.

```

C c=new C();
c.methodOne();//C
((B)c).methodOne();//B
((A)((B)c)).methodOne();//A
  
```

## Example 4:



```
C c=new C();
System.out.println(c.x);//999
System.out.println(((B)c).x);//888
System.out.println(((A)((B)c)).x);//777
```

- Variable resolution is always based on reference type only.
- If we are changing variable as static then also we will get the same output.

## Coupling :

The degree of dependency between the components is called coupling.

Example:

```
class A
{
    static int i=B.j;
}
class B extends A
{
    static int j=C.methodOne();
}
class C extends B
{
    public static int methodOne()
    {
        return D.k;
    }
}
class D extends C
{
    static int k=10;
    public static void main(String[] args)
    {
        D d=new D();
    }
}
```

The above components are said to be tightly coupled to each other because the dependency between the components is more.

Tightly coupling is not a good programming practice because it has several serious disadvantages.

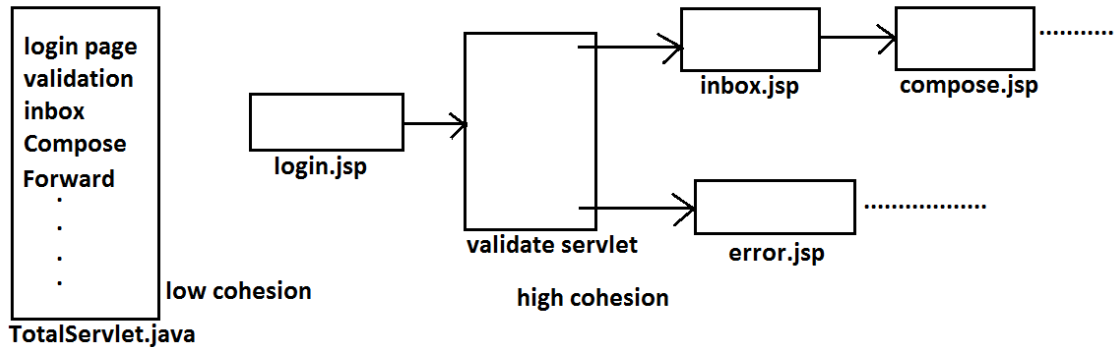
1. Without effecting remaining components we can't modify any component hence enhancement(development) will become difficult.
2. It reduces maintainability of the application.
3. It doesn't promote reusability of the code.

It is always recommended to maintain loosely coupling between the components.

## Cohesion:

For every component we have to maintain a clear well defined functionality such type of component is said to be follow high cohesion.

Diagram:



High cohesion is always good programming practice because it has several advantages.

1. Without effecting remaining components we can modify any component hence enhancement will become very easy.
2. It improves maintainability of the application.
3. It promotes reusability of the application.(where ever validation is required we can reuse the same validate servlet without rewriting )

**Note:** It is highly recommended to follow loosely coupling and high cohesion.

# **CORE JAVA**

## **With**

# **SCJP / OCJP**

### **Study Material**

#### **Chapter 6: Exception Handling**



**DURGA M.Tech**

**(Sun certified & Realtime Expert)**

**Ex. IBM Employee**

**Trained Lakhs of Students  
for last 14 years across INDIA**

**India's No.1 Software Training Institute**

# **DURGASOFT**

**www.durgasoft.com Ph: 9246212143 ,8096969696**

**EXCEPTION HANDLING AGENDA:**

1. Introduction
2. Runtime stack mechanism
3. Default exception handling in java
4. Exception hierarchy
5. Customized exception handling by try catch
6. Control flow in try catch
7. Methods to print exception information
8. Try with multiple catch blocks
9. Finally
10. Difference between final, finally, finalize
11. Control flow in try catch finally
12. Control flow in nested try catch finally
13. Various possible combinations of try catch finally
14. throw keyword
15. throws keyword
16. Exception handling keywords summary
17. Various possible compile time errors in exception handling
18. Customized exceptions
19. Top-10 exceptions
20. 1.7 Version Enhancements
  1. try with resources
  2. multi catch block
21. Exception Propagation
22. Rethrowing an Exception

## Introduction

**Exception:** An unwanted unexpected event that disturbs normal flow of the program is called exception.

*Example:*

SleepingException

TyrePuncturedException

FileNotFoundException ...etc

- It is highly recommended to handle exceptions. The main objective of exception handling is graceful (normal) termination of the program.

What is the meaning of exception handling?

Exception handling doesn't mean repairing an exception. We have to define alternative way to continue rest of the program normally. This way of defining alternative is nothing but exception handling.

**Example:** Suppose our programming requirement is to read data from remote file locating at London. At runtime if London file is not available then our program should not be terminated abnormally.

We have to provide a local file to continue rest of the program normally. This way of defining alternative is nothing but exception handling.

Example:

```
try
{
    read data from London file
}
catch(FileNotFoundException e)
{
    use local file and continue rest of the program normally
}
```

For every thread JVM will create a separate stack at the time of Thread creation. All method calls performed by that thread will be stored in that stack. Each entry in the stack is called "Activation record" (or) "stack frame".

After completing every method call JVM removes the corresponding entry from the stack.

After completing all method calls JVM destroys the empty stack and terminates the program normally.

Example:

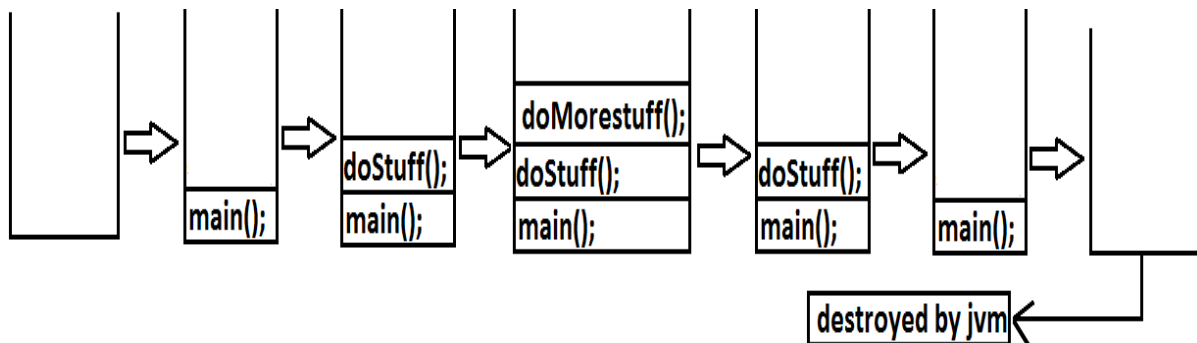
```
class Test
{
public static void main(String[] args){
doStuff();
}
public static void doStuff(){
doMoreStuff();
}
public static void doMoreStuff(){
System.out.println("Hello");
}}

```

Output:

Hello

Diagram:



### Default Exception Handling in Java:

1. If an exception raised inside any method then that method is responsible to create Exception object with the following information.
  1. Name of the exception.
  2. Description of the exception.
  3. Location of the exception.(StackTrace)
2. After creating that Exception object, the method handovers that object to the JVM.
3. JVM checks whether the method contains any exception handling code or not. If method won't contain any handling code then JVM terminates that method abnormally and removes corresponding entry form the stack.
4. JVM identifies the caller method and checks whether the caller method contain any handling code or not. If the caller method also does not contain handling code then JVM terminates that caller method also abnormally and removes corresponding entry from the stack.
5. This process will be continued until main() method and if the main() method also doesn't contain any exception handling code then JVM terminates main() method also and removes corresponding entry from the stack.
6. Then JVM handovers the responsibility of exception handling to the default exception handler.



7. Default exception handler just print exception information to the console in the following format and terminates the program abnormally.

*Exception in thread "xxx(main)" Name of exception: description  
Location of exception (stack trace)*

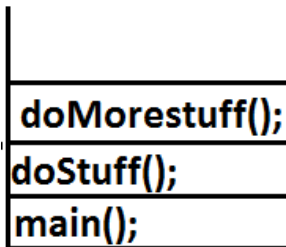
Example:

```
class Test
{
    public static void main(String[] args){
        doStuff();
    }
    public static void doStuff(){
        doMoreStuff();
    }
    public static void doMoreStuff(){
        System.out.println(10/0);
    }
}
```

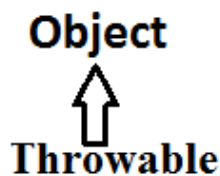
Output:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
at Test.doMoreStuff(Test.java:10)
at Test.doStuff(Test.java:7)
at Test.main(Test.java:4)
```

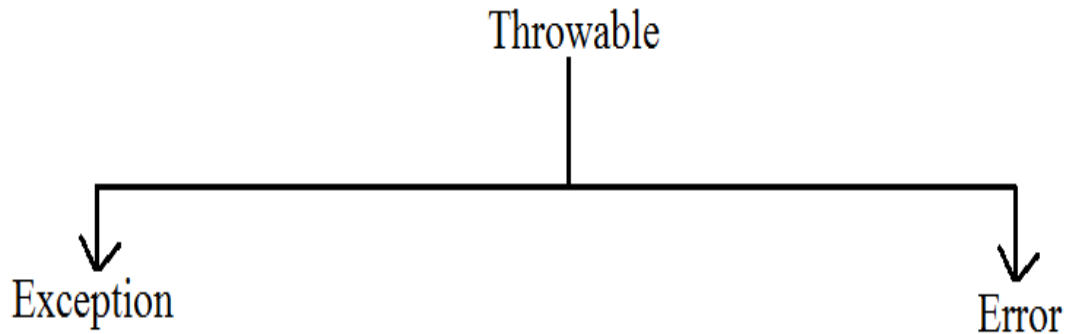
Diagram:



## Exception Hierarchy:



Throwable acts as a root for exception hierarchy.  
Throwable class contains the following two child classes.

**Exception:**

Most of the cases exceptions are caused by our program and these are recoverable.

**Ex :** If `FileNotFoundException` occurs then we can use local file and we can continue rest of the program execution normally.

**Error:**

Most of the cases errors are not caused by our program these are due to lack of system resources and these are non-recoverable.

**Ex :** If `OutOfMemoryError` occurs being a programmer we can't do anything the program will be terminated abnormally. System Admin or Server Admin is responsible to raise/increase heap memory.

**Checked Vs Unchecked Exceptions:**

- The exceptions which are checked by the compiler whether programmer handling or not, for smooth execution of the program at runtime, are called checked exceptions.
  1. `HallTicketMissingException`
  2. `PenNotWorkingException`
  3. `FileNotFoundException`
- The exceptions which are not checked by the compiler whether programmer handling or not, are called unchecked exceptions.
  1. `BombBlastException`
  2. `ArithmeticException`
  3. `NullPointerException`

**Note:** `RuntimeException` and its child classes, `Error` and its child classes are unchecked and all the remaining are considered as checked exceptions.

**Note:** Whether exception is checked or unchecked compulsory it should occur at runtime only and there is no chance of occurring any exception at compile time.

**Fully checked Vs Partially checked :**

A checked exception is said to be fully checked if and only if all its child classes are also checked.

Example:

- 1) IOException
- 2) InterruptedException

A checked exception is said to be partially checked if and only if some of its child classes are unchecked.

Example:

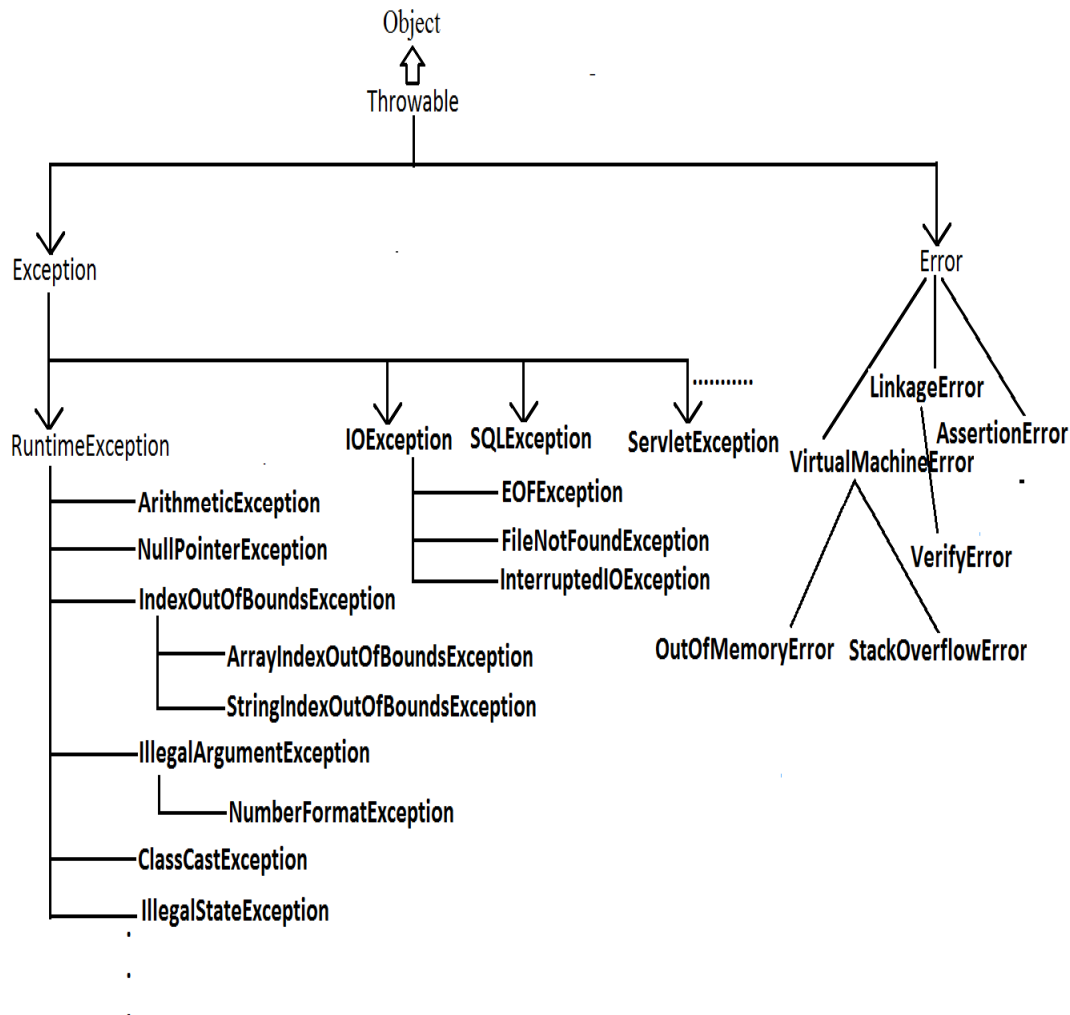
Exception

**Note :** The only possible partially checked exceptions in java are:

1. Throwable.
2. Exception.

**Q:** Describe behavior of following exceptions ?

1. RuntimeException-----unchecked
2. Error-----unchecked
3. IOException-----fully checked
4. Exception-----partially checked
5. InterruptedException-----fully checked
6. Throwable-----partially checked
7. ArithmeticException ----- unchecked
8. NullPointerException ----- unchecked
9. FileNotFoundException ----- fully checked

**Diagram:****Customized Exception Handling by using try-catch:**

- It is highly recommended to handle exceptions.
- In our program the code which may raise exception is called risky code, we have to place risky code inside try block and the corresponding handling code inside catch block.

Example:

```

try
{
    Risky code
}
catch(Exception e)
{
    Handling code
}
  
```

| Without try catch                                                                                                                                                                                                                                                      | With try catch                                                                                                                                                                                                                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>class Test { public static void main(String[] args){ System.out.println("statement1"); System.out.println(10/0); System.out.println("statement3"); } }</pre> <p>output:<br/>statement1<br/>RE:AE:/by zero<br/>at Test.main()<br/><b>Abnormal termination.</b></p> | <pre>class Test{ public static void main(String[] args){ System.out.println("statement1"); try{ System.out.println(10/0); } catch(ArithmeticException e){ System.out.println(10/2); } } System.out.println("statement3"); } }</pre> <p>Output:<br/>statement1<br/>5<br/>statement3<br/><b>Normal termination.</b></p> |

### Control flow in try catch:

```
try{
    statement1;
    statement2;
    statement3;
}
catch(X e) {
    statement4;
}
statement5;
```

- **Case 1:** If there is no exception.  
1, 2, 3, 5 normal termination.
- **Case 2:** if an exception raised at statement 2 and corresponding catch block matched  
  
1, 4, 5 normal termination.
- **Case 3:** if an exception raised at statement 2 but the corresponding catch block not matched  
  
1 followed by abnormal termination.
- **Case 4:** if an exception raised at statement 4 or statement 5 then it's always abnormal termination of the program.

#### Note:

1. Within the try block if anywhere an exception raised then rest of the try block won't be executed even though we handled that exception. Hence we have to place/take only risk code inside try block and length of the try block should be as less as possible.

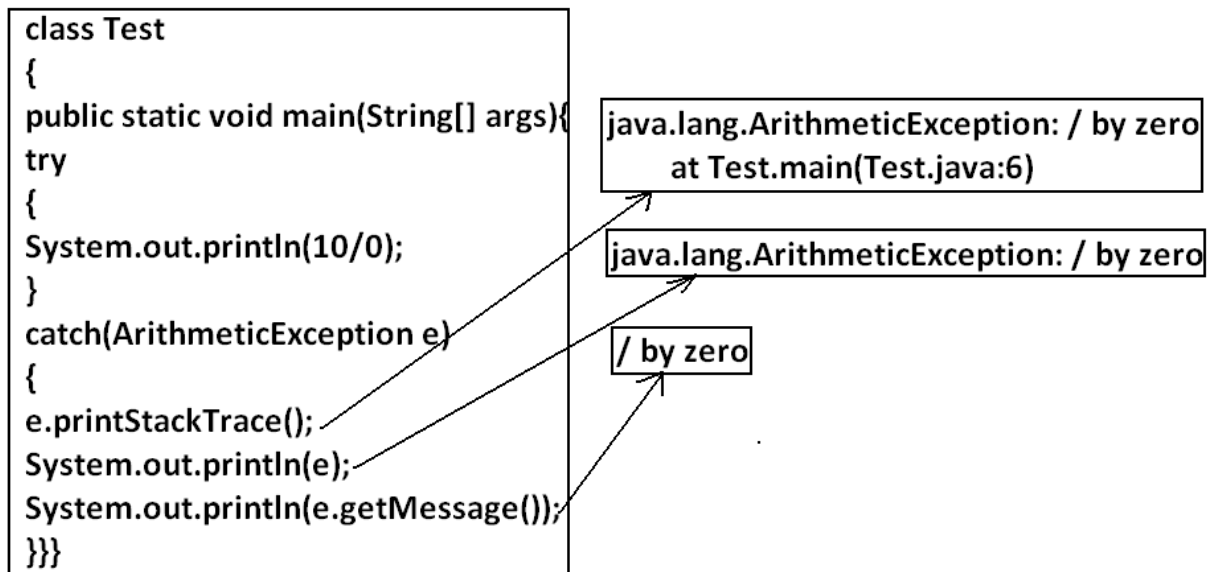
2. If any statement which raises an exception and it is not part of any try block then it is always abnormal termination of the program.
3. There may be a chance of raising an exception inside catch and finally blocks also in addition to try block.

### Various methods to print exception information:

Throwable class defines the following methods to print exception information to the console.

|                    |                                                                                                                                                   |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| printStackTrace(): | This method prints exception information in the following format.<br><u>Name of the exception: description of exception</u><br><u>Stack trace</u> |
| toString():        | This method prints exception information in the following format.<br><u>Name of the exception: description of exception</u>                       |
| getMessage():      | This method returns only description of the exception.<br><u>Description.</u>                                                                     |

Example:



**Note:** Default exception handler internally uses printStackTrace() method to print exception information to the console.

### Try with multiple catch blocks:

The way of handling an exception is varied from exception to exception. Hence for every exception type it is recommended to take a separate catch block. That is try with multiple catch blocks is possible and recommended to use.

**Example:**

|                                                                     |                                                                                                                                                                                                                                                                 |
|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>try { . . . } catch(Exception e) {     default handler }</pre> | <pre>try { . . . catch(FileNotFoundException e) {     use local file } catch(ArithmeticException e) {     perform these Arithmetic operations } catch(SQLException e) {     don't use oracle db, use mysqldb } catch(Exception e) {     default handler }</pre> |
|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

This approach is not recommended because for any type of Exception we are using the same catch block.

This approach is highly recommended because for any exception raise we are defining a separate catch block.

- If try with multiple catch blocks present then order of catch blocks is very important. It should be from child to parent by mistake if we are taking from parent to child then we will get Compile time error saying

"exception xxx has already been caught"

**Example:**

|                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>class Test { public static void main(String[] args) { try { System.out.println(10/0); } catch(Exception e) { e.printStackTrace(); } catch(ArithmeticException e) { e.printStackTrace(); }} } CE:exception java.lang.ArithmeticException has already been caught</pre> | <pre>class Test { public static void main(String[] args) { try { System.out.println(10/0); } catch(ArithmeticException e) { e.printStackTrace(); } catch(Exception e) { e.printStackTrace(); }} } Output: Compile successfully.</pre> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Finally block:**

- It is not recommended to take clean up code inside try block because there is no guarantee for the execution of every statement inside a try.
- It is not recommended to place clean up code inside catch block because if there is no exception then catch block won't be executed.
- We require some place to maintain clean up code which should be executed always irrespective of whether exception raised or not raised and whether handled or not handled. Such type of best place is nothing but finally block.
- Hence the main objective of finally block is to maintain cleanup code.

Example:

```
try
{
    risky code
}
catch(x e)
{
    handling code
}
finally
{
    cleanup code
}
```

The speciality of finally block is it will be executed always irrespective of whether the exception raised or not raised and whether handled or not handled.

**Case-1: If there is no Exception:**

```
class Test
{
    public static void main(String[] args)
    {
        try
        {
            System.out.println("try block executed");
        }
        catch(ArithmeticException e)
        {
            System.out.println("catch block executed");
        }
        finally
        {
            System.out.println("finally block executed");
        }
    }
}
```

Output:

```
try block executed
Finally block executed
```



**Case-2: If an exception raised but the corresponding catch block matched:**

```
class Test
{
    public static void main(String[] args)
    {
        try
        {
            System.out.println("try block executed");
            System.out.println(10/0);
        }
        catch(ArithmeticException e)
        {
            System.out.println("catch block executed");
        }
        finally
        {
            System.out.println("finally block executed");
        }
    }
}
```

Output:

Try block executed  
Catch block executed  
Finally block executed

**Case-3: If an exception raised but the corresponding catch block not matched:**

```
class Test
{
    public static void main(String[] args)
    {
        try
        {
            System.out.println("try block executed");
            System.out.println(10/0);
        }
        catch(NullPointerException e)
        {
            System.out.println("catch block executed");
        }
        finally
        {
            System.out.println("finally block executed");
        }
    }
}
```

Output:

Try block executed  
Finally block executed  
Exception in thread "main" java.lang.ArithmeticException: / by zero  
at Test.main(Test.java:8)

**return Vs finally:**

Even though return statement present in try or catch blocks first finally will be executed and after that only return statement will be considered. i.e. finally block dominates return statement.

Example:

```
class Test
{
    public static void main(String[] args)
    {
        try
        {
            System.out.println("try block executed");
            return;
        }
        catch(ArithmeticException e)
        {
            System.out.println("catch block executed");
        }
        finally
        {
            System.out.println("finally block executed");
        }
    }
}
```

Output:

try block executed

Finally block executed

If return statement present try, catch and finally blocks then finally block return statement will be considered.

Example:

```
class Test
{
    public static void main(String[] args)
    {
        System.out.println(m1());
    }
    public static int m1(){
        try
        {
            System.out.println(10/0);
            return 777;
        }
        catch(ArithmeticException e)
        {
            return 888;
        }
        finally{
            return 999;
        }
    }
}
```

Output:

999

**finally vs System.exit(0):**  
 =====

There is only one situation where the finally block won't be executed is whenever we are using System.exit(0) method.

When ever we are using System.exit(0) then JVM itself will be shutdown , in this case finally block won't be executed.

i.e., System.exit(0) dominates finally block.

Example:

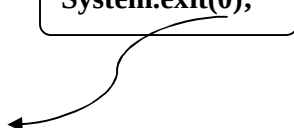
```
class Test
{
    public static void main(String[] args)
    {
        try
        {
            System.out.println("try");
            System.exit(0);
        }
        catch(ArithmeticException e)
        {
            System.out.println("catch block executed");
        }
        finally
        {
            System.out.println("finally block executed");
        }
    }
}
```

Output:

try

Note :

System.exit(0);



1. This argument acts as status code. Instead of zero, we can take any integer value
2. zero means normal termination , non-zero means abnormal termination
3. This status code internally used by JVM, whether it is zero or non-zero there is no change in the result and effect is same wrt program.

### **Difference between final, finally, and finalize:**

**final:**

- final is the modifier applicable for classes, methods and variables.
- If a class declared as the final then child class creation is not possible.
- If a method declared as the final then overriding of that method is not possible.
- If a variable declared as the final then reassignment is not possible.

**finally:**

- **finally** is the block always associated with try-catch to maintain clean up code which should be executed always irrespective of whether exception raised or not raised and whether handled or not handled.

**finalize:**

- **finalize** is a method, always invoked by Garbage Collector just before destroying an object to perform cleanup activities.

**Note:**

1. **finally** block meant for cleanup activities related to try block where as **finalize()** method meant for cleanup activities related to object.
2. To maintain clean up code **finally** block is recommended over **finalize()** method because we can't expect exact behavior of GC.

### Control flow in try catch finally:

**Example:**

```
try
{
    Stmt 1;
    Stmt-2;
    Stmt-3;
}
catch(Exception e)
{
    Stmt-4;
}
finally
{
    stmt-5;
}
Stmt-6;
```

- **Case 1:** If there is no exception. 1, 2, 3, 5, 6 normal termination.
- **Case 2:** if an exception raised at statement 2 and corresponding catch block matched. 1,4,5,6 normal terminations.
- **Case 3:** if an exception raised at statement 2 and corresponding catch block is not matched. 1,5 abnormal termination.
- **Case 4:** if an exception raised at statement 4 then it's always abnormal termination but before the finally block will be executed.
- **Case 5:** if an exception raised at statement 5 or statement 6 its always abnormal termination.

**Control flow in Nested try-catch-finally:**

```
try
{
    stmt-1;
    stmt-2;
    stmt-3;
    try
    {
        stmt-4;
        stmt-5;
        stmt-6;
    }
    catch (X e)
    {
        stmt-7;
    }
    finally
    {
        stmt-8;
    }
    stmt-9;
}
catch (Y e)
{
    stmt-10;
}
finally
{
    stmt-11;
}
stmt-12;
```

- **Case 1:** if there is no exception. 1, 2, 3, 4, 5, 6, 8, 9, 11, 12 normal termination.
- **Case 2:** if an exception raised at statement 2 and corresponding catch block matched 1,10,11,12 normal terminations.
- **Case 3:** if an exception raised at statement 2 and corresponding catch block is not matched 1, 11 abnormal termination.
- **Case 4:** if an exception raised at statement 5 and corresponding inner catch has matched 1, 2, 3, 4, 7, 8, 9, 11, 12 normal termination.
- **Case 5:** if an exception raised at statement 5 and inner catch has not matched but outer catch block has matched. 1, 2, 3, 4, 8, 10, 11, 12 normal termination.
- **Case 6:** if an exception raised at statement 5 and both inner and outer catch blocks are not matched. 1, 2, 3, 4, 8, 11 abnormal termination.
- **Case 7:** if an exception raised at statement 7 and the corresponding catch block matched 1, 2, 3, 4, 5, 6, 8, 10, 11, 12 normal termination.
- **Case 8:** if an exception raised at statement 7 and the corresponding catch block not matched 1, 2, 3, 4, 5, 6, 8, 11 abnormal terminations.

- **Case 9:** if an exception raised at statement 8 and the corresponding catch block has matched 1, 2, 3, 4, 5, 6, 7, 10, 11, 12 normal termination.
- **Case 10:** if an exception raised at statement 8 and the corresponding catch block not matched 1, 2, 3, 4, 5, 6, 7, 11 abnormal terminations.
- **Case 11:** if an exception raised at statement 9 and corresponding catch block matched 1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 12 normal termination.
- **Case 12:** if an exception raised at statement 9 and corresponding catch block not matched 1, 2, 3, 4, 5, 6, 7, 8, 11 abnormal termination.
- **Case 13:** if an exception raised at statement 10 is always abnormal termination but before that finally block 11 will be executed.
- **Case 14:** if an exception raised at statement 11 or 12 is always abnormal termination.

**Note:**

1. if we are not entering into the try block then the finally block won't be executed. Once we entered into the try block without executing finally block we can't come out.

2. We can take try-catch inside try i.e., nested try-catch is possible

3. The most specific exceptions can be handled by using inner try-catch and generalized exceptions can be handle by using outer try-catch.

**Example:**

```
class Test
{
public static void main(String[] args){
    try{
        System.out.println(10/0);
    }
    catch(ArithmeticException e)
    {
        System.out.println(10/0);
    }
    finally{
        String s=null;
        System.out.println(s.length());
    }
}}
```

**output :**

RE:NullPointerException

**Note:** Default exception handler can handle only one exception at a time and that is the most recently raised exception.

### Various possible combinations of try catch finally:

1. Whenever we are writing try block compulsory we should write either catch or finally. i.e., try without catch or finally is invalid.
2. Whenever we are writing catch block compulsory we should write try. i.e., catch without try is invalid.
3. Whenever we are writing finally block compulsory we should write try. i.e., finally without try is invalid.

4. In try-catch-finally order is important.
5. With in the try-catch -finally blocks we can take try-catch-finally.  
i.e., nesting of try-catch-finally is possible.
6. For try-catch-finally blocks curly braces are mandatory.

```
try {}
catch (X e) {}
```

 ✓

```
try {}
catch (X e) {}
catch (Y e) {}
```

 ✓

```
try {}
catch (X e) {}
catch (X e) {} //CE:exception ArithmeticException has already been caught
```

 X

```
try {}
catch (X e) {}
finally {}
```

 ✓

```
try {}
finally {}
```

 ✓

```
try {} //CE: 'try' without 'catch', 'finally' or resource declarations
```

 X

```
catch (X e) {} //CE: 'catch' without 'try'
```

 X

```
finally {} //CE: 'finally' without 'try'
```

 X

```
try {} //CE: 'try' without 'catch', 'finally' or resource declarations
System.out.println("Hello");
catch {} //CE: 'catch' without 'try'
```

 X

```
try {}
catch (X e) {}
System.out.println("Hello");
catch (Y e) {} //CE: 'catch' without 'try'
```

 X

```
try {}
catch (X e) {}
System.out.println("Hello");
finally {} //CE: 'finally' without 'try'
```

 X

```
try {}
finally {}
catch (X e) {} //CE: 'catch' without 'try'
```

 X

```
try {}  
catch (X e) {}  
try {}  
finally {}
```

✓

```
try {}  
catch (X e) {}  
finally {}  
finally {} //CE: 'finally' without 'try'
```

✗

```
try {}  
catch (X e) {  
    try {}  
    catch (Y e1) {}  
}
```

✓

```
try {}  
catch (X e) {}  
finally {  
    try {}  
    catch (Y e1) {}  
    finally {}  
}
```

✓

```
try {  
    try {} //CE: 'try' without 'catch', 'finally' or resource declarations  
}  
catch (X e) {}
```

✗

```
try //CE: '{' expected  
System.out.println("Hello");  
catch (X e1) {} //CE: 'catch' without 'try'
```

✗

```
try {}  
catch (X e) //CE: '{' expected  
System.out.println("Hello");
```

✗

```
try {}  
catch (NullPointerException e1) {}  
finally //CE: '{' expected  
System.out.println("Hello");
```

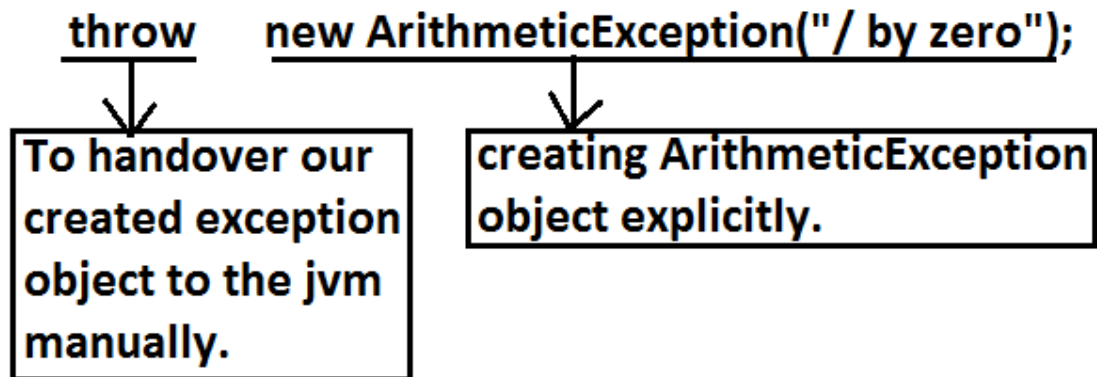
✗



## throw statement:

Sometimes we can create Exception object explicitly and we can hand over to the JVM manually by using throw keyword.

Example:



The result of following 2 programs is exactly same.

```
class Test
{
    public static void main(String[] args){
        System.out.println(10/0);
    }
}
```

In this case creation of ArithmeticException object and handover to the jvm will be performed automatically by the main() method.

```
class Test
{
    public static void main(String[] args){
        throw new ArithmeticException("/ by zero");
    }
}
```

In this case we are creating exception object explicitly and handover to the JVM manually.

Note: In general we can use throw keyword for customized exceptions but not for predefined exceptions.

Case 1:

throw e;

←  
If e refers null then we will get NullPointerException.

**Example:**

|                                                                                                                                                                                                                            |                                                                                                                                                                                                           |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>class Test3 { static ArithmeticException e=new ArithmeticException(); public static void main(String[] args){ throw e; } } Output: Runtime exception: Exception in thread "main"  java.lang.ArithmeticException</pre> | <pre>class Test3 { static ArithmeticException e; public static void main(String[] args){ throw e; } } Output: Exception in thread "main" java.lang.NullPointerException at Test3.main(Test3.java:5)</pre> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Case 2:**

After throw statement we can't take any statement directly otherwise we will get compile time error saying unreachable statement.

**Example:**

|                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>class Test3 { public static void main(String[] args){ System.out.println(10/0); System.out.println("hello"); } } Output: Runtime error: Exception in thread "main" java.lang.ArithmeticException: / by zero at Test3.main(Test3.java:4)</pre> | <pre>class Test3 { public static void main(String[] args){ throw new ArithmeticException("/ by zero"); System.out.println("hello"); } } Output: Compile time error. Test3.java:5: unreachable statement System.out.println("hello");</pre> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Case 3:**

We can use throw keyword only for Throwable types otherwise we will get compile time error saying incompatible types.

**Example:**

|                                                                                                                                                                                                                       |                                                                                                                                                                                                      |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>class Test3 { public static void main(String[] args){ throw new Test3(); } } Output: Compile time error. Test3.java:4: incompatible types found   : Test3 required: java.lang.Throwable throw new Test3();</pre> | <pre>class Test3 extends RuntimeException { public static void main(String[] args){ throw new Test3(); } } Output: Runtime error: Exception in thread "main" Test3 at Test3.main(Test3.java:4)</pre> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Throws statement:

In our program if there is any chance of raising checked exception then compulsory we should handle either by try catch or by throws keyword otherwise the code won't compile.

Example:

```
import java.io.*;
class Test3
{
    public static void main(String[] args){
        PrintWriter out=new PrintWriter("abc.txt");
        out.println("hello");
    }
}
```

CE :

Unreported exception java.io.FileNotFoundException; must be caught or declared to be thrown.

Example:

```
class Test3
{
    public static void main(String[] args){
        Thread.sleep(5000);
    }
}
```

Unreported exception java.lang.InterruptedException; must be caught or declared to be thrown.

We can handle this compile time error by using the following 2 ways.

Example:

| By using try catch                                                                                                                                                                                                               | By using throws keyword                                                                                                                                                                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>class Test3 {     public static void     main(String[] args){         try{             Thread.sleep(5000);         }         catch(InterruptedException         e){}     } } Output: Compile and running successfully</pre> | <p>We can use throws keyword to delegate the responsibility of exception handling to the caller method. Then caller method is responsible to handle that exception.</p> <pre>class Test3 {     public static void main(String[] args)throws     InterruptedException{         Thread.sleep(5000);     } } Output: Compile and running successfully</pre> |

**Note :**

- Hence the main objective of "throws" keyword is to delegate the responsibility of exception handling to the caller method.
- "throws" keyword required only checked exceptions. Usage of throws for unchecked exception there is no use.
- "throws" keyword required only to convince compiler. Usage of throws keyword doesn't prevent abnormal termination of the program.

Hence recommended to use try-catch over throws keyword.

Example:

```
class Test
{
public static void main(String[] args)throws InterruptedException{
doStuff();
}
public static void doStuff()throws InterruptedException{
doMoreStuff();
}
public static void doMoreStuff()throws InterruptedException{
Thread.sleep(5000);
}
}
```

Output:

Compile and running successfully.

In the above program if we are removing at least one throws keyword then the program won't compile.

**Case 1:**

we can use throws keyword only for Throwable types otherwise we will get compile time error saying incompatible types.

Example:

|                                                                                                                                                                                                                                                |                                                                                                                                                     |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>class Test3{ public static void main(String[] args) throws Test3 {} } Output: Compile time error Test3.java:2: incompatible types found   : Test3 required: java.lang.Throwable public static void main(String[] args) throws Test3</pre> | <pre>class Test3 extends RuntimeException{ public static void main(String[] args) throws Test3 {} } Output: Compile and running successfully.</pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|

**Case 2:Example:**

```
class Test3{
public static void main(String[]
args){
throw new Exception();
}
}
Output:
Compile time error.
Test3.java:3: unreported exception
      java.lang.Exception;
must be caught or declared to be
thrown
```

```
class Test3{
public static void main(String[]
args){
throw new Error();
}
}
Output:
Runtime error
Exception in thread "main"
java.lang.Error
      at Test3.main(Test3.java:3)
```

**Case 3:**

In our program with in the try block, if there is no chance of rising an exception then we can't right catch block for that exception otherwise we will get compile time error saying exception XXX is never thrown in body of corresponding try statement. But this rule is applicable only for fully checked exception.

**Example:**

```
class Test
{
public static void main(String[] args){
try{
System.out.println("hello");
}
catch(Exception e)
{ } output:
} hello
} partial checked
```

```
class Test
{
public static void main(String[] args){
try{
System.out.println("hello");
}
catch(ArithmeticException e)
{ } output:
} hello
} unchecked
```

```
class Test
{
public static void main(String[] args){
try{
System.out.println("hello");
}
catch(java.io.IOException e)
{ } output:
} compile time error
} fully checked
```

```

class Test
{
public static void main(String[] args){
try{
System.out.println("hello");
}
catch(InterruptedException e)
{} output:
} compile time error
} Fully checked

```

```

class Test
{
public static void main(String[] args){
try{
System.out.println("hello");
}
catch(Error e)
{} output:
} compile successfully
} unchecked

```

**Case 4:**

We can use throws keyword only for constructors and methods but not for classes.

Example:

```

class Test throws Exception    //invalid
{
    Test() throws Exception    //valid
    {}
    methodOne() throws Exception    //valid
    { }
}

```

**Exception handling keywords summary:**

1. try: To maintain risky code.
2. catch: To maintain handling code.
3. finally: To maintain cleanup code.
4. throw: To handover our created exception object to the JVM manually.
5. throws: To delegate responsibility of exception handling to the caller method.

**Various possible compile time errors in exception handling:**

1. Exception XXX has already been caught.
2. Unreported exception XXX must be caught or declared to be thrown.
3. Exception XXX is never thrown in body of corresponding try statement.
4. Try without catch or finally.
5. Catch without try.

6. Finally without try.
7. Incompatible types.  
found:Test  
required:java.lang.Throwable;
8. Unreachable statement.

### Customized Exceptions (User defined Exceptions):

Sometimes we can create our own exception to meet our programming requirements. Such type of exceptions are called customized exceptions (user defined exceptions).

*Example:*

1. InsufficientFundsException
2. TooYoungException
3. TooOldException

Program:

```
class TooYoungException extends RuntimeException
{
    TooYoungException(String s)
    {
        super(s);
    }
}
class TooOldException extends RuntimeException
{
    TooOldException(String s)
    {
        super(s);
    }
}
class CustomizedExceptionDemo
{
    public static void main(String[] args){
        int age=Integer.parseInt(args[0]);
        if(age>60)
        {
            throw new TooYoungException("please wait some more time.... u will get best match");
        }
        else if(age<18)
        {
            throw new TooOldException("u r age already crossed....no chance of getting married");
        }
        else
        {
            System.out.println("you will get match details soon by e-mail");
        }
    }
}
```

Output:

```
1)E:\scjp>java CustomizedExceptionDemo 61
Exception in thread "main" TooYoungException:
please wait some more time.... u will get best match
at CustomizedExceptionDemo.main(CustomizedExceptionDemo.java:21)
```

```
2)E:\scjp>java CustomizedExceptionDemo 27
You will get match details soon by e-mail
```

```
3)E:\scjp>java CustomizedExceptionDemo 9
Exception in thread "main" TooOldException:
u r age already crossed....no chance of getting married
at CustomizedExceptionDemo.main(CustomizedExceptionDemo.java:25)
```

**Note:** It is highly recommended to maintain our customized exceptions as unchecked by extending `RuntimeException`.  
We can catch any `Throwable` type including `Errors` also.

Example:

```
try
{
    catch(Error e) valid
}
```

## Top-10 Exceptions:

Based on the person who is raising exception, all exceptions are divided into two types.

They are:

- 1) JVM Exceptions:
- 2) Programmatic exceptions:

### JVM Exceptions:

The exceptions which are raised automatically by the jvm whenever a particular event occurs, are called JVM Exceptions.

Example:

- 1) `ArrayIndexOutOfBoundsException(AIOOBE)`
- 2) `NullPointerException (NPE)`.

### Programmatic Exceptions:

The exceptions which are raised explicitly by the programmer (or) by the API developer are called programmatic exceptions.

Example: 1) `IllegalArgumentException(IAE)`.



**Top 10 Exceptions :****1. ArrayIndexOutOfBoundsException:**

It is the child class of RuntimeException and hence it is unchecked. Raised automatically by the JVM whenever we are trying to access array element with out of range index. Example:

```
class Test{
public static void main(String[] args){
int[] x=new int[10];
System.out.println(x[0]); //valid
System.out.println(x[100]); //AI00BE
System.out.println(x[-100]); //AI00BE
}
}
```

**2. NullPointerException:**

It is the child class of RuntimeException and hence it is unchecked. Raised automatically by the JVM, whenever we are trying to call any method on null.

Example:

```
class Test{
public static void main(String[] args){
String s=null;
System.out.println(s.length()); //R.E: NullPointerException
}
}
```

**3. StackOverflowError:**

It is the child class of Error and hence it is unchecked. Whenever we are trying to invoke recursive method call JVM will raise StackOverFloeError automatically.

Example:

```
class Test
{
public static void methodOne()
{
methodTwo();
}
public static void methodTwo()
{
methodOne();
}
public static void main(String[] args)
{
methodOne();
}
}
```

Output:

Run time error: StackOverFloeError

**4. NoClassDefFoundError:**

It is the child class of Error and hence it is unchecked. JVM will raise this error automatically whenever it is unable to find required .class file. Example: java Test If Test.class is not available. Then we will get NoClassDefFound error.

**5. ClassCastException:**

It is the child class of RuntimeException and hence it is unchecked. Raised automatically by the JVM whenever we are trying to type cast parent object to child type.

Example:

|                                                                                                                                                      |                                                                                                                                                                     |                                                                                                                                                      |
|------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>class Test { public static void main(String[] args) { String s=new String("bhaskar"); Object o=(Object)s; } <u>output:</u> } <u>valid</u></pre> | <pre>class Test { public static void main(String[] args) { Object o=new Object(); String s=(String)o; } <u>output:</u> } Runtime exception:ClassCastException</pre> | <pre>class Test { public static void main(String[] args) { Object o=new String("bhaskar"); String s=(String)o; } <u>output:</u> } <u>valid</u></pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|

**6. ExceptionInInitializerError:**

It is the child class of Error and it is unchecked. Raised automatically by the JVM, if any exception occurs while performing static variable initialization and static block execution.

Example 1:

```
class Test{
static int i=10/0;
}
```

Output:

Runtime exception:

Exception in thread "main" java.lang.ExceptionInInitializerError

Example 2:

```
class Test{
static {
String s=null;
System.out.println(s.length());
}}
}
```

Output:

Runtime exception:

Exception in thread "main" java.lang.ExceptionInInitializerError

**7. IllegalArgumentException:**

It is the child class of RuntimeException and hence it is unchecked. Raised explicitly by the programmer (or) by the API developer to indicate that a method has been invoked with inappropriate argument.

Example:

```
class Test{
```

```
public static void main(String[] args){
    Thread t=new Thread();
    t.setPriority(10);//valid
    t.setPriority(100);//invalid
}
}
Output:
Runtime exception
Exception in thread "main" java.lang.IllegalArgumentException.
```

### 8. NumberFormatException:

It is the child class of `IllegalArgumentException` and hence is unchecked. Raised explicitly by the programmer or by the API developer to indicate that we are attempting to convert string to the number. But the string is not properly formatted.

Example:

```
class Test{
    public static void main(String[] args){
        int i=Integer.parseInt("10");
        int j=Integer.parseInt("ten");
    }
}
Output:
Runtime Exception
Exception in thread "main" java.lang.NumberFormatException: For input
string: "ten"
```

### 9. IllegalStateException:

It is the child class of `RuntimeException` and hence it is unchecked. Raised explicitly by the programmer or by the API developer to indicate that a method has been invoked at inappropriate time.

Example:

Once session expires we can't call any method on the session object otherwise we will get `IllegalStateException`

```
HttpSession session=req.getSession();
System.out.println(session.getId());
session.invalidate();
System.out.println(session.getId()); // illgalStateException
```

### 10. AssertionError:

It is the child class of `Error` and hence it is unchecked. Raised explicitly by the programmer or by API developer to indicate that Assert statement fails.

Example:  
`assert(false);`

| Exception/Error                                                                                                                                                                                                                               | Raised by                                                                            |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <ol style="list-style-type: none"> <li>1. AIOOBE</li> <li>2. NPE(NullPointerException)</li> <li>3. StackOverflowError</li> <li>4. NoClassDefFoundError</li> <li>5. CCE(ClassCastException)</li> <li>6. ExceptionInInitializerError</li> </ol> | Raised automatically by JVM(JVM Exceptions)                                          |
| <ol style="list-style-type: none"> <li>1. IAE(IllegalArgumentException)</li> <li>2. NFE(NumberFormatException)</li> <li>3. ISE(IllegalStateException)</li> <li>4. AE(AssertionError)</li> </ol>                                               | Raised explicitly either by programmer or by API developer (Programatic Exceptions). |

## 1.7 Version Enhancements :

As part of 1.7 version enhancements in Exception Handling the following 2 concepts introduced

1. try with resources
2. multi catch block

### 1.try with resources

Untill 1.6 version it is highly recommended to write finally block to close all resources which are open as part of try block.

```
BufferedReader br=null;
try{
br=new BufferedReader(new FileReader("abc.txt"));
//use br based on our requirements
}
catch(IOException e) {
// handling code
}
finally {
if(br != null)
br.close();
}
```

### problems in this approach :

- Compulsory programmer is required to close all opened resources with increases the complexity of the programming
- Compulsory we should write finally block explicitly which increases length of the code and reviews readability.

To overcome these problems Sun People introduced "try with resources" in 1.7 version.

**The main advantage of "try with resources" is**

the resources which are opened as part of try block will be closed automatically. Once the control reaches end of the try block either normally or abnormally and hence we are not required to close explicitly so that the complexity of programming will be reduced. It is not required to write finally block explicitly and hence length of the code will be reduced and readability will be improved.

```
try(BufferedReader br=new BufferedReader(new FileReader("abc.txt")))
{
    use be based on our requirement, br will be closed automatically ,
    Once control reaches end of try either normally
    or abnormally and we are not required to close explicitly
}
catch(IOException e) {
    // handling code
}
```

**Conclusions:**

1. We can declare any no of resources but all these resources should be separated with ;(semicolon)

```
try(R1 ; R2 ; R3)
{
    -----
    -----
}
```

2. All resources should be AutoCloseable resources. A resource is said to be auto closable if and only if the corresponding class implements the java.lang.AutoCloseable interface either directly or indirectly.

All database related, network related and file io related resources already implemented AutoCloseable interface. Being a programmer we should aware and we are not required to do anything extra.

3. All resource reference variables are implicitly final and hence we can't perform reassignment with in the try block.

```
try(BufferedReader br=new BufferedReader(new FileReader("abc.txt"))) ;
{
    br=new BufferedReader(new FileReader("abc.txt"));
}
```

output :

CE : Can't reassign a value to final variable br

4.Untill 1.6 version try should be followed by either catch or finally but 1.7 version we can take only try with resource without catch or finally

```
try(R)
```

```
{
    //valid
}
```

5.The main advantage of "try with resources" is finally block will become dummy because we are not required to close resources of explicitly.

### Multi catch block :

Until 1.6 version ,Eventhough Multiple Exceptions having same handling code we have to write a separate catch block for every exceptions, it increases length of the code and reviews readability

```
try{
    -----
    -----
}
catch(ArithmeticException e) {
    e.printStackTrace();
}
catch(NullPointerException e) {
    e.printStackTrace();
}
catch(ClassCastException e) {
    System.out.println(e.getMessage());
}
catch(IOException e) {
    System.out.println(e.getMessage());
}
```

To overcome this problem Sun People introduced "Multi catch block" concept in 1.7 version.

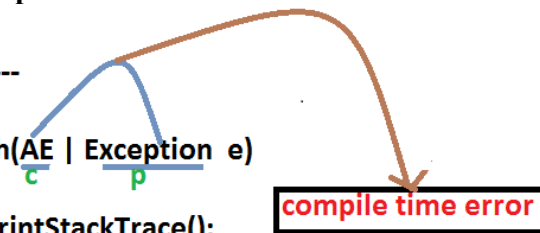
The main advantage of multi catch block is we can write a single catch block , which can handle multiple different exceptions

```
try{
    -----
    -----
}
catch(ArithmeticException | NullPointerException e) {
    e.printStackTrace();
}
catch(ClassCastException | IOException e) {
    System.out.println(e.getMessage());
}
```

In multi catch block, there should not be any relation between Exception types(either child to parent Or parent to child Or same type , otherwise we will get Compile time error )

Example:

```
try {
    -----
}
catch(AE | Exception e)
{
    e.printStackTrace();
}
```



**compile time error**

invalid

### Exception Propagation :

With in a method if an exception raised and if that method doesn't handle that exception, then Exception object will be propagated to the caller then caller method is responsible to handle that exceptions. This process is called Exception Propagation.

### Rethrowing an Exception :

To convert the one exception type to another exception type , we can use rethrowing exception concept.

```
class Test
{
    public static void main(String[] args){
        try {
            System.out.println(10/0);
        }
        catch(ArithmeticException e) {
            throw new NullPointerException();
        }
    }
}
```

output:  
RE:NPE

# **CORE JAVA**

## **With**

# **SCJP / OCJP**

### **Study Material**

#### **Chapter 7 : Multi Threading**



**DURGA M.Tech**

**(Sun certified & Realtime Expert)**

**Ex. IBM Employee**

**Trained Lakhs of Students  
for last 14 years across INDIA**

**India's No.1 Software Training Institute**

# **DURGASOFT**

**www.durgasoft.com Ph: 9246212143 ,8096969696**



# Multi Threading

## Agenda

1. Introduction.
2. The ways to define, instantiate and start a new Thread.
  1. By extending Thread class
  2. By implementing Runnable interface
3. Thread class constructors
4. Thread priority
5. Getting and setting name of a Thread.
6. The methods to prevent(stop) Thread execution.
  1. yield()
  2. join()
  3. sleep()
7. Synchronization.
8. Inter Thread communication.
9. Deadlock
10. Daemon Threads.
11. Various Conclusion
  1. To stop a Thread
  2. Suspend & resume of a thread
  3. Thread group
  4. Green Thread
  5. Thread Local
12. Life cycle of a Thread

## Introduction

**Multitasking:** Executing several tasks simultaneously is the concept of multitasking. There are two types of multitasking's.

1. Process based multitasking.
2. Thread based multitasking.

### Diagram:



### Process based multitasking:

Executing several tasks simultaneously where each task is a separate independent process such type of multitasking is called process based multitasking.

Example:

- While typing a java program in the editor we can able to listen mp3 audio songs at the same time we can download a file from the net all these tasks are independent of each other and executing simultaneously and hence it is Process based multitasking.
- This type of multitasking is best suitable at "os level".

### Thread based multitasking:

Executing several tasks simultaneously where each task is a separate independent part of the same program, is called Thread based multitasking.

And each independent part is called a "Thread".

1. This type of multitasking is best suitable for "programatic level".
2. When compared with "C++", developing multithreading examples is very easy in java because java provides in built support for multithreading through a rich API (Thread, Runnable, ThreadGroup, ThreadLocal...etc).
3. In multithreading on 10% of the work the programmer is required to do and 90% of the work will be down by java API.
4. *The main important application areas of multithreading are:*
  1. To implement multimedia graphics.
  2. To develop animations.
  3. To develop video games etc.
  4. To develop web and application servers
5. Whether it is process based or Thread based the main objective of multitasking is to improve performance of the system by reducing response time.

## **The ways to define instantiate and start a new Thread:**

What is singleton? Give example?

We can define a Thread in the following 2 ways.

1. By extending Thread class.
2. By implementing Runnable interface.

### Defining a Thread by extending "Thread class":

Example:

defining a Thread.

```

class MyThread extends Thread
{
    public void run()
    {
        for(int i=0;i<10;i++)
        {
            System.out.println("child Thread");
        }
    }
}

```

Job of a Thread.

```

class ThreadDemo
{
    public static void main(String[] args)
    {
        MyThread t=new MyThread();//Instantiation of a Thread
        t.start();//starting of a Thread

        for(int i=0;i<5;i++)
        {
            System.out.println("main thread");
        }
    }
}

```

### Case 1: Thread Scheduler:

- If multiple Threads are waiting to execute then which Thread will execute 1st is decided by "Thread Scheduler" which is part of JVM.
- Which algorithm or behavior followed by Thread Scheduler we can't expect exactly it is the JVM vendor dependent hence in multithreading examples we can't expect exact execution order and exact output.

- The following are various possible outputs for the above program.

| p1           | p2           | p3           |
|--------------|--------------|--------------|
| main thread  | main thread  | main thread  |
| main thread  | main thread  | main thread  |
| main thread  | main thread  | main thread  |
| main thread  | main thread  | main thread  |
| main thread  | main thread  | main thread  |
| child thread | child thread | child thread |
| child thread | child thread | child thread |
| child thread | child thread | child thread |
| child thread | child thread | child thread |
| child thread | child thread | child thread |
| child thread | child thread | child thread |
| child thread | child thread | child thread |
| child thread | child thread | child thread |
| child thread | child thread | child thread |
| child thread | child thread | child thread |

Case 2: Difference between t.start() and t.run() methods.

- In the case of t.start() a new Thread will be created which is responsible for the execution of run() method.
- But in the case of t.run() no new Thread will be created and run() method will be executed just like a normal method by the main Thread.
- In the above program if we are replacing t.start() with t.run() the following is the output.

Output:

```
child thread
child thread
child thread
child thread
child thread
child thread
child thread
child thread
child thread
child thread
child thread
main thread
main thread
main thread
main thread
```

main thread

Entire output produced by only main Thread.

### Case 3: importance of Thread class start() method.

For every Thread the required mandatory activities like registering the Thread with Thread Scheduler will takes care by Thread class start() method and programmer is responsible just to define the job of the Thread inside run() method.

That is start() method acts as best assistant to the programmer.

Example:

```
start()
{
    1. Register Thread with Thread Scheduler
    2. All other mandatory low level activities.
    3. Invoke or calling run() method.
}
```

We can conclude that without executing Thread class start() method there is no chance of starting a new Thread in java. Due to this start() is considered as heart of multithreading.

### Case 4: If we are not overriding run() method:

If we are not overriding run() method then Thread class run() method will be executed which has empty implementation and hence we won't get any output.

Example:

```
class MyThread extends Thread
{
}
class ThreadDemo
{
    public static void main(String[] args)
    {
        MyThread t=new MyThread();
        t.start();
    }
}
```

It is highly recommended to override run() method. Otherwise don't go for multithreading concept.

### Case 5: Overloading of run() method.

We can overload run() method but Thread class start() method always invokes no argument run() method the other overload run() methods we have to call explicitly then only it will be executed just like normal method.

Example:

```
class MyThread extends Thread
{
    public void run()
    {
        System.out.println("no arg method");
    }
    public void run(int i)
    {
        System.out.println("int arg method");
    }
}
```

```
class ThreadDemo
{
    public static void main(String[] args)
    {
        MyThread t=new MyThread();
        t.start();
    }
}
```

Output:  
No arg method

### Case 6: overriding of start() method:

If we override start() method then our start() method will be executed just like a normal method call and no new Thread will be started.

Example:

```
class MyThread extends Thread
{
    public void start()
    {
        System.out.println("start method");
    }
    public void run()
    {
        System.out.println("run method");
    }
}
class ThreadDemo
{
    public static void main(String[] args)
    {
        MyThread t=new MyThread();
        t.start();
        System.out.println("main method");
    }
}
```

Output:  
start method  
main method

Entire output produced by only main Thread.

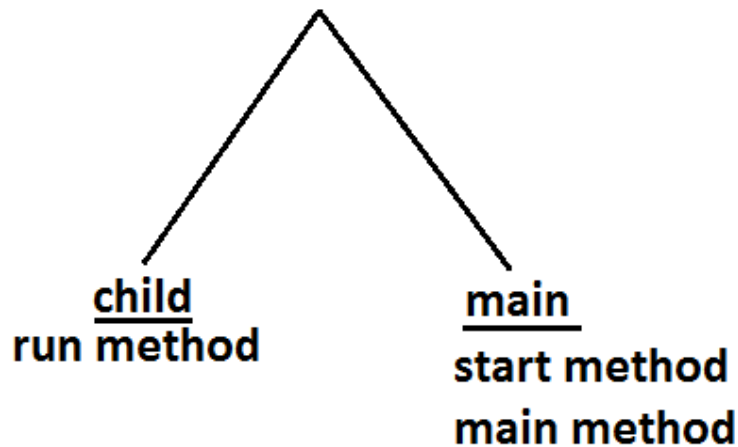
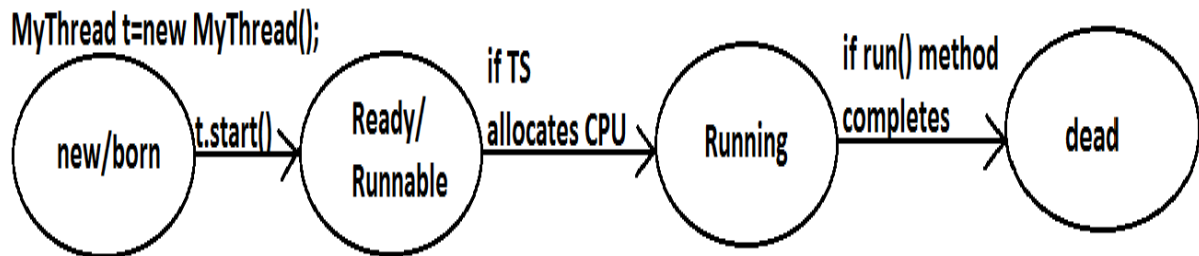
Note : It is never recommended to override start() method.

Case 7:Example 1:

|                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class MyThread extends Thread {     public void start()     {         System.out.println("start method");     }     public void run()     {         System.out.println("run method");     } } </pre> | <pre> class ThreadDemo {     public static void main(String[] args)     {         MyThread t=new MyThread();         t.start();         System.out.println("main method");     } } </pre> <p><u>output:</u><br/>main thread<br/>start method<br/>main method</p> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Example 2:

|                                                                                                                                                                                                                                   |                                                                                                                                                                                           |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class MyThread extends Thread {     public void start()     {         super.start();         System.out.println("start method");     }     public void run()     {         System.out.println("run method");     } } </pre> | <pre> class ThreadDemo {     public static void main(String[] args)     {         MyThread t=new MyThread();         t.start();         System.out.println("main method");     } } </pre> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Output:Case 8: life cycle of the Thread:Diagram:

- Once we created a Thread object then the Thread is said to be in new state or born state.
- Once we call start() method then the Thread will be entered into Ready or Runnable state.
- If Thread Scheduler allocates CPU then the Thread will be entered into running state.
- Once run() method completes then the Thread will entered into dead state.

Case 9:

After starting a Thread we are not allowed to restart the same Thread once again otherwise we will get runtime exception saying "IllegalThreadStateException".

Example:

```

MyThread t=new MyThread();
t.start();//valid
;;;;;;;;;
t.start();//we will get R.E saying: IllegalThreadStateException
  
```

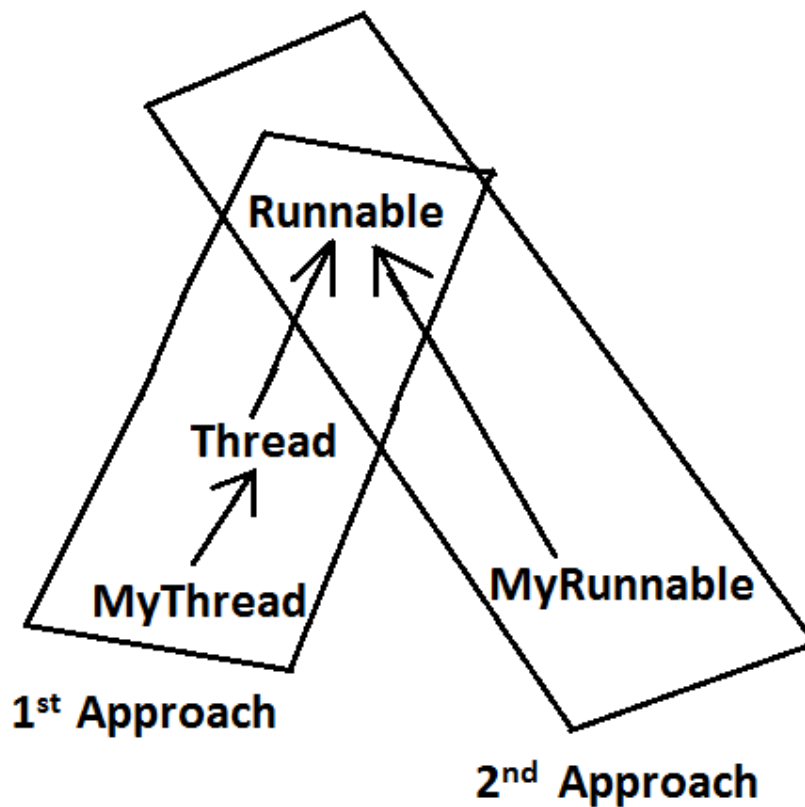


## Defining a Thread by implementing Runnable interface:

We can define a Thread even by implementing Runnable interface also.

Runnable interface present in java.lang.pkg and contains only one method run().

Diagram:



Example:

defining a  
Thread

```
class MyRunnable implements Runnable
{
    public void run()
    {
        for(int i=0;i<10;i++)
        {
            System.out.println("child Thread");
        }
    }
}
```

job of a Thread

```
class ThreadDemo
{
    public static void main(String[] args)
    {
        MyRunnable r=new MyRunnable();
        Thread t=new Thread(r);//here r is a Target Runnable
        t.start();

        for(int i=0;i<10;i++)
        {
            System.out.println("main thread");
        }
    }
}
```

Output:

```
main thread
main thread
main thread
main thread
main thread
main thread
main thread
main thread
main thread
main thread
child Thread
```

```
child Thread  
child Thread  
child Thread  
child Thread  
child Thread  
child Thread  
child Thread  
child Thread  
child Thread  
child Thread
```

We can't expect exact output but there are several possible outputs.

## Case study:

```
MyRunnable r=new MyRunnable();  
Thread t1=new Thread();  
Thread t2=new Thread(r);
```

### Case 1: t1.start():

A new Thread will be created which is responsible for the execution of Thread class run()method.

Output:

```
main thread  
main thread  
main thread  
main thread  
main thread
```

### Case 2: t1.run():

No new Thread will be created but Thread class run() method will be executed just like a normal method call.

Output:

```
main thread  
main thread  
main thread  
main thread  
main thread
```

### Case 3: t2.start():

New Thread will be created which is responsible for the execution of MyRunnable run() method.

Output:

```
main thread  
main thread  
main thread  
main thread  
main thread  
child Thread  
child Thread  
child Thread  
child Thread  
child Thread
```

**Case 4: t2.run():**

No new Thread will be created and MyRunnable run() method will be executed just like a normal method call.

Output:

```
child Thread
child Thread
child Thread
child Thread
child Thread
main thread
main thread
main thread
main thread
main thread
```

**Case 5: r.start():**

We will get compile time error saying start() method is not available in MyRunnable class.

Output:

Compile time error

E:\SCJP>javac ThreadDemo.java

ThreadDemo.java:18: cannot find symbol

Symbol: method start()

Location: class MyRunnable

**Case 6: r.run():**

No new Thread will be created and MyRunnable class run() method will be executed just like a normal method call.

Output:

```
child Thread
child Thread
child Thread
child Thread
child Thread
main thread
main thread
main thread
main thread
main thread
```

In which of the above cases a new Thread will be created which is responsible for the execution of MyRunnable run() method ?

t2.start();

In which of the above cases a new Thread will be created ?

t1.start();

t2.start();

In which of the above cases MyRunnable class run() will be executed ?

t2.start();

```
t2.run();
r.run();
```

### Best approach to define a Thread:

- Among the 2 ways of defining a Thread, implements Runnable approach is always recommended.
- In the 1st approach our class should always extends Thread class there is no chance of extending any other class hence we are missing the benefits of inheritance.
- But in the 2nd approach while implementing Runnable interface we can extend some other class also. Hence implements Runnable mechanism is recommended to define a Thread.

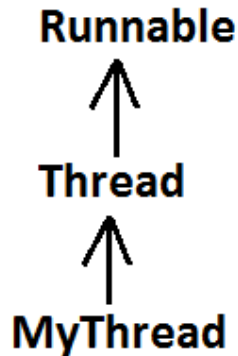
### Thread class constructors:

1. Thread t=new Thread();
2. Thread t=new Thread(Runnable r);
3. Thread t=new Thread(String name);
4. Thread t=new Thread(Runnable r,String name);
5. Thread t=new Thread(ThreadGroup g,String name);
6. Thread t=new Thread(ThreadGroup g,Runnable r);
7. Thread t=new Thread(ThreadGroup g,Runnable r,String name);
8. Thread t=new Thread(ThreadGroup g,Runnable r,String name,long stackSize);

### Ashok's approach to define a Thread(not recommended to use):

```
class MyThread extends Thread
{
    public void run()
    {
        System.out.println("run method");
    }
}
```

```
class ThreadDemo
{
    public static void main(String[] args)
    {
        MyThread t=new MyThread();
        Thread t1=new Thread(t);
        t1.start();
        System.out.println("main method");
    }
}
```

Diagram:

Output:  
main method  
run method

**Getting and setting name of a Thread:**

- Every Thread in java has some name it may be provided explicitly by the programmer or automatically generated by JVM.
- Thread class defines the following methods to get and set name of a Thread.

Methods:

1. `public final String getName()`
2. `public final void setName(String name)`

Example:

```

class MyThread extends Thread
{
}
class ThreadDemo
{
    public static void main(String[] args)
    {
        System.out.println(Thread.currentThread().getName()); //main
        MyThread t=new MyThread();
        System.out.println(t.getName()); //Thread-0
        Thread.currentThread().setName("Bhaskar Thread");

        System.out.println(Thread.currentThread().getName()); //Bhaskar
        Thread
    }
}
  
```

Note: We can get current executing Thread object reference by using `Thread.currentThread()` method.

**Thread Priorities**

- Every Thread in java has some priority it may be default priority generated by JVM (or) explicitly provided by the programmer.

- The valid range of Thread priorities is 1 to 10[but not 0 to 10] where 1 is the least priority and 10 is highest priority.
- Thread class defines the following constants to represent some standard priorities.
  1. Thread.MIN\_PRIORITY-----1
  2. Thread.MAX\_PRIORITY-----10
  3. Thread.NORM\_PRIORITY-----5
- There are no constants like Thread.LOW\_PRIORITY, Thread.HIGH\_PRIORITY
- Thread scheduler uses these priorities while allocating CPU.
- The Thread which is having highest priority will get chance for 1st execution.
- If 2 Threads having the same priority then we can't expect exact execution order it depends on Thread scheduler whose behavior is vendor dependent.
- We can get and set the priority of a Thread by using the following methods.
  1. public final int getPriority()
  2. public final void setPriority(int newPriority);//the allowed values are 1 to 10
- The allowed values are 1 to 10 otherwise we will get runtime exception saying "IllegalArgumentException".

### Default priority:

The default priority only for the main Thread is 5. But for all the remaining Threads the default priority will be inheriting from parent to child. That is whatever the priority parent has by default the same priority will be for the child also.

Example 1:

```
class MyThread extends Thread
{
}
class ThreadPriorityDemo
{
    public static void main(String[] args)
    {
        System.out.println(Thread.currentThread().getPriority());//5
        Thread.currentThread().setPriority(9);
        MyThread t=new MyThread();
        System.out.println(t.getPriority());//9
    }
}
```

Example 2:

```
class MyThread extends Thread
{
    public void run()
    {
        for(int i=0;i<10;i++)
        {
            System.out.println("child thread");
        }
    }
}
class ThreadPriorityDemo
{
    public static void main(String[] args)
    {
        MyThread t=new MyThread();
    }
}
```

```

        //t.setPriority(10);          //----> 1
        t.start();
        for(int i=0;i<10;i++)
        {
            System.out.println("main thread");
        }
    }
}

```

- If we are commenting line 1 then both main and child Threads will have the same priority and hence we can't expect exact execution order.
- If we are not commenting line 1 then child Thread has the priority 10 and main Thread has the priority 5 hence child Thread will get chance for execution and after completing child Thread main Thread will get the chance in this the output is:

Output:

```

child thread
child thread
child thread
child thread
child thread
child thread
child thread
child thread
child thread
child thread
child thread
main thread
main thread
main thread
main thread
main thread
main thread
main thread
main thread
main thread
main thread

```

Some operating systems(like windowsXP) may not provide proper support for Thread priorities. We have to install separate bats provided by vendor to provide support for priorities.

## The Methods to Prevent a Thread from Execution:

We can prevent(stop) a Thread execution by using the following methods.

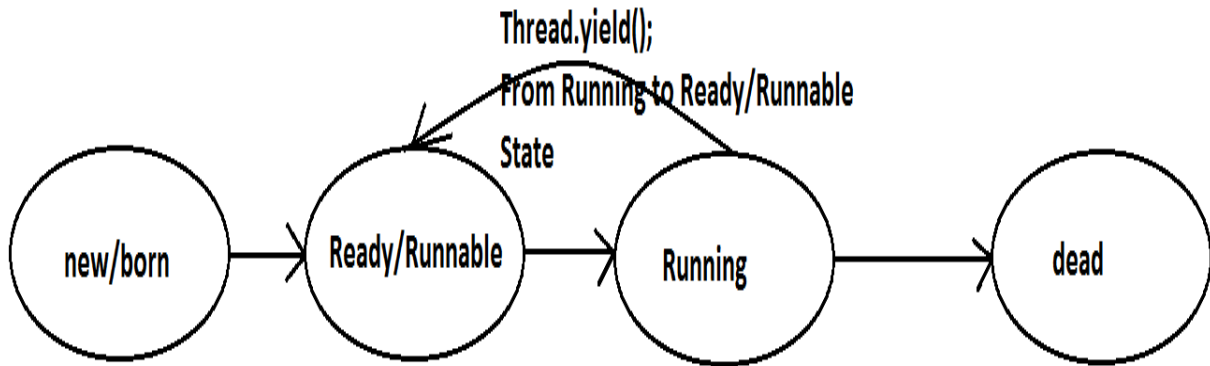
1. `yield();`
2. `join();`
3. `sleep();`

**yield():**

1. `yield()` method causes "to pause current executing Thread for giving the chance of remaining waiting Threads of same priority".



2. If all waiting Threads have the low priority or if there is no waiting Threads then the same Thread will be continued its execution.
3. If several waiting Threads with same priority available then we can't expect exact which Thread will get chance for execution.
4. The Thread which is yielded when it get chance once again for execution is depends on mercy of the Thread scheduler.
5. `public static native void yield();`

Diagram:

Example:

```

class MyThread extends Thread
{
    public void run()
    {
        for(int i=0;i<5;i++)
        {
            Thread.yield();
            System.out.println("child thread");
        }
    }
}
class ThreadYieldDemo
{
    public static void main(String[] args)
    {
        MyThread t=new MyThread();
        t.start();
        for(int i=0;i<5;i++)
        {
            System.out.println("main thread");
        }
    }
}
  
```

Output:

```

main thread
main thread
main thread
main thread
main thread
child thread
  
```

```
child thread  
child thread  
child thread  
child thread
```

In the above program child Thread always calling yield() method and hence main Thread will get the chance more number of times for execution.

Hence the chance of completing the main Thread first is high.

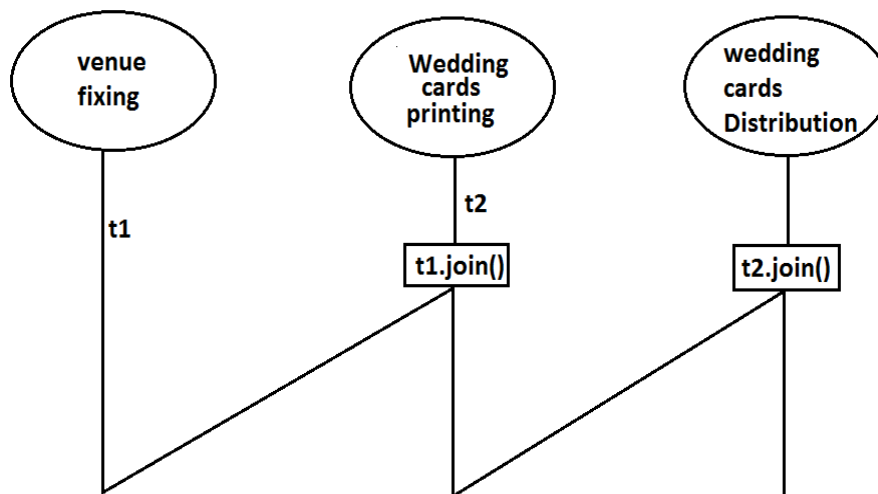
**Note :** Some operating systems may not provide proper support for yield() method.

### Join():

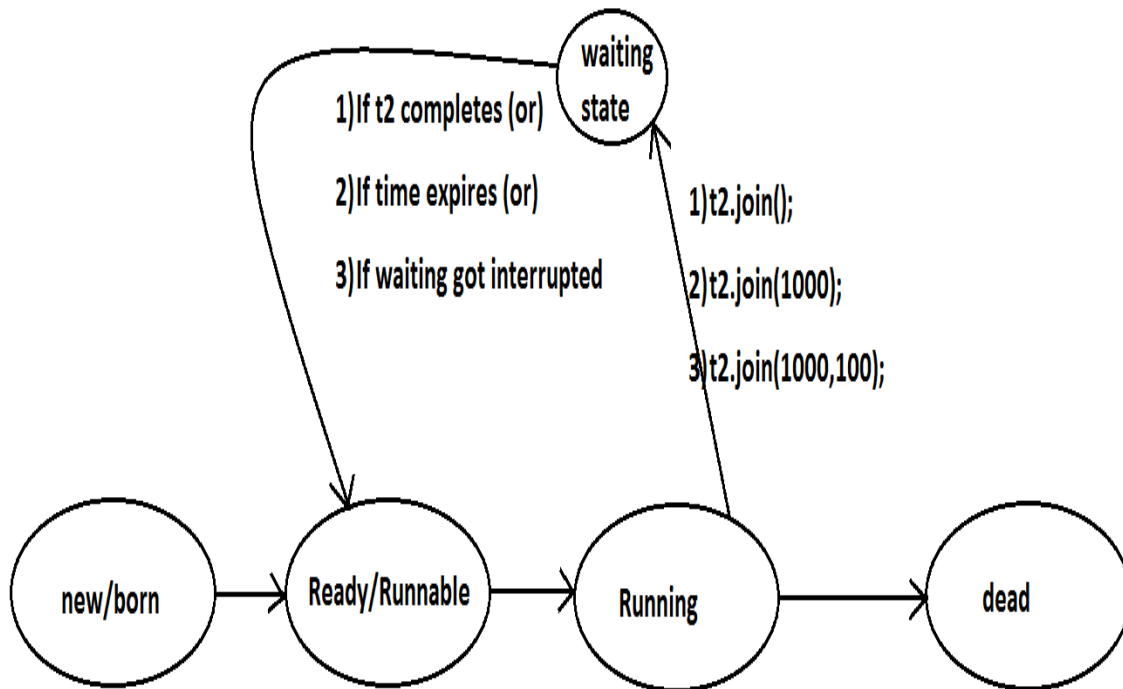
If a Thread wants to wait until completing some other Thread then we should go for join() method.

**Example:** If a Thread t1 executes t2.join() then t1 should go for waiting state until completing t2.

**Diagram:**



1. public final void join()throws InterruptedException
2. public final void join(long ms) throws InterruptedException
3. public final void join(long ms,int ns) throws InterruptedException

Diagram:

Every `join()` method throws `InterruptedException`, which is checked exception hence compulsory we should handle either by try catch or by throws keyword.

Otherwise we will get compiletime error.

Example:

```

class MyThread extends Thread
{
    public void run()
    {
        for(int i=0;i<5;i++)
        {
            System.out.println("Sita Thread");
            try
            {
                Thread.sleep(2000);
            }
            catch (InterruptedException e){}
        }
    }
}

class ThreadJoinDemo
{
    public static void main(String[] args)throws InterruptedException
    {

```

```

        MyThread t=new MyThread();
        t.start();
        //t.join();    //--->1
        for(int i=0;i<5;i++)
        {
            System.out.println("Rama Thread");
        }
    }
}

```

- If we are commenting line 1 then both Threads will be executed simultaneously and we can't expect exact execution order.
- If we are not commenting line 1 then main Thread will wait until completing child Thread in this the output is sita Thread 5 times followed by Rama Thread 5 times.

## Waiting of child Thread untill completing main Thread :

Example:

```

class MyThread extends Thread
{
    static Thread mt;
    public void run()
    {
        try
        {
            mt.join();
        }
        catch (InterruptedException e){}

        for(int i=0;i<5;i++)
        {
            System.out.println("Child Thread");
        }
    }
}
class ThreadJoinDemo
{
    public static void main(String[] args)throws InterruptedException
    {
        MyThread mt=Thread.currentThread();
        MyThread t=new MyThread();
        t.start();

        for(int i=0;i<5;i++)
        {
            Thread.sleep(2000);
            System.out.println("Main Thread");
        }
    }
}

```

Output :

```

Main Thread
Main Thread
Main Thread

```

Main Thread  
 Main Thread  
 Child Thread  
 Child Thread  
 Child Thread  
 Child Thread  
 Child Thread

**Note :**

If main thread calls join() on child thread object and child thread called join() on main thread object then both threads will wait for each other forever and the program will be hanged(like deadlock if a Thread class join() method on the same thread itself then the program will be hanged ).

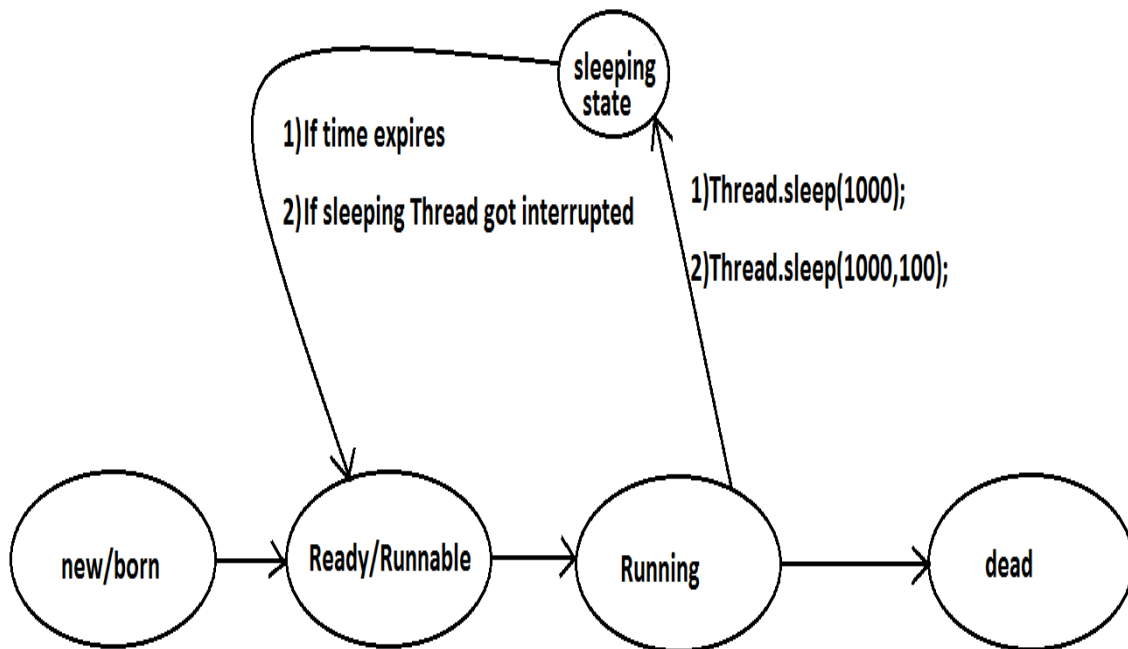
**Example :**

```
class ThreadDemo {
public static void main() throws InterruptedException {
  Thread.currentThread().join();
    -----
    main          main
}
}
```

### **Sleep() method:**

If a Thread don't want to perform any operation for a particular amount of time then we should go for sleep() method.

1. public static native void sleep(long ms) throws InterruptedException
2. public static void sleep(long ms,int ns)throws InterruptedException

Diagram:

Example:

```

class ThreadJoinDemo
{
    public static void main(String[] args) throws InterruptedException
    {
        System.out.println("M");
        Thread.sleep(3000);
        System.out.println("E");
        Thread.sleep(3000);
        System.out.println("G");
        Thread.sleep(3000);
        System.out.println("A");
    }
}

```

Output:

M  
E  
G  
A

Interrupting a Thread:

How a Thread can interrupt another thread ?

If a Thread can interrupt a sleeping or waiting Thread by using interrupt()(break off) method of Thread class.

public void interrupt();

Example:

class MyThread extends Thread

```

{
    public void run()
    {
        try
        {
            for(int i=0;i<5;i++)
            {
                System.out.println("i am lazy Thread :"+i);
                Thread.sleep(2000);
            }
        }
        catch (InterruptedException e)
        {
            System.out.println("i got interrupted");
        }
    }
}
class ThreadInterruptDemo
{
    public static void main(String[] args)
    {
        MyThread t=new MyThread();
        t.start();
        //t.interrupt();          //-->1
        System.out.println("end of main thread");
    }
}

```

- If we are commenting line 1 then main Thread won't interrupt child Thread and hence child Thread will be continued until its completion.
- If we are not commenting line 1 then main Thread interrupts child Thread and hence child Thread won't continued until its completion in this case the output is:

End of main thread  
 I am lazy Thread: 0  
 I got interrupted

Note:

- Whenever we are calling interrupt() method we may not see the effect immediately, if the target Thread is in sleeping or waiting state it will be interrupted immediately.
- If the target Thread is not in sleeping or waiting state then interrupt call will wait until target Thread will enter into sleeping or waiting state. Once target Thread entered into sleeping or waiting state it will effect immediately.
- In its lifetime if the target Thread never entered into sleeping or waiting state then there is no impact of interrupt call simply interrupt call will be wasted.

Example:

```

class MyThread extends Thread
{
    public void run()
    {
        for(int i=0;i<5;i++)
        {
            System.out.println("iam lazy thread");
        }
    }
}

```

```

        System.out.println("I'm entered into sleeping stage");
        try
        {
            Thread.sleep(3000);
        }
        catch (InterruptedException e)
        {
            System.out.println("i got interrupted");
        }
    }
}
class ThreadInterruptDemo1
{
    public static void main(String[] args)
    {
        MyThread t=new MyThread();
        t.start();
        t.interrupt();
        System.out.println("end of main thread");
    }
}

```

- In the above program interrupt() method call invoked by main Thread will wait until child Thread entered into sleeping state.
- Once child Thread entered into sleeping state then it will be interrupted immediately.

### Compression of yield, join and sleep() method?

| property                              | Yield()                                                                                                | Join()                                                                                   | Sleep()                                                                                                               |
|---------------------------------------|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| 1) Purpose?                           | To pause current executing Thread for giving the chance of remaining waiting Threads of same priority. | If a Thread wants to wait until completing some other Thread then we should go for join. | If a Thread don't want to perform any operation for a particular amount of time then we should go for sleep() method. |
| 2) Is it static?                      | yes                                                                                                    | no                                                                                       | yes                                                                                                                   |
| 3) Is it final?                       | no                                                                                                     | yes                                                                                      | no                                                                                                                    |
| 4) Is it overloaded?                  | No                                                                                                     | yes                                                                                      | yes                                                                                                                   |
| 5) Is it throws InterruptedException? | no                                                                                                     | yes                                                                                      | yes                                                                                                                   |
| 6) Is it native method?               | yes                                                                                                    | no                                                                                       | sleep(long ms) -->native<br>sleep(long ms,int ns) -->non-native                                                       |



## Synchronization

1. Synchronized is the keyword applicable for methods and blocks but not for classes and variables.
2. If a method or block declared as the synchronized then at a time only one Thread is allow to execute that method or block on the given object.
3. The main advantage of synchronized keyword is we can resolve date inconsistency problems.
4. But the main disadvantage of synchronized keyword is it increases waiting time of the Thread and effects performance of the system.
5. Hence if there is no specific requirement then never recommended to use synchronized keyword.
6. Internally synchronization concept is implemented by using lock concept.
7. Every object in java has a unique lock. Whenever we are using synchronized keyword then only lock concept will come into the picture.
8. If a Thread wants to execute any synchronized method on the given object 1st it has to get the lock of that object. Once a Thread got the lock of that object then it's allow to execute any synchronized method on that object. If the synchronized method execution completes then automatically Thread releases lock.
9. While a Thread executing any synchronized method the remaining Threads are not allowed execute any synchronized method on that object simultaneously. But remaining Threads are allowed to execute any non-synchronized method simultaneously. [lock concept is implemented based on object but not based on method].

Example:

```
class Display
{
    public synchronized void wish(String name)
    {
        for(int i=0;i<5;i++)
        {
            System.out.print("good morning:");
            try
            {
                Thread.sleep(1000);
            }
            catch (InterruptedException e)
            {}
            System.out.println(name);
        }
    }
}

class MyThread extends Thread
{
    Display d;
    String name;
    MyThread(Display d,String name)
    {
        this.d=d;
        this.name=name;
    }
    public void run()
    {
        d.wish(name);
    }
}
```

```

    }
}
class SynchronizedDemo
{
    public static void main(String[] args)
    {
        Display d1=new Display();
        MyThread t1=new MyThread(d1,"dhoni");
        MyThread t2=new MyThread(d1,"yuvaraj");
        t1.start();
        t2.start();
    }
}

```

If we are not declaring wish() method as synchronized then both Threads will be executed simultaneously and we will get irregular output.

Output:

```

good morning:good morning:yuvaraj
good morning:dhoni
good morning:yuvaraj
good morning:dhoni
good morning:yuvaraj
good morning:dhoni
good morning:yuvaraj
good morning:dhoni
good morning:yuvaraj
dhoni

```

If we declare wish() method as synchronized then the Threads will be executed one by one that is until completing the 1st Thread the 2nd Thread will wait in this case we will get regular output which is nothing but

Output:

```

good morning:dhoni
good morning:dhoni
good morning:dhoni
good morning:dhoni
good morning:dhoni
good morning:yuvaraj
good morning:yuvaraj
good morning:yuvaraj
good morning:yuvaraj
good morning:yuvaraj

```

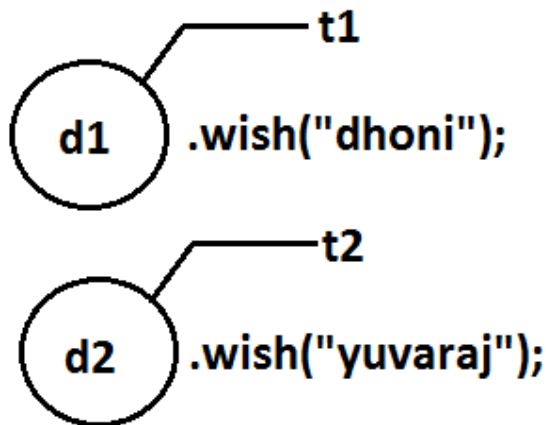
## Case study:

### Case 1:

```

Display d1=new Display();
Display d2=new Display();
MyThread t1=new MyThread(d1,"dhoni");
MyThread t2=new MyThread(d2,"yuvaraj");
t1.start();
t2.start();

```

Diagram:

Even though we declared wish() method as synchronized but we will get irregular output in this case, because both Threads are operating on different objects.

**Conclusion :** If multiple threads are operating on multiple objects then there is no impact of Synchronization.

If multiple threads are operating on same java objects then synchronized concept is required(applicable).

Class level lock:

1. Every class in java has a unique lock. If a Thread wants to execute a static synchronized method then it required class level lock.
2. Once a Thread got class level lock then it is allow to execute any static synchronized method of that class.
3. While a Thread executing any static synchronized method the remaining Threads are not allow to execute any static synchronized method of that class simultaneously.
4. But remaining Threads are allowed to execute normal synchronized methods, normal static methods, and normal instance methods simultaneously.
5. Class level lock and object lock both are different and there is no relationship between these two.

Synchronized block:

1. If very few lines of the code required synchronization then it's never recommended to declare entire method as synchronized we have to enclose those few lines of the code with in synchronized block.
2. The main advantage of synchronized block over synchronized method is it reduces waiting time of Thread and improves performance of the system.

**Example 1:** To get lock of current object we can declare synchronized block as follows.  
If Thread got lock of current object then only it is allowed to execute this block.  
`Synchronized(this){}`

**Example 2:** To get the lock of a particular object 'b' we have to declare a synchronized block as follows.  
If thread got lock of 'b' object then only it is allowed to execute this block.  
`Synchronized(b){}`

**Example 3:** To get class level lock we have to declare synchronized block as follows.  
`Synchronized(Display.class){}`  
If thread got class level lock of Display then only it allowed to execute this block.

**Note:**As the argument to the synchronized block we can pass either object reference or ".class file" and we can't pass primitive values as argument [because lock concept is dependent only for objects and classes but not for primitives].

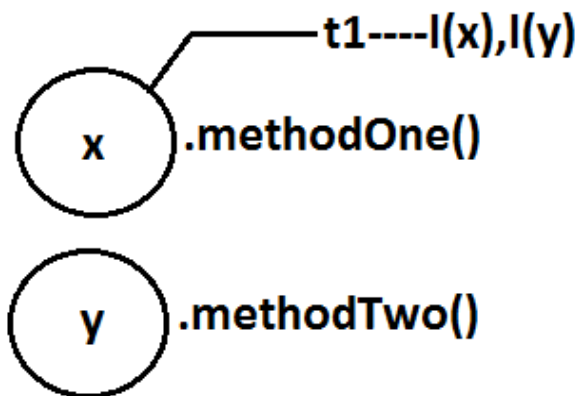
Example:  
`Int x=b;`  
`Synchronized(x){}`  
Output:  
Compile time error.  
Unexpected type.  
Found: int  
Required: reference

### **Questions:**

1. Explain about synchronized keyword and its advantages and disadvantages?
2. What is object lock and when a Thread required?
3. What is class level lock and when a Thread required?
4. What is the difference between object lock and class level lock?
5. While a Thread executing a synchronized method on the given object is the remaining Threads are allowed to execute other synchronized methods simultaneously on the same object?  
Ans: No.
6. What is synchronized block and explain its declaration?
7. What is the advantage of synchronized block over synchronized method?
8. Is a Thread can hold more than one lock at a time?  
Ans: Yes, up course from different objects. Example:

|                                                                                                                    |                                                                        |
|--------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| <pre> class X {     synchronized void methodOne()     {         Y y=new Y();         y.methodTwo();     } } </pre> | <pre> class Y {     synchronized void methodTwo()     {     } } </pre> |
|--------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|

Diagram:



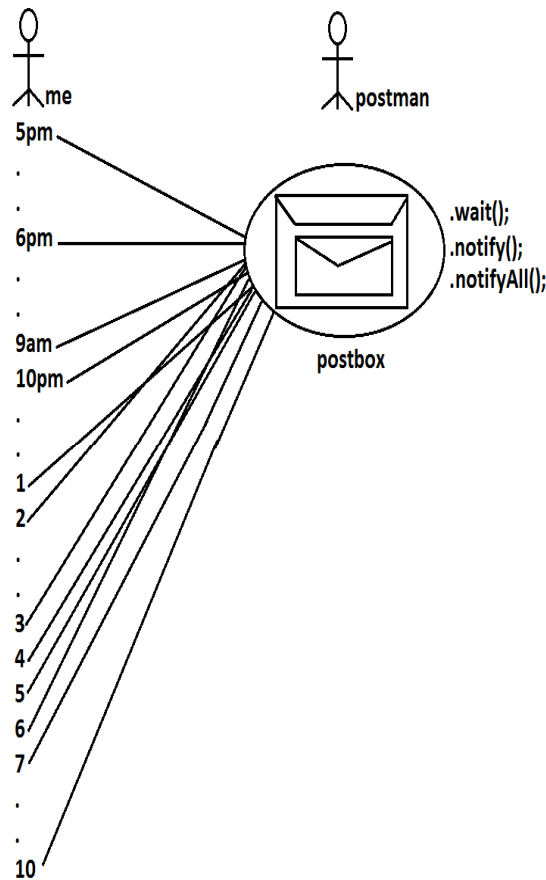
9. What is synchronized statement?

Ans: The statements which present inside synchronized method and synchronized block are called synchronized statements. [Interview people created terminology].

### Inter Thread communication (wait(), notify(), notifyAll()):

- Two Threads can communicate with each other by using wait(), notify() and notifyAll() methods.
- The Thread which is required updation it has to call wait() method on the required object then immediately the Thread will entered into waiting state. The Thread which is performing updation of object, it is responsible to give notification by calling notify() method. After getting notification the waiting Thread will get those updations.

Diagram:

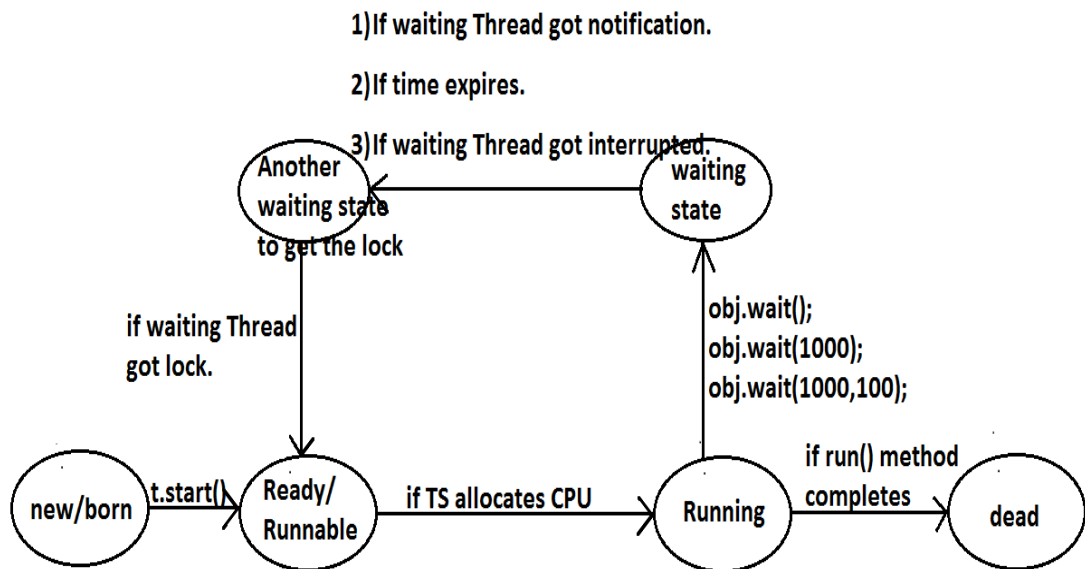


- `wait()`, `notify()` and `notifyAll()` methods are available in `Object` class but not in `Thread` class because `Thread` can call these methods on any common object.
- To call `wait()`, `notify()` and `notifyAll()` methods compulsory the current `Thread` should be owner of that object  
i.e., current `Thread` should has lock of that object  
i.e., current `Thread` should be in synchronized area. Hence we can call `wait()`, `notify()` and `notifyAll()` methods only from synchronized area otherwise we will get runtime exception saying `IllegalMonitorStateException`.
- Once a `Thread` calls `wait()` method on the given object 1st it releases the lock of that object immediately and entered into waiting state.
- Once a `Thread` calls `notify()` (or) `notifyAll()` methods it releases the lock of that object but may not immediately.
- Except these (`wait()`, `notify()`, `notifyAll()`) methods there is no other place(method) where the lock release will be happen.

| Method               | Is Thread Releases Lock? |
|----------------------|--------------------------|
| <code>yield()</code> | No                       |
| <code>join()</code>  | No                       |
| <code>sleep()</code> | No                       |
| <code>wait()</code>  | Yes                      |

|             |     |
|-------------|-----|
| notify()    | Yes |
| notifyAll() | Yes |

- Once a Thread calls wait(), notify(), notifyAll() methods on any object then it releases the lock of that particular object but not all locks it has.
  - public final void wait()throws InterruptedException
  - public final native void wait(long ms)throws InterruptedException
  - public final void wait(long ms,int ns)throws InterruptedException
  - public final native void notify()
  - public final void notifyAll()

Diagram:

## Example 1:

```

class ThreadA
{
    public static void main(String[] args)throws InterruptedException
    {
        ThreadB b=new ThreadB();
        b.start();
        synchronized(b)
        {
            System.out.println("main Thread calling wait() method");//step-1
            b.wait();
            System.out.println("main Thread got notification call");//step-4
            System.out.println(b.total);
        }
    }
}
class ThreadB extends Thread
{
    int total=0;
    public void run()
    {
        synchronized(this)
  
```

```

    {
        System.out.println("child thread starts calcuation");//step-2
        for(int i=0;i<=100;i++)
        {
            total=total+i;
        }
        System.out.println("child thread giving notification call");//step-
3        this.notify();
        }
    }
}

```

Output:

```

main Thread calling wait() method
child thread starts calculation
child thread giving notification call
main Thread got notification call
5050

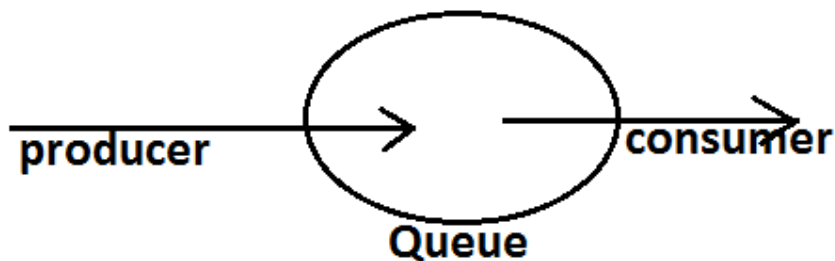
```

Example 2:

### Producer consumer problem:

- Producer(producer Thread) will produce the items to the queue and consumer(consumer thread) will consume the items from the queue. If the queue is empty then consumer has to call wait() method on the queue object then it will entered into waiting state.
- After producing the items producer Thread call notify() method on the queue to give notification so that consumer Thread will get that notification and consume items.

### Diagram:



### Example:



```

class Producer
{
    Producer()
    {
        synchronized(q)
        {
            produce items to the queue
            q.notify();
        }
    }
}

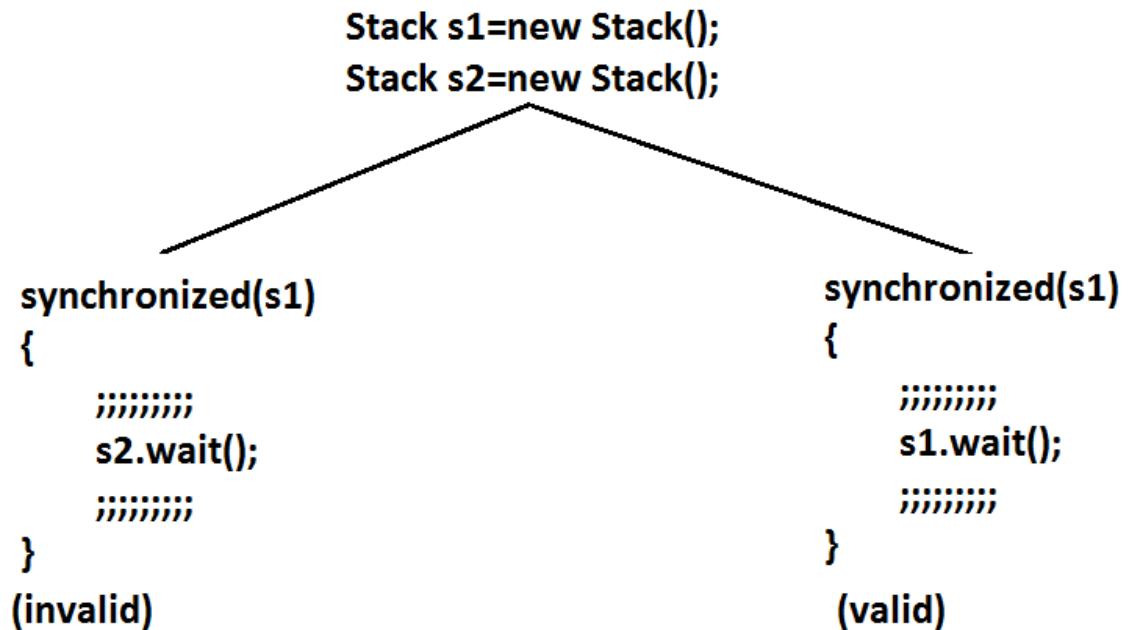
Consumer()
{
    synchronized(q)
    {
        if(q is empty)
        {
            q.wait();
        }
        else
            continue items;
    }
}

```

**Notify vs notifyAll():**

- We can use notify() method to give notification for only one Thread. If multiple Threads are waiting then only one Thread will get the chance and remaining Threads has to wait for further notification. But which Thread will be notify(inform) we can't expect exactly it depends on JVM.
- We can use notifyAll() method to give the notification for all waiting Threads. All waiting Threads will be notified and will be executed one by one, because they are required lock

**Note:** On which object we are calling wait(), notify() and notifyAll() methods that corresponding object lock we have to get but not other object locks.

Example:**R.E:IllegalMonitorStateException**Which of the following statements are True ?

1. Once a Thread calls wait() on any Object immediately it will entered into waiting state without releasing the lock ?  
NO
2. Once a Thread calls wait() on any Object it reduces the lock of that Object but may not immediately ?  
NO
3. Once a Thread calls wait() on any Object it immediately releases all locks whatever it has and entered into waiting state ?  
NO
4. Once a Thread calls wait() on any Object it immediately releases the lock of that particular Object and entered into waiting state ?  
YES
5. Once a Thread calls notify() on any Object it immediately releases the lock of that Object ?  
NO
6. Once a Thread calls notify() on any Object it releases the lock of that Object but may not immediately ?  
YES

## Dead lock:

- If 2 Threads are waiting for each other forever(without end) such type of situation(infinite waiting) is called dead lock.
- There are no resolution techniques for dead lock but several prevention(avoidance) techniques are possible.
- Synchronized keyword is the cause for deadlock hence whenever we are using synchronized keyword we have to take special care.

Example:

```
class A
{
    public synchronized void foo(B b)
    {
        System.out.println("Thread1 starts execution of foo()
method");
        try
        {
            Thread.sleep(2000);
        }
        catch (InterruptedException e)
        {}
        System.out.println("Thread1 trying to call b.last()");
        b.last();
    }
    public synchronized void last()
    {
        System.out.println("inside A, this is last()method");
    }
}
class B
{
    public synchronized void bar(A a)
    {
        System.out.println("Thread2 starts execution of bar() method");
        try
        {
            Thread.sleep(2000);
        }
        catch (InterruptedException e)
        {}
        System.out.println("Thread2 trying to call a.last()");
        a.last();
    }
    public synchronized void last()
    {
        System.out.println("inside B, this is last() method");
    }
}
class DeadLock implements Runnable
{
    A a=new A();
    B b=new B();
    DeadLock()
    {
        Thread t=new Thread(this);
        t.start();
        a.foo(b); //main thread
    }
}
```

```

    }
    public void run()
    {
        b.bar(a); //child thread
    }
    public static void main(String[] args)
    {
        new DeadLock(); //main thread
    }
}

```

Output:

```

Thread1 starts execution of foo() method
Thread2 starts execution of bar() method
Thread2 trying to call a.last()
Thread1 trying to call b.last()
//here cursor always waiting.

```

**Note :** If we remove atleast one synchronized keyword then we won't get DeadLock. Hence synchronized keyword is the only reason for DeadLock due to this while using synchronized keyword we have to handle carefully.

## Daemon Threads:

The Threads which are executing in the background are called daemon Threads. The main objective of daemon Threads is to provide support for non-daemon Threads like main Thread.

### Example:

#### Garbage collector

When ever the program runs with low memory the JVM will execute Garbage Collector to provide free memory. So that the main Thread can continue its execution.

- We can check whether the Thread is daemon or not by using `isDaemon()` method of Thread class.  
`public final boolean isDaemon();`
- We can change daemon nature of a Thread by using `setDaemon()` method.  
`public final void setDaemon(boolean b);`
- But we can change daemon nature before starting Thread only. That is after starting the Thread if we are trying to change the daemon nature we will get R.E saying *IllegalThreadStateException*.
- **Default Nature :** Main Thread is always non daemon and we can't change its daemon nature because it's already started at the beginning only.
- Main Thread is always non daemon and for the remaining Threads daemon nature will be inheriting from parent to child that is if the parent is daemon child is also daemon and if the parent is non daemon then child is also non daemon.
- Whenever the last non daemon Thread terminates automatically all daemon Threads will be terminated.

Example:

```

class MyThread extends Thread
{

```

```

}

class DaemonThreadDemo
{
    public static void main(String[] args)
    {
        System.out.println(Thread.currentThread().isDaemon());
        MyThread t=new MyThread();
        System.out.println(t.isDaemon());           1
        t.start();
        t.setDaemon(true);
        System.out.println(t.isDaemon());
    }
}

```

Output:

false

false

RE:IllegalThreadStateException

Example:

```

class MyThread extends Thread
{
    public void run()
    {
        for(int i=0;i<10;i++)
        {
            System.out.println("lazy thread");
            try
            {
                Thread.sleep(2000);
            }
            catch (InterruptedException e)
            {}
        }
    }
}

```

```

}
class DaemonThreadDemo
{
    public static void main(String[] args)
    {
        MyThread t=new MyThread();
        t.setDaemon(true);    //-->1
        t.start();
        System.out.println("end of main Thread");
    }
}

```

Output:

End of main Thread

- If we comment line 1 then both main & child Threads are non-Daemon , and hence both threads will be executed untill there completion.
- If we are not comment line 1 then main thread is non-Daemon and child thread is Daemon. Hence when ever main Thread terminates automatically child thread will be terminated.

### Lazy thread

- If we are commenting line 1 then both main and child Threads are non daemon and hence both will be executed until they completion.
- If we are not commenting line 1 then main Thread is non daemon and child Thread is daemon and hence whenever main Thread terminates automatically child Thread will be terminated.

### Deadlock vs Starvation:

- A long waiting of a Thread which never ends is called deadlock.
- A long waiting of a Thread which ends at certain point is called starvation.
- A low priority Thread has to wait until completing all high priority Threads.
- This long waiting of Thread which ends at certain point is called starvation.

### How to kill a Thread in the middle of the line?

- We can call stop() method to stop a Thread in the middle then it will be entered into dead state immediately.  
public final void stop();
- stop() method has been deprecated and hence not recommended to use.

### suspend and resume methods:

- A Thread can suspend another Thread by using suspend() method then that Thread will be paused temporarily.
- A Thread can resume a suspended Thread by using resume() method then suspended Thread will continue its execution.
  1. public final void suspend();
  2. public final void resume();
- Both methods are deprecated and not recommended to use.

### RACE condition:

Executing multiple Threads simultaneously and causing data inconsistency problems is nothing but Race condition

we can resolve race condition by using synchronized keyword.

### ThreadGroup:

Based on functionality we can group threads as a single unit which is nothing but ThreadGroup.

ThreadGroup provides a convenient way to perform common operations for all threads belongs to a particular group.

We can create a ThreadGroup by using the following constructors

ThreadGroup g=new ThreadGroup(String gName);

We can attach a Thread to the ThreadGroup by using the following constructor of Thread class

```
Thread t=new Thread(ThreadGroup g, String name);
```

```
ThreadGroup g=new ThreadGroup("Printing Threads");
MyThread t1=new MyThread(g,"Header Printing");
MyThread t2=new MyThread(g,"Footer Printing");
MyThread t3=new MyThread(g,"Body Printing");
-----
g.stop();
```

### ThreadLocal(1.2 v):

We can use ThreadLocal to define local resources which are required for a particular Thread like DBConnections, counterVariables etc.,

We can use ThreadLocal to define Thread scope like Servlet Scopes(page,request,session,application).

### GreenThread:

Java multiThreading concept is implementing by using the following 2 methods :

1. GreenThread Model
2. Native OS Model

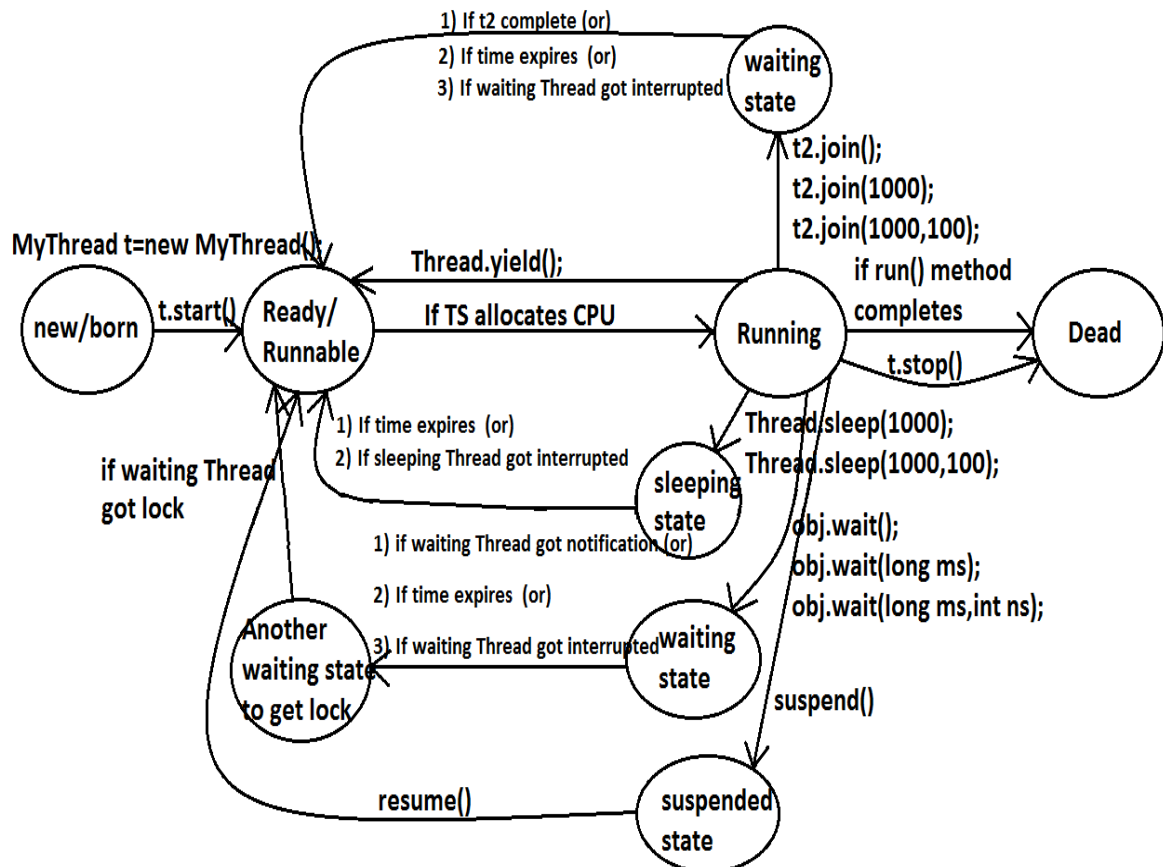
### **GreenThread Model**

The threads which are managed completely by JVM without taking support for underlying OS, such type of threads are called Green Threads.

### **Native OS Model**

- The Threads which are managed with the help of underlying OS are called Native Threads.
- Windows based OS provide support for Native OS Model
- Very few OS like SunSolaries provide support for GreenThread Model
- Anyway GreenThread model is deprecated and not recommended to use.

## Life cycle of a Thread:



What is the difference between extends Thread and implements Runnable?

1. Extends Thread is useful to override the public void run() method of Thread class.
2. Implements Runnable is useful to implement public void run() method of Runnable interface.

Extends Thread, implements Runnable which one is advantage?

If we extend Thread class, there is no scope to extend another class.

**Example:**

Class MyClass extends Frame,Thread//invalid

If we write implements Runnable still there is a scope to extend one more class.

**Example:**



1. class MyClass extends Thread implements Runnable
2. class MyClass extends Frame implements Runnable

**How can you stop a Thread which is running?**

**Step 1:** Declare a boolean type variable and store false in that variable.

`boolean stop=false;`

**Step 2:** If the variable becomes true return from the run() method.

`If(stop) return;`

**Step 3:** Whenever to stop the Thread store true into the variable.

`System.in.read();//press enter`

`Obj.stop=true;`

## Questions:

1. What is a Thread?
2. Which Thread by default runs in every java program?  
Ans: By default main Thread runs in every java program.
3. What is the default priority of the Thread?
4. How can you change the priority number of the Thread?
5. Which method is executed by any Thread?  
Ans: A Thread executes only public void run() method.
6. How can you stop a Thread which is running?
7. Explain the two types of multitasking?
8. What is the difference between a process and a Thread?
9. What is Thread scheduler?
10. Explain the synchronization of Threads?
11. What is the difference between synchronized block and synchronized keyword?
12. What is Thread deadlock? How can you resolve deadlock situation?
13. Which methods are used in Thread communication?
14. What is the difference between notify() and notifyAll() methods?
15. What is the difference between sleep() and wait() methods?
16. Explain the life cycle of a Thread?
17. What is daemon Thread?

# CORE JAVA

## With

# SCJP / OCJP

### Study Material

Chapter 8: Multi Threading Enhancements



**DURGA M.Tech**

(Sun certified & Realtime Expert)

# DURGA SOFTWARE SOLUTIONS

[www.durgasoft.com](http://www.durgasoft.com) Ph: 9246212143 8096969696

# Multi Threading Enhancements

8.1) ThreadGroup

8.2) ThreadLocal

8.3) java.util.concurrent.locks package

->Lock(l)

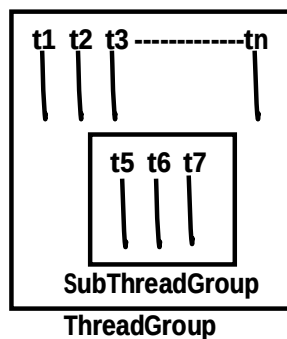
->ReentrantLock(C )

8.4) Thread Pools

8.5) Callable and Future

## ThreadGroup:

- Based on the Functionality we can Group Threads into a Single Unit which is Nothing but ThreadGroup i.e. ThreadGroup Represents a Set of Threads.
- In Addition a ThreadGroup can Also contains Other SubThreadGroups.



- ThreadGroup Class Present in java.lang Package and it is the Direct Child Class of Object.
- ThreadGroup provides a Convenient Way to Perform Common Operation for all Threads belongs to a Particular Group.

**Eg:** Stop All Consumer Threads.

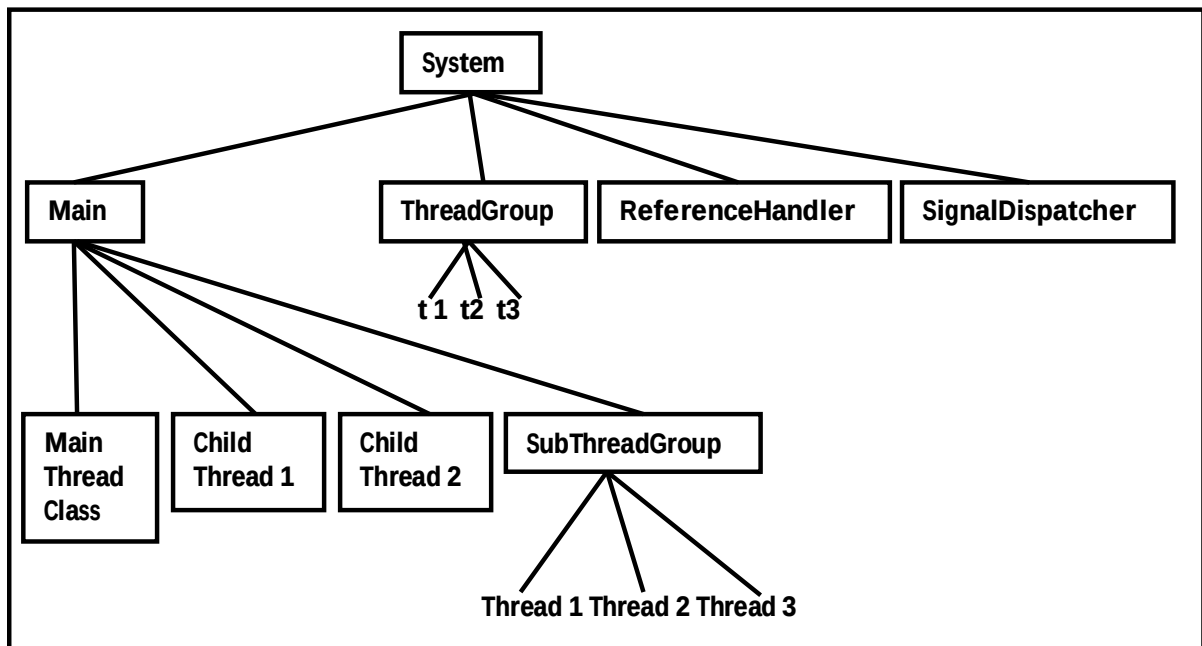
Suspend All Producer Threads.

**Constructors:**

- 1) `ThreadGroup g = new ThreadGroup(String gname);`
  - Creates a New ThreadGroup.
  - The Parent of this New Group is the ThreadGroup of Currently Running Thread.
- 2) `ThreadGroup g = new ThreadGroup(ThreadGroupppg, String gname);`
  - Creates a New ThreadGroup.
  - The Parent of this ThreadGroup is the specified ThreadGroup.

**Note:**

- In Java Every Thread belongs to Some Group.
- Every ThreadGroup is the Child Group of *System Group* either Directly OR Indirectly. Hence SystemGroup Acts as Root for all ThreadGroup's in Java.
- System ThreadGroup Represents System Level Threads Like ReferenceHandler, SignalDispatcher, Finalizer, AttachListener Etc.



```

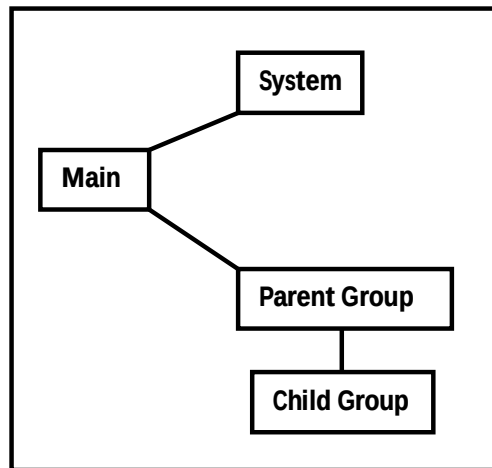
class ThreadGroupDemo {
public static void main(String[] args) {
    System.out.println(Thread.currentThread().getThreadGroup().getName());
    System.out.println(Thread.currentThread().getThreadGroup().getParent().getName());
    ThreadGroup pg = new ThreadGroup("Parent Group");
    System.out.println(pg.getParent().getName());
    ThreadGroup cg = new ThreadGroup(pg, "Child Group");
    System.out.println(cg.getParent().getName());
}
}

```

```

main
system
main
Parent Group

```



### Important Methods of ThreadGroup Class:

- 1) String getName(); Returns Name of the ThreadGroup.
- 2) int getMaxPriority(); Returns the Maximum Priority of ThreadGroup.
- 3) void setMaxPriority();
  - To Set Maximum Priority of ThreadGroup.
  - The Default Maximum Priority is 10.
  - Threads in the ThreadGroup that Already have Higher Priority, Not effected but Newly Added Threads this MaxPriority is Applicable.

```
class ThreadGroupDemo {  
    public static void main(String[] args) {  
        ThreadGroup g1 = new ThreadGroup("tg");  
        Thread t1 = new Thread(g1, "Thread 1");  
        Thread t2 = new Thread(g1, "Thread 2");  
        g1.setMaxPriority(3);  
        Thread t3 = new Thread(g1, "Thread 3");  
        System.out.println(t1.getPriority()); → 5  
        System.out.println(t2.getPriority()); → 5  
        System.out.println(t3.getPriority()); → 3  
    }  
}
```

- 4) **ThreadGroup.getParent():** Returns Parent Group of Current ThreadGroup.
- 5) **void list():** It Prints Information about ThreadGroup to the Console.
- 6) **int activeCount():** Returns Number of Active Threads Present in the ThreadGroup.
- 7) **int activeGroupCount():** It Returns Number of Active ThreadGroups Present in the Current ThreadGroup.
- 8) **int enumerate(Thread[] t):** To Copy All Active Threads of this Group into provided Thread Array. In this Case SubThreadGroup Threads also will be Considered.
- 9) **int enumerate(ThreadGroup[] g):** To Copy All Active SubThreadGroups into ThreadGroupArray.
- 10) **boolean isDaemon():**
- 11) **void setDaemon(boolean b):**
- 12) **void interrupt():** To Interrupt All Threads Present in the ThreadGroup.
- 13) **void destroy():** To Destroy ThreadGroup and its SubThreadGroups.

```

class MyThread extends Thread {
    MyThread(ThreadGroup g, String name) {
        super(g, name);
    }
    public void run() {
        System.out.println("Child Thread");
        try {
            Thread.sleep(2000);
        }
        catch (InterruptedException e) {}
    }
}

class ThreadGroupDemo {
    public static void main(String[] args) throws InterruptedException {
        ThreadGroup pg = new ThreadGroup("Parent Group");
        ThreadGroup cg = new ThreadGroup(pg, "Child Group");
        MyThread t1 = new MyThread(pg, "Child Thread 1");
        MyThread t2 = new MyThread(pg, "Child Thread 2");
        t1.start();
        t2.start();
        System.out.println(pg.activeCount());
        System.out.println(pg.activeGroupCount());
        pg.list();
        Thread.sleep(5000);
        System.out.println(pg.activeCount());
        pg.list();
    }
}

```

```

2
1
java.lang.ThreadGroup[name=Parent Group,maxpri=10]
Thread[Child Thread 1,5,Parent Group]
Thread[Child Thread 2,5,Parent Group]
java.lang.ThreadGroup[name=Child Group,maxpri=10]
Child Thread
Child Thread
0
java.lang.ThreadGroup[name=Parent Group,maxpri=10]
java.lang.ThreadGroup[name=Child Group,maxpri=10]

```

**Write a Program to Display All Thread Names belongs to System Group**

```

class ThreadGroupDemo {
    public static void main(String[] args) {
        ThreadGroup system = Thread.currentThread().getThreadGroup().getParent();
        Thread[] t = new Thread[system.activeCount()];
        system.enumerate(t);
        for (Thread t1: t) {
            System.out.println(t1.getName()+"-----"+t1.isDaemon());
        }
    }
}

```

```

Reference Handler-----true
Finalizer-----true
Signal Dispatcher-----true
Attach Listener-----true
main-----false

```

**ThreadLocal:**

- ThreadLocal Provides ThreadLocal Variables.
- ThreadLocal Class Maintains Values for Thread Basis.
- Each ThreadLocal Object Maintains a Separate Value Like userID, transactionID etc for Each Thread that Accesses that Object.
- Thread can Access its Local Value, can Manipulates its Value and Even can Remove its Value.
- In Every Part of the Code which is executed by the Thread we can Access its Local Variables.

**Eg:**

- Consider a Servlet which Calls Some Business Methods. we have a Requirement to generate a Unique transactionID for Each and Every Request and we have to Pass this transactionID to the Business Methods for Logging Purpose.
- For this Requirement we can Use ThreadLocal to Maintain a Separate transactionID for Every Request and for Every Thread.

**Note:**

- ☀ ThreadLocal Class introduced in 1.2 Version.
- ☀ ThreadLocal can be associated with Thread Scope.
- ☀ All the Code which is executed by the Thread has Access to Corresponding ThreadLocal Variables.
- ☀ A Thread can Access its Own Local Variables and can't Access Other Threads Local Variables.
- ☀ Once Thread Entered into Dead State All Local Variables are by Default Eligible for Garbage Collection.

**Constructor:** ThreadLocal tl = new ThreadLocal();  
Creates a ThreadLocal Variable.



**Methods:**

- 1) **Object get();** Returns the Value of ThreadLocal Variable associated with Current Thread.
- 2) **Object initialValue();**
  - Returns the initialValue of ThreadLocal Variable of Current Thread.
  - The Default Implementation of initialValue() Returns null.
  - To Customize Our initialValue we have to Override initialValue().
- 3) **void set(Object newValue);**To Set a New Value.
- 4) **void remove();**
  - To Remove the Current Threads Local Variable Value.
  - After Remove if we are trying to Access it will be reinitialized Once Again by invoking its initialValue().
  - This Method Newly Added in 1.5 Version.

```
class ThreadLocalDemo {
    public static void main(String[] args) {
        ThreadLocal tl = new ThreadLocal();
        System.out.println(tl.get()); //null
        tl.set("Durga");
        System.out.println(tl.get()); //Durga
        tl.remove();
        System.out.println(tl.get()); //null
    }
}
```

```
//Overriding of initialValue()
class ThreadLocalDemo {
    public static void main(String[] args) {
        ThreadLocal tl = new ThreadLocal() {
            protected Object initialValue() {
                return "abc";
            }
        };
        System.out.println(tl.get()); //abc
        tl.set("Durga");
        System.out.println(tl.get()); //Durga
        tl.remove();
        System.out.println(tl.get()); //abc
    }
}
```

```

class CustomerThread extends Thread {
    static Integer custID = 0;
    private static ThreadLocal tl = new ThreadLocal() {
        protected Integer initialValue() {
            return ++custID;
        }
    };
    CustomerThread(String name) {
        super(name);
    }
    public void run() {
        for (int i=0; i<5; i++) {
            SOP(Thread.currentThread().getName()+" Executing with Customer ID:"+tl.get());
        }
    }
}

class ThreadLocalDemo {
    public static void main(String[] args) {
        CustomerThread c1 = new CustomerThread("CustomerThread - 1");
        CustomerThread c2 = new CustomerThread("CustomerThread - 2");
        CustomerThread c3 = new CustomerThread("CustomerThread - 3");
        CustomerThread c4 = new CustomerThread("CustomerThread - 4");
        c1.start();
        c2.start();
        c3.start();
        c4.start();
    }
}

```

CustomerThread - 1 Executing with Customer ID:1  
 CustomerThread - 1 Executing with Customer ID:1  
 CustomerThread - 1 Executing with Customer ID:1  
 CustomerThread - 1 Executing with Customer ID:1  
 CustomerThread - 1 Executing with Customer ID:1

CustomerThread - 2 Executing with Customer ID:2  
 CustomerThread - 2 Executing with Customer ID:2  
 CustomerThread - 2 Executing with Customer ID:2  
 CustomerThread - 2 Executing with Customer ID:2  
 CustomerThread - 2 Executing with Customer ID:2

CustomerThread - 3 Executing with Customer ID:3  
 CustomerThread - 3 Executing with Customer ID:3  
 CustomerThread - 3 Executing with Customer ID:3  
 CustomerThread - 3 Executing with Customer ID:3  
 CustomerThread - 3 Executing with Customer ID:3

CustomerThread - 4 Executing with Customer ID:4  
 CustomerThread - 4 Executing with Customer ID:4  
 CustomerThread - 4 Executing with Customer ID:4  
 CustomerThread - 4 Executing with Customer ID:4  
 CustomerThread - 4 Executing with Customer ID:4

CustomerThread - 4 Executing with Customer ID:4

In the Above Program for Every Customer Thread a Separate customerID will be maintained by ThreadLocal Object.

### ThreadLocalVs Inheritance:

- Parent Threads ThreadLocal Variables are by Default Not Available to the Child Threads.
- If we want to Make Parent Threads Local Variables Available to Child Threads we should go for InheritableThreadLocal Class.
- It is the Child Class of ThreadLocal Class.
- By Default Child Thread Values are Same as Parent Thread Values but we can Provide Customized Values for Child Threads by Overriding childValue().

**Constructor:** `InheritableThreadLocal itl = new InheritableThreadLocal();`  
**InheritableThreadLocal** is the Child Class of **ThreadLocal** and Hence All Methods Present in **ThreadLocal** by Default Available to the **InheritableThreadLocal**.

**Method:** `public Object childValue(Object pvalue);`

```
class ParentThread extends Thread {
    public static InheritableThreadLocal itl = new InheritableThreadLocal() {
        public Object childValue(Object p) {
            return "cc";
        }
    };
    public void run() {
        itl.set("pp");
        System.out.println("Parent Thread --"+itl.get());
        ChildThread ct = new ChildThread();
        ct.start();
    }
    class ChildThread extends Thread {
        public void run() {
            System.out.println("Child Thread --"+ParentThread.itl.get());
        }
    }
}
class ThreadLocalDemo {
    public static void main(String[] args) {
        ParentThread pt = new ParentThread();
        pt.start();
    }
}
```

Parent Thread --pp  
 Child Thread --cc

## Java.util.concurrent.locks package:

### Problems with Traditional synchronized Key Word

- If a Thread Releases the Lock then which waiting Thread will get that Lock we are Not having any Control on this.
- We can't Specify Maximum waiting Time for a Thread to get Lock so that it will Wait until getting Lock, which May Effect Performance of the System and Causes Dead Lock.
- We are Not having any Flexibility to Try for Lock without waiting.
- There is No API to List All Waiting Threads for a Lock.
- The synchronized Key Word Compulsory we have to Define within a Method and it is Not Possible to Declare Over Multiple Methods.
- To Overcome Above Problems SUN People introduced *java.util.concurrent.locks* Package in 1.5 Version.
- It Also Provides Several Enhancements to the Programmer to Provide More Control on Concurrency.

### Lock(I):

- A Lock Object is Similar to Implicit Lock acquired by a Thread to Execute synchronized Method OR synchronized Block
- Lock Implementations Provide More Extensive Operations than Traditional Implicit Locks.

### Important Methods of Lock Interface

#### 1) void lock();

- It Locks the Lock Object.
- If Lock Object is Already Locked by Other Thread then it will wait until it is Unlocked.

#### 2) boolean tryLock();

- To Acquire the Lock if it is Available.
- If the Lock is Available then Thread Acquires the Lock and Returns true.
- If the Lock Unavailable then this Method Returns false and Continue its Execution.
- In this Case Thread is Never Blocked.

```
if (l.tryLock()) {  
    Perform Safe Operations  
}  
else {  
    Perform Alternative Operations  
}
```

**3) boolean tryLock(long time, TimeUnit unit);**

- To Acquire the Lock if it is Available.
- If the Lock is Unavailable then Thread can Wait until specified Amount of Time.
- Still if the Lock is Unavailable then Thread can Continue its Execution.

**Eg:** if (l.tryLock(1000, TimeUnit.SECONDS)) {}

**TimeUnit:** TimeUnit is an *enum* Present in *java.util.concurrent* Package.

```
enum TimeUnit {
    DAYS, HOURS, MINUTES, SECONDS, MILLI SECONDS, MICRO SECONDS, NANO SECONDS;
}
```

**4) void lockInterruptibly();**

Acquired the Lock Unless the Current Thread is Interrupted.

Acquires the Lock if it is Available and Returns Immediately.

If it is Unavailable then the Thread will wait while waiting if it is Interrupted then it won't get the Lock.

**5) void unlock(); To Release the Lock.****ReentrantLock**

- It implements Lock Interface and it is the Direct Child Class of an Object.
- Reentrant Means a Thread can acquires Same Lock Multiple Times without any Issue.
- Internally ReentrantLock Increments Threads Personal Count whenever we Call lock() and Decrements Counter whenever we Call unlock() and Lock will be Released whenever Count Reaches '0'.

**Constructors:****1) ReentrantLock rl = new ReentrantLock();**

Creates an Instance of ReentrantLock.

**2) ReentrantLock rl = new ReentrantLock(boolean fairness);**

- Creates an Instance of ReentrantLock with the Given Fairness Policy.
- If Fairness is true then Longest Waiting Thread can acquired Lock Once it is Available i.e. if follows First - In First - Out.
- If Fairness is false then we can't give any Guarantee which Thread will get the Lock Once it is Available.

**Note:** If we are Not specifying any Fairness Property then by Default it is Not Fair.

**Which of the following 2 Lines are Equal?**

```
ReentrantLockrl = new ReentrantLock(); ✓  
ReentrantLockrl = new ReentrantLock(true);  
ReentrantLockrl = new ReentrantLock(false); ✓
```

**Important Methods of ReentrantLock**

- 1) **void lock();**
- 2) **booleantryLock();**
- 3) **booleantryLock(long l, TimeUnit t);**
- 4) **void lockInterruptibly();**
- 5) **void unlock();**
  - To Release the Lock.
  - If the Current Thread is Not Owner of the Lock then we will get Runtime Exception Saying IllegalMonitorStateException.
- 6) **intgetHoldCount();** Returns Number of Holds on this Lock by Current Thread.
- 7) **booleanisHeldByCurrentThread();** Returns true if and Only if Lock is Hold by Current Thread.
- 8) **intgetQueueLength();** Returns the Number of Threads waiting for the Lock.
- 9) **Collection getQueuedThreads();** Returns a Collection containing Thread Objects which are waiting to get the Lock.
- 10) **booleanhasQueuedThreads();** Returns true if any Thread waiting to get the Lock.
- 11) **booleanisLocked();** Returns true if the Lock acquired by any Thread.
- 12) **booleanisFair();** Returns true if the Lock's Fairness Set to true.
- 13) **Thread getOwner();** Returns the Thread which acquires the Lock.

```
import java.util.concurrent.locks.ReentrantLock;
class Test {
    public static void main(String[] args) {
        ReentrantLock l = new ReentrantLock();
        l.lock();

        l.lock();
        System.out.println(l.isLocked()); //true
        System.out.println(l.isHeldByCurrentThread()); //true
        System.out.println(l.getQueueLength()); //0

        l.unlock();
        System.out.println(l.getHoldCount()); //1
        System.out.println(l.isLocked()); //true

        l.unlock();
        System.out.println(l.isLocked()); //false
        System.out.println(l.isFair()); //false
    }
}
```

```

import java.util.concurrent.locks.ReentrantLock;
class Display {
    ReentrantLock l = new ReentrantLock(true);
    public void wish(String name) {
        l.lock(); → 1
        for(int i=0; i<5; i++) {
            System.out.println("Good Morning");
            try {
                Thread.sleep(2000);
            }
            catch (InterruptedException e) {}
            System.out.println(name);
        }
        l.unlock(); → 2
    }
}
class MyThread extends Thread {
    Display d;
    String name;
    MyThread(Display d, String name) {
        this.d = d;
        this.name = name;
    }
    public void run() {
        d.wish(name);
    }
}
class ReentrantLockDemo {
    public static void main(String[] args) {
        Display d = new Display();
        MyThread t1 = new MyThread(d, "Dhoni");
        MyThread t2 = new MyThread(d, "Yuva Raj");
        MyThread t3 = new MyThread(d, "ViratKohli");
        t1.start();
        t2.start();
        t3.start();
    }
}

```

```

Good Morning
Dhoni
Good Morning
Dhoni
Good Morning
Dhoni
Good Morning
Dhoni
Good Morning
Dhoni
Good Morning
Yuva Raj
Good Morning
Yuva Raj
Good Morning
Yuva Raj
Good Morning
Yuva Raj
Good Morning
Yuva Raj
Good Morning
Yuva Raj
Good Morning
ViratKohli
Good Morning
ViratKohli
Good Morning
ViratKohli
Good Morning
ViratKohli
Good Morning
ViratKohli

```

If we Comment Both Lines 1 and 2 then All Threads will be executed Simultaneously and Hence we will get Irregular Output.

If we are Not Commenting then the Threads will be executed One by One and Hence we will get Regular Output



**Demo Program To Demonstrate tryLock();**

```
import java.util.concurrent.locks.ReentrantLock;
class MyThread extends Thread {
    static ReentrantLock l = new ReentrantLock();
    MyThread(String name) {
        super(name);
    }
    public void run() {
        if(l.tryLock()) {
            SOP(Thread.currentThread().getName()+" Got Lock and Performing Safe Operations");
            try {
                Thread.sleep(2000);
            }
            catch (InterruptedException e) {}
            l.unlock();
        }
        else {
            System.out.println(Thread.currentThread().getName()+" Unable To Get Lock
            and Hence Performing Alternative Operations");
        }
    }
}
class ReentrantLockDemo {
    public static void main(String args[]) {
        MyThread t1 = new MyThread("First Thread");
        MyThread t2 = new MyThread("Second Thread");
        t1.start();
        t2.start();
    }
}
```

First Thread Got Lock and Performing Safe Operations

Second Thread Unable To Get Lock and Hence Performing Alternative Operations

```

import java.util.concurrent.TimeUnit;
import java.util.concurrent.locks.ReentrantLock;
class MyThread extends Thread {
    static ReentrantLock l = new ReentrantLock();
    MyThread(String name) {
        super(name);
    }
    public void run() {
        do {
            try {
                if(l.tryLock(1000, TimeUnit.MILLISECONDS)) {
                    SOP(Thread.currentThread().getName()+"----- Got Lock");
                    Thread.sleep(5000);
                    l.unlock();
                    SOP(Thread.currentThread().getName()+"----- Releases Lock");
                    break;
                }
                else {
                    SOP(Thread.currentThread().getName()+"----- Unable To Get Lock And Will Try Again");
                }
            }
            catch (InterruptedException e) {}
        }
        while(true);
    }
}
class ReentrantLockDemo {
    public static void main(String args[]) {
        MyThread t1 = new MyThread("First Thread");
        MyThread t2 = new MyThread("Second Thread");
        t1.start();
        t2.start();
    }
}

```

```

First Thread----- Got Lock
Second Thread----- Unable To Get Lock And Will Try Again
Second Thread----- Unable To Get Lock And Will Try Again
Second Thread----- Unable To Get Lock And Will Try Again
Second Thread----- Unable To Get Lock And Will Try Again
Second Thread----- Got Lock
First Thread----- Releases Lock
Second Thread----- Releases Lock

```

## Thread Pools:

- Creating a New Thread for Every Job May Create Performance and Memory Problems.
- To Overcome this we should go for Thread Pool Concept.
- Thread Pool is a Pool of Already Created Threads Ready to do Our Job.
- Java 1.5 Version Introduces Thread Pool Framework to Implement Thread Pools.
- Thread Pool Framework is Also Known as Executor Framework.
- We can Create a Thread Pool as follows  
`ExecutorService service = Executors.newFixedThreadPool(3);` //Our Choice
- We can Submit a Runnable Job by using `submit()`.  
`service.submit(job);`
- We can Shutdown `ExecutorService` by using `shutdown()`.  
`service.shutdown();`

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
class PrintJob implements Runnable {
    String name;
    PrintJob(String name) {
        this.name = name;
    }
    public void run() {
        SOP(name+".....Job Started By Thread:" +Thread.currentThread().getName());
        try {
            Thread.sleep(10000);
        }
        catch (InterruptedException e) {}
        SOP(name+".....Job Completed By Thread:" +Thread.currentThread().getName());
    }
}
class ExecutorDemo {
    public static void main(String[] args) {
        PrintJob[] jobs = {
            new PrintJob("Durga"),
            new PrintJob("Ravi"),
            new PrintJob("Nagendra"),
            new PrintJob("Pavan"),
            new PrintJob("Bhaskar"),
            new PrintJob("Varma")
        };
        ExecutorService service = Executors.newFixedThreadPool(3);
        for (PrintJob job : jobs) {
            service.submit(job);
        }
        service.shutdown();
    }
}
```

**Output**

```
Durga.....Job Started By Thread:pool-1-thread-1
Ravi.....Job Started By Thread:pool-1-thread-2
Nagendra.....Job Started By Thread:pool-1-thread-3
Ravi.....Job Completed By Thread:pool-1-thread-2
Pavan.....Job Started By Thread:pool-1-thread-2
Durga.....Job Completed By Thread:pool-1-thread-1
Bhaskar.....Job Started By Thread:pool-1-thread-1
Nagendra.....Job Completed By Thread:pool-1-thread-3
Varma.....Job Started By Thread:pool-1-thread-3
Pavan.....Job Completed By Thread:pool-1-thread-2
Bhaskar.....Job Completed By Thread:pool-1-thread-1
Varma.....Job Completed By Thread:pool-1-thread-3
```

On the Above Program 3 Threads are Responsible to Execute 6 Jobs. So that a Single Thread can be reused for Multiple Jobs.

**Note:** Usually we can Use ThreadPool Concept to Implement Servers (Web Servers And Application Servers).

**Callable and Future:**

- In the Case of Runnable Job Thread won't Return anything.
- If a Thread is required to Return Some Result after Execution then we should go for Callable.
- Callable Interface contains Only One Method *public Object call() throws Exception*.
- If we Submit a Callable Object to Executor then the Framework Returns an Object of Type *java.util.concurrent.Future*
- The Future Object can be Used to Retrieve the Result from Callable Job.

```

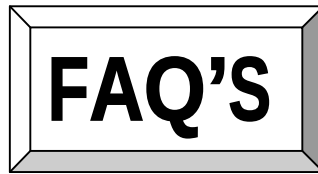
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

class MyCallable implements Callable {
    int num;
    MyCallable(int num) {
        this.num = num;
    }
    public Object call() throws Exception {
        int sum = 0;
        for(int i=0; i<num; i++) {
            sum = sum+i;
        }
        return sum;
    }
}

class CallableFutureDemo {
    public static void main(String args[]) throws Exception {
        MyCallable[] jobs = {
            new MyCallable(10),
            new MyCallable(20),
            new MyCallable(30),
            new MyCallable(40),
            new MyCallable(50),
            new MyCallable(60)
        };
        ExecutorService service = Executors.newFixedThreadPool(3);
        for(MyCallable job : jobs) {
            Future f = service.submit(job);
            System.out.println(f.get());
        }
        service.shutdown();
    }
}

```

45  
190  
435  
780  
1225  
1770



## FAQ'S

- 1) What Is Multi Tasking?
- 2) What Is Multi Threading And Explain Its Application Areas?
- 3) What Is Advantage Of Multi Threading?
- 4) When Compared With C++ What Is The Advantage In Java With Respect To Multi Threading?
- 5) In How Many Ways We Can Define A Thread?
- 6) Among Extending Thread And Implementing Runnable Which Approach Is Recommended?
- 7) Difference Between t.start() And t.run()?
- 8) Explain About Thread Scheduler?
- 9) If We Are Not Overriding run() What Will Happen?
- 10) Is It Possible Overloading Of run()?
- 11) Is It Possible To Override a start() And What Will Happen?
- 12) Explain Life Cycle Of A Thread?
- 13) What Is The Importance Of Thread Class start()?
- 14) After Starting A Thread If We Try To Restart The Same Thread Once Again What Will Happen?
- 15) Explain Thread Class Constructors?
- 16) How To Get And Set Name Of A Thread?
- 17) Who Uses Thread Priorities?
- 18) Default Priority For Main Thread?
- 19) Once We Create A New Thread What Is Its Priority?

- 20) How To Get Priority From Thread And Set Priority To A Thread?
- 21) If We Are Trying To Set Priority Of Thread As 100, What Will Happen?
- 22) If 2 Threads Having Different Priority Then Which Thread Will Get Chance First For Execution?
- 23) If 2 Threads Having Same Priority Then Which Thread Will Get Chance First For Execution?
- 24) How We Can Prevent Thread From Execution?
- 25) What Is yield() And Explain Its Purpose?
- 26) Is Join Is Overloaded?
- 27) Purpose Of sleep()?
- 28) What Is synchronized Key Word? Explain Its Advantages And Disadvantages?
- 29) What Is Object Lock And When It Is Required?
- 30) What Is A Class Level Lock When It Is Required?
- 31) While A Thread Executing Any Synchronized Method On The Given Object Is It Possible To Execute Remaining Synchronized Methods On The Same Object Simultaneously By Other Thread?
- 32) Difference Between Synchronized Method And Static Synchronized Method?
- 33) Advantages Of Synchronized Block Over Synchronized Method?
- 34) What Is Synchronized Statement?
- 35) How 2 Threads Will Communicate With Each Other?
- 36) wait(), notify(), notifyAll() Are Available In Which Class?
- 37) Why wait(), notify(), notifyAll() Methods Are Defined In Object Instead Of Thread Class?
- 38) Without Having The Lock Is It Possible To Call wait()?
- 39) If A Waiting Thread Gets Notification Then It Will Enter Into Which State?
- 40) In Which Methods Thread Can Release Lock?
- 41) Explain wait(), notify() and notifyAll()?

- 42) Difference Between notify() and notifyAll()?
- 43) Once A Thread Gives Notification Then Which Waiting Thread Will Get A Chance?
- 44) How A Thread Can Interrupt Another Thread?
- 45) What Is Deadlock? Is It Possible To Resolve Deadlock Situation?
- 46) Which Keyword Causes Deadlock Situation?
- 47) How We Can Stop A Thread Explicitly?
- 48) Explain About suspend() And resume()?
- 49) What Is Starvation And Explain Difference Between Deadlock and Starvation?
- 50) What Is Race Condition?
- 51) What Is Daemon Thread? Give An Example Purpose Of Daemon Thread?
- 52) How We Can Check Daemon Nature Of A Thread? Is It Possible To Change Daemon Nature Of A Thread? Is Main Thread Daemon OR Non-Daemon?
- 53) Explain About ThreadGroup?
- 54) What Is ThreadLocal?



# CORE JAVA

## With

# SCJP / OCJP

## Study Material

### Chapter 9 : InnerClasses



**DURGA M.Tech**

(Sun certified & Realtime Expert)

Ex. IBM Employee

Trained Lakhs of Students  
for last 14 years across INDIA

India's No.1 Software Training Institute

# DURGASOFT

**www.durgasoft.com Ph: 9246212143 ,8096969696**

## Inner Classes

### Agenda

1. Introduction.
2. Normal or Regular inner classes
  - Accessing inner class code from static area of outer class
  - Accessing inner class code from instance area of outer class
  - Accessing inner class code from outside of outer class
  - The applicable modifiers for outer & inner classes
  - Nesting of Inner classes
3. Method Local inner classes
4. Anonymous inner classes
  - Anonymous inner class that extends a class
  - Anonymous Inner Class that implements an interface
  - Anonymous Inner Class that define inside method arguments
  - Difference between general class and anonymous inner classes
  - Explain the application areas of anonymous inner classes ?
5. Static nested classes
  - Comparison between normal or regular class and static nested class ?
6. Various possible combinations of nested class & interfaces
  - class inside a class
  - interface inside a class
  - interface inside a interface
  - class inside a interface
7. Conclusions

## Introduction

- Sometimes we can declare a class inside another class such type of classes are called inner classes.

### Diagram:

- Sun people introduced inner classes in 1.1 version as part of "EventHandling" to resolve GUI bugs.
- But because of powerful features and benefits of inner classes slowly the programmers starts using in regular coding also.
- Without existing one type of object if there is no chance of existing another type of object then we should go for inner classes.

### Example:

Without existing University object there is no chance of existing Department object hence we have to define Department class inside University class.

Example1:

Example 2:

Without existing Bank object there is no chance of existing Account object hence we have to define Account class inside Bank class.

Example:

Example 3:

Without existing Map object there is no chance of existing Entry object hence Entry interface is define inside Map interface.

Map is a collection of key-value pairs, each key-value pair is called an Entry.

Example:

Diagram:

**Note :** Without existing Outer class Object there is no chance of existing Inner class Object.

Note: The relationship between outer class and inner class is not IS-A relationship and it is Has-A relationship.

Based on the purpose and position of declaration all inner classes are divided into 4 types. They are:

1. Normal or Regular inner classes
2. Method Local inner classes
3. Anonymous inner classes
4. Static nested classes.

## 1. Normal (or) Regular inner class:

If we are declaring any named class inside another class directly without static modifier such type of inner classes are called normal or regular inner classes.

Example:

```
class Outer
{
    class Inner
    {
    }
}
Output:
```

Example:

```
class Outer
{
    class Inner
    {
    }
    public static void main(String[] args)
    {
        System.out.println("outer class main method");
    }
}
Output:
```

- Inside inner class we can't declare static members. Hence it is not possible to declare main() method and we can't invoke inner class directly from the command prompt.

Example:

```
class Outer
```

```
{
    class Inner
    {
        public static void main(String[] args)
        {
            System.out.println("inner class main
method");
        }
    }
}
```

Output:

E:\scjp>javac Outer.java

Outer.java:5: inner classes cannot have static declarations  
public static void main(String[] args)

Accessing inner class code from static area of outer class:

Example:

```
class Outer
{
    class Inner
    {
        public void methodOne(){
            System.out.println("inner class method");
        }
    }
    public static void main(String[] args)
    {

    }
}
```

Accessing inner class code from instance area of outer class:

Example:

```
class Outer
{
    class Inner
    {
        public void methodOne()
        {
            System.out.println("inner class method");
        }
    }
    public void methodTwo()
    {
        Inner i=new Inner();
    }
}
```

```

        i.methodOne();
    }
    public static void main(String[] args)
    {
        Outer o=new Outer();
        o.methodTwo();
    }
}

```

Output:

E:\scjp>javac Outer.java

E:\scjp>java Outer

Inner class method

### Accessing inner class code from outside of outer class:

#### Example:

```

class Outer
{
    class Inner
    {
        public void methodOne()
        {
            System.out.println("inner class method");
        }
    }
}
class Test
{
    public static void main(String[] args)
    {
        new Outer().new Inner().methodOne();
    }
}

```

Output:

Inner class method

- From inner class we can access all members of outer class (both static and non-static, private and non private methods and variables) directly.

#### Example:

```

class Outer
{
    int x=10;
    static int y=20;
    class Inner{

```

```
        public void methodOne()
        {
            System.out.println(x);//10
            System.out.println(y);//20
        }
    }
    public static void main(String[] args)
    {
        new Outer().new Inner().methodOne();
    }
}
```

- Within the inner class "this" always refers current inner class object. To refer current outer class object we have to use "outer class name.this".

**Example:**

```
class Outer
{
    int x=10;
    class Inner
    {
        int x=100;
        public void methodOne()
        {
            int x=1000;
            System.out.println(x);//1000
            System.out.println(this.x);//100
            System.out.println(Outer.this.x);//10
        }
    }
    public static void main(String[] args)
    {
        new Outer().new Inner().methodOne();
    }
}
```

**The applicable modifiers for outer classes are:**

1. public
2. default
3. final
4. abstract
5. strictfp

But for the inner classes in addition to this the following modifiers also allowed.

Diagram:

Nesting of Inner classes :

We can declare an inner class inside another inner class

### Method local inner classes:

- Sometimes we can declare a class inside a method such type of inner classes are called method local inner classes.
- The main objective of method local inner class is to define method specific repeatedly required functionality.
- Method Local inner classes are best suitable to meet nested method requirement.
- We can access method local inner class only within the method where we declared it. That is from outside of the method we can't access. As the scope of method local inner classes is very less, this type of inner classes are most rarely used type of inner classes.

Example:

```
class Test
{
    public void methodOne()
    {
        class Inner
        {
            public void sum(int i,int j)
            {
                System.out.println("The sum:"+(i+j));
            }
        }
        Inner i=new Inner();
        i.sum(10,20);
        ///////////////
        i.sum(100,200);
        ///////////////
        i.sum(1000,2000);
        ///////////////
    }
    public static void main(String[] args)
    {
```



```

        new Test().methodOne();
    }
}

```

Output:

```

The sum: 30
The sum: 300
The sum: 3000

```

- If we are declaring inner class inside instance method then we can access both static and non static members of outer class directly.
- But if we are declaring inner class inside static method then we can access only static members of outer class directly and we can't access instance members directly.

Example:

```

class Test
{
    int x=10;
    static int y=20;
    public void methodOne()
    {
        class Inner
        {
            public void methodTwo()
            {
                System.out.println(x);//10
                System.out.println(y);//20
            }
        }
        Inner i=new Inner();
        i.methodTwo();
    }
    public static void main(String[] args)
    {
        new Test().methodOne();
    }
}

```

- If we declare methodOne() method as static then we will get compile time error saying "non-static variable x cannot be referenced from a static context".
- From method local inner class we can't access local variables of the method in which we declared it. But if that local variable is declared as final then we won't get any compile time error.

Example:

```

class Test
{
    int x=10;
    public void methodOne()

```

```

    {
        int y=20;
        class Inner
        {
            public void methodTwo()
            {
                System.out.println(x); //10
                System.out.println(y); //C.E: local
            }
        }
    }
    Inner i=new Inner();
    i.methodTwo();
}
public static void main(String[] args)
{
    new Test().methodOne();
}

```

variable y is accessed from within inner class; needs to be declared final.

- If we declared y as final then we won't get any compile time error.
- Consider the following declaration.

```

class Test
{
    int i=10;
    static int j=20;
    public void methodOne()
    {
        int k=30;
        final int l=40;
        class Inner
        {
            public void methodTwo()
            {
                System.out.println(i);
                System.out.println(j); //-->line 1
                System.out.println(k);
                System.out.println(l);
            }
        }
        Inner i=new Inner();
        i.methodTwo();
    }
    public static void main(String[] args)
    {
        new Test().methodOne();
    }
}

```

```

    }
}

```

At line 1 which of the following variables we can access ?

If we declare methodOne() method as static then which variables we can access at line 1 ?

- If we declare methodTwo() as static then we will get compile time error because we can't declare static members inside inner classes.
- The only applicable modifiers for method local inner classes are:
  1. final
  2. abstract
  3. strictfp
- By mistake if we are declaring any other modifier we will get compile time error.

### Anonymous inner classes:

- Sometimes we can declare inner class without name such type of inner classes are called anonymous inner classes.
- The main objective of anonymous inner classes is "just for instant use".
- There are 3 types of anonymous inner classes
  1. Anonymous inner class that extends a class.
  2. Anonymous inner class that implements an interface.
  3. Anonymous inner class that defined inside method arguments.

Anonymous inner class that extends a class:

```

class PopCorn
{
    public void taste()
    {
        System.out.println("spicy");
    }
}
class Test
{
    public static void main(String[] args)
    {
        PopCorn p=new PopCorn()
        {

```

```

        public void taste()
        {
            System.out.println("salty");
        }
    };
    p.taste();//salty
    PopCorn p1=new PopCorn()
    p1.taste();//spicy
}
}

```

**Analysis:**

1. PopCorn p=new PopCorn();  
We are just creating a PopCorn object.
  2. PopCorn p=new PopCorn()
  3. {
  4. };
  5. We are creating child class without name for the PopCorn class and for that child class we are creating an object with Parent PopCorn reference.
  6. PopCorn p=new PopCorn()
  7. {
  8.       public void taste()
  9.       {
  10.             System.out.println("salty");
  11.       }
  12.     };
  - 13.
1. We are creating child class for PopCorn without name.
  2. We are overriding taste() method.
  3. We are creating object for that child class with parent reference.

**Note:** Inside Anonymous inner classes we can take or declare new methods but outside of anonymous inner classes we can't call these methods directly because we are depending on parent reference.[parent reference can be used to hold child class object but by using that reference we can't call child specific methods]. These methods just for internal purpose only.

**Example 1:**

```

class PopCorn
{
    public void taste()
    {
        System.out.println("spicy");
    }
}
class Test
{
    public static void main(String[] args)

```

```

    {
        PopCorn p=new PopCorn()
        {
            public void taste()
            {
                methodOne();//valid call(internal
purpose)
                System.out.println("salty");
            }
            public void methodOne()
            {
                System.out.println("child specific
method");
            }
        };
        //p.methodOne();//here we can not call(outside
inner class)
        p.taste();//salty
        PopCorn p1=new PopCorn();
        p1.taste();//spicy
    }
}

```

Output:

Child specific method

Salty

Spicy

Example 2:

```

class Test
{
    public static void main(String[] args)
    {
        Thread t=new Thread()
        {
            public void run()
            {
                for(int i=0;i<10;i++)
                {
                    System.out.println("child
thread");
                }
            }
        };
        t.start();
        for(int i=0;i<10;i++)
        {
            System.out.println("main thread");
        }
    }
}

```

**Anonymous Inner Class that implements an interface:****Example:**

```

class InnerClassesDemo
{
    public static void main(String[] args)
    {
        Runnable r=new Runnable() //here we are not
creating for                      Runnable interface, we are
creating                          implements class object.
        {
            public void run()
            {
                for(int i=0;i<10;i++)
                {
                    System.out.println("Child
thread");
                }
            }
        };
        Thread t=new Thread(r);
        t.start();
        for(int i=0;i<10;i++)
        {
            System.out.println("Main thread");
        }
    }
}

```

**Anonymous Inner Class that define inside method arguments:****Example:**

```

class Test
{
    public static void main(String[] args)
    {
        new Thread(
            new Runnable()
            {
                public void run()
                {
                    for(int i=0;i<10;i++)
                    {

```

```

        System.out.println("child
thread");
    }
    }
    }).start();
    for(int i=0;i<10;i++)
    {
        System.out.println("main thread");
    }
}

```

Output:

- This output belongs to example 2, anonymous inner class that implements an interface example and anonymous inner class that define inside method arguments example.

```

Main thread
Main thread
Main thread
Main thread
Main thread
Main thread
Main thread
Main thread
Main thread
Main thread
Main thread
Child thread
Child thread
Child thread
Child thread
Child thread
Child thread
Child thread
Child thread
Child thread
Child thread

```

### Difference between general class and anonymous inner classes:

| General Class                                                         | Anonymous Inner Class                                                                        |
|-----------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| 1) A general class can extends only one class at a time.              | 1) Ofcourse anonymous inner class also can extends only one class at a time.                 |
| 2) A general class can implement any no. Of interfaces at a time.     | 2) But anonymous inner class can implement only one interface at a time.                     |
| 3) A general class can extends a class and can implement an interface | 3) But anonymous inner class can extends a class or can implements an interface but not both |

|                                                                                     |                                                                                                               |
|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| simultaneously.                                                                     | simultaneously.                                                                                               |
| 4) In normal Java class we can write constructor because we know name of the class. | 4) But in anonymous inner class we can't write constructor because anonymous inner class not having any name. |

**Explain the application areas of anonymous inner classes ?**

anonymous inner classes are best suitable to define call back functions in GUI components

```
import java.awt.*;
import java.awt.event.*;

public class AnonymousInnerClassDemo {
    public static void main(String args[]) {
        Frame f=new Frame();

        f.addWindowListener(new WindowAdaptor(){
            public void windowClosing(WindowEvent e) {
                System.exit(0);
            }
        });

        f.add(new Label("Anonymous Inner class Demo !!!"));
        f.setSize(500,500);
        f.setVisible(true);
    }
}
```

**Without Anonumous Inner class :**

```
class GUI extends JFrame implements ActionListener {
    JButton b1,b2,b3,b4;
    -----
    public void actionPerformed(ActionEvent e) {
        if(e.getSource()==b1) {
            //perform b1 specific functionality
        }
        else if(e.getSource()==b2){
            //perform b2 specific functionality
        }

        -----
    }
    -----
}
```



With Anonumous Inner class :

```

class GUI extends JFrame {
    JButton b1,b2,b3,b4 ;
    -----

    b1.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            //perform b1 specific functionality
        }
    });

    b2.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            //perform b2 specific functionality
        }
    });

    -----
}

```

**Static nested classes:**

- Sometimes we can declare inner classes with static modifier such type of inner classes are called static nested classes.
- In the case of normal or regular inner classes without existing outer class object there is no chance of existing inner class object.  
i.e., inner class object is always strongly associated with outer class object.
- But in the case of static nested class without existing outer class object there may be a chance of existing static nested class object.  
i.e., static nested class object is not strongly associated with outer class object.

Example:

```

class Test
{
    static class Nested
    {
        public void methodOne()
        {
            System.out.println("nested class method");
        }
    }
    public static void main(String[] args)

```

```

    {
        Test.Nested t=new Test.Nested();
        t.methodOne();
    }
}

```

- Inside static nested classes we can declare static members including main() method also. Hence it is possible to invoke static nested class directly from the command prompt.

**Example:**

```

class Test
{
    static class Nested
    {
        public static void main(String[] args)
        {
            System.out.println("nested class main
method");
        }
    }
    public static void main(String[] args)
    {
        System.out.println("outer class main method");
    }
}

```

**Output:**

```

E:\SCJP>javac Test.java
E:\SCJP>java Test
Outer class main method
E:\SCJP>java Test$Nested
Nested class main method

```

- From the normal inner class we can access both static and non static members of outer class but from static nested class we can access only static members of outer class.

**Example:**

```

class Test
{
    int x=10;
    static int y=20;
    static class Nested
    {
        public void methodOne()
        {
            System.out.println(x); //C.E:non-static
variable x                                cannot be referenced
from a static context

```

```

        System.out.println(y);
    }
}

```

### Compression between normal or regular class and static nested class ?

| Normal /regular inner class                                                                                                                                        | Static nested class                                                                                                                                                                |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1) Without existing outer class object there is no chance of existing inner class object. That is inner class object is always associated with outer class object. | 1) Without existing outer class object there may be a chance of existing static nested class object. That is static nested class object is not associated with outer class object. |
| 2) Inside normal or regular inner class we can't declare static members.                                                                                           | 2) Inside static nested class we can declare static members.                                                                                                                       |
| 3) Inside normal inner class we can't declare main() method and hence we can't invoke regular inner class directly from the command prompt.                        | 3) Inside static nested class we can declare main() method and hence we can invoke static nested class directly from the command prompt.                                           |
| 4) From the normal or regular inner class we can access both static and non static members of outer class directly.                                                | 4) From static nested class we can access only static members of outer class directly.                                                                                             |

### Various possible combinations of nested class & interfaces :

#### 1. class inside a class :

- We can declare a class inside another class
- Without existing one type of object, if there is no chance of existing another type of object, then we should go for class inside a class

```

class University {
    class Department {
    }
}

```

Without existing University object, there is no chance of existing Department object. i.e., Department object is always associated with University

#### 2. interface inside a class :

We can declare interface inside a class

```
class X {
    interface Y {
    }
}
```

Inside class if we required multiple implements of an interface and these implementations of relevant to a particular class, then we should declare interface inside a class.

```
class VehicleType {
    interface Vehicle {
        public int getNoOfWheels();
    }

    class Bus implements Vehicle {
        public int getNoOfWheels() {
            return 6;
        }
    }

    class Auto implements Vehicle {
        public int getNoOfWheels() {
            return 3;
        }
    }
}
```

### 3. interface inside a interface :

We can declare an interface inside another interface.

```
interface Map {
    interface Entry {
        public Object getKey();
        public Object getValue();
        public Object getValue(Object new );
    }
}
```

Nested interfaces are always public,static whether we are declaring or not. Hence we can implements inner interface directly with out implementing outer interface.

```
interface Outer {
    public void methodOne();
    interface Inner {
        public void methodTwo();
    }
}

class Test implements Outer.Inner {
    public void methodTwo() {
```

```

    System.out.println("Inner interface method");
}
public static void main(String args[]) {
    Test t=new Test();
    t.methodTwo();
}
}

```

Whenever we are implementing Outer interface , it is not required to implement Inner interfaces.

```

class Test implements Outer {
    public void methodOne() {
        System.out.println("Outer interface method ");
    }
    public static void main(String args[]) {
        Test t=new Test();
        t.methodOne();
    }
}

```

i.e., Both Outer and Inner interfaces we can implement independently.

#### 4. class inside a interface :

We can declare a class inside interface. If a class functionality is closely associated with the use interface then it is highly recommended to declare class inside interface

Example:

```

interface EmailServer {
    public void sendEmail(EmailDetails e);

    class EmailDetails {
        String from;
        String to;
        String subject;
    }
}

```

In the above example Emaildetails functionality is required for EmailService and we are not using anywhere else . Hence we can declare EmailDetails class inside EmailService interface .

We can also declare a class inside interface to provide default implementation for that interface.

Example :

```

interface Vehicle {
    public int getNoOfWheels();
}

```

```
class DefaultVehicle implements Vehicle {
    public int getNoOfWheels() {
        return 3;
    }
}

class Bus implements Vehicle {
    public int getNoOfWheels() {
        return 6;
    }
}

class Test {
    public static void main(String args[]) {
        Bus b=new Bus();
        System.out.println(b.getNoOfWheels());

        Vehicle.DefaultVehicle d=new Vehicle.DefaultVehicle();
        System.out.println(d.getNoOfWheels());
    }
}
```

In the above example DefaultVehicle in the default implementation of Vehicle interface where as Bus customized implementation of Vehicle interface.

The class which is declared inside interface is always static ,hence we can create object directly without having outer interface type object.

### **Conclusions :**

1. We can declare anything inside any thing with respect to classes and interfaces.
2. Nesting interfaces are always public, static whether we are declaring or not.
3. class which is declared inside interface is always public,static whether we are declaring or not.

# CORE JAVA

## With

# SCJP / OCJP

### Study Material

Chapter 10 : Java.lang Package



**DURGA M.Tech**

(Sun certified & Realtime Expert)

**Ex. IBM Employee**

**Trained Lakhs of Students  
for last 14 years across INDIA**

**India's No.1 Software Training Institute**

# DURGASOFT

**www.durgasoft.com Ph: 9246212143 ,8096969696**

# java.lang package

## Agenda

1. Introduction
2. Java.lang.Object class
  - toString( ) method
  - hashCode() method
  - toString() method vs hashCode() method
  - equals() method
  - Simplified version of .equals() method
  - More simplified version of .equals() method
  - Relationship between .equals() method and ==(double equal operator)
  - Differences between ==(double equal operator) and .equals() method?
  - Contract between .equals() method and hashCode() method
  - Clone () method
  - Shallow cloning
  - Deep Cloning
  - getClass() method
  - finalize( )
  - wait( ) , notify( ) , notifyAll( )
3. java.lang.String class
  - Importance of String constant pool (SCP)
  - Interning of String objects
  - String class constructors
  - Important methods of String class
  - Creation of our own immutable class
  - Final vs immutability
4. StringBuffer
  - Constructors
  - Important methods of StringBuffer
5. StringBuilder (1.5v)
  - StringBuffer Vs StringBuilder
  - String vs StringBuffer vs StringBuilder
  - Method chaining
6. Wrapper classes
  - Constructors
  - Wrapper class Constructor summary
7. Utility methods
  - valueOf() method
  - xxxValue() method
  - parseXxx() method
  - toString() method
8. Dancing between String, wrapper object and primitive
9. Partial Hierarchy of java.lang package
  - Void
10. Autoboxing and Autounboxing
  - Autoboxing
  - Autounboxing
  - Conclusions



### 11. Overloading with respect to widening, Autoboxing and var-arg methods

- Case 1: Widening vs Autoboxing
- Case 2: Widening vs var-arg method
- Case 3: Autoboxing vs var-arg method

## Introduction

The following are some of important classes present in java.lang package.

1. Object class
  2. String class
  3. StringBuffer class
  4. StringBuilder class (1.5 v)
  5. Wrapper Classes
  6. Autoboxing and Autounboxing(1.5 v)
- For writing any java program the most commonly required classes and interfaces are encapsulated in the separate package which is nothing but java.lang package.
  - It is not required to import java.lang package in our program because it is available by default to every java program.

What is your favorite package? Why java.lang is your favorite package?

It is not required to import lang package explicitly but the remaining packages we have to import.

## Java.lang.Object class:

1. For any java object whether it is predefined or customized the most commonly required methods are encapsulated into a separate class which is nothing but object class.
2. As object class acts as a root (or) parent (or) super for all java classes, by default its methods are available to every java class.
3. Note : If our class doesn't extend any other class then it is the direct child class of object  
If our class extends any other class then it is the indirect child class of Object.

The following is the list of all methods present in java.lang Object class :

1. public String toString();
2. public native int hashCode();
3. public boolean equals(Object o);
4. protected native Object clone()throws CloneNotSupportedException;
5. public final Class getClass();
6. protected void finalize()throws Throwable;
7. public final void wait() throws InterruptedException;
8. public final native void wait()throws InterruptedException;
9. public final void wait(long ms,int ns)throws InterruptedException;

10. public final native void notify();
11. public final native void notifyAll();

## toString() method :

1. We can use this method to get string representation of an object.
2. Whenever we are try to print any object reference internally toString() method will be executed.
3. If our class doesn't contain toString() method then Object class toString() method will be executed.
4. Example:
5. System.out.println(s1); => super(s1.toString());
6. Example 1:
7. class Student
8. {
9. String name;
10. int rollno;
11. Student(String name, int rollno)
12. {
13. this.name=name;
14. this.rollno=rollno;
15. }
16. public static void main(String args[]){
17. Student s1=new Student("saicharan",101);
18. Student s2=new Student("ashok",102);
19. System.out.println(s1);
20. System.out.println(s1.toString());
21. System.out.println(s2);
22. }
23. }
24. Output:
25. Student@3e25a5
26. Student@3e25a5
27. Student@19821f
- 28.
29. In the above program Object class toString() method got executed which is implemented as follows.
30. public String toString() {
31. return getClass().getName() + "@" +
- Integer.toHexString(hashCode());
32. }
- 33.
34. here getClass().getName() =>
- classname@hexa\_decimal\_String\_representation\_of\_hashCode
35. To provide our own String representation we have to override toString() method in our class.
- Ex: For example whenever we are try to print student reference to print his a name and roll no we have to override toString() method as follows.
36. public String toString(){
37. return name+"....."+rollno;
38. }
39. In String class, StringBuffer, StringBuilder, wrapper classes and in all collection classes toString() method is overridden for meaningful string representation. Hence in our classes also highly recommended to override toString() method.
- 40.

```

41. Example 2:
42.
43. class Test{
44. public String toString(){
45. return "Test";
46. }
47. public static void main(String[] args){
48. Integer i=new Integer(10);
49. String s=new String("ashok");
50. Test t=new Test();
51. System.out.println(i);
52. System.out.println(s);
53. System.out.println(t);
54. }
55. }
56. Output:
57. 10
58. ashok
59. Test

```

### hashCode() method :

1. For every object jvm will generate a unique number which is nothing but hashCode.
2. Jvm will using hashCode while saving objects into hashing related data structures like HashSet, HashMap, and Hashtable etc.
3. If the objects are stored according to hashCode searching will become very efficient (The most powerful search algorithm is hashing which will work based on hashCode).
4. If we didn't override hashCode() method then Object class hashCode() method will be executed which generates hashCode based on address of the object but it doesn't mean hashCode represents address of the object.
5. Based on our programming requirement we can override hashCode() method to generate our own hashCode.
6. Overriding hashCode() method is said to be proper if and only if for every object we have to generate a unique number as hashCode for every object.
7. Example 3:

```

class Student
{
public int hashCode()
{
return 100;
}
}

```

It is *improper* way of overriding hashCode() method because for every object we are generating same hashCode.

```

class Student
{
int rollno;
public int hashCode()
{
return rollno;
}
}

```

It is *proper* way of overriding hashCode() method because for every object we are generating a different hashCode.

**toString() method vs hashCode() method:**

```

class Test
{
    int i;
    Test(int i)
    {
        this.i=i;
    }
    public static void main(String[]
args){
    Test t1=new Test(10);
    Test t2=new Test(100);
    System.out.println(t1);
    System.out.println(t2);
}
}

```

Object==>toString() called.  
Object==>hashCode() called.

In this case *Object class toString()* method got executed which is internally calls *Object class hashCode()* method.

```

class Test{
    int i;
    Test(int i){
        this.i=i;
    }
    public int hashCode(){
        return i;
    }
    public static void main(String[]
args){
    Test t1=new Test(10);
    Test t2=new Test(100);
    System.out.println(t1);
    System.out.println(t2);
}
}

```

Object==>toString() called.  
Test==>hashCode() called.

In this case *Object class toString()* method got executed which is internally calls *Test class hashCode()* method.

8. Example 4:

```

9.
10. class Test
11. {
12.     int i;
13.     Test(int i)
14.     {
15.         this.i=i;
16.     }
17.     public int hashCode(){
18.         return i;
19.     }
20.     public String toString()
21.     {
22.         return i+"";
23.     }
24.     public static void main(String[] args){
25.         Test t1=new Test(10);
26.         Test t2=new Test(100);
27.         System.out.println(t1);
28.         System.out.println(t2);
29.     }
30. }
31. Output:
32. 10
33. 100

```

34. In this case Test class toString() method got executed.

**Note :**

1. if we are giving opportunity to Object class toString() method it internally calls hashCode() method. But if we are overriding toString() method it may not call hashCode() method.

2. We can use toString() method while printing object references and we can use hashCode() method while saving objects into HashSet or Hashtable or HashMap.

### equals() method:

1. We can use this method to check equivalence of two objects.
2. If our class doesn't contain .equals() method then object class .equals() method will be executed which is always meant for reference comparison[address comparison]. i.e., if two references pointing to the same object then only .equals() method returns true .

Example 5:

```
class Student
{
String name;
int rollno;
Student(String name,int rollno)
{
this.name=name;
this.rollno=rollno;
}
public static void main(String[] args){
Student s1=new Student("vijayabhaskar",101);
Student s2=new Student("bhaskar",102);
Student s3=new Student("vijayabhaskar",101);
Student s4=s1;
System.out.println(s1.equals(s2));
System.out.println(s1.equals(s3));
System.out.println(s1.equals(s4));
}}
```

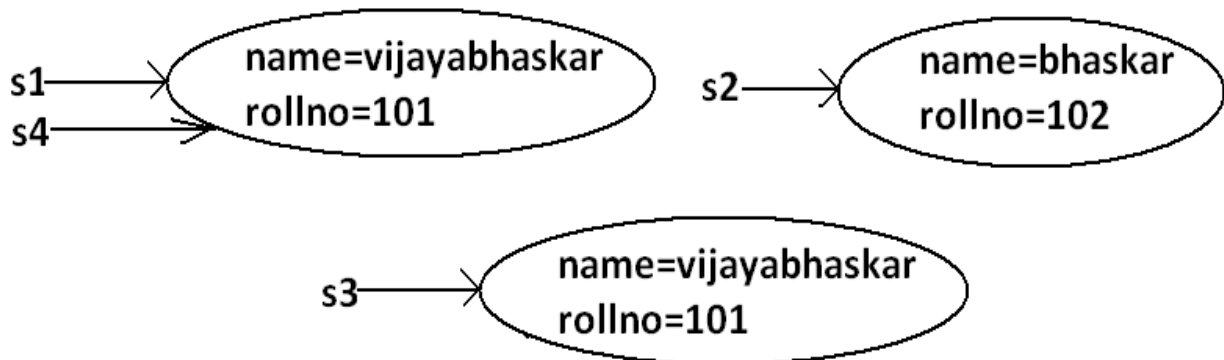
Output:

False

False

True

Diagram:



- In the above program Object class .equals() method got executed which is always meant for reference comparison that is if two references pointing to the same object then only .equals() method returns true.  
In object class .equals() method is implemented as follows which is meant for reference comparison.

- 
- `public boolean equals(Object obj) {`
- `return (this == obj);`
- `}`
- Based on our programming requirement we can override `.equals()` method for content comparison purpose.
  
- When ever we are overriding `.equals()` method we have to consider the following things :
  1. Meaning of content comparison i.e., whether we have to check the names are equal (or) roll numbers (or) both are equal.
  2. If we are passing different type of objects (heterogeneous object) our `.equals()` method should return false but not `ClassCastException` i.e., we have to handle `ClassCastException` to return false.
  3. If we are passing null argument our `.equals()` method should return false but not `NullPointerException` i.e., we have to handle `NullPointerException` to return false.
  4. The following is the proper way of overriding `.equals()` method for content comparison in Student class.
  - 5.
  6. Example 6:
  7. `class Student`
  8. `{`
  9. `String name;`
  10. `int rollno;`
  11. `Student(String name,int rollno)`
  12. `{`
  13. `this.name=name;`
  14. `this.rollno=rollno;`
  15. `}`
  16. `public boolean equals(Object obj)`
  17. `{`
  18. `try{`
  19. `String name1=this.name;`
  20. `int rollno1=this.rollno;`
  21. `Student s2=(Student)obj;`
  22. `String name2=s2.name;`
  23. `int rollno2=s2.rollno;`
  24. `if(name1.equals(name2) && rollno1==rollno2)`
  25. `{`
  26. `return true;`
  27. `}`
  28. `else return false;`
  29. `}`
  30. `catch(ClassCastException e)`
  31. `{`
  32. `return false;`
  33. `}`
  34. `catch(NullPointerException e)`
  35. `{`
  36. `return false;`
  37. `}`
  38. `}`
  39. `public static void main(String[] args){`
  40. `Student s1=new Student("vijayabhaskar",101);`

```

41. Student s2=new Student("bhaskar",102);
42. Student s3=new Student("vijayabhaskar",101);
43. Student s4=s1;
44. System.out.println(s1.equals(s2));
45. System.out.println(s1.equals(s3));
46. System.out.println(s1.equals(s4));
47. System.out.println(s1.equals("vijayabhaskar"));
48. System.out.println(s1.equals("null"));
49. }
50. }
51. Output:
52. False
53. True
54. True
55. False
56. False

```

### Simplified version of .equals() method:

```

public boolean equals(Object o){
try{
    Student s2=(Student)o;
    if(name.equals(s2.name) && rollno==s2.rollno){
        return true;
    }
    else return false;
}
catch(ClassCastException e) {
    return false;
}
catch(NullPointerException e) {
    return false;
}
}

```

### More simplified version of .equals() method :

```

public boolean equals(Object o) {
    if(this==o)
        return true;
    if(o instanceof Student) {
        Student s2=(Student)o;
        if(name.equals(s2.name) && rollno==s2.rollno)
            return true;
        else
            return false;
    }
    return false;
}

```

Example 7 :

```

class Student {
String name;
int rollno;
Student(String name,int rollno) {
this.name=name;
}
}

```

```

this.rollno=rollno;
}
public boolean equals(Object o) {
if(this==o)
return true;
if(o instanceof Student) {
Student s2=(Student)o;
if(name.equals(s2.name) && rollno==s2.rollno)
return true;
else
return false;
}
return false;
}
public static void main(String[] args){
Student s=new Student("vijayabhaskar",101);
Integer i=new Integer(10);
System.out.println(s.equals(i));
}
}

```

Output:

False

To make .equals() method more efficient we have to place the following code at the top inside .equals() method.

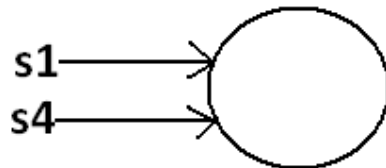
```

if(this==o)
return true;

```

Diagram:

**Student s1=new Student("vijayabhaskar",101);**  
**Student s4=s1;**



If 2 references pointing to the same object then .equals() method return true directly without performing any content comparison this approach improves performance of the system

|                                                                                                                                                                          |                                                                                                                                                                                                                                                                         |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> String s1 = new String("ashok"); String s2 = new String("ashok"); System.out.println(s1==s2); //false System.out.println(s1.equals(s2) ); //true </pre>            | <pre> StringBuffer s1 = new StringBuffer("ashok"); StringBuffer s2 = new StringBuffer("ashok"); System.out.println(s1==s2); //false System.out.println(s1.equals(s2) ); //false </pre>                                                                                  |
| <p>In String class .equals( ) is overridden for content comparison hence if content is same .equals( ) method returns true , even though this objects are different.</p> | <p>In StringBuffer class .equals( ) is not overridden for content comparison hence Object class .equals( ) will be executed which is meant for reference comparison , hence if objects are different .equals( ) method returns false , even though content is same.</p> |



**Note :** In String class , Wrapper classes and all collection classes .equals( ) method is overridden for content comparison

### Relationship between .equals() method and ==(double equal operator) :

1. If `r1==r2` is true then `r1.equals(r2)` is always true i.e., if two objects are equal by `==` operator then these objects are always equal by `.equals( )` method also.
2. If `r1==r2` is false then we can't conclude anything about `r1.equals(r2)` it may return true (or) false.
3. If `r1.equals(r2)` is true then we can't conclude anything about `r1==r2` it may returns true (or) false.
4. If `r1.equals(r2)` is false then `r1==r2` is always false.

### Differences between ==(double equal operator) and .equals() method?

| ==(double equal operator)                                                                                                                                                         | .equals() method                                                                                                                                         |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| It is an operator applicable for both primitives and object references.                                                                                                           | It is a method applicable only for object references but not for primitives.                                                                             |
| In the case of primitives == (double equal operator) meant for content comparison, but in the case of object references == operator meant for reference comparison.               | By default .equals() method present in object class is also meant for reference comparison.                                                              |
| We can't override == operator for content comparison in object references.                                                                                                        | We can override .equals() method for content comparison.                                                                                                 |
| If there is no relationship between argument types then we will get compile time error saying incompatible types.(relation means child to parent or parent to child or same type) | If there is no relationship between argument types then .equals() method simply returns false and we won't get any compile time error and runtime error. |
| For any object reference r, <code>r==null</code> is always false.                                                                                                                 | For any object reference r, <code>r.equals(null)</code> is also returns false.                                                                           |

```
String s = new String("ashok");
StringBuffer sb = new StringBuffer("ashok");
System.out.println(s == sb); // CE : incomparable types : String and
StringBuffer
System.out.println(s.equals(sb)); //false
```

#### **Note:**

in general we can use == (double equal operator) for *reference comparison* whereas .equals() method for *content comparison*.

## Contract between .equals() method and hashCode() method:

1. If 2 objects are equal by .equals() method compulsory their hashcodes must be equal (or) same. That is If *r1.equals(r2)* is true then *r1.hashCode()==r2.hashCode()* must be true.
2. If 2 objects are not equal by .equals() method then there are no restrictions on hashCode() methods. They may be same (or) may be different. That is If *r1.equals(r2)* is false then *r1.hashCode()==r2.hashCode()* may be same (or) may be different.
3. If hashcodes of 2 objects are equal we can't conclude anything about .equals() method it may returns true (or) false. That is If *r1.hashCode()==r2.hashCode()* is true then *r1.equals(r2)* method may returns true (or) false.
4. If hashcodes of 2 objects are not equal then these objects are always not equal by .equals() method also. That is If *r1.hashCode()==r2.hashCode()* is false then *r1.equals(r2)* is always false.

To maintain the above contract between .equals() and hashCode() methods whenever we are overriding .equals() method compulsory we should override hashCode() method. Violation leads to no compile time error and runtime error but it is not good programming practice.

### Example :

Consider the following person class.

Program:

```
class Person {
String name;
int age;
Person(String name,int age) {
this.name=name;
this.age=age;
}
public boolean equals(Object o) {
if(this==o)
return true;
if(o instanceof Person) {
Person p2=(Person)o;
if(name.equals(p2.name) && age==p2.age)
return true;
else
return false;
}
return false;
}
public static void main(String[] args){
Person p1=new Person("vijayabhaskar",101);
Person p2=new Person("vijayabhaskar",101);
Integer i=new Integer(102);
System.out.println(p1.equals(p2));
System.out.println(p1.equals(i));
}
}
```

Output:

True  
False

Which of the following is appropriate way of overriding hashCode() method?

Diagram:

|                                                   |                                                          |                                                                   |                      |
|---------------------------------------------------|----------------------------------------------------------|-------------------------------------------------------------------|----------------------|
| 1) public int hashCode()<br>{<br>return 100;<br>} | 2) public int hashCode()<br>{<br>return age+height;<br>} | 3) public int hashCode()<br>{<br>return name.hashCode()+age;<br>} | 4) Any of the above. |
|---------------------------------------------------|----------------------------------------------------------|-------------------------------------------------------------------|----------------------|

Based on whatever the parameters we override ".equals() method" we should use same parameters while overriding hashCode() method also.

**Note:** in all wrapper classes, in string class, in all collection classes .equals() method is overridden for content comparison in our classes also it is highly recommended to override .equals() method.

Which of the following is valid?

1. If hash Codes of 2 objects are not equal then .equals() method always return false.(valid)
2. Example:
- 3.
4. class Test {
5.     int i;
6.     Test(int i) {
7.         this.i=i;
8.     }
9.     public int hashCode() {
10.         return i;
11.     }
12.     public String toString() {
13.         return i+"";
14.     }
15.     public static void main(String[] args) {
16.         Test t1=new Test(10);
17.         Test t2=new Test(20);
18.         System.out.println(t1.hashCode()); //10
19.         System.out.println(t2.hashCode()); //20
20.         System.out.println(t1.hashCode()==t2.hashCode()); //false
21.         System.out.println(t1.equals(t2)); //false
22.     }
23. }

24. If 2 objects are equal by == operator then their hash codes must be same.(valid)
25. Example:

```

26.
27. class Test {
28.     int i;
29.     Test(int i) {
30.         this.i=i;
31.     }
32.     public int hashCode() {
33.         return i;

```

```

34.     }
35.     public String toString()      {
36.         return i+"";
37.     }
38.     public static void main(String[] args)      {
39.         Test t1=new Test(10);
40.         Test t2=t1;
41.         System.out.println(t1.hashCode()); //10
42.         System.out.println(t2.hashCode()); //10
43.         System.out.println(t1==t2); //true
44.
45.     }
46. }
47.

```

48. If == operator returns false then their hash codes(may be same (or) may be different) must be different.(invalid)

49. Example:

```

50.
51. class Test {
52.     int i;
53.     Test(int i)    {
54.         this.i=i;
55.     }
56.     public int hashCode() {
57.         return i;
58.     }
59.     public String toString()      {
60.         return i+"";
61.     }
62.     public static void main(String[] args){
63.         Test t1=new Test(10);
64.         Test t2=new Test(10);
65.         System.out.println(t1.hashCode()); //10
66.         System.out.println(t2.hashCode()); //10
67.         System.out.println(t1==t2); //false
68.     }
69. }

```

70. If hashcodes of 2 objects are equal then these objects are always equal by == operator also.(invalid)

## Clone () method:

1. The process of creating exactly duplicate object is called cloning.
2. The main objective of cloning is to maintain backup purposes.(i.e., if something goes wrong we can recover the situation by using backup copy.)
3. We can perform cloning by using clone() method of Object class.

protected native object clone() throws CloneNotSupportedException;

Example:

```

class Test implements Cloneable
{
    int i=10;
    int j=20;

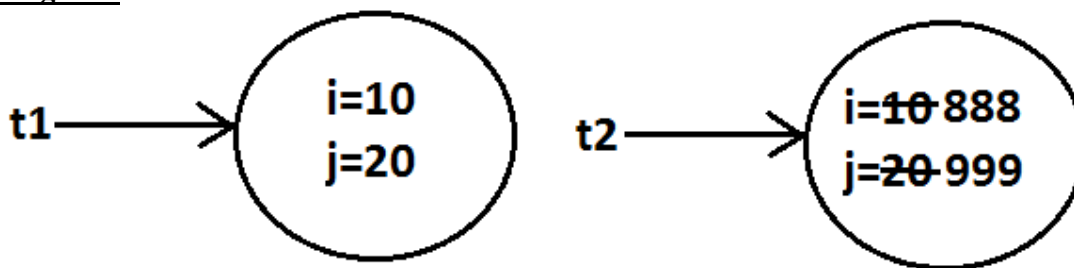
```

```

    public static void main(String[] args) throws
CloneNotSupportedException
    {
        Test t1=new Test();
        Test t2=(Test)t1.clone();
        t2.i=888;
        t2.j=999;
        System.out.println(t1.i+"-----"+t1.j);
        System.out.println(t2.i+"-----"+t2.j);
    }
}
Output:
10-----20
888-----999

```

Diagram:



- We can perform cloning only for Cloneable objects.
- An object is said to be Cloneable if and only if the corresponding class implements Cloneable interface.
- Cloneable interface present in java.lang package and does not contain any methods. It is a marker interface where the required ability will be provided automatically by the JVM.
- If we are trying to perform cloning on non-cloneable objects then we will get RuntimeException saying CloneNotSupportedException.

## Shallow cloning vs Deep cloning :

### Shallow cloning:

1. The process of creating bitwise copy of an object is called Shallow Cloning .
2. If the main object contain any primitive variables then exactly duplicate copies will be created in cloned object.
3. If the main object contain any reference variable then the corresponding object won't be created just reference variable will be created by pointing to old contained object.
4. By using main object reference if we perform any change to the contained object then those changes will be reflected automatically to the cloned object , by default Object class clone( ) meant for Shallow Cloning

```

5. class Cat {
6.     int j ;
7.     Cat(int j) {

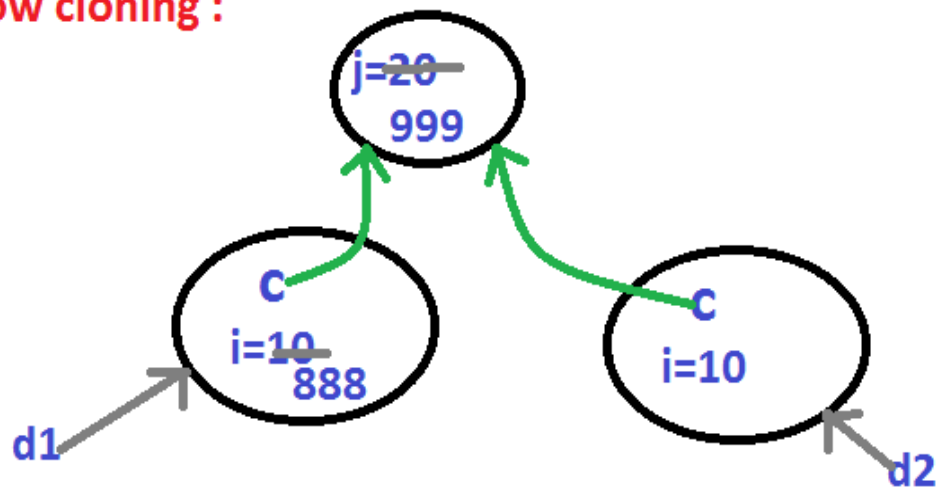
```

```

8.   this.j=j ;
9.   }
10.  }
11.
12.  class Dog implements Clonable {
13.      Cat c ;
14.      int i ;
15.      Dog(Cat c , int i) {
16.          this.c=c ;
17.          this.i=i ;
18.      }
19.      public Object clone( ) throws CloneNotSupportedException {
20.          return super.clone( ) ;
21.      }
22.  }
23.
24.  class ShallowClone {
25.      public static void main(String[ ] ar) {
26.          Cat c=new Cat(20) ;
27.          Dog d1=new Dog(c , 10) ;
28.          System.out.println(d1.i +.....+d1.j);    // 10.....20
29.
30.          Dog d2=(Dog)d1.clone( ) ;
31.          d1.i=888 ;
32.          d1.c.j=999 ;
33.          System.out.println(d2.i +.....+d2.j);    // 10.....999
34.      }
35.  }
36.

```

### shallow cloning :

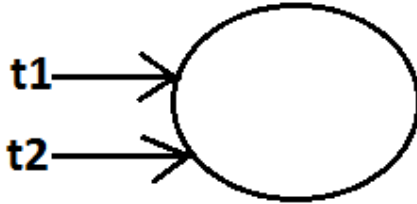


37. Shallow cloning is the best choice , if the Object contains only primitive values.
38. In Shallow cloning by using main object reference , if we perform any change to the contained object then those changes will be reflected automatically in cloned copy.
39. To overcome this problem we should go for Deep cloning.

Example:

```
Test t1=new Test();
Test t2=t1;
```

Diagram:



## Deep Cloning :

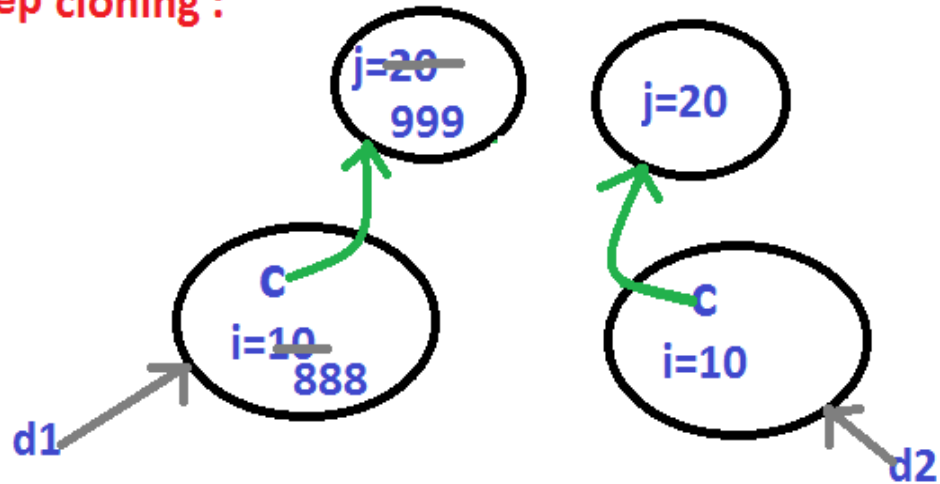
1. The process of creating exactly independent duplicate object(including contained objects also) is called deep cloning.
2. In Deep cloning , if main object contain any reference variable then the corresponding Object copy will also be created in cloned object.
3. Object class clone() method meant for Shallow Cloning , if we want Deep cloning then the programmer is responsible to implement by overriding clone() method.

```

4. class Cat {
5.     int j ;
6.     Cat(int j) {
7.         this.j=j ;
8.     }
9. }
10.
11. class Dog implements Clonable {
12.     Cat c ;
13.     int i ;
14.     Dog(Cat c , int i) {
15.         this.c=c ;
16.         this.i=i ;
17.     }
18.     public Object clone( ) throws CloneNotSupportedException {
19.         Cat c1=new Cat(c.j) ;
20.         Dog d1=new Dog(c1 , i) ;
21.         return d1 ;
22.     }
23. }
24.
25. class DeepClone {
26.     public static void main(String[] ar) {
27.         Cat c=new Cat(20) ;
28.         Dog d1=new Dog(c , 10) ;
29.         System.out.println(d1.i +.....+d1.c.j);    // 10.....20
30.
31.         Dog d2=(Dog)d1.clone( ) ;
32.         d1.i=888 ;
33.         d1.c.j=999 ;
34.         System.out.println(d2.i +.....+d2.c.j);    // 10.....20
35.     }
  
```

```
36.  }
37.
```

**deep cloning :**

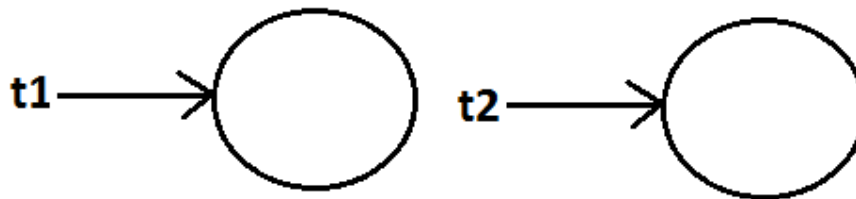


38. In Deep cloning by using main Object reference if we perform any change to the contained Object those changes won't be reflected to the cloned object.

Example:

```
Test t1=new Test();
Test t2=(Test)t1.clone();
System.out.println(t1==t2);           //false
System.out.println(t1.hashCode()==t2.hashCode()); //false
```

Diagram :



Which cloning is best ?

If the Object contain only primitive variable then Shallow Cloning is the best choice ,

If the Object contain reference variables then Deep cloning is the best choice.

Cloning by default deep cloning.



## getClass() method :

This method returns runtime class definition of an object.

Example :

```

class Test implements Cloneable {
    public static void main(String[] args) throws
    CloneNotSupportedException {
        Object o=new String("ashok");
        System.out.println("Runtime object type of o is
        :"+o.getClass().getName());
    }
}

```

Output:

Runtime object type of o is: java.lang. String

Ex : To print Connection interface implemented vendor specific class name .  
 System.out.println(con.getClass( ).getName( ) );

## finalize() :

Just before destroying an object GC calls finalize( ) method to perform CleanUp activities .

## wait() , notify() , notifyAll()

We can use these methods for inter thread communication

## java.lang.String class :

### Case 1:

```

String s=new String("bhaskar");
s.concat("software");
System.out.println(s); //bhaskar

```

Once we create a String object we can't perform any changes in the existing object. If we are try to perform any changes with those changes a new object will be created. This behavior is called immutability of the String object.

Diagram:

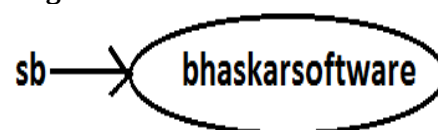
```

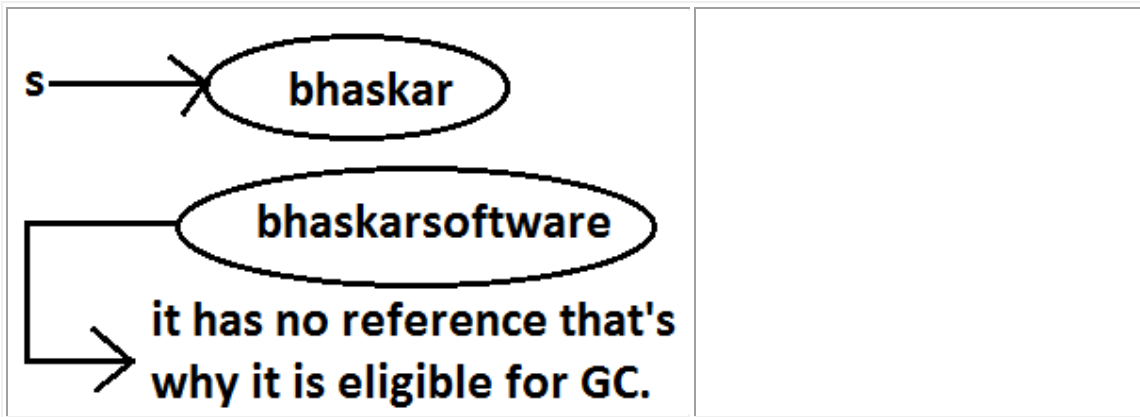
StringBuffer sb=new
StringBuffer("bhaskar");
sb.append("software");
System.out.println(sb);
//bhaskarsoftware

```

Once we created a StringBuffer object we can perform any changes in the existing object. This behavior is called mutability of the StringBuffer object.

Diagram:



Case 2 :

```
String s1=new String("ashok");
String s2=new String("ashok");
System.out.println(s1==s2);//false
System.out.println(s1.equals(s2));//true
```

In String class `.equals()` method is overridden for content comparison hence if the content is same `.equals()` method returns true even though objects are different.

```
StringBuffer sb1=new
StringBuffer("ashok");
StringBuffer sb2=new
StringBuffer("ashok");
System.out.println(sb1==sb2);//false
System.out.println(sb1.equals(sb2));//false
```

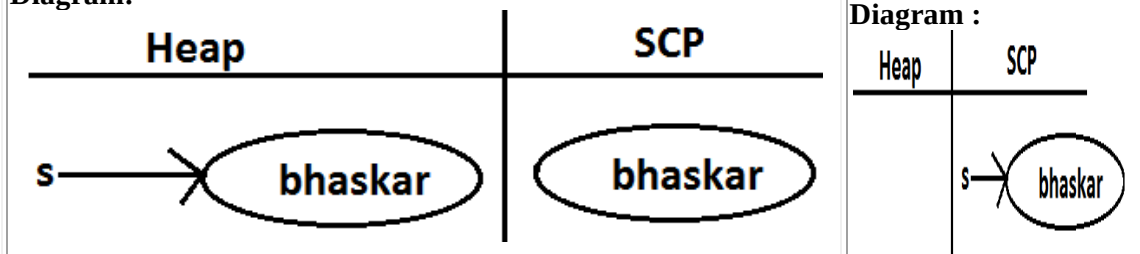
In StringBuffer class `.equals()` method is not overridden for content comparison hence Object class `.equals()` method got executed which is always meant for reference comparison. Hence if objects are different `.equals()` method returns false even though content is same.

Case 3 :

```
String s=new String("bhaskar");
```

In this case two objects will be created one is on the heap the other one is SCP(String constant pool) and `s` is always pointing to heap object.

Diagram:

Note :

1. Object creation in SCP is always optional 1st JVM will check is any object already created with required content or not. If it is already available then it will reuse existing object instead of creating new object. If it is not already there then only a new object will be created. Hence there is no chance of existing 2 objects with same content on SCP that is duplicate objects are not allowed in SCP.
2. Garbage collector can't access SCP area hence even though object doesn't have any reference still that object is not eligible for GC if it is present in SCP.
3. All SCP objects will be destroyed at the time of JVM shutdown automatically.

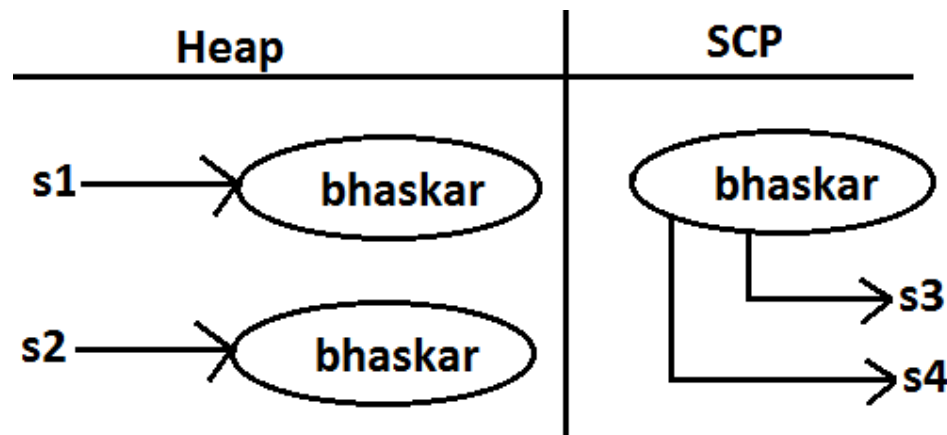
Example 1:

```
String s1=new String("bhaskar");
String s2=new String("bhaskar");
String s3="bhaskar";
String s4="bhaskar";
```

Note :

When ever we are using new operator compulsory a new object will be created on the Heap . There may be a chance of existing two objects with same content on the heap but there is no chance of existing two objects with same content on SCP . i.e., duplicate objects possible in the heap but not in SCP .

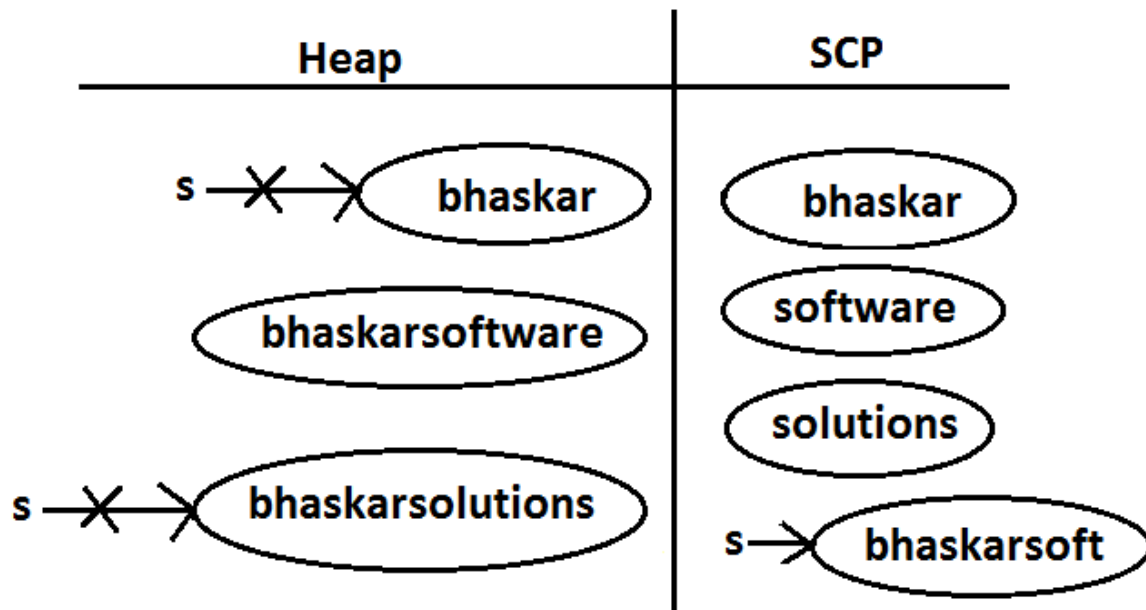
Diagram :



Example 2:

```
String s=new String("bhaskar");
s.concat("software");
s=s.concat("solutions");
s="bhaskarsoft";
```

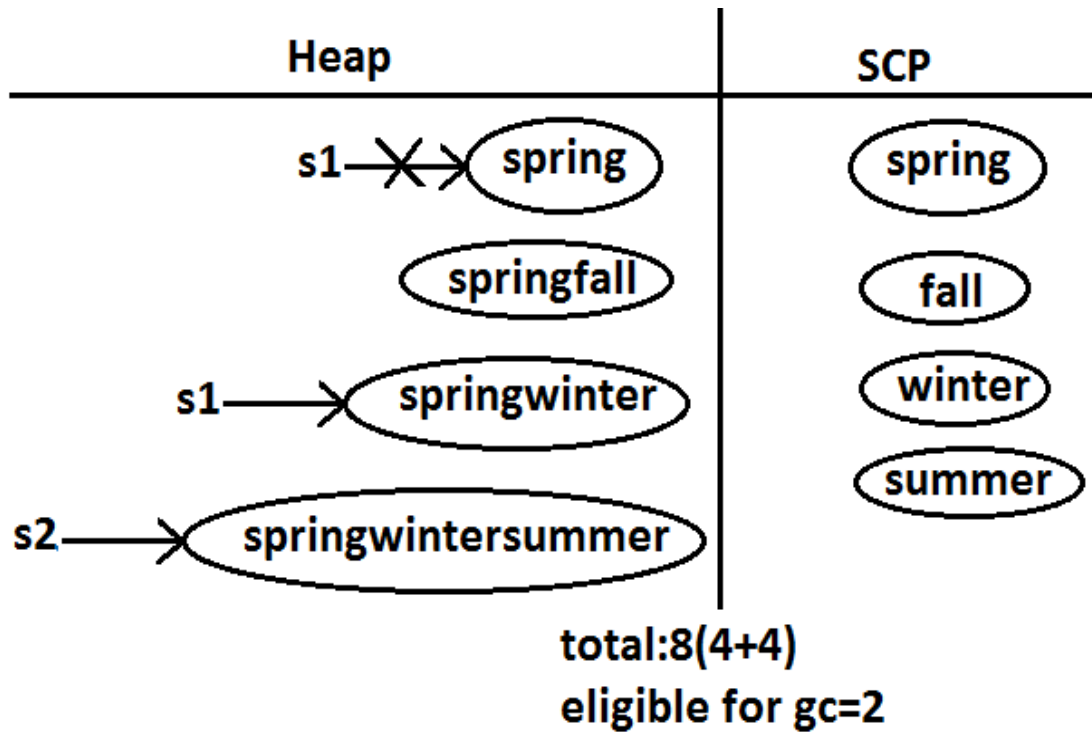
Diagram :



For every String Constant one object will be created in SCP. Because of runtime operation if an object is required to create compulsory that object should be placed on the heap but not SCP.

Example 3:

```
String s1=new String("spring");
s1.concat("fall");
s1=s1+"winter";
String s2=s1.concat("summer");
System.out.println(s1);
System.out.println(s2);
```

Diagram :

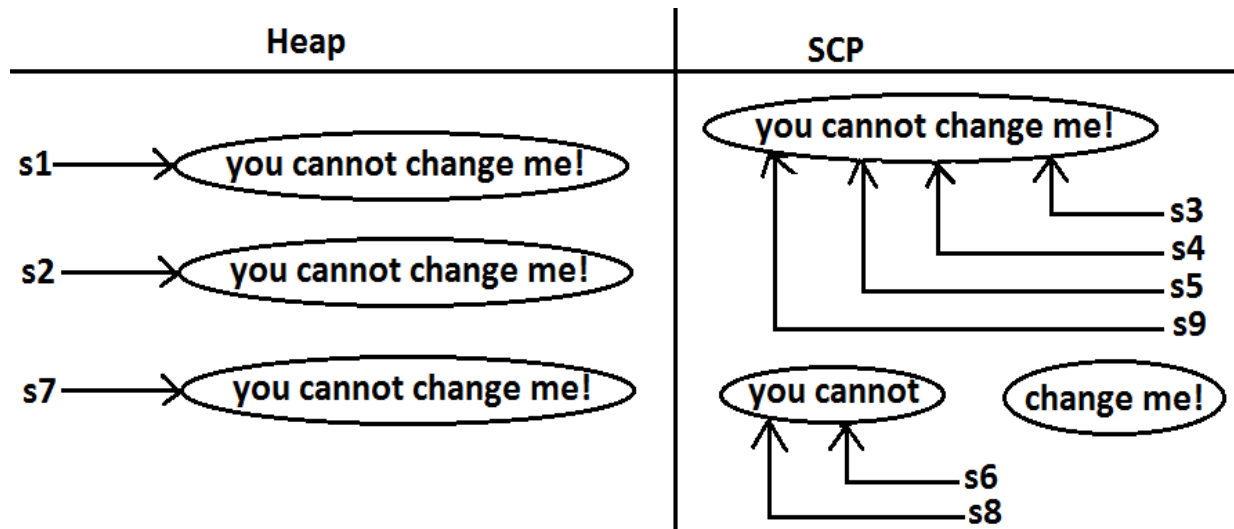
Example:

```

class StringDemo
{
    public static void main(String[] args)
    {
        String s1=new String("you cannot change me!");
        String s2=new String("you cannot change me!");
        System.out.println(s1==s2);//false
        String s3="you cannot change me!";
        System.out.println(s1==s3);//false
        String s4="you cannot change me!";
        System.out.println(s3==s4);//true
        String s5="you cannot "+"change me!";
        System.out.println(s3==s5);//true
        String s6="you cannot ";
        String s7=s6+"change me!";
        System.out.println(s3==s7);//false
        final String s8="you cannot ";
        String s9=s8+"change me!";
        System.out.println(s3==s9);//true
        System.out.println(s6==s8);//true
    }
}

```

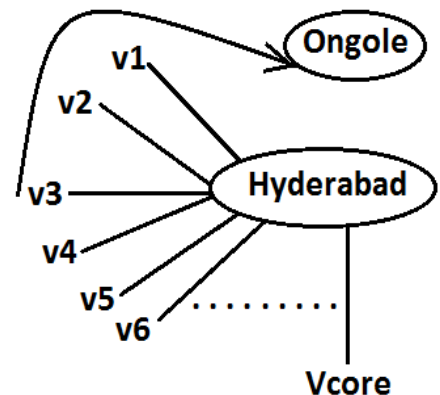
Diagram:



### Importance of String constant pool (SCP) :

**Voter Registration Form**

|                                                 |                                                                                                |
|-------------------------------------------------|------------------------------------------------------------------------------------------------|
| Name: <input type="text" value="BhaskarReddy"/> | <div style="border: 1px solid black; width: 100px; height: 100px; margin: 0 auto;">photo</div> |
| FatherName: <input type="text"/>                |                                                                                                |
| DOB: <input type="text"/>                       |                                                                                                |
| Age: <input type="text"/>                       |                                                                                                |
| Address: <input type="text"/>                   |                                                                                                |
| City: <input type="text"/>                      |                                                                                                |
| State: <input type="text"/>                     |                                                                                                |
| .                                               |                                                                                                |
| .                                               |                                                                                                |
| .                                               |                                                                                                |



1. In our program if any String object is required to use repeatedly then it is not recommended to create multiple object with same content it reduces performance of the system and effects memory utilization.
2. We can create only one copy and we can reuse the same object for every requirement. This approach improves performance and memory utilization we can achieve this by using "scp".
3. In SCP several references pointing to same object the main disadvantage in this approach is by using one reference if we are performing any change the remaining references will be impacted. To overcome this problem sun people implemented immutability concept for String objects.

4. According to this once we create a String object we can't perform any changes in the existing object if we are trying to perform any changes with those changes a new String object will be created hence immutability is the main disadvantage of scp.

## FAQS :

1. What is the main difference between String and StringBuilder?
2. What is the main difference between String and StringBuffer ?
3. Other than immutability and mutability is there any other difference between String and StringBuffer ?

In String .equals( ) method meant for content comparison where as in StringBuffer meant for reference comparison .

4. What is the meaning of immutability and mutability?
5. Explain immutability and mutability with an example?
6. What is SCP?

A specially designed memory area for the String literals/objects .

7. What is the advantage of SCP?

Instead of creating a separate object for every requirement we can create only one object and we can reuse same object for every requirement. This approach improves performance and memory utilization.

8. What is the disadvantage of SCP?

In SCP as several references pointing to the same object by using one reference if we are performing any changes the remaining references will be inflected. To prevent this compulsory String objects should be immutable. That is immutability is the disadvantage of SCP.

9. Why SCP like concept available only for the String but not for the StringBuffer?

As String object is the most commonly used object sun people provided a specially designed memory area like SCP to improve memory utilization and performance.

But StringBuffer object is not commonly used object hence specially designed memory area is not at all required.

10. Why String objects are immutable where as StringBuffer objects are mutable?

In the case of String as several references pointing to the same object, by using one reference if we are allowed perform the change the remaining references will be impacted. To prevent this once we created a String object we can't perform any change in the existing object that is immutability is only due to SCP. But in the case of StringBuffer for every requirement we are creating a separate

object will be created by using one reference if we are performing any change in the object the remaining references won't be impacted hence immutability concept is not require for the StringBuffer.

11. Similar to String objects any other objects are immutable in java?

In addition to String objects , all wrapper objects are immutable in java.

12. Is it possible to create our own mutable class?

Yes.

13. Explain the process of creating our own immutable class with an example?

14. What is the difference between final and immutability?

15. What is interning of String objects?

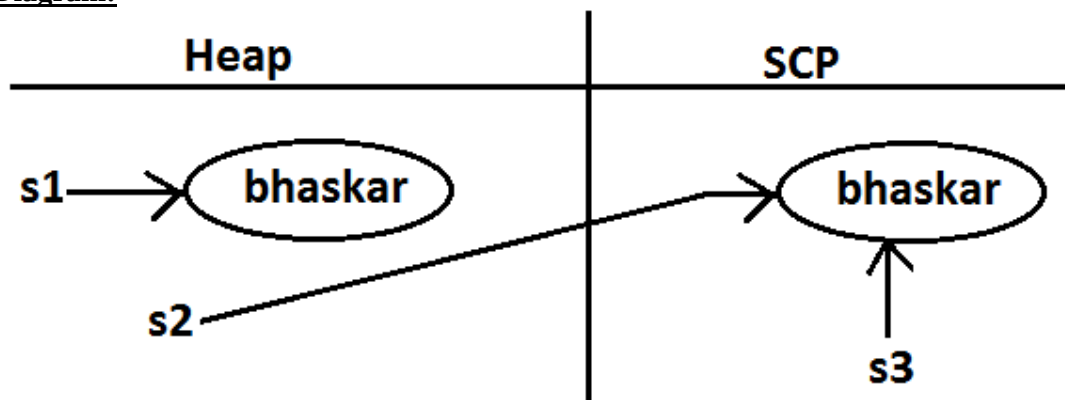
### Interning of String objects :

By using heap object reference, if we want to get corresponding SCP object , then we should go for intern() method.

Example 1:

```
class StringDemo {
    public static void main(String[] args) {
        String s1=new String("bhaskar");
        String s2=s1.intern();
        System.out.println(s1==s2);    //false
        String s3="bhaskar";
        System.out.println(s2==s3);    //true
    }
}
```

Diagram:



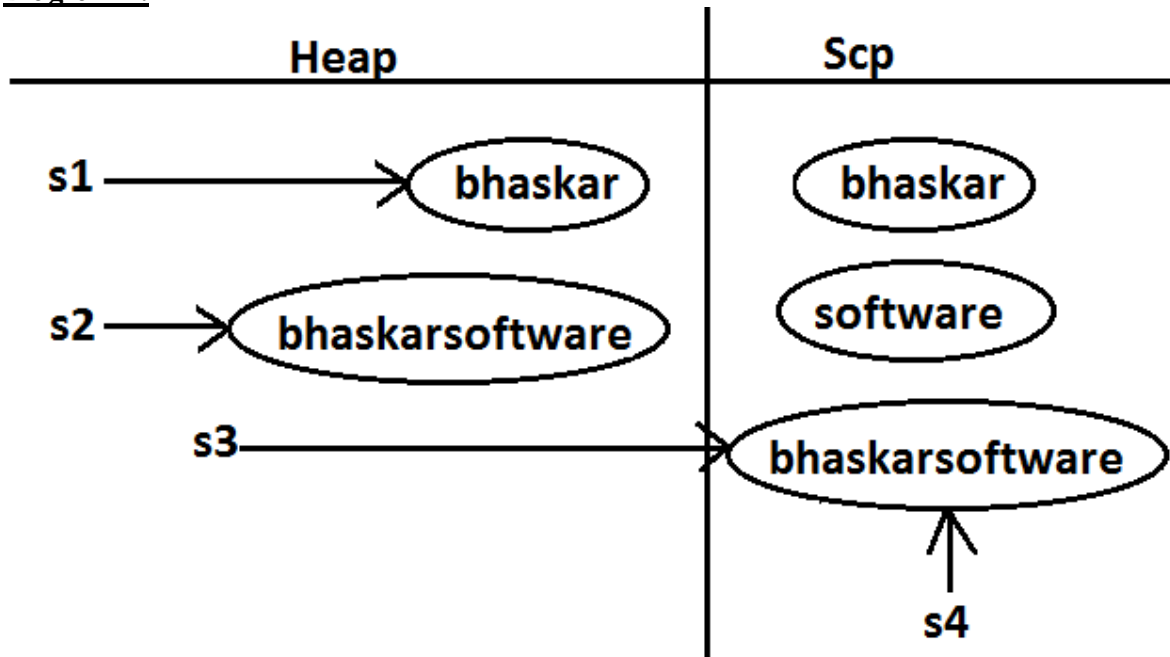
If the corresponding object is not there in SCP then intern() method itself will create that object and returns it.



Example 2:

```
class StringDemo {
    public static void main(String[] args)    {
        String s1=new String("bhaskar");
        String s2=s1.concat("software");
        String s3=s2.intern();
        String s4="bhaskarsoftware";
        System.out.println(s3==s4);//true
    }
}
```

Diagram 2:



### String class constructors :

1. `String s=new String();`  
Creates an empty String Object.
2. `String s=new String(String literals);`  
To create an equivalent String object for the given String literal on the heap.
3. `String s=new String(StringBuffer sb);`  
Creates an equivalent String object for the given StringBuffer.
4. `String s=new String(char[] ch);`

creates an equivalent String object for the given char[ ] array.

Example:

```
class StringDemo {
public static void main(String[] args) {
char[] ch={'a','b','c'} ;
String s=new String(ch);
System.out.println(ch);//abc
}
}
```

#### 5. String s=new String(byte[] b);

Create an equivalent String object for the given byte[] array.

Example:

```
class StringDemo {
public static void main(String[] args) {
byte[] b={100,101,102};
String s=new String(b);
System.out.println(s);//def
}
}
```

### Important methods of String class:

#### 1. public char charAt(int index);

Returns the character locating at specified index.

Example:

```
class StringDemo {
public static void main(String[] args) {
String s="ashok";
System.out.println(s.charAt(3));//o
System.out.println(s.charAt(100));// RE :
StringIndexOutOfBoundsException
}
}
```

// index is zero based

#### 2. public String concat(String str);

#### 3. Example:

```
4. class StringDemo {
5. public static void main(String[] args) {
6. String s="ashok";
7. s=s.concat("software");
8. //s=s+"software";
9. //s+="software";
10. System.out.println(s);//ashoksoftware
11. }
12. }
```

The overloaded "+" and "+=" operators also meant for concatenation purpose only.

**13. public boolean equals(Object o);**

For content comparison where case is important.  
It is the overriding version of Object class .equals() method.

**14. public boolean equalsIgnoreCase(String s);**

For content comparison where case is not important.

Example:

```
class StringDemo {
    public static void main(String[] args) {
        String s="java";
        System.out.println(s.equals("JAVA")); //false
        System.out.println(s.equalsIgnoreCase("JAVA")); //true
    }
}
```

Note: We can validate username by using .equalsIgnoreCase() method where case is not important and we can validate password by using .equals() method where case is important.

**15. public String substring(int begin);**

Return the substring from begin index to end of the string.

Example:

```
class StringDemo {
    public static void main(String[] args) {
        String s="ashoksoft";
        System.out.println(s.substring(5)); //soft
    }
}
```

**16. public String substring(int begin, int end);**

Returns the substring from begin index to end-1 index.

Example:

```
class StringDemo {
    public static void main(String[] args) {
        String s="ashoksoft";
        System.out.println(s.substring(5)); //soft
        System.out.println(s.substring(3,7)); //okso
    }
}
```

**17. public int length();**

Returns the number of characters present in the string.

Example:

```
class StringDemo {
    public static void main(String[] args) {
        String s="jobs4times";
        System.out.println(s.length()); //10
        //System.out.println(s.length()); //compile time error
    }
}
/*
CE :
StringDemo.java:7: cannot find symbol
    symbol : variable length
    location: class java.lang.String
*/
```

Note: length is the variable applicable for arrays where as length() method is applicable for String object.

#### 18. public String replace(char old, char new);

To replace every old character with a new character.

Example:

```
class StringDemo {
    public static void main(String[] args) {
        String s="ababab";
        System.out.println(s.replace('a', 'b')); //bbbbbb
    }
}
```

#### 19. public String toLowerCase();

Converts the all characters of the string to lowercase.

Example:

```
class StringDemo {
    public static void main(String[] args) {
        String s="ASHOK";
        System.out.println(s.toLowerCase()); //ashok
    }
}
```

#### 20. public String toUpperCase();

Converts the all characters of the string to uppercase.

Example :

```
class StringDemo {
    public static void main(String[] args) {
        String s="ashok";
```

```

        System.out.println(s.toUpperCase()); //ASHOK
    }
}

```

## 21. public String trim();

We can use this method to remove blank spaces present at beginning and end of the string but not blank spaces present at middle of the String.

Example:

```

class StringDemo {
    public static void main(String[] args) {
        String s="  sai charan  ";
        System.out.println(s.trim()); //sai charan
    }
}

```

## 22. public int indexOf(char ch);

It returns index of 1st occurrence of the specified character if the specified character is not available then return -1.

Example:

```

class StringDemo {
    public static void main(String[] args) {
        String s="saicharan";
        System.out.println(s.indexOf('c')); // 3
        System.out.println(s.indexOf('z')); // -1
    }
}

```

## 23. public int lastIndexOf(Char ch);

It returns index of last occurrence of the specified character if the specified character is not available then return -1.

Example:

```

class StringDemo {
    public static void main(String[] args) {
        String s="arunkumar";
        System.out.println(s.lastIndexOf('a')); //7
        System.out.println(s.indexOf('z')); //-1
    }
}

```

### Note :

Because runtime operation if there is a change in content with those changes a new object will be created only on the heap but not in SCP.

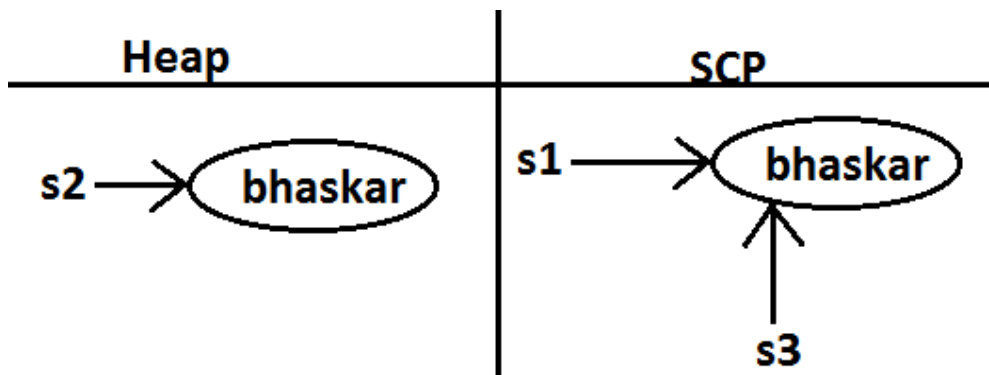
If there is no change in content no new object will be created the same object will be reused.

This rule is same whether object present on the Heap or SCP

Example 1 :

```
class StringDemo {
    public static void main(String[] args) {
        String s1="bhaskar";
        String s2=s1.toUpperCase();
        String s3=s1.toLowerCase();
        System.out.println(s1==s2);//false
        System.out.println(s1==s3);//true
    }
}
```

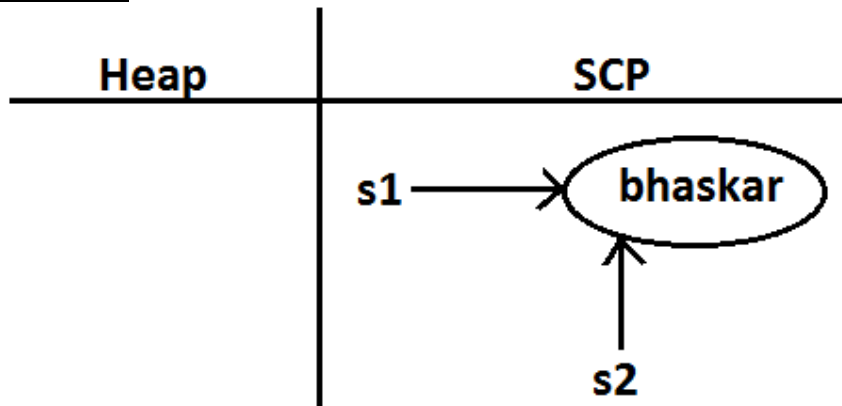
Diagram :



Example 2:

```
class StringDemo {
    public static void main(String[] args) {
        String s1="bhaskar";
        String s2=s1.toString();
        System.out.println(s1==s2);//true
    }
}
```

Diagram :



```
class StringDemo {
    public static void main(String[] args) {
        String s1=new String("ashok");
        String s2=s1.toString();
        String s3=s1.toUpperCase();
        String s4=s1.toLowerCase();
        String s5=s1.toUpperCase();
        String s6=s3.toLowerCase();
    }
}
```

```
System.out.println(s1==s6); //false
System.out.println(s3==s5); //false
```

## Creation of our own immutable class:

Once we created an object we can't perform any changes in the existing object. If we are trying to perform any changes with those changes a new object will be created. If there is no change in the content then existing object will be reused. This behavior is called immutability.

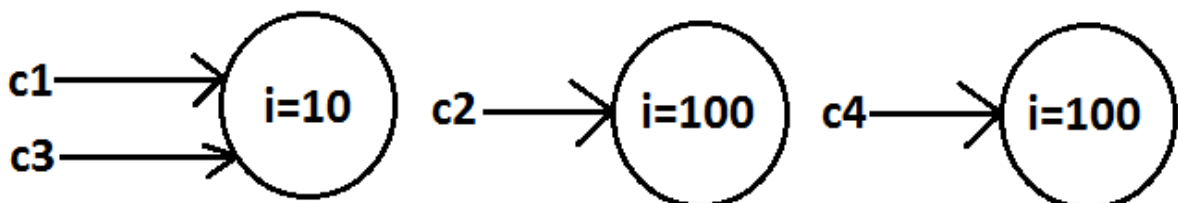
### Immutable program :

```
final class CreateImmutable {
    private int i;
    CreateImmutable(int i) {
        this.i=i;
    }
    public CreateImmutable modify(int i) {
        if(this.i==i)
            return this;
        else
            return (new CreateImmutable(i));
    }
    public static void main(String[] args) {
        CreateImmutable c1=new CreateImmutable(10);
        CreateImmutable c2=c1.modify(100);
        CreateImmutable c3=c1.modify(10);
        System.out.println(c1==c2); //false
        System.out.println(c1==c3); //true
        CreateImmutable c4=c1.modify(100);
        System.out.println(c2==c4); //false
    }
}
```

Once we create a CreateImmutable object we can't perform any changes in the existing object, if we are trying to perform any changes with those changes a new object will be created.

If there is no change in the content then existing object will be reused

### Diagram:



## Final vs immutability :

1. final modifier applicable for variables where as immutability concept applicable for objects
2. If reference variable declared as final then we can't perform reassignment for the reference variable it doesn't mean we can't perform any change in that object.
3. That is by declaring a reference variable as final we won't get any immutability nature .
4. final and immutability both are different concepts .

Example:

```
class Test
{
    public static void main(String[] args)
    {
        final StringBuffer sb=new StringBuffer("ashok");
        sb.append("software");
        System.out.println(sb);//ashoksoftware
        sb=new StringBuffer("solutions");//C.E: cannot assign a
value to final variable sb
    }
}
```

In the above example even though "sb" is final we can perform any type of change in the corresponding object. That is through final keyword we are not getting any immutability nature.

Which of the following are meaning ful ?

1. final variable (valid)
2. final object (invalid)
3. immutable variable (invalid)
4. immutable object (valid)

## StringBuffer :

1. If the content will change frequently then never recommended to go for String object because for every change a new object will be created internally.
2. To handle this type of requirement we should go for StringBuffer concept.
3. The main advantage of StringBuffer over String is, all required changes will be performed in the existing object only instead of creating new object.(won't create new object)



## Constructors :

### 1. `StringBuffer sb=new StringBuffer();`

Creates an empty `StringBuffer` object with default initialcapacity "16".  
Once `StringBuffer` object reaches its maximum capacity a new `StringBuffer` object will be created with  
 $\text{Newcapacity}=(\text{currentcapacity}+1)*2$ .

Example:

```
class StringBufferDemo {
    public static void main(String[] args) {
        StringBuffer sb=new StringBuffer();
        System.out.println(sb.capacity());//16
        sb.append("abcdefghijklmnop");
        System.out.println(sb.capacity());//16
        sb.append("q");
        System.out.println(sb.capacity());//34
    }
}
```

### 2. `StringBuffer sb=new StringBuffer(int initialcapacity);`

Creates an empty `StringBuffer` object with the specified initial capacity.

Example:

```
class StringBufferDemo {
    public static void main(String[] args) {
        StringBuffer sb=new StringBuffer(19);
        System.out.println(sb.capacity());//19
    }
}
```

### 3. `StringBuffer sb=new StringBuffer(String s);`

Creates an equivalent `StringBuffer` object for the given `String` with  
 $\text{capacity}=\text{s.length}()+16$ ;

Example:

```
class StringBufferDemo {
    public static void main(String[] args) {
        StringBuffer sb=new StringBuffer("ashok");
        System.out.println(sb.capacity());//21
    }
}
```

## Important methods of StringBuffer :

1. `public int length();`

Return the no of characters present in the StringBuffer.

2. `public int capacity();`

Returns the total no of characters StringBuffer can accommodate(hold).

3. `public char charAt(int index);`

It returns the character located at specified index.

Example:

```
class StringBufferDemo {
    public static void main(String[] args) {
        StringBuffer sb=new StringBuffer("saiashokkumarreddy");
        System.out.println(sb.length()); //18
        System.out.println(sb.capacity()); //34
        System.out.println(sb.charAt(14)); //e
        System.out.println(sb.charAt(30)); //RE :
        StringIndexOutOfBoundsException
    }
}
```

4. `public void setCharAt(int index, char ch);`

To replace the character locating at specified index with the provided character.

Example:

```
class StringBufferDemo {
    public static void main(String[] args) {
        StringBuffer sb=new StringBuffer("ashokkumar");
        sb.setCharAt(8, 'A');
        System.out.println(sb);
    }
}
```

5. `public StringBuffer append(String s);`
6. `public StringBuffer append(int i);`
7. `public StringBuffer append(long l);`
8. `public StringBuffer append(boolean b);`      All these are overloaded methods.

9. `public StringBuffer append(double d);`
10. `public StringBuffer append(float f);`
11. `public StringBuffer append(int index, Object o);`

12. Example:

```
13. class StringBufferDemo {
14.     public static void main(String[] args) {
15.         StringBuffer sb=new StringBuffer();
16.         sb.append("PI value is :");
17.         sb.append(3.14);
18.         sb.append(" this is exactly ");
19.         sb.append(true);
20.         System.out.println(sb); //PI value is :3.14 this is exactly
    true
}
```

```

21.         }
22.     }
23.
24.     public StringBuffer insert(int index,String s);
25.     public StringBuffer insert(int index,int i);
26.     public StringBuffer insert(int index,long l);
27.     public StringBuffer insert(int index,double d);           All are
    overloaded methods
28.     public StringBuffer insert(int index,boolean b);
29.     public StringBuffer insert(int index,float f);
30.     public StringBuffer insert(int index, Object o);
31. To insert at the specified location.
32. Example :
33. class StringBufferDemo {
34.     public static void main(String[] args) {
35.         StringBuffer sb=new StringBuffer("abcdefgh");
36.         sb.insert(2, "xyz");
37.         sb.insert(11,"9");
38.         System.out.println(sb);//abxyzcdefgh9
39.     }
40. }
41. public StringBuffer delete(int begin,int end);

```

To delete characters from begin index to end n-1 index.

```

42. public StringBuffer deleteCharAt(int index);

```

To delete the character locating at specified index.

Example:

```

class StringBufferDemo {
    public static void main(String[] args) {
        StringBuffer sb=new StringBuffer("saicharankumar");
        System.out.println(sb);//saicharankumar
        sb.delete(6,13);
        System.out.println(sb);//saichar
        sb.deleteCharAt(5);
        System.out.println(sb);//saichr
    }
}

```

```

43. public StringBuffer reverse();

```

44. Example :

```

45. class StringBufferDemo {
46.     public static void main(String[] args) {
47.         StringBuffer sb=new StringBuffer("ashokkumar");
48.         System.out.println(sb);//ashokkumar
49.         System.out.println(sb.reverse());//ramukkohsa
50.     }
51. }

```

```

52. public void setLength(int length);

```

Consider only specified no of characters and remove all the remaining characters.

Example:

```

class StringBufferDemo {

```

```

    public static void main(String[] args)    {
        StringBuffer sb=new StringBuffer("ashokkumar");
        sb.setLength(6);
        System.out.println(sb);//ashokk
    }
}

```

### 53. public void trimToSize();

To deallocate the extra allocated free memory such that capacity and size are equal.

Example:

```

class StringBufferDemo {
    public static void main(String[] args)    {
        StringBuffer sb=new StringBuffer(1000);
        System.out.println(sb.capacity());//1000
        sb.append("ashok");
        System.out.println(sb.capacity());//1000
        sb.trimToSize();
        System.out.println(sb.capacity());//5
    }
}

```

### 54. public void ensureCapacity(int initialcapacity);

To increase the capacity dynamically(fly) based on our requirement.

Example:

```

class StringBufferDemo {
    public static void main(String[] args){
        StringBuffer sb=new StringBuffer();
        System.out.println(sb.capacity());//16
        sb.ensureCapacity(1000);
        System.out.println(sb.capacity());//1000
    }
}

```

### Note :

Every method present in StringBuffer is synchronized hence at a time only one thread is allowed to operate on StringBuffer object , it increases waiting time of the threads and creates performance problems , to overcome this problem we should go for StringBuilder.

## StringBuilder (1.5v)

1. Every method present in StringBuffer is declared as synchronized hence at a time only one thread is allowed to operate on the StringBuffer object due to this, waiting time of the threads will be increased and effects performance of the system.
2. To overcome this problem sun people introduced StringBuilder concept in 1.5v.

## StringBuffer Vs StringBuilder

StringBuilder is exactly same as StringBuffer(including constructors and methods ) except the following differences :

| StringBuffer                                                                                                       | StringBuilder                                                                                                                        |
|--------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Every method present in StringBuffer is synchronized.                                                              | No method present in StringBuilder is synchronized.                                                                                  |
| At a time only one thread is allow to operate on the StringBuffer object hence StringBuffer object is Thread safe. | At a time Multiple Threads are allowed to operate simultaneously on the StringBuilder object hence StringBuilder is not Thread safe. |
| It increases waiting time of the Thread and hence relatively performance is low.                                   | Threads are not required to wait and hence relatively performance is high.                                                           |
| Introduced in 1.0 version.                                                                                         | Introduced in 1.5 versions.                                                                                                          |

### String vs StringBuffer vs StringBuilder :

1. If the content is fixed and won't change frequently then we should go for String.
2. If the content will change frequently but Thread safety is required then we should go for StringBuffer.
3. If the content will change frequently and Thread safety is not required then we should go for StringBuilder.

### Method chaining:

1. For most of the methods in String, StringBuffer and StringBuilder the return type is same type only. Hence after applying method on the result we can call another method which forms method chaining.
2. Example:
3. `sb.m1().m2().m3().....`
4. In method chaining all methods will be evaluated from left to right.
5. Example:
6. 

```
class StringBufferDemo {
7.     public static void main(String[] args)      {
8.         sb.append("ashok").insert(5,"arunkumar").delete(11,13)
9.         .reverse().append("solutions").insert(18,"abcd").reverse();
10.        System.out.println(sb); // snofdcbaitsulosashokarunkur
11.    }
12. }
```

## Wrapper classes :

The main objectives of wrapper classes are:

1. To wrap primitives into object form so that we can handle primitives also just like objects.
2. To define several utility functions which are required for the primitives.

## Constructors :

1. All most all wrapper classes define the following 2 constructors one can take corresponding primitive as argument and the other can take String as argument.

2. Example:

```
3. 1) Integer i=new Integer(10);
4. 2) Integer i=new Integer("10");
```

5. If the String is not properly formatted i.e., if it is not representing number then we will get runtime exception saying "NumberFormatException".

6. Example:

```
7. class WrapperClassDemo {
8.     public static void main(String[] args)throws Exception {
9.         Integer i=new Integer("ten");
10.        System.out.println(i);//NumberFormatException
11.    }
12. }
```

13. Float class defines 3 constructors with float, String and double arguments.

```
14. 1) Float f=new Float (10.5f);
15. 2) Float f=new Float ("10.5f");
16. 3) Float f=new Float(10.5);
17. 4) Float f=new Float ("10.5");
```

18. Character class defines only one constructor which can take char primitive as argument there is no String argument constructor.

```
19. Character ch=new Character('a');//valid
20. Character ch=new Character("a");//invalid
```

21. Boolean class defines 2 constructors with boolean primitive and String arguments.

If we want to pass boolean primitive the only allowed values are true, false where case should be lower case.

22. Example:

```
23. Boolean b=new Boolean(true);
24. Boolean b=new Boolean(false);
25.        //Boolean b1=new Boolean(True);//C.E
26.        //Boolean b=new Boolean(False);//C.E
27.        //Boolean b=new Boolean(TRUE);//C.E
```

28. If we are passing String argument then case is not important and content is not important. If the content is case insensitive String of true then it is treated as true in all other cases it is treated as false.

29. Example 1:

```
30. class WrapperClassDemo {
31.     public static void main(String[] args)throws Exception {
32.         Boolean b1=new Boolean("true");
33.         Boolean b2=new Boolean("True");
34.         Boolean b3=new Boolean("false");
35.         Boolean b4=new Boolean("False");
36.         Boolean b5=new Boolean("ashok");
```

```

37.         Boolean b6=new Boolean("TRUE");
38.         System.out.println(b1);//true
39.         System.out.println(b2);//true
40.         System.out.println(b3);//false
41.         System.out.println(b4);//false
42.         System.out.println(b5);//false
43.         System.out.println(b6);//true
44.     }
45. }
46. Example 2(for exam purpose):
47. class WrapperClassDemo {
48.     public static void main(String[] args)throws Exception {
49.         Boolean b1=new Boolean("yes");
50.         Boolean b2=new Boolean("no");
51.         System.out.println(b1);//false
52.         System.out.println(b2);//false
53.         System.out.println(b1.equals(b2));//true
54.         System.out.println(b1==b2);//false
55.     }
56. }

```

## Wrapper class Constructor summery :

| Wrapper class | Constructor summery     |
|---------------|-------------------------|
| Byte          | byte, String            |
| Short         | short, String           |
| Integer       | Int, String             |
| Long          | long, String            |
| Float         | float, String, double   |
| Double        | double, String          |
| Character     | <del>char, String</del> |
| Boolean       | boolean, String         |

### Note :

- 1) In all wrapper classes toString() method is overridden to return its content.
- 2) In all wrapper classes .equals() method is overridden for content comparison.

### Example :

```

Integer i1 = new Integer(10) ;
Integer i2 = new Integer(10);
System.out.println(i1); //10
System.out.println(i1.equals(i2)); //true

```

## Utility methods :

1. `valueOf()` method.
2. `XXXValue()` method.
3. `parseXxx()` method.
4. `toString()` method.

## valueOf() method :

We can use `valueOf()` method to create wrapper object for the given primitive or String this method is alternative to constructor.

### Form 1:

Every wrapper class except Character class contains a static `valueOf()` method to create wrapper object for the given String.

`public static wrapper valueOf(String s);`

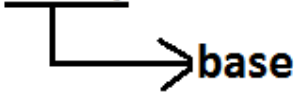
Example:

```
class WrapperClassDemo {
    public static void main(String[] args) throws Exception {
        Integer i=Integer.valueOf("10");
        Double d=Double.valueOf("10.5");
        Boolean b=Boolean.valueOf("ashok");
        System.out.println(i);//10
        System.out.println(d);//10.5
        System.out.println(b);//false
    }
}
```

### Form 2:

Every integral type wrapper class (Byte, Short, Integer, and Long) contains the following `valueOf()` method to convert specified radix string to wrapper object.

`public static Integer valueOf(String s, int radix)`



`public static wrapper valueOf(String s , int radix ) ;`

//radix means base

### Note:

the allowed radix range is 2 to 36.



base 2 ————— 0 To 1  
 base 8 ————— 0 To 7  
 base 10 ————— 0 To 9  
 base 11 ————— 0 To 9,a  
 base 16 ————— 0 To 9,a To f  
 base 36 ————— 0 To 9,a to z

Example:

```

class WrapperClassDemo {
    public static void main(String[] args) {
        Integer i=Integer.valueOf("100",2);
        System.out.println(i);//4
    }
}
  
```

Analysis:

|                |                |                |                               |
|----------------|----------------|----------------|-------------------------------|
| 1              | 0              | 0              | $2^0 * 0 + 2^1 * 0 + 2^2 * 1$ |
|                |                |                |                               |
| 2 <sup>2</sup> | 2 <sup>1</sup> | 2 <sup>0</sup> | $1 * 0 + 2 * 0 + 4 * 1$       |
|                |                |                | $0 + 0 + 4 = 4$               |

Form 3 :

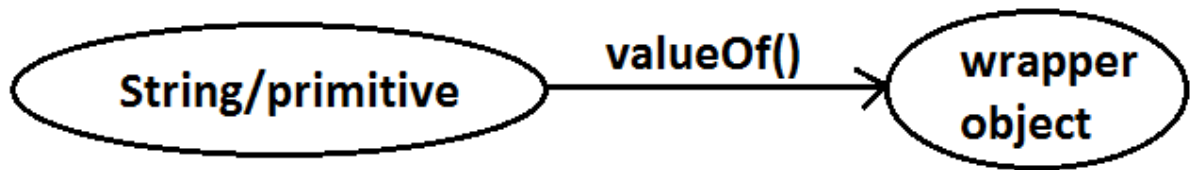
Every wrapper class including Character class defines valueOf() method to convert primitive to wrapper object.

public static wrapper valueOf(primitive p);

Example:

```

class WrapperClassDemo {
    public static void main(String[] args) throws Exception {
        Integer i=Integer.valueOf(10);
        Double d=Double.valueOf(10.5);
        Boolean b=Boolean.valueOf(true);
        Character ch=Character.valueOf('a');
        System.out.println(ch); //a
        System.out.println(i);//10
        System.out.println(d);//10.5
        System.out.println(b);//true
    }
}
  
```

Diagram:xxxValue() method :

We can use xxxValue() methods to convert wrapper object to primitive.

Every number type wrapper class (Byte, Short, Integer, Long, Float, Double) contains the following 6 xxxValue() methods to convert wrapper object to primitives.

- 1) public byte byteValue()
- 2) public short shortValue()
- 3) public int intValue()
- 4) public long longValue()
- 5) public float floatValue()
- 6) public double doubleValue();

Example:

```

class WrapperClassDemo {
    public static void main(String[] args) throws Exception {
        Integer i = new Integer(130);
        System.out.println(i.byteValue()); // -126
        System.out.println(i.shortValue()); // 130
        System.out.println(i.intValue()); // 130
        System.out.println(i.longValue()); // 130
        System.out.println(i.floatValue()); // 130.0
        System.out.println(i.doubleValue()); // 130.0
    }
}
  
```

charValue() method:

Character class contains charValue() method to convert Character object to char primitive.

```
public char charValue();
```

Example:

```

class WrapperClassDemo {
    public static void main(String[] args) {
        Character ch = new Character('a');
        char c = ch.charValue();
        System.out.println(c); // a
    }
}
  
```

## booleanValue() method:

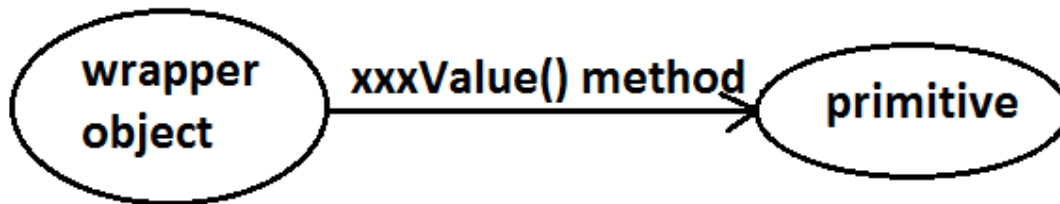
Boolean class contains booleanValue() method to convert Boolean object to boolean primitive.

public boolean booleanValue();

Example:

```
class WrapperClassDemo {
    public static void main(String[] args) {
        Boolean b=new Boolean("ashok");
        boolean b1=b.booleanValue();
        System.out.println(b1);//false
    }
}
```

Diagram :



In total there are 38(= 6\*6+1+1) xxxValue() methods are possible.

## parseXxx() method :

We can use this method to convert String to corresponding primitive.

Form1 :

Every wrapper class except Character class contains a static parseXxx() method to convert String to corresponding primitive.

public static primitive parseXxx(String s);

Example:

```
class WrapperClassDemo {
    public static void main(String[] args) {
        int i=Integer.parseInt("10");
        boolean b=Boolean.parseBoolean("ashok");
        double d=Double.parseDouble("10.5");
        System.out.println(i);//10
        System.out.println(b);//false
        System.out.println(d);//10.5
    }
}
```

```
}
```

**Form 2:**

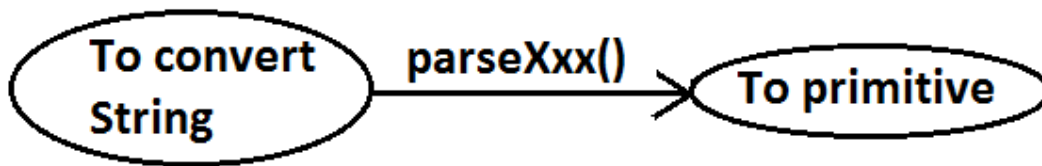
integral type wrapper classes(Byte, Short, Integer, Long) contains the following `parseXxx()` method to convert specified radix String form to corresponding primitive.

`public static primitive parseXxx(String s,int radix);`

The allowed range of radix is : 2 to 36

Example:

```
class WrapperClassDemo {  
    public static void main(String[] args) {  
        int i=Integer.parseInt("100",2);  
        System.out.println(i);//4  
    }  
}
```

**Diagram :****toString() method :**

We can use `toString()` method to convert wrapper object (or) primitive to String.

**Form 1 :**

`public String toString();`

1. Every wrapper class (including Character class) contains the above `toString()` method to convert wrapper object to String.
2. It is the overriding version of Object class `toString()` method.
3. Whenever we are trying to print wrapper object reference internally this `toString()` method only executed.

Example:

```
class WrapperClassDemo {  
    public static void main(String[] args) {  
        Integer i=Integer.valueOf("10");  
        System.out.println(i);//10  
        System.out.println(i.toString());//10  
    }  
}
```

**Form 2:**

Every wrapper class contains a static toString() method to convert primitive to String.

**public static String toString(primitive p);**

Example:

```
class WrapperClassDemo {
    public static void main(String[] args)    {
        String s1=Integer.toString(10);
        String s2=Boolean.toString(true);
        String s3=Character.toString('a');
        System.out.println(s1);              //10
        System.out.println(s2);              //true
        System.out.println(s3);              //a
    }
}
```

**Form 3:**

Integer and Long classes contains the following static toString() method to convert the primitive to specified radix String form.

**public static String toString(primitive p, int radix);**

Example:

```
class WrapperClassDemo {
    public static void main(String[] args)    {
        String s1=Integer.toString(7,2);
        String s2=Integer.toString(17,2);
        System.out.println(s1);              //111
        System.out.println(s2);              //10001
    }
}
```

**Form 4:**

Integer and Long classes contains the following toXxxString() methods.

**public static String toBinaryString(primitive p);**

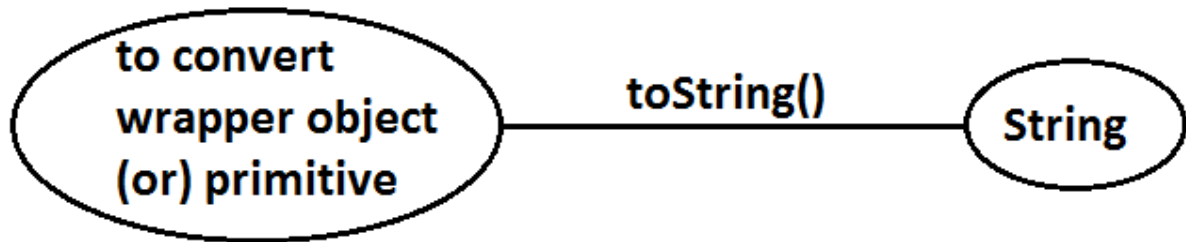
**public static String toOctalString(primitive p);**

**public static String toHexString(primitive p);**

Example:

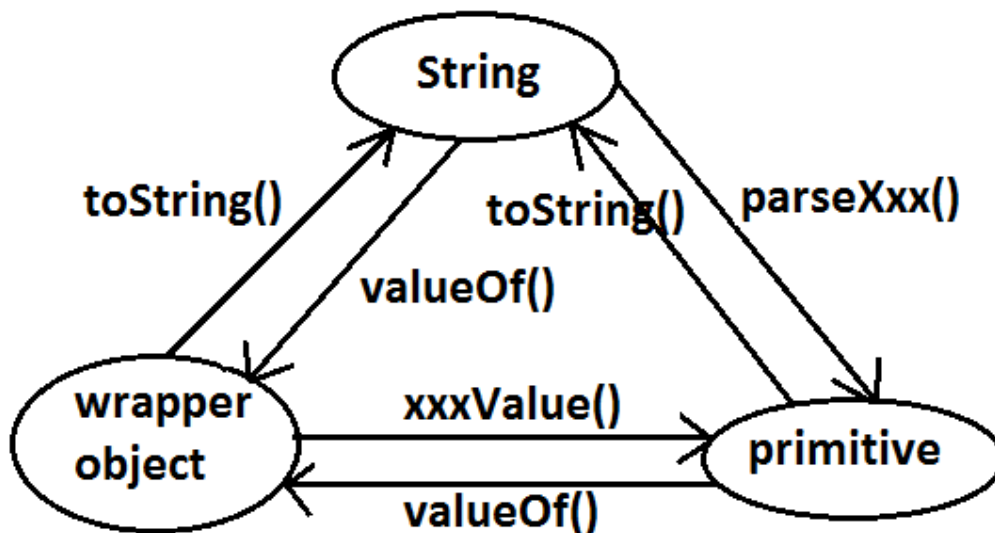
```
class WrapperClassDemo {
    public static void main(String[] args)    {
        String s1=Integer.toBinaryString(7);
        String s2=Integer.toOctalString(10);
        String s3=Integer.toHexString(20);
        String s4=Integer.toHexString(10);
        System.out.println(s1);              //111
        System.out.println(s2);              //12
        System.out.println(s3);              //14
        System.out.println(s4);              //a
    }
}
```

Diagram:



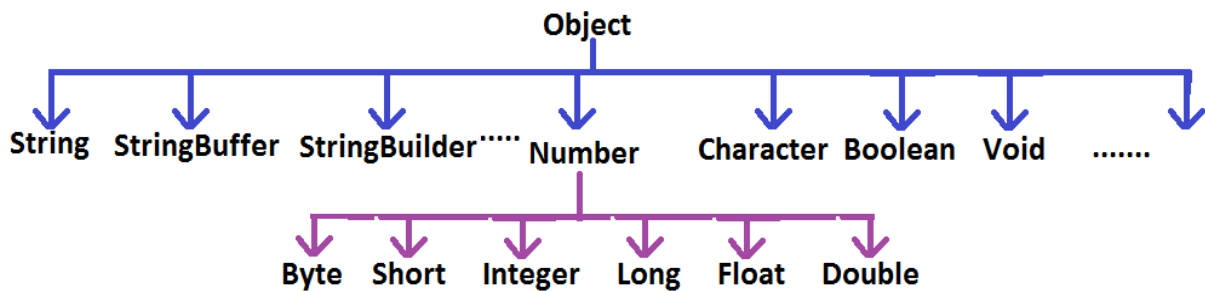
### Dancing between String, wrapper object and primitive :

Diagram:



## Partial Hierarchy of java.lang package :

Diagram :



1. String, StringBuffer, StringBuilder and all wrapper classes are final classes.
2. The wrapper classes which are not child class of Number are Boolean and Character.
3. The wrapper classes which are not direct class of Object are Byte, Short, Integer, Long, Float, Double.
4. Sometimes we can consider Void is also as wrapper class.
5. In addition to String objects , all wrapper class objects also immutable in java.

## Void :

1. Sometimes Void class is also considered as wrapper class.
2. Void is class representation of void java keyword.
3. Void class is the direct child class of Object and it doesn't contains any method and it contains only one static variable Void.TYPE
4. we can use Void class in reflections

Ex : To check whether return type of m1( ) is void or not .

```

if(ob.getClass( ).getMethod("m1").getReturnType( )==Void.TYPE) {
    .....
}
  
```

## Autoboxing and Autounboxing (1.5v):

Until 1.4 version we can't provide wrapper object in the place of primitive and primitive in the place of wrapper object all the required conversions should be performed explicitly by the programmer.

**Example 1 :****Program 1 :**

```

class AutoBoxingAndUnboxingDemo
{
    public static void main(String[] args)
    {
        Boolean b=new Boolean(true);
        if(b)
        {
            System.out.println("hello");
        }
    }
}

```

E:\scjp>javac -source 1.4 AutoBoxingAndUnboxingDemo.java  
 AutoBoxingAndUnboxingDemo.java:6: incompatible types  
 found : java.lang.Boolean  
 required: boolean

**Program 2:**

```

class AutoBoxingAndUnboxingDemo {
    public static void main(String[] args) {
        Boolean b=new Boolean(true);
        if(b) {
            System.out.println("hello");
        }
    }
}
Output:
hello

```

**Example 2:****Program 1:**

```

import java.util.*;
class AutoBoxingAndUnboxingDemo
{
    public static void main(String[] args)
    {
        ArrayList l=new ArrayList();
        l.add(10);
    }
}

```

E:\scjp>javac -source 1.4 AutoBoxingAndUnboxingDemo.java  
 AutoBoxingAndUnboxingDemo.java:7: cannot find symbol  
 symbol : method add(int)  
 location: class java.util.ArrayList

**Program 2:**

```

import java.util.*;
class AutoBoxingAndUnboxingDemo {
    public static void main(String[] args) {
        ArrayList l=new ArrayList();
        Integer i=new Integer(10);
        l.add(i);
    }
}

```



But from 1.5 version onwards we can provide primitive value in the place of wrapper and wrapper object in the place of primitive all required conversions will be performed automatically by compiler. These automatic conversions are called Autoboxing and Autounboxing.

### Autoboxing :

Automatic conversion of primitive to wrapper object by compiler is called Autoboxing.

#### Example :

```
Integer i=10;
```

[compiler converts "int" to "Integer" automatically by Autoboxing]

After compilation the above line will become.

```
Integer i=Integer.valueOf(10);
```

That is internally Autoboxing concept is implemented by using valueOf() method.

### Autounboxing :

automatic conversion of wrapper object to primitive by compiler is called Autounboxing.

#### Example:

```
Integer I=new Integer(10);
```

```
Int i=I;
```

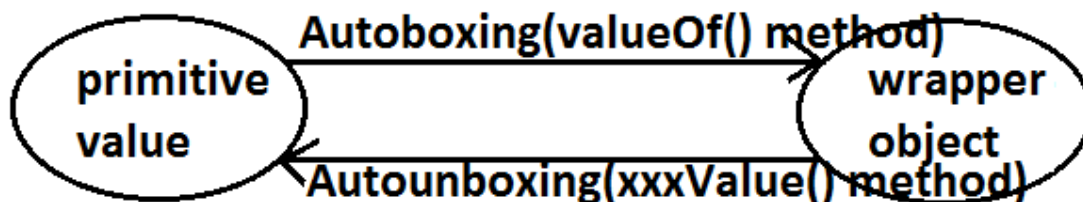
[ compiler converts "Integer" to "int" automatically by Autounboxing ]

After compilation the above line will become.

```
Int i=I.intValue();
```

That is Autounboxing concept is internally implemented by using xxxValue() method.

#### Diagram :



#### Example :

```

import java.util.*;
class AutoBoxingAndUnboxingDemo
{
    static Integer I=10;—————① Autoboxing.
    public static void main(String[] args)
    {
        int i=I;—————② Autounboxing
        methodOne(i);—————③ Autoboxing.
    }
    public static void methodOne(Integer I)
    {
        int k=I;—————④ Autounboxing
        System.out.println(k);//10
    }
}

```

It is valid in 1.5 version but invalid in 1.4 version.

**Note:**

From 1.5 version onwards we can use primitives and wrapper objects interchangeably the required conversions will be performed automatically by compiler.

**Example 1:**

```

import java.util.*;
class AutoBoxingAndUnboxingDemo {
    static Integer I=0;
    public static void main(String[] args) {
        int i=I;
        System.out.println(i);//0
    }
}

```

**Example 2 :**

```

import java.util.*;
class AutoBoxingAndUnboxingDemo
{
    static Integer l;    null
    public static void main(String[] args)
    {
        int i=l;         → R.E:NullPointerException
        System.out.println(i);
        }
    }

```

→ int i=l.intValue();

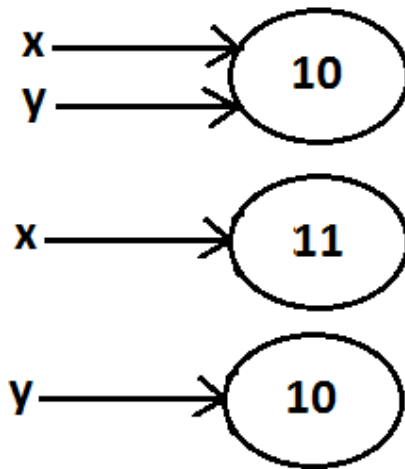
If we provide null reference for autounboxing , we will get NullPointerException

Example 3:

```

import java.util.*;
class AutoBoxingAndUnboxingDemo {
    public static void main(String[] args) {
        Integer x=10;
        Integer y=x;
        ++x;
        System.out.println(x);//11
        System.out.println(y);//10
        System.out.println(x==y);//false
    }
}

```

Diagram :Note :

All wrapper objects are immutable that is once we created a wrapper object we can't perform any changes in the existing object.

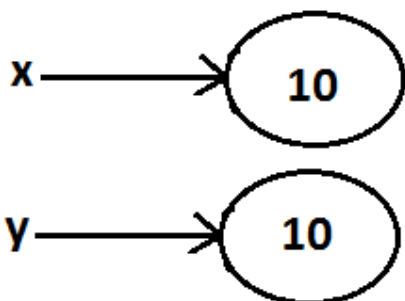
If we are trying to perform any changes with those changes a new object will be created.

• ----  
• ----  
• ----  
• ----  
• ----  
• ----  
• ----  
• ----  
• ----  
• ----

## Example 1:

```

import java.util.*;
class AutoBoxingAndUnboxingDemo {
    public static void main(String[] args) {
        Integer x=new Integer(10);
        Integer y=new Integer(10);
        System.out.println(x==y);//false
    }
}
  
```

Diagram :

## Example 2 :

```

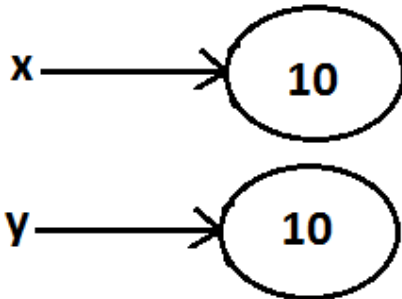
import java.util.*;
  
```

```

class AutoBoxingAndUnboxingDemo {
    public static void main(String[] args) {
        Integer x=new Integer(10);
        Integer y=10;
        System.out.println(x==y);//false
    }
}

```

Diagram:



Example 3:

```

import java.util.*;
class AutoBoxingAndUnboxingDemo {
    public static void main(String[] args) {
        Integer x=new Integer(10);
        Integer y=x;
        System.out.println(x==y);//true
    }
}

```

Diagram :



Example 4 :

```

import java.util.*;
class AutoBoxingAndUnboxingDemo {
    public static void main(String[] args) {
        Integer x=10;
        Integer y=10;
        System.out.println(x==y);//true
    }
}

```

Diagram:



Example 5 :

```

import java.util.*;
class AutoBoxingAndUnboxingDemo {

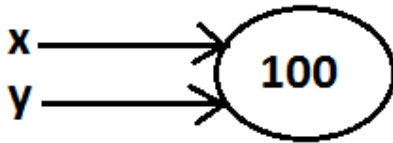
```

```

    public static void main(String[] args)    {
        Integer x=100;
        Integer y=100;
        System.out.println(x==y);//true
    }
}

```

Diagram :



Example 6 :

```

import java.util.*;
class AutoBoxingAndUnboxingDemo {
    public static void main(String[] args)    {
        Integer x=1000;
        Integer y=1000;
        System.out.println(x==y);//false
    }
}

```

Diagram :

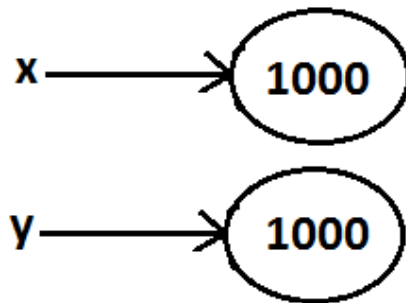
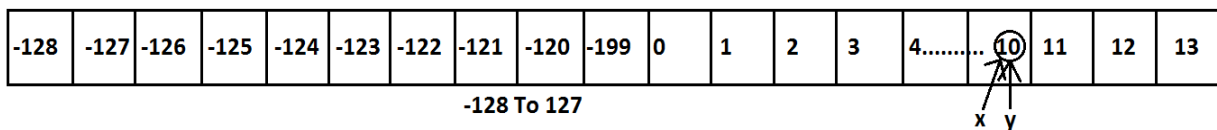


Diagram :



## Conclusions :

1. To implement the Autoboxing concept in every wrapper class a buffer of objects will be created at the time of class loading.
2. By Autoboxing if an object is required to create 1st JVM will check whether that object is available in the buffer or not.

3. If it is available then JVM will reuse that buffered object instead of creating new object.
4. If the object is not available in the buffer then only a new object will be created.  
This approach improves performance and memory utilization.

But this buffer concept is available only in the following cases :

|           |             |
|-----------|-------------|
| Byte      | Always      |
| Short     | -128 To 127 |
| Integer   | -128 To 127 |
| Long      | -128 To 127 |
| Character | 0 To 127    |
| Boolean   | Always      |

In all the remaining cases compulsory a new object will be created.

Examples :

- |                                                                                                                                                                |                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>① Integer x=127;<br/>Integer y=127;<br/>System.out.println(x==y);//true</p> <p>② Integer x=128;<br/>Integer y=128;<br/>System.out.println(x==y);//false</p> | <p>③ Boolean b1=true;<br/>Boolean b2=true;<br/>System.out.println(b1==b2);//true</p> <p>④ Double d1=10.0;<br/>Double d2=10.0;<br/>System.out.println(d1==d2);//false</p> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Internally Autoboxing concept is implemented by using valueOf() method hence the above rule applicable even for valueOf() method also.

Examples :

- |                                                                                                                                                                                      |                                                                                                                                                                                                              |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>① Integer x=new Integer(10);<br/>Integer y=new Integer(10);<br/>System.out.println(x==y);//false</p> <p>② Integer x=10;<br/>Integer y=10;<br/>System.out.println(x==y);//true</p> | <p>③ Integer x=Integer.valueOf(10);<br/>Integer y=Integer.valueOf(10);<br/>System.out.println(x==y);//true</p> <p>④ Integer x=10;<br/>Integer y=Integer.valueOf(10);<br/>System.out.println(x==y);//true</p> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Note:**

When compared with constructors it is recommended to use `valueOf()` method to create wrapper object.

## Overloading with respect to widening, Autoboxing and var-arg methods :

- Case 1: Widening vs Autoboxing :

**Widening:**

Converting a lower data type into a higher data type is called widening.

Example:

```
import java.util.*;
class AutoBoxingAndUnboxingDemo {
    public static void methodOne(long l) {
        System.out.println("widening");
    }
    public static void methodOne(Integer i) {
        System.out.println("autoboxing");
    }
    public static void main(String[] args) {
        int x=10;
        methodOne(x);
    }
}
```

Output:  
Widening

Widening dominates Autoboxing.

- Case 2: Widening vs var-arg method :

- 

- Example:

```
import java.util.*;
class AutoBoxingAndUnboxingDemo {
    public static void methodOne(long l) {
        System.out.println("widening");
    }
    public static void methodOne(int... i) {
        System.out.println("var-arg method");
    }
    public static void main(String[] args) {
        int x=10;
        methodOne(x);
    }
}
```



- }
- Output:
- Widening

Widening dominates var-arg method.

### • Case 3: Autoboxing vs var-arg method :

- 
- Example:
- `import java.util.*;`
- `class AutoBoxingAndUnboxingDemo {`
- `public static void methodOne(Integer i) {`
- `System.out.println("Autoboxing");`
- `}`
- `public static void methodOne(int... i) {`
- `System.out.println("var-arg method");`
- `}`
- `public static void main(String[] args) {`
- `int x=10;`
- `methodOne(x);`
- `}`
- `}`
- Output:
- Autoboxing

Autoboxing dominates var-arg method.

In general var-arg method will get least priority i.e., if no other method matched then only var-arg method will get chance. It is exactly same as "default" case inside a switch.

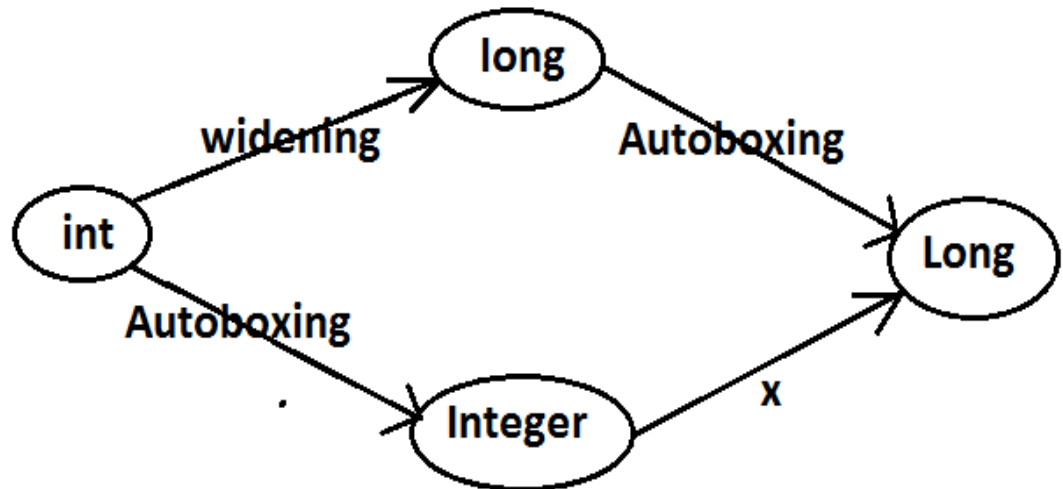
**Note :** While resolving overloaded methods compiler will always gives the precedence in the following order :

1. Widening
2. Autoboxing
3. Var-arg method.

- 
- 
- Case 4:
- `import java.util.*;`
- `class AutoBoxingAndUnboxingDemo {`
- `public static void methodOne(Long l) {`
- `System.out.println("Long");`
- `}`
- `public static void main(String[] args) {`
- `int x=10;`
- `methodOne(x);`
- `}`
- `}`
- Output:

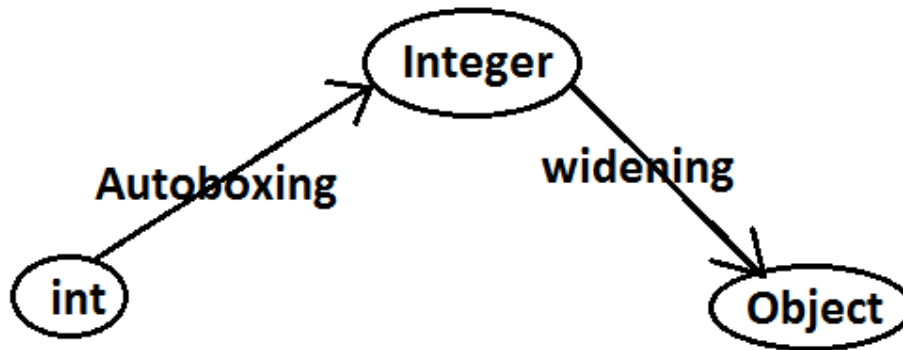
- `methodOne(java.lang.Long)` in `AutoBoxingAndUnboxingDemo` cannot be applied to `(int)`

**Diagram:**



- **imp :**  
Widening followed by Autoboxing is not allowed in java but Autoboxing followed by widening is allowed.
- 
- **Case 5:**
- `import java.util.*;`
- `class AutoBoxingAndUnboxingDemo {`
- `public static void methodOne(Object o) {`
- `System.out.println("Object");`
- `}`
- `public static void main(String[] args) {`
- `int x=10;`
- `methodOne(x);`
- `}`
- `}`
- **Output:**
- **Object**

- Diagram :



- Which of the following declarations are valid ?
  1. `int i=10 ; //valid`
  2. `Integer I=10 ; //valid`
  3. `int i=10L ; //invalid CE :`
  4. `Long l = 10L ; // valid`
  5. `Long l = 10 ; // invalid CE :`
  6. `long l = 10 ; //valid`
  7. `Object o=10 ; //valid (autoboxing followed by widening)`
  8. `double d=10 ; //valid`
  9. `Double d=10 ; //invalid`
  10. `Number n=10; //valid (autoboxing followed by widening)`

# **CORE JAVA**

## **With**

# **SCJP / OCJP**

### **Study Material**

**Chapter 11 : File IO**



**DURGA M.Tech**

**(Sun certified & Realtime Expert)**

**Ex. IBM Employee**

**Trained Lakhs of Students  
for last 14 years across INDIA**

**India's No.1 Software Training Institute**

# **DURGASOFT**

**www.durgasoft.com Ph: 9246212143 ,8096969696**

## File IO

### Agenda:

1. File
2. FileWriter
3. FileReader
4. BufferedWriter
5. BufferedReader

### File:

`File f=new File("abc.txt");`

This line 1st checks whether abc.txt file is already available (or) not if it is already available then "f" simply refers that file.

If it is not already available then it won't create any physical file just creates a java File object represents name of the file.

### Example:

```
import java.io.*;
class FileDemo
{
    public static void main(String[] args)throws IOException
    {
        File f=new File("cricket.txt");
        System.out.println(f.exists()); //false
        f.createNewFile();
        System.out.println(f.exists()); //true
    }
}
```

output :

1<sup>st</sup> run :

false

true

2<sup>nd</sup> run :

true

true

A java File object can represent a directory also.

### Example:

```
import java.io.*;
class FileDemo
{
    public static void main(String[] args)throws IOException
    {
```

```

        File f=new File("cricket123");
        System.out.println(f.exists()); //false
        f.mkdir();
        System.out.println(f.exists()); //true
    }
}

```

**Note:** in UNIX everything is a file, java "file IO" is based on UNIX operating system hence in java also we can represent both files and directories by File object only.

**File class constructors:**

1. `File f=new File(String name);`  
Creates a java File object that represents name of the file or directory in current working directory.
2. `File f=new File(String subdirname,String name);`  
Creates a File object that represents name of the file or directory present in specified sub directory.
3. `File f=new File(File subdir,String name);`

**Requirement:** Write code to create a file named with demo.txt in current working directory.

**Program:**

```

import java.io.*;
class FileDemo
{
    public static void main(String[] args)throws IOException
    {
        File f=new File("demo.txt");
        f.createNewFile();
    }
}

```

**Requirement:** Write code to create a directory named with SaiCharan123 in current working directory and create a file named with abc.txt in that directory.

**Program:**

```

import java.io.*;
class FileDemo
{
    public static void main(String[] args)throws IOException
    {
        File f1=new File("SaiCharan123");
        f1.mkdir();
        File f2=new File("SaiCharan123","abc.txt");
        f2.createNewFile();
    }
}

```

**Requirement:** Write code to create a file named with demo.txt present in c:\saicharan folder.

**Program:**

```

import java.io.*;

```

```
class FileDemo
{
    public static void main(String[] args)throws IOException
    {
        File f=new File("c:\\saiCharan","demo.txt");
        f.createNewFile();
    }
}
```

### Import methods of file class:

1. boolean exists();  
Returns true if the physical file or directory available.
2. boolean createNewFile();  
This method 1st checks whether the physical file is already available or not if it is already available then this method simply returns false without creating any physical file.  
If this file is not already available then it will create a new file and returns true
3. boolean mkdir();  
This method 1st checks whether the directory is already available or not if it is already available then this method simply returns false without creating any directory.  
If this directory is not already available then it will create a new directory and returns true
4. boolean isFile();  
Returns true if the File object represents a physical file.
5. boolean isDirectory();  
Returns true if the File object represents a directory.
6. String[] list();  
It returns the names of all files and subdirectories present in the specified directory.
7. long length();  
Returns the no of characters present in the file.
8. boolean delete();  
To delete a file or directory.

**Requirement:** Write a program to display the names of all files and directories present in c:\\charan\_classes.

**Program:**

```
import java.io.*;
class FileDemo
{
    public static void main(String[] args)throws IOException
    {
        int count=0;
        File f=new File("c:\\charan_classes");
```

```

        String[] s=f.list();
        for(String s1=s) {
            count++;
            System.out.println(s1);
        }
        System.out.println("total number : "+count);
    }
}

```

**Requirement:** Write a program to display only file names

**Program:**

```

import java.io.*;
class FileDemo
{
    public static void main(String[] args)throws IOException
    {
        int count=0;
        File f=new File("c:\\charan_classes");
        String[] s=f.list();
        for(String s1=s) {
            File f1=new file(f,s1);
            if(f1.isFile()) {
                count++;
                System.out.println(s1);
            }
        }
        System.out.println("total number : "+count);
    }
}

```

**Requirement:** Write a program to display only directory names

**Program:**

```

import java.io.*;
class FileDemo
{
    public static void main(String[] args)throws IOException
    {
        int count=0;
        File f=new File("c:\\charan_classes");
        String[] s=f.list();
        for(String s1=s) {
            File f1=new file(f,s1);
            if(f1.isDirectory()) {
                count++;
                System.out.println(s1);
            }
        }
        System.out.println("total number : "+count);
    }
}

```



## FileWriter:

By using FileWriter object we can write character data to the file.

## Constructors:

```
FileWriter fw=new FileWriter(String name);  
FileWriter fw=new FileWriter(File f);
```

The above 2 constructors meant for overriding.

Instead of overriding if we want append operation then we should go for the following 2 constructors.

```
FileWriter fw=new FileWriter(String name,boolean append);  
FileWriter fw=new FileWriter(File f,boolean append);
```

If the specified physical file is not already available then these constructors will create that file.

## Methods:

1. `write(int ch);`  
To write a single character to the file.
2. `write(char[] ch);`  
To write an array of characters to the file.
3. `write(String s);`  
To write a String to the file.
4. `flush();`  
To give the guarantee the total data include last character also written to the file.
5. `close();`  
To close the stream.

Example:

```
import java.io.*;  
class FileWriterDemo  
{  
    public static void main(String[] args)throws IOException  
    {  
        FileWriter fw=new FileWriter("cricket.txt",true);  
        fw.write(99);//adding a single character  
        fw.write("haran\nsoftware solutions");  
        fw.write("\n");  
        char[] ch={'a','b','c'};  
        fw.write(ch);  
        fw.write("\n");  
        fw.flush();  
        fw.close();  
    }  
}
```

```

    }
}
Output:
charan
software solutions
abc
Note :

```

- The main problem with `FileWriter` is we have to insert line separator manually , which is difficult to the programmer. ('\\n')
- And even line separator varing from system to system.

### FileReader:

By using `FileReader` object we can read character data from the file.

### Constructors:

```

FileReader fr=new FileReader(String name);
FileReader fr=new FileReader (File f);

```

### Methods:

1. `int read();`  
It attempts to read next character from the file and return its Unicode value. If the next character is not available then we will get -1.
2. `int i=fr.read();`
3. `System.out.println((char)i);`

As this method returns unicode value , while printing we have to perform type casting.

4. `int read(char[] ch);`  
It attempts to read enough characters from the file into `char[]` array and returns the no of characters copied from the file into `char[]` array.
5. `File f=new File("abc.txt");`
6. `Char[] ch=new Char[(int)f.length()];`

7. `void close();`

Approach 1:

```

import java.io.*;
class FileReaderDemo
{
    public static void main(String[] args)throws IOException

```

```

    {
        FileReader fr=new FileReader("cricket.txt");
        int i=fr.read();    //more amount of data
        while(i!=-1)
        {
            System.out.print((char)i);
            i=fr.read();
        }
    }
}

```

Output:

Charan

Software solutions

ABC

Approach 2:

```
import java.io.*;
```

```
class FileReaderDemo
```

```

{
    public static void main(String[] args)throws IOException
    {
        File f=new File("cricket.txt");
        FileReader fr=new FileReader(f);
        char[] ch=new char[(int)f.length()]; //small
amount of data
        fr.read(ch);
        for(char ch1:ch)
        {
            System.out.print(ch1);
        }
    }
}

```

Output:

XYZ

Software solutions.

Usage of FileWriter and FileReader is not recommended because :

1. While writing data by FileWriter compulsory we should insert line separator(\n) manually which is a bigger headache to the programmer.
2. While reading data by FileReader we have to read character by character instead of line by line which is not convenient to the programmer.
3. To overcome these limitations we should go for BufferedWriter and BufferedReader concepts.

### **BufferedWriter:**

By using BufferedWriter object we can write character data to the file.

### Constructors:

```
BufferedWriter bw=new BufferedWriter(writer w);
```

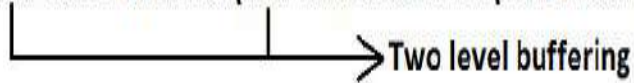
```
BufferedWriter bw=new BufferedWriter(writer w,int buffersize);
```

**Note:** `BufferedWriter` never communicates directly with the file it should communicate via some writer object.

**Which of the following declarations are valid?**

1. `BufferedWriter bw=new BufferedWriter("cricket.txt");` (invalid)
2. `BufferedWriter bw=new BufferedWriter (new File("cricket.txt"));` (invalid)
3. `BufferedWriter bw=new BufferedWriter (new FileWriter("cricket.txt"));` (valid)

4) `BufferedWriter bw=new BufferedWriter(new BufferedWriter(new FileWriter("cricket.txt")));` (valid)



**Methods:**

1. `write(int ch);`
2. `write(char[] ch);`
3. `write(String s);`
4. `flush();`
5. `close();`
6. `newline();`  
Inserting a new line character to the file.

When compared with `FileWriter` which of the following capability(facility) is available as method in `BufferedWriter`.

1. Writing data to the file.
2. Closing the writer.
3. Flush the writer.
4. Inserting newline character.

Ans : 4

Example:

```
import java.io.*;
class BufferedWriterDemo
{
    public static void main(String[] args)throws IOException
    {
        FileWriter fw=new FileWriter("cricket.txt");
        BufferedWriter bw=new BufferedWriter(fw);
        bw.write(100);
        bw.newLine();
        char[] ch={'a','b','c','d'};
        bw.write(ch);
        bw.newLine();
        bw.write("SaiCharan");
        bw.newLine();
        bw.write("software solutions");
    }
}
```

```

        bw.flush();
        bw.close();
    }
}

```

Output:

d

abcd

SaiCharan

software solutions

**Note :** When ever we are closing BufferedWriter automatically underlying writer will be closed and we are not close explicitly.

### **BufferedReader:**

This is the most enhanced(better) Reader to read character data from the file.

#### Constructors:

```
BufferedReader br=new BufferedReader(Reader r);
```

```
BufferedReader br=new BufferedReader(Reader r,int buffersize);
```

**Note:** BufferedReader can not communicate directly with the File it should communicate via some Reader object.

*The main advantage of BufferedReader over FileReader is we can read data line by line instead of character by character.*

#### Methods:

1. int read();
2. int read(char[] ch);
3. String readLine();  
It attempts to read next line and return it , from the File. if the next line is not available then this method returns null.
4. void close();

Example:

```

import java.io.*;
class BufferedReaderDemo
{
    public static void main(String[] args)throws IOException
    {
        FileReader fr=new FileReader("cricket.txt");
        BufferedReader br=new BufferedReader(fr);
        String line=br.readLine();
        while(line!=null)
        {
            System.out.println(line);
            line=br.readLine();
        }
        br.close();
    }
}

```

```
    }
}
```

**Note:**

- Whenever we are closing **BufferedReader** automatically underlying **FileReader** will be closed it is not required to close explicitly.
- Even this rule is applicable for **BufferedWriter** also.

```
fr.close(); | br.close(); | fr.close();
  x         ✓             x
```

**PrintWriter:**

- This is the most enhanced **Writer** to write text data to the file.
- By using **FileWriter** and **BufferedWriter** we can write only character data to the File but by using **PrintWriter** we can write any type of data to the File.

**Constructors:**

```
PrintWriter pw=new PrintWriter(String name);
```

```
PrintWriter pw=new PrintWriter(File f);
```

```
PrintWriter pw=new PrintWriter(Writer w);
```

**PrintWriter** can communicate either directly to the File or via some **Writer** object also.

**Methods:**

1. write(int ch);
2. write (char[] ch);
3. write(String s);
4. flush();
5. close();
6. print(char ch);
7. print (int i);
8. print (double d);
9. print (boolean b);
10. print (String s);
11. println(char ch);
12. println (int i);
13. println(double d);
14. println(boolean b);
15. println(String s);

Example:

```
import java.io.*;
class PrintWriterDemo {
    public static void main(String[] args)throws IOException
    {
        FileWriter fw=new FileWriter("cricket.txt");
        PrintWriter out=new PrintWriter(fw);
        out.write(100);
        out.println(100);
        out.println(true);
        out.println('c');
        out.println("SaiCharan");
        out.flush();
        out.close();
    }
}
```

Output:

```
d100
true
c
SaiCharan
```

What is the difference between write(100) and print(100)?

In the case of write(100) the corresponding character "d" will be added to the File but in the case of print(100) "100" value will be added directly to the File.

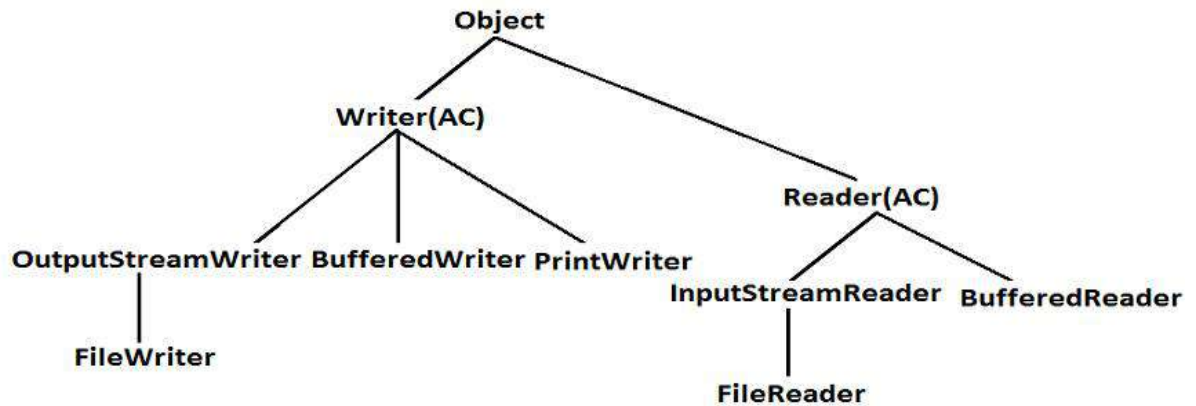
**Note 1:**

1. The most enhanced Reader to read character data from the File is **BufferedReader**.
2. The most enhanced Writer to write character data to the File is **PrintWriter**.

**Note 2:**

1. In general we can use Readers and Writers to handle character data. Where as we can use InputStreams and OutputStreams to handle binary data(like images, audio files, video files etc).
2. We can use OutputStream to write binary data to the File and we can use InputStream to read binary data from the File.

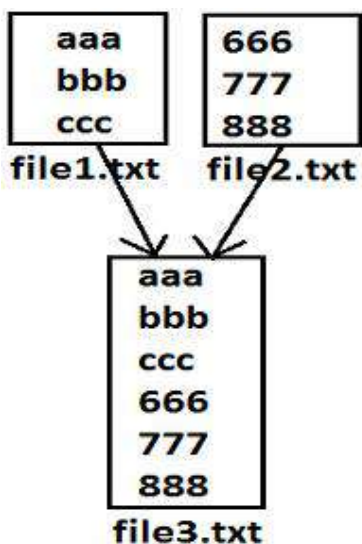
Diagram:



Requirement:

Write a program to perform File merge(combine) operation.

Diagram:



Program:

```

import java.io.*;
class FileWriterDemo1
{
    public static void main(String[] args) throws IOException {
        PrintWriter pw=new PrintWriter("file3.txt");
        BufferedReader br=new BufferedReader(new
        FileReader("file1.txt"));
        String line=br.readLine();
        while(line!=null)
        {
            pw.println(line);
        }
    }
}
  
```



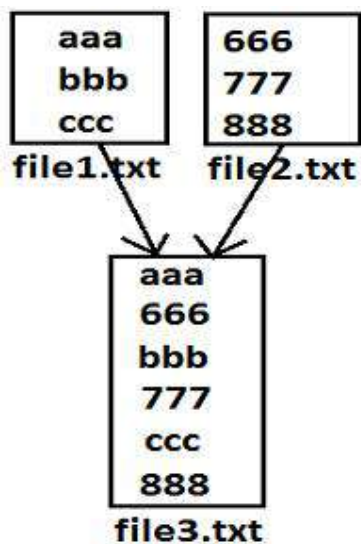
```

        line=br.readLine();
    }
    br=new BufferedReader(new FileReader("file2.txt"));
//reuse
    line=br.readLine();
    while(line!=null)
    {
        pw.println(line);
        line=br.readLine();
    }
    pw.flush();
    br.close();
    pw.close();
}

```

**Requirement:** Write a program to perform file merge operation where merging should be performed line by line alternatively.

Diagram:



Program:

```

import java.io.*;
class FileWriterDemo1 {
    public static void main(String[] args)throws IOException {
        PrintWriter pw=new PrintWriter("file3.txt");
        BufferedReader br1=new BufferedReader(new
FileReader("file1.txt"));
        BufferedReader br2=new BufferedReader(new
FileReader("file2.txt"));
        String line1=br1.readLine();
        String line2=br2.readLine();
        while(line1!=null||line2!=null)

```

```

    {
        if(line1!=null)
        {
            pw.println(line1);
            line1=br1.readLine();
        }
        if(line2!=null)
        {
            pw.println(line2);
            line2=br2.readLine();
        }
    }

    pw.flush();
    br1.close();
    br2.close();
    pw.close();
}
}

```

**Requirement:** Write a program to merge data from all files present in a folder into a new file

**Program:**

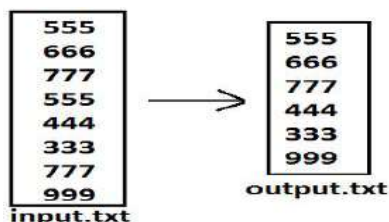
```

import java.io.*;
class TotalFileMerge {
    public static void main(String[] args)throws IOException {
        PrintWriter pw=new PrintWriter("output.txt");
        File f=new File("E:\\xyz");
        String[] s=f.list();
        for(String s1:s) {
            BufferedReader br1=new BufferedReader(new File(f,s1));
            String line=br.readLine();
            while(line!=null)
            {
                pw.println(line);
                line=br.readLine();
            }
        }
        pw.flush();
    }
}

```

**Requirement:** Write a program to delete duplicate numbers from the file.

**Diagram:**

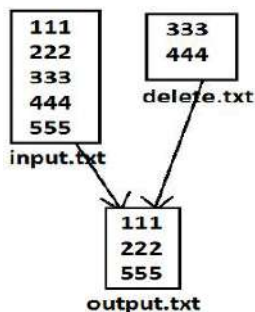


Program:

```
import java.io.*;
class FileWriterDemo1
{
    public static void main(String[] args) throws IOException
    {
        BufferedReader br1=new BufferedReader(new
        FileReader("input.txt"));
        PrintWriter out=new PrintWriter("output.txt");
        String target=br1.readLine();
        while(target!=null)
        {
            boolean available=false;
            BufferedReader br2=new BufferedReader(new
            FileReader("output.txt"));
            String line=br2.readLine();
            while(line!=null)
            {
                if(target.equals(line))
                {
                    available=true;
                    break;
                }
                line=br2.readLine();
            }
            if(available==false)
            {
                out.println(target);
                out.flush();
            }
            target=br1.readLine();
        }
    }
}
```

**Requirement:** write a program to perform file extraction operation.

Diagram:



Program:

```
import java.io.*;
class FileWriterDemo1
{
    public static void main(String[] args)throws IOException
    {
        BufferedReader br1=new BufferedReader(new
FileReader("input.txt"));
        PrintWriter pw=new PrintWriter("output.txt");
        String line=br1.readLine();
        while(line!=null)

        {
            boolean available=false;
            BufferedReader br2=new BufferedReader(new
FileReader("delete.txt"));
            String target=br2.readLine();
            while(target!=null)
            {
                if(line.equals(target))
                {
                    available=true;
                    break;
                }
                target=br2.readLine();
            }
            if(available==false)
            {
                pw.println(line);
            }
            line=br1.readLine();
        }
        pw.flush();
    }
}
```

# CORE JAVA

## With

# SCJP / OCJP

## Study Material

### Chapter 12: Serialization



**DURGA M.Tech**

(Sun certified & Realtime Expert)

**Ex. IBM Employee**

**Trained Lakhs of Students  
for last 14 years across INDIA**

**India's No.1 Software Training Institute**

# DURGASOFT

**www.durgasoft.com Ph: 9246212143 ,8096969696**

# Serialization

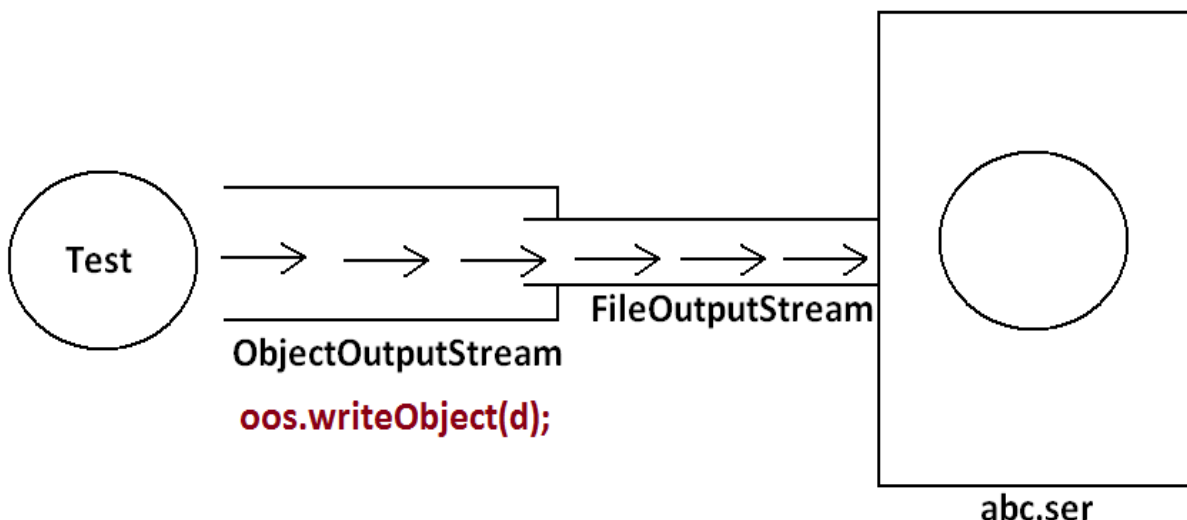
## Agenda :

1. Serialization
2. Deserialization
3. transient keyword
4. static Vs transient
5. transient Vs final
6. Object graph in serialization.
7. customized serialization.
8. Serialization with respect inheritance.
9. Externalization
10. Difference between Serialization & Externalization
11. Serializable

## Serialization: (1.1 v)

1. The process of saving (or) writing state of an object to a file is called serialization
2. but strictly speaking it is the process of converting an object from java supported form to either network supported form (or) file supported form.
3. By using `FileOutputStream` and `ObjectOutputStream` classes we can achieve serialization process.
4. Ex: big ballon

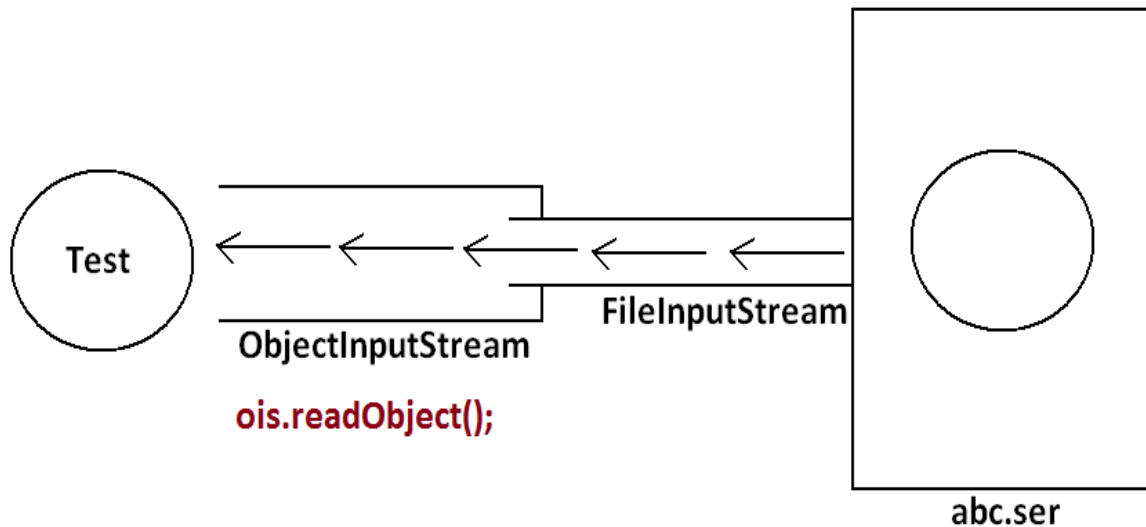
## Diagram:



## De-Serialization:

1. The process of reading state of an object from a file is called DeSerialization
2. but strictly speaking it is the process of converting an object from file supported form (or) network supported form to java supported form.
3. By using `FileInputStream` and `ObjectInputStream` classes we can achieve DeSerialization.

Diagram:

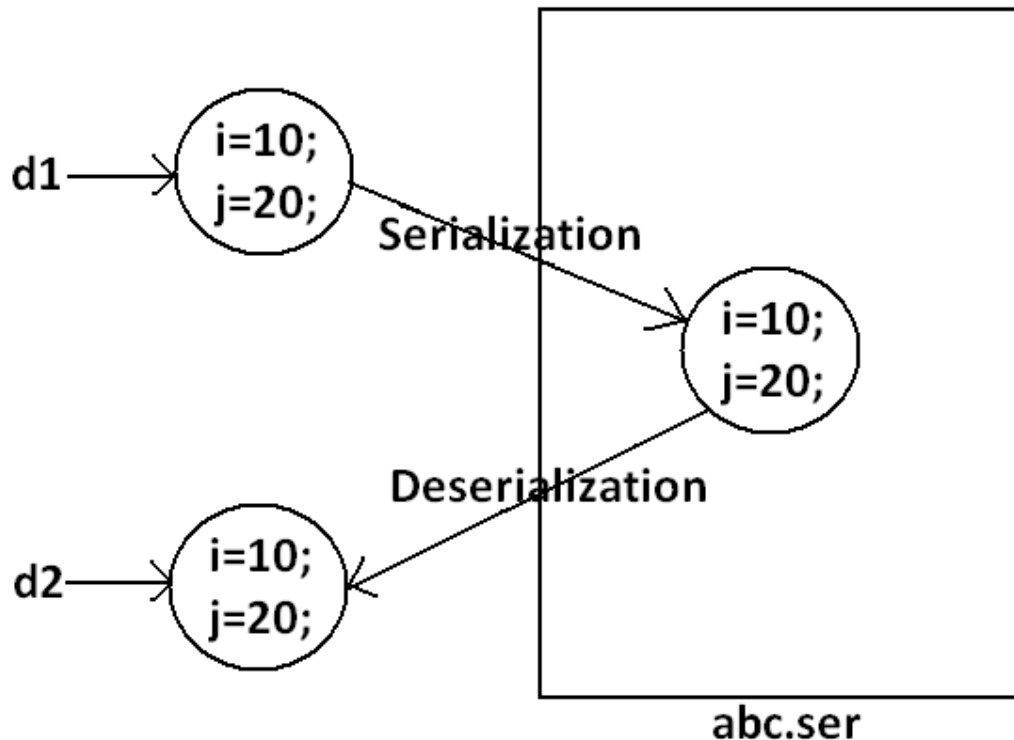


Example 1:

```
import java.io.*;
class Dog implements Serializable
{
    int i=10;
    int j=20;
}
class SerializableDemo
{
    public static void main(String args[])throws Exception{
        Dog d1=new Dog();
        System.out.println("Serialization started");
        FileOutputStream fos=new FileOutputStream("abc.ser");
        ObjectOutputStream oos=new ObjectOutputStream(fos);
        oos.writeObject(d1);
        System.out.println("Serialization ended");
        System.out.println("Deserialization started");
        FileInputStream fis=new FileInputStream("abc.ser");
        ObjectInputStream ois=new ObjectInputStream(fis);
        Dog d2=(Dog)ois.readObject();
        System.out.println("Deserialization ended");
        System.out.println(d2.i+"....."+d2.j);
    }
}
```

Output:  
 Serialization started  
 Serialization ended  
 Deserialization started  
 Deserialization ended  
 10.....20

Diagram:



**Note:**

1. We can perform Serialization only for Serializable objects.
2. An object is said to be Serializable if and only if the corresponding class implements Serializable interface.
3. Serializable interface present in java.io package and does not contain any methods. It is marker interface. The required ability will be provided automatically by JVM.
4. We can add any no. Of objects to the file and we can read all those objects from the file but in which order we wrote objects in the same order only the objects will come back. That is order is important.
5. If we are trying to serialize a non-serializable object then we will get RuntimeException saying "NotSerializableException".

Example2:  

```

import java.io.*;
class Dog implements Serializable
{
    int i=10;
    int j=20;
}

```



```

class Cat implements Serializable
{
    int i=30;
    int j=40;
}
class SerializableDemo
{
    public static void main(String args[])throws Exception{
        Dog d1=new Dog();
        Cat c1=new Cat();
        System.out.println("Serialization started");
        FileOutputStream fos=new FileOutputStream("abc.ser");
        ObjectOutputStream oos=new ObjectOutputStream(fos);
        oos.writeObject(d1);
        oos.writeObject(c1);
        System.out.println("Serialization ended");
        System.out.println("Deserialization started");
        FileInputStream fis=new FileInputStream("abc.ser");
        ObjectInputStream ois=new ObjectInputStream(fis);
        Dog d2=(Dog)ois.readObject();
        Cat c2=(Cat)ois.readObject();
        System.out.println("Deserialization ended");
        System.out.println(d2.i+"....."+d2.j);
        System.out.println(c2.i+"....."+c2.j);
    }
}
Output:
Serialization started
Serialization ended
Deserialization started
Deserialization ended
10.....20
30.....40

```

### Transient keyword:

1. transient is the modifier applicable only for variables.
2. While performing serialization if we don't want to save the value of a particular variable to meet security constant such type of variable , then we should declare that variable with "transient" keyword.
3. At the time of serialization JVM ignores the original value of transient variable and save default value to the file .
4. That is transient means "not to serialize".

### Static Vs Transient :

1. static variable is not part of object state hence they won't participate in serialization because of this declaring a static variable as transient there is no use.

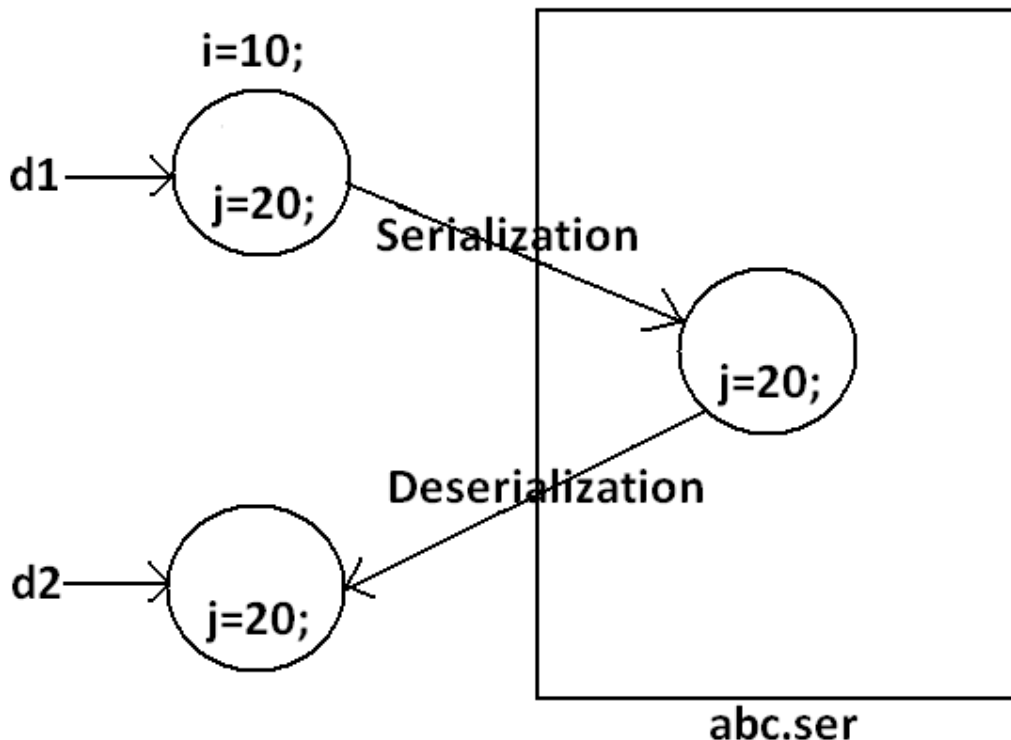
## Transient Vs Final:

1. final variables will be participated into serialization directly by their values.  
Hence declaring a final variable as transient there is no use.  
//the compiler assign the value to final variable

Example 3:

```
import java.io.*;
class Dog implements Serializable
{
    static transient int i=10;
    final transient int j=20;
}
class SerializableDemo
{
    public static void main(String args[])throws Exception{
        Dog d1=new Dog();
        FileOutputStream fos=new FileOutputStream("abc.ser");
        ObjectOutputStream oos=new ObjectOutputStream(fos);
        oos.writeObject(d1);
        FileInputStream fis=new FileInputStream("abc.ser");
        ObjectInputStream ois=new ObjectInputStream(fis);
        Dog d2=(Dog)ois.readObject();
        System.out.println(d2.i+"....."+d2.j);
    }
}
Output:
10.....20
```

Diagram:



**Table:**

| declaration                                             | output    |
|---------------------------------------------------------|-----------|
| int i=10;<br>int j=20;                                  | 10.....20 |
| transient int i=10;<br>int j=20;                        | 0.....20  |
| transient int i=10;<br>transient static int j=20;       | 0.....20  |
| transient final int i=10;<br>transient int j=20;        | 10.....0  |
| transient final int i=10;<br>transient static int j=20; | 10.....20 |

We can serialize any no of objects to the file but in which order we serialized in the same order only we have to deserialize.

Example :

```
Dog d1=new Dog( );
Cat c1=new Cat( );
Rat r1=new Rat( );
```

```
FileOutputStream fos=new FileOutputStream("abc.ser");
ObjectOutputStream oos=new ObjectOutputStream(fos);
oos.writeObject(d1);
oos.writeObject(c1);
oos.writeObject(r1);
```

```
FileInputStream fis=new FileInputStream("abc.ser");
ObjectInputStream ois=new ObjectInputStream(fis);
Dog d2=(Dog)ois.readObject();
Cat c2=(Cat)ois.readObject();
Rat r2=(Rat)ois.readObject();
```

**If we don't know order of objects :**

Example :

```
FileInputStream fis=new FileInputStream("abc.ser");
ObjectInputStream ois=new ObjectInputStream(fis);
Object o=ois.readObject( );
```

```
    if(o instanceof Dog) {
        Dog d2=(Dog)o;
        //perform Dog specific functionality
    }
    else if(o instanceof Cat) {
        Cat c2=(Cat)o;
        //perform Cat specific functionality
    }
    .
    .
    .}
```

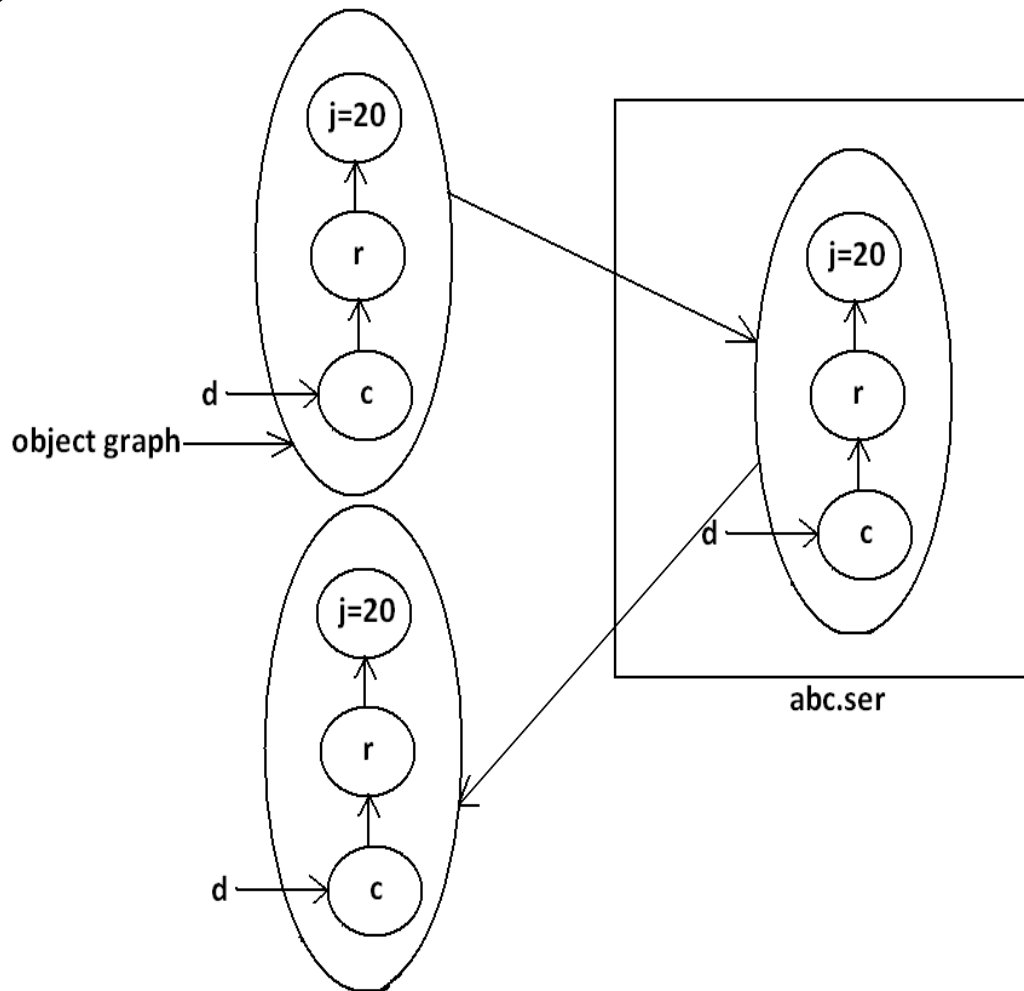
## Object graph in serialization:

1. Whenever we are serializing an object the set of all objects which are reachable from that object will be serialized automatically. This group of objects is nothing but object graph in serialization.
2. In object graph every object should be Serializable otherwise we will get runtime exception saying "*NotSerializableException*".

Example 4:

```
import java.io.*;
class Dog implements Serializable
{
    Cat c=new Cat();
}
class Cat implements Serializable
{
    Rat r=new Rat();
}
class Rat implements Serializable
{
    int j=20;
}
class SerializableDemo
{
    public static void main(String args[])throws Exception{
        Dog d1=new Dog();
        FileOutputStream fos=new FileOutputStream("abc.ser");
        ObjectOutputStream oos=new ObjectOutputStream(fos);
        oos.writeObject(d1);
        FileInputStream fis=new FileInputStream("abc.ser");
        ObjectInputStream ois=new ObjectInputStream(fis);
        Dog d2=(Dog)ois.readObject();
        System.out.println(d2.c.r.j);
    }
}
Output:
20
```

Diagram:



- In the above example whenever we are serializing Dog object automatically Cat and Rat objects will be serialized because these are part of object graph of Dog object.
- Among Dog, Cat, Rat if at least one object is not serializable then we will get runtime exception saying "NotSerializableException".

## Customized serialization:

During default Serialization there may be a chance of lose of information due to transient keyword.(Ex : mango ,money , box)

Example 5:  
import java.io.\*;

```

class Account implements Serializable
{
String userName="Bhaskar";
transient String pwd="kajal";
}
class CustomizedSerializeDemo
{
public static void main(String[] args)throws Exception{
Account a1=new Account();
System.out.println(a1.userName+"....."+a1.pwd);

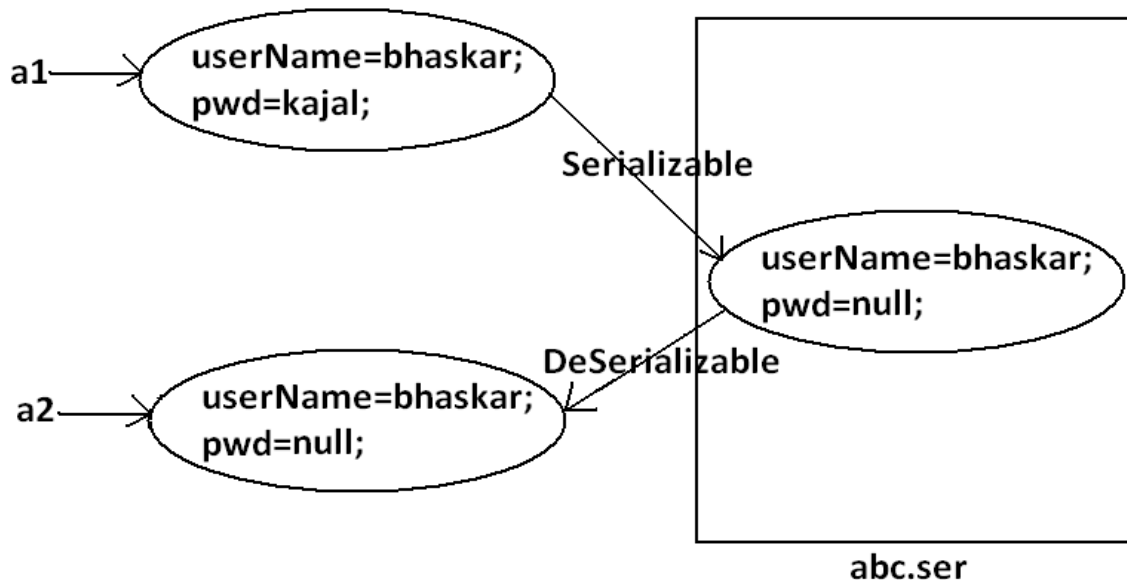
FileOutputStream fos=new FileOutputStream("abc.ser");
ObjectOutputStream oos=new ObjectOutputStream(fos);
oos.writeObject(a1);

FileInputStream fis=new FileInputStream("abc.ser");
ObjectInputStream ois=new ObjectInputStream(fis);
Account a2=(Account)ois.readObject();
System.out.println(a2.userName+"....."+a2.pwd);
}
}

```

Output:  
Bhaskar.....kajal  
Bhaskar.....null

Diagram:



- In the above example before serialization `Account` object can provide proper username and password. But after Deserialization `Account` object can provide only username but not password. This is due to declaring password as transient. Hence doing default serialization there may be a chance of loss of information due to transient keyword.

- We can recover this loss of information by using customized serialization.

We can implements customized serialization by using the following two methods.

1. private void writeObject(ObjectOutputStream os) throws Exception.

This method will be executed automatically by jvm at the time of serialization. It is a callback method. Hence at the time of serialization if we want to perform any extra work we have to define that in this method only.  
(prepare encrypted password and write encrypted password seperate to the file )

2. private void readObject(ObjectInputStream is) throws Exception.

This method will be executed automatically by JVM at the time of Deserialization. Hence at the time of Deserialization if we want to perform any extra activity we have to define that in this method only.  
(read encrypted password , perform decryption and assign decrypted password to the current object password variable )

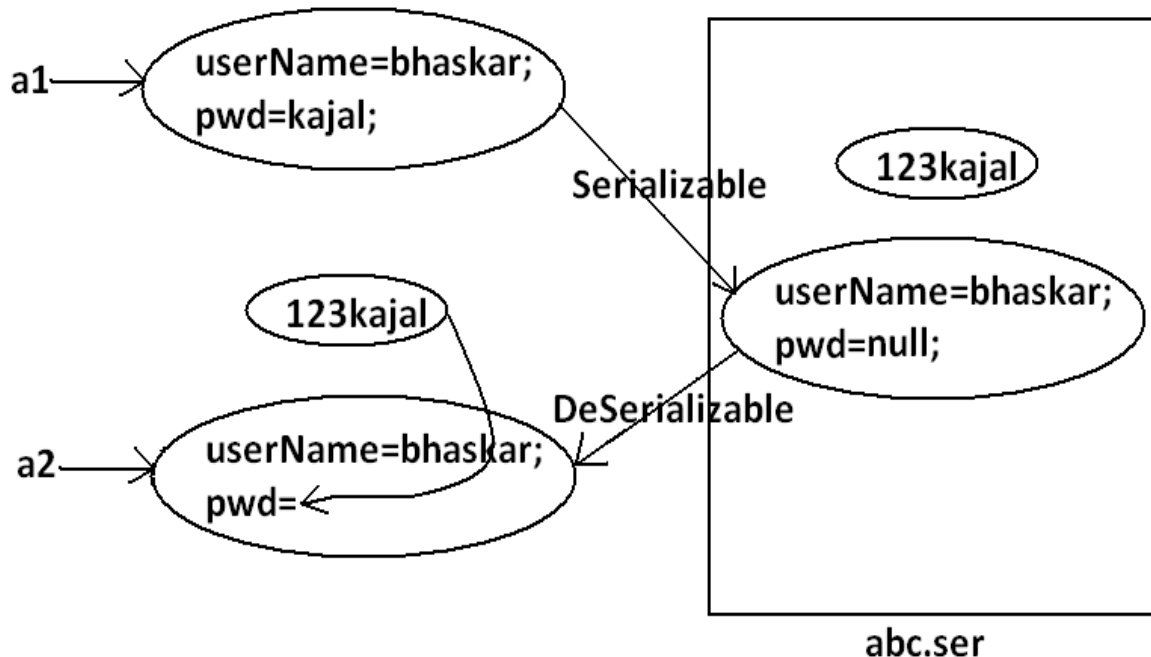
#### Example 6:

Demo program for customized serialization to recover loss of information which is happen due to transient keyword.

```
import java.io.*;
class Account implements Serializable
{
    String userName="Bhaskar";
    transient String pwd="kajal";
    private void writeObject(ObjectOutputStream os)throws Exception
    {
        os.defaultWriteObject();
        String epwd="123"+pwd;
        os.writeObject(epwd);
    }
    private void readObject(ObjectInputStream is)throws Exception{
        is.defaultReadObject();
        String epwd=(String)is.readObject();
        pwd=epwd.substring(3);
    }
}
class CustomizedSerializeDemo
{
    public static void main(String[] args)throws Exception{
        Account a1=new Account();
        System.out.println(a1.userName+"....."+a1.pwd);
        FileOutputStream fos=new FileOutputStream("abc.ser");
        ObjectOutputStream oos=new ObjectOutputStream(fos);
        oos.writeObject(a1);
        FileInputStream fis=new FileInputStream("abc.ser");
        ObjectInputStream ois=new ObjectInputStream(fis);
        Account a2=(Account)ois.readObject();
        System.out.println(a2.userName+"....."+a2.pwd);
    }
}
```

Output :  
 Bhaskar.....kajal  
 Bhaskar.....kajal

Diagram:



At the time of Account object serialization JVM will check is there any writeObject() method in Account class or not. If it is not available then JVM is responsible to perform serialization(default serialization). If Account class contains writeObject() method then JVM feels very happy and executes that Account class writeObject() method. The same rule is applicable for readObject() method also.

### Serialization with respect to inheritance :

#### Case 1:

If parent class implements Serializable then automatically every child class by default implements Serializable. That is Serializable nature is inheriting from parent to child.

Hence even though child class doesn't implements Serializable , we can serialize child class object if parent class implements serializable interface.

Example 7:  

```
import java.io.*;
class Animal implements Serializable
{
    int i=10;
}
class Dog extends Animal
```



```

{
int j=20;
}
class SerializableWRTInheritance
{
public static void main(String[] args)throws Exception{
Dog d1=new Dog();
System.out.println(d1.i+"....."+d1.j);
FileOutputStream fos=new FileOutputStream("abc.ser");
ObjectOutputStream oos=new ObjectOutputStream(fos);
oos.writeObject(d1);
FileInputStream fis=new FileInputStream("abc.ser");
ObjectInputStream ois=new ObjectInputStream(fis);
Dog d2=(Dog)ois.readObject();
System.out.println(d2.i+"....."+d2.j);
}
}
Output:
10.....20
10.....20

```

Even though Dog class does not implements Serializable interface explicitly but we can Serialize Dog object because its parent class animal already implements Serializable interface.

**Note :** *Object class doesn't implement Serializable interface.*

#### Case 2:

1. Even though parent class does not implements Serializable we can serialize child object if child class implements Serializable interface.
2. At the time of serialization JVM ignores the values of instance variables which are coming from non Serializable parent then instead of original value JVM saves default values for those variables to the file.
3. At the time of Deserialization JVM checks whether any parent class is non Serializable or not. If any parent class is non Serializable JVM creates a separate object for every non Serializable parent and shares its instance variables to the current object.
4. To create an object for non-serializable parent JVM always calls no arg constructor(default constructor) of that non Serializable parent hence every non Serializable parent should compulsory contain no arg constructor otherwise we will get runtime exception "InvalidClassException" .
5. If non-serializable parent is abstract class then just instance control flow will be performed and share it's instance variable to the current object.

Example 8:

```

import java.io.*;
class Animal
{
int i=10;
Animal(){
System.out.println("Animal constructor called");
}
}

```

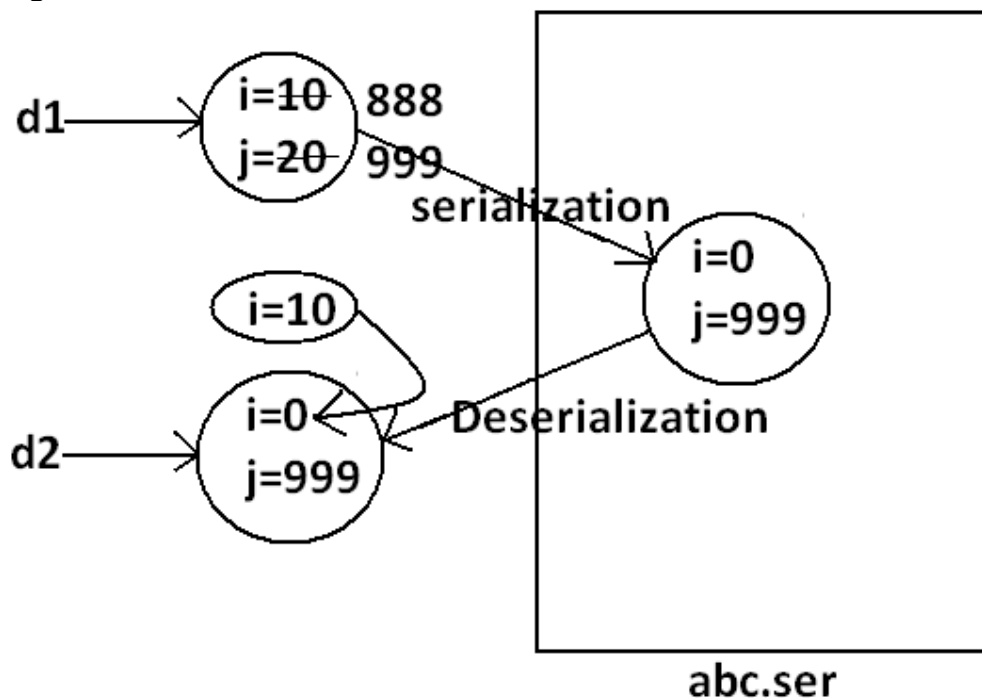
```

class Dog extends Animal implements Serializable
{
    int j=20;
    Dog(){
        System.out.println("Dog constructor called");
    }
}
class SerializableWRTInheritance
{
    public static void main(String[] args)throws Exception{
        Dog d1=new Dog();
        d1.i=888;
        d1.j=999;
        FileOutputStream fos=new FileOutputStream("abc.ser");
        ObjectOutputStream oos=new ObjectOutputStream(fos);
        oos.writeObject(d1);
        System.out.println("Deserialization started");
        FileInputStream fis=new FileInputStream("abc.ser");
        ObjectInputStream ois=new ObjectInputStream(fis);
        Dog d2=(Dog)ois.readObject();
        System.out.println(d2.i+"....."+d2.j);
    }
}

```

Output:  
 Animal constructor called  
 Dog constructor called  
 Deserialization started  
 Animal constructor called  
 10.....999

Diagram:



## Externalization : ( 1.1 v )

1. In default serialization every thing takes care by JVM and programmer doesn't have any control.
2. In serialization total object will be saved always and it is not possible to save part of the object , which creates performance problems at certain point.
3. To overcome these problems we should go for externalization where every thing takes care by programmer and JVM doesn't have any control.
4. The main advantage of externalization over serialization is we can save either total object or part of the object based on our requirement.
5. To provide Externalizable ability for any object compulsory the corresponding class should implements externalizable interface.
6. Externalizable interface is child interface of serializable interface.

### Externalizable interface defines 2 methods :

1. writeExternal()
2. readExternal()

*public void writeExternal(ObjectOutput out) throws IOException*

This method will be executed automatically at the time of Serialization with in this method , we have to write code to save required variables to the file .

*public void readExternal(ObjectInput in) throws IOException ,  
ClassNotFoundException*

This method will be executed automatically at the time of deserialization with in this method , we have to write code to save read required variable from file and assign to the current object

At the time of deserialization Jvm will create a separate new object by executing public no-arg constructor on that object JVM will call readExternal() method.

Every Externalizable class should compulsory contains public no-arg constructor otherwise we will get RuntimeException saying "InvalidClassException" .

Example :

```
import java.io.*;

class ExternalDemo implements Externalizable {
    String s ;
    int i ;
    int j ;

    public ExternalDemo() {
        System.out.println("public no-arg constructor");
    }
}
```

```

    }
    public ExternalDemo(String s , int i, int j) {
        this.s=s ;
        this.i=i ;
        this.j=j ;
    }

    public void writeExternal(ObjectOutput out) throws IOException {
        out.writeObject(s);
        out.writeInt(i);
    }

    public void readExternal(ObjectInput in) throws IOException ,
    ClassNotFoundException {
        s=(String)in.readObject();
        i= in.readInt();
    }
}

public class Externalizable1 {
    public static void main(String[] args)throws Exception {
        ExternalDemo t1=new ExternalDemo("ashok", 10, 20);
        FileOutputStream fos=new FileOutputStream("abc.ser");
        ObjectOutputStream oos=new ObjectOutputStream(fos);
        oos.writeObject(t1);

        FileInputStream fis=new FileInputStream("abc.ser");
        ObjectInputStream ois=new ObjectInputStream(fis);
        ExternalDemo t2=(ExternalDemo)ois.readObject();
        System.out.println(t2.s+"-----"+t2.i+"-----"+t2.j);
    }
}

```

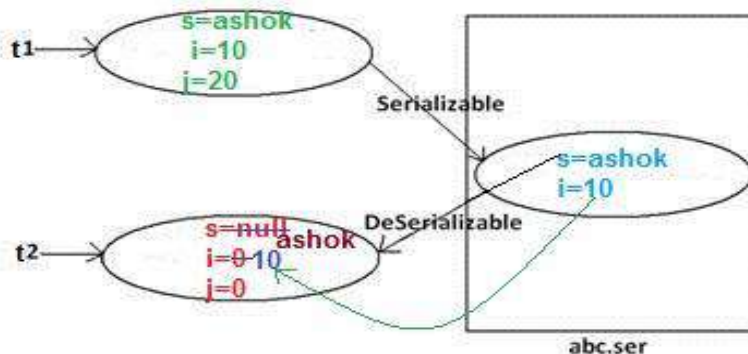
output :

```

public no-arg constructor
ashok ----- 10 ----- 0

```

Diagram :



1. If the class implements `Externalizable` interface then only part of the object will be saved in the case output is
2. public no-arg constructor
3. ashok ---- 10 ----- 0
4. If the class implements `Serializable` interface then the output is ashok --- 10 --- 20

5. In externalization transient keyword won't play any role , hence transient keyword not required.

### Difference between Serialization & Externalization :

| Serialization                                                                             | Externalization                                                                                                                                 |
|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| It is meant for default Serialization                                                     | It is meant for Customized Serialization                                                                                                        |
| Here every thing takes care by JVM and programmer doesn't have any control                | Here every thing takes care by programmer and JVM doesn't have any control.                                                                     |
| Here total object will be saved always and it is not possible to save part of the object. | Here based on our requirement we can save either total object or part of the object.                                                            |
| Serialization is the best choice if we want to save total object to the file.             | Externalization is the best choice if we want to save part of the object.                                                                       |
| relatively performance is low                                                             | relatively performance is high                                                                                                                  |
| Serializable interface doesn't contain any method , and it is marker interface.           | Externalizable interface contains 2 methods :<br>1.writeExternal()<br>2. readExternal()<br>It is not a marker interface.                        |
| Serializable class not required to contains public no-arg constructor.                    | Externalizable class should compulsory contains public no-arg constructor otherwise we will get RuntimeException saying "InvalidClassException" |
| transient keyword play role in serialization                                              | transient keyword don't play any role in Externalization                                                                                        |

### serialVersionUID :

To perform Serialization & Deserialization internally JVM will use a unique identifier , which is nothing but serialVersionUID .

At the time of serialization JVM will save serialVersionUID with object.

At the time of Deserialization JVM will compare serialVersionUID and if it is matched then only object will be Deserialized otherwise we will get RuntimeException saying "InvalidClassException".

The process in depending on default serialVersionUID are :

1. After Serializing object if we change the .class file then we can't perform deserialization because of mismatch in serialVersionUID of local class and

serialized object in this case at the time of Deserialization we will get `RuntimeException` saying in `"InvalidClassException"`.

- Both sender and receiver should use the same version of JVM if there any incompatibility in JVM versions then receive unable to deserializable because of different `serialVersionUID`, in this case receiver will get `RuntimeException` saying `"InvalidClassException"`.
- To generate `serialVersionUID` internally JVM will use complexAlgorithm which may create performance problems.

We can solve above problems by configuring our own `serialVersionUID`.

we can configure `serialVersionUID` as follows :

`private static final long serialVersionUID = 1L;`

Example :

```
class Dog implements Serializable {
    private static final long serialVersionUID=1L;
    int i=10;
    int j=20;
}
class Sender {
    public static void main(String[] args) throws Exception {
        Dog d1=new Dog();
        FileOutputStream fos=new FileOutputStream("abc.ser");
        ObjectOutputStream oos= new ObjectOutputStream(fos);
        oos.writeObject(d1);
    }
}
class Receiver {
    public static void main(String[] args)throws Exception {
        FileInputStream fis=new FileInputStream("abc.ser");
        ObjectInputStream ois=new ObjectInputStream(fis);
        Dog d2=(Dog) ois.readObject();
        System.out.println(d2.i+"-----"+d2.j);
    }
}
```

In the above program after serialization even though if we perform any change to `Dog.class` file, we can deserialize object.

We if configure our own `serialVersionUID` both sender and receiver not required to maintain the same JVM versions.

Note : some IDE's generate explicit `serialVersionUID`.

# **CORE JAVA**

## **With**

# **SCJP / OCJP**

### **Study Material**

#### **Chapter 13: Regular Expression**



**DURGA M.Tech**

**(Sun certified & Realtime Expert)**

**Ex. IBM Employee**

**Trained Lakhs of Students  
for last 14 years across INDIA**

**India's No.1 Software Training Institute**

# **DURGASOFT**

**www.durgasoft.com Ph: 9246212143 ,8096969696**

## Regular Expression

### Agenda

1. Introduction.
2. The main important application areas of Regular Expression
3. Pattern class
4. Matcher class
5. Important methods of Matcher class
6. Character classes
7. Predefined character classes
8. Quantifiers
9. Pattern class split() method
10. String class split() method
11. StringTokenizer
12. Requirements:
  - o Write a regular expression to represent all valid identifiers in java language
  - o Write a regular expression to represent all mobile numbers
  - o Write a regular expression to represent all Mail Ids
  - o Write a program to extract all valid mobile numbers from a file
  - o Write a program to extract all Mail IDS from the File
  - o Write a program to display all .txt file names present in specific(E:\scjp) folder

### Introduction

A Regular Expression is a expression which represents a group of Strings according to a particular pattern.

#### Example:

- We can write a Regular Expression to represent all valid mail ids.
- We can write a Regular Expression to represent all valid mobile numbers.

#### The main important application areas of Regular Expression are:

- To implement validation logic.
- To develop Pattern matching applications.
- To develop translators like compilers, interpreters etc.
- To develop digital circuits.
- To develop communication protocols like TCP/IP, UDP etc.



Example:

```
import java.util.regex.*;
class RegularExpressionDemo
{
    public static void main(String[] args)
    {
        int count=0;
        Pattern p=Pattern.compile("ab");
        Matcher m=p.matcher("abbbabbaba");
        while(m.find())
        {
            count++;
            System.out.println(m.start()+"-----"+m.end()+"--
            ----"+m.group());
        }
        System.out.println("The no of occurrences
        : "+count);
    }
}
```

Output:

0-----2-----ab

4-----6-----ab

7-----9-----ab

The no of occurrences: 3

### Pattern class:

- A Pattern object represents "compiled version of Regular Expression".
- We can create a Pattern object by using compile() method of Pattern class.

```
public static Pattern compile(String regex);
```

Example:

```
Pattern p=Pattern.compile("ab");
```

Note: if we refer API we will get more information about pattern class.

### Matcher:

A Matcher object can be used to match character sequences against a Regular Expression.

We can create a Matcher object by using matcher() method of Pattern class.

```
public Matcher matcher(String target);
        Matcher m=p.matcher("abbbabbaba");
```

Important methods of Matcher class:

1. boolean find();  
It attempts to find next match and returns true if it is available otherwise returns false.

2. `int start();`  
Returns the start index of the match.
3. `int end();`  
Returns the offset(equalize) after the last character matched.(or)  
Returns the "end+1" index of the matched.
4. `String group();`  
Returns the matched Pattern.

**Note:** Pattern and Matcher classes are available in `java.util.regex` package, and introduced in 1.4 version

### Character classes:

1. `[abc]`-----Either 'a' or 'b' or 'c'
2. `[^abc]` -----Except 'a' and 'b' and 'c'
3. `[a-z]` -----Any lower case alphabet symbol
4. `[A-Z]` -----Any upper case alphabet symbol
5. `[a-zA-Z]` -----Any alphabet symbol
6. `[0-9]` -----Any digit from 0 to 9
7. `[a-zA-Z0-9]` -----Any alphanumeric character
8. `[^a-zA-Z0-9]` -----Any special character

**Example:**

```
import java.util.regex.*;
class RegularExpressionDemo
{
    public static void main(String[] args)
    {
        Pattern p=Pattern.compile("x");
        Matcher m=p.matcher("a1b7@z#");
        while(m.find())
        {
            System.out.println(m.start()+"-----
"+m.group());
        }
    }
}
```

Output:

| <u>x=[abc]</u> | <u>x=[^abc]</u> | <u>x=[0-9]</u> | <u>x=[a-z]</u> |
|----------------|-----------------|----------------|----------------|
| 0-----a        | 1-----1         | 1-----1        | 0-----a        |
| 2-----b        | 3-----7         | 3-----7        | 2-----b        |
|                | 4-----@         |                | 5-----z        |
|                | 5-----z         |                |                |
|                | 6-----#         |                |                |

**Predefined character classes:**

\s-----space character  
 \d-----Any digit from 0 to 9[o-9]  
 \w-----Any word character[a-zA-Z0-9\_]  
 . -----Any character including special characters.  
  
 \S-----any character except space character  
 \D-----any character except digit  
 \W-----any character except word character(special character)

Example:

```
import java.util.regex.*;
class RegularExpressionDemo
{
    public static void main(String[] args)
    {
        Pattern p=Pattern.compile("x");
        Matcher m=p.matcher("a1b7 @z#");
        while(m.find())
        {
            System.out.println(m.start()+"-----"
+m.group());
        }
    }
}
```

Output:

| <u>x=\\s</u> | <u>x=\\d</u> | <u>x=\\w</u> | <u>x=.</u> |
|--------------|--------------|--------------|------------|
| 4-----       | 1-----1      | 0-----a      | 0-----a    |
|              | 3-----7      | 1-----1      | 1-----1    |
|              |              | 2-----b      | 2-----b    |
|              |              | 3-----7      | 3-----7    |
|              |              | 6-----z      | 4-----     |
|              |              |              | 5-----@    |
|              |              |              | 6-----z    |
|              |              |              | 7-----#    |

### Quantifiers:

Quantifiers can be used to specify no of characters to match.

a-----Exactly one 'a'

a+-----At least one 'a'

a\*-----Any no of a's including zero number

a? -----At most one 'a'

Example:

```
import java.util.regex.*;
class RegularExpressionDemo
{
    public static void main(String[] args)
    {
        Pattern p=Pattern.compile("x");
        Matcher m=p.matcher("abaabaaab");
        while(m.find())
        {
            System.out.println(m.start()+"-----
"+m.group());
        }
    }
}
```

Output:

| <u>x=a</u> | <u>x=a+</u> | <u>x=a*</u> | <u>x=a?</u> |
|------------|-------------|-------------|-------------|
| 0-----a    | 0-----a     | 0-----a     | 0-----a     |
| 2-----a    | 2-----aa    | 1-----      | 1-----      |
| 3-----a    | 5-----aaa   | 2-----aa    | 2-----a     |
| 5-----a    |             | 4-----      | 3-----a     |
| 6-----a    |             | 5-----aaa   | 4-----      |
| 7-----a    |             | 8-----      | 5-----a     |
|            |             | 9-----      | 6-----a     |
|            |             |             | 7-----a     |
|            |             |             | 8-----      |
|            |             |             | 9-----      |

### Pattern class split() method:

Pattern class contains split() method to split the given string against a regular expression.

Example 1:

```
import java.util.regex.*;
class RegularExpressionDemo
{
    public static void main(String[] args)
    {
        Pattern p=Pattern.compile("\\s");
        String[] s=p.split("ashok software solutions");
        for(String s1:s)
        {
            System.out.println(s1);//ashok
                                   //software
                                   //solutions
        }
    }
}
```

Example 2:

```
import java.util.regex.*;
class RegularExpressionDemo
{
```

```

    public static void main(String[] args)
    {
        Pattern p=Pattern.compile("\\."); //(or)[.]
        String[] s=p.split("www.dugrajobs.com");
        for(String s1:s)
        {
            System.out.println(s1);//www
                                   //dugrajobs
                                   //com
        }
    }
}

```

### String class split() method:

String class also contains split() method to split the given string against a regular expression.

Example:

```

import java.util.regex.*;
class RegularExpressionDemo
{
    public static void main(String[] args)
    {
        String s="www.saijobs.com";
        String[] s1=s.split("\\.");
        for(String s2:s1)
        {
            System.out.println(s2);//www
                                   //saijobs
                                   //com
        }
    }
}

```

**Note :** String class split() method can take regular expression as argument where as pattern class split() method can take target string as the argument.

### StringTokenizer:

- This class present in java.util package.
- It is a specially designed class to perform string tokenization.

Example 1:

```

import java.util.*;
class RegularExpressionDemo
{
    public static void main(String[] args)
    {
        StringTokenizer st=new StringTokenizer("sai
software solutions");
    }
}

```

```

        while(st.hasMoreTokens())
        {
            System.out.println(st.nextToken()); //sai
                                                    //software
                                                    //solutions
        }
    }
}

```

*The default regular expression for the StringTokenizer is space.*

Example 2:

```

import java.util.*;
class RegularExpressionDemo
{
    public static void main(String[] args)
    {

```

```

        StringTokenizer st=new
StringTokenizer("1,99,988",",");
        while(st.hasMoreTokens())
        {
            System.out.println(st.nextToken()); //1
                                                    //99
                                                    //988
        }
    }
}

```

Requirement:

*Write a regular expression to represent all valid identifiers in java language.*

Rules:

The allowed characters are:

1. a to z, A to Z, 0 to 9, -, #
2. The 1st character should be alphabet symbol only.
3. The length of the identifier should be at least 2.

Program:

```

import java.util.regex.*;
class RegularExpressionDemo
{
    public static void main(String[] args)
    {
        Pattern p=Pattern.compile("[a-zA-Z][a-zA-Z0-9-#]*"); (or)

```

```

        Pattern p=Pattern.compile("[a-zA-Z][a-zA-Z0-9-#][a-zA-Z0-9-#]*");
        Matcher m=p.matcher(args[0]);
        if(m.find()&&m.group().equals(args[0]))
        {
            System.out.println("valid identifier");
        }
        else
        {
            System.out.println("invalid identifier");
        }
    }
}

```

Output:

```

E:\scjp>javac RegularExpressionDemo.java
E:\scjp>java RegularExpressionDemo ashok
Valid identifier

```

```

E:\scjp>java RegularExpressionDemo ?ashok
Invalid identifier

```

### Requirement:

Write a regular expression to represent all mobile numbers.

1. Should contain exactly 10 digits.
2. The 1st digit should be 7 to 9.

Program:

```

import java.util.regex.*;
class RegularExpressionDemo
{
    public static void main(String[] args)
    {
        Pattern p=Pattern.compile("
                                [7-9][0-9][0-9][0-9][0-9][0-9]
                                9][0-9][0-9][0-9][0-9]");
        //Pattern p=Pattern.compile("[7-9][0-9]{9}");
        Matcher m=p.matcher(args[0]);
        if(m.find()&&m.group().equals(args[0]))
        {
            System.out.println("valid number");
        }
        else
        {
            System.out.println("invalid number");
        }
    }
}

```



**Analysis:****10 digits mobile:**

[7-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]     (or)  
 [7-9][0-9]{9}

**Output:**

```
E:\scjp>javac RegularExpressionDemo.java
E:\scjp>java RegularExpressionDemo 9989123456
Valid number
```

```
E:\scjp>java RegularExpressionDemo 6989654321
Invalid number
```

**10 digits (or) 11 digits:**  
 (0?[7-9][0-9]{9})

**Output:**

```
E:\scjp>javac RegularExpressionDemo.java
```

```
E:\scjp>java RegularExpressionDemo 9989123456
Valid number
```

```
E:\scjp>java RegularExpressionDemo 09989123456
Valid number
```

```
E:\scjp>java RegularExpressionDemo 919989123456
Invalid number
```

**10 digits (or) 11 digit (or) 12 digits:**  
 (0|91)?[7-9][0-9]{9}     (or)  
 (91)?(0?[7-9][0-9]{9})

```
E:\scjp>javac RegularExpressionDemo.java
E:\scjp>java RegularExpressionDemo 9989123456
Valid number
E:\scjp>java RegularExpressionDemo 09989123456
Valid number
E:\scjp>java RegularExpressionDemo 919989123456
Valid number
E:\scjp>java RegularExpressionDemo 69989123456
Invalid number
```

**Requirement:**

**Write a regular expression to represent all Mail Ids.**

**Program:**

```
import java.util.regex.*;
class RegularExpressionDemo
{
    public static void main(String[] args)
    {
        Pattern p=Pattern.compile("

```

```

                                [a-zA-Z][a-zA-Z0-9- . ]*@[a-zA-Z0-9
9]^([.][a-zA-Z]^)+");
        Matcher m=p.matcher(args[0]);
        if(m.find()&&m.group().equals(args[0]))
        {
            System.out.println("valid mail id");
        }
        else
        {
            System.out.println("invalid mail id");
        }
    }
}

```

Output:

```

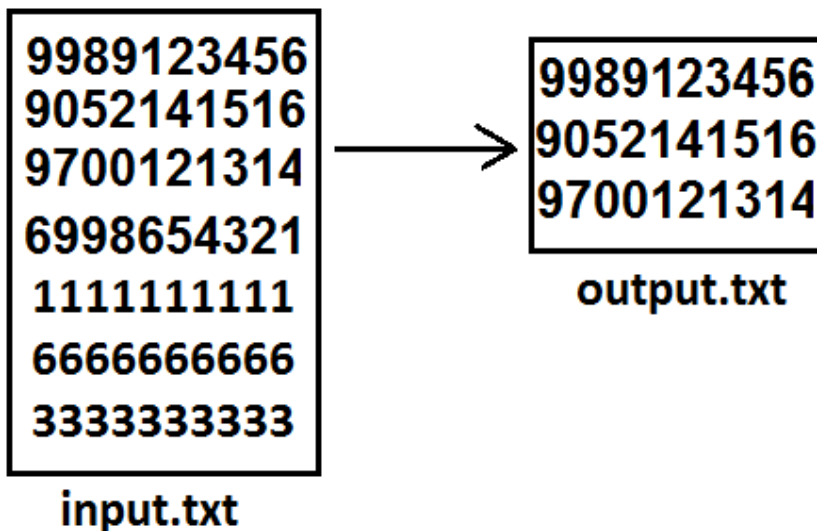
E:\scjp>javac RegularExpressionDemo.java
E:\scjp>java RegularExpressionDemo sunmicrosystem@gmail.com
Valid mail id
E:\scjp>java RegularExpressionDemo 999sunmicrosystem@gmail.com
Invalid mail id
E:\scjp>java RegularExpressionDemo 999sunmicrosystem@gmail.co9
Invalid mail id

```

#### Requirement:

Write a program to extract all valid mobile numbers from a file.

#### Diagram:



Program:

```

import java.util.regex.*;
import java.io.*;
class RegularExpressionDemo
{
    public static void main(String[] args)throws IOException

```

```

        {
            PrintWriter out=new PrintWriter("output.txt");
            BufferedReader br=new BufferedReader(new
FileReader("input.txt"));
            Pattern p=Pattern.compile("(0|91)?[7-9][0-
9]{9}");
            String line=br.readLine();
            while(line!=null)
            {
                Matcher m=p.matcher(line);
                while(m.find())
                {
                    out.println(m.group());
                }
                line=br.readLine();
            }
            out.flush();
        }
    }
}

```

**Requirement:**

Write a program to extract all Mail IDS from the File.

**Note:** In the above program replace mobile number regular expression with MAIL ID regular expression.

**Requirement:**

Write a program to display all .txt file names present in E:\scjp folder.

Program:

```

import java.util.regex.*;
import java.io.*;
class RegularExpressionDemo
{
    public static void main(String[] args)throws IOException
    {
        int count=0;
        Pattern p=Pattern.compile("[a-zA-Z0-9-
$.]+[.]txt");
        File f=new File("E:\\scjp");
        String[] s=f.list();
        for(String s1:s)
        {
            Matcher m=p.matcher(s1);
            if(m.find()&&m.group().equals(s1))
            {
                count++;
                System.out.println(s1);
            }
        }
    }
}

```

```

        System.out.println(count);
    }
}
Output:
input.txt
output.txt
outut.txt
3

```

Write a program to check whether the given mailid is valid or not.

In the above program we have to replace mobile number regular expression with mailid regular expression

Write a regular expressions to represent valid Gmail mail id's :

`[a-zA-Z0-9][a-zA-Z0-9-]*@gmail[.]com`

Write a regular expressions to represent all Java language identifiers :

Rules :

- The length of the identifier should be atleast two.
- The allowed characters are
  - a - z
  - A - Z
  - 0 - 9
  - #
  - \$
  - 
  -
- The first character should be lower case alphabet symbol k-z , and second character should be a digit divisible by 3

`[k-z][0369][a-zA-Z0-9#$]*`

Write a regular expressions to represent all names starts with 'a'

`[aA][a-zA-Z]*`

To represent all names starts with 'A' ends with 'K'

`[aA][a-zA-Z]*[kK]`

# **CORE JAVA**

## **With**

# **SCJP / OCJP**

### **Study Material**

### **Chapter 14: Collections**



**DURGA M.Tech**

**(Sun certified & Realtime Expert)**

**Ex. IBM Employee**

**Trained Lakhs of Students  
for last 14 years across INDIA**

**India's No.1 Software Training Institute**

# **DURGASOFT**

**www.durgasoft.com Ph: 9246212143 ,8096969696**

# Collections

## Agenda

1. Introduction
2. Limitations of Object[] array
3. Differences between Arrays and Collections ?
4. 9(Nine) key interfaces of collection framework
  - i. Collection
  - ii. List
  - iii. Set
  - iv. SortedSet
  - v. NavigableSet
  - vi. Queue
  - vii. Map
  - viii. SortedMap
  - ix. NavigableMap
5. What is the difference between Collection and Collections ?
6. In collection framework the following are legacy characters
7. Collection interface
8. List interface
9. ArrayList
  - o Differences between ArrayList and Vector ?
  - o Getting synchronized version of ArrayList object
- LinkedList
- Vector
- Stack
- The 3 cursors of java
  0. Enumeration
  1. Iterator
  2. ListIterator
  - o Compression of Enumeration , Iterator and ListIterator ?
- Set interface
- HashSet
- LinkedHashSet
- Diff b/w HashSet & LinkedHashSet
- SortedSet
- TreeSet
  - o Null acceptance
- Comparable interface
- compareTo() method analysis
- Comparator interface
- Compression of Comparable and Comparator ?

Compression of Set implemented class objects

Map

Entry interface

HashMap

- Differences between HashMap and Hashtable ?
- How to get synchronized version of HashMap

LinkedHashMap

IdentityHashMap

WeakHashMap

SortedMap

TreeMap

Hashtable

Properties

1.5v enhancements

- Queue interface
- PriorityQueue

1.6v Enhancements

- NavigableSet
- NavigableMap

Utility classes :

- Collections class
  - Sorting the elements of a List
  - Searching the elements of a List
  - Conclusions
- Arrays class
  - Sorting the elements of array
  - Searching the elements of array
  - Converting array to List

### Introduction:

1. An array is an indexed collection of fixed no of homogeneous data elements. (or)
2. An array represents a group of elements of same data type.
3. The main advantage of array is we can represent huge no of elements by using single variable. So that readability of the code will be improved.

### Limitations of Object[] array:

1. Arrays are fixed in size that is once we created an array there is no chance of increasing (or) decreasing the size based on our requirement hence to use arrays concept compulsory we should know the size in advance which may not possible always.
2. Arrays can hold only homogeneous data elements.

### Example:

```
Student[] s=new Student[10000];  
s[0]=new Student();//valid
```

```
s[1]=new Customer();//invalid(compile time error)
```

**Compile time error:**

Test.java:7: cannot find symbol

Symbol: class Customer

Location: class Test

```
s[1]=new Customer();
```

3) But we can resolve this problem by using object type array(Object[]).

**Example:**

```
Object[] o=new Object[10000];
```

```
o[0]=new Student();
```

```
o[1]=new Customer();
```

4) Arrays concept is not implemented based on some data structure hence ready-made methods support we can't expect. For every requirement we have to write the code explicitly.

To overcome the above limitations we should go for collections concept.

1. Collections are growable in nature that is based on our requirement we can increase (or) decrease the size hence memory point of view collections concept is recommended to use.
2. Collections can hold both homogeneous and heterogeneous objects.
3. Every collection class is implemented based on some standard data structure hence for every requirement ready-made method support is available being a programmer we can use these methods directly without writing the functionality on our own.

### **Differences between Arrays and Collections ?**

| Arrays                                                                    | Collections                                                                              |
|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| 1) Arrays are fixed in size.                                              | 1) Collections are growable in nature.                                                   |
| 2) Memory point of view arrays are not recommended to use.                | 2) Memory point of view collections are highly recommended to use.                       |
| 3) Performance point of view arrays are recommended to use.               | 3) Performance point of view collections are not recommended to use.                     |
| 4) Arrays can hold only homogeneous data type elements.                   | 4) Collections can hold both homogeneous and heterogeneous elements.                     |
| 5) There is no underlying data structure for arrays and hence there is no | 5) Every collection class is implemented based on some standard data structure and hence |



|                                                      |                                                          |
|------------------------------------------------------|----------------------------------------------------------|
| readymade method support.                            | readymade method support is available.                   |
| 6) Arrays can hold both primitives and object types. | 6) Collections can hold only objects but not primitives. |

**Collection:**

If we want to represent a group of objects as single entity then we should go for collections.

**Collection framework:**

It defines several classes and interfaces to represent a group of objects as a single entity.

|                      |                                |
|----------------------|--------------------------------|
| Java                 | C++                            |
| Collection           | Containers                     |
| Collection framework | STL(Standard Template Library) |

**9(Nine) key interfaces of collection framework:**

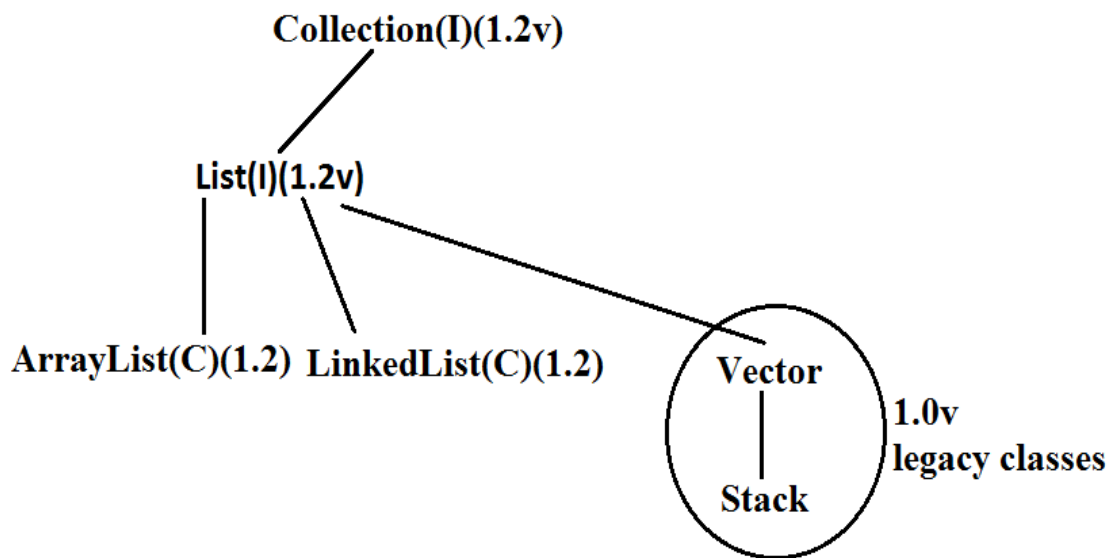
1. Collection
2. List
3. Set
4. SortedSet
5. NavigableSet
6. Queue
7. Map
8. SortedMap
9. NavigableMap

**Collection:**

1. If we want to represent a group of "individual objects" as a single entity then we should go for collection.
2. In general we can consider collection as root interface of entire collection framework.
3. Collection interface defines the most common methods which can be applicable for any collection object.
4. There is no concrete class which implements Collection interface directly.

**List:**

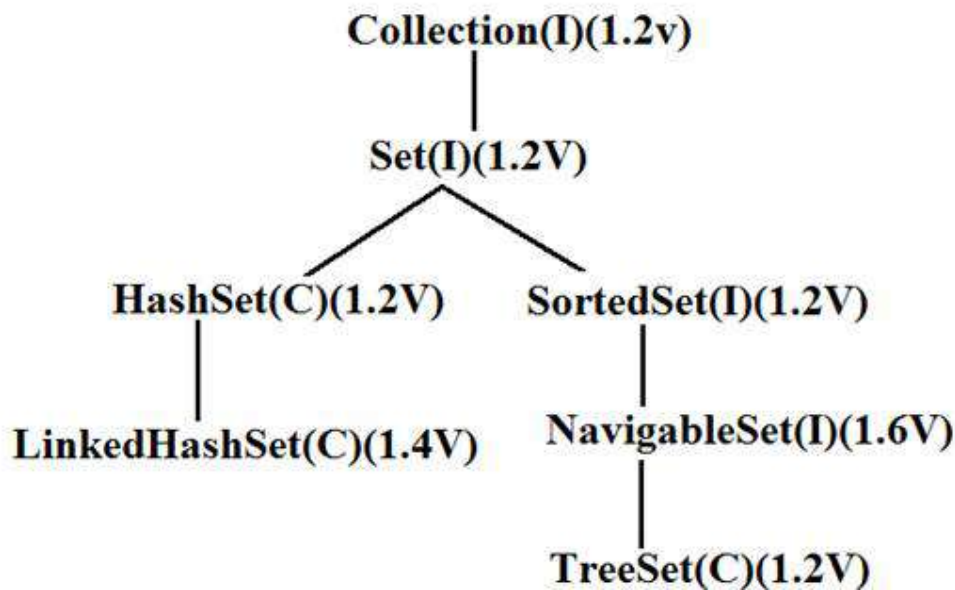
1. It is the child interface of Collection.
2. If we want to represent a group of individual objects as a single entity where "duplicates are allow and insertion order must be preserved" then we should go for List interface.

**Diagram:**

Vector and Stack classes are re-engineered in 1.2 versions to implement List interface.

**Set:**

1. It is the child interface of Collection.
2. If we want to represent a group of individual objects as single entity "where duplicates are not allow and insertion order is not preserved" then we should go for Set interface.

Diagram:**SortedSet:**

1. It is the child interface of Set.
2. If we want to represent a group of individual objects as single entity "where duplicates are not allow but all objects will be insertion according to some sorting order then we should go for SortedSet.  
(or)
3. If we want to represent a group of "unique objects" according to some sorting order then we should go for SortedSet.

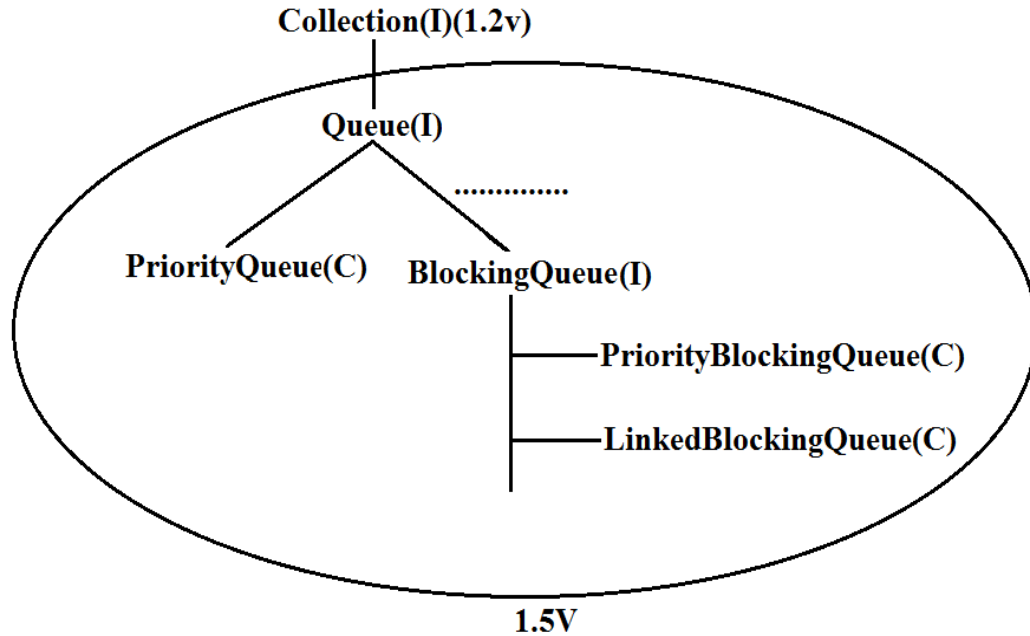
**NavigableSet:**

1. It is the child interface of SortedSet.
2. It provides several methods for navigation purposes.

**Queue:**

1. It is the child interface of Collection.
2. If we want to represent a group of individual objects prior to processing then we should go for queue concept.

Diagram:

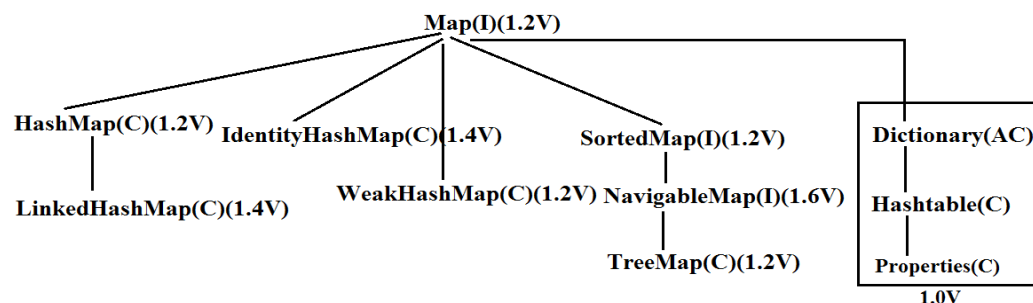


**Note:** All the above interfaces (Collection, List, Set, SortedSet, NavigableSet, and Queue) meant for representing a group of individual objects.  
If we want to represent a group of objects as key-value pairs then we should go for Map.

### Map:

1. Map is not child interface of Collection.
2. If we want to represent a group of objects as key-value pairs then we should go for Map interface.
3. Duplicate keys are not allowed but values can be duplicated.

### Diagram:



**SortedMap:**

1. It is the child interface of Map.
2. If we want to represent a group of objects as key value pairs "according to some sorting order of keys" then we should go for SortedMap.

**NavigableMap:**

1) It is the child interface of SortedMap and defines several methods for navigation purposes.

**What is the difference between Collection and Collections ?**

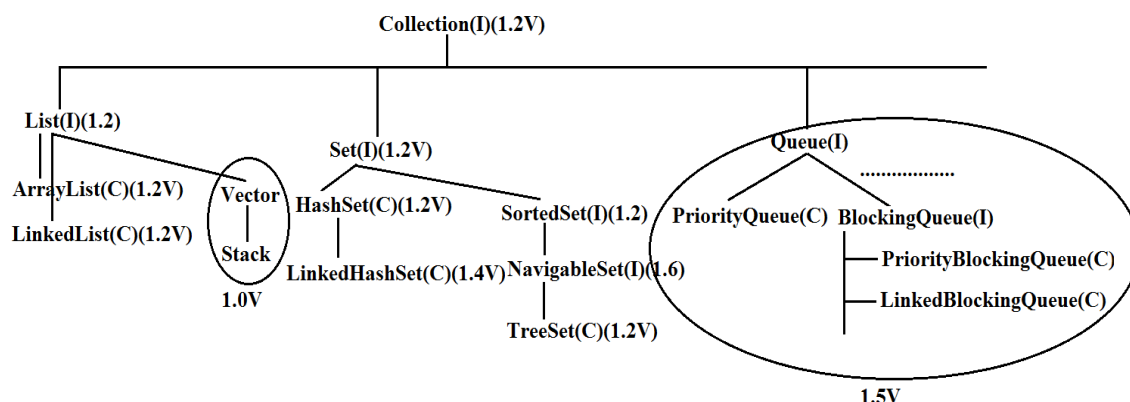
"Collection is an "interface" which can be used to represent a group of objects as a single entity. Whereas "Collections is an utility class" present in java.util package to define several utility methods for Collection objects.

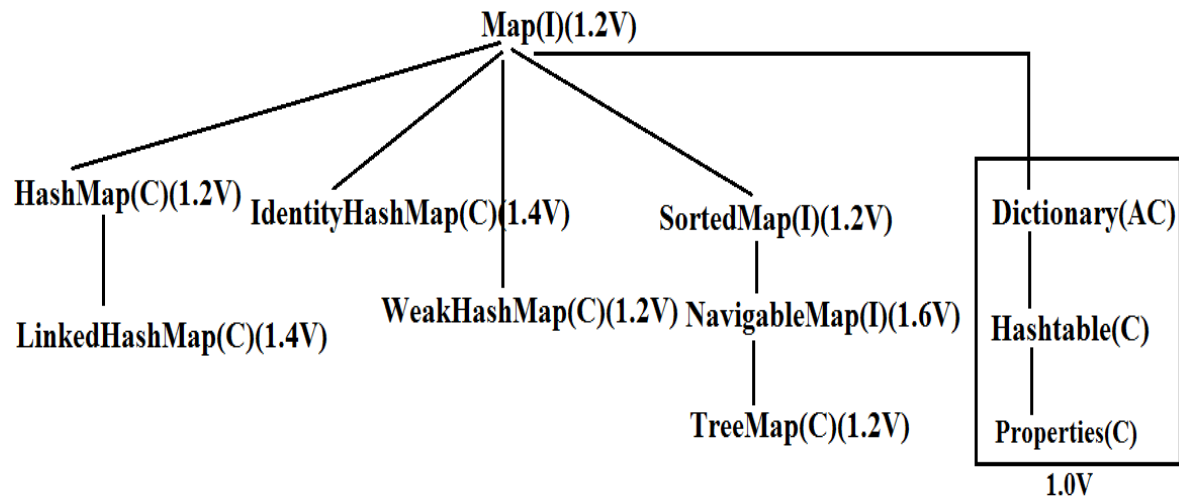
Collection-----interface

Collections-----class

*In collection framework the following are legacy characters.*

1. Enumeration(I)
2. Dictionary(AC)
3. Vector(C)
4. Stack(C)
5. Hashtable(C)
6. Properties(C)

**Diagram:****Diagram:**



### Collection interface:

- If we want to represent a group of individual objects as a single entity then we should go for Collection interface. This interface defines the most common general methods which can be applicable for any Collection object.
- The following is the list of methods present in Collection interface.
  1. boolean add(Object o);
  2. boolean addAll(Collection c);
  3. boolean remove(Object o);
  4. boolean removeAll(Collection c);
  5. boolean retainAll(Collection c);  
To remove all objects except those present in c.
  6. void clear();
  7. boolean contains(Object o);
  8. boolean containsAll(Collection c);
  9. boolean isEmpty();
  10. int size();
  11. Object[] toArray();
  12. Iterator iterator();

There is no concrete class which implements Collection interface directly.

### List interface:

- It is the child interface of Collection.
- If we want to represent a group of individual objects as a single entity where duplicates are allowed and insertion order is preserved. Then we should go for List.
- We can differentiate duplicate objects and we can maintain insertion order by means of index hence "index plays a very important role in List".

List interface defines the following specific methods.

1. `boolean add(int index, Object o);`
2. `boolean addAll(int index, Collection c);`
3. `Object get(int index);`
4. `Object remove(int index);`
5. `Object set(int index, Object new);` //to replace
6. `int indexOf(Object o);`  
Returns index of first occurrence of "o".
7. `int lastIndexOf(Object o);`
8. `ListIterator listIterator();`

### ArrayList:

1. The underlying data structure is resizable array (or) growable array.
2. Duplicate objects are allowed.
3. Insertion order preserved.
4. Heterogeneous objects are allowed. (except TreeSet, TreeMap everywhere heterogeneous objects are allowed)
5. Null insertion is possible.

#### Constructors:

1) `ArrayList a = new ArrayList();`

Creates an empty ArrayList object with default initial capacity "10" if ArrayList reaches its max capacity then a new ArrayList object will be created with

$$\text{New capacity} = (\text{current capacity} * 3/2) + 1$$

2) `ArrayList a = new ArrayList(int initialCapacity);`

Creates an empty ArrayList object with the specified initial capacity.

3) `ArrayList a = new ArrayList(Collection c);`

Creates an equivalent ArrayList object for the given Collection that is this constructor meant for inter conversation between collection objects. That is to dance between collection objects.

#### Demo program for ArrayList:

```
import java.util.*;
class ArrayListDemo
{
    public static void main(String[] args)
    {
        ArrayList a = new ArrayList();
```

```

        a.add("A");
        a.add(10);
        a.add("A");
        a.add(null);
        System.out.println(a);//[A, 10, A, null]
        a.remove(2);
        System.out.println(a);//[A, 10, null]
        a.add(2, "m");
        a.add("n");
        System.out.println(a);//[A, 10, m, null, n]
    }
}

```

- Usually we can use collection to hold and transfer objects from one tier to another tier. To provide support for this requirement every Collection class already implements Serializable and Cloneable interfaces.
- ArrayList and Vector classes implements RandomAccess interface so that any random element we can access with the same speed. Hence ArrayList is the best choice of "retrival operation".
- RandomAccess interface present in util package and doesn't contain any methods. It is a marker interface.

**Example :**

```

ArrayList a1=new ArrayList();
LinkedList a2=new LinkedList();

```

```

System.out.println(a1 instanceof Serializable ); //true
System.out.println(a2 instanceof Clonable); //true

```

```

System.out.println(a1 instanceof RandomAccess); //true
System.out.println(a2 instanceof RandomAccess); //false

```

**Differences between ArrayList and Vector ?**

| ArrayList                                                                                                             | Vector                                                                                                    |
|-----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| 1) No method is synchronized                                                                                          | 1) Every method is synchronized                                                                           |
| 2) At a time multiple Threads are allow to operate on ArrayList object and hence ArrayList object is not Thread safe. | 2) At a time only one Thread is allow to operate on Vector object and hence Vector object is Thread safe. |
| 3) Relatively performance is high because Threads are not required to wait.                                           | 3) Relatively performance is low because Threads are required to wait.                                    |



4) It is non legacy and introduced in 1.2v

4) It is legacy and introduced in 1.0v

### Getting synchronized version of ArrayList object:

- Collections class defines the following method to return synchronized version of List.  
Public static List synchronizedList(list l);

- Example:

```
ArrayList a=new arrayList();
```

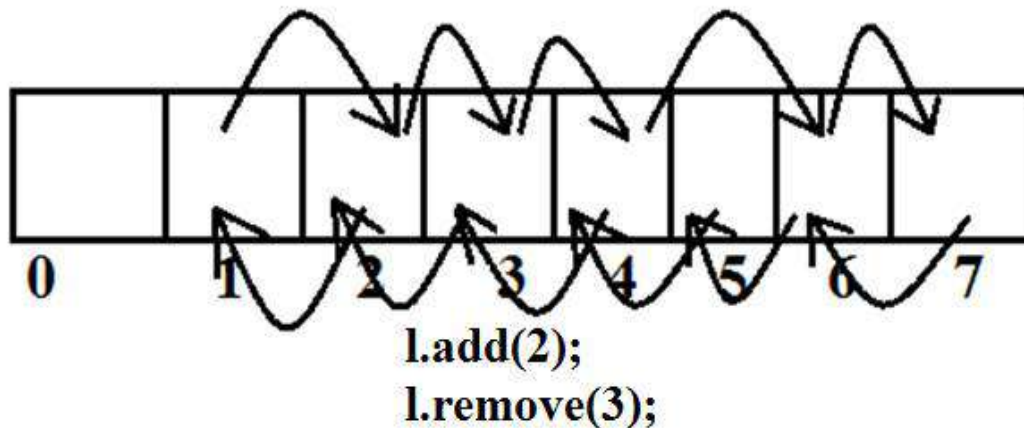
```
List l1=collections.synchronizedList(a);
```

**synchronized  
version**

**nonsynchronized  
version**

- Similarly we can get synchronized version of Set and Map objects by using the following methods.  
1) public static Set synchronizedSet(Set s);  
2) public static Map synchronizedMap(Map m);
- ArrayList is the best choice if our frequent operation is retrieval.
- ArrayList is the worst choice if our frequent operation is insertion (or) deletion in the middle because it requires several internal shift operations.

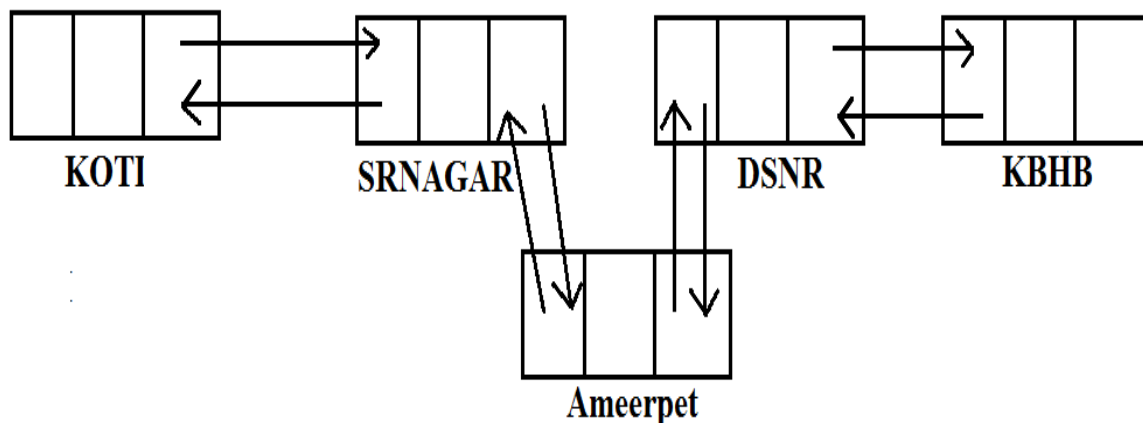
Diagram:



### **LinkedList:**

1. The underlying data structure is double LinkedList
2. If our frequent operation is insertion (or) deletion in the middle then LinkedList is the best choice.
3. If our frequent operation is retrieval operation then LinkedList is worst choice.
4. Duplicate objects are allowed.
5. Insertion order is preserved.
6. Heterogeneous objects are allowed.
7. Null insertion is possible.
8. Implements Serializable and Cloneable interfaces but not RandomAccess.

Diagram:



Usually we can use LinkedList to implement Stacks and Queues.

To provide support for this requirement LinkedList class defines the following 6 specific methods.

1. void addFirst(Object o);
2. void addLast(Object o);
3. Object getFirst();
4. Object getLast();
5. Object removeFirst();
6. Object removeLast();

We can apply these methods only on LinkedList object.

Constructors:

1. LinkedList l=new LinkedList();  
Creates an empty LinkedList object.

2. `LinkedList l=new LinkedList(Collection c);`  
To create an equivalent `LinkedList` object for the given collection.

**Example:**

```
import java.util.*;
class LinkedListDemo
{
    public static void main(String[] args)
    {
        LinkedList l=new LinkedList();
        l.add("ashok");
        l.add(30);
        l.add(null);
        l.add("ashok");
        System.out.println(l);//[ashok, 30, null, ashok]
        l.set(0,"software");
        System.out.println(l);//[software, 30, null,
ashok]
        l.set(0,"venky");
        System.out.println(l);//[venky, 30, null, ashok]
        l.removeLast();
        System.out.println(l);//[venky, 30, null]
        l.addFirst("vvv");
        System.out.println(l);//[vvv, venky, 30, null]
    }
}
```

**Vector:**

1. The underlying data structure is resizable array (or) growable array.
2. Duplicate objects are allowed.
3. Insertion order is preserved.
4. Heterogeneous objects are allowed.
5. Null insertion is possible.
6. Implements `Serializable`, `Cloneable` and `RandomAccess` interfaces.

Every method present in `Vector` is synchronized and hence `Vector` is Thread safe.

**Vector specific methods:****To add objects:**

1. `add(Object o);`-----Collection
2. `add(int index,Object o);`-----List
3. `addElement(Object o);`-----Vector

**To remove elements:**

1. remove(Object o);-----Collection
2. remove(int index);-----List
3. removeElement(Object o);----Vector
4. removeElementAt(int index);----Vector
5. removeAllElements();----Vector
6. clear();-----Collection

#### To get objects:

1. Object get(int index);-----List
2. Object elementAt(int index);----Vector
3. Object firstElement();-----Vector
4. Object lastElement();-----Vector

#### Other methods:

1. Int size();//How many objects are added
2. Int capacity();//Total capacity
3. Enumeration elements();

#### Constructors:

1. Vector v=new Vector();
  - o Creates an empty Vector object with default initial capacity 10.
  - o Once Vector reaches its maximum capacity then a new Vector object will be created with double capacity. That is "newcapacity=currentcapacity\*2".
2. Vector v=new Vector(int initialcapacity);
3. Vector v=new Vector(int initialcapacity, int incrementalcapacity);
4. Vector v=new Vector(Collection c);

#### Example:

```
import java.util.*;
class VectorDemo
{
    public static void main(String[] args)
    {
        Vector v=new Vector();
        System.out.println(v.capacity());//10
        for(int i=1;i<=10;i++)
        {
            v.addElement(i);
        }
        System.out.println(v.capacity());//10
        v.addElement("A");
        System.out.println(v.capacity());//20
        System.out.println(v);//[1, 2, 3, 4, 5, 6, 7, 8, 9, 10,
A]
    }
}
```

}

**Stack:**

1. It is the child class of Vector.
2. Whenever last in first out(LIFO) order required then we should go for Stack.

**Constructor:**

It contains only one constructor.

Stack s= new Stack();

**Methods:**

1. Object push(Object o);  
To insert an object into the stack.
2. Object pop();  
To remove and return top of the stack.
3. Object peek();  
To return top of the stack without removal.
4. boolean empty();  
Returns true if Stack is empty.
5. Int search(Object o);  
Returns offset if the element is available otherwise returns "-1"

**Example:**

```
import java.util.*;
class StackDemo
{
    public static void main(String[] args)
    {
        Stack s=new Stack();
        s.push("A");
        s.push("B");
        s.push("C");
        System.out.println(s);//[A, B, C]
        System.out.println(s.pop());//C
        System.out.println(s);//[A, B]
        System.out.println(s.peek());//B
        System.out.println(s.search("A"));//2
        System.out.println(s.search("Z"));//-1
        System.out.println(s.empty());//false
    }
}
```

### The 3 cursors of java:

If we want to get objects one by one from the collection then we should go for cursor.  
There are 3 types of cursors available in java. They are:

1. Enumeration
2. Iterator
3. ListIterator

### Enumeration:

1. We can use Enumeration to get objects one by one from the legacy collection objects.
2. We can create Enumeration object by using elements() method.  

```
public Enumeration elements();
Enumeration e=v.elements();
using Vector Object
```

Enumeration interface defines the following two methods

1. `public boolean hasMoreElements();`
2. `public Object nextElement();`

#### Example:

```
import java.util.*;
class EnumerationDemo
{
    public static void main(String[] args)
    {
        Vector v=new Vector();
        for(int i=0;i<=10;i++)
        {
            v.addElement(i);
        }
        System.out.println(v);//[0, 1, 2, 3, 4, 5, 6, 7,
8, 9, 10]
        Enumeration e=v.elements();
        while(e.hasMoreElements())
        {
            Integer i=(Integer)e.nextElement();
            if(i%2==0)
                System.out.println(i);//0 2 4 6 8 10
        }
        System.out.print(v);//[0, 1, 2, 3, 4, 5, 6, 7, 8,
9, 10]
    }
}
```

**Limitations of Enumeration:**

1. We can apply Enumeration concept only for legacy classes and it is not a universal cursor.
2. By using Enumeration we can get only read access and we can't perform remove operations.
3. To overcome these limitations sun people introduced Iterator concept in 1.2v.

**Iterator:**

1. We can use Iterator to get objects one by one from any collection object.
2. We can apply Iterator concept for any collection object and it is a universal cursor.
3. While iterating the objects by Iterator we can perform both read and remove operations.

We can get Iterator object by using iterator() method of Collection interface.

```
public Iterator iterator();
```

```
Iterator itr=c.iterator();
```

**Iterator interface defines the following 3 methods.**

1. public boolean hasNext();
2. public object next();
3. public void remove();

**Example:**

```
import java.util.*;
class IteratorDemo
{
    public static void main(String[] args)
    {
        ArrayList a=new ArrayList();
        for(int i=0;i<=10;i++)
        {
            a.add(i);
        }
        System.out.println(a);//[0, 1, 2, 3, 4, 5, 6, 7,
8, 9, 10]
        Iterator itr=a.iterator();
        while(itr.hasNext())
        {
            Integer i=(Integer)itr.next();
            if(i%2==0)
                System.out.println(i);//0, 2, 4, 6,
8, 10
            else
                itr.remove();
        }
    }
}
```

```

        }
        System.out.println(a);//[0, 2, 4, 6, 8, 10]
    }
}

```

### Limitations of Iterator:

1. Both enumeration and Iterator are single direction cursors only. That is we can always move only forward direction and we can't move to the backward direction.
2. While iterating by Iterator we can perform only read and remove operations and we can't perform replacement and addition of new objects.
3. To overcome these limitations sun people introduced listIterator concept.

### **ListIterator:**

1. ListIterator is the child interface of Iterator.
2. By using listIterator we can move either to the forward direction (or) to the backward direction that is it is a bi-directional cursor.
3. While iterating by listIterator we can perform replacement and addition of new objects in addition to read and remove operations

By using listIterator method we can create listIterator object.

```

public ListIterator listIterator();
ListIterator itr=l.listIterator();
(I is any List object)

```

### ListIterator interface defines the following 9 methods.

1. public boolean hasNext();
2. public Object next(); forward
3. public int nextIndex();
4. public boolean hasPrevious();
5. public Object previous(); backward
6. public int previousIndex();
7. public void remove();
8. public void set(Object new);
9. public void add(Object new);

### Example:

```

import java.util.*;
class ListIteratorDemo
{
    public static void main(String[] args)
    {

```



```

        LinkedList l=new LinkedList();
        l.add("balakrishna");
        l.add("venki");
        l.add("chiru");
        l.add("nag");
        System.out.println(l);//[balakrishna, venki,
chiru, nag]
        ListIterator itr=l.listIterator();
        while(itr.hasNext())
        {
            String s=(String)itr.next();
            if(s.equals("venki"))
            {
                itr.remove();
            }
        }
        System.out.println(l);//[balakrishna, chiru, nag]
    }
}

```

**Case 1:**

```

if(s.equals("chiru"))
{
    itr.set("chran");
}

```

Output:

```

[balakrishna, venki, chiru, nag]
[balakrishna, venki, chran, nag]

```

**Case 2:**

```

if(s.equals("nag"))
{
    itr.add("chitu");
}

```

Output:

```

[balakrishna, venki, chiru, nag]
[balakrishna, venki, chiru, nag, chitu]

```

The most powerful cursor is listIterator but its limitation is it is applicable only for "List objects".

**Comparison of Enumeration Iterator and ListIterator ?**

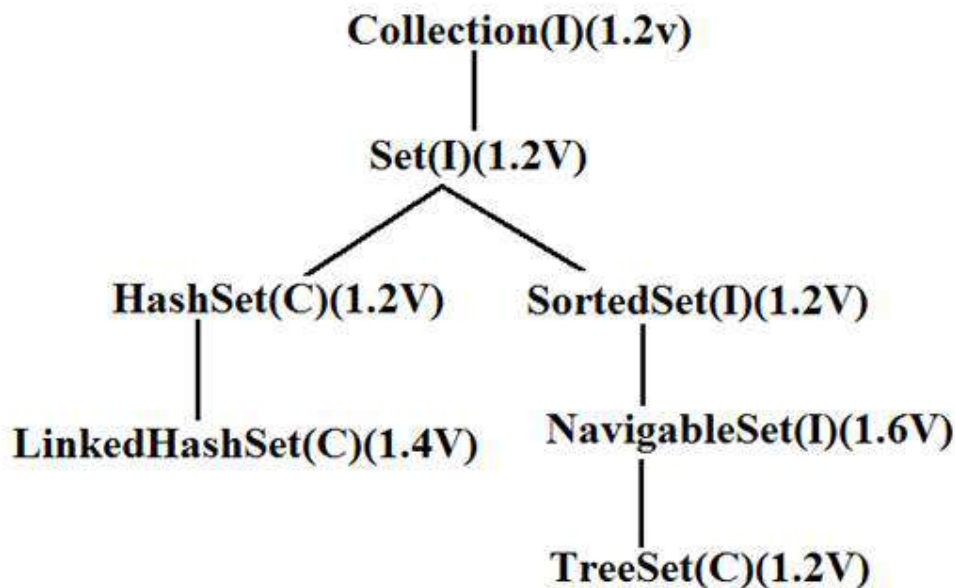
| Property                  | Enumeration          | Iterator                              | ListIterator                      |
|---------------------------|----------------------|---------------------------------------|-----------------------------------|
| 1) Is it legacy ?         | Yes                  | no                                    | no                                |
| 2) It is applicable for ? | Only legacy classes. | Applicable for any collection object. | Applicable for only list objects. |

|                   |                                   |                                  |                                 |
|-------------------|-----------------------------------|----------------------------------|---------------------------------|
| 3) Moment?        | Single direction cursor(forward)  | Single direction cursor(forward) | Bi-directional.                 |
| 4) How to get it? | By using elements() method.       | By using iterator()method.       | By using listIterator() method. |
| 5) Accessibility? | Only read.                        | Both read and remove.            | Read/remove/replace/add.        |
| 6) Methods        | hasMoreElement()<br>nextElement() | hasNext()<br>next()<br>remove()  | 9 methods.                      |

### Set interface:

1. It is the child interface of Collection.
2. If we want to represent a group of individual objects as a single entity where duplicates are not allow and insertion order is not preserved then we should go for Set interface.

### Diagram:



Set interface does not contain any new method we have to use only Collection interface methods.

## HashSet:

1. The underlying data structure is Hashtable.
2. Insertion order is not preserved and it is based on hash code of the objects.
3. Duplicate objects are not allowed.
4. If we are trying to insert duplicate objects we won't get compile time error and runtime error add() method simply returns false.
5. Heterogeneous objects are allowed.
6. Null insertion is possible.(only once)
7. Implements Serializable and Cloneable interfaces but not RandomAccess.
8. HashSet is best suitable, if our frequent operation is "Search".

### Constructors:

1. `HashSet h=new HashSet();`  
Creates an empty HashSet object with default initial capacity 16 and default fill ratio 0.75(fill ratio is also known as load factor).
2. `HashSet h=new HashSet(int initialcapacity);`  
Creates an empty HashSet object with the specified initial capacity and default fill ratio 0.75.
3. `HashSet h=new HashSet(int initialcapacity,float fillratio);`
4. `HashSet h=new HashSet(Collection c);`

**Note :** After filling how much ratio new HashSet object will be created , The ratio is called "FillRatio" or "LoadFactor".

### Example:

```
import java.util.*;
class HashSetDemo
{
    public static void main(String[] args)
    {
        HashSet h=new HashSet();
        h.add("B");
        h.add("C");
        h.add("D");
        h.add("Z");
        h.add(null);
        h.add(10);
        System.out.println(h.add("Z")); //false
        System.out.println(h); // [null, D, B, C, 10, Z]
    }
}
```

**LinkedHashSet:**

1. It is the child class of HashSet.
2. LinkedHashSet is exactly same as HashSet except the following differences.

| HashSet                                        | LinkedHashSet                                                                  |
|------------------------------------------------|--------------------------------------------------------------------------------|
| 1) The underlying data structure is Hashtable. | 1) The underlying data structure is a combination of LinkedList and Hashtable. |
| 2) Insertion order is not preserved.           | 2) Insertion order is preserved.                                               |
| 3) Introduced in 1.2 v.                        | 3) Introduced in 1.4v.                                                         |

In the above program if we are replacing HashSet with LinkedHashSet the output is [B, C, D, Z, null, 10]. That is insertion order is preserved.

**Example:**

```
import java.util.*;
class LinkedHashSetDemo
{
    public static void main(String[] args)
    {
        LinkedHashSet h=new LinkedHashSet();
        h.add("B");
        h.add("C");
        h.add("D");
        h.add("Z");
        h.add(null);
        h.add(10);
        System.out.println(h.add("Z")); //false
        System.out.println(h); // [B, C, D, Z, null, 10]
    }
}
```

**Note:** LinkedHashSet and LinkedHashMap commonly used for implementing "cache applications" where insertion order must be preserved and duplicates are not allowed.

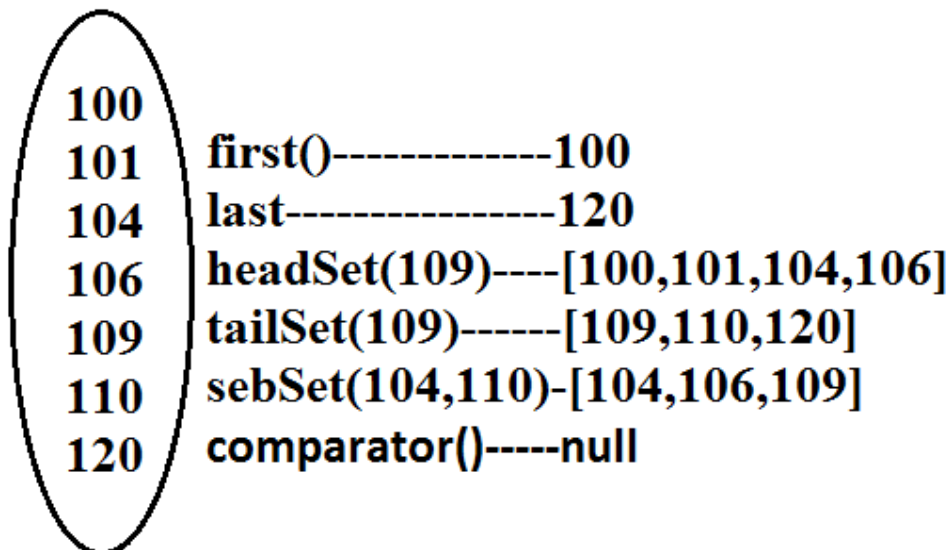
**SortedSet:**

1. It is child interface of Set.
2. If we want to represent a group of "unique objects" where duplicates are not allowed and all objects must be inserting according to some sorting order then we should go for SortedSet interface.
3. That sorting order can be either default natural sorting (or) customized sorting order.

SortedSet interface define the following 6 specific methods.

1. Object first();
2. Object last();
3. SortedSet headSet(Object obj);  
Returns the SortedSet whose elements are <obj.
4. SortedSet tailSet(Object obj);  
It returns the SortedSet whose elements are >=obj.
5. SortedSet subset(Object o1, Object o2);  
Returns the SortedSet whose elements are >=o1 but <o2.
6. Comparator comparator();
  - o Returns the Comparator object that describes underlying sorting technique.
  - o If we are following default natural sorting order then this method returns null.

Diagram:



### TreeSet:

1. The underlying data structure is balanced tree.
2. Duplicate objects are not allowed.
3. Insertion order is not preserved and it is based on some sorting order of objects.
4. Heterogeneous objects are not allowed if we are trying to insert heterogeneous objects then we will get ClassCastException.
5. Null insertion is possible(only once).

**Constructors:**

1. `TreeSet t=new TreeSet();`  
Creates an empty `TreeSet` object where all elements will be inserted according to default natural sorting order.
2. `TreeSet t=new TreeSet(Comparator c);`  
Creates an empty `TreeSet` object where all objects will be inserted according to customized sorting order specified by `Comparator` object.
3. `TreeSet t=new TreeSet(SortedSet s);`
4. `TreeSet t=new TreeSet(Collection c);`

**Example 1:**

```
import java.util.*;
class TreeSetDemo
{
    public static void main(String[] args)
    {
        TreeSet t=new TreeSet();
        t.add("A");
        t.add("a");
        t.add("B");
        t.add("Z");
        t.add("L");
        //t.add(new Integer(10)); //ClassCastException
        //t.add(null); //NullPointerException
        System.out.println(t); // [A, B, L, Z, a]
    }
}
```

**Null acceptance:**

- For the empty `TreeSet` as the 1st element "null" insertion is possible but after inserting that null if we are trying to insert any other we will get `NullPointerException`.
- For the non empty `TreeSet` if we are trying to insert null then we will get `NullPointerException`.

**Example 2:**

```
import java.util.*;
class TreeSetDemo
{
    public static void main(String[] args)
    {
        TreeSet t=new TreeSet();
        t.add(new StringBuffer("A"));
        t.add(new StringBuffer("Z"));
        t.add(new StringBuffer("L"));
        t.add(new StringBuffer("B"));
        System.out.println(t);
    }
}
```

```

    }
}

```

Output:

Runtime Exception.

Note :

- Exception in thread "main" java.lang.ClassCastException: java.lang.StringBuffer cannot be cast to java.lang.Comparable
- If we are depending on default natural sorting order compulsory the objects should be homogeneous and Comparable otherwise we will get ClassCastException.
- An object is said to be Comparable if and only if the corresponding class implements Comparable interface.
- String class and all wrapper classes implements Comparable interface but StringBuffer class doesn't implement Comparable interface hence in the above program we are getting ClassCastException.

### Comparable interface:

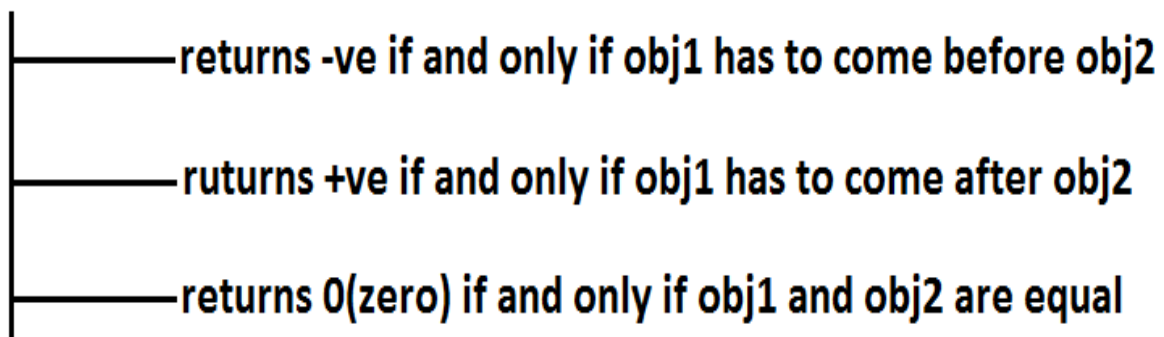
Comparable interface present in java.lang package and contains only one method compareTo() method.

```
public int compareTo(Object obj);
```

Example:

```
obj1.compareTo(obj2);
```

Diagram:



Example 3:

```

class Test
{
    public static void main(String[] args)
    {
        System.out.println("A".compareTo("Z")); //- 25
    }
}

```

```

        System.out.println("Z".compareTo("K")); //15
        System.out.println("A".compareTo("A")); //0
        //System.out.println("A".compareTo(new
Integer(10)));
        //Test.java:8: compareTo(java.lang.String)
in java.lang.String cannot
        be applied to (java.lang.Integer)

        //System.out.println("A".compareTo(null)); //NullPointerException
    }
}

```

If we are depending on default natural sorting order then internally JVM will use `compareTo()` method to arrange objects in sorting order.

#### Example 4:

```

import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        TreeSet t=new TreeSet();
        t.add(10);
        t.add(0);
        t.add(15);
        t.add(10);
        System.out.println(t); //[0, 10, 15]
    }
}

```

#### compareTo() method analysis:

```

TreeSet t=new TreeSet();
t.add(10); → [10]
t.add(0); → -ve → 0.compareTo(10); [0,10]
t.add(15); → +ve → 15.compareTo(0); [0,15,10]
           → +ve → 15.compareTo(10); [0,10,15]
t.add(10); → +ve → 10.compareTo(0); 10
           → 0(zero) → 10.compareTo(10); [0,10,15]

```



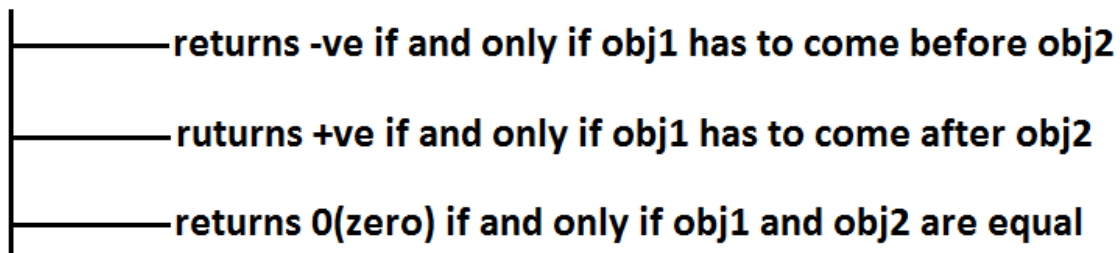
- If we are not satisfying with default natural sorting order (or) if default natural sorting order is not available then we can define our own customized sorting by Comparator object.
- Comparable meant for default natural sorting order.
- Comparator meant for customized sorting order.

### Comparator interface:

Comparator interface present in java.util package this interface defines the following 2 methods.

1) *public int compare(Object obj1, Object Obj2);*

Diagram:



2) *public boolean equals(Object obj);*

- Whenever we are implementing Comparator interface we have to provide implementation only for compare() method.
- Implementing equals() method is optional because it is already available from Object class through inheritance.

Requirement: Write a program to insert integer objects into the TreeSet where the sorting order is descending order.

Program:

```

import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        TreeSet t=new TreeSet(new MyComparator());  //--
->(1)        t.add(10);
              t.add(0);
              t.add(15);
              t.add(5);
              t.add(20);
    }
}
  
```

```

        System.out.println(t);//[20, 15, 10, 5, 0]
    }
}
class MyComparator implements Comparator
{
    public int compare(Object obj1,Object obj2)
    {
        Integer i1=(Integer)obj1;
        Integer i2=(Integer)obj2;
        if(i1<i2)
            return +1;
        else if(i1 > i2)
            return -100;
        else return 0;
    }
}

```

- At line "1" if we are not passing Comparator object then JVM will always calls compareTo() method which is meant for default natural sorting order(ascending order)hence in this case the output is [0, 5, 10, 15, 20].
- At line "1" if we are passing Comparator object then JVM calls compare() method of MyComparator class which is meant for customized sorting order(descending order) hence in this case the output is [20, 15, 10, 5, 0].

Diagram:

```

TreeSet t=new TreeSet(new MyComparator());
t.add(10);    [10]
t.add(0);     $\xrightarrow{+ve}$  compare(0,10) [10,0]
t.add(15);    $\xrightarrow{-ve}$  compare(15,10)[15,10,0]
t.add(5);     $\xrightarrow{+ve}$  compare(5,15) [15,5,10,0]
               $\xrightarrow{+ve}$  compare(5,10) [15,10,5,0]
               $\xrightarrow{-ve}$  compare(5,0) [15,10,5,0]
t.add(20);    $\longrightarrow$  compare(20,15) [20,15,10,5,0]

```

Various alternative implementations of compare() method:

```

public int compare(Object obj1,Object obj2)
{
    Integer i1=(Integer)obj1;
    Integer i2=(Integer)obj2;

```

```

        //return i1.compareTo(i2);//[0, 5, 10, 15, 20]
        //return -i1.compareTo(i2);//[20, 15, 10, 5, 0]
        //return i2.compareTo(i1);//[20, 15, 10, 5, 0]
        //return -i2.compareTo(i1);//[0, 5, 10, 15, 20]
        //return -1;//[20, 5, 15, 0, 10]//reverse of
insertion order
        //return +1;//[10, 0, 15, 5, 20]//insertion order
        //return 0;//[10]and all the remaining elements
treated as duplicate.
    }

```

**Requirement:** Write a program to insert String objects into the TreeSet where the sorting order is reverse of alphabetical order.

**Program:**

```

import java.util.*;
class TreeSetDemo
{
    public static void main(String[] args)
    {
        TreeSet t=new TreeSet(new MyComparator());
        t.add("Roja");
        t.add("ShobaRani");
        t.add("RajaKumari");
        t.add("GangaBhavani");
        t.add("Ramulamma");
        System.out.println(t);//[ShobaRani, Roja,
Ramulamma, RajaKumari, GangaBhavani]
    }
}
class MyComparator implements Comparator
{
    public int compare(Object obj1,Object obj2)
    {
        String s1=obj1.toString();
        String s2=(String)obj2;
        //return s2.compareTo(s1);
        return -s1.compareTo(s2);
    }
}

```

**Requirement:** Write a program to insert StringBuffer objects into the TreeSet where the sorting order is alphabetical order.

**Program:**

```

import java.util.*;
class TreeSetDemo
{
    public static void main(String[] args)
    {
        TreeSet t=new TreeSet(new MyComparator());
        t.add(new StringBuffer("A"));
    }
}

```

```

        t.add(new StringBuffer("Z"));
        t.add(new StringBuffer("K"));
        t.add(new StringBuffer("L"));
        System.out.println(t);// [A, K, L, Z]
    }
}
class MyComparator implements Comparator
{
    public int compare(Object obj1,Object obj2)
    {
        String s1=obj1.toString();
        String s2=obj2.toString();
        return s1.compareTo(s2);
    }
}

```

**Note:** Whenever we are defining our own customized sorting by Comparator then the objects need not be Comparable.

**Example:** StringBuffer

**Requirement:** Write a program to insert String and StringBuffer objects into the TreeSet where the sorting order is increasing length order. If 2 objects having the same length then consider they alphabetical order.

**Program:**

```

import java.util.*;
class TreeSetDemo
{
    public static void main(String[] args)
    {
        TreeSet t=new TreeSet(new MyComparator());
        t.add("A");
        t.add(new StringBuffer("ABC"));
        t.add(new StringBuffer("AA"));
        t.add("xx");
        t.add("ABCD");
        t.add("A");
        System.out.println(t);//[A, AA, xx, ABC, ABCD]
    }
}
class MyComparator implements Comparator
{
    public int compare(Object obj1,Object obj2)
    {
        String s1=obj1.toString();
        String s2=obj2.toString();
        int l1=s1.length();
        int l2=s2.length();
        if(l1 < l2)
            return -1;
        else if(l1 > l2)

```

```

        return 1;
    else
        return s1.compareTo(s2);
    }
}

```

**Note:** If we are depending on default natural sorting order then the objects should be "homogeneous and comparable" otherwise we will get ClassCastException. If we are defining our own sorting by Comparator then objects "need not be homogeneous and comparable".

### Comparable vs Comparator:

- For predefined Comparable classes default natural sorting order is already available if we are not satisfied with default natural sorting order then we can define our own customized sorting order by Comparator.
- For predefined non Comparable classes [like StringBuffer] default natural sorting order is not available we can define our own sorting order by using Comparator object.
- For our own classes [like Customer, Student, and Employee] we can define default natural sorting order by using Comparable interface. The person who is using our class, if he is not satisfied with default natural sorting order then he can define his own sorting order by using Comparator object.

### Example:

```

import java.util.*;
class Employee implements Comparable
{
    String name;
    int eid;
    Employee(String name,int eid)
    {
        this.name=name;
        this.eid=eid;
    }
    public String toString()
    {
        return name+"----"+eid;
    }
    public int compareTo(Object o)
    {
        int eid1=this.eid;
        int eid2=((Employee)o).eid;
        if(eid1 < eid2)
        {
            return -1;
        }
    }
}

```

```

    }
    else if(eid1 > eid2)
    {
        return 1;
    }
    else return 0;
}
}
class CompComp
{
    public static void main(String[] args)
    {
        Employee e1=new Employee("nag",100);
        Employee e2=new Employee("balaiah",200);
        Employee e3=new Employee("chiru",50);
        Employee e4=new Employee("venki",150);
        Employee e5=new Employee("nag",100);
        TreeSet t1=new TreeSet();
        t1.add(e1);
        t1.add(e2);
        t1.add(e3);
        t1.add(e4);
        t1.add(e5);
        System.out.println(t1);//[chiru----50, nag----
100, venki----150, balaiah----200]
        TreeSet t2=new TreeSet(new MyComparator());
        t2.add(e1);
        t2.add(e2);
        t2.add(e3);
        t2.add(e4);
        t2.add(e5);
        System.out.println(t2);//[balaiah----200, chiru--
--50, nag----100, venki----150]
    }
}
class MyComparator implements Comparator
{
    public int compare(Object obj1,Object obj2)
    {
        Employee e1=(Employee)obj1;
        Employee e2=(Employee)obj2;
        String s1=e1.name;
        String s2=e2.name;
        return s1.compareTo(s2);
    }
}

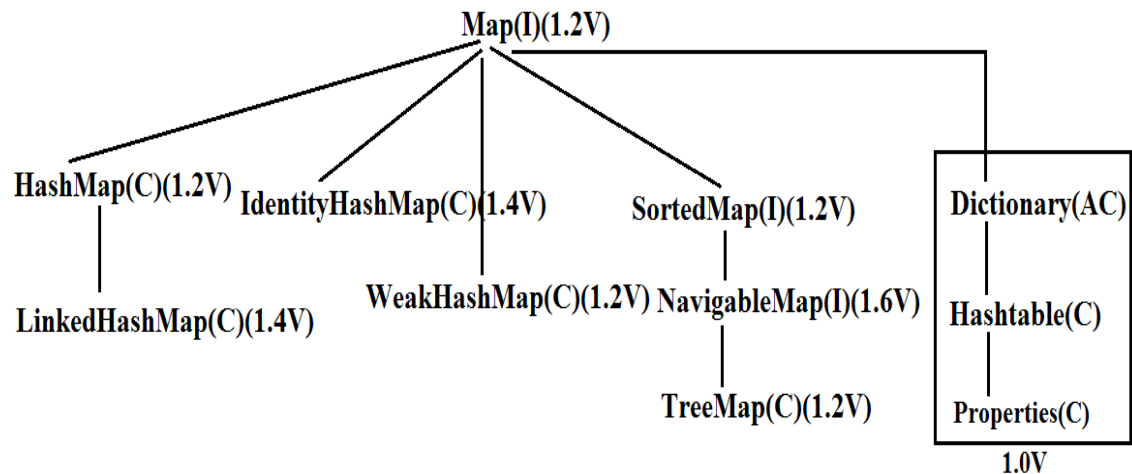
```

### Compression of Comparable and Comparator ?

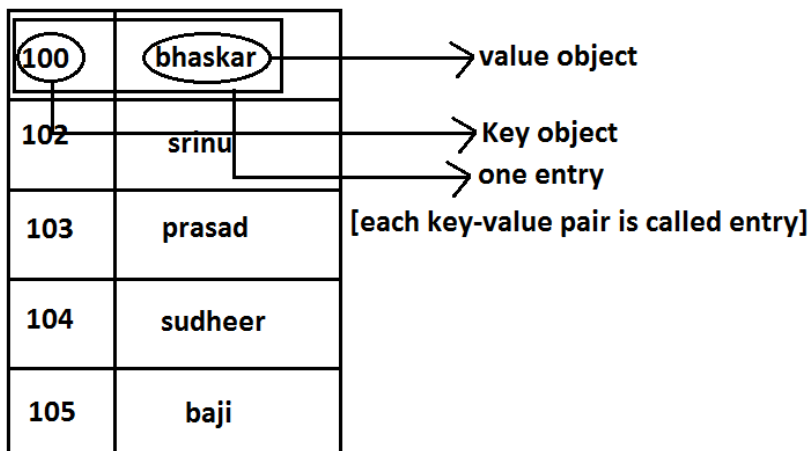
| Comparable                                                               | Comparator                                                                                      |
|--------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| 1) Comparable meant for default natural sorting order.                   | 1) Comparator meant for customized sorting order.                                               |
| 2) Present in java.lang package.                                         | 2) Present in java.util package.                                                                |
| 3) Contains only one method. compareTo() method.                         | 3) Contains 2 methods. Compare() method. Equals() method.                                       |
| 4) String class and all wrapper Classes implements Comparable interface. | 4) The only implemented classes of Comparator are Collator and RuleBasedCollator. (used in GUI) |

**Comparison of Set implemented class objects:**

| Property                      | HashSet         | LinkedHashSet           | TreeSet                                                                                                 |
|-------------------------------|-----------------|-------------------------|---------------------------------------------------------------------------------------------------------|
| 1) Underlying Data structure. | Hashtable.      | LinkedList + Hashtable. | Balanced Tree.                                                                                          |
| 2) Insertion order.           | Not preserved.  | Preserved.              | Not preserved (by default).                                                                             |
| 3) Duplicate objects.         | Not allowed.    | Not allowed.            | Not allowed.                                                                                            |
| 4) Sorting order.             | Not applicable. | Not applicable.         | Applicable.                                                                                             |
| 5) Heterogeneous objects.     | Allowed.        | Allowed.                | Not allowed.                                                                                            |
| 6) Null insertion.            | Allowed.        | Allowed.                | For the empty TreeSet as the 1st element null insertion is possible in all other cases we will get NPE. |

**Map:****Diagram:**

1. If we want to represent a group of objects as "key-value" pair then we should go for Map interface.
2. Both key and value are objects only.
3. Duplicate keys are not allowed but values can be duplicated
4. Each key-value pair is called "one entry".

**Diagram:**



- Map interface is not child interface of Collection and hence we can't apply Collection interface methods here.
  - Map interface defines the following specific methods.
1. Object put(Object key, Object value);  
To add an entry to the Map, if key is already available then the old value replaced with new value and old value will be returned.

Example:

- ```
2. import java.util.*;
3. class Map
4. {
5.     public static void main(String[] args)
6.     {
7.         HashMap m=new HashMap();
8.         m.put("100", "vijay");
9.         System.out.println(m); //{100=vijay}
10.        m.put("100", "ashok");
11.        System.out.println(m); //{100=ashok}
12.    }
13. }
```
14. void putAll(Map m);
  15. Object get(Object key);
  16. Object remove(Object key);  
It removes the entry associated with specified key and returns the corresponding value.
  17. boolean containsKey(Object key);
  18. boolean containsValue(Object value);
  19. boolean isEmpty();
  20. int size();
  21. void clear();
  22. Set keySet();  
The set of keys we are getting.
  23. Collection values();  
The set of values we are getting.
  24. Set entrySet();  
The set of entryset we are getting.

**Entry interface:**

Each key-value pair is called one entry. Hence Map is considered as a group of entry Objects, without existing Map object there is no chance of existing entry object hence interface entry is define inside Map interface(inner interface).

Example:

```
interface Map
```

```

{
    .....;
    .....;
    .....;
    interface Entry
    {
        Object getKey();
        Object getValue();
        Object setValue(Object new);
    }
}

```

on Entry we can apply these 3 methods.

### HashMap:

1. The underlying data structure is Hashtable.
2. Duplicate keys are not allowed but values can be duplicated.
3. Insertion order is not preserved and it is based on hash code of the keys.
4. Heterogeneous objects are allowed for both key and value.
5. Null is allowed for keys(only once) and for values(any number of times).
6. It is best suitable for Search operations.

### Differences between HashMap and Hashtable ?

| HashMap                                                                                           | Hashtable                                                                                                       |
|---------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| 1) No method is synchronized.                                                                     | 1) Every method is synchronized.                                                                                |
| 2) Multiple Threads can operate simultaneously on HashMap object and hence it is not Thread safe. | 2) Multiple Threads can't operate simultaneously on Hashtable object and hence Hashtable object is Thread safe. |
| 3) Relatively performance is high.                                                                | 3) Relatively performance is low.                                                                               |
| 4) Null is allowed for both key and value.                                                        | 4) Null is not allowed for both key and value otherwise we will get NullPointerException.                       |
| 5) It is non legacy and introduced in 1.2v.                                                       | 5) It is legacy and introduced in 1.0v.                                                                         |

### How to get synchronized version of HashMap:

By default HashMap object is not synchronized. But we can get synchronized version by using the following method of Collections class.

```
public static Map synchronizedMap(Map m1)
```

Constructors:

1. `HashMap m=new HashMap();`  
Creates an empty HashMap object with default initial capacity 16 and default fill ratio "0.75".
2. `HashMap m=new HashMap(int initialcapacity);`
3. `HashMap m =new HashMap(int initialcapacity, float fillratio);`
4. `HashMap m=new HashMap(Map m);`

Example:

```
import java.util.*;
class HashMapDemo
{
    public static void main(String[] args)
    {
        HashMap m=new HashMap();
        m.put("chiranjeevi",700);
        m.put("balaiah",800);
        m.put("venkatesh",200);
        m.put("nagarjuna",500);

        System.out.println(m);//{nagarjuna=500,venkatesh=200,bal
aiah=800,chiranjeevi=700}

        System.out.println(m.put("chiranjeevi",100));//700
        Set s=m.keySet();

        System.out.println(s);//[nagarjuna,venkatesh,balaiah,chi
ranjeevi]
        Collection c=m.values();
        System.out.println(c);//[500, 200, 800, 100]
        Set s1=m.entrySet();

        System.out.println(s1);//[nagarjuna=500,venkatesh=200,ba
laiah=800,chiranjeevi=100]
        Iterator itr=s1.iterator();
        while(itr.hasNext())
        {
            Map.Entry m1=(Map.Entry)itr.next();

            System.out.println(m1.getKey()+"....."+m1.getValue());
            //nagarjuna.....500
            //venkatesh.....200 //
            //balaiah.....800
            //chiranjeevi.....100
            if(m1.getKey().equals("nagarjuna"))
            {
                m1.setValue(1000);
            }
        }
    }
}
```

```

    }
    System.out.println(m);

//{nagarjuna=1000, venkatesh=200, balaiah=800, chiranjeevi=100}
}
}

```

### LinkedHashMap:

It is exactly same as HashMap except the following differences :

| HashMap                                        | LinkedHashMap                                                               |
|------------------------------------------------|-----------------------------------------------------------------------------|
| 1) The underlying data structure is Hashtable. | 1) The underlying data structure is a combination of Hashtable+ LinkedList. |
| 2) Insertion order is not preserved.           | 2) Insertion order is preserved.                                            |
| 3) introduced in 1.2.v.                        | 3) Introduced in 1.4v.                                                      |

**Note:** in the above program if we are replacing HashMap with LinkedHashMap then the output is {chiranjeevi=100, balaiah.....800, venkatesh.....200, nagarjuna.....1000} that is insertion order is preserved.

**Note:** in general we can use LinkedHashMap and LinkedHashMap for implementing cache applications.

### IdentityHashMap:

It is exactly same as HashMap except the following differences:

1. In the case of HashMap JVM will always use ".equals()" method to identify duplicate keys, which is meant for content comparison.
2. But in the case of IdentityHashMap JVM will use == (double equal operator) to identify duplicate keys, which is meant for reference comparison.

**Example:**

```

import java.util.*;
class HashMapDemo
{
    public static void main(String[] args)
    {
        HashMap m=new HashMap();
        Integer i1=new Integer(10);
    }
}

```

```

        Integer i2=new Integer(10);
        m.put(i1,"pavan");
        m.put(i2,"kalyan");
        System.out.println(m);
    }
}

```

- In the above program i1 and i2 are duplicate keys because i1.equals(i2) returns true.
- In the above program if we replace HashMap with IdentityHashMap then i1 and i2 are not duplicate keys because i1==i2 is false hence in this case the output is {10=pavan, 10=kalyan}.

```

System.out.println(m.get(10)); //null
10==i1-----false
10==i2-----false

```

### WeakHashMap:

It is exactly same as HashMap except the following differences:

- In the case of normal HashMap, an object is not eligible for GC even though it doesn't have any references if it is associated with HashMap. That is HashMap dominates garbage collector.
- But in the case of WeakHashMap if an object does not have any references then it's always eligible for GC even though it is associated with WeakHashMap that is garbage collector dominates WeakHashMap.

#### Example:

```

import java.util.*;
class WeakHashMapDemo
{
    public static void main(String[] args)throws Exception
    {
        WeakHashMap m=new WeakHashMap();
        Temp t=new Temp();
        m.put(t,"ashok");
        System.out.println(m); //{Temp=ashok}
        t=null;
        System.gc();
        Thread.sleep(5000);
        System.out.println(m); //{ }
    }
}
class Temp
{

```

```

    public String toString()
    {
        return "Temp";
    }
    public void finalize()
    {
        System.out.println("finalize() method called");
    }
}

```

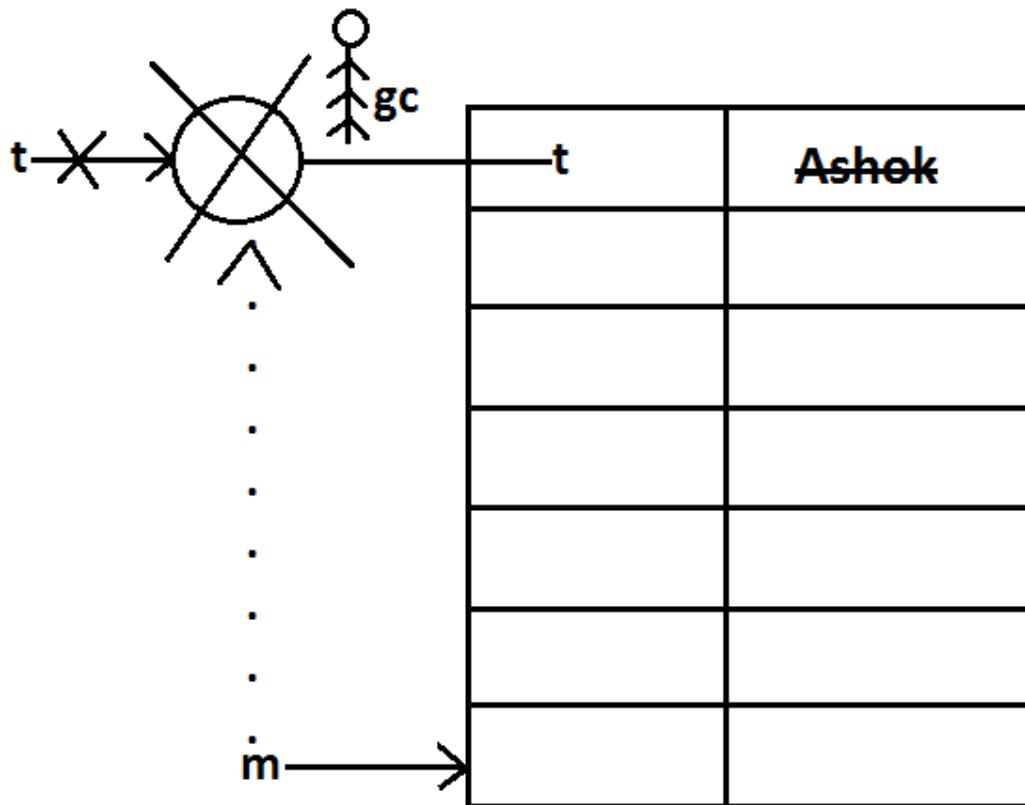
Output:

```

{Temp=ashok}
finalize() method called
{}

```

Diagram:



In the above program if we replace `WeakHashMap` with normal `HashMap` then object won't be destroyed by the garbage collector in this the output is

```

{Temp=ashok}
{Temp=ashok}

```

## SortedMap:

- It is the child interface of Map.
- If we want to represent a group of key-value pairs according to some sorting order of keys then we should go for SortedMap.
- Sorting is possible only based on the keys but not based on values.
- SortedMap interface defines the following 6 specific methods.
  1. Object firstKey();
  2. Object lastKey();
  3. SortedMap headMap(Object key);
  4. SortedMap tailMap(Object key);
  5. SortedMap subMap(Object key1, Object key2);
  6. Comparator comparator();

## TreeMap:

1. The underlying data structure is RED-BLACK Tree.
2. Duplicate keys are not allowed but values can be duplicated.
3. Insertion order is not preserved and all entries will be inserted according to some sorting order of keys.
4. If we are depending on default natural sorting order keys should be homogeneous and Comparable otherwise we will get ClassCastException.
5. If we are defining our own sorting order by Comparator then keys can be heterogeneous and non Comparable.
6. There are no restrictions on values they can be heterogeneous and non Comparable.
7. For the empty TreeMap as first entry null key is allowed but after inserting that entry if we are trying to insert any other entry we will get NullPointerException.
8. For the non empty TreeMap if we are trying to insert an entry with null key we will get NullPointerException.
9. There are no restrictions for null values.

### Constructors:

1. TreeMap t=new TreeMap();  
For default natural sorting order.
2. TreeMap t=new TreeMap(Comparator c);  
For customized sorting order.
3. TreeMap t=new TreeMap(SortedMap m);
4. TreeMap t=new TreeMap(Map m);

### Example 1:

```
import java.util.*;  
class TreeMapDemo  
{
```

```

public static void main(String[] args)
{
    TreeMap t=new TreeMap();
    t.put(100,"ZZZ");
    t.put(103,"YYY");
    t.put(101,"XXX");
    t.put(104,106);
    t.put(107,null);
    //t.put("FFF","XXX");//ClassCastException
    //t.put(null,"xxx");//NullPointerException
    System.out.println(t);//{100=ZZZ, 101=XXX,
103=YYY, 104=106, 107=null}
}
}

```

**Example 2:**

```

import java.util.*;
class TreeMapDemo
{
    public static void main(String[] args)
    {
        TreeMap t=new TreeMap(new MyComparator());
        t.put("XXX",10);
        t.put("AAA",20);
        t.put("ZZZ",30);
        t.put("LLL",40);
        System.out.println(t);//{ZZZ=30, XXX=10, LLL=40,
AAA=20}
    }
}
class MyComparator implements Comparator
{
    public int compare(Object obj1,Object obj2)
    {
        String s1=obj1.toString();
        String s2=obj2.toString();
        return s2.compareTo(s1);
    }
}

```

**Hashtable:**

1. The underlying data structure is Hashtable.
2. Insertion order is not preserved and it is based on hash code of the keys.
3. Heterogeneous objects are allowed for both keys and values.
4. Null key (or) null value is not allowed otherwise we will get NullPointerException.
5. Duplicate keys are allowed but values can be duplicated.
6. Every method present inside Hashtable is synchronized and hence Hashtable object is Thread-safe.

**Constructors:**



1. `Hashtable h=new Hashtable();`  
Creates an empty Hashtable object with default initialcapacity 11 and default fill ratio 0.75.
2. `Hashtable h=new Hashtable(int initialcapacity);`
3. `Hashtable h=new Hashtable(int initialcapacity,float fillratio);`
4. `Hashtable h=new Hashtable (Map m);`

Example:

```
import java.util.*;
class HashtableDemo
{
    public static void main(String[] args)
    {
        Hashtable h=new Hashtable();
        h.put(new Temp(5), "A");

        h.put(new Temp(2), "B");
        h.put(new Temp(6), "C");
        h.put(new Temp(15), "D");
        h.put(new Temp(23), "E");
        h.put(new Temp(16), "F");
        System.out.println(h);//{6=C, 16=F, 5=A, 15=D,
        2=B, 23=E}
    }
}
class Temp
{
    int i;
    Temp(int i)
    {
        this.i=i;
    }
    public int hashCode()
    {
        return i;
    }
    public String toString()
    {
        return i+"";
    }
}
```

Diagram:

|    |             |
|----|-------------|
| 10 |             |
| 9  |             |
| 8  |             |
| 7  |             |
| 6  | 6=C         |
| 5  | 5=A    16=F |
| 4  | 15=D        |
| 3  |             |
| 2  | 2=B         |
| 1  | 23=E        |
| 0  |             |

Note: if we change hashCode() method of Temp class as follows.

```
public int hashCode()
{
    return i%9;
}
```

Then the output is {16=F, 15=D, 6=C, 23=E, 5=A, 2=B}.

Diagram:

|    |           |
|----|-----------|
| 10 |           |
| 9  |           |
| 8  |           |
| 7  | 16=F      |
| 6  | 6=C,15=D  |
| 5  | 5=A, 23=E |
| 4  |           |
| 3  |           |
| 2  | 2=B       |
| 1  |           |
| 0  |           |

Note: if we change initial capacity as 25.

Hashtable h=new Hashtable(25);

output is : { 23=E, 16=F, 15=D, 6=C, 5=A, 2=B }

Diagram:

|    |      |
|----|------|
| 24 |      |
| 23 | 23=E |
| 22 |      |
| 21 |      |
| 20 |      |
| 19 |      |
| 18 |      |
| 17 |      |
| 16 | 16=F |
| 15 | 15=D |
| 14 |      |
| 13 |      |
| 12 |      |
| 11 |      |
| 10 |      |
| 9  |      |
| 8  |      |
| 7  |      |
| 6  | 6=C  |
| 5  | 5=A  |
| 4  |      |
| 3  |      |
| 2  | 2=B  |
| 1  |      |
| 0  |      |

## **Properties:**

1. Properties class is the child class of Hashtable.
2. In our program if anything which changes frequently like DBUserName, Password etc., such type of values not recommended to hardcode in java application because for every change we have to recompile, rebuild and redeployed the application and even server restart also required sometimes it creates a big business impact to the client.
3. Such type of variable things we have to hardcode in property files and we have to read the values from the property files into java application.
4. The main advantage in this approach is if there is any change in property files automatically those changes will be available to java application just redeployment is enough.
5. By using Properties object we can read and hold properties from property files into java application.

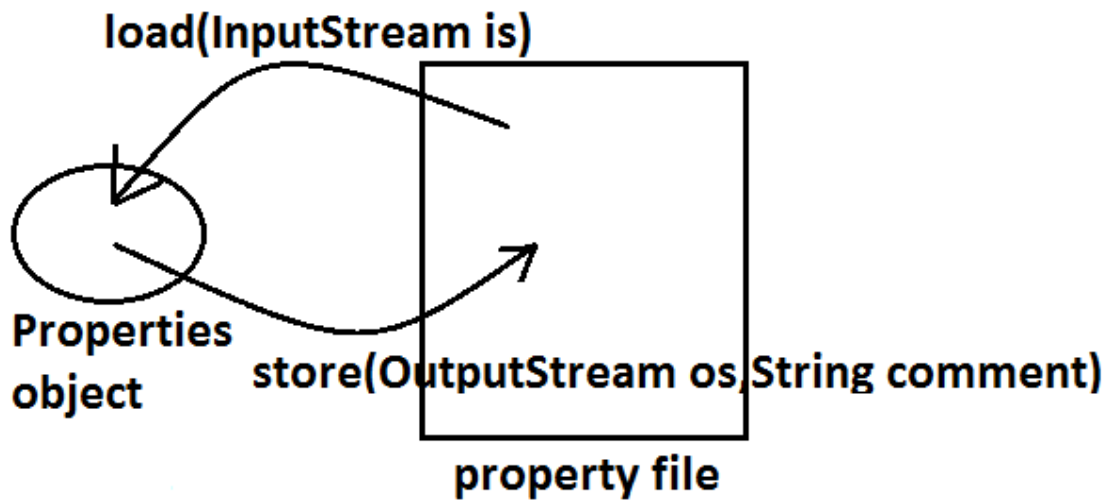
### Constructor:

Properties p=new Properties();

In properties both key and value "should be String type only".

### Methods:

1. String getProperty(String propertyname) ;  
Returns the value associated with specified property.
2. String setProperty(String propertyname,String propertyvalue);  
To set a new property.
3. Enumeration propertyNames();
4. void load(InputStream is); //Any InputStream we can pass.  
To load Properties from property files into java Properties object.
5. void store(OutputStream os,String comment); //Any OutputStream we can pass.  
To store the properties from Properties object into properties file.

Diagram:Example:

```

import java.util.*;
import java.io.*;
class PropertiesDemo
{
    public static void main(String[] args)throws Exception
    {
        Properties p=new Properties();
        FileInputStream fis=new
FileInputStream("abc.properties");
        p.load(fis);
        System.out.println(p);//{user=scott,
password=tiger, venki=8888}
        String s=p.getProperty("venki");
        System.out.println(s);//8888
        p.setProperty("nag", "9999999");
        Enumeration e=p.propertyNames();
        while(e.hasMoreElements())
        {
            String s1=(String)e.nextElement();
            System.out.println(s1);//nag
                                                //user
                                                //password
                                                //venki
        }
        FileOutputStream fos=new
FileOutputStream("abc.properties");
        p.store(fos,"updated by ashok for scjp demo
class");
    }
}
  
```

```
    }
}
```

Property file:

```
user=scott
nag=99999999
password=tiger
venki=8888
```

**abc.properties**

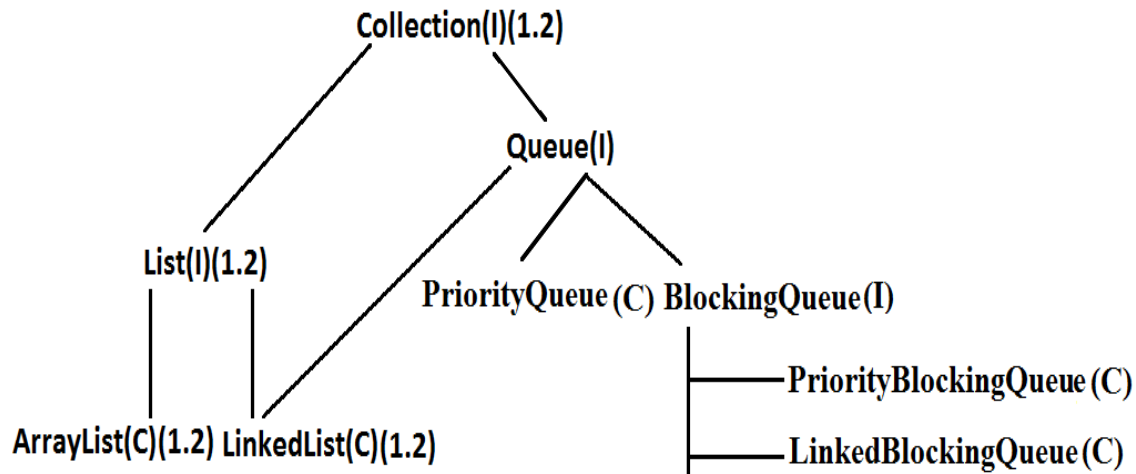
Example:

```
import java.util.*;
import java.io.*;
class PropertiesDemo
{
    public static void main(String[] args)throws Exception
    {
        Properties p=new Properties();
        FileInputStream fis=new
FileInputStream("db.properties");
        p.load(fis);
        String url=p.getProperty("url");
        String user=p.getProperty("user");
        String pwd=p.getProperty("pwd");
        Connection con=DriverManager.getConnection(url,
user, pwd);
        -----
        -----
        -----
        FileOutputStream fos=new
FileOutputStream("db.properties");
        p.store(fos,"updated by ashok for scjp demo
class");
    }
}
```

## 1.5 enhancements

### Queue interface

Diagram:



1. Queue is child interface of Collections.
2. If we want to represent a group of individual objects prior (happening before something else) to processing then we should go for Queue interface.
3. Usually Queue follows first in first out(FIFO) order but based on our requirement we can implement our own order also.
4. From 1.5v onwards LinkedList also implements Queue interface.
5. LinkedList based implementation of Queue always follows first in first out order.

Assume we have to send sms for one lakh mobile numbers , before sending messages we have to store all mobile numbers into Queue so that for the first inserted number first message will be triggered(FIFO).

#### Queue interface methods:

1. `boolean offer(Object o);`  
To add an object to the Queue.
2. `Object poll() ;`  
To remove and return head element of the Queue, if Queue is empty then we will get null.
3. `Object remove();`  
To remove and return head element of the Queue. If Queue is empty then this method raises Runtime Exception saying `NoSuchElementException`.



4. **Object peek();**  
To return head element of the Queue without removal, if Queue is empty this method returns null.
5. **Object element();**  
It returns head element of the Queue and if Queue is empty then it will raise Runtime Exception saying NoSuchElementException.

**PriorityQueue:**

1. PriorityQueue is a data structure to represent a group of individual objects prior to processing according to some priority.
2. The priority order can be either default natural sorting order (or) customized sorting order specified by Comparator object.
3. If we are depending on default natural sorting order then the objects must be homogeneous and Comparable otherwise we will get ClassCastException.
4. If we are defining our own customized sorting order by Comparator then the objects need not be homogeneous and Comparable.
5. Duplicate objects are not allowed.
6. Insertion order is not preserved but all objects will be inserted according to some priority.
7. Null is not allowed even as the 1st element for empty PriorityQueue. Otherwise we will get the "NullPointerException".

**Constructors:**

1. **PriorityQueue q=new PriorityQueue();**  
Creates an empty PriorityQueue with default initial capacity 11 and default natural sorting order.
2. **PriorityQueue q=new PriorityQueue(int initialcapacity,Comparator c);**
3. **PriorityQueue q=new PriorityQueue(int initialcapacity);**
4. **PriorityQueue q=new PriorityQueue(Collection c);**
5. **PriorityQueue q=new PriorityQueue(SortedSet s);**

**Example 1:**

```
import java.util.*;
class PriorityQueueDemo
{
    public static void main(String[] args)
    {
        PriorityQueue q=new PriorityQueue();
        //System.out.println(q.peek()); //null

        //System.out.println(q.element()); //NoSuchElementException
on
        for(int i=0;i<=10;i++)
        {
            q.offer(i);
        }
    }
}
```

```

        System.out.println(q);//[0, 1, 2, 3, 4, 5, 6, 7,
8, 9, 10]
        System.out.println(q.poll());//0
        System.out.println(q);//[1, 3, 2, 7, 4, 5, 6, 10,
8, 9]
    }
}

```

**Note:** Some platforms may not provide proper supports for PriorityQueue [windowsXP].

#### Example 2:

```

import java.util.*;
class PriorityQueueDemo
{
    public static void main(String[] args)
    {
        PriorityQueue q=new PriorityQueue(15,new
MyComparator());
        q.offer("A");
        q.offer("Z");
        q.offer("L");
        q.offer("B");
        System.out.println(q);//[Z, B, L, A]
    }
}
class MyComparator implements Comparator
{
    public int compare(Object obj1,Object obj2)
    {
        String s1=(String)obj1;
        String s2=obj2.toString();
        return s2.compareTo(s1);
    }
}

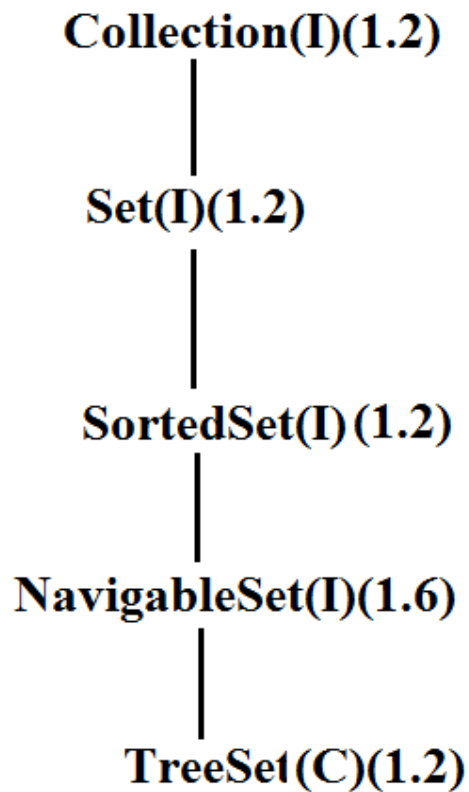
```

#### 1.6v Enhancements :

##### **NavigableSet:**

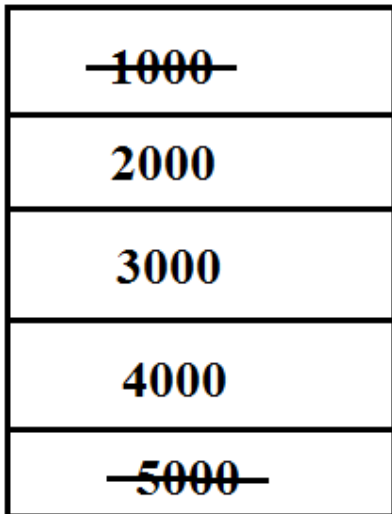
1. It is the child interface of SortedSet.
2. It provides several methods for navigation purposes.

Diagram:



NavigableSet interface defines the following methods.

1. **ceiling(e);**  
It returns the lowest element which is  $\geq e$ .
2. **higher(e);**  
It returns the lowest element which is  $> e$ .
3. **floor(e);**  
It returns highest element which is  $\leq e$ .
4. **lower(e);**  
It returns height element which is  $< e$ .
5. **pollFirst ();**  
Remove and return 1st element.
6. **pollLast ();**  
Remove and return last element.
7. **descendingSet ();**  
Returns SortedSet in reverse order.

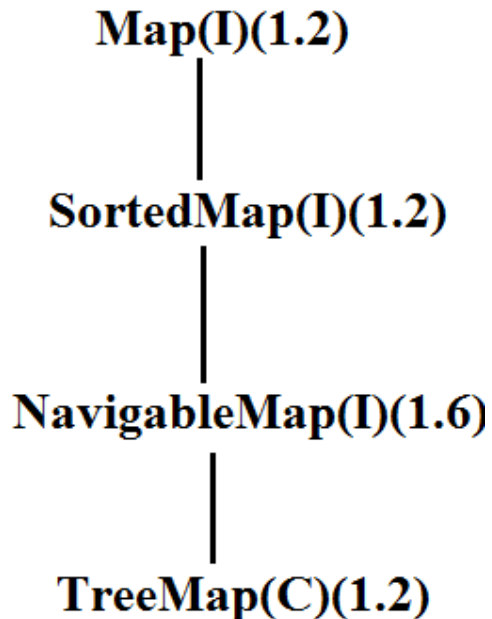
Diagram:Example:

```
import java.util.*;
class NavigableSetDemo
{
    public static void main(String[] args)
    {
        TreeSet<Integer> t=new TreeSet<Integer>();
        t.add(1000);
        t.add(2000);
        t.add(3000);
        t.add(4000);
        t.add(5000);
        System.out.println(t);//[1000, 2000, 3000, 4000,
5000]
        System.out.println(t.ceiling(2000));//2000
        System.out.println(t.higher(2000));//3000
        System.out.println(t.floor(3000));//3000
        System.out.println(t.lower(3000));//2000
        System.out.println(t.pollFirst());//1000
        System.out.println(t.pollLast());//5000
        System.out.println(t.descendingSet());//[4000,
3000, 2000]
        System.out.println(t);//[2000, 3000, 4000]
    }
}
```

**NavigableMap:**

It is the child interface of SortedMap and it defines several methods for navigation purpose.

Diagram:



NavigableMap interface defines the following methods.

1. ceilingKey(e);
2. higherKey(e);
3. floorKey(e);
4. lowerKey(e);
5. pollFirstEntry();
6. pollLastEntry();
7. descendingMap();

Example:

```
import java.util.*;
class NavigableMapDemo
{
    public static void main(String[] args)
    {
        TreeMap<String,String> t=new
TreeMap<String,String>();
        t.put("b", "banana");
        t.put("c", "cat");
        t.put("a", "apple");
        t.put("d", "dog");
```

```

        t.put("g", "gun");
        System.out.println(t); //{a=apple, b=banana,
c=cat, d=dog, g=gun}
        System.out.println(t.ceilingKey("c")); //c
        System.out.println(t.higherKey("e")); //g
        System.out.println(t.floorKey("e")); //d
        System.out.println(t.lowerKey("e")); //d
        System.out.println(t.pollFirstEntry()); //a=apple
        System.out.println(t.pollLastEntry()); //g=gun
        System.out.println(t.descendingMap()); //{d=dog,
c=cat, b=banana}
        System.out.println(t); //{b=banana, c=cat, d=dog}
    }
}

```

Diagram:

|                 |
|-----------------|
| <b>a=apple</b>  |
| <b>b=banana</b> |
| <b>c=cat</b>    |
| <b>d=dog</b>    |
| <b>g=gun</b>    |

### **Collections class:**

Collections class defines several utility methods for collection objects.

#### Sorting the elements of a List:

Collections class defines the following methods to perform sorting the elements of a List.

```
public static void sort(List l);
```

- To sort the elements of List according to default natural sorting order in this case the elements should be homogeneous and comparable otherwise we will get **ClassCastException**.
- The List should not contain null otherwise we will get **NullPointerException**.

```
public static void sort(List l,Comparator c);
```

- To sort the elements of List according to customized sorting order.

**Program 1:** To sort elements of List according to natural sorting order.

```
import java.util.*;
class CollectionsDemo
{
    public static void main(String[] args)
    {
        ArrayList l=new ArrayList();
        l.add("Z");
        l.add("A");
        l.add("K");
        l.add("N");
        //l.add(new Integer(10));//ClassCastException
        //l.add(null);//NullPointerException
        System.out.println("Before sorting :"+l);//[Z, A,
K, N]
        Collections.sort(l);
        System.out.println("After sorting :"+l);//[A, K,
N, Z]
    }
}
```

**Program 2:** To sort elements of List according to customized sorting order.

```
import java.util.*;
class CollectionsDemo
{
    public static void main(String[] args)
    {
        ArrayList l=new ArrayList();
        l.add("Z");
        l.add("A");
        l.add("K");
        l.add("L");
        l.add(new Integer(10));
        //l.add(null);//NullPointerException
        System.out.println("Before sorting :"+l);//[Z, A,
K, L, 10]
        Collections.sort(l,new MyComparator());
        System.out.println("After sorting :"+l);//[Z, L,
K, A, 10]
    }
}
class MyComparator implements Comparator
{
    public int compare(Object obj1,Object obj2)
    {
        String s1=(String)obj1;
```

```

        String s2=obj2.toString();
        return s2.compareTo(s1);
    }
}

```

### Searching the elements of a List:

Collections class defines the following methods to search the elements of a List.

```
public static int binarySearch(List l, Object obj);
```

- If the List is sorted according to default natural sorting order then we have to use this method.

```
public static int binarySearch(List l, Object obj, Comparator c);
```

- If the List is sorted according to Comparator then we have to use this method.

**Program 1:** To search elements of List.

```

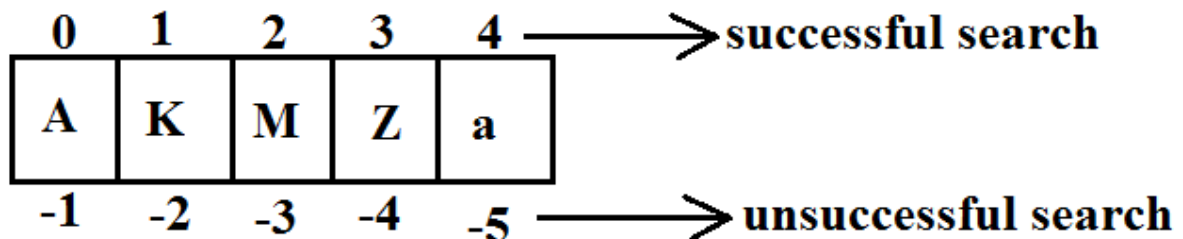
import java.util.*;
class CollectionsSearchDemo
{
    public static void main(String[] args)
    {
        ArrayList l=new ArrayList();
        l.add("Z");
        l.add("A");
        l.add("M");
        l.add("K");
        l.add("a");
        System.out.println(l);//[Z, A, M, K, a]
        Collections.sort(l);
        System.out.println(l);//[A, K, M, Z, a]

        System.out.println(Collections.binarySearch(l,"Z"));//3

        System.out.println(Collections.binarySearch(l,"J"));//-2
    }
}

```

Diagram:





**Program 2:**

```

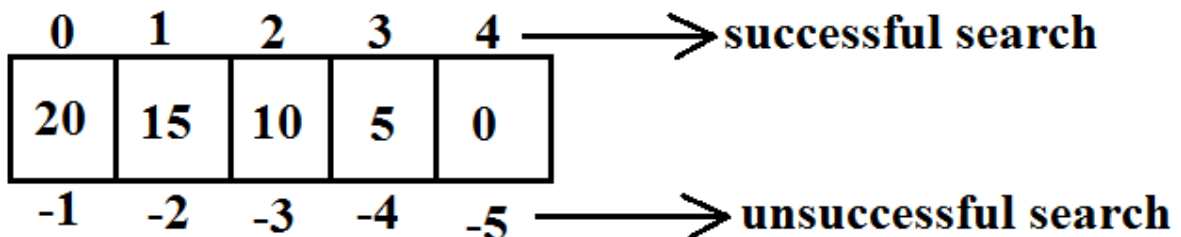
import java.util.*;
class CollectionsSearchDemo
{
    public static void main(String[] args)
    {
        ArrayList l=new ArrayList();
        l.add(15);
        l.add(0);
        l.add(20);
        l.add(10);
        l.add(5);
        System.out.println(l);//[15, 0, 20, 10, 5]
        Collections.sort(l,new MyComparator());
        System.out.println(l);//[20, 15, 10, 5, 0]

        System.out.println(Collections.binarySearch(l,10,new
MyComparator()));//2

        System.out.println(Collections.binarySearch(l,13,new
MyComparator()));//-3

        System.out.println(Collections.binarySearch(l,17));//-6
    }
}
class MyComparator implements Comparator
{
    public int compare(Object obj1,Object obj2)
    {
        Integer i1=(Integer)obj1;
        Integer i2=(Integer)obj2;
        return i2.compareTo(i1);
    }
}

```

**Diagram:**

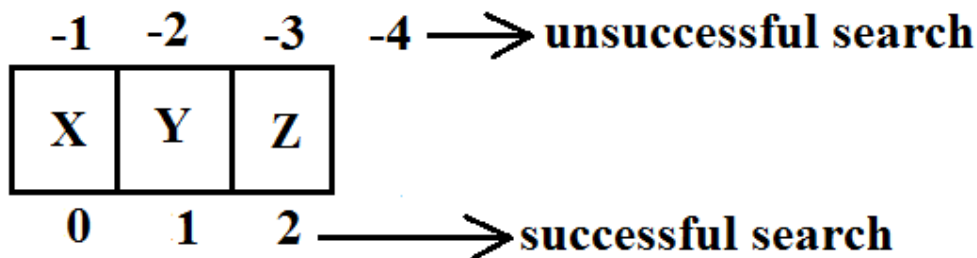
**Conclusions:**

1. Internally these search methods will use binary search algorithm.
2. Successful search returns index unsuccessful search returns insertion point.
3. Insertion point is the location where we can place the element in the sorted list.
4. Before calling `binarySearch()` method compulsory the list should be sorted otherwise we will get unpredictable results.
5. If the list is sorted according to `Comparator` then at the time of search operation also we should pass the same `Comparator` object otherwise we will get unpredictable results.

**Note:**

For the list of n elements with respect to `binary Search()` method.

- Successful search range is: 0 to n-1.
- Unsuccessful search results range is: -(n+1)to -1.
- Total result range is: -(n+1)to n-1.

**Example:**

**successful result range is: 0 to 2**

**unsuccessful result range is: -4 to -1**

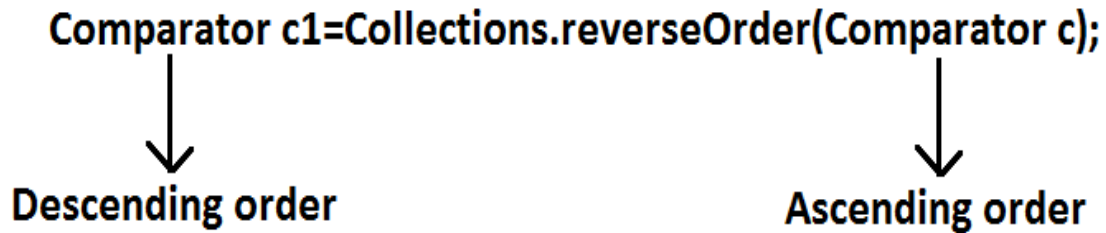
**Total result range is : -4 to 2**

**Reversing the elements of List:**

```
public static void reverse(List l);
```

**reverse() vs reverseOrder() method**

- We can use `reverse()` method to reverse the elements of List.
- Where as we can use `reverseOrder()` method to get reversed `Comparator`.



**Program:** To reverse elements of list.

```
import java.util.*;
class CollectionsReverseDemo
{
    public static void main(String[] args)
    {
        ArrayList l=new ArrayList();
        l.add(15);
        l.add(0);
        l.add(20);
        l.add(10);
        l.add(5);
        System.out.println(l);//[15, 0, 20, 10, 5]
        Collections.reverse(l);
        System.out.println(l);//[5, 10, 20, 0, 15]
    }
}
```

### **Arrays class:**

Arrays class defines several utility methods for arrays.

#### Sorting the elements of array:

- **public static void sort(primitive[] p);**//any primitive data type we can give  
To sort the elements of primitive array according to default natural sorting order.
- **public static void sort(object[] o);**  
To sort the elements of object[] array according to default natural sorting order.  
In this case objects should be homogeneous and Comparable.
- **public static void sort(object[] o,Comparator c);**  
To sort the elements of object[] array according to customized sorting order.

**Note:** We can sort object[] array either by default natural sorting order (or) customized sorting order but we can sort primitive arrays only by default natural sorting order.

**Program:** To sort elements of array.

```
import java.util.*;
class ArraySortDemo
{
    public static void main(String[] args)
```

```

        {
            int[] a={10,5,20,11,6};
            System.out.println("primitive array before
sorting");
            for(int a1:a)
            {
                System.out.println(a1);
            }
            Arrays.sort(a);
            System.out.println("primitive array after
sorting");
            for(int a1: a)
            {
                System.out.println(a1);
            }
            String[] s={"A","Z","B"};
            System.out.println("Object array before
sorting");
            for(String s1: s)
            {
                System.out.println(s1);
            }
            Arrays.sort(s);
            System.out.println("Object array after sorting");
            for(String s1:s)
            {
                System.out.println(s1);
            }
            Arrays.sort(s,new MyComparator());
            System.out.println("Object array after sorting by
Comparator:");
            for(String s1: s)
            {
                System.out.println(s1);
            }
        }
    }
}
class MyComparator implements Comparator
{
    public int compare(Object obj1,Object obj2)
    {
        String s1=obj1.toString();
        String s2=obj2.toString();
        return s2.compareTo(s1);
    }
}

```

Searching the elements of array:

Arrays class defines the following methods to search elements of array.

1. `public static int binarySearch(primitive[] p, primitive key);`
2. `public static int binarySearch(Object[] p, object key);`
3. `public static int binarySearch(Object[] p, Object key, Comparator c);`

All rules of Arrays class `binarySearch()` method are exactly same as Collections class `binarySearch()` method.

**Program:** To search elements of array.

```
import java.util.*;
class ArraysSearchDemo
{
    public static void main(String[] args)
    {
        int[] a={10,5,20,11,6};
        Arrays.sort(a);
        System.out.println(Arrays.binarySearch(a,6));//1
        System.out.println(Arrays.binarySearch(a,14));//-
5
        String[] s={"A","Z","B"};
        Arrays.sort(s);

        System.out.println(Arrays.binarySearch(s,"Z"));//2

        System.out.println(Arrays.binarySearch(s,"S"));//-3
        Arrays.sort(s,new MyComparator());
        System.out.println(Arrays.binarySearch(s,"Z",new
MyComparator()));//0
        System.out.println(Arrays.binarySearch(s,"S",new
MyComparator()));//-2

        System.out.println(Arrays.binarySearch(s,"N"));//-
4(unpredictable result)
    }
}
```

### Converting array to List:

Arrays class defines the following method to view array as List.

`public static List asList (Object[] o);`

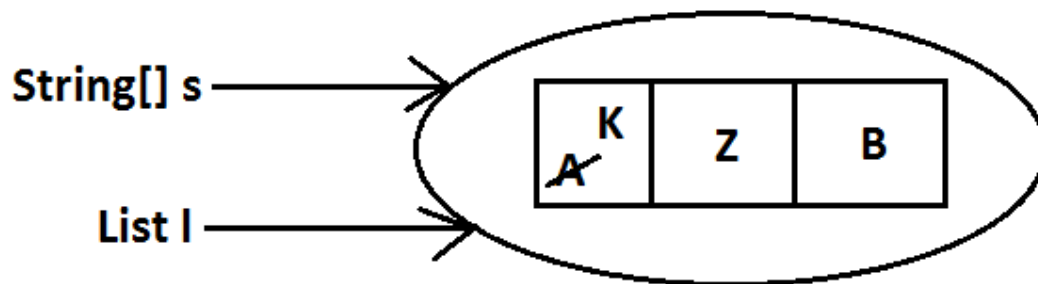
- Strictly speaking we are not creating an independent List object just we are viewing array in List form.
- By using List reference if we are performing any change automatically these changes will be reflected to array reference similarly by using array reference if we are performing any change automatically these changes will be reflected to the List reference.

- By using List reference if we are trying to perform any operation which varies the size then we will get runtime exception saying `UnsupportedOperationException`.
- By using List reference if we are trying to insert heterogeneous objects we will get runtime exception saying `ArrayStoreException`.

**Program:** To view array in List form.

```
import java.util.*;
class ArraysAsListDemo
{
    public static void main(String[] args)
    {
        String[] s={"A","Z","B"};
        List l=Arrays.asList(s);
        System.out.println(l);//[A, Z, B]
        s[0]="K";
        System.out.println(l);//[K, Z, B]
        l.set(1,"L");
        for(String s1: s)
            System.out.println(s1);//K,L,B
        //l.add("ashok");//UnsupportedOperationException
        //l.remove(2);//UnsupportedOperationException
        //l.set(1,new Integer(10));//ArrayStoreException
    }
}
```

**Diagram:**



# **CORE JAVA**

## **With**

# **SCJP / OCJP**

### **Study Material**

### **Chapter 15: Generics**



**DURGA M.Tech**

**(Sun certified & Realtime Expert)**

**Ex. IBM Employee**

**Trained Lakhs of Students  
for last 14 years across INDIA**

**India's No.1 Software Training Institute**

# **DURGASOFT**

**www.durgasoft.com Ph: 9246212143 ,8096969696**

# GENERICS

## Agenda:

1. Introduction
2. Type-Safety
3. Type-Casting
4. Generic Classes
5. Bounded Types
6. Generic methods and wild card character(?)
7. Communication with non generic code
8. Conclusions

## Introduction:

**Deff :** The main objective of Generics is to provide Type-Safety and to resolve Type-Casting problems.

### Case 1: Type-Safety

- Arrays are always type safe that is we can give the guarantee for the type of elements present inside array.
- For example if our programming requirement is to hold String type of objects it is recommended to use String array. In the case of string array we can add only string type of objects by mistake if we are trying to add any other type we will get compile time error.

### Example:

```
String[] s=new String[600];
```

```
s[0]="durga";
```

```
s[1]="pavan";
```

```
s[2]=new Integer(10);(invalid)
```

C.E

```
E:\SCJP>javac Test.java  
Test.java:8: incompatible types  
found   : java.lang.Integer  
required: java.lang.String
```



- That is we can always provide guarantee for the type of elements present inside array and hence arrays are safe to use with respect to type that is arrays are type safe.
- But collections are not type safe that is we can't provide any guarantee for the type of elements present inside collection.
- For example if our programming requirement is to hold only string type of objects it is never recommended to go for ArrayList.
- By mistake if we are trying to add any other type we won't get any compile time error but the program may fail at runtime.

Example:

```
ArrayList l=new ArrayList();  
l.add("vijaya");  
l.add("bhaskara");  
l.add(new Integer(10));  
.   
.  
.
```

```
String name1=(String)l.get(0);
```

```
String name2=(String)l.get(1);
```

```
String name3=(String)l.get(2);(invalid)
```

R.E

Exception in thread "main" java.lang.ClassCastException:  
java.lang.Integer cannot be cast to java.lang.String

Hence we can't provide guarantee for the type of elements present inside collections that is collections are not safe to use with respect to type.

Case 2: Type-Casting

In the case of array at the time of retrieval it is not required to perform any type casting.

Example:

```
String[] s=new String[600];
s[0]="vijaya";
s[1]="bhaskara";
```

```
-----
-----
-----
```

```
String name1=s[0];
```

→ At the time of retrieval  
type casting is not required.

But in the case of collection at the time of retrieval compulsory we should perform type casting otherwise we will get compile time error.

Example:

```
ArrayList l=new ArrayList();
l.add("vijaya");
l.add("bhaskara");
```

```
String name1=l.get(0);
```

Test.java:9: incompatible types  
found : java.lang.Object  
required: java.lang.String  
String name1=l.get(0);

→ At the time of retrieval type  
casting is Mandatory.

```
String name1=(String)l.get(0);
```

- That is in collections type casting is bigger headache.

- To overcome the above problems of collections(type-safety, type casting)sun people introduced generics concept in 1.5v hence the main objectives of generics are:
  - To provide type safety to the collections.
  - To resolve type casting problems.
- To hold only string type of objects we can create a generic version of ArrayList as follows.

Example:

```
ArrayList<String> l=new ArrayList<String>();
l.add("vijaya");
l.add(10);(invalid) → C.E → Test.java:8: cannot find symbol
symbol : method add(int)
location: class java.util.ArrayList<java.lang.String>
l.add(10);
```

- For this ArrayList we can add only string type of objects by mistake if we are trying to add any other type we will get compile time error that is through generics we are getting type safety.
- At the time of retrieval it is not required to perform any type casting we can assign elements directly to string type variables.

Example:

```
ArrayList<String> l=new ArrayList<String>();
l.add("A");
.
.
.
String name1= l.get(0);
→ Type casting is not required
```

- That is through generic syntax we can resolve type casting problems.

#### Conclusion1:

- Polymorphism concept is applicable only for the base type but not for parameter type[usage of parent reference to hold child object is called polymorphism].

#### Example:

parameter type  
 ArrayList<String> l1=new ArrayList<String>();  
 base Type ———  
 List<String> l2=new ArrayList<String>();  
 Collection<String> l3=new ArrayList<String>();  
 ArrayList<Object> l4=new ArrayList<String>(); C.E →

Test.java:9: incompatible types  
 found : java.util.ArrayList<java.lang.String>  
 required: java.util.ArrayList<java.lang.Object>  
 ArrayList<Object> l4=new ArrayList<String>();

#### Conclusion2:

Collections concept applicable only for objects , Hence for the parameter type we can use any class or interface name but not primitive value(type).Otherwise we will get compile time error.

#### Example:

ArrayList<int> l=new ArrayList<int>();

Test.java:6: unexpected type  
 found : int  
 required: reference  
 ArrayList<int> l=new ArrayList<int>();

**Generic classes:**

Until 1.4v a non-generic version of ArrayList class is declared as follows.

**Example:**

```
class ArrayList
{
    add(Object o);
    Object get(int index);
}
```

- add() method can take object as the argument and hence we can add any type of object to the ArrayList. Due to this we are not getting type safety.
- The return type of get() method is object hence at the time of retrieval compulsory we should perform type casting.

But in 1.5v a generic version of ArrayList class is declared as follows.

**Example:**

Type parameter

```
class ArrayList<T>
{
    add(T t);
    T get(int index);
}
```

- Based on our requirement T will be replaced with our provided type.
- For Example to hold only string type of objects we can create ArrayList object as follows.

**Example:**

```
ArrayList<String> l=new ArrayList<String>();
```

- For this requirement compiler considered ArrayList class is

Example:

```
class ArrayList<String>
{
    add(String s);
    String get(int index);
}
```

add() method can take only string type as argument hence we can add only string type of objects to the List.

By mistake if we are trying to add any other type we will get compile time error.

Example:

```
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        ArrayList<String> l=new ArrayList<String>();
        l.add("A");
        l.add(10);
    }
}
```

Test.java:8: cannot find symbol  
symbol : method add(int)  
location: class java.util.ArrayList<java.lang.String>  
l.add(10);

- Hence through generics we are getting type safety.
- At the time of retrieval it is not required to perform any type casting we can assign its values directly to string variables.

Example:

```

import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        ArrayList<String> l=new ArrayList<String>();
        l.add("A");
        l.add("10");
        String name1=l.get(0);
    }
}

```

→ Type casting is not required

In Generics we are associating a type-parameter to the class, such type of parameterised classes are nothing but Generic classes.

Generic class : class with type-parameter

Based on our requirement we can create our own generic classes also.

Example:

```

class Account<T>
{
}
Account<Gold> g1=new Account<Gold>();
Account<Silver> g2=new Account<Silver>();

```

Example:

```

class Gen<T>
{
    T obj;
    Gen(T obj)
    {
        this.obj=obj;
    }
    public void show()
    {
        System.out.println("The type of object is
        :"+obj.getClass().getName());
    }
    public T getObject()
    {
        return obj;
    }
}

```

```
}  
  
class GenericsDemo  
{  
    public static void main(String[] args)  
    {  
        Gen<Integer> g1=new Gen<Integer>(10);  
        g1.show();  
        System.out.println(g1.getObject());  
  
        Gen<String> g2=new Gen<String>("Akshay");  
        g2.show();  
        System.out.println(g2.getObject());  
  
        Gen<Double> g3=new Gen<Double>(10.5);  
        g3.show();  
        System.out.println(g3.getObject());  
    }  
}
```

Output:

The type of object is: java.lang.Integer  
10

The type of object is: java.lang. String  
Akshay

The type of object is: java.lang. Double  
10.5

### **Bounded types:**

We can bound the type parameter for a particular range by using extends keyword such types are called bounded types.

#### Example 1:

```
class Test<T>  
{  
}  
Test <Integer> t1=new Test < Integer>();  
Test <String> t2=new Test < String>();
```

- Here as the type parameter we can pass any type and there are no restrictions hence it is unbounded type.

#### Example 2:

```
class Test<T extends X>  
{  
}
```

- If x is a class then as the type parameter we can pass either x or its child classes.
- If x is an interface then as the type parameter we can pass either x or its implementation classes.



**Example 1:**

```
class Test<T extends Number>
{
class Test1
{
    public static void main(String[] args)
    {
        Test<Integer> t1=new Test<Integer>();
        Test<String> t2=new Test<String>();
    }
}
```

→ type parameter java.lang.String is not within its bound  
Test<String> t2=new Test<String>();

**Example 2:**

```
class Test<T extends Runnable>
{
class Test1
{
    public static void main(String[] args)
    {
        Test<Thread> t1=new Test<Thread>();
        Test<String> t2=new Test<String>();
    }
}
```

→ C.E Test1.java:8: type parameter java.lang.String is not within its bound  
Test<String> t2=new Test<String>();

- We can't define bounded types by using implements and super keyword.

Example:

|                                                                    |                                                             |
|--------------------------------------------------------------------|-------------------------------------------------------------|
| <pre>class Test&lt;T implements Runnable&gt; {     (invalid)</pre> | <pre>class Test&lt;T super String&gt; {     (invalid)</pre> |
|--------------------------------------------------------------------|-------------------------------------------------------------|

- But implements keyword purpose we can replace with extends keyword.
- As the type parameter we can use any valid java identifier but it convention to use T always.

Example:

|                                  |                                        |
|----------------------------------|----------------------------------------|
| <pre>class Test&lt;X&gt; {</pre> | <pre>class Test&lt;bhaskar&gt; {</pre> |
|----------------------------------|----------------------------------------|

We can pass any no of type parameters need not be one.

Example:

```
class HashMap<K,V>
{
```

key type

value type

```
HashMap<Integer,String> h=new HashMap<Integer,String>();
```

We can define bounded types even in combination also.

Example 1:

```
class Test <T extends Number&Runnable> {}(valid)
```

As the type parameter we can pass any type which extends Number class and implements Runnable interface.

**Example 2:**

```
class Test<T extends Number&Runnable&Comparable> {}(valid)
```

**Example 3:**

```
class Test<T extends Number&String> {}(invalid)
```

We can't extend more than one class at a time.

**Example 4:**

```
class Test<T extends Runnable&Comparable> {}(valid)
```

**Example 5:**

```
class Test<T extends Runnable&Number> {}(invalid)
```

We have to take 1st class followed by interface.

**Generic methods and wild-card character (?) :**

```
methodOne(ArrayList<String> l):
```

This method is applicable for ArrayList of only String type.

**Example:**

```
l.add("A");  
l.add(null);  
l.add(10); //(invalid)
```

Within the method we can add only String type of objects and null to the List.

```
methodOne(ArrayList<?> l):
```

We can use this method for ArrayList of any type but within the method we can't add anything to the List except null.

**Example:**

```
l.add(null); //(valid)  
l.add("A"); //(invalid)  
l.add(10); //(invalid)
```

- This method is useful whenever we are performing only read operation.

```
methodOne(ArrayList<? Extends x> l):
```

- If x is a class then this method is applicable for ArrayList of either x type or its child classes.
- If x is an interface then this method is applicable for ArrayList of either x type or its implementation classes.
- In this case also within the method we can't add anything to the List except null.

**methodOne(ArrayList<? super x> l):**

- If x is a class then this method is applicable for ArrayList of either x type or its super classes.
- If x is an interface then this method is applicable for ArrayList of either x type or super classes of implementation class of x.
- But within the method we can add x type objects and null to the List.

**Which of the following declarations are allowed?**

1. ArrayList<String> l1=new ArrayList<String>(); //(valid)
2. ArrayList<?> l2=new ArrayList<String>(); //(valid)
3. ArrayList<?> l3=new ArrayList<Integer>(); //(valid)
4. ArrayList<? extends Number> l4=new ArrayList<Integer>(); //(valid)
5. ArrayList<? extends Number> l5=new ArrayList<String>(); (invalid)
6. Output:
7. Compile time error.
8. Test.java:10: incompatible types
9. Found : java.util.ArrayList<java.lang.String>
10. Required: java.util.ArrayList<? extends java.lang.Number>
11. ArrayList<? extends Number> l5=new ArrayList<String>();
12. ArrayList<?> l6=new ArrayList<? extends Number>();
13. Output:
14. Compile time error
15. Test.java:11: unexpected type
16. found : ? extends java.lang.Number
17. required: class or interface without bounds
18. ArrayList<?> l6=new ArrayList<? extends Number>();
19. ArrayList<?> l7=new ArrayList<?>();
20. Output:
21. Test.java:12: unexpected type
22. Found :?
23. Required: class or interface without bounds
24. ArrayList<?> l7=new ArrayList<?>();

- We can declare the type parameter either at class level or method level.

**Declaring type parameter at class level:**

```
class Test<T>
{
    We can use anywhere this 'T'.
}
```

**Declaring type parameter at method level:**

We have to declare just before return type.

**Example:**

```
public <T> void methodOne1(T t){} //valid
public <T extends Number> void methodOne2(T t){} //valid
```

```
public <T extends Number&Comparable> void methodOne3(T t){} //valid
public <T extends Number&Comparable&Runnable> void methodOne4(T
t){} //valid
public <T extends Number&Thread> void methodOne(T t){} //invalid
Output:
Compile time error.
Test.java:7: interface expected here
```

```
public <T extends Number&Thread> void methodOne(T t){} //valid
public <T extends Runnable&Number> void methodOne(T t){} //invalid
Output:
Compile time error.
Test.java:8: interface expected here
```

```
public <T extends Number&Runnable> void methodOne(T t){} //valid
```

### Communication with non generic code:

To provide compatibility with old version sun people compromised the concept of generics in very few areas the following is one such area.

#### Example:

```
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        ArrayList<String> l=new ArrayList<String>();
        l.add("A");
        //l.add(10); //C.E:cannot find symbol,method add(int)
        methodOne(l);
        l.add(10.5); //C.E:cannot find symbol,method
add(double)
        System.out.println(l); // [A, 10, 10.5, true]
    }
    public static void methodOne(ArrayList l)
    {
        l.add(10);
        l.add(10.5);
        l.add(true);
    }
}
```

### Conclusions :

- Generics concept is applicable only at compile time, at runtime there is no such type of concept. Hence the following declarations are equal.

```
ArrayList l=new ArrayList<String>();
ArrayList l=new ArrayList<Integer>();           All are equal at
runtime.
ArrayList l=new ArrayList();
```

**Example 1:**

```
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        ArrayList l=new ArrayList<String>();
        l.add(10);
        l.add(10.5);
        l.add(true);
        System.out.println(l);// [10, 10.5, true]
    }
}
```

**Example 2:**

```
import java.util.*;
class Test
{
    public void methodOne(ArrayList<String> l){}
    public void methodOne(ArrayList<Integer> l){}
}
```

Output:

Compile time error.

Test.java:4: name clash:

methodOne(java.util.ArrayList<java.lang.String>) and methodOne(java.util.ArrayList<java.lang.Integer>) have the same erasure

```
public void methodOne(ArrayList<String> l){}
```

- The following 2 declarations are equal.

```
ArrayList<String> l1=new ArrayList();
ArrayList<String> l2=new ArrayList<String>();
```

- For these ArrayList objects we can add only String type of objects.

**Example:**

```
l1.add("A");//valid
l1.add(10); //invalid
```

# **CORE JAVA**

## **With**

# **SCJP / OCJP**

### **Study Material**

#### **Chapter 16: Garbage Collection**



**DURGA M.Tech**

**(Sun certified & Realtime Expert)**

**Ex. IBM Employee**

**Trained Lakhs of Students  
for last 14 years across INDIA**

**India's No.1 Software Training Institute**

# **DURGASOFT**

**www.durgasoft.com Ph: 9246212143 ,8096969696**

## Garbage Collection

1. Introduction:
  2. The way to make an object eligible for GC
    - i. Nullifying the reference variable
    - ii. Reassign the reference variable
    - iii. Objects created inside a method
    - iv. Island of Isolation
  3. The methods for requesting JVM to run GC
    - i. By System class
    - ii. By Runtime class
  4. Finalization
    - o Case 1 : Just before destroying any object GC calls finalize() method on the object
    - o Case 2 : We can call finalize() method explicitly
    - o Case 3 : finalize() method can be call either by the programmer or by the GC
    - o Case 4 : On any object GC calls finalize() method only once
- Memory leaks

### Introduction:

- In old languages like C++ programmer is responsible for both creation and destruction of objects. Usually programmer is taking very much care while creating object and neglect destruction of useless objects .Due to his negligence at certain point of time for creation of new object sufficient memory may not be available and entire application may be crashed due to memory problems.
- But in java programmer is responsible only for creation of new object and his not responsible for destruction of objects.
- Sun people provided one assistant which is always running in the background for destruction at useless objects. Due to this assistant the chance of failing java program is very rare because of memory problems.
- This assistant is nothing but garbage collector. Hence the main objective of GC is to destroy useless objects.

### The ways to make an object eligible for GC:

- Even through programmer is not responsible for destruction of objects but it is always a good programming practice to make an object eligible for GC if it is no longer required.
- An object is eligible for GC if and only if it does not have any references.

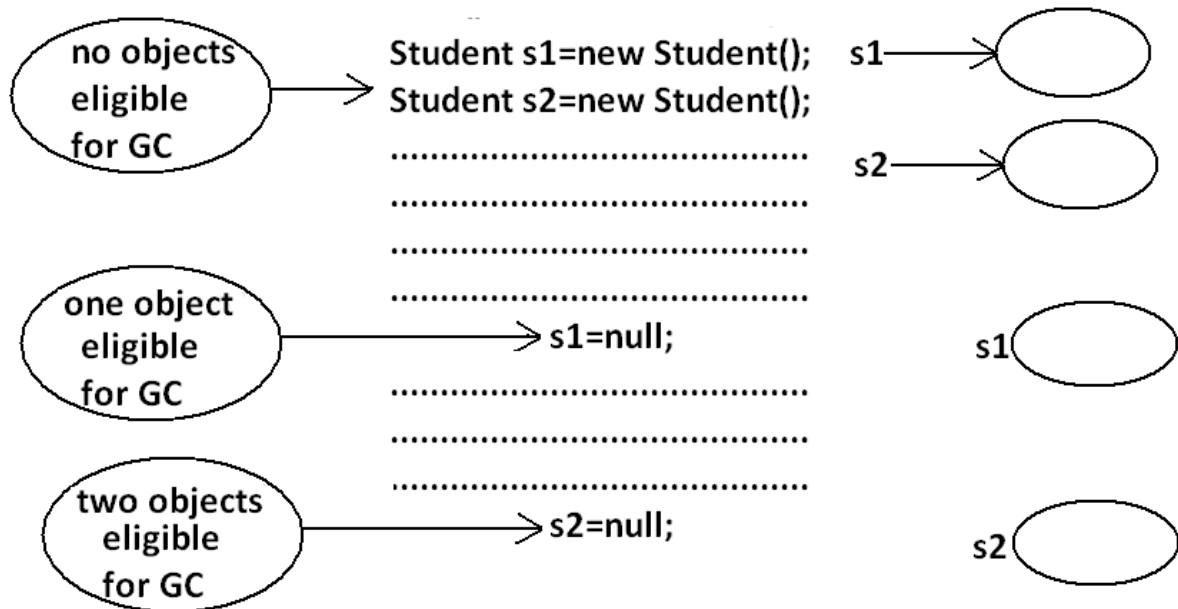
### The following are various possible ways to make an object eligible for GC:



**1.Nullifying the reference variable:**

If an object is no longer required then we can make eligible for GC by assigning "null" to all its reference variables.

**Example:**

**2.Reassign the reference variable:**

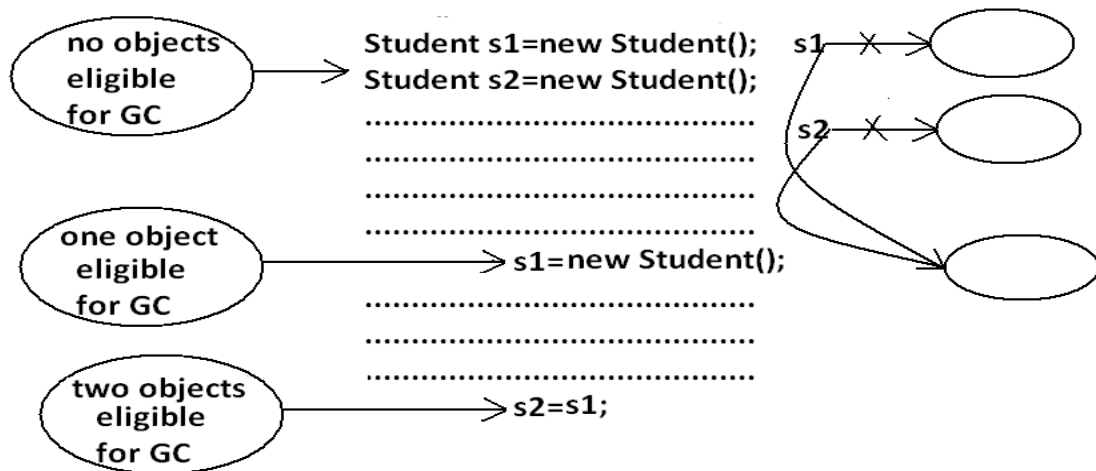
If an object is no longer required then reassign all its reference variables to some other objects then old object is by default eligible for GC.

**Example:**

**3. Objects created inside a method:**

Objects created inside a method are by default eligible for GC once method completes.

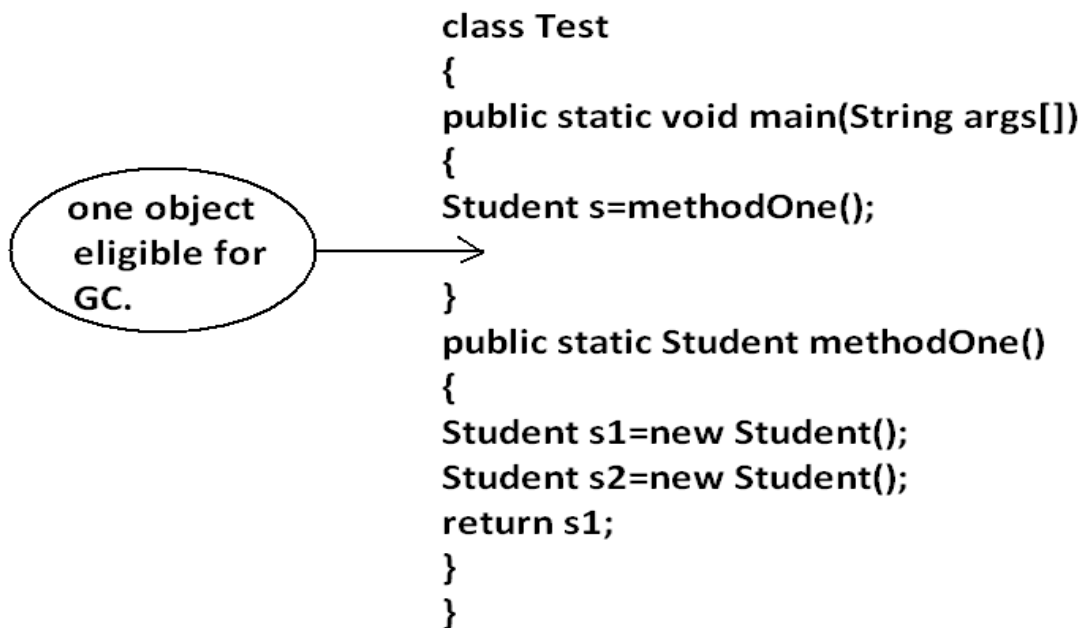
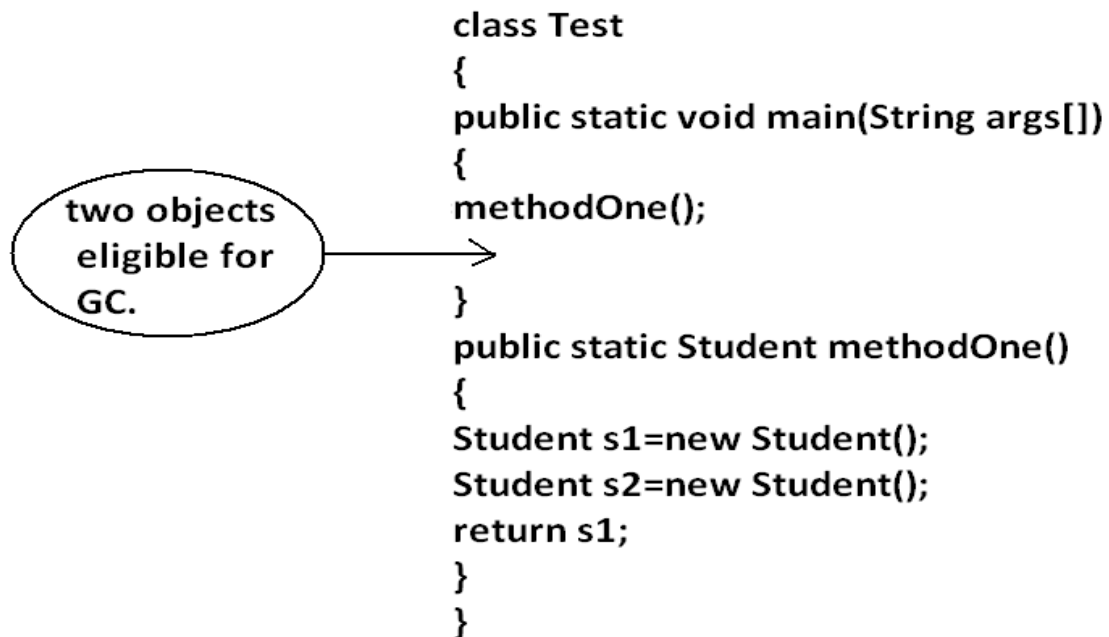
**Example 1:**

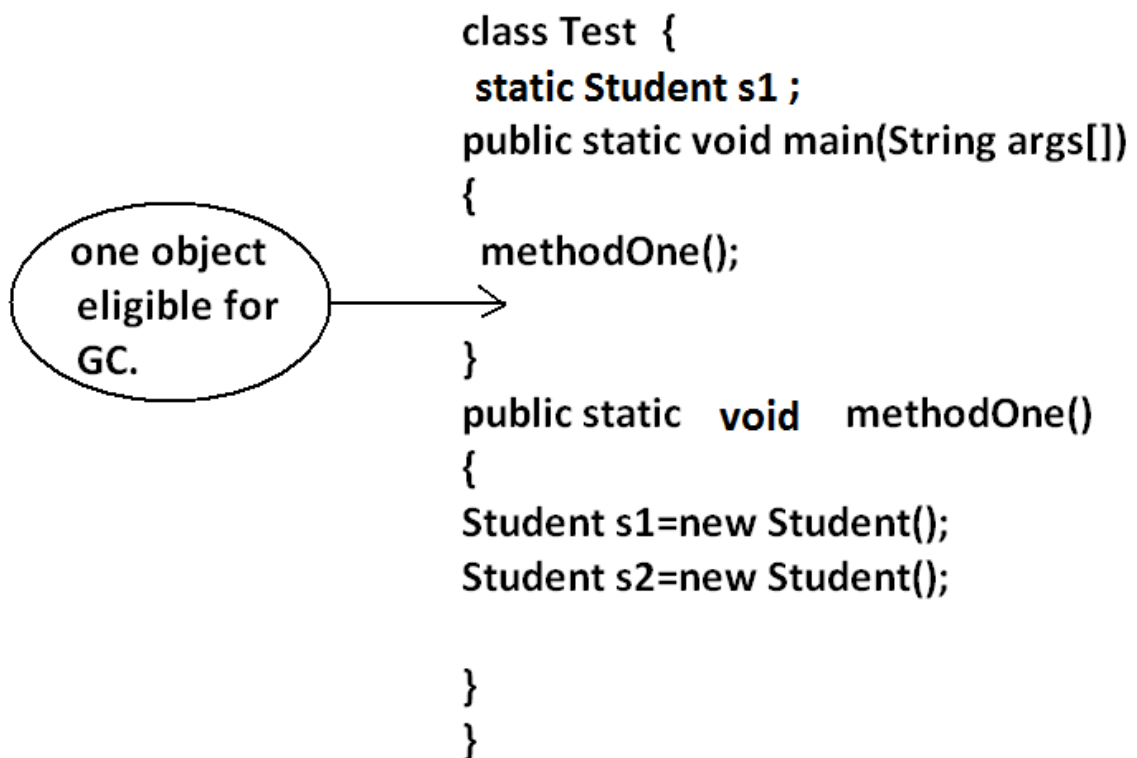


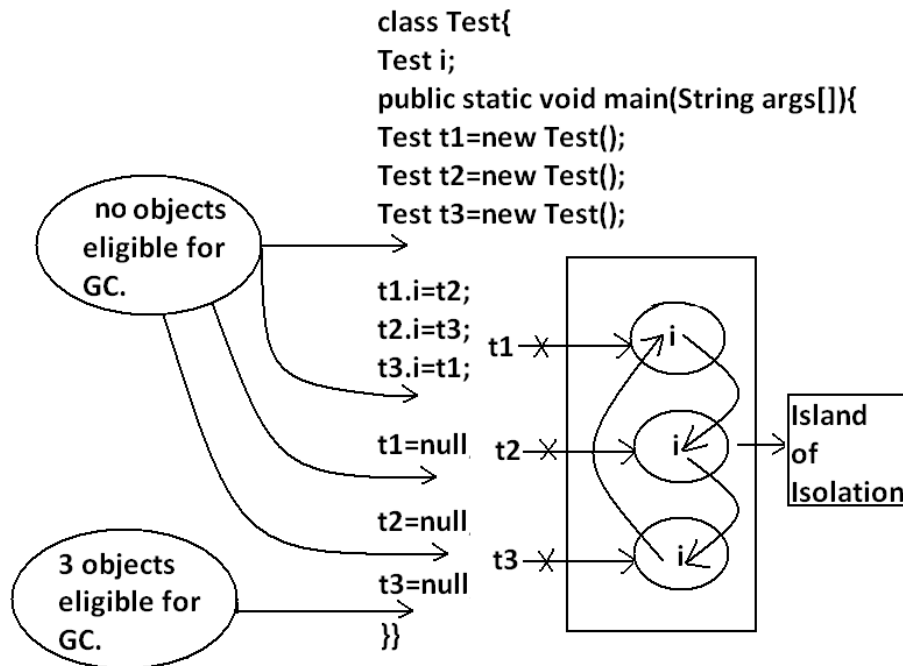
```

class Test
{
    public static void main(String args[]){
        methodOne();
    }
    public static void methodOne()
    {
        Student s1=new Student();
        Student s2=new Student();
        //.....
        //.....
        //.....
    }
}

```

**Example 2:****Example 3:**

Example 4:

**4.Island of Isolation:**

**Note:** if an object doesn't have any reference then it always eligible for GC.

**Note:** Even though object having reference still it is eligible for GC some times.

**Example:**

island of isolation. (Island of Isolation all references are internal references )

**The methods for requesting JVM to run GC:**

- Once we made an object eligible for GC it may not be destroyed immediately by the GC. Whenever jvm runs GC then only object will be destroyed by the GC. But when exactly JVM runs GC we can't expect it is vendor dependent.
- We can request jvm to run garbage collector programmatically, but whether jvm accept our request or not there is no guaranty. But most of the times JVM will accept our request.

**The following are various ways for requesting jvm to run GC:****By System class:**

System class contains a static method GC for this purpose.

Example:

System.gc();

**By Runtime class:**

- A java application can communicate with jvm by using Runtime object.
- Runtime class is a singleton class present in java.lang. Package.
- We can create Runtime object by using factory method getRuntime().

Example:

Runtime r=Runtime.getRuntime();

Once we got Runtime object we can call the following methods on that object.

**freeMemory()**: returns the free memory present in the heap.

**totalMemory()**: returns total memory of the heap.

**gc()**: for requesting jvm to run gc.

**Example:**

```
import java.util.Date;
class RuntimeDemo
{
    public static void main(String args[]){
        Runtime r=Runtime.getRuntime();
        System.out.println("total memory of the heap :"+r.totalMemory());
        System.out.println("free memory of the heap :"+r.freeMemory());
        for(int i=0;i<10000;i++)
        {
            Date d=new Date();
            d=null;
        }
        System.out.println("free memory of the heap :"+r.freeMemory());
        r.gc();
        System.out.println("free memory of the heap :"+r.freeMemory());
    }
}
```

Output:

Total memory of the heap: 5177344

Free memory of the heap: 4994920

Free memory of the heap: 4743408

Free memory of the heap: 5049776

Note : Runtime class is a singleton class so not create the object to use constructor.

Which of the following are valid ways for requesting jvm to run GC ?

System.gc(); (valid)

Runtime.gc(); (invalid)

(new Runtime).gc(); (invalid)  
Runtime.getRuntime().gc(); (valid)

**Note:** gc() method present in System class is static, where as it is instance method in Runtime class.

**Note:** Over Runtime class gc() method, System class gc() method is recommended to use.

**Note:** in java it is not possible to find size of an object and address of an object.

### Finalization:

- Just before destroying any object gc always calls finalize() method to perform cleanup activities.
- If the corresponding class contains finalize() method then it will be executed otherwise Object class finalize() method will be executed.

which is declared as follows.

protected void finalize() throws Throwable

#### Case 1:

Just before destroying any object GC calls finalize() method on the object which is eligible for GC then the corresponding class finalize() method will be executed.

For Example if String object is eligible for GC then String class finalize() method is executed but not Test class finalize() method.

#### Example:

```
class Test
{
    public static void main(String args[]){
        String s=new String("bhaskar");
        Test t=new Test();
        s=null;
        System.gc();
        System.out.println("End of main.");
    }
    public void finalize(){
        System.out.println("finalize() method is executed");
    }
}
```

Output:

End of main.

In the above program String class finalize() method got executed. Which has empty implementation.

If we replace String object with Test object then Test class finalize() method will be executed.

The following program is an Example of this.

**Example:**

```
class Test
{
    public static void main(String args[]){
        String s=new String("bhaskar");
        Test t=new Test();
        t=null;
        System.gc();
        System.out.println("End of main.");
    }
    public void finalize(){
        System.out.println("finalize() method is executed");
    }
}
```

**Output:**

```
finalize() method is executed
End of main
```

**Case 2:**

We can call finalize() method explicitly then it will be executed just like a normal method call and object won't be destroyed. But before destroying any object GC always calls finalize() method.

**Example:**

```
class Test
{
    public static void main(String args[]){
        Test t=new Test();
        t.finalize();
        t.finalize();
        t=null;
        System.gc();
        System.out.println("End of main.");
    }
    public void finalize(){
        System.out.println("finalize() method called");
    }
}
```

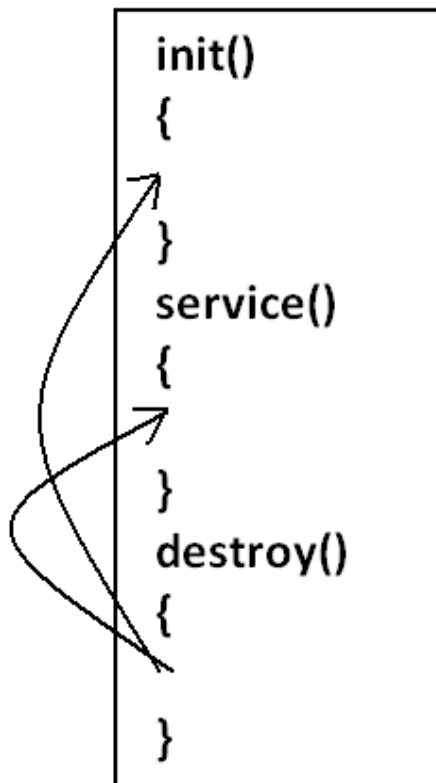
**Output:**

```
finalize() method called.
finalize() method called.
finalize() method called.
End of main.
```

In the above program finalize() method got executed 3 times in that 2 times explicitly by the programmer and one time by the gc.

**Note:** In Servlets we can call destroy() method explicitly from init() and service() methods. Then it will be executed just like a normal method call and Servlet object won't be destroyed.



Diagram:Case 3:

**finalize()** method can be call either by the programmer or by the GC .

If the programmer calls explicitly **finalize()** method and while executing the **finalize()** method if an exception raised and uncaught then the program will be terminated abnormally.

If GC calls **finalize()** method and while executing the **finalize()** method if an exception raised and uncaught then JVM simply ignores that exception and the program will be terminated normally.

Example:

```
class Test
{
    public static void main(String args[]){
        Test t=new Test();
        //t.finalize();-----line(1)
```

```

t=null;
System.gc();
System.out.println("End of main.");
}
public void finalize(){
System.out.println("finalize() method called");
System.out.println(10/0);
}

```

If we are not comment line1 then programmer calling finalize() method explicitly and while executing the finalize() method ArithmeticException raised which is uncaught hence the program terminated abnormally.

If we are comment line1 then GC calls finalize() method and JVM ignores ArithmeticException and program will be terminated normally.

#### Which of the following is true?

While executing finalize() method JVM ignores every exception(Invalid).

While executing finalize() method JVM ignores only uncaught exception(valid).

#### Case 4:

On any object GC calls finalize() method only once.

#### Example:

```

class FinalizeDemo
{
static FinalizeDemo s;
public static void main(String args[])throws Exception{
FinalizeDemo f=new FinalizeDemo();
System.out.println(f.hashCode());
f=null;
System.gc();
Thread.sleep(5000);
System.out.println(s.hashCode());
s=null;
System.gc();
Thread.sleep(5000);
System.out.println("end of main method");
}
public void finalize()
{
System.out.println("finalize method called");
s=this;
}
}

```

Output:

```

D:\Enum>java FinalizeDemo
4072869
finalize method called
4072869
End of main method

```

#### Note:

The behavior of the GC is vendor dependent and varied from JVM to JVM hence we can't expect exact answer for the following.

1. What is the algorithm followed by GC.
2. Exactly at what time JVM runs GC.
3. In which order GC identifies the eligible objects.
4. In which order GC destroys the object etc.
5. Whether GC destroys all eligible objects or not.

When ever the program runs with low memory then the JVM runs GC, but we can't except exactly at what time.

Most of the GC's followed mark & sweep algorithm , but it doesn't mean every GC follows the same algorithm.

### Memory leaks:

- An object which is not using in our application and it is not eligible for GC such type of objects are called "memory leaks".
- In the case of memory leaks GC also can't do anything the application will be crashed due to memory problems.
- In our program if memory leaks present then certain point we will get OutOfMemoryException. Hence if an object is no longer required then it's highly recommended to make that object eligible for GC.
- By using monitoring tools we can identify memory leaks.

Example:

HPJ meter  
HP ovo  
IBM Tivoli  
J Probe  
Patrol and etc

These are monitoring tools.  
(or memory management tools)

# CORE JAVA

## With

# SCJP / OCJP

### Study Material

#### Chapter 17: ENUM



**DURGA M.Tech**

(Sun certified & Realtime Expert)

**Ex. IBM Employee**

Trained Lakhs of Students  
for last 14 years across INDIA

**India's No.1 Software Training Institute**

# DURGASOFT

**www.durgasoft.com Ph: 9246212143 ,8096969696**

# ENUM

## Agenda

1. Introduction
2. Internal implementation of enum
3. Declaration and usage of enum
4. Enum vs switch statement
5. enum outside the class allowed modifiers
6. enum inside a class allowed modifiers
7. Enum vs inheritance
8. Java.lang.Enum class
  - o values() method
  - o ordinal() method
9. Speciality of java enum
10. Enum vs constructor
11. enum Vs Enum Vs Enumeration

## **Introduction :**

We can use enum to define a group of named constants.

### Example 1:

```
enum Month
{
    JAN, FEB, MAR,    ... DEC;    //; -->optional
}
```

### Example 2:

```
enum Beer
{
    KF, KO, RC, FO;
}
```

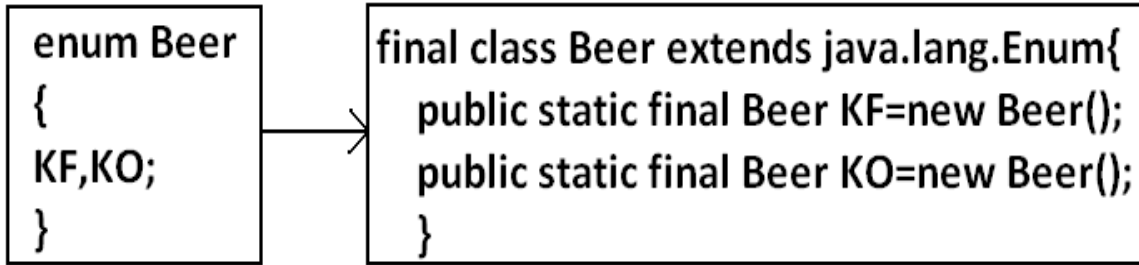
- Enum concept introduced in 1.5 versions.
- When compared with old languages enum java's enum is more powerful.
- By using enum we can define our own data types which are also come enumerated data types.

### Internal implementation of enum:

Internally enum's are implemented by using class concept.

Every enum constant is a reference variable to that enum type object.

Every enum constant is implicitly public static final always.

Example 3:Diagram:Declaration and usage of enum:Example 4:

```
enum Beer
{
    KF,KO,RC,F0;//here semicolon is optional.
}
class Test
{
    public static void main(String args[]){
        Beer b1=Beer.KF;
        System.out.println(b1);
    }
}
```

Output:

```
D:\Enum>java Test
```

```
KF
```

Note:

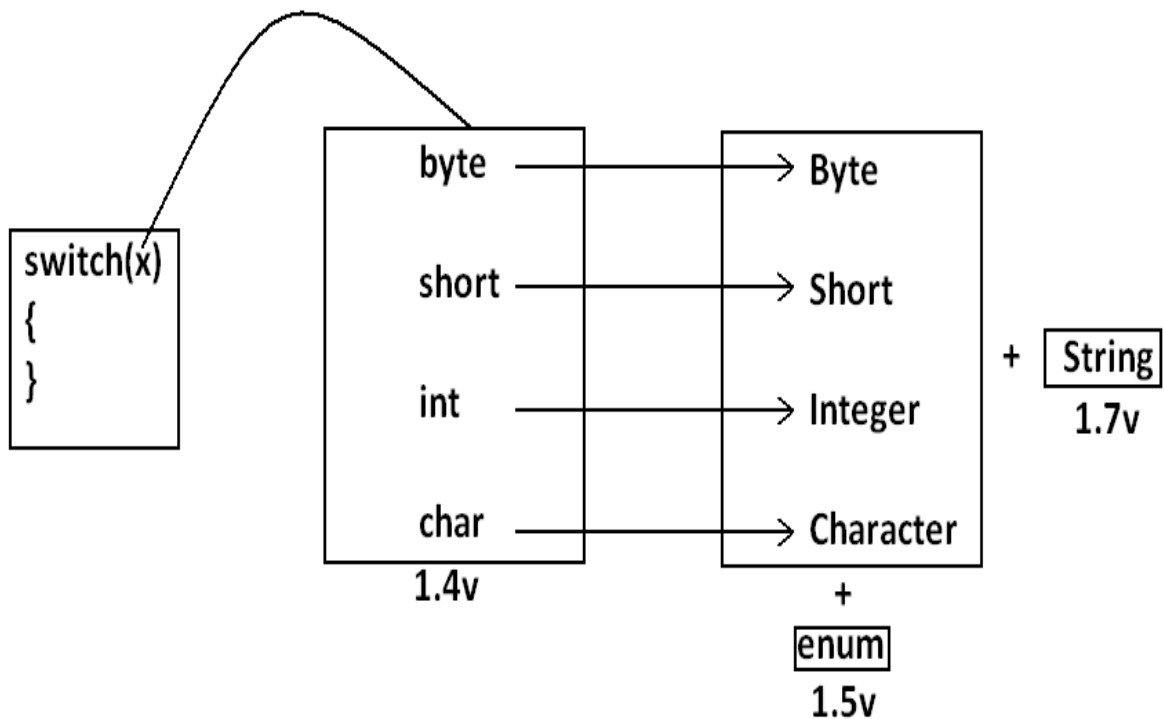
- Every enum constant internally static hence we can access by using "enum name".
- Internally inside every enum toString() method is implemented to return name of the constant.

Enum vs switch statement:

Until 1.4 versions the allowed argument types for the switch statement are byte, short, char int.

But from 1.5 version onwards in addition to this the corresponding wrapper classes and enum type also allowed.

That is from 1.5 version onwards we can use enum type as argument to switch statement.

Diagram:Example:

```
enum Beer
{
    KF, KO, RC, FO;
}
class Test{
    public static void main(String args[]){
        Beer b1=Beer.RC;
        switch(b1){
            case KF:
                System.out.println("it is childrens brand");
```

```

        break;
    case K0:
        System.out.println("it is too lite");
        break;
    case RC:
        System.out.println("it is too hot");
        break;

    case F0:
        System.out.println("buy one get one");
        break;
    default:
        System.out.println("other brands are not good");
    }
}
}
Output:
D:\Enum>java Test
It is too hot

```

If we are passing enum type as argument to switch statement then every case label should be a valid enum constant otherwise we will get compile time error.

#### Example:

```

enum Beer
{
    KF, K0, RC, F0;
}
class Test{
    public static void main(String args[]){
        Beer b1=Beer.RC;
        switch(b1){
            case KF:
            case RC:
            case KALYANI:
        }
    }
}
Output:
Compile time error.
D:\Enum>javac Test.java
Test.java:11: unqualified enumeration constant name required
case KALYANI:
We can declare enum either outside the class or within the class but not inside a method.

```

#### If we declare enum outside the class the allowed modifiers are :

```

public
default
strictfp.

```



If we declare enum inside a class then the allowed modifiers are :

public                      private  
default +                protected  
strictfp                static

Example:

|         |         |         |  |
|---------|---------|---------|--|
| enum X  | class X | class X |  |
| { }     | {       | {       |  |
| class Y | enum Y  | enum X  |  |
| { }     | {       | { }     |  |
| (valid) | }       | }       |  |
|         | (valid) | }       |  |

output:  
compile time error.  
D:\Enum>javac X.java  
X.java:4: enum types must not be local  
enum X

Enum vs inheritance:

- Every enum in java is the direct child class of java.lang.Enum class hence it is not possible to extends any other enum.
- Every enum is implicitly final hence we can't create child enum.
- Because of above reasons we can conclude inheritance concept is not applicable for enum's explicitly. Hence we can't apply extends keyword for enum's .
- But enum can implement any no. Of interfaces simultaneously.

Example:

```
enum X
{
enum Y extends X
{

```

(invalid)

```
enum X extends Enum
{

```

(invalid)

```
class X
{
enum Y extends X
{

```

(invalid)

Example:

```
enum X
{
class Y extends X
{

```

output:

compile time error.

D:\Enum&gt;javac Y.java

Y.java:3: cannot inherit from final X

class Y extends X

Y.java:3: enum types are not extensible

class Y extends X

(invalid)

interface X

{

enum Y implements X

{

(valid)

Java.lang.Enum class:

- Every enum in java is the direct child class of java.lang.Enum class. Hence this class acts as base class for all java enums.
- It is abstract class and it is direct child class of "Object class"
- It implements Serializable and Comparable.

**values() method:**

Every enum implicitly contains a static values() method to list all constants of enum.

Example: Beer[] b=Beer.values();

**ordinal() method:**

Within enum the order of constants is important we can specify by its ordinal value. We can find ordinal value(index value) of enum constant by using ordinal() method.

Example: public final int ordinal();

**Example:**

```
enum Beer
{
    KF, KO, RC, FO;
}
class Test{
    public static void main(String args[]){
        Beer[] b=Beer.values();
        for(Beer b1:b)//this is forEach loop.
        {
            System.out.println(b1+"....."+b1.ordinal());
        }
    }
}
```

Output:

```
D:\Enum>java Test
KF.....0
KO.....1
RC.....2
FO.....3
```

**Speciality of java enum:**

When compared with old languages enum, java's enum is more powerful because in addition to constants we can take normal variables, constructors, methods etc which may not possible in old languages.

Inside enum we can declare main method and even we can invoke enum directly from the command prompt.

**Example:**

```
enum Fish{
    GOLD, APOLO, STAR;
    public static void main(String args[]){
        System.out.println("enum main() method called");
    }
}
```

Output:

```
D:\Enum>java Fish
enum main() method called
```

In addition to constants if we are taking any extra members like methods then the list of constants should be in the 1st line and should ends with semicolon.

If we are taking any extra member then enum should contain at least one constant. Any way an empty enum is always valid.

#### Example:

```
enum X{
A,B,C;//here semicolon mandatory.
public void methodOne(){
}
}          (valid)
```

```
enum X{
public void methodOne(){
}
A,B,C;
}          (invalid)
```

```
enum X
{
public void methodOne(){
}
}          (invalid)
```

```
enum X
{
}          (valid)
```

```
enum X
{
;
public void methodOne(){
}
}          (valid)
```

#### **Enum vs constructor:**

Enum can contain constructor. Every enum constant represents an object of that enum class which is static hence all enum constants will be created at the time of class loading automatically and hence constructor will be executed at the time of enum class loading for every enum constants.

#### Example:

```
enum Beer{
KF, KO, RC, FO;
```

```

Beer(){
System.out.println("Constructor called.");
}
}
class Test{
public static void main(String args[]){
Beer b=Beer.KF;    // --->1
System.out.println("hello.");
}}

```

Output:

```

D:\Enum>java Test
Constructor called.
Constructor called.
Constructor called.
Constructor called.
Hello.

```

If we comment line 1 then the output is Hello.

We can't create enum object explicitly and hence we can't invoke constructor directly.

Example:

```

enum Beer{
KF, KO, RC, FO;
Beer(){
System.out.println("constructor called");
}
}
class Test{
public static void main(String args[]){
Beer b=new Beer();
System.out.println(b);
}}

```

Output:

```

Compile time error.
D:\Enum>javac Test.java
Test.java:9: enum types may not be instantiated
Beer b=new Beer();

```

Example:

```

KF==>public static final Beer KF=new Beer();
KF(100)==>public static final Beer KF=new Beer(100);

```

```
enum Beer
{
    KF(100), KO(70), RC(65), Fo(90), KALYANI;
    int price;
    Beer(int price){
        this.price=price;
    }
    Beer()
    {
        this.price=125;
    }
    public int getPrice()
    {
        return price;
    }
}
class Test{
    public static void main(String args[]){
        Beer[] b=Beer.values();
        for(Beer b1:b)
        {
            System.out.println(b1+"....."+b1.getPrice());
        }
    }
}
```

output :

```
KF.....100
KO.....70
RC.....65
FO .....90
KALYANI.....125
```

Inside enum we can take both instance and static methods but it is not possible to take abstract methods.

#### Case 1:

Every enum constant represents an object hence whatever the methods we can apply on the normal objects we can apply the same methods on enum constants also.

Which of the following expressions are valid ?

```
Beer.KF==Beer.RC-----> false
Beer.KF.equals(Beer.RC) -----> false
Beer.KF < Beer.RC-----> invalid
Beer.KF.ordinal() < Beer.RC.ordinal()-----> valid
```

#### Case 2:

##### Example 1:

```
package pack1;
```

```
public enum Fish
{
    STAR, GUPPY;
}
```

Example 2:

```
package pack2;
//import static pack1.Fish.*;
import static pack1.Fish.STAR;
class A
{
    public static void main(String args[]){
        System.out.println(STAR);
    }
}
Import pack1.*; ----->invalid
Import pack1.Fish; ----->invalid
import static pack1.Fish.*; ----->valid
import static pack1.Fish.STAR; ----->valid
```

Example 3:

```
package pack3;
//import pack1.Fish;
import pack1.*;
//import static pack1.Fish.GUPPY;
import static pack1.Fish.*;
class B
{
    public static void main(String args[]){
        Fish f=Fish.STAR;
        System.out.println(GUPPY);
    }
}
```

**Note :**

If we want to use classname directly from outside package we should write normal import , If we want to access static method or static variable without classname directly then static import is required.

Case 3:Example 1:

```
enum Color
{
    BLUE, RED, GREEN;
    public void info()
    {
        System.out.println("Universal color");
    }
}
```

```
class Test
{
    public static void main(String args[]){
        Color[] c=Color.values();
        for(Color c1:c)
        {
            c1.info();
        }
    }
}
Output:
Universal color
Universal color
Universal color
```

Example 2:

```
enum Color
{
    BLUE, RED
    {
        public void info(){
            System.out.println("Dangerous color");
        }
    }, GREEN;
    public void info()
    {
        System.out.println("Universal color");
    }
}
class Test
{
    public static void main(String args[]){
        Color[] c=Color.values();
        for(Color c1:c)
        {
            c1.info();
        }
    }
}
Output:
Universal color
Dangerous color
Universal color
```



## **enum Vs Enum Vs Enumeration :**

**enum** : enum is a keyword which can be used to define a group of named constants.

**Enum** :

It is a class present in java.lang package .

Every enum in java is the direct child class of this class. Hence this Enum class acts as base class for all java enum's .

**Enumeration** :

It is a interface present in java.util package .

We can use Enumeration to get the objects one by one from the Collections.

# **CORE JAVA**

## **With**

# **SCJP / OCJP**

### **Study Material**

**Chapter 18: Internationalization (I18N)**



**DURGA M.Tech**

**(Sun certified & Realtime Expert)**

**Ex. IBM Employee**

**Trained Lakhs of Students  
for last 14 years across INDIA**

**India's No.1 Software Training Institute**

# **DURGASOFT**

**www.durgasoft.com Ph: 9246212143 ,8096969696**

# Internationalization (I18N)

## Agenda

1. Introduction
2. Locale
  - o How to create a Locale object
  - o Important methods of Locale class
3. NumberFormat
  - o Getting NumberFormat object for the default Locale
  - o Getting NumberFormat object for the specific Locale
  - o Requirements
    - Write a program to display java number form into Italy specific form
    - Write a program to print a java number in INDIA, UK, US and ITALY currency formats
  - o Setting Maximum, Minimum, Fraction and Integer digits
4. DateFormat
  - o Getting DateFormat object for default Locale
  - o Getting DateFormat object for specific Locale
  - o Requirements
    - Write a program to represent current system date in all possible styles of us format
    - Write a program to represent current system date in UK, US and ITALY styles
  - o Getting DateFormat object to get both date and time

## Introduction

The process of designing a web application such that it supports various countries, various languages without performing any changes in the application is called Internationalization.

If the request is coming from India then the response should be in India specific form , and if the request is from US then the response should be in US specific form.

We can implement Internationalization by using the following classes.

*They are:*

1. Locale
2. NumberFormat
3. DateFormat

## 1. Locale:

- A Locale object can be used to represent a geographic (country) location (or) language.
- Locale class present in java.util package.
- It is a final class and direct child class of Object and , implements Cloneable, and Serializable Interfaces.

## How to create a Locale object:

We can create a Locale object by using the following constructors of Locale class.

```
Locale l=new Locale(String language);  
Locale l=new Locale(String language,String country);
```

Locale class already defines some predefined Locale constants. We can use these constants directly.

Example:

```
Locale. UK  
Locale. US  
Locale. ITALY  
Locale. CHINA
```

## Important methods of Locale class:

1. public static Locale getDefault()
2. public static void setDefault(Locale l)
3. public String getLanguage()
4. public String getDisplayLanguage(Locale l)
5. public String getCountry()
6. public String getDisplayCountry(Locale l)
7. public static String[] getISOLanguages()
8. public static String[] getISOCountries()
9. public static Locale[] getAvailableLocales()

Example for Locale:

```
import java.util.*;  
class LocaleDemo{  
public static void main(String args[]){  
Locale l1=Locale.getDefault();  
//System.out.println(l1.getCountry()+"....."+l1.getLanguage())  
;  
//System.out.println(l1.getDisplayCountry()+"....."+l1.getDisplayLanguage());  
Locale l2=new Locale("pa","IN");  
Locale.setDefault(l2);  
String[] s3=Locale.getISOLanguages();  
for(String s4:s3)
```

```

{
//System.out.print("ISO language is  :");
//System.out.println(s4);
}
String[] s4=Locale.getISOCountries();
for(String s5:s4)
{
System.out.print("ISO Country is:");
System.out.println(s5);
}
Locale[] s=Locale.getAvailableLocales();
for(Locale s1:s)
{
//System.out.print("Available locales is:");
//System.out.println(s1.getDisplayCountry()+"....."+s1.getDisplayLanguage());
}}}

```

## 2. NumberFormat:

Various countries follow various styles to represent number.

Example:

```
double d=123456.789;
```

```
1,23,456.789-----INDIA
```

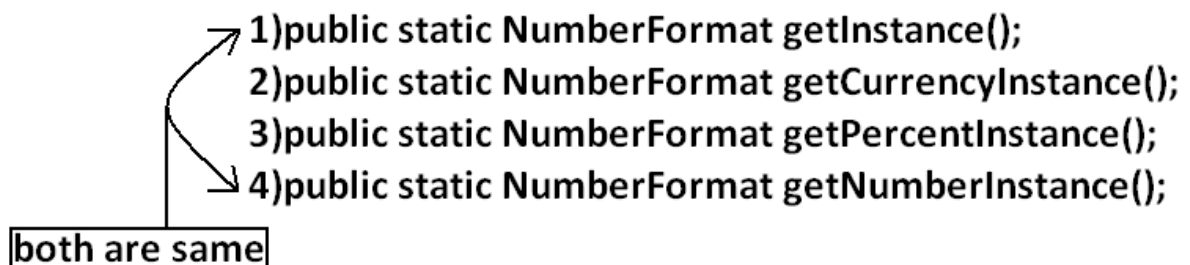
```
123,456.789-----US
```

```
123.456,789-----ITALY
```

- By using NumberFormat class we can format a number according to a particular Locale.
- NumberFormat class present in java.Text package and it is an abstract class. Hence we can't create an object by using constructor.  
NumberFormat nf=new NumberFormat(); -----invalid

Getting NumberFormat object for the default Locale:

NumberFormat class defines the following methods for this.



```

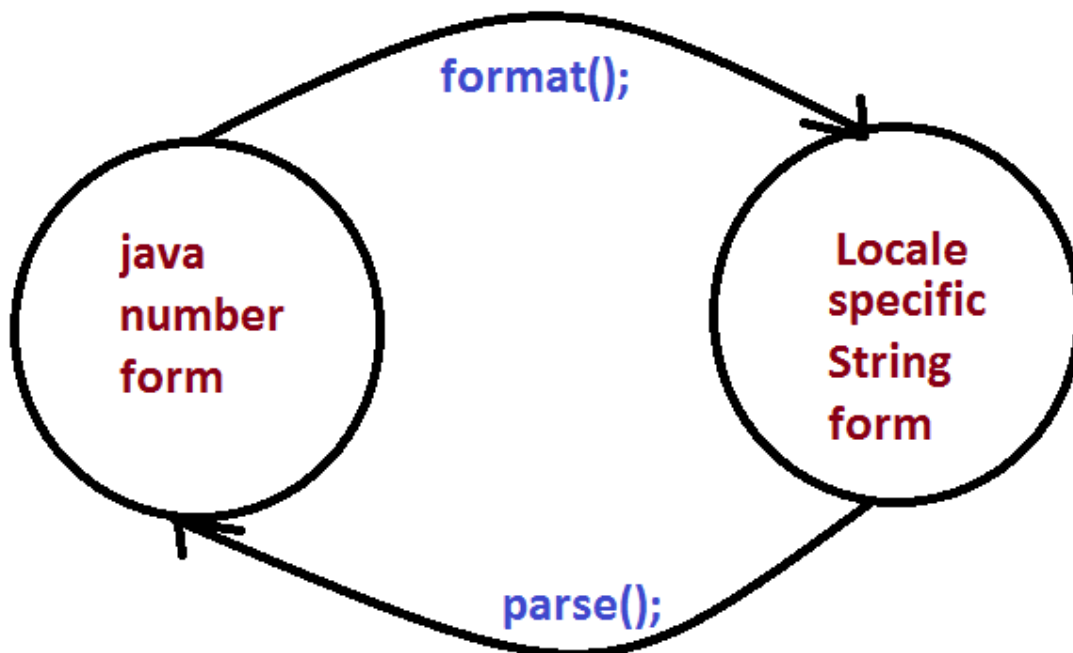
1)public static NumberFormat getInstance();
2)public static NumberFormat getCurrencyInstance();
3)public static NumberFormat getPercentInstance();
4)public static NumberFormat getNumberInstance();

```

both are same

Getting NumberFormat object for the specific Locale:

- The methods are exactly same but we have to pass the corresponding Locale object as argument.  
*Example: public static NumberFormat getInstance(Locale l);*
- Once we got NumberFormat object we can call the following methods to format and parse numbers.  
*public String format(long l);*  
*public String format(double d);*
- To convert a number from java form to Locale specific form.  
*public Number parse(String source)throws ParseException*
- To convert from Locale specific String form to java specific form.



**Requirement:** Write a program to display java number form into Italy specific form.

Example:

```
import java.util.*;  
import java.text.*;  
class NumberFormatDemo  
{
```

```
public static void main(String args[]){
double d=123456.789;
NumberFormat nf=NumberFormat.getInstance(Locale.ITALY);
System.out.println("ITALY form is :"+nf.format(d));
}
}
```

Output:

ITALY form is :123.456,789

**Requirement:** Write a program to print a java number in INDIA, UK, US and ITALY currency formats.

Program:

```
import java.util.*;
import java.text.*;
class NumberFormatDemo
{
public static void main(String args[]){
double d=123456.789;
Locale INDIA=new Locale("pa", "IN");
NumberFormat nf=NumberFormat.getCurrencyInstance(INDIA);
System.out.println("INDIA notation is :"+nf.format(d));

NumberFormat nf1=NumberFormat.getCurrencyInstance(Locale.UK);
System.out.println("UK notation is :"+nf1.format(d));
NumberFormat nf2=NumberFormat.getCurrencyInstance(Locale.US);
System.out.println("US notation is :"+nf2.format(d));
NumberFormat
nf3=NumberFormat.getCurrencyInstance(Locale.ITALY);
System.out.println("ITALY notation is :"+nf3.format(d));
}}
```

Output:

INDIA notation is: INR 123,456.79

UK notation is: £123,456.79

US notation is: \$123,456.79

ITALY notation is: € 123.456,79

**Setting Maximum, Minimum, Fraction and Integer digits:**

NumberFormat class defines the following methods for this purpose.

1. public void setMaximumFractionDigits(int n);
2. public void setMinimumFractionDigits(int n);
3. public void setMaximumIntegerDigits(int n);
4. public void setMinimumIntegerDigits(int n);

Example:

```
import java.text.*;
public class NumberFormatExample
{
public static void main(String[] args){
```

```

NumberFormat nf=NumberFormat.getInstance();
nf.setMaximumFractionDigits(3);
System.out.println(nf.format(123.4));
System.out.println(nf.format(123.4567));
nf.setMinimumFractionDigits(3);
System.out.println(nf.format(123.4));
System.out.println(nf.format(123.4567));
nf.setMaximumIntegerDigits(3);
System.out.println(nf.format(1.234));
System.out.println(nf.format(123456.789));
nf.setMinimumIntegerDigits(3);
System.out.println(nf.format(1.234));
System.out.println(nf.format(123456.789));
}}

```

Output:

```

123.4
123.457
123.400
123.457
1.234
456.789
001.234
456.789

```

### 3. DateFormat:

Various countries follow various styles to represent Date. We can format the date according to a particular locale by using DateFormat class.

DateFormat class present in java.text package and it is an abstract class.

Getting DateFormat object for default Locale:

DateFormat class defines the following methods for this purpose.

- 1)public static DateFormat getInstance();
- 2)public static DateFormat getDateInstance();
- 3)public static DateFormat getDateInstance(int style);



```

DateFormat.FULL----->0
DateFormat.LONG----->1
DateFormat.MEDIUM----->2
DateFormat.SHORT----->3

```

The default style is Medium style

Getting DateFormat object for the specific Locale:



```
public static DateFormat getInstance(int style, Locale l);
```

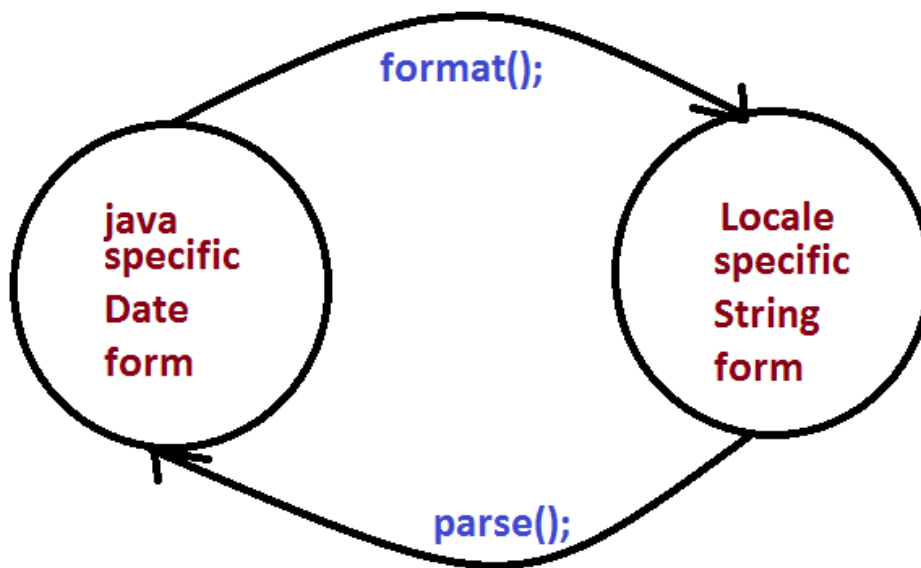
Once we got DateFormat object we can format and parse Date by using the following methods.

```
public String format(Date date);
```

To convert the date from java form to locale specific string form.

```
public Date parse(String source)throws ParseException
```

To convert the date from locale specific form to java form.



**Requirement:** Write a program to represent current system date in all possible styles of us format.

- Questions and answers
  - Full Form
  - Study Material
- Interview question and answer
- Interview questions and answers
  - Iso
  - Date and time

Program:

```
import java.text.*;  
import java.util.*;  
public class DateFormatDemo  
{
```

```
public static void main(String args[]){
System.out.println("full form is
:"+DateFormat.getDateInstance(0).format(new Date()));
System.out.println("long form is
:"+DateFormat.getDateInstance(1).format(new Date()));
System.out.println("medium form is
:"+DateFormat.getDateInstance(2).format(new Date()));
System.out.println("short form is
:"+DateFormat.getDateInstance(3).format(new Date()));
}
}
```

Output:

Full form is: Wednesday, July 20, 2011

Long form is: July 20, 2011

Medium form is: Jul 20, 2011

Short form is: 7/20/11

Note: The default style is medium style.

Requirement: Write a program to represent current system date in UK, US and ITALY styles.

Program:

```
import java.text.*;
import java.util.*;
public class DateFormatDemo
{
public static void main(String args[]){
DateFormat UK=DateFormat.getDateInstance(0,Locale.UK);
DateFormat US=DateFormat.getDateInstance(0,Locale.US);
DateFormat ITALY=DateFormat.getDateInstance(0,Locale.ITALY);
System.out.println("UK style is :"+UK.format(new Date()));
System.out.println("US style is :"+US.format(new Date()));
System.out.println("ITALY style is :"+ITALY.format(new
Date()));
}
}
```

Output:

UK style is: Wednesday, 20 July 2011

US style is: Wednesday, July 20, 2011

ITALY style is: mercoled 20 luglio 2011

Getting DateFormat object to get both date and time:

DateFormat class defines the following methods for this.

- 1)public static DateFormat getDateTImeInstance();
- 2)public static DateFormat getDateTImeInstance(int dateStyle, int timeStyle);
- 3)public static DateFormat getDateTImeInstance(int dateStyle, int timeStyle,Locale l);



Example:

```
import java.text.*;
import java.util.*;
public class DateFormatDemo
{
    public static void main(String args[]){
        DateFormat
        ITALY=DateFormat.getDateTImeInstance(0,0,Locale.ITALY);
        System.out.println("ITALY style is:"+ITALY.format(new
        Date()));
    }
}
```

Output:

ITALY style is: mercoled 20 luglio 2011 23.21.30 IST

# **CORE JAVA**

## **With**

# **SCJP / OCJP**

### **Study Material**

**Chapter 19: Development**



**DURGA M.Tech**

**(Sun certified & Realtime Expert)**

**Ex. IBM Employee**

**Trained Lakhs of Students  
for last 14 years across INDIA**

**India's No.1 Software Training Institute**

# **DURGASOFT**

**www.durgasoft.com Ph: 9246212143 ,8096969696**

# Development

## Agenda

1. Introduction
2. Javac
3. Java
4. Classpath
5. Jar file
6. What is the difference between Jar, War and Ear ?
7. Various Commands
  - o To create a jar file
  - o To extract a jar file
8. System properties
9. How to set system property from the command prompt
10. What is the difference between path and classpath ?
11. What is the difference between JDK, JRE and JVM ?
12. Shortcut way to place a jar files
13. Web Applications Vs Enterprise Applications
14. Web Server Vs Application Server
15. Creation of executable jar file
16. In how many ways we can run a java program

## Introduction

Javac:

we can use Javac to compile a single or group of ".java files".

Syntax:

```

javac    [options]    Test.java (valid)
          ↓
          -source      Test.java  Demo.java (valid)
          -version     *.java (valid)
          -cp <path>
          -d <directory>
          .
          .
          .
          .

```

Java:

we can use java command to run a single ".class file".

Syntax:

```

java    [options]    classfile    arg[0]    arg[1].....
          -version
          -ea/-esa/-da/-dsa
          -D
          -cp/-classpath
          .
          .
          .
          .

```

Note :

We can compile any number of source files at a time but we can run only one .class file.

Classpath:

Class path describes the location where the required ".class files" are available.  
Java compiler & JVM will use classpath to locate required class files.

We can set the class path in the following 3 ways.

1. Permanently by using environment variable "classpath". This class path will be preserved after system restart also.
2. Temporary for a particular command prompt level by using "set" command.

Example:

```
set classpath=%classpath%;D:\durga_classes;.
```

Once if you close the command prompt automatically this class path will be lost.

3. We can set the class path for a particular command level by using "-cp" (or) "-classpath". This class path is applicable only for that command execution. After executing the command this classpath will be lost.

Among the 3 ways of setting the class path the most common way is setting class path at command level by using "-cp".

**Note :**

- By default java compiler & JVM will search in current working directory for the required .class files
- If we set the classpath explicitly then JVM will search only in our specified location for .class file and it won't search in current working directory.
- Once we set the classpath we can run our program from any location.

Example 1:

```
class Rain
{
    public static void main(String args[]){
        System.out.println("Raining of jobs these days");
    }
}
```

Analysis:

D:\Java>javac Rain.java (valid)

D:\Java>java Rain (valid)

Raining of jobs these days.

D:\>java Rain (invalid)

Exception in thread "main" java.lang.NoClassDefFoundError: Rain

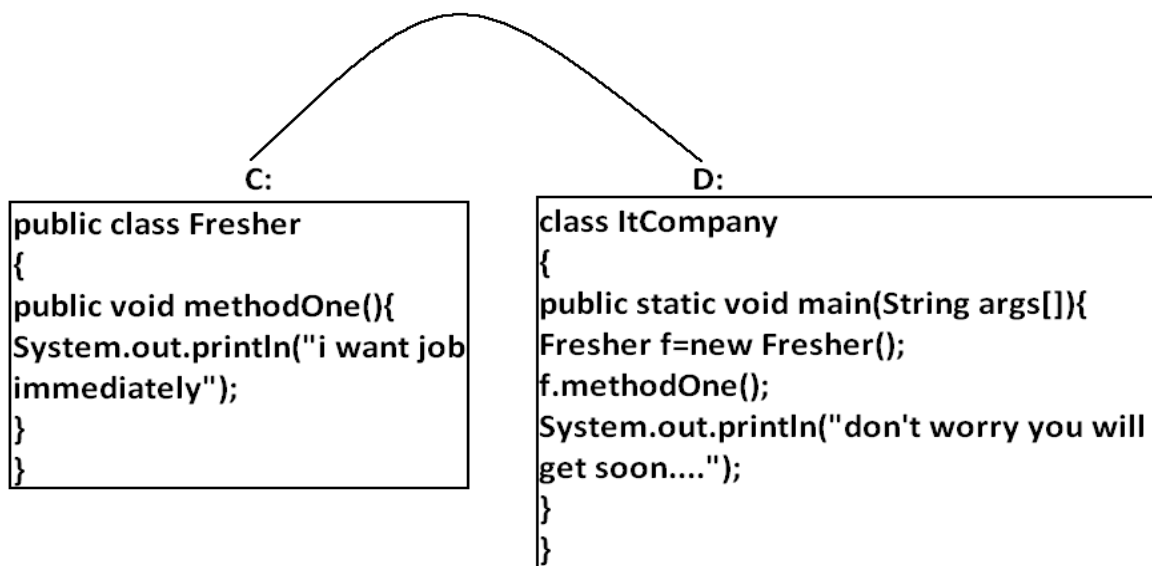
D:\>java -cp D:\java Rain (valid)

Raining of jobs these days.

D:\>java Rain (invalid)

C:\>java -cp d:\java Rain (valid)

Raining of jobs these days

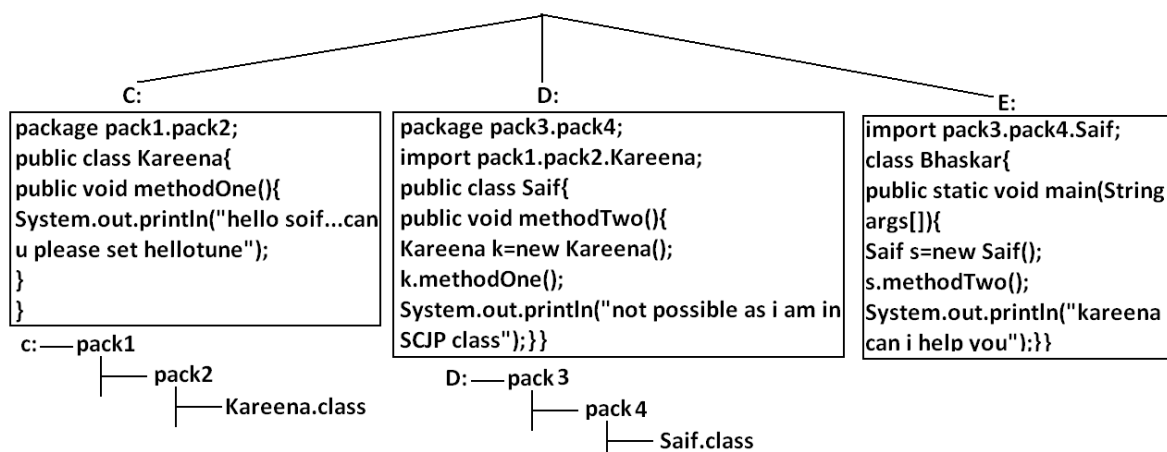
Example 2:Analysis:



```

C:\>javac Fresher.java (valid)
D:\>javac ItCompany.java (invalid)
compile time error: ItCompany.java:4: cannot find symbol
        symbol : class Fresher
        location: class ItCompany
        Fresher f=new Freaher();
D:\>javac -cp c: ItCompany.java(valid)
D:\>java ItCompany
Runtime error:NoClassDefFoundError: Fresher
D:\>java -cp c: ItCompany
Runtime error:NoClassDefFoundError: ItCompany
D:\>java -cp .;c: ItCompany (valid)
i want job immediately
don't worry you will get soon....
E:\>java -cp D;;C: ItCompany (valid)

```

**Example 3:****Analysis:**

```

C:\>javac -d . Kareena.java (valid)
D:\>javac -d . Saif.java (invalid) ———> compile time error
                                           Saif.java:5: cannot find symbol
                                           symbol  : class Kareena
                                           location: class pack3.pack4.Saif
                                           Kareena k=new Kareena();

D:\>javac -cp c: -d . Saif.java (valid)
E:\>javac Bhaskar.java (invalid) ———> compile time error
                                           Bhaskar.java:4: cannot find symbol
                                           symbol  : class Saif
                                           location: class Bhaskar
                                           Saif s=new Saif();

E:\>javac -cp d: Bhaskar.java (valid)
E:\>java Bhaskar (invalid) ———> Runtime error
                                           NoClassDefFoundError: pack3/pack4/Saif

E:\>java -cp d: Bhaskar (invalid) ———> Runtime error
                                           NoClassDefFoundError: Bhaskar

E:\>java -cp .;d: Bhaskar (invalid) ———> Runtime error
                                           NoClassDefFoundError: pack1/pack2/Kareena

E:\>java -cp .;d;;c: Bhaskar (valid) ———> output:
                                           hello soif...can u please set hellotune
                                           not possible as i am in SCJP class
                                           kareena can i help you

F:\>java -cp E;;D;;C: Bhaskar (valid) ———> output:
                                           hello soif...can u please set hellotune
                                           not possible as i am in SCJP class
                                           kareena can i help you

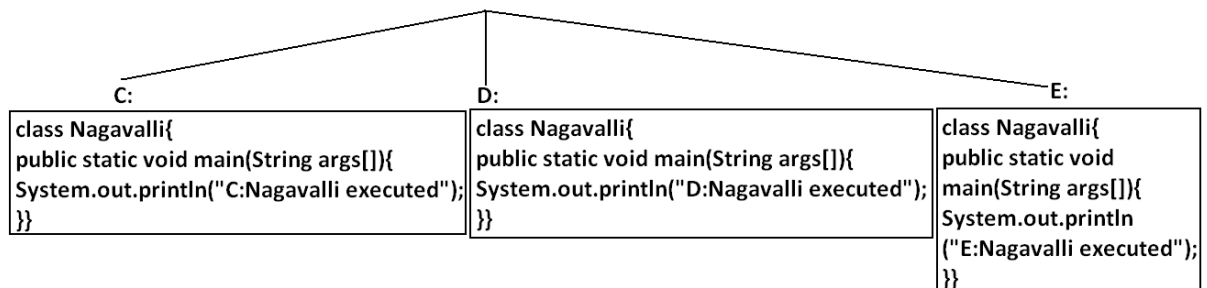
```

**Note:** If any folder structure created because of package statement. It should be resolved by import statement only and the location of base package should be make it available in class path.

**Note:** Compiler will always checks one level of dependency, where as the JVM will check all levels of dependency.

**Note:** In classpath the order of locations is very important and compiler & JVM will always search from left to right for thr required .class file untill match is available.

#### Example 4:



**Analysis:**

```
C:\>javac Nagavalli.java (valid)
D:\>javac Nagavalli.java (valid)
E:\>javac Nagavalli.java (valid)
C:\>java Nagavalli           output:
                               C:Nagavalli executed
C:\>java -cp e;;d;;c: Nagavalli output:
                               E:Nagavalli executed
C:\>java -cp d;;e;;c: Nagavalli output:
                               D:Nagavalli executed
```

**Jar file:**

If several dependent classes present then it is never recommended to set the classpath individual for every component. We have to group all these ".class files" into a single zip file and we have to make that zip file available to the classpath. This zip file is nothing but jar file.

**Example 1 :** To develop a Servlet class all dependent classes are available into a single jar file (Servlet-api.jar) hence we have to place this jar file available in the classpath to compile and run Servlet program.

**Example 2 :** To use Log4J in our application all dependent classes are available in log4j.jar hence to use Log4J in our application. We have to use this jar file in the classpath.

**What is the difference between Jar, War and Ear ?**

**Jar (java archive):** Represents a group of ".class files".

**War (web archive):** Represents a web application which may contains Servlets, JSP, HTML pages, JavaScript files etc.

If we maintain web application in the form of war file, the project delevering , transportation and deployment will become easy.

**Ear (Enterprise archive):** it represents an enterprise application which may contain Servlets, JSP, EJB'S, JMS component etc.

In generally an ear file consists of a group of war files and jar files.

Ear=war+ jar

### Various Commands:

To create a jar file:

```
D:\Enum>jar -cvf praveen.jar Beer.class Test.class X.class
```

```
D:\Enum>jar -cvf praveen.jar *.class
```

```
D:\Enum>jar -cvf praveen.jar *.*
```

To extract a jar file:

```
D:\Enum>jar -xvf bhaskar.jar
```

To display table of contents of a jar file:

```
D:\Enum>jar -tvf bhaskar.jar
```

Example 5:

```
public class BhaskarColorFulCalc{
public static int add(int x,int y){
return x*y;
}
public static int multiply(int x,int y){
return 2*x*y;
}}

```

Analysis:

```
C:\>javac BhaskarColorFulCalc.java
```

```
C:\>jar -cvf bhaskar.jar BhaskarColorFulCalc.class
```

Example 6:

```
class Client{
public static void main(String args[]){
System.out.println(BhaskarColorFulCalc.add(10,20));
System.out.println(BhaskarColorFulCalc.multiply(10,20));
}}

```

Analysis:

```
D:\Enum>javac Client.java (invalid)
```

```
D:\Enum>javac -cp c: Client.java (invalid)
```

```
D:\Enum>javac -cp c:\bhaskar.jar Client.java (valid)
```

```
D:\Enum>java -cp .;c:\bhaskar.jar Client (valid)
```

**Note:** Whenever we are placing jar file in the classpath compulsory we have to specify the name of the jar file also and just location is not enough.

### System properties:

- For every system some persistence information is available in the form of system properties. These may include name of the os, java version, vendor of jvm , userCountry etc.
- We can get system properties by using `getProperties()` method of system class.

The following program displays all the system properties.

#### Example 7:

```
import java.util.*;
class Test{
public static void main(String args[]){
//Properties is a class in util package.
//here getProperties() method returns the Properties object.
Properties p=System.getProperties();
p.list(System.out);
}
}
```

### How to set system property from the command prompt:

We can set a system property explicitly from the command prompt by using `-D` option.

#### Command:

D:\Enum>java -Dbhaskar=scjp Test

                    ↓                    ↓

propertyname      propertyvalue

The main advantage of setting System Properties is we can customize the behaviour of java program.

```
class Test {
public static void main(String args[]) {
String course=System.getProperty("course");
if(course.equals("scjp")) {
    System.out.println("SCJP Information");
}
else
```

```
    System.out.println("other course information");  
}  
}
```

output:

```
c:> java -Dcourse=scjp Test  
    SCJP Information  
c:> java -Dcourse=scwcd Test  
    other course information
```

What is the difference between path and classpath ?

**Path:** We can use "path variable" to specify the location where required binary executables are available.

If we are not setting path then "java" and "Javac" commands won't work.

**Classpath:** We can use "classpath variable" to describe location where required class files are available.

If we are not setting classpath then our program won't compile and run.

What is the difference between JDK, JRE and JVM ?

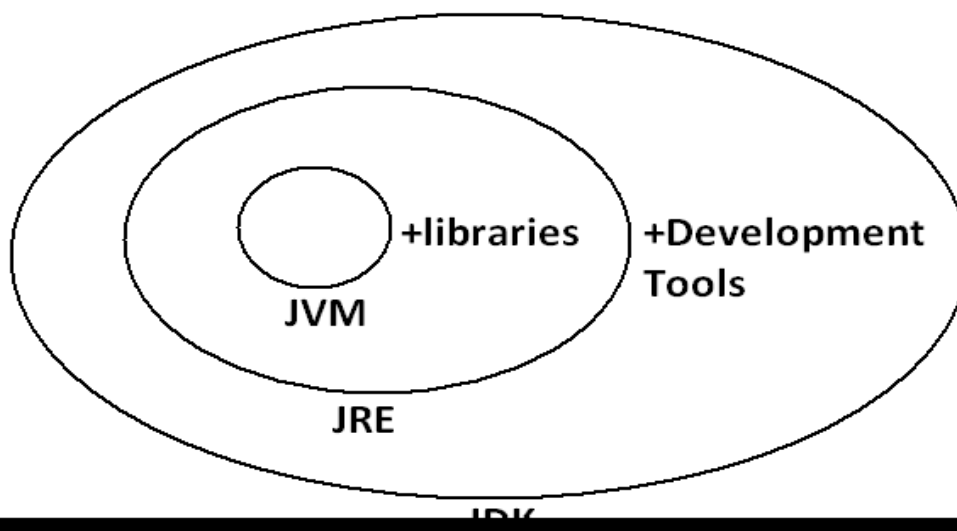
**JDK (java development kit):** To develop and run java applications the required environment is JDK.

**JRE (java runtime environment):** To run java application the required environment is JRE.

**JVM (java virtual machine):** To execute java application the required virtual machine is JVM.

JVM is an interpreter which is responsible to run our program line by line.

Diagram:



- JDK=JRE+Development Tools.
- JRE=JVM+Libraries.
- JRE is the part of JDK.
- Jvm is the part of JRE.

**Note:** At client side JRE is required and at developers side JDK is required.

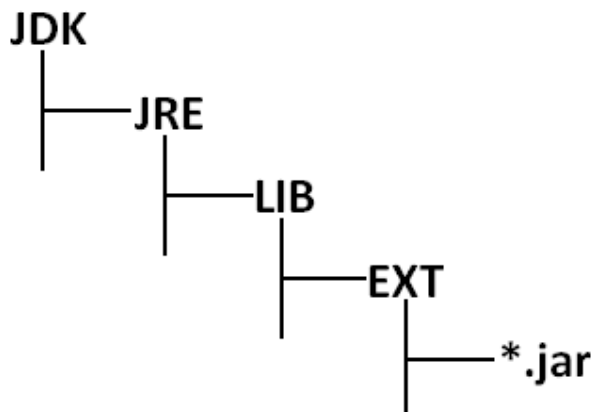
On the developers machine we have to install JDK , where as on the client machine we have to install JRE.

### Shortcut way to place a jar files available in the classpath:

If we are placing jar file in the following location then it is not required to set classpath explicitly.

By default it's available for Java compiler and JVM.

Diagram:



### Web Applications Vs Enterprise Applications:

A web application contains only web related technologies Servlets, Jsps, HTML etc., where as an enterprise applications can be developed by any technology from Java J2EE like Servlets, Jsps, EJB, JMS components etc.,

J2EE compatible application is Enterprise Application.

### Web Server Vs Application Server :

Web Server provides environment to run web applications, webserver provides support only for web related technologies like Servlets, jsp.

Ex: Tomcat server

An Application Server provides environment to run enterprise applications. Application server provides support for any technology from J2EE like Servlet, Jsp, EJB, JMS components etc.,

Ex: weblogic, web sphere , J Boss etc.,

J2EE compatible server is Application Server.

Every application server contains in built web server.

### Creation of executable jar file :

```
import java.awt.*;  
import java.awt.event.*;  
  
public class JarDemo {  
    public static void main(String args[]) {  
        Frame f=new Frame();  
        f.addWindowListener(new WindowAdaptor {  
            public void windowClosing(WindowEvent e) {  
                System.exit(0);  
            }  
        });  
  
        f.add(new Label("I can create Executable Jar File"));  
        f.setSize(500,500);  
        f.setVisible(true);  
    }  
}
```

**Main\_Class : JarDemo**

**manifest.mf**

```
javac JarDemo.java
```

```
1. java JarDemo
```

```
jar -cvfm demo.jar manifest.mf JarDemo.class  
JarDemo$1.class
```

```
2. java -jar demo.jar
```

```
3. By clicking jar file we can execute
```



**Creation of Batch file :**

```
java -cp c:\charan_classes JarDemo
```

**abc.bat**

By clicking this batch file , the java program will be executed.

**In how many ways we can run a java program :**

1. by executing class file.  
Ex: java Test
2. by executing a jar file from the command prompt.  
Ex: java -jar demo.jar
3. By double clicking jar file
4. By double clicking batch file

# **CORE JAVA**

## **With**

# **SCJP / OCJP**

### **Study Material**

#### **Chapter 20: Assertions**



**DURGA M.Tech**

**(Sun certified & Realtime Expert)**

**Ex. IBM Employee**

**Trained Lakhs of Students  
for last 14 years across INDIA**

**India's No.1 Software Training Institute**

# **DURGASOFT**

**www.durgasoft.com Ph: 9246212143 ,8096969696**

## Assertions

### Agenda

1. Introduction
2. Assert as keyword and identifier
3. Types of assert statements
  - i. Simple version
  - ii. Argumented version
4. Various runtime flags
5. Appropriate and Inappropriate use of assertions
6. AssertionError

### Introduction:

1. The most common way of debugging is uses of sops. But the main disadvantage of sops is after fixing the bug compulsory we should delete extra added sops otherwise these sops also will be executed at runtime which impacts performance of the system and disturbs logging mechanism.
2. To overcome these problems sun people introduced assertions concept in 1.4 versions.
3. The main advantage of assertions over sops is based on our requirement we can enable or disable assertions and by default assertions are disable hence after fixing the bug it is not required to delete assert statements explicitly.
4. Hence the main objective of assertions is to perform debugging.
5. Usually we can perform debugging either in development environment or Test environment but not in production environment hence assertions concept is applicable for the development and test environments but not for the production.

### Assert as keyword and identifier:

assert keyword is introduced in 1.4 version hence from 1.4 version onwards we can't use assert as identifier but until 1.3 we can use assert as an identifier.

Example:

```
class Test
{
    public static void main(String[] args)
    {
        int assert=10;
        System.out.println(assert);
    }
}
```

Output:

- javac Test.java(invalid)
- As of release 1.4, 'assert' is a keyword, and may not be used as an identifier. (Use -source 1.3 or lower to use 'assert' as an identifier)
- javac -source 1.3 Test.java(valid)
- The code compiles fine but warnings will be generated.

- java Test
- 10

**Note:** It is always possible to compile a java program according to a particular version by using -source option.

#### Types of assert statements:

There are 2 types of asset statements.

1. Simple version
2. Argumented version

#### **Simple version:**

Syntax: `assert(b);`//b should be boolean type.

- If 'b' is true then our assumption is correct and continue rest of the program normally.
- If 'b' is false our assumption is fails and hence stop the program execution by raising assertion error.

**Example:**

```
class Test
{
    public static void main(String[] args)
    {
        int x=10;
        ;;;;;;;;;;
        assert(x>10);
        ;;;;;;;;;;
        System.out.println(x);
    }
}
```

**Output:**

```
javac Test.java
java Test
10
java -ea Test(Invalid)
R.E: AssertionError
```

**Note:** By default assertions are disable and hence they won't be executed by default we have to enable assertions explicitly by using -ea option.

#### **Argumented version:**

By using argumented version we can argument some extra information with the assertion error.

Syntax: `assert(b):e;`

'b' should be boolean type.

'e' can be any type.

Example:

```
class Test
{
    public static void main(String[] args)
    {
        int x=10;
        ;;;;;;;;;;
        assert(x>10):"here x value should be >10 but it
is not";
        ;;;;;;;;;;
        System.out.println(x);
    }
}
```

Output:

```
javac Test.java
```

```
java Test
```

```
10
```

```
java -ea Test(invalid)
```

```
R.E: AssertionError: here x value should be >10 but it is not
```

Conclusion 1:

```
assert(b):e;
```

'e' will be evaluated if and only if 'b' is false that is if 'b' is true then 'e' won't be evaluated.

Example:

```
class Test
{
    public static void main(String[] args)
    {
        int x=10;
        ;;;;;;;;;;
        assert(x==10):++x;
        ;;;;;;;;;;
        System.out.println(x);
    }
}
```

Output:

```
javac Test.java
```

```
java Test
```

```
10
```

```
java -ea Test
```

```
10
```

Conclusion 2:

```
assert(b):e;
```

For the 2nd argument we can take method call also but void type method call not allowed.

Example:

```
class Test
{
    public static void main(String[] args)
    {
        int x=10;
        //////////
        assert(x>10):methodOne();
        //////////
        System.out.println(x);
    }
    public static int methodOne()
    {
        return 999;
    }
}
```

Output:

```
javac Test.java
```

```
java Test
```

```
10
```

```
java -ea Test
```

```
R.E: AssertionError: 999
```

If methodOne() method return type is void then we will get compile time error saying void type not allowed here.

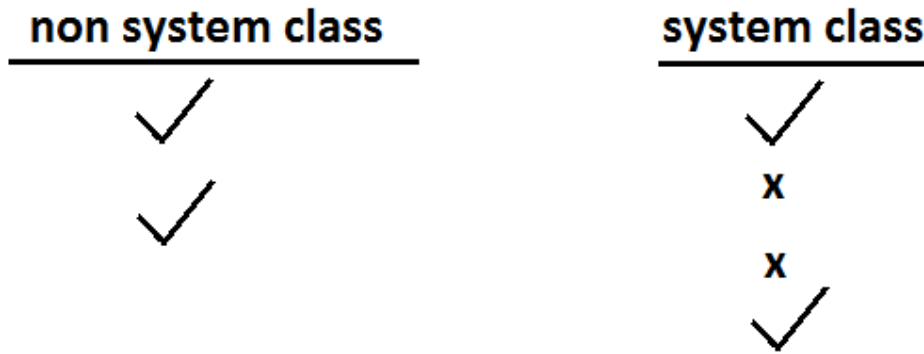
#### Various runtime flags:

1. -ea: To enable assertions in every non system class(user defined classes).  
-enableassertions: It is exactly same as -ea.
2. -da: To disable assertions in every non system class.  
-disableassertions: It is exactly same as -da.
3. -esa: To enable assertions in every system class(predefined classes or application classes).  
-enablesystemassertions: It is exactly same as -esa.
4. -dsa: To disable assertions in every system class.  
-disablesystemassertions: It is exactly same as -dsa.

Example: We can use these flags in combination also but all these flags will be executed from left to right.

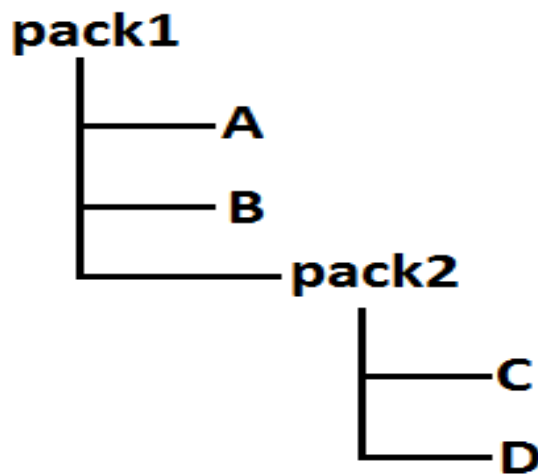
Example:

```
java -ea -esa -dsa -ea -dsa -esa Test
```



At the end in both system and non system class assertions are enabled.

Example:



1. To enable assertions only in B class.  
java -ea: pack1.B
2. To enable assertions only in B and C classes.  
java -ea: pack1.B -ea:pack1.pack2.C
3. To enable assertions in every class of pack1.  
java -ea:pack1
4. To enable assertions everywhere inside pack1 except B class.  
java -ea:pack1 -da:pack1.B
5. To enable assertions in every class of pack1 except pack2 classes.  
java -ea:pack1... -da:pack1.pack2...

It is possible to enable (or) disable assertions either class wise (or) package wise also.

#### Appropriate and inappropriate use of assertions:

- It is always inappropriate to mix programming logic with assert statements, because there is no guaranty for the execution of assert statement always at runtime.

Example:

|                                                                                                                        |                                                                                                                                                                                                      |
|------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>withdraw(double d) {     assert(d&gt;=100);     .     . }</pre> <p>inappropriate use of<br/>assert statement.</p> | <pre>withdraw(double d) {     if(d&lt;100)     {         throw new IllegalArgumentException     }     else     {         withdraw code     } }</pre> <p>appropriate use of<br/>assert statement.</p> |
|------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

- For validating public method arguments usage of assertions is always inappropriate, because outside person is not aware whether assertions are enabled or disabled in our local system.
- While perform debugging if any place where the control is not allow to reach then that is the best place to use assertions.

Example:

```
switch(x)
{
    case 1:
        System.out.println("Jan");
        break;
    case 2:
        System.out.println("Feb");
        break;
    case 3:
        System.out.println("Mar");
        break;
    case 12:
        System.out.println("Dec");
        break;
    default: assert(false);
}
```

- It is always inappropriate to use assertions for validating public method arguments.



- It is always appropriate to use assertions for validating private method arguments.
- It is always inappropriate to use assertions for validating command line arguments because these are arguments to public method main.

### AssertionError:

1. It is the child class of Error and it is unchecked.
2. Raised explicitly whenever assert statement fails.
3. Even though it is legal but it is not recommended to catch AssertionError.

Example:

```
class Test {
public static void main(String[] args){
    int x=10;
    try {
        assert(x>10);
    }
    catch (AssertionError e) {
        System.out.println("not a good programming practice to
catch AssertionError");
    }
        System.out.println(x);
    }
}
```

Output:

```
javac Test.java
```

```
java Test
```

```
10
```

```
Not a good programming practice to catch AssertionError
```

```
10
```

Example 1:

```
class One
{
    public static void main(String[] args)
    {
        int assert=0;
    }
}
class Two
{
    public static void main(String[] args)
    {
        assert(false);
    }
}
```

Output:

```
Javac -source 1.3 one.java//compiles with warnings.
```

```
Javac -source 1.4 one.java//compile time error.
```

```
Javac -source 1.3 Two.java//compile time error.
```

**Javac -source 1.4 Two.java//compiles without warnings.**

**Example 2:**

```
class Test
{
    public static void main(String[] args)
    {
        assert(args.length==1);
    }
}
```

**Which two will produce AssertionError?**

- 1) Java Test
- 2) Java -ea Test//R.E: AssertionError
- 3) Java Test file1
- 4) Java -ea Test file1
- 5) java -ea Test file1 file2//R.E: AssertionError
- 6) java -ea:Test Test file1

**To enable the assertions in a particular class.**

**Example 3:**

```
class Test
{
    public static void main(String[] args)
    {
        boolean assertOn=true;
        assert(assertOn):assertOn=true;
        if(assertOn)
        {
            System.out.println("assert is on");
        }
    }
}
```

**Output:**

**Java Test**

**Assert is on**

**Java -ea Test**

**Assert is on**

**In the above example boolean assertOn=false then answer following questions.**

**Javac Test.java**

**Java Test**

**java -ea Test**

**R.E: AssertionError: true**

**Example 4:**

```
class Test
{
    int z=5;
    public void stuff1(int x)
    {
        assert(x>0);—————> Inappropriate
        switch(x)
        {
            case 2:
                x=3;
            default:
                assert(false);—————> appropriate
        }
    }
}
```

because inside public method we are not using assert statements.

**Example 5:**

```
private void stuff2(int y)
{
    assert(y<0);————— appropriate
}
```

Example 6:

```
int z=5;
private void stuff3()
{
    assert(stuff4()); —— inappropriate
}
private boolean stuff4()
{
    z=6;
    return false;
}
```

Note: Because assert statement changes the value of Z. By using assert statement we can not changes the value that is why it is inappropriate.

# **CORE JAVA**

## **With**

# **SCJP / OCJP**

## **Study Material**

### **Chapter 21: JVM Architecture**



**DURGA M.Tech**

**(Sun certified & Realtime Expert)**

**Ex. IBM Employee**

**Trained Lakhs of Students  
for last 14 years across INDIA**

**India's No.1 Software Training Institute**

# **DURGASOFT**

**www.durgasoft.com Ph: 9246212143 ,8096969696**

# JVM Architecture

- 1) Virtual Machine
- 2) Types of Virtual Machines
- 3) Basic JVM Architecture
- 4) ClassLoader Sub System
  - Loading
  - Linking
  - Initialization
- 5) Types of ClassLoaders
  - Boot Strap ClassLoader
  - Extension ClassLoader
  - Application ClassLoader
- 6) How ClassLoader Works?
- 7) Customized ClassLoader
  - Need of Customized ClassLoader
  - Pseudo Code to Define Customized ClassLoader
- 8) Various Memory Areas of JVM
  - Method Area
  - Heap Area
  - Stack Memory
  - PC Registers Area
  - Native Method Stacks Area
- 9) Importance of Runtime Class
- 10) Program to Display Statistics of Heap Memory
  - MaxMemory
  - TotalMemory
  - FreeMemory
- 11) How to Set Maximum and Minimum Heap Size
- 12) Execution Engine
  - Interpreter
  - JIT Compiler
- 13) Java Native Interface (JNI)
- 14) Class File Structure

## Virtual Machine:

It is a Software Simulation of a Machine which can Perform Operations Like a Physical Machine.

## Types of Virtual Machines

There are 2 Types of Virtual Machines

- 1) Hardware Based OR System Based Virtual Machines
- 2) Software Based OR Application Based OR Process Based Virtual Machines

### 1) Hardware Based OR System Based Virtual Machines

It Provides Several Logical Systems on the Same Computer with Strong Isolation from Each Other.

#### Examples:

- 1) KVM (Kernel Based Virtual Machine) for Linux Systems
- 2) VMware (Virtual Machine ware)
- 3) Xen
- 4) Cloud Computing

The main advantage of Hard-ware based Virtual Machines is for effective utilization of hardware resources.

### 2) Software Based OR Application Based OR Process Based Virtual Machines

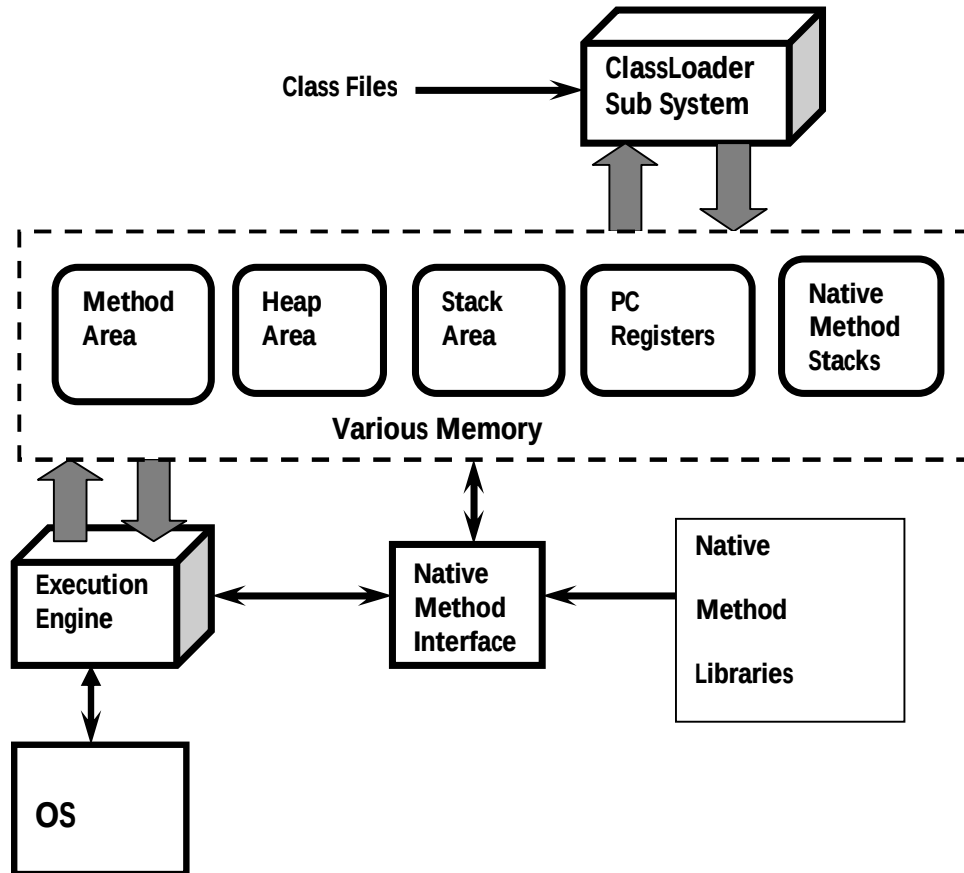
These Virtual Machines Acts as Runtime Engines to Run a Particular Programming Language Application.

#### Examples:

- 1) JVM Acts as Runtime Engine to Run Java Applications
- 2) PVM (Parrot VM) Acts as Runtime Engine to Run Scripting Languages Like PERL.
- 3) CLR (Common Language Runtime) Acts as Runtime Engine to Run .Net Based Applications.

## JVM

- JVM is the Part of JRE.
- JVM is Responsible to Load and Run Java Applications.

**Basic JVM Architecture****ClassLoader Sub System:**

ClassLoader Sub System is Responsible for the following 3 Activities.

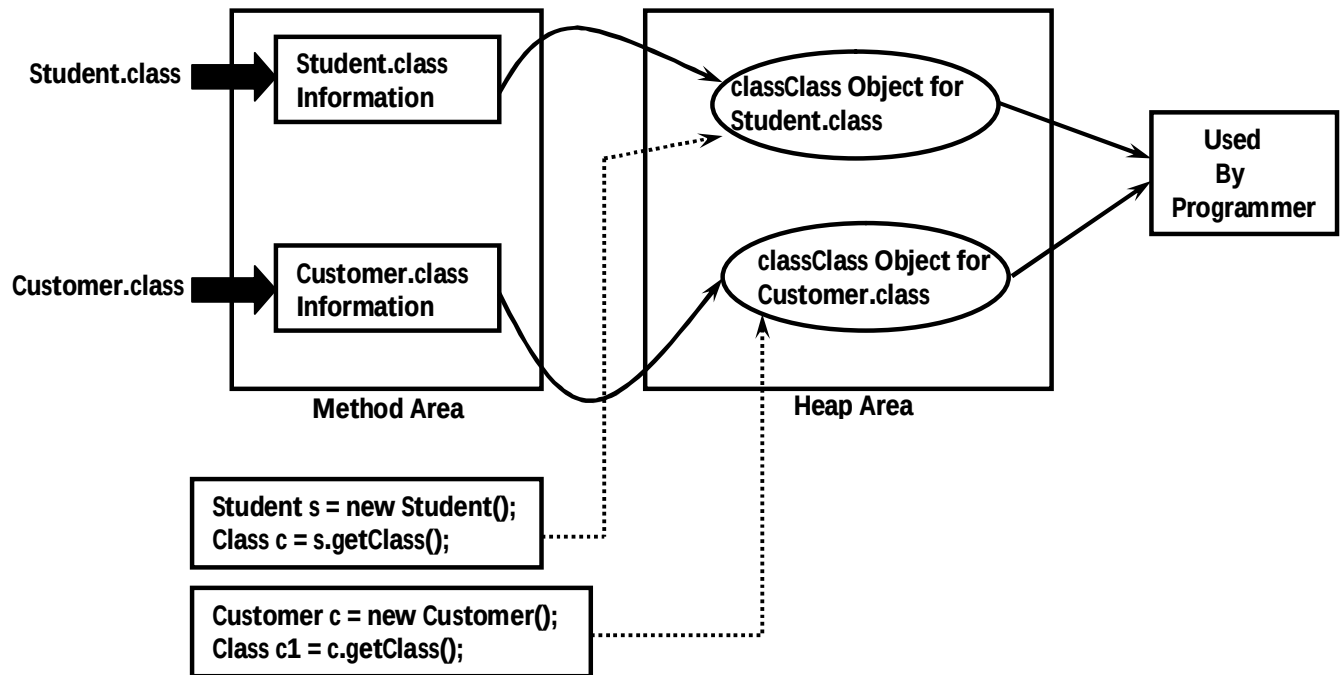
- 1) Loading
- 2) Linking
  - Verification
  - Preparation
  - Resolution
- 3) Initialization

**1) Loading:**

- Loading Means Reading Class Files and Store Corresponding Binary Data in Method Area.
- For Each Class File JVM will Store the following Information in Method Area.
  - 1) Fully Qualified Name of the Loaded Class OR Interface OR enum.
  - 2) Fully Qualified Name of its Immediate Parent Class.
  - 3) Whether .class File is related to Class OR Interface OR enum.
  - 4) The Modifiers Information
  - 5) Variables OR Fields Information
  - 6) Methods Information
  - 7) Constant Pool Information and so on.



- After loading .class File Immediately JVM will Creates an Object of the Type class Class to Represent Class Level Binary Information on the Heap Memory.



The Class Object can be used by Programmer to get Class Level Information Like Fully Qualified Name of the Class, Parent Name, Methods and Variables Information Etc.

**Program to print methods and variables information by using Class object:**

```
import java.lang.reflect.*;
class Student {
    private String name;
    private int rollNo;
    public String getName() {
        return name;
    }
    public void setRollNo(int rollNo) {
        this.rollNo = rollNo;
    }
}
class Test1 {
    public static void main(String args[]) {
        Student s = new Student();
        Class c = s.getClass();
        System.out.println(c.getName());
        Method[] m = c.getDeclaredMethods();
        for (int i=0; i<m.length; i++)
            System.out.println(m[i]);
        Field[] f = c.getDeclaredFields();
        for (int i=0; i<f.length; i++)
            System.out.println(f[i]);
    }
}
```

```
Student
public void Student.setRollNo(int)
public java.lang.String Student.getName()
private java.lang.String Student.name
private int Student.rollNo
```

In the Above Example by using Student class Class Object we can get its Methods and Variable Information.

**Note:** For Every loaded .class file Only One Class Object will be Created, even though we are using Class Multiple Times in Our Application.

```
class Test2 {
    public static void main(String args[]) {
        Student s1 = new Student();
        Student s2 = new Student();
        Class c1 = s1.getClass();
        Class c2 = s2.getClass();
    }
}
```

**2) Linking:**

Linking Consists of 3 Activities

- 1) Verification
- 2) Preparation
- 3) Resolution

**Verification:**

- It is the Process of ensuring that Binary Representation of a Class is Structurally Correct OR Not.
- That is JVM will Check whether .class File generated by Valid Compiler OR Not.i.e.whether .class File is Properly Formatted OR Not.
- Internally Byte Code Verifier which is Part of ClassLoader Sub System is Responsible for this Activity.
- If Verification Fails then we will get Runtime Exception Saying *java.lang.VerifyError*.

**Preparation:**

In this Phase JVM will Allocate Memory for the Class Level Static Variables and Assign DefaultValues (But Not Original Values).

**Note:**Original Values will be assigned in Initialization Phase.

**Resolution:**

- It is the Process of Replaced Symbolic References used by the Loaded Type with Original References.
- Symbolic References are Resolved into Direct References by searching through Method Area to Locate the Referenced Entity.

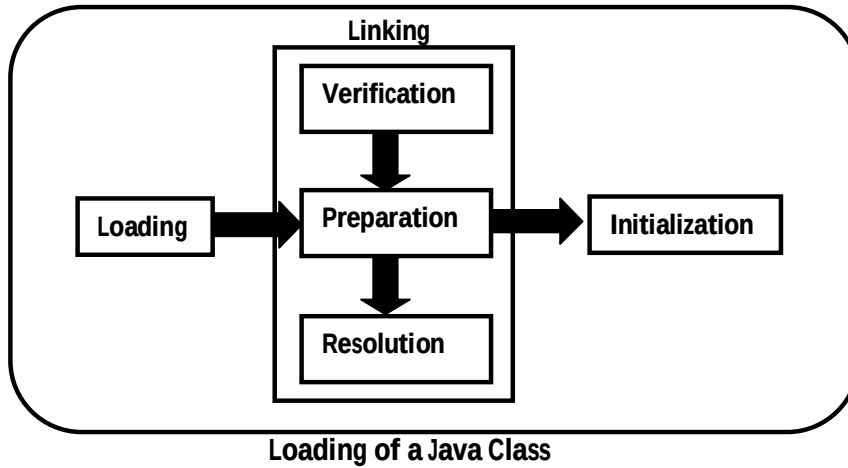
```
class Test {  
    public static void main(String[] args) {  
        String s = new String("Durga");  
        Student s1 = new Student();  
    }  
}
```

- Test.class
- String.class
- Student.class
- Object.class

- For the Above Class, ClassLoadersub system Loads *Test.class*, *String.class*,*Student.class*, and *Object.class*.
- The Names of these Class Names are stored in *Constant Pool* of Test Class.
- In Resolution Phase these Names are Replaced with Actual References from Method Area.

**3) Initialization:**

In this Phase All Static Variables will be assigned with Original Values and Static Blocks will be executed from fromtop to bottom and from Parent to Child.



**Note:** While Loading, Linking and Initialization if any Error Occurs then we will get Runtime Exception Saying `java.lang.LinkageError`. Of course `VerifyError` is child class of `LinkageError` only.

### Types of ClassLoaders:

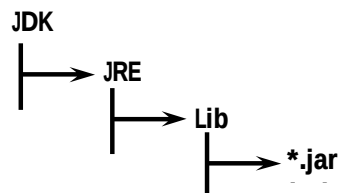
Every ClassLoader Sub System contains the following 3 ClassLoaders.

- 1) BootstrapClassLoader OR PrimordialClassLoader
- 2) ExtensionClassLoader
- 3) ApplicationClassLoader OR SystemClassLoader

### BootstrapClassLoader

- This ClassLoader is Responsible to load classes from `jdk\jre\lib` folder.
- All core java API classes present in `rt.jar` which is present in this location only. Hence all API classes (like `String`, `StringBuffer` etc) will be loaded by Bootstrap class Loader only.

- Location:

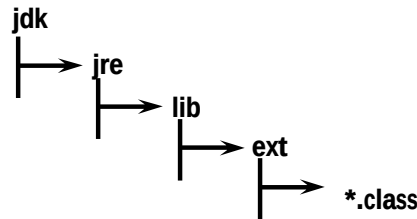


- This Location is Called `BootstrapClassPath`.
- That is `BootstrapClassLoader` is Responsible to Load Classes from `BootstrapClassPath`.
- `BootstrapClassLoader` is by Default Available with the JVM.
- It is implemented in Native Languages Like C and C++.

**Extension ClassLoader:**

- It is the Child of Bootstrap ClassLoader.
- ThisClassLoader is Responsible to Load Classes from Extension Class Path.

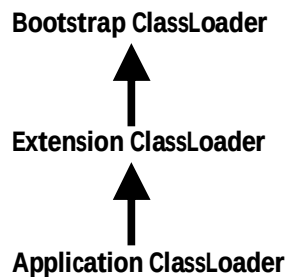
**Location:** jdk\jre\lib\ext



- This ClassLoader is implemented in Java and the corresponding .class File Name is [\*sun.misc.Launcher\\$ExtClassLoader.class\*](#)

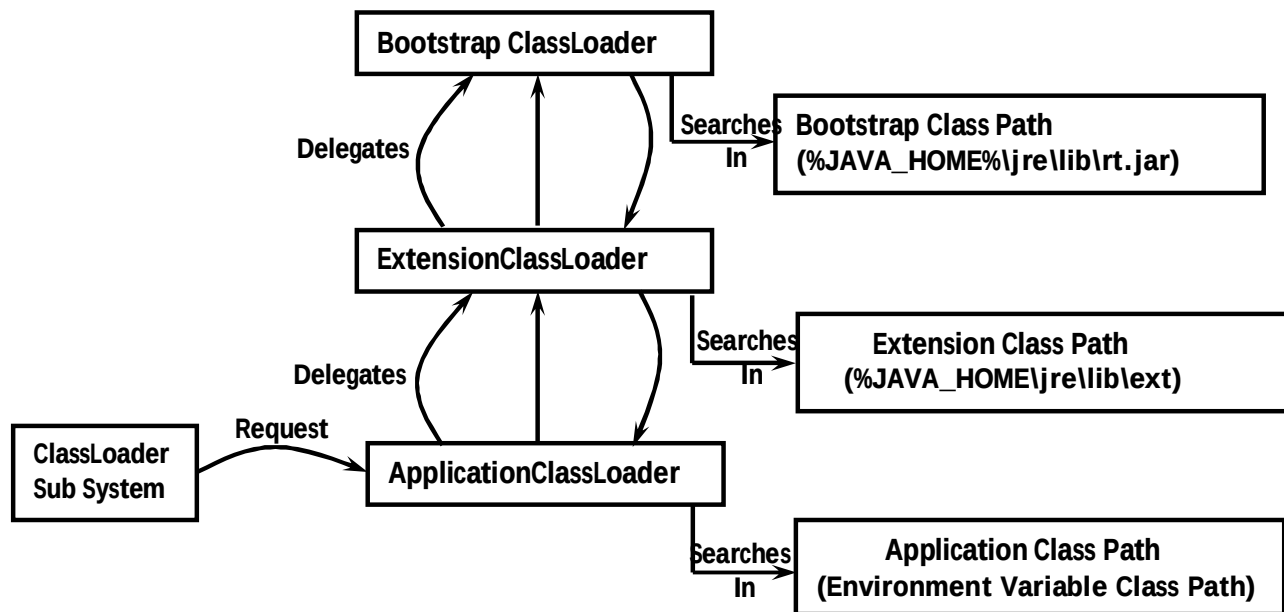
**Application ClassLoader OR System ClassLoader:**

- It is the Child of Extension ClassLoader.
- This ClassLoader is Responsible to Load Classes from Application Class Path (Current Working Directory).
- It Internally Uses Environment Variable Class Path.
- Application ClassLoader is implemented in Java and the corresponding .class File Name is [\*sun.misc.Launcher\\$AppClassLoader.class\*](#)

**How Java ClassLoader Works?**

- ClassLoader follows *Delegation Hierarchy* Principle.
- Whenever JVM Come Across a Particular Class, first it will Check whether the corresponding Class is Already Loaded OR Not.
- If it is Already Loaded in Method Area then JVM will Use that Loaded Class.
- If it is Not Already Loaded then JVM Requests ClassLoaderSub System to Load that Particular Class.
- Then ClassLoaderSub System Handovers the Request to ApplicationClassLoader.
- ApplicationClassLoader Delegates that Request to ExtensionClassLoader and ExtensionClassLoader in-turn Delegates that Request to BootstrapClassLoader.
- BootstrapClassLoader Searches in Bootstrap Class Path for the required .class File (jdk/jre/lib)
- If the required .class is Available, then it will be Loaded. Otherwise BootstrapClassLoader Delegates that Request to ExtensionClassLoader.

- **ExtensionClassLoader** will Search in Extension Class Path (jdk/jre/lib/ext). If the required .class File is Available then it will be Loaded, Otherwise it Delegates that Request to **ApplicationClassLoader**.
- **ApplicationClassLoader** will Search in Application Class Path (Current Working Directory). If the specified .class is Already Available, then it will be Loaded. Otherwise we will get Runtime Exception Saying *ClassNotFoundException* OR *NoClassDefFoundError*.



### Example:

```

class Test {
    public static void main(String[] args) {
        System.out.println(String.class.getClassLoader());
        System.out.println(Student.class.getClassLoader());
        System.out.println(Test.class.getClassLoader());
    }
}
  
```

- **For String Class:** From Bootstrap Class Path by Bootstrap ClassLoader Output is null
- **For Student Class:** From Extension Class Path by Extension ClassLoader Output is `sun.misc.Launcher$ExtClassLoader@1234`
- **For Test Class:** From Application Class Path by Application ClassLoader Output is `sun.misc.Launcher$AppClassLoader@3456`

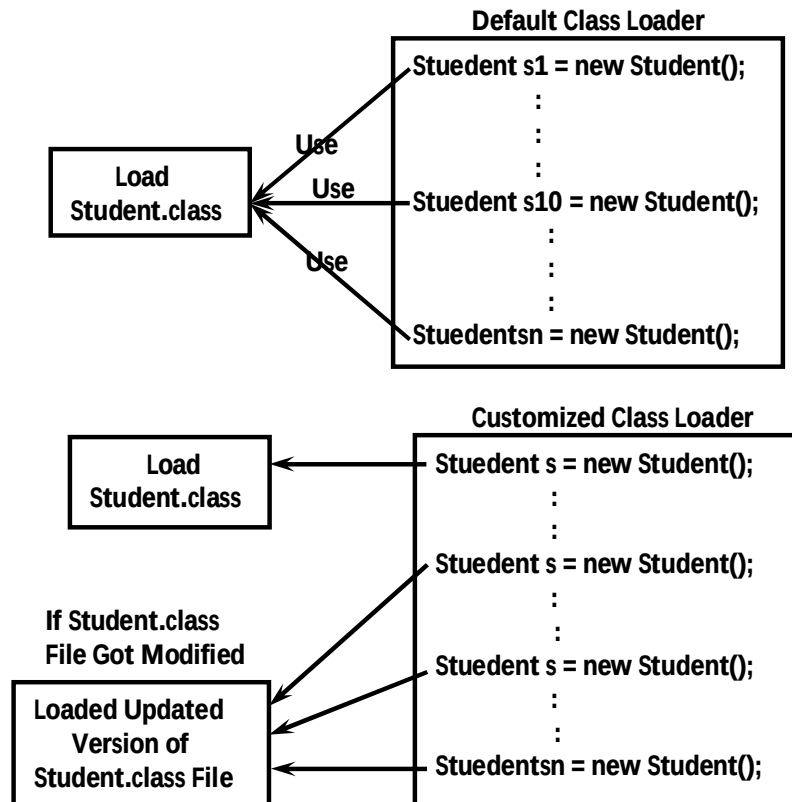
**Note:** Assume that *Student.class* Present in Both *Extension Class Path* and *Application Class Path* and *Test.class* Present in Only in *Application Class Path*.

**Note:**

- Bootstrap ClassLoader is Not Java Object. Hence we are getting null in the 1<sup>st</sup> Case but Extension ClassLoader and Application ClassLoader are Java Objects and Hence we get Proper Output in remaining 2 Cases.  
     ClassName@HexaDecimal.String\_of\_Hashcode
- ClassLoader Subsystem will give Highest Priority for Bootstrap Class Path and then Extension Class followed by Application Class Path.

### What is the Need of Customized ClassLoader?

- Default ClassLoader will load .class Files Only Once Eventhough we are using Multiple Times that Class in Our Program.
- After loading .class File if it is modified Outside, then Default ClassLoaderwon't Load Updated Version of Class File on Fly (Dynamically). Because .class File already there in Method Area.
- We can Resolve this Problem by defining Our Own Customized ClassLoader.
- The Main Advantage of Customized ClassLoader is we can Control Class loading Mechanism Based on Our Requirement.
- For Example we can Load Class File Separately Every Time. So that Updated Version Available to Our Program.



### How to Define Our Own ClassLoader?

We can Define Our Own Customized ClassLoader by extending *java.lang.ClassLoader* Class.

### Pseudo Code to Define Customized Class Loader

```

public class CustomClassLoader extends ClassLoader{
    public Class loadClass(String name) throws ClassNotFoundException{
        //Rad and Written Updated Class
        ---
        ---
        ---
    }
}
class CustomClassLoaderTest{
    public static void main(String[] args){
        Dog d = new Dog();
        .
        .
        .
        CustomClassLoader c = new CustomClassLoader();
        c.loadClass("Dog");
        .
        .
        c.loadClass("Dog");
    }
}

```

### What is the Purpose of java.lang.ClassLoader Class?

- This Class Act as Base Class for designing Our Own Customized ClassLoaders.
- Hence Every Customized ClassLoader Class should extends *java.lang.ClassLoader* either Directly OR Indirectly.

### Various Memory Areas of JVM:

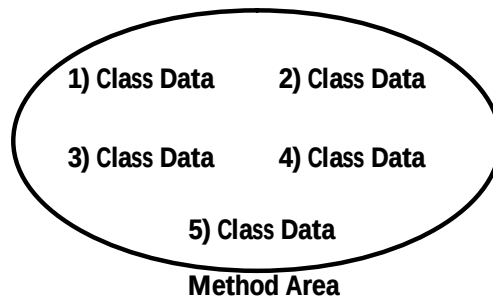
- Whole Loading and Running a Java Program JVM required Memory to Store Several Things Like Byte Code, Objects, Variables, Etc.
- Total JVM Memory organized in the following 5 Categories:
  - 1) Method Area
  - 2) Heap Area OR Heap Memory
  - 3) Java Stacks Area
  - 4) PC Registers Area
  - 5) Native Method Stacks Area

#### 1) Method Area:

- Method Area will be Created at the Time of JVM Start- Up.
- It will be Shared by All Threads (Global Memory).
- This Memory Area Need Not be Continuous.
- Method area shows runtime constant pool.

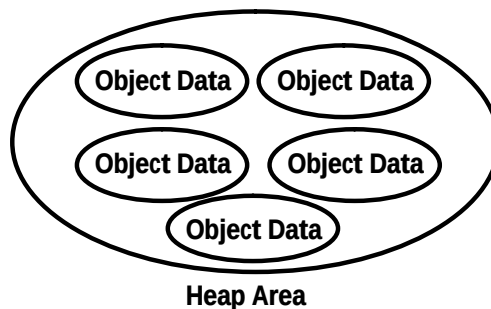


- Total Class Level Binary Information including Static Variables Stored in Method Area.



## 2) Heap Area:

- Programmer Point of View Heap Area is Consider as Important Memory Area.
- Heap Area will be Created at the Time of JVM Start- Up.
- Heap Area can be accessed by All Threads (Global OR Sharable Memory).
- Heap Area Need Not be Continuous.
- All Objects and corresponding Instance Variables will be stored in the Heap Area.
- Every Array in Java is an Object and Hence Arrays Also will be stored in Heap Memory Only.



## Program to Display Heap Memory Statistics

- A Java Application can Communicate with the JVM by using Runtime Object.
  - Runtime Class Present in java.lang Package and it is a Singleton Class.
  - We can Create Runtime Object by using  
Runtime r = Runtime.getRuntime();
  - Once we got Runtime Object we can Call the following Methods on that Object.
- 1) maxMemory(): Returns Number of Bytes of Max Memory allocated to the Heap.
  - 2) totalMemory(): Returns Number of Bytes of Total (Initial) Memory allocated to the Heap.
  - 3) freeMemory(): Returns Number of Bytes of Free Memory Present in Heap.

```

classHeapDemo {
public static void main(String[] args) {
    longmb = 1024*1024;
    Runtime r = Runtime.getRuntime();
    System.out.println("Max Memory: "+r.maxMemory()/mb);
    System.out.println("Total Memory: "+r.totalMemory()/mb);
    System.out.println("Free Memory: "+r.freeMemory()/mb);
    System.out.println("Consumed memory:"+(r.totalMemory()-r.freeMemory())/mb);
}
}

```

1 KB = 1024 Bytes  
1MB = (1024\*1024) Bytes

#### Output in Terms of MB's

Max Memory: 247  
Total Memory: 15  
Free Memory: 15  
Consumed memory:0

#### Output in Terms of Bytes

Max Memory: 253440  
Total Memory: 15872  
Free Memory: 15582  
Consumed memory:289

### How to Set Maximum and Minimum Heap Size?

- Heap Memory Size is Finite, Based on Our Requirement we can *IncreaseORDecrease* Heap Size.
- The Default Heap Size is 64.
- We can Use the following Flags with Java Command.

❖ -Xmx: To Set Maximum Heap Size.

Eg: java -Xmx128m HeapDemo

This Command will be Set as Maximum Heap Size as 128mb.

Max Memory: 123  
Total Memory: 14  
Free Memory: 14  
Consumed Memory: 0

❖ -Xms : To Set Minimum Heap Size.

Eg: java -Xms64m HeapDemo

This Command Set Minimum Heap Size as 64 mb.

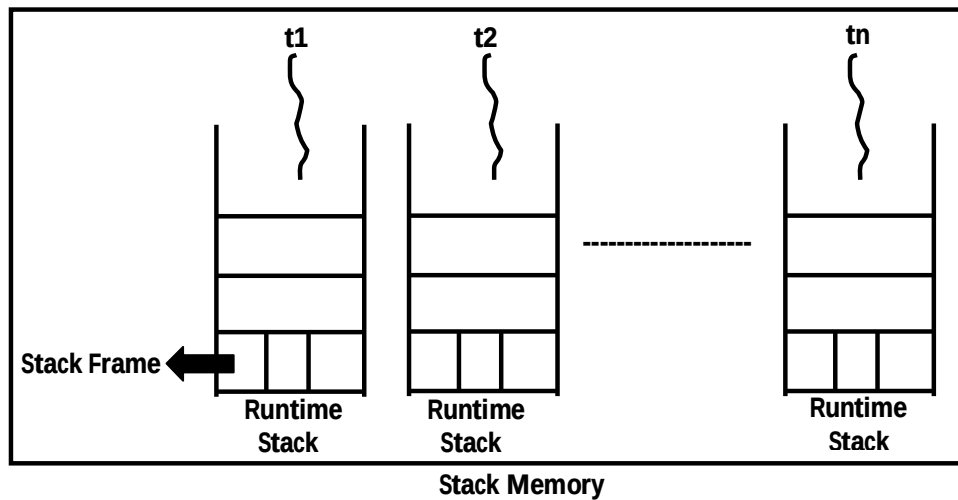
Max Memory: 232  
Total Memory: 61  
Free Memory: 61  
Consumed Memory: 0

❖ `java -Xmx128m -Xms64m HeapDemo`

|                                                                              |
|------------------------------------------------------------------------------|
| Max Memory: 123<br>Total Memory: 61<br>Free Memory: 61<br>Consumed Memory: 0 |
|------------------------------------------------------------------------------|

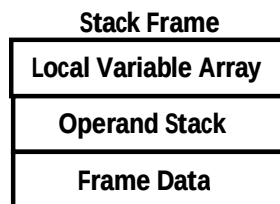
### 3) Stack Memory:

- For Every Thread JVM will Create a Separate Runtime Stack.
- Runtime Stack will be Created Automatically at the Time of Thread Creation.
- All Method Calls and corresponding Local Variables, Intermediate Results will be stored in the Stack.
- For Every Method Call a Separate Entry will be Added to the Stack and that Entry is Called *Stack Frame* OR *Activation Record*.
- After completing that Method Call the corresponding Entry from the Stack will be Removed.
- After completing All Method Calls, Just Before terminating the Thread, the Runtime Stack will be destroyed by the JVM.
- The Data stored in the Stack can be accessed by Only the corresponding Thread and it is Not Available to Other Threads.



#### Stack Frame Structure:

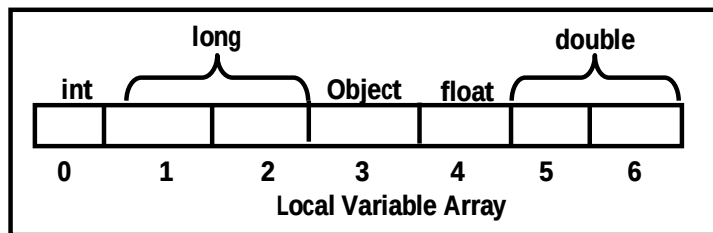
Each Stack Frame contains 3 Parts



❖ Local Variable Array:

- It Contains All Parameters and Local Variables of the Method.
- Each Slot in the Array is of 4 Bytes.
- Values of Type int, float, and Referenced Variables Occupy One Entry in that Array.
- Values of Type long and double Occupy 2 Consecutive Entries in Array.
- byte, short and char Values will be converted in to int Type before storing and Occupy One Slot.
- But the Way of storing boolean Values is varied from JVM to JVM. But Most of the JVM's follow One Entry OR One Slot for boolean Values.

**Eg:** public void m1(int i, long l, Object o, byte b, double d){}

❖ Operand Stack:

- JVM Uses Operand Stack as Work Space.
- Some Instructions can Push the Values to the Operand Stack and Some Instructions can Pop the Values from Operand Stack and Perform required Operations and Store Result Once Again Back to the Operand Stack.

| Program                                                                                                                                   |                      | Before Storing | After i-load 0 | After i-load 1 | After i-add | After i-store |
|-------------------------------------------------------------------------------------------------------------------------------------------|----------------------|----------------|----------------|----------------|-------------|---------------|
| <div style="border: 1px solid black; padding: 5px; display: inline-block;"> i - load 0<br/> i - load 1<br/> i - add<br/> i - store </div> | Local Variable Array | 0 100          | 0 100          | 0 100          | 0 100       | 0 100         |
|                                                                                                                                           |                      | 1 80           | 1 80           | 1 80           | 1 80        | 1 80          |
|                                                                                                                                           |                      | 2              | 2              | 2              | 2           | 2 180         |
|                                                                                                                                           |                      |                |                |                |             |               |
|                                                                                                                                           | Operand Stack        |                | 100            | 100<br>80      | 180         |               |
|                                                                                                                                           |                      |                |                |                |             |               |

❖ Frame Data:

- Frame Data contains All Symbolic References (Constant Pool) related to that Method.
- It also contains a Reference to Exception Table which Provides corresponding catch Block Information in the Case of Exceptions.

4) PC (Program Counter) Registers Area:

- For Every Thread a Separate PC Register will be Created at the Time of Thread Creation.
- PC Registers contains Address of Current executing Instruction.

- Once Instruction Execution Completes Automatically PC Register will be incremented to Hold Address of Next Instruction.

### 5) Native Method Stacks:

- For Every Thread JVM will Create a Separate Native Method Stack.
- All Native Method Calls invoked by the Thread will be stored in the corresponding Native Method Stack.

#### Note:

- Method Area, Heap Area and Stack Area are considered as *Major Memory Areas* with Respect to Programmers Point of View.
- Method Area and Heap Area are for JVM. Whereas Stack Area, PC Registers Area and Native Method Stack Area are for Thread. That is
  - One Separate Heap for Every JVM
  - One Separate Method Area for Every JVM
  - One Separate Stack for Every Thread
  - One Separate PC Register for Every Thread
  - One Separate Native Method Stack for Every Thread
- Static Variables will be stored in Method Area whereas Instance Variables will be stored in Heap Area and Local Variables will be stored in Stack Area.

### Execution Engine:

- This is the Central Component of JVM.
- Execution Engine is Responsible to Execute Java Class Files.
- Execution Engine contains 2 Components for executing Java Classes.
  - Interpreter
  - JIT Compiler

#### Interpreter:

- It is Responsible to Read Byte Code and Interpret (Convert) into Machine Code (Native Code) and Execute that Machine Code Line by Line.
- The Problem with Interpreter is it Interpreters Every Time Even the Same Method Multiple Times. Which Reduces Performance of the System.
- To Overcome this Problem SUN People Introduced JIT Compilers in 1.1 Version.

#### JIT Compiler:

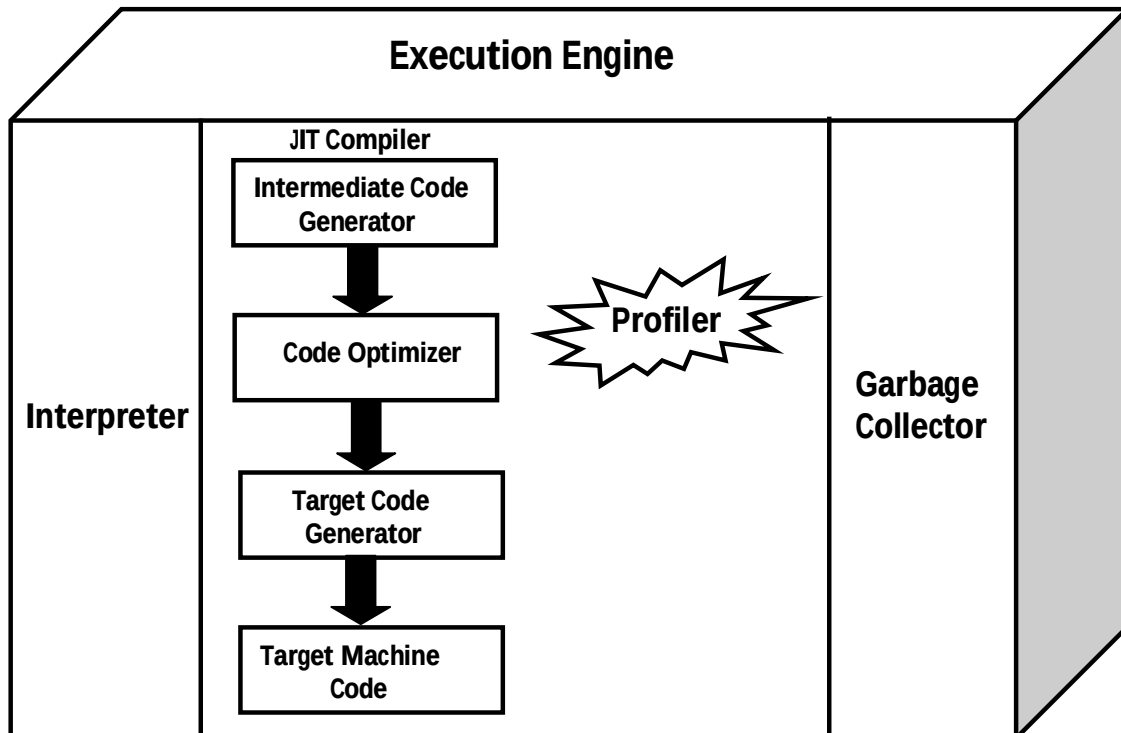
- The Main Purpose of JIT Compiler is to Improve Performance.
- Internally JIT Compiler Maintains a Separate Count for Every Method whenever JVM Come Across any Method Call.
- First that Method will be interpreted normally by the Interpreter and JIT Compiler Increments the corresponding Count Variable.
- This Process will be continued for Every Method.
- Once if any Method Count Reaches Threshold (The Starting Point for a New State) Value, then JIT Compiler Identifies that Method Repeatedly used Method (HOT SPOT).
- Immediately JIT Compiler Compiles that Method and Generates the corresponding Native Code. Next Time JVM Come Across that Method Call then JVM Directly Use Native Code

and Executes it Instead of interpreting Once Again. So that Performance of the System will be Improved.

- The Threshold Count Value varied from JVM to JVM.
- Some Advanced JIT Compilers will Re-compile generated Native Code if Count Reaches Threshold Value Second Time, So that More optimized Machine Code will be generated.
- **Profiler** which is the Part of JIT Compiler is Responsible to Identify HOT SPOTS.

**Note:**

- JVM Interprets Total Program Line by Line at least Once.
- JIT Compilation is Applicable Only for Repeatedly invoked Methods. But Not for Every Method.



**Java Native Interface (JNI):**

JNI Acts as Bridge (Mediator) between Java Method Calls and corresponding Native Libraries.

**Eg:** hashCode()

## Class File Structure

```
class File {
    Magic_Number;
    Minor_Version;
    Major_Version;
    Constant_Pool_Cont;
    Constant_Pool[];
    access_Flash;
    this_class;
    super_class;
    interface_count;
    interface[];
    fields_count;
    fields[];
    Methods_count;
    methods[];
    attributes_count;
    attributes[];
}
```

### 1) Magic Number

- The 1<sup>st</sup> 4 Bytes of Class File is Magic Number.
- This is a Predefined Value to Identify Java Class File.
- This Value should be 0XCAFEBABE.
- JVM will Use this Magic Number to Identify whether the Class File is Valid OR Not i.e. whether it is generated by Valid Compiler OR Not.

**Note:** Whenever we are executing a Java Class if JVM Unable to Find Valid Magic Number then we get RuntimeException Saying ClassFormatError: incompatible magic value.

### 2) Minor Version and Major Version

- Minor and Major Versions Represents Class File Version.
- JVM will Use these Versions to Identify which Version of Compiler Generates Current .class File



### Note:

- Higher Version JVM can Always Run Lower Version Class Files But Lower Version JVM can't Run Class Files generated by Higher Version Compiler.
- Whenever we are trying to Execute Higher Version Compiler generated Class File with Lower Version JVM we will get RuntimeException Saying java.lang.UnsupportedClassVersionError: Employee (Unsupported major.minor version 51.0)

- 3) **Constant Pool Count**: It Represents the Number of Constants Present in Constant Table of the Class.
- 4) **Constant Pool[]**: It Represents Information About Constants Present in Constant Table of the Class.
- 5) **Access Flags**: It Shows the Modifiers which are declared for the Current Class OR Interface.
- 6) **this\_class**: It Represents the Name of the Class OR Interface defined by Class File.
- 7) **super\_class**: It Represents the Name of the Super Class Represented by Class File.

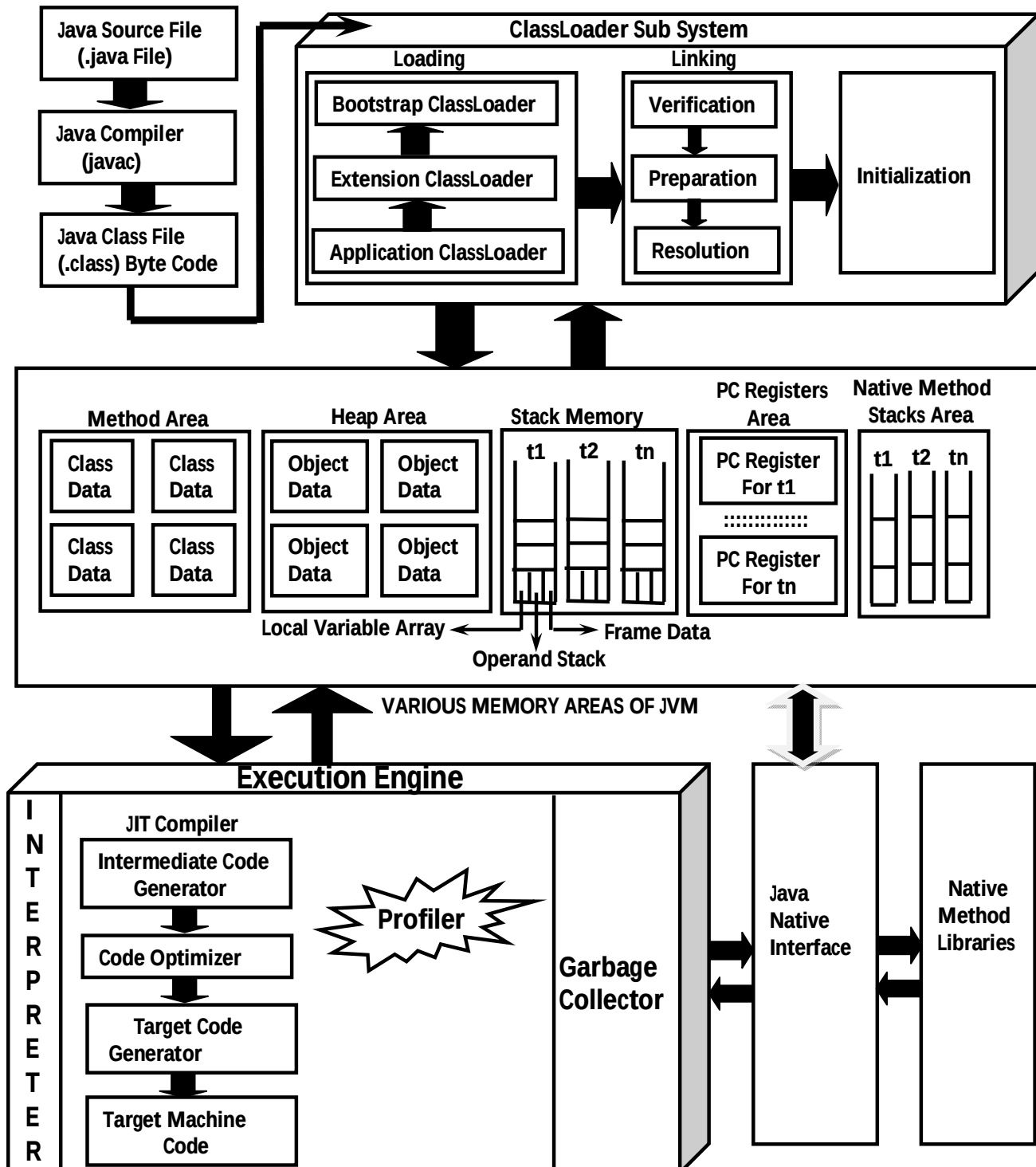


Test.class

this\_class: Test  
super\_class: java.lang.Object

- 8) **interface\_count**: It Represents Number of Interfaces implemented by Current Class File.
- 9) **interface[]**: It Represents the Names of Interfaces which are implemented by Current Class File.
- 10) **fields\_count**: It Represents Number of Fields Present in the Current Class File.
- 11) **fields[]**: It Provides Names of All Fields Present in the Current Class File.
- 12) **method\_count**: It Represents Number of Methods Present in the Current Class File.
- 13) **methods[]**: It Returns the Name of the Method Present in the Current Class File.
- 14) **attributes\_count**: It Represents Number of Attributes Present in the Current Class File.
- 15) **attributes[]**: It Provides Information About All Attributes Present in the Current Class File.





# **CORE JAVA**

## **With**

# **SCJP / OCJP**

## **Study Material**

### **Chapter 22: Java 8 New Features**



**DURGA M.Tech**

**(Sun certified & Realtime Expert)**

**Ex. IBM Employee**

**Trained Lakhs of Students  
for last 14 years across INDIA**

**India's No.1 Software Training Institute**

# **DURGASOFT**

**www.durgasoft.com Ph: 9246212143 ,8096969696**

# Java 8 New Features

java 7 – July 28<sup>th</sup> 2011  
2 Years 7 Months 18 Days

Java 8 - March 18<sup>th</sup> 2014

Java 9 - September 22<sup>nd</sup> 2016

Java 10 - 2018

After java 1.5version, java 8 is the next major version.

Before java 8, sun people gave importance only for objects but in 1.8version oracle people gave the importance for functional aspects of programming to bring its benefits to java.ie it doesn't mean java is functional oriented programming language.

## Java 8 New Features:

- 1) Lambda Expression
  - 2) FunctionalInterfaces
  - 3) Default methods
  - 4) Predicates
  - 5) Functions
  - 6) Double colon operator(::)
  - 7) Stream API
  - 8) Date and Time API
- Etc.....

## Lambda ( $\lambda$ ) Expression

- ☀ Lambda calculus is a big change in mathematical world which has been introduced in 1930. Because of benefits of Lambda calculus slowly this concepts started using in programming world. "LISP" is the first programming which uses Lambda Expression.
- ☀ The other languages which uses lambda expressions are:
  - C#.Net
  - C Objective
  - C
  - C++
  - Python
  - Ruby etc.
  - and finally in java also.

- ☀ The Main Objective of  $\lambda$  Lambda Expression is to bring benefits of functional programming into java.

### What is Lambda Expression ( $\lambda$ ):

Lambda Expression is just an anonymous(nameless) function. That means the function which doesn't have the name, return type and access modifiers.

Lambda Expression also known as anonymous functions or closures.

Ex: 1

|                                                   |                                                                                                         |
|---------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| <pre>public void m1() {     sop("hello"); }</pre> | $\rightarrow$ {<br>sop("hello");<br>}<br>$\rightarrow$ { sop("hello"); }<br>$\rightarrow$ sop("hello"); |
|---------------------------------------------------|---------------------------------------------------------------------------------------------------------|

Ex:2

|                                                           |                                                     |
|-----------------------------------------------------------|-----------------------------------------------------|
| <pre>public void add(inta, int b) {     sop(a+b); }</pre> | $\rightarrow$ (inta, int b) $\rightarrow$ sop(a+b); |
|-----------------------------------------------------------|-----------------------------------------------------|

- If the type of the parameter can be decided by compiler automatically based on the context then we can remove types also.
- The above Lambda expression we can rewrite as  $(a,b) \rightarrow \text{sop}(a+b)$ ;

Ex: 3

|                                                              |                                                                                                                                                                                       |
|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>public String str(String str) {     return str; }</pre> | $(\text{String str}) \rightarrow \text{return str};$<br><br>$(\text{str}) \rightarrow \text{str};$ |
|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### Conclusions:

- 1) A lambda expression can have zero or more number of parameters(arguments).

Ex:

$() \rightarrow \text{sop}(\text{"hello"});$   
 $(\text{int a}) \rightarrow \text{sop}(\text{a});$   
 $(\text{inta, int b}) \rightarrow \text{return a+b};$

- 2) Usually we can specify type of parameter. If the compiler expect the type based on the context then we can remove type. i.e., programmer is not required.

Ex:

$(\text{inta, int b}) \rightarrow \text{sop}(\text{a+b});$   
 $(\text{a,b}) \rightarrow \text{sop}(\text{a+b});$

- 3) If multiple parameters present then these parameters should be separated with comma(,).

- 4) If zero number of parameters available then we have to use empty parameter [ like ()].

Ex:

$() \rightarrow \text{sop}(\text{"hello"});$

- 5) If only one parameter is available and if the compiler can expect the type then we can remove the type and parenthesis also.

Ex:

(int a) → sop(a);



(a) → sop(a);



A → sop(a);

- 6) Similar to method body lambda expression body also can contain multiple statements. if more than one statements present then we have to enclose inside within curly braces. if one statement present then curly braces are optional.
- 7) Once we write lambda expression we can call that expression just like a method, for this functional interfaces are required.

### Functional Interfaces:

if an interface contain only one abstract method, such type of interfaces are called functional interfaces and the method is called functional method or single abstract method(SAM).

Ex:

- |                   |   |                                     |
|-------------------|---|-------------------------------------|
| 1) Runnable       | → | It contains only run() method       |
| 2) Comparable     | → | It contains only compareTo() method |
| 3) ActionListener | → | It contains only actionPerformed()  |
| 4) Callable       | → | It contains only call() method      |

Inside functional interface in addition to single Abstract method(SAM) we write any number of default and static methods.

Ex:

```

1) interface Interf {
2)     public abstract void m1();
3)     default void m2() {
4)         System.out.println ("hello" );
5)     }
6) }
```

In Java 8 ,SunMicroSystem introduced @FunctionalInterface annotation to specify that the interface is FunctionalInterface.

Ex:

```

@FunctionalInterface
    Interface Interf {
        public void m1();
    }
```

} this code compiles without any compilation errors.

Inside `FunctionalInterface` we can take only one abstract method, if we take more than one abstract method then compiler raise an error message that is called we will get compilation error.

Ex:

```
@FunctionalInterface {
    public void m1();
    public void m2();
}
```

} this code gives compilation error.

Inside `FunctionalInterface` we have to take exactly only one abstract method. If we are not declaring that abstract method then compiler gives an error message.

Ex:

```
@FunctionalInterface {
    interface Interface {
    }
```

} compilation error

### FunctionalInterface with respect to Inheritance:

If an interface extends `FunctionalInterface` and child interface doesn't contain any abstract method then child interface is also `FunctionalInterface`

Ex:

```
1) @FunctionalInterface
2) interface A {
3)     public void methodOne();
4) }
5) @FunctionalInterface
6) Interface B extends A {
7) }
```

In the child interface we can define exactly same parent interface abstract method.

Ex:

```
1) @FunctionalInterface
2) interface A {
3)     public void methodOne();
4) }
5) @FunctionalInterface
6) interface B extends A {
7)     public void methodOne();
8) }
```

} No Compile Time Error

In the child interface we can't define any new abstract methods otherwise child interface won't be `FunctionalInterface` and if we are trying to use `@FunctionalInterface` annotation then compiler gives an error message.

```

1) @FunctionalInterface {
2) interface A {
3)     public void methodOne();
4) }
5) @FunctionalInterface
6) interface B extends A {
7)     public void methodTwo();
8) }

```

} Compiletime Error

Ex:

```

@FunctionalInterface
interface A {
    public void methodOne();
}
interface B extends A {
    public void methodTwo();
}

```

} No compile time error

this's Normal interface so that code compiles without error

In the above example in both parent & child interface we can write any number of default methods and there are no restrictions. Restrictions are applicable only for abstract methods.

### FunctionalInterface Vs Lambda Expressions:

Once we write Lambda expressions to invoke it's functionality, then FunctionalInterface is required. We can use FunctionalInterface reference to refer Lambda Expression.

Where ever FunctionalInterface concept is applicable there we can use Lambda Expressions

Ex:1

Without Lambda Expression

```

1) interface Interf {
2)     public void methodOne() {}
3)     public class Demo implements Interface {
4)         public void methodOne() {
5)             System.out.println( " method one execution " );
6)         }
7)     public class Test {
8)         public static void main(String[] args) {
9)             Interfi = new Demo();
10)            i.methodOne();
11)        }
12) }

```

Above code With Lambda expression

```
1) interface Interf {  
2)     public void methodOne() {}  
3)     class Test {  
4)         public static void main(String[] args) {  
5)             Interfi = () → System.out.println("MethodOne Execution");  
6)             i.methodOne();  
7)         }  
8)     }
```

Without Lambda Expression

```
1) interface Interf {  
2)     public void sum(inta,int b);  
3) }  
4) class Demo implements Interf {  
5)     public void sum(inta,int b) {  
6)         System.out.println("The sum:" + (a+b));  
7)     }  
8) }  
9) public class Test {  
10)     public static void main(String[] args) {  
11)         Interfi = new Demo();  
12)         i.sum(20,5);  
13)     }  
14) }
```

Above code With Lambda Expression

```
1) interface Interf {  
2)     public void sum(inta, int b);  
3) }  
4) class Test {  
5)     public static void main(String[] args) {  
6)         Interfi = (a,b) → System.out.println("The Sum:" + (a+b));  
7)         i.sum(5,10);  
8)     }  
9) }
```

Without Lambda Expressions

```
1) interface Interf {  
2)     public int square(int x);  
3) }  
4) class Demo implements Interf {
```



```
5)      public int square(int x) {
6)          return x*x; OR (int x) → x*x
7)      }
8)  }
9)  class Test {
10)      public static void main(String[] args) {
11)          Interfi = new Demo();
12)          System.out.println("The Square of 7 is: " +i.square(7));
13)      }
14) }
```

#### Above code with Lambda Expression

```
1)  interface Interf {
2)      public int square(int x);
3)  }
4)  class Test {
5)      public static void main(String[] args) {
6)          Interfi = x → x*x;
7)          System.out.println("The Square of 5 is:" +i.square(5));
8)      }
9)  }
```

#### Without Lambda expression

```
1)  class MyRunnable implements Runnable {
2)      public void main() {
3)          for(int i=0; i<10; i++) {
4)              System.out.println("Child Thread");
5)          }
6)      }
7)  }
8)  class ThreadDemo {
9)      public static void main(String[] args) {
10)          Runnable r = new myRunnable();
11)          Thread t = new Thread(r);
12)          t.start();
13)          for(int i=0; i<10; i++) {
14)              System.out.println("Main Thread")
15)          }
16)      }
17) }
```

#### With Lambda expression

```
1)  class ThreadDemo {
2)      public static void main(String[] args) {
```

```
3)         Runnable r = () → {
4)             for(int i=0; i<10; i++) {
5)                 System.out.println("Child Thread");
6)             }
7)         };
8)         Thread t = new Thread(r);
9)         t.start();
10)        for(i=0; i<10; i++) {
11)            System.out.println("Main Thread");
12)        }
13)    }
14) }
```

### Anonymous inner classes vs Lambda Expressions

Wherever we are using anonymous inner classes there may be a chance of using Lambda expression to reduce length of the code and to resolve complexity.

Ex: With anonymous inner class

```
1) class Test {
2)     public static void main(String[] args) {
3)         Thread t = new Thread(new Runnable() {
4)             public void run() {
5)                 for(int i=0; i<10; i++) {
6)                     System.out.println("Child Thread");
7)                 }
8)             }
9)         });
10)        t.start();
11)        for(int i=0; i<10; i++)
12)            System.out.println("Main thread");
13)
14)    }
15) }
```

### With Lambda expression

```
1) class Test {
2)     public static void main(String[] args) {
3)         Thread t = new Thread() → {
4)             for(int i=0; i<10; i++) {
5)                 System.out.println("Child Thread");
6)             }
7)         });
8)        t.start();
9)        for(int i=0; i<10; i++) {
```

```
10)          System.out.println("Main Thread");
11)          }
12)      }
13) }
```

### What are the advantages of Lambda expression?

- ☀ We can reduce length of the code so that readability of the code will be improved.
- ☀ We can resolve complexity of anonymous inner classes.
- ☀ We can provide Lambda expression in the place of object.
- ☀ We can pass lambda expression as argument to methods.

### Note:

- ☀ Anonymous inner class can extend concrete class, can extend abstract class, can implement interface with any number of methods but
- ☀ Lambda expression can implement an interface with only single abstract method(FunctionalInterface).
- ☀ Hence if anonymous inner class implements functionalinterface in that particular case only we can replace with lambda expressions.hence wherever anonymous inner class concept is there,it may not possible to replace with Lambda expressions.
- ☀ Anonymous inner class! = Lambda Expression
- ☀ Inside anonymous inner class we can declare instance variables.
- ☀ Inside anonymous inner class “this” always refers current inner class object(anonymous inner class) but not related outer class object

### Ex:

- ☀ Inside lambda expression we can't declare instance variables.
- ☀ Whatever the variables declare inside lambda expression are simply acts as local variables
- ☀ Within lambda expression “this” keyword represents current outer class object reference (that is current enclosing class reference in which we declare lambda expression)

### Ex:

```
1) interface Interf {
2)     public void m1();
3) }
4) class Test {
5)     int x = 777;
6)     public void m2() {
7)         Interfi = () -> {
8)             int x = 888;
9)             System.out.println(x); 888
10)            System.out.println(this.x); 777
```

```
11)         };
12)         i.m1();
13)     }
14)     public static void main(String[] args) {
15)         Test t = new Test();
16)         t.m2();
17)     }
18) }
```

- ☀ From lambda expression we can access enclosing class variables and enclosing method variables directly.
- ☀ The local variables referenced from lambda expression are implicitly final and hence we can't perform re-assignment for those local variables otherwise we get compile time error

Ex:

```
1) interface Interf {
2)     public void m1();
3) }
4) class Test {
5)     int x = 10;
6)     public void m2() {
7)         int y = 20;
8)         Interfi = () → {
9)             System.out.println(x); 10
10)            System.out.println(y); 20
11)            x = 888;
12)            y = 999; //CE
13)        };
14)        i.m1();
15)        y = 777;
16)    }
17)    public static void main(String[] args) {
18)        Test t = new Test();
19)        t.m2();
20)    }
21) }
```

**Differences between anonymous inner classes and Lambda expression**

| Anonymous Inner class                                                                                                             | Lambda Expression                                                                                                                         |
|-----------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| It's a class without name                                                                                                         | It's a method without name(anonymous function)                                                                                            |
| Anonymous inner class can extend Abstract and concrete classes                                                                    | lambda expression can't extend Abstract and concrete classes                                                                              |
| Anonymous inner class can implement An interface that contains any number of Abstract methods                                     | lambda expression can implement an Interface which contains single abstract method (FunctionalInterface)                                  |
| Inside anonymous inner class we can Declare instance variables.                                                                   | Inside lambda expression we can't Declare instance variables, whatever the variables declare are simply acts as local variables.          |
| Anonymous inner classes can be Instantiated                                                                                       | lambda expressions can't be instantiated                                                                                                  |
| Inside anonymous inner class "this" Always refers current anonymous Inner class object but not outer class Object.                | Inside lambda expression "this" Always refers current outer class object. that is enclosing class object.                                 |
| Anonymous inner class is the best choice If we want to handle multiple methods.                                                   | Lambda expression is the best Choice if we want to handle interface With single abstract method (FunctionalInterface).                    |
| In the case of anonymous inner class At the time of compilation a separate Dot class file will be generated (outerclass\$1.class) | At the time of compilation no dot Class file will be generated for Lambda expression. it simply convert in to private method outer class. |
| Memory allocated on demand Whenever we are creating an object                                                                     | Reside in permanent memory of JVM (Method Area).                                                                                          |

**Default methods**

- ☀ Until 1.7 version onwards inside interface we can take only public abstract methods and public static final variables (every method present inside interface is always public and abstract whether we are declaring or not).
- ☀ Every variable declared inside interface is always public static final whether we are declaring or not.
- ☀ But from 1.8 version onwards in addition to these, we can declare default concrete methods also inside interface, which are also known as defender methods.
- ☀ We can declare default method with the keyword "default" as follows

```

1) default void m1(){
2) System.out.println ("Default Method");
3) }
```

Interface default methods are by-default available to all implementation classes. Based on requirement implementation class can use these default methods directly or can override.

Ex:

```
1) interface Interf {  
2)     default void m1() {  
3)         System.out.println("Default Method");  
4)     }  
5) }  
6) class Test implements Interf {  
7)     public static void main(String[] args) {  
8)         Test t = new Test();  
9)         t.m1();  
10)    }  
11) }
```

Default methods also known as defender methods or virtual extension methods.

The main advantage of default methods is without effecting implementation classes we can add new functionality to the interface (backward compatibility).

Note:

We can't override object class methods as default methods inside interface otherwise we get compiletime error.

Ex:

```
1) interface Interf {  
2)     default int hashCode() {  
3)         return 10;  
4)     }  
5) }
```

**CompileTimeError**

Reason: object class methods are by-default available to every java class hence it's not required to bring through default methods.

### Default method vs multiple inheritance

Two interfaces can contain default method with same signature then there may be a chance of ambiguity problem (diamond problem) to the implementation class. To overcome this problem compulsory we should override default method in the implementation class otherwise we get compiletime error.

```
1) Eg 1:
2) interface Left {
3)     default void m1() {
4)         System.out.println("Left Default Method");
5)     }
6) }
7)
8) Eg 2:
9) interface Right {
10)     default void m1() {
11)         System.out.println("Right Default Method");
12)     }
13) }
14)
15) Eg 3:
16) class Test implements Left, Right {}
```

### How to override default method in the implementation class?

In the implementation class we can provide complete new implementation or we can call any interface method as follows.

interfacename.super.m1();

Ex:

```
1) class Test implements Left, Right {
2)     public void m1() {
3)         System.out.println("Test Class Method"); OR Left.super.m1();
4)     }
5)     public static void main(String[] args) {
6)         Test t = new Test();
7)         t.m1();
8)     }
9) }
```

**Differences between interface with default methods and abstract class**

Eventhough we can add concrete methods in the form of default methods to the interface , it wont be equal to abstract class.

| Interface with Default Methods                                                                             | Abstract Class                                                                                           |
|------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Inside interface every variable is Always public static final and there is No chance of instance variables | Inside abstract class there may be a Chance of instance variables which Are required to the child class. |
| Interface never talks about state of Object.                                                               | Abstract class can talk about state of Object.                                                           |
| Inside interface we can't declare Constructors.                                                            | Inside abstract class we can declare Constructors.                                                       |
| Inside interface we can't declare Instance and static blocks.                                              | Inside abstract class we can declare Instance and static blocks.                                         |
| Functional interface with default Methods Can refer lambda expression.                                     | Abstract class can't refer lambda Expressions.                                                           |
| Inside interface we can't override Object class methods.                                                   | Inside abstract class we can override Object class methods.                                              |

**Interface with default method != abstract class****Static methods inside interface:**

From 1.8version onwards in addition to default methods we can write static methods also inside interface to define utility functions.

Interface static methods by-default not available to the implementation classes hence by using implementation class reference we can't call interface static methods.we should call interface static methods by using interface name.

**Ex:**

```

1) interface Interf {
2)     public static void sum(int a, int b) {
3)         System.out.println("The Sum:"+(a+b));
4)     }
5) }
6) class Test implements Interf {
7)     public static void main(String[] args) {
8)         Test t = new Test();
9)         t.sum(10, 20); //CE
10)        Test.sum(10, 20); //CE
11)        Interf.sum(10, 20);
12)    }
13) }
```



As interface static methods by default not available to the implementation class, overriding concept is not applicable.

Based on our requirement we can define exactly same method in the implementation class, it's valid but not overriding.

**Ex:1**

```
1) interface Interf {  
2)     public static void m1() {}  
3) }  
4) class Test implements Interf {  
5)     public static void m1() {}  
6) }
```

It's valid but not overriding

**Ex:2**

```
1) interface Interf {  
2)     public static void m1() {}  
3) }  
4) class Test implements Interf {  
5)     public void m1() {}  
6) }
```

This's valid but not overriding

**Ex3:**

```
1) class P {  
2)     private void m1() {}  
3) }  
4) class C extends P {  
5)     public void m1() {}  
6) }
```

This's valid but not overriding

From 1.8 version onwards we can write main() method inside interface and hence we can run interface directly from the command prompt.

**Ex:**

```
1) interface Interf {  
2)     public static void main(String[] args) {  
3)         System.out.println("Interface Main Method");  
4)     }  
5) }
```

At the command prompt:

```
javac Interf.java  
javaInterf
```

## Predicates

A predicate is a function with a single argument and returns boolean value.

To implement predicate functions in java, oracle people introduced Predicate interface in 1.8 version (i.e., Predicate<T>).

Predicate interface present in java.util.function package.

It's a functional interface and it contains only one method i.e., test()

Ex:

```
interface Predicate<T> {  
    public boolean test(T t);  
}
```

As predicate is a functional interface and hence it can refer lambda expression

Ex:1

Write a predicate to check whether the given integer is greater than 10 or not.

Ex:

```
public boolean test(Integer I) {  
    if (I > 10) {  
        return true;  
    }  
    else {  
        return false;  
    }  
}
```



```
(Integer I) → { if(I > 10)  
                return true;  
                else  
                return false;  
            }
```



```
I → (I > 10);
```



```
predicate<Integer> p = I → (I > 10);  
System.out.println (p.test(100)); true  
System.out.println (p.test(7)); false
```

**Program:**

```

1) import java.util.function;
2) class Test {
3)     public static void main(String[] args) {
4)         predicate<Integer> p = i → (i>10);
5)         System.out.println(p.test(100));
6)         System.out.println(p.test(7));
7)         System.out.println(p.test(true)); //CE
8)     }
9) }

```

# 1 Write a predicate to check the length of given string is greater than 3 or not.

```

Predicate<String> p = s → (s.length() > 3);
System.out.println (p.test("rvkb")); true
System.out.println (p.test("rk")); false

```

#-2 write a predicate to check whether the given collection is empty or not.

```

Predicate<collection> p = c → c.isEmpty();

```

**Predicate joining**

It's possible to join predicates into a single predicate by using the following methods.

and()

or()

negate()

these are exactly same as logical AND ,OR complement operators

**Ex:**

```

1) import java.util.function.*;
2) class test {
3)     public static void main(string[] args) {
4)         int[] x = {0, 5, 10, 15, 20, 25, 30};
5)         predicate<integer> p1 = i->i>10;
6)         predicate<integer> p2=i -> i%2==0;
7)         System.out.println("The Numbers Greater Than 10:");
8)         m1(p1, x);
9)         System.out.println("The Even Numbers Are:");
10)        m1(p2, x);
11)        System.out.println("The Numbers Not Greater Than 10:");
12)        m1(p1.negate(), x);
13)        System.out.println("The Numbers Greater Than 10 And Even Are:â€  );
14)        m1(p1.and(p2), x);
15)        System.out.println("The Numbers Greater Than 10 OR Even:â€  );
16)        m1(p1.or(p2), x);
17)    }
18)    public static void m1(predicate<integer>p, int[] x) {

```

```
19)      for(int x1:x) {  
20)          if(p.test(x1))  
21)              System.out.println(x1);  
22)      }  
23)  }  
24) }
```

### Function

Functions are exactly same as predicates except that functions can return any type of result but function should(can) return only one value and that value can be any type as per our requirement.

To implement functions oracle people introduced Function interface in 1.8 version.

Function interface present in java.util.function package.

Functional interface contains only one method i.e., apply()

```
interface function(T,R) {  
    public R apply(T t);  
}
```

### Assignment:

Write a function to find length of given input string.

### Ex:

```
1) import java.util.function.*;  
2) class Test {  
3)     public static void main(String[] args) {  
4)         Function<String, Integer> f = s -> s.length();  
5)         System.out.println(f.apply("Durga"));  
6)         System.out.println(f.apply("Soft"));  
7)     }  
8) }
```

### Note:

Function is a functional interface and hence it can refer lambda expression.

**Difference between predicate and function**

| Predicate                                                                                   | Function                                                                                                                           |
|---------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| To implement conditional checks<br>We should go for predicate                               | To perform certain operation And to return some result we Should go for function.                                                  |
| Predicate can take one type Parameter which represents Input argument type.<br>Predicate<T> | Function can take 2 type Parameters.first one represent Input argument type and Second one represent return Type.<br>Function<T,R> |
| Predicate interface defines only one method called test()                                   | Function interface defines only one Method called apply().                                                                         |
| public boolean test(T t)                                                                    | public R apply(T t)                                                                                                                |
| Predicate can return only boolean value.                                                    | Function can return any type of value                                                                                              |

**Note:**

Predicate is a boolean valued function  
and(), or(), negate() are default methods present inside Predicate interface.

**Method and Constructor references by using ::(double colon)operator**

functionalInterface method can be mapped to our specified method by using :: (double colon)operator. This is called method reference.

Our specified method can be either static method or instance method.  
FunctionalInterface method and our specified method should have same argument types ,except this the remaining things like  
return type,method name,modifiers etc are not required to match.

**Syntax:**

if our specified method is static method

**Classname::methodName**

if the method is instance method

**Objref::methodName**

FunctionalInterface can refer lambda expression and FunctionalInterface can also refer method reference . Hence lambda expression can be replaced with method reference.  
hence method reference is alternative syntax to lambda expression.

Ex: With Lambda Expression

```
1) class Test {  
2)     public static void main(String[] args) {  
3)         Runnable r = () -> {  
4)             for(int i=0; i<=10; i++) {  
5)                 System.out.println("Child Thread");  
6)             }  
7)         };  
8)         Thread t = new Thread(r);  
9)         t.start();  
10)        for(int i=0; i<=10; i++) {  
11)            System.out.println("Main Thread");  
12)        }  
13)    }  
14) }
```

With Method Reference

```
1) class Test {  
2)     public static void m1() {  
3)         for(int i=0; i<=10; i++) {  
4)             System.out.println("Child Thread");  
5)         }  
6)     }  
7)     public static void main(String[] args) {  
8)         Runnable r = Test:: m1;  
9)         Thread t = new Thread(r);  
10)        t.start();  
11)        for(int i=0; i<=10; i++) {  
12)            System.out.println("Main Thread");  
13)        }  
14) }
```

In the above example Runnable interface run() method referring to Test class static method m1().  
Method reference to Instance method:

Ex:

```
1) interface Interf {  
2)     public void m1(int i);  
3) }  
4) class Test {  
5)     public void m2(int i) {
```

```
6)      System.out.println("From Method Reference:"+i);
7)      }
8)      public static void main(String[] args) {
9)          Interf f = I ->sop("From Lambda Expression:"+i);
10)         f.m1(10);
11)         Test t = new Test();
12)         Interf i1 = t::m2;
13)         i1.m1(20);
14)     }
15) }
```

In the above example functional interface method m1() referring to Test class instance method m2(). The main advantage of method reference is we can use already existing code to implement functional interfaces(code reusability).

### Constructor References

We can use :: ( double colon )operator to refer constructors also

Syntax: classname :: new

Ex:

Interf f = sample :: new;  
functional interface f referring sample class constructor

Ex:

```
1) class Sample {
2)     private String s;
3)     Sample(String s) {
4)         this.s = s;
5)         System.out.println("Constructor Executed:"+s);
6)     }
7) }
8) interface Interf {
9)     public Sample get(String s);
10) }
11) class Test {
12)     public static void main(String[] args) {
13)         Interf f = s -> new Sample(s);
14)         f.get("From Lambda Expression");
15)         Interf f1 = Sample :: new;
16)         f1.get("From Constructor Reference");
17)     }
18) }
```

**Note:**

In method and constructor references compulsory the argument types must be matched.

**Streams**

To process objects of the collection, in 1.8 version Streams concept introduced.

**What is the differences between java.util.streams and java.io streams?**

java.util streams meant for processing objects from the collection. I.e, it represents a stream of objects from the collection but java.io streams meant for processing binary and character data with respect to file. I.e it represents stream of binary data or character data from the file .hence java.io streams and java.util streams both are different.

**What is the difference between collection and stream?**

if we want to represent a group of individual objects as a single entity then we should go for collection.

if we want to process a group of objects from the collection then we should go for streams.

we can create a stream object to the collection by using stream() method of Collection interface. stream() method is a default method added to the Collection in 1.8 version.

default Stream stream()

**Ex:**

```
Stream s = c.stream();
```

Stream is an interface present in java.util.stream.

once we got the stream, by using that we can process objects of that collection.

we can process the objects in the following two phases

1.configuration

2.processing

**configuration:**

we can configure either by using filter mechanism or by using map mechanism.

**Filtering:**

we can configure a filter to filter elements from the collection based on some boolean condition by using filter() method of Stream interface.

```
public Stream filter(Predicate<T> t)
```

here (Predicate<T > t ) can be a boolean valued function/lambda expression





```
7)      }
8)      System.out.println(l1);
9)      ArrayList<Integer> l2 = new ArrayList<Integer>();
10)     for(Integer i:l1) {
11)         if(i%2 == 0)
12)             l2.add(i);
13)     }
14)     System.out.println(l2);
15) }
16) }
```

#### Approach-2: With Streams

```
1) import java.util.*;
2) import java.util.stream.*;
3) class Test {
4)     public static void main(String[] args) {
5)         ArrayList<Integer> l1 = new ArrayList<Integer>();
6)         for(int i=0; i<=10; i++) {
7)             l1.add(i);
8)         }
9)         System.out.println(l1);
10)        List<Integer> l2 = l1.stream().filter(i -> i%2==0).collect(Collectors.toList());
11)        System.out.println(l2);
12)    }
13) }
```

#### Ex: Program for map() and collect() Method

```
1) import java.util.*;
2) import java.util.stream.*;
3) class Test {
4)     public static void main(String[] args) {
5)         ArrayList<String> l = new ArrayList<String>();
6)         l.add("rvk"); l.add("rk"); l.add("rkv"); l.add("rvki"); l.add("rvkir");
7)         System.out.println(l);
8)         List<String> l2 = l.stream().map(s -> s.toUpperCase()).collect(Collectors.toList());
9)         System.out.println(l2);
10)    }
11) }
```

**II.Processing by count()method**

this method returns number of elements present in the stream.

```
public long count()
```

**Ex:**

```
long count=l.stream().filter(s ->s.length()==5).count();  
sop("the number of 5 length strings is:"+count);
```

**III.Processing by sorted()method**

if we sort the elements present inside stream then we should go for sorted() method.

the sorting can either default natural sorting order or customized sorting order specified by comparator.

sorted()- default natural sorting order

sorted(Comparator c)-customized sorting order.

**Ex:**

```
List<String> l3=l.stream().sorted().collect(Collectors.toList());  
sop("according to default natural sorting order:"+l3);
```

```
List<String> l4=l.stream().sorted((s1,s2) -> -s1.compareTo(s2)).collect(Collectors.toList());  
sop("according to customized sorting order:"+l4);
```

**IV.Processing by min() and max() methods**

```
min(Comparator c)
```

returns minimum value according to specified comparator.

```
max(Comparator c)
```

returns maximum value according to specified comparator

**Ex:**

```
String min=l.stream().min((s1,s2) -> s1.compareTo(s2)).get();  
sop("minimum value is:"+min);
```

```
String max=l.stream().max((s1,s2) -> s1.compareTo(s2)).get();  
sop("maximum value is:"+max);
```

**V.forEach() method**

this method will not return anything.

this method will take lambda expression as argument and apply that lambda expression for each element present in the stream.

**Ex:**

```
l.stream().forEach(s->sop(s));  
l3.stream().forEach(System.out::println);
```

**Ex:**

```

1) import java.util.*;
2) import java.util.stream.*;
3) class Test1 {
4)     public static void main(String[] args) {
5)         ArrayList<Integer> l1 = new ArrayList<Integer>();
6)         l1.add(0); l1.add(15); l1.add(10); l1.add(5); l1.add(30); l1.add(25); l1.add(20);
7)         System.out.println(l1);
8)         ArrayList<Integer> l2=l1.stream().map(i-> i+10).collect(Collectors.toList());
9)         System.out.println(l2);
10)        long count = l1.stream().filter(i->i%2==0).count();
11)        System.out.println(count);
12)        List<Integer> l3=l1.stream().sorted().collect(Collectors.toList());
13)        System.out.println(l3);
14)        Comparator<Integer> comp=(i1,i2)->i1.compareTo(i2);
15)        List<Integer> l4=l1.stream().sorted(comp).collect(Collectors.toList());
16)        System.out.println(l4);
17)        Integer min=l1.stream().min(comp).get();
18)        System.out.println(min);
19)        Integer max=l1.stream().max(comp).get();
20)        System.out.println(max);
21)        l3.stream().forEach(i->sop(i));
22)        l3.stream().forEach(System.out:: println);
23)
24)    }
25) }

```

**VI.toArray() method**

we can use toArray() method to copy elements present in the stream into specified array

```

Integer[] ir = l1.stream().toArray(Integer[] :: new);
for(Integer i: ir) {
    sop(i);
}

```

**VII.Stream.of()method**

we can also apply a stream for group of values and for arrays.

**Ex:**

```

Stream s=Stream.of(99,999,9999,99999);
s.forEach(System.out:: println);

```

```

Double[] d={10.0,10.1,10.2,10.3};
Stream s1=Stream.of(d);
s1.forEach(System.out :: println);

```

### Date and Time API: (Joda-Time API)

until java 1.7 version the classes present in java.util package to handle Date and Time (like Date, Calendar, TimeZone etc) are not upto the mark with respect to convenience and performance.

To overcome this problem in the 1.8 version oracle people introduced Joda-Time API. This API developed by joda.org and available in java in the form of java.time package.

# program for to display System Date and time.

```
1) import java.time.*;
2) public class DateTime {
3)     public static void main(String[] args) {
4)         LocalDate date = LocalDate.now();
5)         System.out.println(date);
6)         LocalTime time=LocalTime.now();
7)         System.out.println(time);
8)     }
9) }
```

O/p:

2015-11-23

12:39:26:587

Once we get LocalDate object we can call the following methods on that object to retrieve Day, month and year values separately.

Ex:

```
1) import java.time.*;
2) class Test {
3)     public static void main(String[] args) {
4)         LocalDate date = LocalDate.now();
5)         System.out.println(date);
6)         int dd = date.getDayOfMonth();
7)         int mm = date.getMonthValue();
8)         int yy = date.getYear();
9)         System.out.println(dd+"..."+"mm+"..."+"yy);
10)        System.out.printf("\n%d-%d-%d",dd,mm,yy);
11)    }
12) }
```

Once we get LocalTime object we can call the following methods on that object.

Ex:

```
1) import java.time.*;
2) class Test {
3)     public static void main(String[] args) {
4)         LocalTime time = LocalTime.now();
```

```
5)    int h = time.getHour();
6)    int m = time.getMinute();
7)    int s = time.getSecond();
8)    int n = time.getNano();
9)    System.out.printf("\n%d:%d:%d:%d",h,m,s,n);
10)   }
11) }
```

If we want to represent both Date and Time then we should go for LocalDateTime object.

```
LocalDateTimedt = LocalDateTime.now();
System.out.println(dt);
```

O/p:2015-11-23T12:57:24.531

We can represent a particular Date and Time by using LocalDateTime object as follows.

Ex:

```
LocalDateTime dt1=LocalDateTime.of(1995,Month.APRIL,28,12,45);
sop(dt1);
```

Ex:

```
LocalDateTime dt1=LocalDateTime.of(1995,04,28,12,45);
sop(dt1);
```

```
Sop("After six months:"+dt.plusMonths(6));
Sop("Before six months:"+dt.minusMonths(6));
```

#### To Represent Zone:

Zoneld object can be used to represent Zone.

Ex:

```
1) import java.time.*;
2) class ProgramOne {
3)     public static void main(String[] args) {
4)         Zoneld zone = Zoneld.systemDefault();
5)         System.out.println(zone);
6)     }
7) }
```

We can create Zoneld for a particular zone as follows

Ex:

```
Zoneld la = Zoneld.of("America/Los_Angeles");
ZonedDateTmezt = ZonedDateTime.now(la);
System.out.println(zt);
```

**Period Object:**

Period object can be used to represent quantity of time

Ex:

```
LocalDate today = LocalDate.now();
```

```
LocalDate birthday = LocalDate.of(1989,06,15);
```

```
Period p = Period.between(birthday,today);
```

```
System.out.printf("age is %d year %d months %d days",p.getYears(),p.getMonths(),p.getDays());
```

**# write a program to check the given year is leap year or not.**

```
1) import java.time.*;
2) public class Leapyear {
3)     int n = Integer.parseInt(args[0]);
4)     Year y = Year.of(n);
5)     if(y.isLeap())
6)         System.out.printf("%d is Leap year",n);
7)     else
8)         System.out.printf("%d is not Leap year",n);
9) }
```